Structure and Interpretation of Computer Programs

JavaScript Edition

Structure and Interpretation of Computer Programs

JavaScript Edition

Harold Abelson and Gerald Jay Sussman

adapted to JavaScript by Martin Henz and Tobias Wrigstad

with Julie Sussman

The MIT Press
Cambridge, Massachusetts
London, England

© 2022 Massachusetts Institute of Technology

This book is published by The MIT Press under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA). To view a copy of this license, visit http://creativecommons.org/licenses/by-nc-sa/4.0/.



The text of this book is derived from the book *Structure and Interpretation of Computer Programs, Second Edition*, 1996, (SICP) and is subject to a Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA). A comparison edition available at http://sicp.sourceacademy.org indicates the changes that were made to the text. The figures are derived from figures created by Andres Raba in 2015 and are also subject to CC BY-SA. To view a copy of this license, visit http://creativecommons.org/licenses/by-sa/4.0/.



The JavaScript programs in this book are derived from the Scheme programs in SICP and are subject to the GNU General Public License v3.0. To view a copy of this license, visit https://www.gnu.org/licenses/gpl-3.0.html.



The original image of MIT founder William Barton Rogers in section 2.2.4 is courtesy MIT Museum.

The cover image is adapted from *Le Moyen Age et la Renaissance*, Paris, 1848–1851.

This book was set in Times by the authors using the LaTeX typesetting system and ancillary scripts (see https://github.com/source-academy/sicp), and was printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Data

Names: Abelson, Harold, author. | Sussman, Gerald Jay, author. | Henz, Martin, adapter. | Wrigstad, Tobias, adapter. | Sussman, Julie, other.
Title: Structure and interpretation of computer programs / Harold Abelson and Gerald Jay Sussman; adapted to JavaScript by Martin Henz and Tobias Wrigstad; with Julie Sussman.
Description: Javascript edition. | Cambridge : The MIT Press, [2022] | Series: MIT electrical engineering and computer science series | Includes bibliographical references and index.
Identifiers: LCCN 2021047249 | ISBN 9780262543231 (paperback)
Subjects: LCSH: Computer programming. | JavaScript (Computer program language)
Classification: LCC QA76.6 .A255 2022 | DDC 005.13–dc23
LC record available at https://lccn.loc.gov/2021047249

10 9 8 7 6 5 4 3 2 1

This book is dedicated, in respect and admiration,
to the spirit that lives in the computer.

"I think it's extraordinarily important that we in computer science keep fun in computing. When it started out, it was an awful lot of fun. Of course, the paying customers got shafted every now and then, and after a while we began to take their complaints seriously. We began to feel as though we really were responsible for the successful, error-free, perfect use of these machines. I don't think we are. I think we're responsible for stretching them, setting them off in new directions, and keeping fun in the house. Fun comes in many ways. Fun comes in making a discovery, proving a theorem, writing a program, breaking a code. Whatever form or sense it comes in I hope the field of computer science never loses its sense of fun. Above all, I hope we don't become missionaries. What you know about computing other people will learn. Don't feel as though the key to successful computing is only in your hands. What's in your hands, I think and hope, is intelligence: the ability to see the machine as more than when you were first led up to it, that you can make it more."

—Alan J. Perlis (April 1, 1922–February 7, 1990)

Contents

Foreword xiii

Foreword to Structure and Interpretation of Computer Programs, 1984 xvii

Preface xxi

Prefaces to Structure and Interpretation of Computer Programs, 1996 & 1984 xxiii

Acknowledgments xxvii

1 Building Abstractions with Functions 1

1.1 The Elements of Programming 3
1.1.1 Expressions 3
1.1.2 Naming and the Environment 5
1.1.3 Evaluating Operator Combinations 6
1.1.4 Compound Functions 8
1.1.5 The Substitution Model for Function Application 11
1.1.6 Conditional Expressions and Predicates 13
1.1.7 Example: Square Roots by Newton's Method 18
1.1.8 Functions as Black-Box Abstractions 22

1.2 Functions and the Processes They Generate 26
1.2.1 Linear Recursion and Iteration 27
1.2.2 Tree Recursion 32
1.2.3 Orders of Growth 36
1.2.4 Exponentiation 38
1.2.5 Greatest Common Divisors 41
1.2.6 Example: Testing for Primality 43

1.3 Formulating Abstractions with Higher-Order Functions 48
1.3.1 Functions as Arguments 49
1.3.2 Constructing Functions using Lambda Expressions 53
1.3.3 Functions as General Methods 58
1.3.4 Functions as Returned Values 63

2 Building Abstractions with Data 69

2.1 Introduction to Data Abstraction 72
2.1.1 Example: Arithmetic Operations for Rational Numbers 72
2.1.2 Abstraction Barriers 76
2.1.3 What Is Meant by Data? 78
2.1.4 Extended Exercise: Interval Arithmetic 81

2.2 Hierarchical Data and the Closure Property 84
2.2.1 Representing Sequences 85
2.2.2 Hierarchical Structures 92
2.2.3 Sequences as Conventional Interfaces 98
2.2.4 Example: A Picture Language 110

2.3 Symbolic Data 124
2.3.1 Strings 124
2.3.2 Example: Symbolic Differentiation 126
2.3.3 Example: Representing Sets 131
2.3.4 Example: Huffman Encoding Trees 140

2.4 Multiple Representations for Abstract Data 147
2.4.1 Representations for Complex Numbers 149
2.4.2 Tagged data 152
2.4.3 Data-Directed Programming and Additivity 156

2.5 Systems with Generic Operations 163
2.5.1 Generic Arithmetic Operations 164
2.5.2 Combining Data of Different Types 169
2.5.3 Example: Symbolic Algebra 176

3 Modularity, Objects, and State 189

3.1 Assignment and Local State 190
3.1.1 Local State Variables 190
3.1.2 The Benefits of Introducing Assignment 197
3.1.3 The Costs of Introducing Assignment 200

3.2 The Environment Model of Evaluation 206
3.2.1 The Rules for Evaluation 207
3.2.2 Applying Simple Functions 210
3.2.3 Frames as the Repository of Local State 213
3.2.4 Internal Declarations 218
3.2.5 CSE Machine 222

3.3 Modeling with Mutable Data 222
3.3.1 Mutable List Structure 223
3.3.2 Representing Queues 231
3.3.3 Representing Tables 236
3.3.4 A Simulator for Digital Circuits 242
3.3.5 Propagation of Constraints 253

3.4 Concurrency: Time Is of the Essence 263
3.4.1 The Nature of Time in Concurrent Systems 265
3.4.2 Mechanisms for Controlling Concurrency 269

3.5 Streams 281
3.5.1 Streams Are Delayed Lists 282
3.5.2 Infinite Streams 289
3.5.3 Exploiting the Stream Paradigm 296
3.5.4 Streams and Delayed Evaluation 306

3.5.5 Modularity of Functional Programs and Modularity of Objects 312

4 Metalinguistic Abstraction 317

4.1 The Metacircular Evaluator 319
4.1.1 The Core of the Evaluator 321
4.1.2 Representing Components 328
4.1.3 Evaluator Data Structures 339
4.1.4 Running the Evaluator as a Program 344
4.1.5 Data as Programs 348
4.1.6 Internal Declarations 351
4.1.7 Separating Syntactic Analysis from Execution 355

4.2 Lazy Evaluation 360
4.2.1 Normal Order and Applicative Order 361
4.2.2 An Interpreter with Lazy Evaluation 362
4.2.3 Streams as Lazy Lists 370

4.3 Nondeterministic Computing 373
4.3.1 Search and `amb` 374
4.3.2 Examples of Nondeterministic Programs 378
4.3.3 Implementing the `amb` Evaluator 386

4.4 Logic Programming 398
4.4.1 Deductive Information Retrieval 401
4.4.2 How the Query System Works 411
4.4.3 Is Logic Programming Mathematical Logic? 419
4.4.4 Implementing the Query System 424

5 Computing with Register Machines 449

5.1 Designing Register Machines 450
5.1.1 A Language for Describing Register Machines 452
5.1.2 Abstraction in Machine Design 456
5.1.3 Subroutines 457
5.1.4 Using a Stack to Implement Recursion 462
5.1.5 Instruction Summary 467

5.2 A Register-Machine Simulator 468
5.2.1 The Machine Model 469
5.2.2 The Assembler 473
5.2.3 Instructions and Their Execution Functions 477
5.2.4 Monitoring Machine Performance 484

5.3 Storage Allocation and Garbage Collection 487
5.3.1 Memory as Vectors 488
5.3.2 Maintaining the Illusion of Infinite Memory 493

5.4 The Explicit-Control Evaluator 499
5.4.1 The Dispatcher and Basic Evaluation 500
5.4.2 Evaluating Function Applications 504

5.4.3 Blocks, Assignments, and Declarations 512
5.4.4 Running the Evaluator 513

5.5 Compilation 519
5.5.1 Structure of the Compiler 522
5.5.2 Compiling Components 527
5.5.3 Compiling Applications and Return Statements 535
5.5.4 Combining Instruction Sequences 543
5.5.5 An Example of Compiled Code 546
5.5.6 Lexical Addressing 554
5.5.7 Interfacing Compiled Code to the Evaluator 557

References 565

Index 571

List of Exercises 607

Foreword

I had the pleasure of meeting the amazing Alan Perlis and talking with him a few times, when I was still a student. He and I had in common a deep love and respect for two very different programming languages: Lisp and APL. Following in his footsteps is a daunting task, even though he blazed an excellent trail. Still, I would like to reexamine one comment he made in the original foreword to this book (and, please, I suggest that you read *his* foreword, which immediately follows this one, before you finish this one). Is it really true that it is better to have 100 functions operate on one data structure than to have 10 functions operate on 10 data structures?

To answer that question carefully, we first need to ask whether that one data structure is "universal": can it conveniently fulfill the roles of those 10 more specialized data structures?

For that matter, we can also ask: do we really need 100 functions? Is there a single universal function that can fulfill the roles of all those other functions?

The surprising answer to that last question is "yes"; it is only slightly tricky to construct a function that accepts (1) a data structure that serves as a description of some other function, and (2) a list of arguments, and behaves exactly as that other function would when applied to the given arguments. And it is only slightly tricky to design a data structure capable of describing any computation whatsoever. One such data structure (the tagged-list representation of expressions and statements, paired with environments that associate names with values) and one such universal function (`apply`) are described in Chapter 4 of this book. So maybe we need only one function and one data structure.

That is true in theory. In practice, we find it convenient to draw distinctions that help us, as human beings constructing descriptions of computations, to organize the structure of our code so that we can better understand them. I believe that Perlis was making a remark not about computational capability, but about human abilities and human limitations.

One thing the human mind seems to do well is to name things; we have powerful associative memories. Given a name, we can quickly recall some associated thing to mind. This is why we typically find it easier to work with the lambda calculus than the combinatory calculus; it is much easier for most people to interpret the Lisp expression `(lambda (x) (lambda (y) (+ x y)))` or the JavaScript expression `x => y => x + y` than the combinatory expression

((S ((S (K S)) ((S ((S (K S)) ((S (K K)) (K +)))) ((S (K K)) I)))) (K I))

even though there is a direct structural correspondence, easily expressed in five lines of Lisp code.

So while in principle we could get by with just one universal function, we prefer to modularize our code, to give names to the various pieces, and to mention the names of function descriptions rather than constantly feeding the descriptions themselves to the universal function.

In my 1998 talk "Growing a Language," I commented that a good programmer "does not just write programs. A good programmer builds a working vocabulary." As we design and define more and more parts of our programs, we give names to those parts, and the result is that we have a richer language in which to write the rest.

But we also find it natural to draw distinctions among data structures, and to give them names.

It may be that nested lists are a universal data structure (and it is worth noting that many modern and widely used data structures, such as HTML and XML and JSON, are also parenthetically nested representations, only slightly more elaborate than Lisp's bare parentheses). There are also many functions, such as finding the length of a list or applying a function to every element of a list and getting back a list of the results, that are useful in a wide variety of situations. And yet, when I am thinking about a specific computation, I often say to myself, "This list of two things I expect to be a personal name and a surname, but that list of two things I expect to be the real and imaginary parts of a complex number, and that other list of two things I will regard as the numerator and denominator of a fraction." In other words, I draw distinctions—and it may be useful to represent those distinctions explicitly in the data structure, in part to prevent mistakes such as accidentally treating a complex number as a fraction. (Again, this is a comment about human abilities and human limitations.)

Since the first edition of this book was written, almost four decades ago, a lot more ways of organizing data have become relatively standard, in particular the "object-oriented" approach, and many languages, including JavaScript, support specialized data structures such as objects and strings and heaps and maps with a variety of built-in mechanisms and libraries. But in doing so, many languages abandoned support for more general, universal notions. Java, for example, originally did not support first-class functions, and has incorporated them only relatively recently, greatly increasing its expressive power.

APL, likewise, originally did not support first-class functions, and moreover its original single data structure—arrays of any number of dimensions—was not so conveniently useful as a universal data structure because arrays could not contain other arrays as elements. More recent versions of APL do support anonymous function values and nested arrays, and these have made APL dramatically more expressive. (The original design of APL did have two very good things going for it: a comprehensive set of functions applicable to that one data structure, and moreover an extremely well chosen set of names for those functions. I'm not talking about the funny symbols and Greek letters, but the spoken words that APL programmers use when mentioning them, words like `shape`, `reshape`, `compress`, `expand`, and `laminate`; these are names not for the symbols, but for the functions they represent. Ken Iverson had a real knack for choosing short, memorable, vivid names for functions on arrays.)

While JavaScript, like Java, was originally designed with objects and methods in mind, it also incorporated first-class functions from the beginning, and it is not difficult to use its objects to define a universal data structure. As a result, JavaScript is not as distant from Lisp as you would think, and as this edition of *Structure and*

Interpretation of Computer Programs demonstrates, it is a good alternate framework for presenting the key ideas. *SICP* was never about a programming language; it presents powerful, general ideas for program organization that ought to be useful in any language.

What do Lisp and JavaScript have in common? The ability to abstract a computation (code plus some associated data) for later execution as a function; the ability to embed references to such functions within data structures; the ability to invoke functions on arguments; the ability to draw a distinction (conditional execution); a convenient universal data structure; completely automatic storage management for that data (which seems like a no-brainer, given everything else, until you realize that many widely used programming languages don't have it); a large set of useful functions for operating on that universal data structure; and standard strategies for using the universal data structure to represent more specialized data structures.

So maybe the truth is somewhere in between the extremes that Perlis so eloquently posited. Maybe the sweet spot is something more like 40 functions general enough to operate usefully on a universal data structure such as lists, but also 10 sets of 6 functions each that are relevant when we take one of 10 specialized views of that universal data structure. This is manageable if we give good names to these functions and specialized views.

As you read this book, please pay attention not only to the programming language constructs and how they are used, but also to the *names* given to functions and variables and data structures. They are not all as short and vivid as the names Iverson chose for his APL functions, but they have been chosen in a deliberate and systematic way to enhance your understanding of the overall program structure.

Primitives, means of combination, functional abstraction, naming, and conventions for using a universal data structure in specialized ways by drawing distinctions: these are the fundamental building blocks of a good programming language. From there, imagination and good engineering judgment based on experience can do the rest.

—Guy L. Steele Jr., Lexington, Massachusetts, 2021

Foreword to Structure and Interpretation of Computer Programs, 1984

Educators, generals, dieticians, psychologists, and parents program. Armies, students, and some societies are programmed. An assault on large problems employs a succession of programs, most of which spring into existence en route. These programs are rife with issues that appear to be particular to the problem at hand. To appreciate programming as an intellectual activity in its own right you must turn to computer programming; you must read and write computer programs—many of them. It doesn't matter much what the programs are about or what applications they serve. What does matter is how well they perform and how smoothly they fit with other programs in the creation of still greater programs. The programmer must seek both perfection of part and adequacy of collection. In this book the use of "program" is focused on the creation, execution, and study of programs written in a dialect of Lisp for execution on a digital computer. Using Lisp we restrict or limit not what we may program, but only the notation for our program descriptions.

Our traffic with the subject matter of this book involves us with three foci of phenomena: the human mind, collections of computer programs, and the computer. Every computer program is a model, hatched in the mind, of a real or mental process. These processes, arising from human experience and thought, are huge in number, intricate in detail, and at any time only partially understood. They are modeled to our permanent satisfaction rarely by our computer programs. Thus even though our programs are carefully handcrafted discrete collections of symbols, mosaics of interlocking functions, they continually evolve: we change them as our perception of the model deepens, enlarges, generalizes until the model ultimately attains a metastable place within still another model with which we struggle. The source of the exhilaration associated with computer programming is the continual unfolding within the mind and on the computer of mechanisms expressed as programs and the explosion of perception they generate. If art interprets our dreams, the computer executes them in the guise of programs!

For all its power, the computer is a harsh taskmaster. Its programs must be correct, and what we wish to say must be said accurately in every detail. As in every other symbolic activity, we become convinced of program truth through argument. Lisp itself can be assigned a semantics (another model, by the way), and if a program's function can be specified, say, in the predicate calculus, the proof methods of logic can be used to make an acceptable correctness argument. Unfortunately, as programs get large and complicated, as they almost always do, the adequacy, consistency, and correctness of the specifications themselves become open to doubt, so that complete formal arguments of correctness seldom accompany large programs. Since large programs grow from small ones, it is crucial that we develop an arsenal of standard program structures of whose correctness we have become sure—we call them idioms—and learn to combine them into larger structures using organizational techniques of proven value. These techniques are treated at length in this book, and understanding them is essential to participation in the Promethean enterprise called programming. More than anything else, the uncovering and mastery of powerful

organizational techniques accelerates our ability to create large, significant programs. Conversely, since writing large programs is very taxing, we are stimulated to invent new methods of reducing the mass of function and detail to be fitted into large programs.

Unlike programs, computers must obey the laws of physics. If they wish to perform rapidly—a few nanoseconds per state change—they must transmit electrons only small distances (at most $1\frac{1}{2}$ feet). The heat generated by the huge number of devices so concentrated in space has to be removed. An exquisite engineering art has been developed balancing between multiplicity of function and density of devices. In any event, hardware always operates at a level more primitive than that at which we care to program. The processes that transform our Lisp programs to "machine" programs are themselves abstract models which we program. Their study and creation give a great deal of insight into the organizational programs associated with programming arbitrary models. Of course the computer itself can be so modeled. Think of it: the behavior of the smallest physical switching element is modeled by quantum mechanics described by differential equations whose detailed behavior is captured by numerical approximations represented in computer programs executing on computers composed of . . . !

It is not merely a matter of tactical convenience to separately identify the three foci. Even though, as they say, it's all in the head, this logical separation induces an acceleration of symbolic traffic between these foci whose richness, vitality, and potential is exceeded in human experience only by the evolution of life itself. At best, relationships between the foci are metastable. The computers are never large enough or fast enough. Each breakthrough in hardware technology leads to more massive programming enterprises, new organizational principles, and an enrichment of abstract models. Every reader should ask himself periodically "Toward what end, toward what end?"—but do not ask it too often lest you pass up the fun of programming for the constipation of bittersweet philosophy.

Among the programs we write, some (but never enough) perform a precise mathematical function such as sorting or finding the maximum of a sequence of numbers, determining primality, or finding the square root. We call such programs algorithms, and a great deal is known of their optimal behavior, particularly with respect to the two important parameters of execution time and data storage requirements. A programmer should acquire good algorithms and idioms. Even though some programs resist precise specifications, it is the responsibility of the programmer to estimate, and always to attempt to improve, their performance.

Lisp is a survivor, having been in use for about a quarter of a century. Among the active programming languages only Fortran has had a longer life. Both languages have supported the programming needs of important areas of application, Fortran for scientific and engineering computation and Lisp for artificial intelligence. These two areas continue to be important, and their programmers are so devoted to these two languages that Lisp and Fortran may well continue in active use for at least another quarter-century.

Lisp changes. The Scheme dialect used in this text has evolved from the original Lisp and differs from the latter in several important ways, including static scoping for variable binding and permitting functions to yield functions as values. In its semantic structure Scheme is as closely akin to Algol 60 as to early Lisps. Algol 60,

never to be an active language again, lives on in the genes of Scheme and Pascal. It would be difficult to find two languages that are the communicating coin of two more different cultures than those gathered around these two languages. Pascal is for building pyramids—imposing, breathtaking, static structures built by armies pushing heavy blocks into place. Lisp is for building organisms—imposing, breathtaking, dynamic structures built by squads fitting fluctuating myriads of simpler organisms into place. The organizing principles used are the same in both cases, except for one extraordinarily important difference: The discretionary exportable functionality entrusted to the individual Lisp programmer is more than an order of magnitude greater than that to be found within Pascal enterprises. Lisp programs inflate libraries with functions whose utility transcends the application that produced them. The list, Lisp's native data structure, is largely responsible for such growth of utility. The simple structure and natural applicability of lists are reflected in functions that are amazingly nonidiosyncratic. In Pascal the plethora of declarable data structures induces a specialization within functions that inhibits and penalizes casual cooperation. It is better to have 100 functions operate on one data structure than to have 10 functions operate on 10 data structures. As a result the pyramid must stand unchanged for a millennium; the organism must evolve or perish.

To illustrate this difference, compare the treatment of material and exercises within this book with that in any first-course text using Pascal. Do not labor under the illusion that this is a text digestible at MIT only, peculiar to the breed found there. It is precisely what a serious book on programming Lisp must be, no matter who the student is or where it is used.

Note that this is a text about programming, unlike most Lisp books, which are used as a preparation for work in artificial intelligence. After all, the critical programming concerns of software engineering and artificial intelligence tend to coalesce as the systems under investigation become larger. This explains why there is such growing interest in Lisp outside of artificial intelligence.

As one would expect from its goals, artificial intelligence research generates many significant programming problems. In other programming cultures this spate of problems spawns new languages. Indeed, in any very large programming task a useful organizing principle is to control and isolate traffic within the task modules via the invention of language. These languages tend to become less primitive as one approaches the boundaries of the system where we humans interact most often. As a result, such systems contain complex language-processing functions replicated many times. Lisp has such a simple syntax and semantics that parsing can be treated as an elementary task. Thus parsing technology plays almost no role in Lisp programs, and the construction of language processors is rarely an impediment to the rate of growth and change of large Lisp systems. Finally, it is this very simplicity of syntax and semantics that is responsible for the burden and freedom borne by all Lisp programmers. No Lisp program of any size beyond a few lines can be written without being saturated with discretionary functions. Invent and fit; have fits and reinvent! We toast the Lisp programmer who pens his thoughts within nests of parentheses.

—Alan J. Perlis, New Haven, Connecticut

Preface

The book *Structure and Interpretation of Computer Programs* (SICP) introduces the reader to central ideas of computation by establishing a series of mental models for computation. Chapters 1–3 cover programming concepts that are common to all modern high-level programming languages. The original first two editions of SICP use the programming language Scheme in their program examples, whose minimalist, expression-oriented syntax allows the book to focus on the underlying ideas rather than the design of the chosen language. Chapters 4 and 5 use Scheme to formulate language processors for Scheme, deepening the readers' understanding of the mental models and exploring language extensions and alternatives.

Since its publication in 1984 and its second edition in 1996, SICP has been adopted as a textbook by universities and colleges around the world, including the National University of Singapore (NUS), which introduced the SICP-based introductory course CS1101S in 1997. In the mid-1990s, the languages Python, JavaScript, and Ruby emerged, which share central design elements with Scheme, but which employ a more complex, statement-oriented syntax that uses familiar algebraic (infix) notation. Their rise in popularity led instructors to adapt their SICP-based courses, typically by translating the example programs to their language of choice, by adding material specific to that language, and by omitting the material of chapters 4 and 5.

Adapting SICP to JavaScript

The work on adapting the second edition of SICP to JavaScript (SICP JS) started at NUS in 2008, and CS1101S switched to JavaScript in 2012. The language standard ECMAScript 2015 introduced lambda expressions, tail recursion, and block-scoped variables and constants, which enabled the adaptation to become quite close to the original. We made substantial changes to SICP only when we felt they were forced by differences between JavaScript and Scheme. The book covers just a small fraction of JavaScript, so a reader would be ill-advised to use it to learn the language. For example, the notion of a JavaScript object—considered one of its fundamental ingredients by any measure—is not even mentioned!

It was straightforward to translate the programs of chapters 1–3 to JavaScript by adding libraries that mirror Scheme primitives—including support for list structure—and adapting the text accordingly. However, the switch to JavaScript forced us to make subtle changes in the interpreters and compiler of chapters 4 and 5 in order to handle return statements. Scheme's expression-oriented syntax doesn't have return statements, which are a prominent feature of statement-oriented languages.

By using JavaScript, chapters 1–3 introduce the reader to the syntactic style of most mainstream languages today. However, that same syntactic style gave rise to significant changes in chapter 4, because the direct representation of programs as data structures could no longer be taken for granted. This provided us with an opportunity to introduce the reader to the notion of program parsing in section 4.1, an important component of programming-language processors. In section 4.4, the rigid syntactic structure of JavaScript complicated the implementation of the presented

logic programming system and exposed the limitations of JavaScript as a tool for programming language design.

Resources for using SICP JS

The MIT Press web page for SICP JS links to support for users of this book. This provides all programs from the book and extensive instructor resources, including a large collection of additional exercises and recommendations on selecting a subset of SICP JS that can be covered in a typical college semester. The JavaScript programs in the book run in the recommended strict mode in any JavaScript interpreter that complies with the ECMAScript 2020 specification of JavaScript (ECMA 2020). The MIT Press web page includes the JavaScript package `sicp`, which provides all JavaScript functions that are considered "primitive" in the book.

To the reader

We sincerely hope that if this book is your first encounter with programming you will use your newly gained understanding of the structure and interpretation of computer programs to learn more programming languages, including Scheme and the full JavaScript language. If you have learned JavaScript prior to picking up SICP JS, you might gain new insights into the fundamental concepts that underlie the language and discover how much can be achieved with so little. If you come to SICP JS with a knowledge of the original SICP, you might enjoy seeing familiar ideas presented in a new format—and might appreciate the online comparison edition, available at the book's web page, where SICP JS and SICP can be viewed side by side.

—Martin Henz and Tobias Wrigstad

Prefaces to Structure and Interpretation of Computer Programs, 1996 & 1984

Preface to the Second Edition of SICP, 1996

> Is it possible that software is not like anything else, that it is meant to be discarded: that the whole point is to always see it as a soap bubble?
>
> —Alan J. Perlis

The material in this book has been the basis of MIT's entry-level computer science subject since 1980. We had been teaching this material for four years when the first edition was published, and twelve more years have elapsed until the appearance of this second edition. We are pleased that our work has been widely adopted and incorporated into other texts. We have seen our students take the ideas and programs in this book and build them in as the core of new computer systems and languages. In literal realization of an ancient Talmudic pun, our students have become our builders. We are lucky to have such capable students and such accomplished builders.

In preparing this edition, we have incorporated hundreds of clarifications suggested by our own teaching experience and the comments of colleagues at MIT and elsewhere. We have redesigned most of the major programming systems in the book, including the generic-arithmetic system, the interpreters, the register-machine simulator, and the compiler; and we have rewritten all the program examples to ensure that any Scheme implementation conforming to the IEEE Scheme standard (IEEE 1990) will be able to run the code.

This edition emphasizes several new themes. The most important of these is the central role played by different approaches to dealing with time in computational models: objects with state, concurrent programming, functional programming, lazy evaluation, and nondeterministic programming. We have included new sections on concurrency and nondeterminism, and we have tried to integrate this theme throughout the book.

The first edition of the book closely followed the syllabus of our MIT one-semester subject. With all the new material in the second edition, it will not be possible to cover everything in a single semester, so the instructor will have to pick and choose. In our own teaching, we sometimes skip the section on logic programming (section 4.4), we have students use the register-machine simulator but we do not cover its implementation (section 5.2), and we give only a cursory overview of the compiler (section 5.5). Even so, this is still an intense course. Some instructors may wish to cover only the first three or four chapters, leaving the other material for subsequent courses.

The World Wide Web site of MIT Press provides support for users of this book. This includes programs from the book, sample programming assignments, supplementary materials, and downloadable implementations of the Scheme dialect of Lisp.

—Harold Abelson and Gerald Jay Sussman

Preface to the First Edition of SICP, 1984

> A computer is like a violin. You can imagine a novice trying first a phonograph and then a violin. The latter, he says, sounds terrible. That is the argument we have heard from our humanists and most of our computer scientists. Computer programs are good, they say, for particular purposes, but they aren't flexible. Neither is a violin, or a typewriter, until you learn how to use it.
>
> —Marvin Minsky, "Why Programming Is a Good Medium for Expressing Poorly-Understood and Sloppily-Formulated Ideas"

"The Structure and Interpretation of Computer Programs" is the entry-level subject in computer science at the Massachusetts Institute of Technology. It is required of all students at MIT who major in electrical engineering or in computer science, as one-fourth of the "common core curriculum," which also includes two subjects on circuits and linear systems and a subject on the design of digital systems. We have been involved in the development of this subject since 1978, and we have taught this material in its present form since the fall of 1980 to between 600 and 700 students each year. Most of these students have had little or no prior formal training in computation, although many have played with computers a bit and a few have had extensive programming or hardware-design experience.

Our design of this introductory computer-science subject reflects two major concerns. First, we want to establish the idea that a computer language is not just a way of getting a computer to perform operations but rather that it is a novel formal medium for expressing ideas about methodology. Thus, programs must be written for people to read, and only incidentally for machines to execute. Second, we believe that the essential material to be addressed by a subject at this level is not the syntax of particular programming-language constructs, nor clever algorithms for computing particular functions efficiently, nor even the mathematical analysis of algorithms and the foundations of computing, but rather the techniques used to control the intellectual complexity of large software systems.

Our goal is that students who complete this subject should have a good feel for the elements of style and the aesthetics of programming. They should have command of the major techniques for controlling complexity in a large system. They should be capable of reading a 50-page-long program, if it is written in an exemplary style. They should know what not to read, and what they need not understand at any moment. They should feel secure about modifying a program, retaining the spirit and style of the original author.

These skills are by no means unique to computer programming. The techniques we teach and draw upon are common to all of engineering design. We control complexity by building abstractions that hide details when appropriate. We control complexity by establishing conventional interfaces that enable us to construct systems by combining standard, well-understood pieces in a "mix and match" way. We control complexity by establishing new languages for describing a design, each of which emphasizes particular aspects of the design and deemphasizes others.

Underlying our approach to this subject is our conviction that "computer science" is not a science and that its significance has little to do with computers. The

computer revolution is a revolution in the way we think and in the way we express what we think. The essence of this change is the emergence of what might best be called *procedural epistemology*—the study of the structure of knowledge from an imperative point of view, as opposed to the more declarative point of view taken by classical mathematical subjects. Mathematics provides a framework for dealing precisely with notions of "what is." Computation provides a framework for dealing precisely with notions of "how to."

In teaching our material we use a dialect of the programming language Lisp. We never formally teach the language, because we don't have to. We just use it, and students pick it up in a few days. This is one great advantage of Lisp-like languages: They have very few ways of forming compound expressions, and almost no syntactic structure. All of the formal properties can be covered in an hour, like the rules of chess. After a short time we forget about syntactic details of the language (because there are none) and get on with the real issues—figuring out what we want to compute, how we will decompose problems into manageable parts, and how we will work on the parts. Another advantage of Lisp is that it supports (but does not enforce) more of the large-scale strategies for modular decomposition of programs than any other language we know. We can make procedural and data abstractions, we can use higher-order functions to capture common patterns of usage, we can model local state using assignment and data mutation, we can link parts of a program with streams and delayed evaluation, and we can easily implement embedded languages. All of this is embedded in an interactive environment with excellent support for incremental program design, construction, testing, and debugging. We thank all the generations of Lisp wizards, starting with John McCarthy, who have fashioned a fine tool of unprecedented power and elegance.

Scheme, the dialect of Lisp that we use, is an attempt to bring together the power and elegance of Lisp and Algol. From Lisp we take the metalinguistic power that derives from the simple syntax, the uniform representation of programs as data objects, and the garbage-collected heap-allocated data. From Algol we take lexical scoping and block structure, which are gifts from the pioneers of programming-language design who were on the Algol committee. We wish to cite John Reynolds and Peter Landin for their insights into the relationship of Church's lambda calculus to the structure of programming languages. We also recognize our debt to the mathematicians who scouted out this territory decades before computers appeared on the scene. These pioneers include Alonzo Church, Barkley Rosser, Stephen Kleene, and Haskell Curry.

—Harold Abelson and Gerald Jay Sussman

Acknowledgments

The JavaScript adaptation of *Structure and Interpretation of Computer Programs* (SICP JS) was developed at the National University of Singapore (NUS) for the course CS1101S. The course was co-taught for six years and counting by Low Kok Lim, whose sound pedagogical judgment was crucial for the success of the course and this project. The CS1101S teaching team has included many NUS colleagues and more than 300 undergraduate student assistants. Their continuous feedback over the past nine years allowed us to resolve countless JavaScript-specific issues and remove unnecessary complications and yet retain the essential features of both SICP and JavaScript.

SICP JS is a software project in addition to a book project. We obtained the LaTeX book sources from the original authors in 2008. An early SICP JS tool chain was developed by Liu Hang and refined by Feng Piaopiao. Chan Ger Hean developed the first tools for the print edition, based on which Jolyn Tan developed the tools for the first e-book edition and He Xinyue and Wang Qian repurposed these tools for the current comparison edition. Samuel Fang designed and developed the online edition of SICP JS.

The online edition of SICP JS and CS1101S rely heavily on a software system called *Source Academy*, and the JavaScript sublanguages it supports are called *Source*. Many dozens of students have contributed to the Source Academy during the preparation of SICP JS, and the system lists them prominently as "Contributors." Since 2020, the students of the NUS course CS4215, Programming Language Implementation, contributed several programming language implementations that are used in SICP JS: The concurrent version of Source used in section 3.4 was developed by Zhengqun Koo and Jonathan Chan; the lazy implementation used in section 4.2 was developed by Jellouli Ahmed, Ian Kendall Duncan, Cruz Jomari Evangelista, and Alden Tan; the nondeterministic implementation used in section 4.3 was developed by Arsalan Cheema and Anubhav; and Daryl Tan helped integrate these implementations into the Academy.

We are grateful to STINT, The Swedish Foundation for International Cooperation in Research and Higher Education, whose sabbatical program connected Martin and Tobias and allowed Tobias to work as a co-teacher of CS1101S and join the SICP JS project.

We would like to acknowledge the courageous work of the committee of ECMAScript 2015, led by Allen Wirfs-Brock. SICP JS relies heavily on constant and let declarations and lambda expressions, all of which were added to JavaScript with ECMAScript 2015. Those additions allowed us to stay close to the original in the presentation of the most important ideas of SICP. Guy Lewis Steele Jr., who led the first ECMAScript standardization, provided detailed and useful feedback on some exercises of chapter 4.

—Martin Henz and Tobias Wrigstad

Acknowledgments from the Second Edition of SICP, 1996

We would like to thank the many people who have helped us develop this book and this curriculum.

Our subject is a clear intellectual descendant of "6.231," a wonderful subject on programming linguistics and the lambda calculus taught at MIT in the late 1960s by Jack Wozencraft and Arthur Evans, Jr.

We owe a great debt to Robert Fano, who reorganized MIT's introductory curriculum in electrical engineering and computer science to emphasize the principles of engineering design. He led us in starting out on this enterprise and wrote the first set of subject notes from which this book evolved.

Much of the style and aesthetics of programming that we try to teach were developed in conjunction with Guy Lewis Steele Jr., who collaborated with Gerald Jay Sussman in the initial development of the Scheme language. In addition, David Turner, Peter Henderson, Dan Friedman, David Wise, and Will Clinger have taught us many of the techniques of the functional programming community that appear in this book.

Joel Moses taught us about structuring large systems. His experience with the Macsyma system for symbolic computation provided the insight that one should avoid complexities of control and concentrate on organizing the data to reflect the real structure of the world being modeled.

Marvin Minsky and Seymour Papert formed many of our attitudes about programming and its place in our intellectual lives. To them we owe the understanding that computation provides a means of expression for exploring ideas that would otherwise be too complex to deal with precisely. They emphasize that a student's ability to write and modify programs provides a powerful medium in which exploring becomes a natural activity.

We also strongly agree with Alan Perlis that programming is lots of fun and we had better be careful to support the joy of programming. Part of this joy derives from observing great masters at work. We are fortunate to have been apprentice programmers at the feet of Bill Gosper and Richard Greenblatt.

It is difficult to identify all the people who have contributed to the development of our curriculum. We thank all the lecturers, recitation instructors, and tutors who have worked with us over the past fifteen years and put in many extra hours on our subject, especially Bill Siebert, Albert Meyer, Joe Stoy, Randy Davis, Louis Braida, Eric Grimson, Rod Brooks, Lynn Stein, and Peter Szolovits. We would like to specially acknowledge the outstanding teaching contributions of Franklyn Turbak, now at Wellesley; his work in undergraduate instruction set a standard that we can all aspire to. We are grateful to Jerry Saltzer and Jim Miller for helping us grapple with the mysteries of concurrency, and to Peter Szolovits and David McAllester for their contributions to the exposition of nondeterministic evaluation in chapter 4.

Many people have put in significant effort presenting this material at other universities. Some of the people we have worked closely with are Jacob Katzenelson at the Technion, Hardy Mayer at the University of California at Irvine, Joe Stoy at Oxford, Elisha Sacks at Purdue, and Jan Komorowski at the Norwegian University of Science and Technology. We are exceptionally proud of our colleagues who have received major teaching awards for their adaptations of this subject at other universities,

including Kenneth Yip at Yale, Brian Harvey at the University of California at Berkeley, and Dan Huttenlocher at Cornell.

Al Moyé arranged for us to teach this material to engineers at Hewlett-Packard, and for the production of videotapes of these lectures. We would like to thank the talented instructors—in particular Jim Miller, Bill Siebert, and Mike Eisenberg—who have designed continuing education courses incorporating these tapes and taught them at universities and industry all over the world.

Many educators in other countries have put in significant work translating the first edition. Michel Briand, Pierre Chamard, and André Pic produced a French edition; Susanne Daniels-Herold produced a German edition; and Fumio Motoyoshi produced a Japanese edition. We do not know who produced the Chinese edition, but we consider it an honor to have been selected as the subject of an "unauthorized" translation.

It is hard to enumerate all the people who have made technical contributions to the development of the Scheme systems we use for instructional purposes. In addition to Guy Steele, principal wizards have included Chris Hanson, Joe Bowbeer, Jim Miller, Guillermo Rozas, and Stephen Adams. Others who have put in significant time are Richard Stallman, Alan Bawden, Kent Pitman, Jon Taft, Neil Mayle, John Lamping, Gwyn Osnos, Tracy Larrabee, George Carrette, Soma Chaudhuri, Bill Chiarchiaro, Steven Kirsch, Leigh Klotz, Wayne Noss, Todd Cass, Patrick O'Donnell, Kevin Theobald, Daniel Weise, Kenneth Sinclair, Anthony Courtemanche, Henry M. Wu, Andrew Berlin, and Ruth Shyu.

Beyond the MIT implementation, we would like to thank the many people who worked on the IEEE Scheme standard, including William Clinger and Jonathan Rees, who edited the R^{4}RS, and Chris Haynes, David Bartley, Chris Hanson, and Jim Miller, who prepared the IEEE standard.

Dan Friedman has been a long-time leader of the Scheme community. The community's broader work goes beyond issues of language design to encompass significant educational innovations, such as the high-school curriculum based on EdScheme by Schemer's Inc., and the wonderful books by Mike Eisenberg and by Brian Harvey and Matthew Wright.

We appreciate the work of those who contributed to making this a real book, especially Terry Ehling, Larry Cohen, and Paul Bethge at the MIT Press. Ella Mazel found the wonderful cover image. For the second edition we are particularly grateful to Bernard and Ella Mazel for help with the book design, and to David Jones, TEX wizard extraordinaire. We also are indebted to those readers who made penetrating comments on the new draft: Jacob Katzenelson, Hardy Mayer, Jim Miller, and especially Brian Harvey, who did unto this book as Julie did unto his book *Simply Scheme*.

Finally, we would like to acknowledge the support of the organizations that have encouraged this work over the years, including support from Hewlett-Packard, made possible by Ira Goldstein and Joel Birnbaum, and support from DARPA, made possible by Bob Kahn.

—Harold Abelson and Gerald Jay Sussman

Structure and Interpretation of Computer Programs

JavaScript Edition

1 Building Abstractions with Functions

> The acts of the mind, wherein it exerts its power over simple ideas, are chiefly these three: 1. Combining several simple ideas into one compound one, and thus all complex ideas are made. 2. The second is bringing two ideas, whether simple or complex, together, and setting them by one another so as to take a view of them at once, without uniting them into one, by which it gets all its ideas of relations. 3. The third is separating them from all other ideas that accompany them in their real existence: this is called abstraction, and thus all its general ideas are made.
>
> —John Locke, An Essay Concerning Human Understanding (1690)

We are about to study the idea of a *computational process*. Computational processes are abstract beings that inhabit computers. As they evolve, processes manipulate other abstract things called *data*. The evolution of a process is directed by a pattern of rules called a *program*. People create programs to direct processes. In effect, we conjure the spirits of the computer with our spells.

A computational process is indeed much like a sorcerer's idea of a spirit. It cannot be seen or touched. It is not composed of matter at all. However, it is very real. It can perform intellectual work. It can answer questions. It can affect the world by disbursing money at a bank or by controlling a robot arm in a factory. The programs we use to conjure processes are like a sorcerer's spells. They are carefully composed from symbolic expressions in arcane and esoteric *programming languages* that prescribe the tasks we want our processes to perform.

A computational process, in a correctly working computer, executes programs precisely and accurately. Thus, like the sorcerer's apprentice, novice programmers must learn to understand and to anticipate the consequences of their conjuring. Even small errors (usually called *bugs*) in programs can have complex and unanticipated consequences.

Fortunately, learning to program is considerably less dangerous than learning sorcery, because the spirits we deal with are conveniently contained in a secure way. Real-world programming, however, requires care, expertise, and wisdom. A small bug in a computer-aided design program, for example, can lead to the catastrophic collapse of an airplane or a dam or the self-destruction of an industrial robot.

Master software engineers have the ability to organize programs so that they can be reasonably sure that the resulting processes will perform the tasks intended. They can visualize the behavior of their systems in advance. They know how to structure programs so that unanticipated problems do not lead to catastrophic consequences, and when problems do arise, they can *debug* their programs. Well-designed computational systems, like well-designed automobiles or nuclear reactors, are designed in a modular manner, so that the parts can be constructed, replaced, and debugged separately.

Programming in JavaScript

We need an appropriate language for describing processes, and we will use for this purpose the programming language JavaScript. Just as our everyday thoughts are usually expressed in our natural language (such as English, Swedish, or Chinese), and descriptions of quantitative phenomena are expressed with mathematical notations, our procedural thoughts will be expressed in JavaScript. JavaScript was developed in 1995 as a programming language for controlling the behavior of World Wide Web browsers through scripts that are embedded in web pages. The language was conceived by Brendan Eich, originally under the name *Mocha*, which was later renamed to *LiveScript*, and finally to JavaScript. The name "JavaScript" is a trademark of Oracle Corporation.

Despite its inception as a language for scripting the web, JavaScript is a general-purpose programming language. A JavaScript *interpreter* is a machine that carries out processes described in the JavaScript language. The first JavaScript interpreter was implemented by Eich at Netscape Communications Corporation for the Netscape Navigator web browser. JavaScript inherited its core features from the Scheme and Self programming languages. Scheme is a dialect of Lisp, and was used as the programming language for the original version of this book. From Scheme, JavaScript inherited its most fundamental design principles, such as lexically scoped first-class functions and dynamic typing.

JavaScript bears only superficial resemblance to the language Java, after which it was (eventually) named; both Java and JavaScript use the block structure of the language C. In contrast with Java and C, which usually employ compilation to lower-level languages, JavaScript programs were initially *interpreted* by web browsers. After Netscape Navigator, other web browsers provided interpreters for the language, including Microsoft's Internet Explorer, whose JavaScript version is called *JScript*. The popularity of JavaScript for controlling web browsers gave rise to a standardization effort, culminating in *ECMAScript*. The first edition of the ECMAScript standard was led by Guy Lewis Steele Jr. and completed in June 1997 (ECMA 1997). The sixth edition, known as ECMAScript 2015, was led by Allen Wirfs-Brock and adopted by the General Assembly of ECMA in June 2015 (ECMA 2015).

The practice of embedding JavaScript programs in web pages encouraged the developers of web browsers to implement JavaScript interpreters. As these programs became more complex, the interpreters became more efficient in executing them, eventually using sophisticated implementation techniques such as Just-In-Time (JIT) compilation. The majority of JavaScript programs as of this writing (2021) are embedded in web pages and interpreted by browsers, but JavaScript is increasingly used as a general-purpose programming language, using systems such as Node.js.

ECMAScript 2015 possesses a set of features that make it an excellent medium for studying important programming constructs and data structures and for relating them to the linguistic features that support them. Its lexically scoped first-class functions and their syntactic support through lambda expressions provide direct and concise access to functional abstraction, and dynamic typing allows the adaptation to remain close to the Scheme original throughout the book. Above and beyond these considerations, programming in JavaScript is great fun.

1.1 The Elements of Programming

A powerful programming language is more than just a means for instructing a computer to perform tasks. The language also serves as a framework within which we organize our ideas about processes. Thus, when we describe a language, we should pay particular attention to the means that the language provides for combining simple ideas to form more complex ideas. Every powerful language has three mechanisms for accomplishing this:

- **primitive expressions**, which represent the simplest entities the language is concerned with,
- **means of combination**, by which compound elements are built from simpler ones, and
- **means of abstraction**, by which compound elements can be named and manipulated as units.

In programming, we deal with two kinds of elements: functions and data. (Later we will discover that they are really not so distinct.) Informally, data is "stuff" that we want to manipulate, and functions are descriptions of the rules for manipulating the data. Thus, any powerful programming language should be able to describe primitive data and primitive functions and should have methods for combining and abstracting functions and data.

In this chapter we will deal only with simple numerical data so that we can focus on the rules for building functions.[1] In later chapters we will see that these same rules allow us to build functions to manipulate compound data as well.

1.1.1 Expressions

One easy way to get started at programming is to examine some typical interactions with an interpreter for the JavaScript language. You type a *statement*, and the interpreter responds by displaying the result of its *evaluating* that statement.

One kind of statement you might type is an expression statement, which consists of an *expression* followed by a semicolon. One kind of primitive expression is a number. (More precisely, the expression that you type consists of the numerals that represent the number in base 10.) If you present JavaScript with the program

1. The characterization of numbers as "simple data" is a barefaced bluff. In fact, the treatment of numbers is one of the trickiest and most confusing aspects of any programming language. Some typical issues involved are these: Some computer systems distinguish *integers*, such as 2, from *real numbers*, such as 2.71. Is the real number 2.00 different from the integer 2? Are the arithmetic operations used for integers the same as the operations used for real numbers? Does 6 divided by 2 produce 3, or 3.0? How large a number can we represent? How many decimal places of accuracy can we represent? Is the range of integers the same as the range of real numbers? Above and beyond these questions, of course, lies a collection of issues concerning roundoff and truncation errors—the entire science of numerical analysis. Since our focus in this book is on large-scale program design rather than on numerical techniques, we are going to ignore these problems. The numerical examples in this chapter will exhibit the usual roundoff behavior that one observes when using arithmetic operations that preserve a limited number of decimal places of accuracy in noninteger operations.

```
486;
```

the interpreter will respond by printing[2]

```
486
```

Expressions representing numbers may be combined with operators (such as + or *) to form a compound expression that represents the application of a corresponding primitive function to those numbers. For example,

```
137 + 349;
486

1000 - 334;
666

5 * 99;
495

10 / 4;
2.5

2.7 + 10;
12.7
```

Expressions such as these, which contain other expressions as components, are called *combinations*. Combinations that are formed by an *operator* symbol in the middle, and *operand* expressions to the left and right of it, are called *operator combinations*. The value of an operator combination is obtained by applying the function specified by the operator to the arguments that are the values of the operands.

The convention of placing the operator between the operands is known as *infix notation*. It follows the mathematical notation that you are most likely familiar with from school and everyday life. As in mathematics, operator combinations can be *nested*, that is, they can have operands that themselves are operator combinations:

```
(3 * 5) + (10 - 6);
19
```

As usual, parentheses are used to group operator combinations in order to avoid ambiguities. JavaScript also follows the usual conventions when parentheses are omitted: multiplication and division bind more strongly than addition and subtraction. For example,

```
3 * 5 + 10 / 2;
```

stands for

```
(3 * 5) + (10 / 2);
```

2. Throughout this book, we distinguish between the input typed by the user and any text printed by the interpreter by showing the latter in slanted characters.

We say that `*` and `/` have *higher precedence* than `+` and `-`. Sequences of additions and subtractions are read from left to right, as are sequences of multiplications and divisions. Thus,

```
1 - 5 / 2 * 4 + 3;
```

stands for

```
(1 - ((5 / 2) * 4)) + 3;
```

We say that the operators `+`, `-`, `*` and `/` are *left-associative*.

There is no limit (in principle) to the depth of such nesting and to the overall complexity of the expressions that the JavaScript interpreter can evaluate. It is we humans who might get confused by still relatively simple expressions such as

```
3 * 2 * (3 - 5 + 4) + 27 / 6 * 10;
```

which the interpreter would readily evaluate to be 57. We can help ourselves by writing such an expression in the form

```
3 * 2 * (3 - 5 + 4)
+
27 / 6 * 10;
```

to visually separate the major components of the expression.

Even with complex expressions, the interpreter always operates in the same basic cycle: It reads a statement typed by the user, evaluates the statement, and prints the result. This mode of operation is often expressed by saying that the interpreter runs in a *read-evaluate-print loop*. Observe in particular that it is not necessary to explicitly instruct the interpreter to print the value of the statement.[3]

1.1.2 Naming and the Environment

A critical aspect of a programming language is the means it provides for using names to refer to computational objects, and our first such means are *constants*. We say that the name identifies a constant whose *value* is the object.

In JavaScript, we name constants with *constant declarations*.

```
const size = 2;
```

causes the interpreter to associate the value 2 with the name `size`.[4] Once the name `size` has been associated with the number 2, we can refer to the value 2 by name:

---

3. JavaScript obeys the convention that every statement has a value (see exercise 4.8). This convention, together with the reputation of JavaScript programmers as not caring about efficiency, leads us to paraphrase a quip on Lisp programmers by Alan Perlis (who was himself paraphrasing Oscar Wilde): *JavaScript programmers know the value of everything but the cost of nothing.*

4. In this book, we do not show the interpreter's response to evaluating programs that end with declarations, since this might depend on previous statements. See exercise 4.8 for details.

```
size;
2

5 * size;
10
```

Here are further examples of the use of `const`:

```
const pi = 3.14159;

const radius = 10;

pi * radius * radius;
314.159

const circumference = 2 * pi * radius;

circumference;
62.8318
```

Constant declaration is our language's simplest means of abstraction, for it allows us to use simple names to refer to the results of compound operations, such as the `circumference` computed above. In general, computational objects may have very complex structures, and it would be extremely inconvenient to have to remember and repeat their details each time we want to use them. Indeed, complex programs are constructed by building, step by step, computational objects of increasing complexity. The interpreter makes this step-by-step program construction particularly convenient because name-object associations can be created incrementally in successive interactions. This feature encourages the incremental development and testing of programs and is largely responsible for the fact that a JavaScript program usually consists of a large number of relatively simple functions.

It should be clear that the possibility of associating values with names and later retrieving them means that the interpreter must maintain some sort of memory that keeps track of the name-object pairs. This memory is called the *environment* (more precisely the *program environment*, since we will see later that a computation may involve a number of different environments).[5]

1.1.3 Evaluating Operator Combinations

One of our goals in this chapter is to isolate issues about thinking procedurally. As a case in point, let us consider that, in evaluating operator combinations, the interpreter is itself following a procedure.

- To evaluate an operator combination, do the following:
  1. Evaluate the operand expressions of the combination.
  2. Apply the function that is denoted by the operator to the arguments that are the values of the operands.

5. Chapter 3 will show that this notion of environment is crucial for understanding how the interpreter works. Chapter 4 will use environments for implementing interpreters.

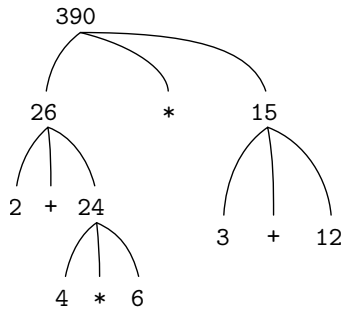


Figure 1.1 Tree representation, showing the value of each subexpression.

Even this simple rule illustrates some important points about processes in general. First, observe that the first step dictates that in order to accomplish the evaluation process for a combination we must first perform the evaluation process on each operand of the combination. Thus, the evaluation rule is *recursive* in nature; that is, it includes, as one of its steps, the need to invoke the rule itself.

Notice how succinctly the idea of recursion can be used to express what, in the case of a deeply nested combination, would otherwise be viewed as a rather complicated process. For example, evaluating

```
(2 + 4 * 6) * (3 + 12);
```

requires that the evaluation rule be applied to four different combinations. We can obtain a picture of this process by representing the combination in the form of a tree, as shown in figure 1.1. Each combination is represented by a node with branches corresponding to the operator and the operands of the combination stemming from it. The terminal nodes (that is, nodes with no branches stemming from them) represent either operators or numbers. Viewing evaluation in terms of the tree, we can imagine that the values of the operands percolate upward, starting from the terminal nodes and then combining at higher and higher levels. In general, we shall see that recursion is a very powerful technique for dealing with hierarchical, treelike objects. In fact, the "percolate values upward" form of the evaluation rule is an example of a general kind of process known as *tree accumulation*.

Next, observe that the repeated application of the first step brings us to the point where we need to evaluate, not combinations, but primitive expressions such as numerals or names. We take care of the primitive cases by stipulating that

- the values of numerals are the numbers that they name, and
- the values of names are the objects associated with those names in the environment.

The key point to notice is the role of the environment in determining the meaning of the names in expressions. In an interactive language such as JavaScript, it is meaningless to speak of the value of an expression such as `x + 1` without specifying any information about the environment that would provide a meaning for the name `x`. As we shall see in chapter 3, the general notion of the environment as providing a context in which evaluation takes place will play an important role in our understanding of program execution.

Notice that the evaluation rule given above does not handle declarations. For instance, evaluating `const x = 3;` does not apply an equality operator `=` to two arguments, one of which is the value of the name `x` and the other of which is 3, since the purpose of the declaration is precisely to associate `x` with a value. (That is, `const x = 3;` is not a combination.)

The letters in `const` are rendered in bold to indicate that it is a *keyword* in JavaScript. Keywords carry a particular meaning, and thus cannot be used as names. A keyword or a combination of keywords in a statement instructs the JavaScript interpreter to treat the statement in a special way. Each such *syntactic form* has its own evaluation rule. The various kinds of statements and expressions (each with its associated evaluation rule) constitute the syntax of the programming language.

1.1.4 Compound Functions

We have identified in JavaScript some of the elements that must appear in any powerful programming language:

- Numbers and arithmetic operations are primitive data and functions.
- Nesting of combinations provides a means of combining operations.
- Constant declarations that associate names with values provide a limited means of abstraction.

Now we will learn about *function declarations*, a much more powerful abstraction technique by which a compound operation can be given a name and then referred to as a unit.

We begin by examining how to express the idea of "squaring." We might say, "To square something, take it times itself." This is expressed in our language as

```
function square(x) {
    return x * x;
}
```

We can understand this in the following way:

```
function square(     x     ) { return x   *     x   ; }
   ↑         ↑         ↑                  ↑   ↑     ↑
   |         |         |                  |   |     |
   To      square  something,          take  it times itself.
```

We have here a *compound function*, which has been given the name `square`. The function represents the operation of multiplying something by itself. The thing to be multiplied is given a local name, `x`, which plays the same role that a pronoun plays in natural language. Evaluating the declaration creates this compound function and associates it with the name `square`.[6]

The simplest form of a function declaration is

`function` *name*`(`*parameters*`) { return` *expression*`; }`

6. Observe that there are two different operations being combined here: we are creating the function, and we are giving it the name `square`. It is possible, indeed important, to be able to separate these two notions—to create functions without naming them, and to give names to functions that have already been created. We will see how to do this in section 1.3.2.

The *name* is a symbol to be associated with the function definition in the environment.[7] The *parameters* are the names used within the body of the function to refer to the corresponding arguments of the function. The *parameters* are grouped within parentheses and separated by commas, as they will be in an application of the function being declared. In the simplest form, the *body* of a function declaration is a single *return statement*,[8] which consists of the keyword `return` followed by the *return expression* that will yield the value of the function application, when the parameters are replaced by the actual arguments to which the function is applied. Like constant declarations and expression statements, return statements end with a semicolon.

Having declared `square`, we can now use it in a *function application* expression, which we turn into a statement using a semicolon:

```
square(21);
441
```

Function applications are—after operator combinations—the second kind of combination of expressions into larger expressions that we encounter. The general form of a function application is

function-expression`(`*argument-expressions*`)`

where the *function-expression* of the application specifies the function to be applied to the comma-separated *argument-expressions*. To evaluate a function application, the interpreter follows a procedure quite similar to the procedure for operator combinations described in section 1.1.3.

- To evaluate a function application, do the following:
  1. Evaluate the subexpressions of the application, namely the function expression and the argument expressions.
  2. Apply the function that is the value of the function expression to the values of the argument expressions.

```
square(2 + 5);
49
```

Here, the argument expression is itself a compound expression, the operator combination `2 + 5`.

```
square(square(3));
81
```

---

7. Throughout this book, we will describe the general syntax of expressions by using italic symbols—e.g., *name*—to denote the "slots" in the expression to be filled in when such an expression is actually used.

8. More generally, the body of the function can be a sequence of statements. In this case, the interpreter evaluates each statement in the sequence in turn until a return statement determines the value of the function application.

Of course function application expressions can also serve as argument expressions.

We can also use `square` as a building block in defining other functions. For example, $x^2 + y^2$ can be expressed as

```
square(x) + square(y)
```

We can easily declare a function `sum_of_squares`[9] that, given any two numbers as arguments, produces the sum of their squares:

```
function sum_of_squares(x, y) {
    return square(x) + square(y);
}

sum_of_squares(3, 4);
```
25

Now we can use `sum_of_squares` as a building block in constructing further functions:

```
function f(a) {
    return sum_of_squares(a + 1, a * 2);
}

f(5);
```
136

In addition to compound functions, any JavaScript environment provides primitive functions that are built into the interpreter or loaded from libraries. Besides the primitive functions provided by the operators, the JavaScript environment used in this book includes additional primitive functions such as the function `math_log`, which computes the natural logarithm of its argument.[10] These additional primitive functions are used in exactly the same way as compound functions; evaluating the application `math_log(1)` results in the number 0. Indeed, one could not tell by looking at the definition of `sum_of_squares` given above whether `square` was built into the interpreter, loaded from a library, or defined as a compound function.

---

9. The way multi-part names such as `sum_of_squares` are written affects the readability of programs, and programming communities differ on this. According to the common JavaScript convention, called *camel case*, the name would be `sumOfSquares`. The convention used in this book is called *snake case*, and was chosen for its closer resemblance to the convention used in the Scheme version of this book, where hyphens play the role of our underscores.

10. Our JavaScript environment includes all functions and constants of ECMAScript's Math object, under the names `math_`.... For example, ECMAScript's `Math.log` is available as `math_log`. The MIT Press web page for this book includes the JavaScript package `sicp` that provides these and all other JavaScript functions that are considered primitive in the book.

1.1.5 The Substitution Model for Function Application

To evaluate a function application, the interpreter follows the process described in section 1.1.4. That is, the interpreter evaluates the elements of the application and applies the function (which is the value of the function expression of the application) to the arguments (which are the values of the argument expressions of the application).

We can assume that the application of primitive functions is handled by the interpreter or libraries. For compound functions, the application process is as follows:

- To apply a compound function to arguments, evaluate the return expression of the function with each parameter replaced by the corresponding argument.[11]

To illustrate this process, let's evaluate the application

```
f(5)
```

where `f` is the function declared in section 1.1.4. We begin by retrieving the return expression of `f`:

```
sum_of_squares(a + 1, a * 2)
```

Then we replace the parameter `a` by the argument 5:

```
sum_of_squares(5 + 1, 5 * 2)
```

Thus the problem reduces to the evaluation of an application with two arguments and a function expression `sum_of_squares`. Evaluating this application involves three subproblems. We must evaluate the function expression to get the function to be applied, and we must evaluate the argument expressions to get the arguments. Now `5 + 1` produces 6 and `5 * 2` produces 10, so we must apply the `sum_of_squares` function to 6 and 10. These values are substituted for the parameters `x` and `y` in the body of `sum_of_squares`, reducing the expression to

```
square(6) + square(10)
```

If we use the declaration of `square`, this reduces to

```
(6 * 6) + (10 * 10)
```

which reduces by multiplication to

```
36 + 100
```

and finally to

```
136
```

The process we have just described is called the *substitution model* for function application. It can be taken as a model that determines the "meaning" of function

11. If the body of the function is a sequence of statements, the body is evaluated with the parameters replaced, and the value of the application is the value of the return expression of the first return statement encountered.

application, insofar as the functions in this chapter are concerned. However, there are two points that should be stressed:

- The purpose of the substitution is to help us think about function application, not to provide a description of how the interpreter really works. Typical interpreters do not evaluate function applications by manipulating the text of a function to substitute values for the parameters. In practice, the "substitution" is accomplished by using a local environment for the parameters. We will discuss this more fully in chapters 3 and 4 when we examine the implementation of an interpreter in detail.
- Over the course of this book, we will present a sequence of increasingly elaborate models of how interpreters work, culminating with a complete implementation of an interpreter and compiler in chapter 5. The substitution model is only the first of these models—a way to get started thinking formally about the evaluation process. In general, when modeling phenomena in science and engineering, we begin with simplified, incomplete models. As we examine things in greater detail, these simple models become inadequate and must be replaced by more refined models. The substitution model is no exception. In particular, when we address in chapter 3 the use of functions with "mutable data," we will see that the substitution model breaks down and must be replaced by a more complicated model of function application.[12]

Applicative order versus normal order

According to the description of evaluation given in section 1.1.4, the interpreter first evaluates the function and argument expressions and then applies the resulting function to the resulting arguments. This is not the only way to perform evaluation. An alternative evaluation model would not evaluate the arguments until their values were needed. Instead it would first substitute argument expressions for parameters until it obtained an expression involving only operators and primitive functions, and would then perform the evaluation. If we used this method, the evaluation of

```
f(5)
```

would proceed according to the sequence of expansions

```
sum_of_squares(5 + 1, 5 * 2)
```

```
square(5 + 1)     + square(5 * 2)
```

```
(5 + 1) * (5 + 1) + (5 * 2) * (5 * 2)
```

followed by the reductions

12. Despite the simplicity of the substitution idea, it turns out to be surprisingly complicated to give a rigorous mathematical definition of the substitution process. The problem arises from the possibility of confusion between the names used for the parameters of a function and the (possibly identical) names used in the expressions to which the function may be applied. Indeed, there is a long history of erroneous definitions of *substitution* in the literature of logic and programming semantics. See Stoy 1977 for a careful discussion of substitution.

```
6    *    6    +    10   *   10

    36         +        100

               136
```

This gives the same answer as our previous evaluation model, but the process is different. In particular, the evaluations of `5 + 1` and `5 * 2` are each performed twice here, corresponding to the reduction of the expression

```
x * x
```

with `x` replaced respectively by `5 + 1` and `5 * 2`.

This alternative "fully expand and then reduce" evaluation method is known as *normal-order evaluation*, in contrast to the "evaluate the arguments and then apply" method that the interpreter actually uses, which is called *applicative-order evaluation*. It can be shown that, for function applications that can be modeled using substitution (including all the functions in the first two chapters of this book) and that yield legitimate values, normal-order and applicative-order evaluation produce the same value. (See exercise 1.5 for an instance of an "illegitimate" value where normal-order and applicative-order evaluation do not give the same result.)

JavaScript uses applicative-order evaluation, partly because of the additional efficiency obtained from avoiding multiple evaluations of expressions such as those illustrated with `5 + 1` and `5 * 2` above and, more significantly, because normal-order evaluation becomes much more complicated to deal with when we leave the realm of functions that can be modeled by substitution. On the other hand, normal-order evaluation can be an extremely valuable tool, and we will investigate some of its implications in chapters 3 and 4.[13]

1.1.6 Conditional Expressions and Predicates

The expressive power of the class of functions that we can define at this point is very limited, because we have no way to make tests and to perform different operations depending on the result of a test. For instance, we cannot declare a function that computes the absolute value of a number by testing whether the number is nonnegative and taking different actions in each case according to the rule

$$|x| = \begin{cases} x & \text{if } x \geq 0 \\ -x & \text{otherwise} \end{cases}$$

This construct is a *case analysis* and can be written in JavaScript using a *conditional expression* as

```
function abs(x) {
    return x >= 0 ? x : - x;
}
```

13. In chapter 3 we will introduce *stream processing*, which is a way of handling apparently "infinite" data structures by incorporating a limited form of normal-order evaluation. In section 4.2 we will modify the JavaScript interpreter to produce a normal-order variant of JavaScript.

which could be expressed in English as "If x is greater than or equal to zero, return x; otherwise return $-x$." The general form of a conditional expression is

predicate `?` *consequent-expression* `:` *alternative-expression*

Conditional expressions begin with a *predicate*—that is, an expression whose value is either *true* or *false*, two distinguished *boolean* values in JavaScript. The primitive boolean expressions `true` and `false` trivially evaluate to the boolean values true and false, respectively. The *predicate* is followed by a question mark, the *consequent-expression*, a colon, and finally the *alternative-expression*.

To evaluate a conditional expression, the interpreter starts by evaluating the *predicate* of the expression. If the *predicate* evaluates to true, the interpreter evaluates the *consequent-expression* and returns its value as the value of the conditional. If the *predicate* evaluates to false, it evaluates the *alternative-expression* and returns its value as the value of the conditional.[14]

The word *predicate* is used for operators and functions that return true or false, as well as for expressions that evaluate to true or false. The absolute-value function `abs` makes use of the primitive predicate `>=`, an operator that takes two numbers as arguments and tests whether the first number is greater than or equal to the second number, returning true or false accordingly.

If we prefer to handle the zero case separately, we can specify the function that computes the absolute value of a number by writing

$$|x| = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -x & \text{otherwise} \end{cases}$$

In JavaScript, we express a case analysis with multiple cases by nesting conditional expressions as alternative expressions inside other conditional expressions:

```
function abs(x) {
    return x > 0
           ? x
           : x === 0
           ? 0
           : - x;
}
```

Parentheses are not needed around the alternative expression `x === 0 ? 0 : - x`, because the conditional-expression syntactic form is right-associative. The interpreter ignores spaces and line breaks, here inserted for readability to align the `?`'s and `:`'s under the first predicate of the case analysis. The general form of a case analysis is

14. Conditionals in full JavaScript accept any value, not just a boolean, as the result of evaluating the *predicate* expression (see footnote 14 in section 4.1.3 for details). The programs in this book use only boolean values as predicates of conditionals.

p_1
`?` e_1
`:` p_2
`?` e_2
⋮
`:` p_n
`?` e_n
`:` *final-alternative-expression*

We call a predicate p_i together with its consequent expression e_i a *clause*. A case analysis can be seen as a sequence of clauses, followed by a final alternative expression. According to the evaluation of conditional expressions, a case analysis is evaluated by first evaluating the predicate p_1. If its value is false, then p_2 is evaluated. If p_2's value is also false, then p_3 is evaluated. This process continues until a predicate is found whose value is true, in which case the interpreter returns the value of the corresponding consequent expression e of the clause as the value of the case analysis. If none of the p's is found to be true, the value of the case analysis is the value of the final alternative expression.

In addition to primitive predicates such as `>=`, `>`, `<`, `<=`, `===`, and `!==` that are applied to numbers,[15] there are logical composition operations, which enable us to construct compound predicates. The three most frequently used are these:

- *expression*$_1$ `&&` *expression*$_2$
  This operation expresses *logical conjunction*, meaning roughly the same as the English word "and." We assume[16] this syntactic form to be syntactic sugar[17] for *expression*$_1$ `?` *expression*$_2$ `: false`.
- *expression*$_1$ `||` *expression*$_2$
  This operation expresses *logical disjunction*, meaning roughly the same as the English word "or." We assume this syntactic form to be syntactic sugar for *expression*$_1$ `?` `true` `:` *expression*$_2$.
- `!` *expression*
  This operation expresses *logical negation*, meaning roughly the same as the English word "not." The value of the expression is true when *expression* evaluates to false, and false when *expression* evaluates to true.

Notice that `&&` and `||` are syntactic forms, not operators; their right-hand expression is not always evaluated. The operator `!`, on the other hand, follows the evaluation rule of section 1.1.3. It is a *unary* operator, which means that it takes only one argument, whereas the arithmetic operators and primitive predicates discussed so far are *binary*, taking two arguments. The operator `!` precedes its argument; we call

15. For now, we restrict these operators to number arguments. In sections 2.3.1 and 3.3.1, we shall generalize the equality and inequality predicates `===` and `!==`.

16. This assumption is justified by the restriction mentioned in footnote 14. Full JavaScript needs to consider the case where the result of evaluating *expression*$_1$ is neither true nor false.

17. Syntactic forms that are simply convenient alternative surface structures for things that can be written in more uniform ways are sometimes called *syntactic sugar*, to use a phrase coined by Peter Landin.

it a *prefix operator*. Another prefix operator is the numeric negation operator, an example of which is the expression `- x` in the `abs` functions above.

As an example of how these predicates are used, the condition that a number x be in the range $5 < x < 10$ may be expressed as

```
x > 5 && x < 10
```

The syntactic form `&&` has lower precedence than the comparison operators `>` and `<`, and the conditional-expression syntactic form $\cdots$`?`$\cdots$`:`$\cdots$ has lower precedence than any other operator we have encountered so far, a property we used in the `abs` functions above.

As another example, we can declare a predicate to test whether one number is greater than or equal to another as

```
function greater_or_equal(x, y) {
    return x > y || x === y;
}
```

or alternatively as

```
function greater_or_equal(x, y) {
    return ! (x < y);
}
```

The function `greater_or_equal`, when applied to two numbers, behaves the same as the operator `>=`. Unary operators have higher precedence than binary operators, which makes the parentheses in this example necessary.

Exercise 1.1

Below is a sequence of statements. What is the result printed by the interpreter in response to each statement? Assume that the sequence is to be evaluated in the order in which it is presented.

```
10;

5 + 3 + 4;

9 - 1;

6 / 2;

2 * 4 + (4 - 6);

const a = 3;

const b = a + 1;

a + b + a * b;

a === b;

b > a && b < a * b ? b : a;
```

```
a === 4
? 6
: b === 4
? 6 + 7 + a
: 25;

2 + (b > a ? b : a);

(a > b
 ? a
 : a < b
 ? b
 : -1)
*
(a + 1);
```

The parentheses around the conditional expressions in the last two statements are necessary because the conditional-expression syntactic form has lower precedence than the arithmetic operators + and ∗.

Exercise 1.2

Translate the following expression into JavaScript

$$\frac{5+4+\left(2-\left(3-(6+\frac{4}{5})\right)\right)}{3(6-2)(2-7)}$$

Exercise 1.3

Declare a function that takes three numbers as arguments and returns the sum of the squares of the two larger numbers.

Exercise 1.4

Observe that our model of evaluation allows for applications whose function expressions are compound expressions. Use this observation to describe the behavior of `a_plus_abs_b`:

```
function plus(a, b) { return a + b; }
function minus(a, b) { return a - b; }
function a_plus_abs_b(a, b) {
    return (b >= 0 ? plus : minus)(a, b);
}
```

Exercise 1.5

Ben Bitdiddle has invented a test to determine whether the interpreter he is faced with is using applicative-order evaluation or normal-order evaluation. He declares the following two functions:

```
function p() { return p(); }

function test(x, y) {
    return x === 0 ? 0 : y;
}
```

Then he evaluates the statement

```
test(0, p());
```

What behavior will Ben observe with an interpreter that uses applicative-order evaluation? What behavior will he observe with an interpreter that uses normal-order evaluation? Explain your answer. (Assume that the evaluation rule for conditional expressions is the same whether the interpreter is using normal or applicative order: The predicate expression is evaluated first, and the result determines whether to evaluate the consequent or the alternative expression.)

1.1.7 Example: Square Roots by Newton's Method

Functions, as introduced above, are much like ordinary mathematical functions. They specify a value that is determined by one or more parameters. But there is an important difference between mathematical functions and computer functions. Computer functions must be effective.

As a case in point, consider the problem of computing square roots. We can define the square-root function as

$$\sqrt{x} = \text{the } y \text{ such that } y \geq 0 \text{ and } y^2 = x$$

This describes a perfectly legitimate mathematical function. We could use it to recognize whether one number is the square root of another, or to derive facts about square roots in general. On the other hand, the definition does not describe a computer function. Indeed, it tells us almost nothing about how to actually find the square root of a given number. It will not help matters to rephrase this definition in pseudo-JavaScript:

```
function sqrt(x) {
    return the y with y >= 0 && square(y) === x;
}
```

This only begs the question.

The contrast between mathematical function and computer function is a reflection of the general distinction between describing properties of things and describing how to do things, or, as it is sometimes referred to, the distinction between declarative knowledge and imperative knowledge. In mathematics we are usually concerned with declarative (what is) descriptions, whereas in computer science we are usually concerned with imperative (how to) descriptions.[18]

---

18. Declarative and imperative descriptions are intimately related, as indeed are mathematics and computer science. For instance, to say that the answer produced by a program is "correct" is to make a declarative statement about the program. There is a large amount of research aimed at establishing techniques for proving that programs are correct, and much of the technical difficulty of this subject has to do with negotiating the transition between imperative statements (from which programs are constructed) and declarative statements (which can be used to deduce things). In a related vein, programming language designers have explored so-called very high-level languages, in which one actually programs in terms of declarative statements. The idea

How does one compute square roots? The most common way is to use Newton's method of successive approximations, which says that whenever we have a guess y for the value of the square root of a number x, we can perform a simple manipulation to get a better guess (one closer to the actual square root) by averaging y with x/y.[19] For example, we can compute the square root of 2 as follows. Suppose our initial guess is 1:

Guess	Quotient	Average
1	$\frac{2}{1} = 2$	$\frac{(2+1)}{2} = 1.5$
1.5	$\frac{2}{1.5} = 1.3333$	$\frac{(1.3333+1.5)}{2} = 1.4167$
1.4167	$\frac{2}{1.4167} = 1.4118$	$\frac{(1.4167+1.4118)}{2} = 1.4142$
1.4142	...	...

Continuing this process, we obtain better and better approximations to the square root.

Now let's formalize the process in terms of functions. We start with a value for the radicand (the number whose square root we are trying to compute) and a value for the guess. If the guess is good enough for our purposes, we are done; if not, we must repeat the process with an improved guess. We write this basic strategy as a function:

```
function sqrt_iter(guess, x) {
    return is_good_enough(guess, x)
           ? guess
           : sqrt_iter(improve(guess, x), x);
}
```

A guess is improved by averaging it with the quotient of the radicand and the old guess:

```
function improve(guess, x) {
    return average(guess, x / guess);
}
```

where

---

is to make interpreters sophisticated enough so that, given "what is" knowledge specified by the programmer, they can generate "how to" knowledge automatically. This cannot be done in general, but there are important areas where progress has been made. We shall revisit this idea in chapter 4.

19. This square-root algorithm is actually a special case of Newton's method, which is a general technique for finding roots of equations. The square-root algorithm itself was developed by Heron of Alexandria in the first century CE. We will see how to express the general Newton's method as a JavaScript function in section 1.3.4.

```
function average(x, y) {
    return (x + y) / 2;
}
```

We also have to say what we mean by "good enough." The following will do for illustration, but it is not really a very good test. (See exercise 1.7.) The idea is to improve the answer until it is close enough so that its square differs from the radicand by less than a predetermined tolerance (here 0.001):[20]

```
function is_good_enough(guess, x) {
    return abs(square(guess) - x) < 0.001;
}
```

Finally, we need a way to get started. For instance, we can always guess that the square root of any number is 1:

```
function sqrt(x) {
    return sqrt_iter(1, x);
}
```

If we type these declarations to the interpreter, we can use `sqrt` just as we can use any function:

```
sqrt(9);
3.00009155413138

sqrt(100 + 37);
11.704699917758145

sqrt(sqrt(2) + sqrt(3));
1.7739279023207892

square(sqrt(1000));
1000.000369924366
```

The `sqrt` program also illustrates that the simple functional language we have introduced so far is sufficient for writing any purely numerical program that one could write in, say, C or Pascal. This might seem surprising, since we have not included in our language any iterative (looping) constructs that direct the computer to do something over and over again. The function `sqrt_iter`, on the other hand, demonstrates how iteration can be accomplished using no special construct other than the ordinary ability to call a function.[21]

Exercise 1.6

Alyssa P. Hacker doesn't like the syntax of conditional expressions, involving the characters `?` and `:`. "Why can't I just declare an ordinary conditional function whose application works

---

20. We will usually give predicates names starting with `is_`, to help us remember that they are predicates.

21. Readers who are worried about the efficiency issues involved in using function calls to implement iteration should note the remarks on "tail recursion" in section 1.2.1.

just like conditional expressions?" she asks.[22] Alyssa's friend Eva Lu Ator claims this can indeed be done, and she declares a `conditional` function as follows:

```
function conditional(predicate, then_clause, else_clause) {
    return predicate ? then_clause : else_clause;
}
```

Eva demonstrates the program for Alyssa:

```
conditional(2 === 3, 0, 5);
5

conditional(1 === 1, 0, 5);
0
```

Delighted, Alyssa uses `conditional` to rewrite the square-root program:

```
function sqrt_iter(guess, x) {
    return conditional(is_good_enough(guess, x),
                       guess,
                       sqrt_iter(improve(guess, x),
                                 x));
}
```

What happens when Alyssa attempts to use this to compute square roots? Explain.

Exercise 1.7

The `is_good_enough` test used in computing square roots will not be very effective for finding the square roots of very small numbers. Also, in real computers, arithmetic operations are almost always performed with limited precision. This makes our test inadequate for very large numbers. Explain these statements, with examples showing how the test fails for small and large numbers. An alternative strategy for implementing `is_good_enough` is to watch how `guess` changes from one iteration to the next and to stop when the change is a very small fraction of the guess. Design a square-root function that uses this kind of end test. Does this work better for small and large numbers?

Exercise 1.8

Newton's method for cube roots is based on the fact that if y is an approximation to the cube root of x, then a better approximation is given by the value

$$\frac{x/y^2 + 2y}{3}$$

Use this formula to implement a cube-root function analogous to the square-root function. (In section 1.3.4 we will see how to implement Newton's method in general as an abstraction of these square-root and cube-root functions.)

---

22. As a Lisp hacker from the original *Structure and Interpretation of Computer Programs*, Alyssa prefers a simpler, more uniform syntax.

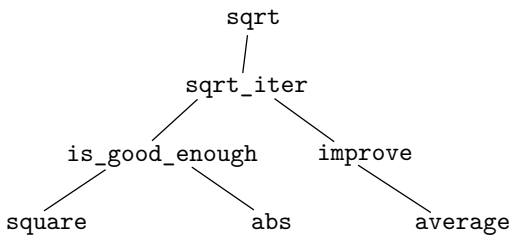


Figure 1.2 Functional decomposition of the `sqrt` program.

1.1.8 Functions as Black-Box Abstractions

The function `sqrt` is our first example of a process defined by a set of mutually defined functions. Notice that the declaration of `sqrt_iter` is *recursive*; that is, the function is defined in terms of itself. The idea of being able to define a function in terms of itself may be disturbing; it may seem unclear how such a "circular" definition could make sense at all, much less specify a well-defined process to be carried out by a computer. This will be addressed more carefully in section 1.2. But first let's consider some other important points illustrated by the `sqrt` example.

Observe that the problem of computing square roots breaks up naturally into a number of subproblems: how to tell whether a guess is good enough, how to improve a guess, and so on. Each of these tasks is accomplished by a separate function. The entire `sqrt` program can be viewed as a cluster of functions (shown in figure 1.2) that mirrors the decomposition of the problem into subproblems.

The importance of this decomposition strategy is not simply that one is dividing the program into parts. After all, we could take any large program and divide it into parts—the first ten lines, the next ten lines, the next ten lines, and so on. Rather, it is crucial that each function accomplishes an identifiable task that can be used as a module in defining other functions. For example, when we define the `is_good_enough` function in terms of `square`, we are able to regard the `square` function as a "black box." We are not at that moment concerned with *how* the function computes its result, only with the fact *that* it computes the square. The details of how the square is computed can be suppressed, to be considered at a later time. Indeed, as far as the `is_good_enough` function is concerned, `square` is not quite a function but rather an abstraction of a function, a so-called *functional abstraction*. At this level of abstraction, any function that computes the square is equally good.

Thus, considering only the values they return, the following two functions squaring a number should be indistinguishable. Each takes a numerical argument and produces the square of that number as the value.[23]

---

23. It is not even clear which of these functions is a more efficient implementation. This depends upon the hardware available. There are machines for which the "obvious" implementation is the less efficient one. Consider a machine that has extensive tables of logarithms and antilogarithms stored in a very efficient manner.

```
function square(x) {
    return x * x;
}

function square(x) {
    return math_exp(double(math_log(x)));
}
function double(x) {
    return x + x;
}
```

So a function should be able to suppress detail. The users of the function may not have written the function themselves, but may have obtained it from another programmer as a black box. A user should not need to know how the function is implemented in order to use it.

Local names

One detail of a function's implementation that should not matter to the user of the function is the implementer's choice of names for the function's parameters. Thus, the following functions should not be distinguishable:

```
function square(x) {
    return x * x;
}

function square(y) {
    return y * y;
}
```

This principle—that the meaning of a function should be independent of the parameter names used by its author—seems on the surface to be self-evident, but its consequences are profound. The simplest consequence is that the parameter names of a function must be local to the body of the function. For example, we used `square` in the declaration of `is_good_enough` in our square-root function:

```
function is_good_enough(guess, x) {
    return abs(square(guess) - x) < 0.001;
}
```

The intention of the author of `is_good_enough` is to determine if the square of the first argument is within a given tolerance of the second argument. We see that the author of `is_good_enough` used the name `guess` to refer to the first argument and `x` to refer to the second argument. The argument of `square` is `guess`. If the author of `square` used `x` (as above) to refer to that argument, we see that the `x` in `is_good_enough` must be a different `x` than the one in `square`. Running the function `square` must not affect the value of `x` that is used by `is_good_enough`, because that value of `x` may be needed by `is_good_enough` after `square` is done computing.

If the parameters were not local to the bodies of their respective functions, then the parameter `x` in `square` could be confused with the parameter `x` in `is_good_enough`, and the behavior of `is_good_enough` would depend upon which version of `square` we used. Thus, `square` would not be the black box we desired.

A parameter of a function has a very special role in the function declaration, in that it doesn't matter what name the parameter has. Such a name is called *bound*, and we say that the function declaration *binds* its parameters. The meaning of a function declaration is unchanged if a bound name is consistently renamed throughout the declaration.[24] If a name is not bound, we say that it is *free*. The set of statements for which a binding declares a name is called the *scope* of that name. In a function declaration, the bound names declared as the parameters of the function have the body of the function as their scope.

In the declaration of `is_good_enough` above, `guess` and `x` are bound names but `abs` and `square` are free. The meaning of `is_good_enough` should be independent of the names we choose for `guess` and `x` so long as they are distinct and different from `abs` and `square`. (If we renamed `guess` to `abs` we would have introduced a bug by *capturing* the name `abs`. It would have changed from free to bound.) The meaning of `is_good_enough` is not independent of the choice of its free names, however. It surely depends upon the fact (external to this declaration) that the name `abs` refers to a function for computing the absolute value of a number. The function `is_good_enough` will compute a different function if we substitute `math_cos` (the primitive cosine function) for `abs` in its declaration.

Internal declarations and block structure

We have one kind of name isolation available to us so far: The parameters of a function are local to the body of the function. The square-root program illustrates another way in which we would like to control the use of names. The existing program consists of separate functions:

```
function sqrt(x) {
    return sqrt_iter(1, x);
}
function sqrt_iter(guess, x) {
    return is_good_enough(guess, x)
           ? guess
           : sqrt_iter(improve(guess, x), x);
}
function is_good_enough(guess, x) {
    return abs(square(guess) - x) < 0.001;
}
function improve(guess, x) {
    return average(guess, x / guess);
}
```

The problem with this program is that the only function that is important to users of `sqrt` is `sqrt`. The other functions (`sqrt_iter`, `is_good_enough`, and

24. The concept of consistent renaming is actually subtle and difficult to define formally. Famous logicians have made embarrassing errors here.

improve) only clutter up their minds. They may not declare any other function called is_good_enough as part of another program to work together with the square-root program, because sqrt needs it. The problem is especially severe in the construction of large systems by many separate programmers. For example, in the construction of a large library of numerical functions, many numerical functions are computed as successive approximations and thus might have functions named is_good_enough and improve as auxiliary functions. We would like to localize the subfunctions, hiding them inside sqrt so that sqrt could coexist with other successive approximations, each having its own private is_good_enough function. To make this possible, we allow a function to have internal declarations that are local to that function. For example, in the square-root problem we can write

```
function sqrt(x) {
    function is_good_enough(guess, x) {
        return abs(square(guess) - x) < 0.001;
    }
    function improve(guess, x) {
        return average(guess, x / guess);
    }
    function sqrt_iter(guess, x) {
        return is_good_enough(guess, x)
               ? guess
               : sqrt_iter(improve(guess, x), x);
    }
    return sqrt_iter(1, x);
}
```

Any matching pair of braces designates a *block*, and declarations inside the block are local to the block. Such nesting of declarations, called *block structure*, is basically the right solution to the simplest name-packaging problem. But there is a better idea lurking here. In addition to internalizing the declarations of the auxiliary functions, we can simplify them. Since x is bound in the declaration of sqrt, the functions is_good_enough, improve, and sqrt_iter, which are declared internally to sqrt, are in the scope of x. Thus, it is not necessary to pass x explicitly to each of these functions. Instead, we allow x to be a free name in the internal declarations, as shown below. Then x gets its value from the argument with which the enclosing function sqrt is called. This discipline is called *lexical scoping*.[25]

25. Lexical scoping dictates that free names in a function are taken to refer to bindings made by enclosing function declarations; that is, they are looked up in the environment in which the function was declared. We will see how this works in detail in chapter 3 when we study environments and the detailed behavior of the interpreter.

```
function sqrt(x) {
    function is_good_enough(guess) {
        return abs(square(guess) - x) < 0.001;
    }
    function improve(guess) {
        return average(guess, x / guess);
    }
    function sqrt_iter(guess) {
        return is_good_enough(guess)
               ? guess
               : sqrt_iter(improve(guess));
    }
    return sqrt_iter(1);
}
```

We will use block structure extensively to help us break up large programs into tractable pieces.[26] The idea of block structure originated with the programming language Algol 60. It appears in most advanced programming languages and is an important tool for helping to organize the construction of large programs.

1.2 Functions and the Processes They Generate

We have now considered the elements of programming: We have used primitive arithmetic operations, we have combined these operations, and we have abstracted these composite operations by declaring them as compound functions. But that is not enough to enable us to say that we know how to program. Our situation is analogous to that of someone who has learned the rules for how the pieces move in chess but knows nothing of typical openings, tactics, or strategy. Like the novice chess player, we don't yet know the common patterns of usage in the domain. We lack the knowledge of which moves are worth making (which functions are worth declaring). We lack the experience to predict the consequences of making a move (executing a function).

The ability to visualize the consequences of the actions under consideration is crucial to becoming an expert programmer, just as it is in any synthetic, creative activity. In becoming an expert photographer, for example, one must learn how to look at a scene and know how dark each region will appear on a print for each possible choice of exposure and processing options. Only then can one reason backward, planning framing, lighting, exposure, and processing to obtain the desired effects. So it is with programming, where we are planning the course of action to be taken by a process and where we control the process by means of a program. To become experts, we must learn to visualize the processes generated by various types of functions. Only after we have developed such a skill can we learn to reliably construct programs that exhibit the desired behavior.

A function is a pattern for the *local evolution* of a computational process. It specifies how each stage of the process is built upon the previous stage. We would

---

26. Embedded declarations must come first in a function body. The management is not responsible for the consequences of running programs that intertwine declaration and use; see also footnotes 55 and 57 in section 1.3.2.

like to be able to make statements about the overall, or *global*, behavior of a process whose local evolution has been specified by a function. This is very difficult to do in general, but we can at least try to describe some typical patterns of process evolution.

In this section we will examine some common "shapes" for processes generated by simple functions. We will also investigate the rates at which these processes consume the important computational resources of time and space. The functions we will consider are very simple. Their role is like that played by test patterns in photography: as oversimplified prototypical patterns, rather than practical examples in their own right.

1.2.1 Linear Recursion and Iteration

We begin by considering the factorial function, defined by

$$n! \quad = \quad n \cdot (n-1) \cdot (n-2) \cdots 3 \cdot 2 \cdot 1$$

There are many ways to compute factorials. One way is to make use of the observation that $n!$ is equal to n times $(n-1)!$ for any positive integer n:

$$n! \quad = \quad n \cdot [(n-1) \cdot (n-2) \cdots 3 \cdot 2 \cdot 1] \quad = \quad n \cdot (n-1)!$$

Thus, we can compute $n!$ by computing $(n-1)!$ and multiplying the result by n. If we add the stipulation that $1!$ is equal to 1, this observation translates directly into a computer function:

```
function factorial(n) {
    return n === 1
           ? 1
           : n * factorial(n - 1);
}
```

We can use the substitution model of section 1.1.5 to watch this function in action computing 6!, as shown in figure 1.3.

Now let's take a different perspective on computing factorials. We could describe a rule for computing $n!$ by specifying that we first multiply 1 by 2, then multiply the result by 3, then by 4, and so on until we reach n. More formally, we maintain a running product, together with a counter that counts from 1 up to n. We can describe the computation by saying that the counter and the product simultaneously change from one step to the next according to the rule

$$\begin{aligned} \text{product} &\leftarrow \text{counter} \cdot \text{product} \\ \text{counter} &\leftarrow \text{counter} + 1 \end{aligned}$$

and stipulating that $n!$ is the value of the product when the counter exceeds n.

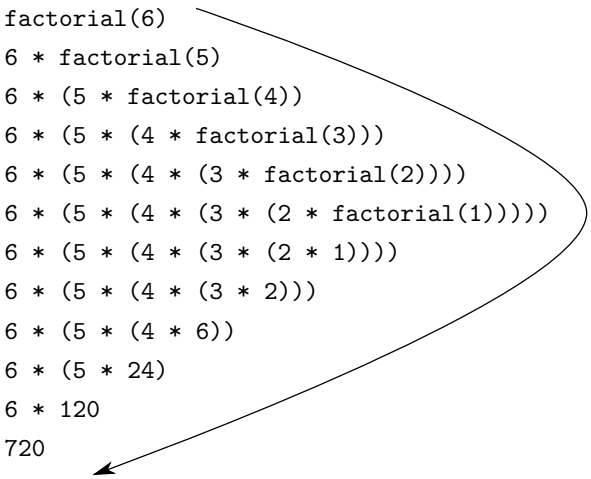


Figure 1.3 A linear recursive process for computing 6!.

Once again, we can recast our description as a function for computing factorials:[27]

```
function factorial(n) {
    return fact_iter(1, 1, n);
}
function fact_iter(product, counter, max_count) {
    return counter > max_count
           ? product
           : fact_iter(counter * product,
                       counter + 1,
                       max_count);
}
```

As before, we can use the substitution model to visualize the process of computing 6!, as shown in figure 1.4.

Compare the two processes. From one point of view, they seem hardly different at all. Both compute the same mathematical function on the same domain, and each requires a number of steps proportional to n to compute n!. Indeed, both processes even carry out the same sequence of multiplications, obtaining the same sequence of partial products. On the other hand, when we consider the "shapes" of the two processes, we find that they evolve quite differently.

---

27. In a real program we would probably use the block structure introduced in the last section to hide the declaration of `fact_iter`:

```
function factorial(n) {
    function iter(product, counter) {
        return counter > n
               ? product
               : iter(counter * product,
                      counter + 1);
    }
    return iter(1, 1);
}
```

We avoided doing this here so as to minimize the number of things to think about at once.

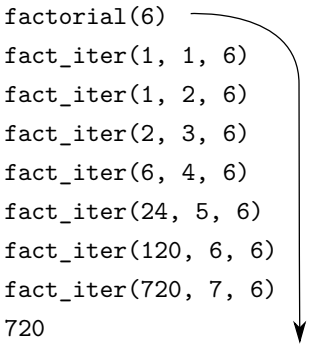


Figure 1.4 A linear iterative process for computing 6!.

Consider the first process. The substitution model reveals a shape of expansion followed by contraction, indicated by the arrow in figure 1.3. The expansion occurs as the process builds up a chain of *deferred operations* (in this case, a chain of multiplications). The contraction occurs as the operations are actually performed. This type of process, characterized by a chain of deferred operations, is called a *recursive process*. Carrying out this process requires that the interpreter keep track of the operations to be performed later on. In the computation of $n!$, the length of the chain of deferred multiplications, and hence the amount of information needed to keep track of it, grows linearly with n (is proportional to n), just like the number of steps. Such a process is called a *linear recursive process*.

By contrast, the second process does not grow and shrink. At each step, all we need to keep track of, for any n, are the current values of the names `product`, `counter`, and `max_count`. We call this an *iterative process*. In general, an iterative process is one whose state can be summarized by a fixed number of *state variables*, together with a fixed rule that describes how the state variables should be updated as the process moves from state to state and an (optional) end test that specifies conditions under which the process should terminate. In computing $n!$, the number of steps required grows linearly with n. Such a process is called a *linear iterative process*.

The contrast between the two processes can be seen in another way. In the iterative case, the state variables provide a complete description of the state of the process at any point. If we stopped the computation between steps, all we would need to do to resume the computation is to supply the interpreter with the values of the three state variables. Not so with the recursive process. In this case there is some additional "hidden" information, maintained by the interpreter and not contained in the state variables, which indicates "where the process is" in negotiating the chain of deferred operations. The longer the chain, the more information must be maintained.[28]

In contrasting iteration and recursion, we must be careful not to confuse the notion of a recursive *process* with the notion of a recursive *function*. When we describe a function as recursive, we are referring to the syntactic fact that the function

28. When we discuss the implementation of functions on register machines in chapter 5, we will see that any iterative process can be realized "in hardware" as a machine that has a fixed set of registers and no auxiliary memory. In contrast, realizing a recursive process requires a machine that uses an auxiliary data structure known as a *stack*.

declaration refers (either directly or indirectly) to the function itself. But when we describe a process as following a pattern that is, say, linearly recursive, we are speaking about how the process evolves, not about the syntax of how a function is written. It may seem disturbing that we refer to a recursive function such as `fact_iter` as generating an iterative process. However, the process really is iterative: Its state is captured completely by its three state variables, and an interpreter need keep track of only three names in order to execute the process.

One reason that the distinction between process and function may be confusing is that most implementations of common languages (including C, Java, and Python) are designed in such a way that the interpretation of any recursive function consumes an amount of memory that grows with the number of function calls, even when the process described is, in principle, iterative. As a consequence, these languages can describe iterative processes only by resorting to special-purpose "looping constructs" such as `do`, `repeat`, `until`, `for`, and `while`. The implementation of JavaScript we shall consider in chapter 5 does not share this defect. It will execute an iterative process in constant space, even if the iterative process is described by a recursive function. An implementation with this property is called *tail-recursive*.[29] With a tail-recursive implementation, iteration can be expressed using the ordinary function call mechanism, so that special iteration constructs are useful only as syntactic sugar.[30]

Exercise 1.9

Each of the following two functions defines a method for adding two positive integers in terms of the functions `inc`, which increments its argument by 1, and `dec`, which decrements its argument by 1.

---

29. Tail recursion has long been known as a compiler optimization trick. A coherent semantic basis for tail recursion was provided by Carl Hewitt (1977), who explained it in terms of the "message-passing" model of computation that we shall discuss in chapter 3. Inspired by this, Gerald Jay Sussman and Guy Lewis Steele Jr. (see Steele 1975) constructed a tail-recursive interpreter for Scheme. Steele later showed how tail recursion is a consequence of the natural way to compile function calls (Steele 1977). The IEEE standard for Scheme requires that Scheme implementations be tail-recursive. The ECMA standard for JavaScript eventually followed suit with ECMAScript 2015 (ECMA 2015). Note, however, that as of this writing (2021), most implementations of JavaScript do not comply with this standard with respect to tail recursion.

30. Exercise 4.7 explores JavaScript's while loops as syntactic sugar for functions that give rise to iterative processes. The full language JavaScript, like other conventional languages, features a plethora of syntactic forms, all of which can be expressed more uniformly in the language Lisp. This, together with the fact that these constructs typically involve semicolons whose placement rules are sometimes not obvious, led Alan Perlis to quip: "Syntactic sugar causes cancer of the semicolon."

```
function plus(a, b) {
    return a === 0 ? b : inc(plus(dec(a), b));
}

function plus(a, b) {
    return a === 0 ? b : plus(dec(a), inc(b));
}
```

Using the substitution model, illustrate the process generated by each function in evaluating `plus(4, 5);`. Are these processes iterative or recursive?

Exercise 1.10

The following function computes a mathematical function called Ackermann's function.

```
function A(x, y) {
    return y === 0
           ? 0
           : x === 0
           ? 2 * y
           : y === 1
           ? 2
           : A(x - 1, A(x, y - 1));
}
```

What are the values of the following statements?

```
A(1, 10);

A(2, 4);

A(3, 3);
```

Consider the following functions, where `A` is the function declared above:

```
function f(n) {
    return A(0, n);
}
function g(n) {
    return A(1, n);
}
function h(n) {
    return A(2, n);
}
function k(n) {
   return 5 * n * n;
}
```

Give concise mathematical definitions for the functions computed by the functions `f`, `g`, and `h` for positive integer values of n. For example, $k(n)$ computes $5n^2$.

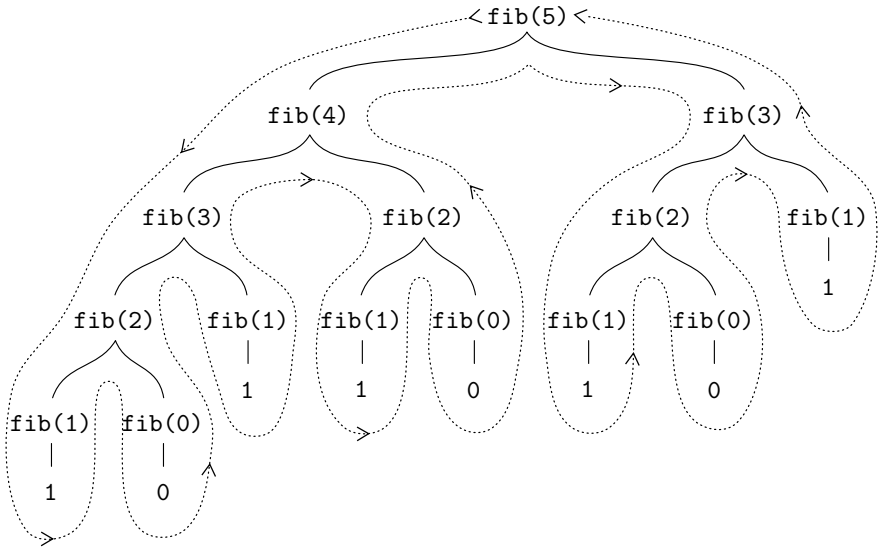


Figure 1.5 The tree-recursive process generated in computing `fib(5)`.

1.2.2 Tree Recursion

Another common pattern of computation is called *tree recursion*. As an example, consider computing the sequence of Fibonacci numbers, in which each number is the sum of the preceding two:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, \ldots$$

In general, the Fibonacci numbers can be defined by the rule

$$\text{Fib}(n) \quad = \quad \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ \text{Fib}(n-1) + \text{Fib}(n-2) & \text{otherwise} \end{cases}$$

We can immediately translate this definition into a recursive function for computing Fibonacci numbers:

```
function fib(n) {
    return n === 0
           ? 0
           : n === 1
           ? 1
           : fib(n - 1) + fib(n - 2);
}
```

Consider the pattern of this computation. To compute `fib(5)`, we compute `fib(4)` and `fib(3)`. To compute `fib(4)`, we compute `fib(3)` and `fib(2)`. In general, the evolved process looks like a tree, as shown in figure 1.5. Notice that the branches split into two at each level (except at the bottom); this reflects the fact that the `fib` function calls itself twice each time it is invoked.

This function is instructive as a prototypical tree recursion, but it is a terrible way to compute Fibonacci numbers because it does so much redundant computation. Notice in figure 1.5 that the entire computation of `fib(3)`—almost half the work—is duplicated. In fact, it is not hard to show that the number of times the function will compute `fib(1)` or `fib(0)` (the number of leaves in the above tree, in general) is precisely $\text{Fib}(n+1)$. To get an idea of how bad this is, one can show that the value of $\text{Fib}(n)$ grows exponentially with n. More precisely (see exercise 1.13), $\text{Fib}(n)$ is the closest integer to $\phi^n/\sqrt{5}$, where

$$\phi \quad = \quad (1+\sqrt{5})/2 \quad \approx \quad 1.6180$$

is the *golden ratio*, which satisfies the equation

$$\phi^2 \quad = \quad \phi + 1$$

Thus, the process uses a number of steps that grows exponentially with the input. On the other hand, the space required grows only linearly with the input, because we need keep track only of which nodes are above us in the tree at any point in the computation. In general, the number of steps required by a tree-recursive process will be proportional to the number of nodes in the tree, while the space required will be proportional to the maximum depth of the tree.

We can also formulate an iterative process for computing the Fibonacci numbers. The idea is to use a pair of integers a and b, initialized to $\text{Fib}(1) = 1$ and $\text{Fib}(0) = 0$, and to repeatedly apply the simultaneous transformations

$$\begin{aligned} a &\leftarrow a + b \\ b &\leftarrow a \end{aligned}$$

It is not hard to show that, after applying this transformation n times, a and b will be equal, respectively, to $\text{Fib}(n+1)$ and $\text{Fib}(n)$. Thus, we can compute Fibonacci numbers iteratively using the function

```
function fib(n) {
    return fib_iter(1, 0, n);
}
function fib_iter(a, b, count) {
    return count === 0
           ? b
           : fib_iter(a + b, a, count - 1);
}
```

This second method for computing $\text{Fib}(n)$ is a linear iteration. The difference in number of steps required by the two methods—one linear in n, one growing as fast as $\text{Fib}(n)$ itself—is enormous, even for small inputs.

One should not conclude from this that tree-recursive processes are useless. When we consider processes that operate on hierarchically structured data rather than numbers, we will find that tree recursion is a natural and powerful tool.[31] But

31. An example of this was hinted at in section 1.1.3: The interpreter itself evaluates expressions using a tree-recursive process.

even in numerical operations, tree-recursive processes can be useful in helping us to understand and design programs. For instance, although the first `fib` function is much less efficient than the second one, it is more straightforward, being little more than a translation into JavaScript of the definition of the Fibonacci sequence. To formulate the iterative algorithm required noticing that the computation could be recast as an iteration with three state variables.

Example: Counting change

It takes only a bit of cleverness to come up with the iterative Fibonacci algorithm. In contrast, consider the following problem: How many different ways can we make change of $1.00 (100 cents), given half-dollars, quarters, dimes, nickels, and pennies (50 cents, 25 cents, 10 cents, 5 cents, and 1 cent, respectively)? More generally, can we write a function to compute the number of ways to change any given amount of money?

This problem has a simple solution as a recursive function. Suppose we think of the types of coins available as arranged in some order. Then the following relation holds:

The number of ways to change amount a using n kinds of coins equals

- the number of ways to change amount a using all but the first kind of coin, plus
- the number of ways to change amount $a - d$ using all n kinds of coins, where d is the denomination of the first kind of coin.

To see why this is true, observe that the ways to make change can be divided into two groups: those that do not use any of the first kind of coin, and those that do. Therefore, the total number of ways to make change for some amount is equal to the number of ways to make change for the amount without using any of the first kind of coin, plus the number of ways to make change assuming that we do use the first kind of coin. But the latter number is equal to the number of ways to make change for the amount that remains after using a coin of the first kind.

Thus, we can recursively reduce the problem of changing a given amount to problems of changing smaller amounts or using fewer kinds of coins. Consider this reduction rule carefully, and convince yourself that we can use it to describe an algorithm if we specify the following degenerate cases:[32]

- If a is exactly 0, we should count that as 1 way to make change.
- If a is less than 0, we should count that as 0 ways to make change.
- If n is 0, we should count that as 0 ways to make change.

We can easily translate this description into a recursive function:

32. For example, work through in detail how the reduction rule applies to the problem of making change for 10 cents using pennies and nickels.

```
function count_change(amount) {
    return cc(amount, 5);
}
function cc(amount, kinds_of_coins) {
    return amount === 0
           ? 1
           : amount < 0 || kinds_of_coins === 0
           ? 0
           : cc(amount, kinds_of_coins - 1)
             +
             cc(amount - first_denomination(kinds_of_coins),
                kinds_of_coins);
}
function first_denomination(kinds_of_coins) {
    return kinds_of_coins === 1 ? 1
         : kinds_of_coins === 2 ? 5
         : kinds_of_coins === 3 ? 10
         : kinds_of_coins === 4 ? 25
         : kinds_of_coins === 5 ? 50
         : 0;
}
```

(The `first_denomination` function takes as input the number of kinds of coins available and returns the denomination of the first kind. Here we are thinking of the coins as arranged in order from largest to smallest, but any order would do as well.) We can now answer our original question about changing a dollar:

```
count_change(100);
```
292

The function `count_change` generates a tree-recursive process with redundancies similar to those in our first implementation of `fib`. On the other hand, it is not obvious how to design a better algorithm for computing the result, and we leave this problem as a challenge. The observation that a tree-recursive process may be highly inefficient but often easy to specify and understand has led people to propose that one could get the best of both worlds by designing a "smart compiler" that could transform tree-recursive functions into more efficient functions that compute the same result.[33]

Exercise 1.11

A function f is defined by the rules $f(n) = n$ if $n < 3$ and $f(n) = f(n-1) + 2f(n-2) + 3f(n-3)$ if $n \geq 3$. Write a JavaScript function that computes f by means of a recursive process. Write a function that computes f by means of an iterative process.

---

33. One approach to coping with redundant computations is to arrange matters so that we automatically construct a table of values as they are computed. Each time we are asked to apply the function to some argument, we first look to see if the value is already stored in the table, in which case we avoid performing the redundant computation. This strategy, known as *tabulation* or *memoization*, can be implemented in a straightforward way. Tabulation can sometimes be used to transform processes that require an exponential number of steps (such as `count_change`) into processes whose space and time requirements grow linearly with the input. See exercise 3.27.

Exercise 1.12

The following pattern of numbers is called *Pascal's triangle*.

```
                1
              1   1
            1   2   1
          1   3   3   1
        1   4   6   4   1
              . . .
```

The numbers at the edge of the triangle are all 1, and each number inside the triangle is the sum of the two numbers above it.[34] Write a function that computes elements of Pascal's triangle by means of a recursive process.

Exercise 1.13

Prove that $\text{Fib}(n)$ is the closest integer to $\phi^n/\sqrt{5}$, where $\phi = (1+\sqrt{5})/2$. Hint: Use induction and the definition of the Fibonacci numbers to prove that $\text{Fib}(n) = (\phi^n - \psi^n)/\sqrt{5}$, where $\psi = (1-\sqrt{5})/2$.

1.2.3 Orders of Growth

The previous examples illustrate that processes can differ considerably in the rates at which they consume computational resources. One convenient way to describe this difference is to use the notion of *order of growth* to obtain a gross measure of the resources required by a process as the inputs become larger.

Let n be a parameter that measures the size of the problem, and let $R(n)$ be the amount of resources the process requires for a problem of size n. In our previous examples we took n to be the number for which a given function is to be computed, but there are other possibilities. For instance, if our goal is to compute an approximation to the square root of a number, we might take n to be the number of digits accuracy required. For matrix multiplication we might take n to be the number of rows in the matrices. In general there are a number of properties of the problem with respect to which it will be desirable to analyze a given process. Similarly, $R(n)$ might measure the number of internal storage registers used, the number of elementary machine operations performed, and so on. In computers that do only a fixed number of operations at a time, the time required will be proportional to the number of elementary machine operations performed.

We say that $R(n)$ has order of growth $\Theta(f(n))$, written $R(n) = \Theta(f(n))$ (pronounced "theta of $f(n)$"), if there are positive constants k_1 and k_2 independent of n such that

$$k_1 f(n) \quad \leq \quad R(n) \quad \leq \quad k_2 f(n)$$

34. The elements of Pascal's triangle are called the *binomial coefficients*, because the nth row consists of the coefficients of the terms in the expansion of $(x+y)^n$. This pattern for computing the coefficients appeared in Blaise Pascal's 1653 seminal work on probability theory, *Traité du triangle arithmétique*. According to Edwards (2019), the same pattern appears in the works of the eleventh-century Persian mathematician Al-Karaji, in the works of the twelfth-century Hindu mathematician Bhaskara, and in the works of the thirteenth-century Chinese mathematician Yang Hui.

for any sufficiently large value of n. (In other words, for large n, the value $R(n)$ is sandwiched between $k_1 f(n)$ and $k_2 f(n)$.)

For instance, with the linear recursive process for computing factorial described in section 1.2.1 the number of steps grows proportionally to the input n. Thus, the steps required for this process grows as $\Theta(n)$. We also saw that the space required grows as $\Theta(n)$. For the iterative factorial, the number of steps is still $\Theta(n)$ but the space is $\Theta(1)$—that is, constant.[35] The tree-recursive Fibonacci computation requires $\Theta(\phi^n)$ steps and space $\Theta(n)$, where ϕ is the golden ratio described in section 1.2.2.

Orders of growth provide only a crude description of the behavior of a process. For example, a process requiring n^2 steps and a process requiring $1000n^2$ steps and a process requiring $3n^2 + 10n + 17$ steps all have $\Theta(n^2)$ order of growth. On the other hand, order of growth provides a useful indication of how we may expect the behavior of the process to change as we change the size of the problem. For a $\Theta(n)$ (linear) process, doubling the size will roughly double the amount of resources used. For an exponential process, each increment in problem size will multiply the resource utilization by a constant factor. In the remainder of section 1.2 we will examine two algorithms whose order of growth is logarithmic, so that doubling the problem size increases the resource requirement by a constant amount.

Exercise 1.14

Draw the tree illustrating the process generated by the `count_change` function of section 1.2.2 in making change for 11 cents. What are the orders of growth of the space and number of steps used by this process as the amount to be changed increases?

Exercise 1.15

The sine of an angle (specified in radians) can be computed by making use of the approximation $\sin x \approx x$ if x is sufficiently small, and the trigonometric identity

$$\sin x = 3 \sin \frac{x}{3} - 4 \sin^3 \frac{x}{3}$$

to reduce the size of the argument of sin. (For purposes of this exercise an angle is considered "sufficiently small" if its magnitude is not greater than 0.1 radians.) These ideas are incorporated in the following functions:

---

35. These statements mask a great deal of oversimplification. For instance, if we count process steps as "machine operations" we are making the assumption that the number of machine operations needed to perform, say, a multiplication is independent of the size of the numbers to be multiplied, which is false if the numbers are sufficiently large. Similar remarks hold for the estimates of space. Like the design and description of a process, the analysis of a process can be carried out at various levels of abstraction.

```
function cube(x) {
    return x * x * x;
}
function p(x) {
    return 3 * x - 4 * cube(x);
}
function sine(angle) {
    return ! (abs(angle) > 0.1)
           ? angle
           : p(sine(angle / 3));
}
```

a. How many times is the function `p` applied when `sine(12.15)` is evaluated?

b. What is the order of growth in space and number of steps (as a function of a) used by the process generated by the `sine` function when `sine(a)` is evaluated?

1.2.4 Exponentiation

Consider the problem of computing the exponential of a given number. We would like a function that takes as arguments a base b and a positive integer exponent n and computes b^n. One way to do this is via the recursive definition

$$\begin{aligned} b^n &= b \cdot b^{n-1} \\ b^0 &= 1 \end{aligned}$$

which translates readily into the function

```
function expt(b, n) {
    return n === 0
           ? 1
           : b * expt(b, n - 1);
}
```

This is a linear recursive process, which requires $\Theta(n)$ steps and $\Theta(n)$ space. Just as with factorial, we can readily formulate an equivalent linear iteration:

```
function expt(b, n) {
    return expt_iter(b, n, 1);
}
function expt_iter(b, counter, product) {
    return counter === 0
           ? product
           : expt_iter(b, counter - 1, b * product);
}
```

This version requires $\Theta(n)$ steps and $\Theta(1)$ space.

We can compute exponentials in fewer steps by using successive squaring. For instance, rather than computing b^8 as

$$b \cdot (b \cdot (b \cdot (b \cdot (b \cdot (b \cdot (b \cdot b))))))$$

we can compute it using three multiplications:

$$\begin{aligned} b^2 &= b \cdot b \\ b^4 &= b^2 \cdot b^2 \\ b^8 &= b^4 \cdot b^4 \end{aligned}$$

This method works fine for exponents that are powers of 2. We can also take advantage of successive squaring in computing exponentials in general if we use the rule

$$\begin{aligned} b^n &= (b^{n/2})^2 && \text{if } n \text{ is even} \\ b^n &= b \cdot b^{n-1} && \text{if } n \text{ is odd} \end{aligned}$$

We can express this method as a function:

```
function fast_expt(b, n) {
    return n === 0
           ? 1
           : is_even(n)
           ? square(fast_expt(b, n / 2))
           : b * fast_expt(b, n - 1);
}
```

where the predicate to test whether an integer is even is defined in terms of the operator %, which computes the remainder after integer division, by

```
function is_even(n) {
    return n % 2 === 0;
}
```

The process evolved by `fast_expt` grows logarithmically with n in both space and number of steps. To see this, observe that computing b^{2n} using `fast_expt` requires only one more multiplication than computing b^n. The size of the exponent we can compute therefore doubles (approximately) with every new multiplication we are allowed. Thus, the number of multiplications required for an exponent of n grows about as fast as the logarithm of n to the base 2. The process has $\Theta(\log n)$ growth.[36]

The difference between $\Theta(\log n)$ growth and $\Theta(n)$ growth becomes striking as n becomes large. For example, `fast_expt` for $n = 1000$ requires only 14 multiplications.[37] It is also possible to use the idea of successive squaring to devise an iterative algorithm that computes exponentials with a logarithmic number of steps

36. More precisely, the number of multiplications required is equal to 1 less than the log base 2 of n, plus the number of ones in the binary representation of n. This total is always less than twice the log base 2 of n. The arbitrary constants k_1 and k_2 in the definition of order notation imply that, for a logarithmic process, the base to which logarithms are taken does not matter, so all such processes are described as $\Theta(\log n)$.

37. You may wonder why anyone would care about raising numbers to the 1000th power. See section 1.2.6.

(see exercise 1.16), although, as is often the case with iterative algorithms, this is not written down so straightforwardly as the recursive algorithm.[38]

Exercise 1.16

Design a function that evolves an iterative exponentiation process that uses successive squaring and uses a logarithmic number of steps, as does `fast_expt`. (Hint: Using the observation that $(b^{n/2})^2 = (b^2)^{n/2}$, keep, along with the exponent n and the base b, an additional state variable a, and define the state transformation in such a way that the product ab^n is unchanged from state to state. At the beginning of the process a is taken to be 1, and the answer is given by the value of a at the end of the process. In general, the technique of defining an *invariant quantity* that remains unchanged from state to state is a powerful way to think about the design of iterative algorithms.)

Exercise 1.17

The exponentiation algorithms in this section are based on performing exponentiation by means of repeated multiplication. In a similar way, one can perform integer multiplication by means of repeated addition. The following multiplication function (in which it is assumed that our language can only add, not multiply) is analogous to the `expt` function:

```
function times(a, b) {
    return b === 0
           ? 0
           : a + times(a, b - 1);
}
```

This algorithm takes a number of steps that is linear in `b`. Now suppose we include, together with addition, the functions `double`, which doubles an integer, and `halve`, which divides an (even) integer by 2. Using these, design a multiplication function analogous to `fast_expt` that uses a logarithmic number of steps.

Exercise 1.18

Using the results of exercises 1.16 and 1.17, devise a function that generates an iterative process for multiplying two integers in terms of adding, doubling, and halving and uses a logarithmic number of steps.[39]

Exercise 1.19

There is a clever algorithm for computing the Fibonacci numbers in a logarithmic number of steps. Recall the transformation of the state variables a and b in the `fib_iter` process of section 1.2.2: $a \leftarrow a + b$ and $b \leftarrow a$. Call this transformation T, and observe that applying T over and over again n times, starting with 1 and 0, produces the pair $\mathrm{Fib}(n+1)$ and $\mathrm{Fib}(n)$.

---

38. This iterative algorithm is ancient. It appears in the *Chandah-sutra* by Áchárya, written before 200 BCE. See Knuth 1997b, section 4.6.3, for a full discussion and analysis of this and other methods of exponentiation.

39. This algorithm, which is sometimes known as the "Russian peasant method" of multiplication, is ancient. Examples of its use are found in the Rhind Papyrus, one of the two oldest mathematical documents in existence, written about 1700 BCE (and copied from an even older document) by an Egyptian scribe named A'h-mose.

In other words, the Fibonacci numbers are produced by applying T^n, the nth power of the transformation T, starting with the pair $(1, 0)$. Now consider T to be the special case of $p = 0$ and $q = 1$ in a family of transformations T_{pq}, where T_{pq} transforms the pair (a, b) according to $a \leftarrow bq + aq + ap$ and $b \leftarrow bp + aq$. Show that if we apply such a transformation T_{pq} twice, the effect is the same as using a single transformation $T_{p'q'}$ of the same form, and compute p' and q' in terms of p and q. This gives us an explicit way to square these transformations, and thus we can compute T^n using successive squaring, as in the `fast_expt` function. Put this all together to complete the following function, which runs in a logarithmic number of steps:[40]

```
function fib(n) {
    return fib_iter(1, 0, 0, 1, n);
}
function fib_iter(a, b, p, q, count) {
    return count === 0
           ? b
           : is_even(count)
           ? fib_iter(a,
                      b,
                      ⟨??⟩,          // compute p'
                      ⟨??⟩,          // compute q'
                      count / 2)
           : fib_iter(b * q + a * q + a * p,
                      b * p + a * q,
                      p,
                      q,
                      count - 1);
}
```

1.2.5 Greatest Common Divisors

The greatest common divisor (GCD) of two integers a and b is defined to be the largest integer that divides both a and b with no remainder. For example, the GCD of 16 and 28 is 4. In chapter 2, when we investigate how to implement rational-number arithmetic, we will need to be able to compute GCDs in order to reduce rational numbers to lowest terms. (To reduce a rational number to lowest terms, we must divide both the numerator and the denominator by their GCD. For example, 16/28 reduces to 4/7.) One way to find the GCD of two integers is to factor them and search for common factors, but there is a famous algorithm that is much more efficient.

The idea of the algorithm is based on the observation that, if r is the remainder when a is divided by b, then the common divisors of a and b are precisely the same as the common divisors of b and r. Thus, we can use the equation

$$\mathrm{GCD}(a, b) \quad = \quad \mathrm{GCD}(b, r)$$

to successively reduce the problem of computing a GCD to the problem of computing the GCD of smaller and smaller pairs of integers. For example,

40. This exercise was suggested by Joe Stoy, based on an example in Kaldewaij 1990.

$$\begin{aligned}\mathrm{GCD}(206, 40) &= \mathrm{GCD}(40, 6)\\ &= \mathrm{GCD}(6, 4)\\ &= \mathrm{GCD}(4, 2)\\ &= \mathrm{GCD}(2, 0)\\ &= 2\end{aligned}$$

reduces GCD(206, 40) to GCD(2, 0), which is 2. It is possible to show that starting with any two positive integers and performing repeated reductions will always eventually produce a pair where the second number is 0. Then the GCD is the other number in the pair. This method for computing the GCD is known as *Euclid's Algorithm*.[41]

It is easy to express Euclid's Algorithm as a function:

```
function gcd(a, b) {
    return b === 0 ? a : gcd(b, a % b);
}
```

This generates an iterative process, whose number of steps grows as the logarithm of the numbers involved.

The fact that the number of steps required by Euclid's Algorithm has logarithmic growth bears an interesting relation to the Fibonacci numbers:

> **Lamé's Theorem:** If Euclid's Algorithm requires k steps to compute the GCD of some pair, then the smaller number in the pair must be greater than or equal to the kth Fibonacci number.[42]

We can use this theorem to get an order-of-growth estimate for Euclid's Algorithm. Let n be the smaller of the two inputs to the function. If the process takes k steps, then we must have $n \geq \mathrm{Fib}(k) \approx \phi^k/\sqrt{5}$. Therefore the number of steps k grows as the logarithm (to the base ϕ) of n. Hence, the order of growth is $\Theta(\log n)$.

---

41. Euclid's Algorithm is so called because it appears in Euclid's *Elements* (Book 7, ca. 300 BCE). According to Knuth (1997a), it can be considered the oldest known nontrivial algorithm. The ancient Egyptian method of multiplication (exercise 1.18) is surely older, but, as Knuth explains, Euclid's Algorithm is the oldest known to have been presented as a general algorithm, rather than as a set of illustrative examples.

42. This theorem was proved in 1845 by Gabriel Lamé, a French mathematician and engineer known chiefly for his contributions to mathematical physics. To prove the theorem, we consider pairs (a_k, b_k), where $a_k \geq b_k$, for which Euclid's Algorithm terminates in k steps. The proof is based on the claim that, if $(a_{k+1},\ b_{k+1}) \to (a_k,\ b_k) \to (a_{k-1},\ b_{k-1})$ are three successive pairs in the reduction process, then we must have $b_{k+1} \geq b_k + b_{k-1}$. To verify the claim, consider that a reduction step is defined by applying the transformation $a_{k-1} = b_k$, b_{k-1} = remainder of a_k divided by b_k. The second equation means that $a_k = qb_k + b_{k-1}$ for some positive integer q. And since q must be at least 1 we have $a_k = qb_k + b_{k-1} \geq b_k + b_{k-1}$. But in the previous reduction step we have $b_{k+1} = a_k$. Therefore, $b_{k+1} = a_k \geq b_k + b_{k-1}$. This verifies the claim. Now we can prove the theorem by induction on k, the number of steps that the algorithm requires to terminate. The result is true for $k = 1$, since this merely requires that b be at least as large as $\mathrm{Fib}(1) = 1$. Now, assume that the result is true for all integers less than or equal to k and establish the result for $k+1$. Let $(a_{k+1},\ b_{k+1}) \to (a_k,\ b_k) \to (a_{k-1},\ b_{k-1})$ be successive pairs in the reduction process. By our induction hypotheses, we have $b_{k-1} \geq \mathrm{Fib}(k-1)$ and $b_k \geq \mathrm{Fib}(k)$. Thus, applying the claim we just proved together with the definition of the Fibonacci numbers gives $b_{k+1} \geq b_k + b_{k-1} \geq \mathrm{Fib}(k) + \mathrm{Fib}(k-1) = \mathrm{Fib}(k+1)$, which completes the proof of Lamé's Theorem.

Exercise 1.20

The process that a function generates is of course dependent on the rules used by the interpreter. As an example, consider the iterative `gcd` function given above. Suppose we were to interpret this function using normal-order evaluation, as discussed in section 1.1.5. (The normal-order-evaluation rule for conditional expressions is described in exercise 1.5.) Using the substitution method (for normal order), illustrate the process generated in evaluating `gcd(206, 40)` and indicate the `remainder` operations that are actually performed. How many `remainder` operations are actually performed in the normal-order evaluation of `gcd(206, 40)`? In the applicative-order evaluation?

1.2.6 Example: Testing for Primality

This section describes two methods for checking the primality of an integer n, one with order of growth $\Theta(\sqrt{n})$, and a "probabilistic" algorithm with order of growth $\Theta(\log n)$. The exercises at the end of this section suggest programming projects based on these algorithms.

Searching for divisors

Since ancient times, mathematicians have been fascinated by problems concerning prime numbers, and many people have worked on the problem of determining ways to test if numbers are prime. One way to test if a number is prime is to find the number's divisors. The following program finds the smallest integral divisor (greater than 1) of a given number n. It does this in a straightforward way, by testing n for divisibility by successive integers starting with 2.

```
function smallest_divisor(n) {
    return find_divisor(n, 2);
}
function find_divisor(n, test_divisor) {
    return square(test_divisor) > n
           ? n
           : divides(test_divisor, n)
           ? test_divisor
           : find_divisor(n, test_divisor + 1);
}
function divides(a, b) {
    return b % a === 0;
}
```

We can test whether a number is prime as follows: n is prime if and only if n is its own smallest divisor.

```
function is_prime(n) {
    return n === smallest_divisor(n);
}
```

The end test for `find_divisor` is based on the fact that if n is not prime it must have a divisor less than or equal to $\sqrt{n}$.[43] This means that the algorithm need only test divisors between 1 and $\sqrt{n}$. Consequently, the number of steps required to identify n as prime will have order of growth $\Theta(\sqrt{n})$.

43. If d is a divisor of n, then so is n/d. But d and n/d cannot both be greater than $\sqrt{n}$.

The Fermat test

The $\Theta(\log n)$ primality test is based on a result from number theory known as Fermat's Little Theorem.[44]

> **Fermat's Little Theorem:** If n is a prime number and a is any positive integer less than n, then a raised to the nth power is congruent to a modulo n.

(Two numbers are said to be *congruent modulo n* if they both have the same remainder when divided by n. The remainder of a number a when divided by n is also referred to as the *remainder of a modulo n*, or simply as *a modulo n*.)

If n is not prime, then, in general, most of the numbers $a < n$ will not satisfy the above relation. This leads to the following algorithm for testing primality: Given a number n, pick a random number $a < n$ and compute the remainder of a^n modulo n. If the result is not equal to a, then n is certainly not prime. If it is a, then chances are good that n is prime. Now pick another random number a and test it with the same method. If it also satisfies the equation, then we can be even more confident that n is prime. By trying more and more values of a, we can increase our confidence in the result. This algorithm is known as the Fermat test.

To implement the Fermat test, we need a function that computes the exponential of a number modulo another number:

```
function expmod(base, exp, m) {
    return exp === 0
           ? 1
           : is_even(exp)
           ? square(expmod(base, exp / 2, m)) % m
           : (base * expmod(base, exp - 1, m)) % m;
}
```

This is very similar to the `fast_expt` function of section 1.2.4. It uses successive squaring, so that the number of steps grows logarithmically with the exponent.[45]

The Fermat test is performed by choosing at random a number a between 1 and $n-1$ inclusive and checking whether the remainder modulo n of the nth power of a is equal to a. The random number a is chosen using the primitive function `math_random`, which returns a nonnegative number less than 1. Hence, to obtain a

---

44. Pierre de Fermat (1601–1665) is considered to be the founder of modern number theory. He obtained many important number-theoretic results, but he usually announced just the results, without providing his proofs. Fermat's Little Theorem was stated in a letter he wrote in 1640. The first published proof was given by Euler in 1736 (and an earlier, identical proof was discovered in the unpublished manuscripts of Leibniz). The most famous of Fermat's results—known as Fermat's Last Theorem—was jotted down in 1637 in his copy of the book *Arithmetic* (by the third-century Greek mathematician Diophantus) with the remark "I have discovered a truly remarkable proof, but this margin is too small to contain it." Finding a proof of Fermat's Last Theorem became one of the most famous challenges in number theory. A complete solution was finally given in 1995 by Andrew Wiles of Princeton University.

45. The reduction steps in the cases where the exponent e is greater than 1 are based on the fact that, for any integers x, y, and m, we can find the remainder of x times y modulo m by computing separately the remainders of x modulo m and y modulo m, multiplying these, and then taking the remainder of the result modulo m. For instance, in the case where e is even, we compute the remainder of $b^{e/2}$ modulo m, square this, and take the remainder modulo m. This technique is useful because it means we can perform our computation without ever having to deal with numbers much larger than m. (Compare exercise 1.25.)

random number between 1 and $n-1$, we multiply the return value of `math_random` by $n-1$, round down the result with the primitive function `math_floor`, and add 1:

```
function fermat_test(n) {
    function try_it(a) {
        return expmod(a, n, n) === a;
    }
    return try_it(1 + math_floor(math_random() * (n - 1)));
}
```

The following function runs the test a given number of times, as specified by a parameter. Its value is true if the test succeeds every time, and false otherwise.

```
function fast_is_prime(n, times) {
    return times === 0
           ? true
           : fermat_test(n)
           ? fast_is_prime(n, times - 1)
           : false;
}
```

Probabilistic methods

The Fermat test differs in character from most familiar algorithms, in which one computes an answer that is guaranteed to be correct. Here, the answer obtained is only probably correct. More precisely, if n ever fails the Fermat test, we can be certain that n is not prime. But the fact that n passes the test, while an extremely strong indication, is still not a guarantee that n is prime. What we would like to say is that for any number n, if we perform the test enough times and find that n always passes the test, then the probability of error in our primality test can be made as small as we like.

Unfortunately, this assertion is not quite correct. There do exist numbers that fool the Fermat test: numbers n that are not prime and yet have the property that a^n is congruent to a modulo n for all integers $a < n$. Such numbers are extremely rare, so the Fermat test is quite reliable in practice.[46] There are variations of the Fermat test that cannot be fooled. In these tests, as with the Fermat method, one tests the primality of an integer n by choosing a random integer $a < n$ and checking some condition that depends upon n and a. (See exercise 1.28 for an example of such a test.) On the other hand, in contrast to the Fermat test, one can prove that, for any n, the condition does not hold for most of the integers $a < n$ unless n is prime. Thus, if n passes the test for some random choice of a, the chances are better than even that n is prime. If n passes the test for two random choices of a, the chances are better than 3 out of 4 that n is prime. By running the test with more and more randomly chosen values of a we can make the probability of error as small as we like.

46. Numbers that fool the Fermat test are called *Carmichael numbers*, and little is known about them other than that they are extremely rare. There are 255 Carmichael numbers below 100,000,000. The smallest few are 561, 1105, 1729, 2465, 2821, and 6601. In testing primality of very large numbers chosen at random, the chance of stumbling upon a value that fools the Fermat test is less than the chance that cosmic radiation will cause the computer to make an error in carrying out a "correct" algorithm. Considering an algorithm to be inadequate for the first reason but not for the second illustrates the difference between mathematics and engineering.

The existence of tests for which one can prove that the chance of error becomes arbitrarily small has sparked interest in algorithms of this type, which have come to be known as *probabilistic algorithms*. There is a great deal of research activity in this area, and probabilistic algorithms have been fruitfully applied to many fields.[47]

Exercise 1.21

Use the `smallest_divisor` function to find the smallest divisor of each of the following numbers: 199, 1999, 19999.

Exercise 1.22

Assume a primitive function `get_time` of no arguments that returns the number of milliseconds that have passed since 00:00:00 UTC on Thursday, 1 January, 1970.[48] The following `timed_prime_test` function, when called with an integer n, prints n and checks to see if n is prime. If n is prime, the function prints three asterisks[49] followed by the amount of time used in performing the test.

```
function timed_prime_test(n) {
    display(n);
    return start_prime_test(n, get_time());
}
function start_prime_test(n, start_time) {
    return is_prime(n)
           ? report_prime(get_time() - start_time)
           : false;
}
function report_prime(elapsed_time) {
    display(" *** ");
    display(elapsed_time);
    return true;
}
```

Using this function, write a function `search_for_primes` that checks the primality of consecutive odd integers in a specified range. Use your function to find the three smallest primes larger than 1000; larger than 10,000; larger than 100,000; larger than 1,000,000. Note the time needed to test each prime. Since the testing algorithm has order of growth of $\Theta(\sqrt{n})$, you should expect that testing for primes around 10,000 should take about

47. One of the most striking applications of probabilistic prime testing has been to the field of cryptography. Although it is computationally infeasible to factor an arbitrary 300-digit number as of this writing (2021), the primality of such a number can be checked in a few seconds with the Fermat test. This fact forms the basis of a technique for constructing "unbreakable codes" suggested by Rivest, Shamir, and Adleman (1977). The resulting *RSA algorithm* has become a widely used technique for enhancing the security of electronic communications. Because of this and related developments, the study of prime numbers, once considered the epitome of a topic in "pure" mathematics to be studied only for its own sake, now turns out to have important practical applications to cryptography, electronic funds transfer, and information retrieval.

48. This date is called the *UNIX epoch* and is part of the specification of functions that deal with time in the UNIX™ operating system.

49. The primitive function `display` returns its argument, but also prints it. Here `" *** "` is a *string*, a sequence of characters that we pass as argument to the `display` function. Section 2.3.1 introduces strings more thoroughly.

$\sqrt{10}$ times as long as testing for primes around 1000. Do your timing data bear this out? How well do the data for 100,000 and 1,000,000 support the $\sqrt{n}$ prediction? Is your result compatible with the notion that programs on your machine run in time proportional to the number of steps required for the computation?

Exercise 1.23

The `smallest_divisor` function shown at the start of this section does lots of needless testing: After it checks to see if the number is divisible by 2 there is no point in checking to see if it is divisible by any larger even numbers. This suggests that the values used for `test_divisor` should not be 2, 3, 4, 5, 6, ... but rather 2, 3, 5, 7, 9, ... To implement this change, declare a function `next` that returns 3 if its input is equal to 2 and otherwise returns its input plus 2. Modify the `smallest_divisor` function to use `next(test_divisor)` instead of `test_divisor + 1`. With `timed_prime_test` incorporating this modified version of `smallest_divisor`, run the test for each of the 12 primes found in exercise 1.22. Since this modification halves the number of test steps, you should expect it to run about twice as fast. Is this expectation confirmed? If not, what is the observed ratio of the speeds of the two algorithms, and how do you explain the fact that it is different from 2?

Exercise 1.24

Modify the `timed_prime_test` function of exercise 1.22 to use `fast_is_prime` (the Fermat method), and test each of the 12 primes you found in that exercise. Since the Fermat test has $\Theta(\log n)$ growth, how would you expect the time to test primes near 1,000,000 to compare with the time needed to test primes near 1000? Do your data bear this out? Can you explain any discrepancy you find?

Exercise 1.25

Alyssa P. Hacker complains that we went to a lot of extra work in writing `expmod`. After all, she says, since we already know how to compute exponentials, we could have simply written

```
function expmod(base, exp, m) {
    return fast_expt(base, exp) % m;
}
```

Is she correct? Would this function serve as well for our fast prime tester? Explain.

Exercise 1.26

Louis Reasoner is having great difficulty doing exercise 1.24. His `fast_is_prime` test seems to run more slowly than his `is_prime` test. Louis calls his friend Eva Lu Ator over to help. When they examine Louis's code, they find that he has rewritten the `expmod` function to use an explicit multiplication, rather than calling `square`:

```
function expmod(base, exp, m) {
    return exp === 0
           ? 1
           : is_even(exp)
           ? (  expmod(base, exp / 2, m)
              * expmod(base, exp / 2, m)) % m
           : (base * expmod(base, exp - 1, m)) % m;
}
```

"I don't see what difference that could make," says Louis. "I do." says Eva. "By writing the function like that, you have transformed the $\Theta(\log n)$ process into a $\Theta(n)$ process." Explain.

Exercise 1.27

Demonstrate that the Carmichael numbers listed in footnote 46 really do fool the Fermat test. That is, write a function that takes an integer n and tests whether a^n is congruent to a modulo n for every $a < n$, and try your function on the given Carmichael numbers.

Exercise 1.28

One variant of the Fermat test that cannot be fooled is called the *Miller–Rabin test* (Miller 1976; Rabin 1980). This starts from an alternate form of Fermat's Little Theorem, which states that if n is a prime number and a is any positive integer less than n, then a raised to the $(n-1)$st power is congruent to 1 modulo n. To test the primality of a number n by the Miller–Rabin test, we pick a random number $a < n$ and raise a to the $(n-1)$st power modulo n using the `expmod` function. However, whenever we perform the squaring step in `expmod`, we check to see if we have discovered a "nontrivial square root of 1 modulo n," that is, a number not equal to 1 or $n-1$ whose square is equal to 1 modulo n. It is possible to prove that if such a nontrivial square root of 1 exists, then n is not prime. It is also possible to prove that if n is an odd number that is not prime, then, for at least half the numbers $a < n$, computing a^{n-1} in this way will reveal a nontrivial square root of 1 modulo n. (This is why the Miller–Rabin test cannot be fooled.) Modify the `expmod` function to signal if it discovers a nontrivial square root of 1, and use this to implement the Miller–Rabin test with a function analogous to `fermat_test`. Check your function by testing various known primes and non-primes. Hint: One convenient way to make `expmod` signal is to have it return 0.

1.3 Formulating Abstractions with Higher-Order Functions

We have seen that functions are, in effect, abstractions that describe compound operations on numbers independent of the particular numbers. For example, when we declare

```
function cube(x) {
    return x * x * x;
}
```

we are not talking about the cube of a particular number, but rather about a method for obtaining the cube of any number. Of course we could get along without ever declaring this function, by always writing expressions such as

```
3 * 3 * 3
x * x * x
y * y * y
```

and never mentioning `cube` explicitly. This would place us at a serious disadvantage, forcing us to work always at the level of the particular operations that happen to be primitives in the language (multiplication, in this case) rather than in terms of higher-level operations. Our programs would be able to compute cubes, but our language would lack the ability to express the concept of cubing. One of the things we should demand from a powerful programming language is the ability to build

abstractions by assigning names to common patterns and then to work in terms of the abstractions directly. Functions provide this ability. This is why all but the most primitive programming languages include mechanisms for declaring functions.

Yet even in numerical processing we will be severely limited in our ability to create abstractions if we are restricted to functions whose parameters must be numbers. Often the same programming pattern will be used with a number of different functions. To express such patterns as concepts, we will need to construct functions that can accept functions as arguments or return functions as values. Functions that manipulate functions are called *higher-order functions.* This section shows how higher-order functions can serve as powerful abstraction mechanisms, vastly increasing the expressive power of our language.

1.3.1 Functions as Arguments

Consider the following three functions. The first computes the sum of the integers from `a` through `b`:

```
function sum_integers(a, b) {
    return a > b
           ? 0
           : a + sum_integers(a + 1, b);
}
```

The second computes the sum of the cubes of the integers in the given range:

```
function sum_cubes(a, b) {
    return a > b
           ? 0
           : cube(a) + sum_cubes(a + 1, b);
}
```

The third computes the sum of a sequence of terms in the series

$$\frac{1}{1\cdot 3}+\frac{1}{5\cdot 7}+\frac{1}{9\cdot 11}+\cdots$$

which converges to $\pi/8$ (very slowly):[50]

```
function pi_sum(a, b) {
    return a > b
           ? 0
           : 1 / (a * (a + 2)) + pi_sum(a + 4, b);
}
```

These three functions clearly share a common underlying pattern. They are for the most part identical, differing only in the name of the function, the function of `a` used to compute the term to be added, and the function that provides the next value of `a`. We could generate each of the functions by filling in slots in the same template:

50. This series, usually written in the equivalent form $\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots$, is due to Leibniz. We'll see how to use this as the basis for some fancy numerical tricks in section 3.5.3.

```
function name(a, b) {
    return a > b
           ? 0
           : term(a) + name(next(a), b);
}
```

The presence of such a common pattern is strong evidence that there is a useful abstraction waiting to be brought to the surface. Indeed, mathematicians long ago identified the abstraction of *summation of a series* and invented "sigma notation," for example

$$\sum_{n=a}^{b} f(n) \quad = \quad f(a)+\cdots+f(b)$$

to express this concept. The power of sigma notation is that it allows mathematicians to deal with the concept of summation itself rather than only with particular sums—for example, to formulate general results about sums that are independent of the particular series being summed.

Similarly, as program designers, we would like our language to be powerful enough so that we can write a function that expresses the concept of summation itself rather than only functions that compute particular sums. We can do so readily in our functional language by taking the common template shown above and transforming the "slots" into parameters:

```
function sum(term, a, next, b) {
    return a > b
           ? 0
           : term(a) + sum(term, next(a), next, b);
}
```

Notice that `sum` takes as its arguments the lower and upper bounds `a` and `b` together with the functions `term` and `next`. We can use `sum` just as we would any function. For example, we can use it (along with a function `inc` that increments its argument by 1) to define `sum_cubes`:

```
function inc(n) {
    return n + 1;
}
function sum_cubes(a, b) {
    return sum(cube, a, inc, b);
}
```

Using this, we can compute the sum of the cubes of the integers from 1 to 10:

```
sum_cubes(1, 10);
```
3025

With the aid of an identity function to compute the term, we can define `sum_integers` in terms of `sum`:

```
function identity(x) {
    return x;
}
```

```
function sum_integers(a, b) {
    return sum(identity, a, inc, b);
}
```

Then we can add up the integers from 1 to 10:

```
sum_integers(1, 10);
55
```

We can also define `pi_sum` in the same way:[51]

```
function pi_sum(a, b) {
    function pi_term(x) {
        return 1 / (x * (x + 2));
    }
    function pi_next(x) {
        return x + 4;
    }
    return sum(pi_term, a, pi_next, b);
}
```

Using these functions, we can compute an approximation to π:

```
8 * pi_sum(1, 1000);
3.139592655589783
```

Once we have `sum`, we can use it as a building block in formulating further concepts. For instance, the definite integral of a function f between the limits a and b can be approximated numerically using the formula

$$\int_a^b f \quad = \quad \left[f\left(a + \frac{dx}{2}\right) + f\left(a + dx + \frac{dx}{2}\right) + f\left(a + 2dx + \frac{dx}{2}\right) + \cdots \right] dx$$

for small values of dx. We can express this directly as a function:

```
function integral(f, a, b, dx) {
    function add_dx(x) {
        return x + dx;
    }
    return sum(f, a + dx / 2, add_dx, b) * dx;
}

integral(cube, 0, 1, 0.01);
0.24998750000000042

integral(cube, 0, 1, 0.001);
0.249999875000001
```

(The exact value of the integral of `cube` between 0 and 1 is 1/4.)

51. Notice that we have used block structure (section 1.1.8) to embed the declarations of `pi_next` and `pi_term` within `pi_sum`, since these functions are unlikely to be useful for any other purpose. We will see how to get rid of them altogether in section 1.3.2.

Exercise 1.29

Simpson's Rule is a more accurate method of numerical integration than the method illustrated above. Using Simpson's Rule, the integral of a function f between a and b is approximated as

$$\frac{h}{3}[y_0 + 4y_1 + 2y_2 + 4y_3 + 2y_4 + \cdots + 2y_{n-2} + 4y_{n-1} + y_n]$$

where $h = (b-a)/n$, for some even integer n, and $y_k = f(a+kh)$. (Increasing n increases the accuracy of the approximation.) Declare a function that takes as arguments f, a, b, and n and returns the value of the integral, computed using Simpson's Rule. Use your function to integrate `cube` between 0 and 1 (with $n = 100$ and $n = 1000$), and compare the results to those of the `integral` function shown above.

Exercise 1.30

The `sum` function above generates a linear recursion. The function can be rewritten so that the sum is performed iteratively. Show how to do this by filling in the missing expressions in the following declaration:

```
function sum(term, a, next, b) {
    function iter(a, result) {
        return ⟨??⟩
               ? ⟨??⟩
               : iter(⟨??⟩, ⟨??⟩);
    }
    return iter(⟨??⟩, ⟨??⟩);
}
```

Exercise 1.31

a. The `sum` function is only the simplest of a vast number of similar abstractions that can be captured as higher-order functions.[52] Write an analogous function called `product` that returns the product of the values of a function at points over a given range. Show how to define `factorial` in terms of `product`. Also use `product` to compute approximations to π using the formula[53]

$$\frac{\pi}{4} = \frac{2 \cdot 4 \cdot 4 \cdot 6 \cdot 6 \cdot 8 \cdots}{3 \cdot 3 \cdot 5 \cdot 5 \cdot 7 \cdot 7 \cdots}$$

b. If your `product` function generates a recursive process, write one that generates an iterative process. If it generates an iterative process, write one that generates a recursive process.

---

52. The intent of exercises 1.31–1.33 is to demonstrate the expressive power that is attained by using an appropriate abstraction to consolidate many seemingly disparate operations. However, though accumulation and filtering are elegant ideas, our hands are somewhat tied in using them at this point since we do not yet have data structures to provide suitable means of combination for these abstractions. We will return to these ideas in section 2.2.3 when we show how to use *sequences* as interfaces for combining filters and accumulators to build even more powerful abstractions. We will see there how these methods really come into their own as a powerful and elegant approach to designing programs.

53. This formula was discovered by the seventeenth-century English mathematician John Wallis.

Exercise 1.32

a. Show that `sum` and `product` (exercise 1.31) are both special cases of a still more general notion called `accumulate` that combines a collection of terms, using some general accumulation function:

```
accumulate(combiner, null_value, term, a, next, b);
```

The function `accumulate` takes as arguments the same term and range specifications as `sum` and `product`, together with a `combiner` function (of two arguments) that specifies how the current term is to be combined with the accumulation of the preceding terms and a `null_value` that specifies what base value to use when the terms run out. Write `accumulate` and show how `sum` and `product` can both be declared as simple calls to `accumulate`.

b. If your `accumulate` function generates a recursive process, write one that generates an iterative process. If it generates an iterative process, write one that generates a recursive process.

Exercise 1.33

You can obtain an even more general version of `accumulate` (exercise 1.32) by introducing the notion of a *filter* on the terms to be combined. That is, combine only those terms derived from values in the range that satisfy a specified condition. The resulting `filtered_accumulate` abstraction takes the same arguments as accumulate, together with an additional predicate of one argument that specifies the filter. Write `filtered_accumulate` as a function. Show how to express the following using `filtered_accumulate`:

a. the sum of the squares of the prime numbers in the interval a to b (assuming that you have an `is_prime` predicate already written)

b. the product of all the positive integers less than n that are relatively prime to n (i.e., all positive integers $i < n$ such that GCD$(i, n) = 1$).

1.3.2 Constructing Functions using Lambda Expressions

In using `sum` as in section 1.3.1, it seems terribly awkward to have to declare trivial functions such as `pi_term` and `pi_next` just so we can use them as arguments to our higher-order function. Rather than declare `pi_next` and `pi_term`, it would be more convenient to have a way to directly specify "the function that returns its input incremented by 4" and "the function that returns the reciprocal of its input times its input plus 2." We can do this by introducing the *lambda expression* as a syntactic form for creating functions. Using lambda expressions, we can describe what we want as

```
x => x + 4
```

and

```
x => 1 / (x * (x + 2))
```

Then we can express our `pi_sum` function without declaring any auxiliary functions:

```
function pi_sum(a, b) {
    return sum(x => 1 / (x * (x + 2)),
               a,
               x => x + 4,
               b);
}
```

Again using a lambda expression, we can write the `integral` function without having to declare the auxiliary function `add_dx`:

```
function integral(f, a, b, dx) {
    return sum(f,
               a + dx / 2,
               x => x + dx,
               b)
           *
           dx;
}
```

In general, lambda expressions are used to create functions in the same way as function declarations, except that no name is specified for the function and the **`return`** keyword and braces are omitted (if there is only one parameter, the parentheses around the parameter list can also be omitted, as in the examples we have seen).[54]

(*parameters*) => *expression*

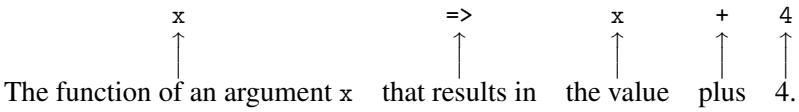
The resulting function is just as much a function as one that is created using a function declaration statement. The only difference is that it has not been associated with any name in the environment. We consider

```
function plus4(x) {
    return x + 4;
}
```

to be equivalent to[55]

```
const plus4 = x => x + 4;
```

We can read a lambda expression as follows:



54. In section 2.2.4, we will extend the syntax of lambda expressions to allow a block as the body rather than just an expression, as in function declaration statements.

55. In JavaScript, there are subtle differences between the two versions: A function declaration statement is automatically "hoisted" (moved) to the beginning of the surrounding block or to the beginning of the program if it occurs outside of any block, whereas a constant declaration is not moved. Names declared with function declaration can be reassigned using assignment (section 3.1.1), whereas names declared with constant declarations can't. In this book, we avoid these features and treat a function declaration as equivalent to the corresponding constant declaration.

Like any expression that has a function as its value, a lambda expression can be used as the function expression in an application such as

```
((x, y, z) => x + y + square(z))(1, 2, 3);
12
```

or, more generally, in any context where we would normally use a function name.[56] Note that => has lower precedence than function application and thus the parentheses around the lambda expression are necessary here.

Using `const` to create local names

Another use of lambda expressions is in creating local names. We often need local names in our functions other than those that have been bound as parameters. For example, suppose we wish to compute the function

$$f(x,y) = x(1+xy)^2 + y(1-y) + (1+xy)(1-y)$$

which we could also express as

$$\begin{aligned} a &= 1+xy \\ b &= 1-y \\ f(x,y) &= xa^2 + yb + ab \end{aligned}$$

In writing a function to compute f, we would like to include as local names not only x and y but also the names of intermediate quantities like a and b. One way to accomplish this is to use an auxiliary function to bind the local names:

```
function f(x, y) {
    function f_helper(a, b) {
        return x * square(a) + y * b + a * b;
    }
    return f_helper(1 + x * y, 1 - y);
}
```

Of course, we could use a lambda expression to specify an anonymous function for binding our local names. The function body then becomes a single call to that function:

```
function f_2(x, y) {
    return ( (a, b) => x * square(a) + y * b + a * b
           )(1 + x * y, 1 - y);
}
```

---

56. It would be clearer and less intimidating to people learning JavaScript if a term more obvious than *lambda expression*, such as *function definition*, were used. But the convention is very firmly entrenched, not just for Lisp and Scheme but also for JavaScript, Java and other languages, no doubt partly due to the influence of the Scheme editions of this book. The notation is adopted from the λ calculus, a mathematical formalism introduced by the mathematical logician Alonzo Church (1941). Church developed the λ calculus to provide a rigorous foundation for studying the notions of function and function application. The λ calculus has become a basic tool for mathematical investigations of the semantics of programming languages.

A more convenient way to declare local names is by using constant declarations within the body of the function. Using `const`, the function can be written as

```
function f_3(x, y) {
    const a = 1 + x * y;
    const b = 1 - y;
    return x * square(a) + y * b + a * b;
}
```

Names that are declared with `const` inside a block have the body of the immediately surrounding block as their scope.[57,58]

Conditional statements

We have seen that it is often useful to declare names that are local to function declarations. When functions become big, we should keep the scope of the names as narrow as possible. Consider for example `expmod` in exercise 1.26.

```
function expmod(base, exp, m) {
    return exp === 0
           ? 1
           : is_even(exp)
           ? (  expmod(base, exp / 2, m)
              * expmod(base, exp / 2, m)) % m
           : (base * expmod(base, exp - 1, m)) % m;
}
```

This function is unnecessarily inefficient, because it contains two identical calls:

```
expmod(base, exp / 2, m);
```

---

57. Note that a name declared in a block cannot be used before the declaration is fully evaluated, regardless of whether the same name is declared outside the block. Thus in the program below, the attempt to use the `a` declared at the top level to provide a value for the calculation of the `b` declared in `f` cannot work.

```
const a = 1;
function f(x) {
    const b = a + x;
    const a = 5;
    return a + b;
}
f(10);
```

The program leads to an error, because the `a` in `a + x` is used before its declaration is evaluated. We will return to this program in section 4.1.6 (exercise 4.19), after we learn more about evaluation.

58. The substitution model can be expanded to say that for a constant declaration, the value of the expression after = is substituted for the name before = in the rest of the block body (after the declaration), similar to the substitution of arguments for parameters in the evaluation of a function application.

While this can be easily fixed in this example using the `square` function, this is not so easy in general. Without using `square`, we would be tempted to introduce a local name for the expression as follows:

```
function expmod(base, exp, m) {
    const half_exp = expmod(base, exp / 2, m);
    return exp === 0
           ? 1
           : is_even(exp)
           ? (half_exp * half_exp) % m
           : (base * expmod(base, exp - 1, m)) % m;
}
```

This would make the function not just inefficient, but actually nonterminating! The problem is that the constant declaration appears outside the conditional expression, which means that it is executed even when the base case `exp === 0` is met. To avoid this situation, we provide for *conditional statements*, and allow return statements to appear in the branches of the statement. Using a conditional statement, we can write the function `expmod` as follows:

```
function expmod(base, exp, m) {
    if (exp === 0) {
        return 1;
    } else {
        if (is_even(exp)) {
            const half_exp = expmod(base, exp / 2, m);
            return (half_exp * half_exp) % m;
        } else {
            return (base * expmod(base, exp - 1, m)) % m;
        }
    }
}
```

The general form of a conditional statement is

`if` (*predicate*) `{` *consequent-statements* `}` `else` `{` *alternative-statements* `}`

As for a conditional expression, the interpreter first evaluates the *predicate*. If it evaluates to true, the interpreter evaluates the *consequent-statements* in sequence, and if it evaluates to false, the interpreter evaluates the *alternative-statements* in sequence. Evaluation of a return statement returns from the surrounding function, ignoring any statements in the sequence after the return statement and any statements after the conditional statement. Note that any constant declarations occurring in either part are local to that part, because each part is enclosed in braces and thus forms its own block.

Exercise 1.34

Suppose we declare

```
function f(g) {
   return g(2);
}
```

Then we have

```
f(square);
4

f(z => z * (z + 1));
6
```

What happens if we (perversely) ask the interpreter to evaluate the application `f(f)`? Explain.

1.3.3 Functions as General Methods

We introduced compound functions in section 1.1.4 as a mechanism for abstracting patterns of numerical operations so as to make them independent of the particular numbers involved. With higher-order functions, such as the `integral` function of section 1.3.1, we began to see a more powerful kind of abstraction: functions used to express general methods of computation, independent of the particular functions involved. In this section we discuss two more elaborate examples—general methods for finding zeros and fixed points of functions—and show how these methods can be expressed directly as functions.

Finding roots of equations by the half-interval method

The *half-interval method* is a simple but powerful technique for finding roots of an equation $f(x) = 0$, where f is a continuous function. The idea is that, if we are given points a and b such that $f(a) < 0 < f(b)$, then f must have at least one zero between a and b. To locate a zero, let x be the average of a and b and compute $f(x)$. If $f(x) > 0$, then f must have a zero between a and x. If $f(x) < 0$, then f must have a zero between x and b. Continuing in this way, we can identify smaller and smaller intervals on which f must have a zero. When we reach a point where the interval is small enough, the process stops. Since the interval of uncertainty is reduced by half at each step of the process, the maximal number of steps required grows as $\Theta(\log(L/T))$, where L is the length of the original interval and T is the error tolerance (that is, the size of the interval we will consider "small enough"). Here is a function that implements this strategy:

```
function search(f, neg_point, pos_point) {
    const midpoint = average(neg_point, pos_point);
    if (close_enough(neg_point, pos_point)) {
        return midpoint;
    } else {
        const test_value = f(midpoint);
        return positive(test_value)
               ? search(f, neg_point, midpoint)
               : negative(test_value)
               ? search(f, midpoint, pos_point)
               : midpoint;
    }
}
```

We assume that we are initially given the function f together with points at which its values are negative and positive. We first compute the midpoint of the two given points. Next we check to see if the given interval is small enough, and if so we simply return the midpoint as our answer. Otherwise, we compute as a test value the value of f at the midpoint. If the test value is positive, then we continue the process with a new interval running from the original negative point to the midpoint. If the test value is negative, we continue with the interval from the midpoint to the positive point. Finally, there is the possibility that the test value is 0, in which case the midpoint is itself the root we are searching for. To test whether the endpoints are "close enough" we can use a function similar to the one used in section 1.1.7 for computing square roots:[59]

```
function close_enough(x, y) {
    return abs(x - y) < 0.001;
}
```

The function `search` is awkward to use directly, because we can accidentally give it points at which f's values do not have the required sign, in which case we get a wrong answer. Instead we will use `search` via the following function, which checks to see which of the endpoints has a negative function value and which has a positive value, and calls the `search` function accordingly. If the function has the

59. We have used 0.001 as a representative "small" number to indicate a tolerance for the acceptable error in a calculation. The appropriate tolerance for a real calculation depends upon the problem to be solved and the limitations of the computer and the algorithm. This is often a very subtle consideration, requiring help from a numerical analyst or some other kind of magician.

same sign on the two given points, the half-interval method cannot be used, in which case the function signals an error.[60]

```
function half_interval_method(f, a, b) {
    const a_value = f(a);
    const b_value = f(b);
    return negative(a_value) && positive(b_value)
           ? search(f, a, b)
           : negative(b_value) && positive(a_value)
           ? search(f, b, a)
           : error("values are not of opposite sign");
}
```

The following example uses the half-interval method to approximate π as the root between 2 and 4 of sin $x = 0$:

```
half_interval_method(math_sin, 2, 4);
```
3.14111328125

Here is another example, using the half-interval method to search for a root of the equation $x^3 - 2x - 3 = 0$ between 1 and 2:

```
half_interval_method(x => x * x * x - 2 * x - 3, 1, 2);
```
1.89306640625

Finding fixed points of functions

A number x is called a *fixed point* of a function f if x satisfies the equation $f(x) = x$. For some functions f we can locate a fixed point by beginning with an initial guess and applying f repeatedly,

$$f(x),\ f(f(x)),\ f(f(f(x))),\ \ldots$$

until the value does not change very much. Using this idea, we can devise a function `fixed_point` that takes as inputs a function and an initial guess and produces an approximation to a fixed point of the function. We apply the function repeatedly until we find two successive values whose difference is less than some prescribed tolerance:

```
const tolerance = 0.00001;
function fixed_point(f, first_guess) {
    function close_enough(x, y) {
        return abs(x - y) < tolerance;
    }
    function try_with(guess) {
        const next = f(guess);
        return close_enough(guess, next)
               ? next
               : try_with(next);
    }
    return try_with(first_guess);
}
```

60. This can be accomplished using `error`, which takes as argument a string that is printed as error message along with the number of the program line that gave rise to the call of `error`.

For example, we can use this method to approximate the fixed point of the cosine function, starting with 1 as an initial approximation:[61]

```
fixed_point(math_cos, 1);
0.7390822985224023
```

Similarly, we can find a solution to the equation $y = \sin y + \cos y$:

```
fixed_point(y => math_sin(y) + math_cos(y), 1);
1.2587315962971173
```

The fixed-point process is reminiscent of the process we used for finding square roots in section 1.1.7. Both are based on the idea of repeatedly improving a guess until the result satisfies some criterion. In fact, we can readily formulate the square-root computation as a fixed-point search. Computing the square root of some number x requires finding a y such that $y^2 = x$. Putting this equation into the equivalent form $y = x/y$, we recognize that we are looking for a fixed point of the function[62] $y \mapsto x/y$, and we can therefore try to compute square roots as

```
function sqrt(x) {
    return fixed_point(y => x / y, 1);
}
```

Unfortunately, this fixed-point search does not converge. Consider an initial guess y_1. The next guess is $y_2 = x/y_1$ and the next guess is $y_3 = x/y_2 = x/(x/y_1) = y_1$. This results in an infinite loop in which the two guesses y_1 and y_2 repeat over and over, oscillating about the answer.

One way to control such oscillations is to prevent the guesses from changing so much. Since the answer is always between our guess y and x/y, we can make a new guess that is not as far from y as x/y by averaging y with x/y, so that the next guess after y is $\frac{1}{2}(y + x/y)$ instead of x/y. The process of making such a sequence of guesses is simply the process of looking for a fixed point of $y \mapsto \frac{1}{2}(y + x/y)$:

```
function sqrt(x) {
    return fixed_point(y => average(y, x / y), 1);
}
```

(Note that $y = \frac{1}{2}(y + x/y)$ is a simple transformation of the equation $y = x/y$; to derive it, add y to both sides of the equation and divide by 2.)

With this modification, the square-root function works. In fact, if we unravel the definitions, we can see that the sequence of approximations to the square root generated here is precisely the same as the one generated by our original square-root function of section 1.1.7. This approach of averaging successive approximations to a solution, a technique we call *average damping*, often aids the convergence of fixed-point searches.

---

61. To obtain a fixed point of cosine on a calculator, set it to radians mode and then repeatedly press the cos button until the value does not change any longer.

62. $\mapsto$ (pronounced "maps to") is the mathematician's way of writing lambda expressions. $y \mapsto x/y$ means `y => x / y`, that is, the function whose value at y is x/y.

Exercise 1.35

Show that the golden ratio ϕ (section 1.2.2) is a fixed point of the transformation $x \mapsto 1 + 1/x$, and use this fact to compute ϕ by means of the `fixed_point` function.

Exercise 1.36

Modify `fixed_point` so that it prints the sequence of approximations it generates, using the primitive function `display` shown in exercise 1.22. Then find a solution to $x^x = 1000$ by finding a fixed point of $x \mapsto \log(1000)/\log(x)$. (Use the primitive function `math_log`, which computes natural logarithms.) Compare the number of steps this takes with and without average damping. (Note that you cannot start `fixed_point` with a guess of 1, as this would cause division by $\log(1) = 0$.)

Exercise 1.37

An infinite *continued fraction* is an expression of the form

$$f = \cfrac{N_1}{D_1 + \cfrac{N_2}{D_2 + \cfrac{N_3}{D_3 + \cdots}}}$$

As an example, one can show that the infinite continued fraction expansion with the N_i and the D_i all equal to 1 produces $1/\phi$, where ϕ is the golden ratio (described in section 1.2.2). One way to approximate an infinite continued fraction is to truncate the expansion after a given number of terms. Such a truncation—a so-called *k-term finite continued fraction*—has the form

$$\cfrac{N_1}{D_1 + \cfrac{N_2}{\ddots + \cfrac{N_K}{D_K}}}$$

a. Suppose that `n` and `d` are functions of one argument (the term index i) that return the N_i and D_i of the terms of the continued fraction. Declare a function `cont_frac` such that evaluating `cont_frac(n, d, k)` computes the value of the k-term finite continued fraction. Check your function by approximating $1/\phi$ using

```
cont_frac(i => 1, i => 1, k);
```

for successive values of `k`. How large must you make `k` in order to get an approximation that is accurate to 4 decimal places?

b. If your `cont_frac` function generates a recursive process, write one that generates an iterative process. If it generates an iterative process, write one that generates a recursive process.

Exercise 1.38

In 1737, the Swiss mathematician Leonhard Euler published a memoir *De Fractionibus Continuis*, which included a continued fraction expansion for $e - 2$, where e is the base of the natural logarithms. In this fraction, the N_i are all 1, and the D_i are successively 1, 2, 1, 1, 4, 1, 1, 6, 1, 1, 8, ... Write a program that uses your `cont_frac` function from exercise 1.37 to approximate e, based on Euler's expansion.

Exercise 1.39

A continued fraction representation of the tangent function was published in 1770 by the German mathematician J.H. Lambert:

$$\tan x = \cfrac{x}{1 - \cfrac{x^2}{3 - \cfrac{x^2}{5 - \cfrac{x^2}{\ddots}}}}$$

where x is in radians. Declare a function `tan_cf(x, k)` that computes an approximation to the tangent function based on Lambert's formula. As in exercise 1.37, `k` specifies the number of terms to compute.

1.3.4 Functions as Returned Values

The above examples demonstrate how the ability to pass functions as arguments significantly enhances the expressive power of our programming language. We can achieve even more expressive power by creating functions whose returned values are themselves functions.

We can illustrate this idea by looking again at the fixed-point example described at the end of section 1.3.3. We formulated a new version of the square-root function as a fixed-point search, starting with the observation that $\sqrt{x}$ is a fixed-point of the function $y \mapsto x/y$. Then we used average damping to make the approximations converge. Average damping is a useful general technique in itself. Namely, given a function f, we consider the function whose value at x is equal to the average of x and $f(x)$.

We can express the idea of average damping by means of the following function:

```
function average_damp(f) {
    return x => average(x, f(x));
}
```

The function `average_damp` takes as its argument a function `f` and returns as its value a function (produced by the lambda expression) that, when applied to a number `x`, produces the average of `x` and `f(x)`. For example, applying `average_damp` to the `square` function produces a function whose value at some number x is the average of x and x^2. Applying this resulting function to 10 returns the average of 10 and 100, or 55:[63]

```
average_damp(square)(10);
55
```

Using `average_damp`, we can reformulate the square-root function as follows:

63. Observe that this is an application whose function expression is itself an application. Exercise 1.4 already demonstrated the ability to form such applications, but that was only a toy example. Here we begin to see the real need for such applications—when applying a function that is obtained as the value returned by a higher-order function.

```
function sqrt(x) {
    return fixed_point(average_damp(y => x / y), 1);
}
```

Notice how this formulation makes explicit the three ideas in the method: fixed-point search, average damping, and the function $y \mapsto x/y$. It is instructive to compare this formulation of the square-root method with the original version given in section 1.1.7. Bear in mind that these functions express the same process, and notice how much clearer the idea becomes when we express the process in terms of these abstractions. In general, there are many ways to formulate a process as a function. Experienced programmers know how to choose process formulations that are particularly perspicuous, and where useful elements of the process are exposed as separate entities that can be reused in other applications. As a simple example of reuse, notice that the cube root of x is a fixed point of the function $y \mapsto x/y^2$, so we can immediately generalize our square-root function to one that extracts cube roots:[64]

```
function cube_root(x) {
    return fixed_point(average_damp(y => x / square(y)), 1);
}
```

Newton's method

When we first introduced the square-root function, in section 1.1.7, we mentioned that this was a special case of *Newton's method*. If $x \mapsto g(x)$ is a differentiable function, then a solution of the equation $g(x) = 0$ is a fixed point of the function $x \mapsto f(x)$ where

$$f(x) \quad = \quad x - \frac{g(x)}{Dg(x)}$$

and $Dg(x)$ is the derivative of g evaluated at x. Newton's method is the use of the fixed-point method we saw above to approximate a solution of the equation by finding a fixed point of the function f.[65] For many functions g and for sufficiently good initial guesses for x, Newton's method converges very rapidly to a solution of $g(x) = 0$.[66]

In order to implement Newton's method as a function, we must first express the idea of derivative. Note that "derivative," like average damping, is something

---

64. See exercise 1.45 for a further generalization.

65. Elementary calculus books usually describe Newton's method in terms of the sequence of approximations $x_{n+1} = x_n - g(x_n)/Dg(x_n)$. Having language for talking about processes and using the idea of fixed points simplifies the description of the method.

66. Newton's method does not always converge to an answer, but it can be shown that in favorable cases each iteration doubles the number-of-digits accuracy of the approximation to the solution. In such cases, Newton's method will converge much more rapidly than the half-interval method.

that transforms a function into another function. For instance, the derivative of the function $x \mapsto x^3$ is the function $x \mapsto 3x^2$. In general, if g is a function and dx is a small number, then the derivative Dg of g is the function whose value at any number x is given (in the limit of small dx) by

$$Dg(x) \quad = \quad \frac{g(x+dx)-g(x)}{dx}$$

Thus, we can express the idea of derivative (taking dx to be, say, 0.00001) as the function

```
function deriv(g) {
    return x => (g(x + dx) - g(x)) / dx;
}
```

along with the declaration

```
const dx = 0.00001;
```

Like `average_damp`, `deriv` is a function that takes a function as argument and returns a function as value. For example, to approximate the derivative of $x \mapsto x^3$ at 5 (whose exact value is 75) we can evaluate

```
function cube(x) { return x * x * x; }

deriv(cube)(5);
75.00014999664018
```

With the aid of `deriv`, we can express Newton's method as a fixed-point process:

```
function newton_transform(g) {
    return x => x - g(x) / deriv(g)(x);
}
function newtons_method(g, guess) {
    return fixed_point(newton_transform(g), guess);
}
```

The `newton_transform` function expresses the formula at the beginning of this section, and `newtons_method` is readily defined in terms of this. It takes as arguments a function that computes the function for which we want to find a zero, together with an initial guess. For instance, to find the square root of x, we can use Newton's method to find a zero of the function $y \mapsto y^2 - x$ starting with an initial guess of 1.[67] This provides yet another form of the square-root function:

```
function sqrt(x) {
    return newtons_method(y => square(y) - x, 1);
}
```

67. For finding square roots, Newton's method converges rapidly to the correct solution from any starting point.

Abstractions and first-class functions

We've seen two ways to express the square-root computation as an instance of a more general method, once as a fixed-point search and once using Newton's method. Since Newton's method was itself expressed as a fixed-point process, we actually saw two ways to compute square roots as fixed points. Each method begins with a function and finds a fixed point of some transformation of the function. We can express this general idea itself as a function:

```
function fixed_point_of_transform(g, transform, guess) {
    return fixed_point(transform(g), guess);
}
```

This very general function takes as its arguments a function g that computes some function, a function that transforms g, and an initial guess. The returned result is a fixed point of the transformed function.

Using this abstraction, we can recast the first square-root computation from this section (where we look for a fixed point of the average-damped version of $y \mapsto x/y$) as an instance of this general method:

```
function sqrt(x) {
    return fixed_point_of_transform(
               y => x / y,
               average_damp,
               1);
}
```

Similarly, we can express the second square-root computation from this section (an instance of Newton's method that finds a fixed point of the Newton transform of $y \mapsto y^2 - x$) as

```
function sqrt(x) {
    return fixed_point_of_transform(
               y => square(y) - x,
               newton_transform,
               1);
}
```

We began section 1.3 with the observation that compound functions are a crucial abstraction mechanism, because they permit us to express general methods of computing as explicit elements in our programming language. Now we've seen how higher-order functions permit us to manipulate these general methods to create further abstractions.

As programmers, we should be alert to opportunities to identify the underlying abstractions in our programs and to build upon them and generalize them to create more powerful abstractions. This is not to say that one should always write programs in the most abstract way possible; expert programmers know how to choose the level of abstraction appropriate to their task. But it is important to be able to think in terms of these abstractions, so that we can be ready to apply them in new contexts. The significance of higher-order functions is that they enable us to represent these abstractions explicitly as elements in our programming language, so that they can be handled just like other computational elements.

In general, programming languages impose restrictions on the ways in which computational elements can be manipulated. Elements with the fewest restrictions are said to have *first-class* status. Some of the "rights and privileges" of first-class elements are:[68]

- They may be referred to using names.
- They may be passed as arguments to functions.
- They may be returned as the results of functions.
- They may be included in data structures.[69]

JavaScript, like other high-level programming languages, awards functions full first-class status. This poses challenges for efficient implementation, but the resulting gain in expressive power is enormous.[70]

Exercise 1.40

Declare a function `cubic` that can be used together with the `newtons_method` function in expressions of the form

```
newtons_method(cubic(a, b, c), 1)
```

to approximate zeros of the cubic $x^3 + ax^2 + bx + c$.

Exercise 1.41

Declare a function `double` that takes a function of one argument as argument and returns a function that applies the original function twice. For example, if `inc` is a function that adds 1 to its argument, then `double(inc)` should be a function that adds 2. What value is returned by

```
double(double(double))(inc)(5);
```

Exercise 1.42

Let f and g be two one-argument functions. The *composition* f after g is defined to be the function $x \mapsto f(g(x))$. Declare a function `compose` that implements composition. For example, if `inc` is a function that adds 1 to its argument,

```
compose(square, inc)(6);
49
```

---

68. The notion of first-class status of programming-language elements is due to the British computer scientist Christopher Strachey (1916–1975).

69. We'll see examples of this after we introduce data structures in chapter 2.

70. The major implementation cost of first-class functions is that allowing functions to be returned as values requires reserving storage for a function's free names even while the function is not executing. In the JavaScript implementation we will study in section 4.1, these names are stored in the function's environment.

Exercise 1.43

If f is a numerical function and n is a positive integer, then we can form the nth repeated application of f, which is defined to be the function whose value at x is $f(f(\ldots(f(x))\ldots))$. For example, if f is the function $x \mapsto x+1$, then the nth repeated application of f is the function $x \mapsto x+n$. If f is the operation of squaring a number, then the nth repeated application of f is the function that raises its argument to the 2^nth power. Write a function that takes as inputs a function that computes f and a positive integer n and returns the function that computes the nth repeated application of f. Your function should be able to be used as follows:

```
repeated(square, 2)(5);
625
```

Hint: You may find it convenient to use `compose` from exercise 1.42.

Exercise 1.44

The idea of *smoothing* a function is an important concept in signal processing. If f is a function and dx is some small number, then the smoothed version of f is the function whose value at a point x is the average of $f(x-dx)$, $f(x)$, and $f(x+dx)$. Write a function `smooth` that takes as input a function that computes f and returns a function that computes the smoothed f. It is sometimes valuable to repeatedly smooth a function (that is, smooth the smoothed function, and so on) to obtained the *n-fold smoothed function*. Show how to generate the n-fold smoothed function of any given function using `smooth` and `repeated` from exercise 1.43.

Exercise 1.45

We saw in section 1.3.3 that attempting to compute square roots by naively finding a fixed point of $y \mapsto x/y$ does not converge, and that this can be fixed by average damping. The same method works for finding cube roots as fixed points of the average-damped $y \mapsto x/y^2$. Unfortunately, the process does not work for fourth roots—a single average damp is not enough to make a fixed-point search for $y \mapsto x/y^3$ converge. On the other hand, if we average-damp twice (i.e., use the average damp of the average damp of $y \mapsto x/y^3$) the fixed-point search does converge. Do some experiments to determine how many average damps are required to compute nth roots as a fixed-point search based upon repeated average damping of $y \mapsto x/y^{n-1}$. Use this to implement a simple function for computing nth roots using `fixed_point`, `average_damp`, and the `repeated` function of exercise 1.43. Assume that any arithmetic operations you need are available as primitives.

Exercise 1.46

Several of the numerical methods described in this chapter are instances of an extremely general computational strategy known as *iterative improvement*. Iterative improvement says that, to compute something, we start with an initial guess for the answer, test if the guess is good enough, and otherwise improve the guess and continue the process using the improved guess as the new guess. Write a function `iterative_improve` that takes two functions as arguments: a method for telling whether a guess is good enough and a method for improving a guess. The function `iterative_improve` should return as its value a function that takes a guess as argument and keeps improving the guess until it is good enough. Rewrite the `sqrt` function of section 1.1.7 and the `fixed_point` function of section 1.3.3 in terms of `iterative_improve`.

2 Building Abstractions with Data

> We now come to the decisive step of mathematical abstraction: we forget about what the symbols stand for. ... [The mathematician] need not be idle; there are many operations which he may carry out with these symbols, without ever having to look at the things they stand for.
>
> —Hermann Weyl, *The Mathematical Way of Thinking*

We concentrated in chapter 1 on computational processes and on the role of functions in program design. We saw how to use primitive data (numbers) and primitive operations (arithmetic operations), how to combine functions to form compound functions through composition, conditionals, and the use of parameters, and how to abstract processes by using function declarations. We saw that a function can be regarded as a pattern for the local evolution of a process, and we classified, reasoned about, and performed simple algorithmic analyses of some common patterns for processes as embodied in functions. We also saw that higher-order functions enhance the power of our language by enabling us to manipulate, and thereby to reason in terms of, general methods of computation. This is much of the essence of programming.

In this chapter we are going to look at more complex data. All the functions in chapter 1 operate on simple numerical data, and simple data are not sufficient for many of the problems we wish to address using computation. Programs are typically designed to model complex phenomena, and more often than not one must construct computational objects that have several parts in order to model real-world phenomena that have several aspects. Thus, whereas our focus in chapter 1 was on building abstractions by combining functions to form compound functions, we turn in this chapter to another key aspect of any programming language: the means it provides for building abstractions by combining data objects to form *compound data*.

Why do we want compound data in a programming language? For the same reasons that we want compound functions: to elevate the conceptual level at which we can design our programs, to increase the modularity of our designs, and to enhance the expressive power of our language. Just as the ability to declare functions enables us to deal with processes at a higher conceptual level than that of the primitive operations of the language, the ability to construct compound data objects enables us to deal with data at a higher conceptual level than that of the primitive data objects of the language.

Consider the task of designing a system to perform arithmetic with rational numbers. We could imagine an operation `add_rat` that takes two rational numbers and produces their sum. In terms of simple data, a rational number can be thought of as two integers: a numerator and a denominator. Thus, we could design a program in which each rational number would be represented by two integers (a numerator and a denominator) and where `add_rat` would be implemented by two functions (one

producing the numerator of the sum and one producing the denominator). But this would be awkward, because we would then need to explicitly keep track of which numerators corresponded to which denominators. In a system intended to perform many operations on many rational numbers, such bookkeeping details would clutter the programs substantially, to say nothing of what they would do to our minds. It would be much better if we could "glue together" a numerator and denominator to form a pair—a *compound data object*—that our programs could manipulate in a way that would be consistent with regarding a rational number as a single conceptual unit.

The use of compound data also enables us to increase the modularity of our programs. If we can manipulate rational numbers directly as objects in their own right, then we can separate the part of our program that deals with rational numbers per se from the details of how rational numbers may be represented as pairs of integers. The general technique of isolating the parts of a program that deal with how data objects are represented from the parts of a program that deal with how data objects are used is a powerful design methodology called *data abstraction*. We will see how data abstraction makes programs much easier to design, maintain, and modify.

The use of compound data leads to a real increase in the expressive power of our programming language. Consider the idea of forming a "linear combination" $ax + by$. We might like to write a function that would accept a, b, x, and y as arguments and return the value of $ax + by$. This presents no difficulty if the arguments are to be numbers, because we can readily declare the function

```
function linear_combination(a, b, x, y) {
    return a * x + b * y;
}
```

But suppose we are not concerned only with numbers. Suppose we would like to describe a process that forms linear combinations whenever addition and multiplication are defined—for rational numbers, complex numbers, polynomials, or whatever. We could express this as a function of the form

```
function linear_combination(a, b, x, y) {
    return add(mul(a, x), mul(b, y));
}
```

where `add` and `mul` are not the primitive functions `+` and `*` but rather more complex things that will perform the appropriate operations for whatever kinds of data we pass in as the arguments `a`, `b`, `x`, and `y`. The key point is that the only thing `linear_combination` should need to know about `a`, `b`, `x`, and `y` is that the functions `add` and `mul` will perform the appropriate manipulations. From the perspective of the function `linear_combination`, it is irrelevant what `a`, `b`, `x`, and `y` are and even more irrelevant how they might happen to be represented in terms of more primitive data. This same example shows why it is important that our programming language provide the ability to manipulate compound objects directly: Without this, there is

no way for a function such as `linear_combination` to pass its arguments along to `add` and `mul` without having to know their detailed structure.[1]

We begin this chapter by implementing the rational-number arithmetic system mentioned above. This will form the background for our discussion of compound data and data abstraction. As with compound functions, the main issue to be addressed is that of abstraction as a technique for coping with complexity, and we will see how data abstraction enables us to erect suitable *abstraction barriers* between different parts of a program.

We will see that the key to forming compound data is that a programming language should provide some kind of "glue" so that data objects can be combined to form more complex data objects. There are many possible kinds of glue. Indeed, we will discover how to form compound data using no special "data" operations at all, only functions. This will further blur the distinction between "function" and "data," which was already becoming tenuous toward the end of chapter 1. We will also explore some conventional techniques for representing sequences and trees. One key idea in dealing with compound data is the notion of *closure*—that the glue we use for combining data objects should allow us to combine not only primitive data objects, but compound data objects as well. Another key idea is that compound data objects can serve as *conventional interfaces* for combining program modules in mix-and-match ways. We illustrate some of these ideas by presenting a simple graphics language that exploits closure.

We will then augment the representational power of our language by introducing *symbolic expressions*—data whose elementary parts can be arbitrary symbols rather than only numbers. We explore various alternatives for representing sets of objects. We will find that, just as a given numerical function can be computed by many different computational processes, there are many ways in which a given data structure can be represented in terms of simpler objects, and the choice of representation can have significant impact on the time and space requirements of processes that manipulate the data. We will investigate these ideas in the context of symbolic differentiation, the representation of sets, and the encoding of information.

Next we will take up the problem of working with data that may be represented differently by different parts of a program. This leads to the need to implement *generic operations*, which must handle many different types of data. Maintaining modularity in the presence of generic operations requires more powerful abstraction barriers than can be erected with simple data abstraction alone. In particular, we introduce *data-directed programming* as a technique that allows individual data

1. The ability to directly manipulate functions provides an analogous increase in the expressive power of a programming language. For example, in section 1.3.1 we introduced the `sum` function, which takes a function `term` as an argument and computes the sum of the values of `term` over some specified interval. In order to define `sum`, it is crucial that we be able to speak of a function such as `term` as an entity in its own right, without regard for how `term` might be expressed with more primitive operations. Indeed, if we did not have the notion of "a function," it is doubtful that we would ever even think of the possibility of defining an operation such as `sum`. Moreover, insofar as performing the summation is concerned, the details of how `term` may be constructed from more primitive operations are irrelevant.

representations to be designed in isolation and then combined *additively* (i.e., without modification). To illustrate the power of this approach to system design, we close the chapter by applying what we have learned to the implementation of a package for performing symbolic arithmetic on polynomials, in which the coefficients of the polynomials can be integers, rational numbers, complex numbers, and even other polynomials.

2.1 Introduction to Data Abstraction

In section 1.1.8, we noted that a function used as an element in creating a more complex function could be regarded not only as a collection of particular operations but also as a functional abstraction. That is, the details of how the function was implemented could be suppressed, and the particular function itself could be replaced by any other function with the same overall behavior. In other words, we could make an abstraction that would separate the way the function would be used from the details of how the function would be implemented in terms of more primitive functions. The analogous notion for compound data is called *data abstraction*. Data abstraction is a methodology that enables us to isolate how a compound data object is used from the details of how it is constructed from more primitive data objects.

The basic idea of data abstraction is to structure the programs that are to use compound data objects so that they operate on "abstract data." That is, our programs should use data in such a way as to make no assumptions about the data that are not strictly necessary for performing the task at hand. At the same time, a "concrete" data representation is defined independent of the programs that use the data. The interface between these two parts of our system will be a set of functions, called *selectors* and *constructors*, that implement the abstract data in terms of the concrete representation. To illustrate this technique, we will consider how to design a set of functions for manipulating rational numbers.

2.1.1 Example: Arithmetic Operations for Rational Numbers

Suppose we want to do arithmetic with rational numbers. We want to be able to add, subtract, multiply, and divide them and to test whether two rational numbers are equal.

Let us begin by assuming that we already have a way of constructing a rational number from a numerator and a denominator. We also assume that, given a rational number, we have a way of extracting (or selecting) its numerator and its denominator. Let us further assume that the constructor and selectors are available as functions:

- `make_rat(`*n*`, `*d*`)` returns the rational number whose numerator is the integer *n* and whose denominator is the integer *d*.
- `numer(`*x*`)` returns the numerator of the rational number *x*.
- `denom(`*x*`)` returns the denominator of the rational number *x*.

We are using here a powerful strategy of synthesis: *wishful thinking*. We haven't yet said how a rational number is represented, or how the functions `numer`, `denom`, and `make_rat` should be implemented. Even so, if we did have these three functions, we could then add, subtract, multiply, divide, and test equality by using the following

relations:

$$\frac{n_1}{d_1} + \frac{n_2}{d_2} = \frac{n_1 d_2 + n_2 d_1}{d_1 d_2}$$

$$\frac{n_1}{d_1} - \frac{n_2}{d_2} = \frac{n_1 d_2 - n_2 d_1}{d_1 d_2}$$

$$\frac{n_1}{d_1} \cdot \frac{n_2}{d_2} = \frac{n_1 n_2}{d_1 d_2}$$

$$\frac{n_1/d_1}{n_2/d_2} = \frac{n_1 d_2}{d_1 n_2}$$

$$\frac{n_1}{d_1} = \frac{n_2}{d_2} \quad \text{if and only if} \quad n_1 d_2 = n_2 d_1$$

We can express these rules as functions:

```
function add_rat(x, y) {
    return make_rat(numer(x) * denom(y) + numer(y) * denom(x),
                    denom(x) * denom(y));
}
function sub_rat(x, y) {
    return make_rat(numer(x) * denom(y) - numer(y) * denom(x),
                    denom(x) * denom(y));
}
function mul_rat(x, y) {
    return make_rat(numer(x) * numer(y),
                    denom(x) * denom(y));
}
function div_rat(x, y) {
    return make_rat(numer(x) * denom(y),
                    denom(x) * numer(y));
}
function equal_rat(x, y) {
    return numer(x) * denom(y) === numer(y) * denom(x);
}
```

Now we have the operations on rational numbers defined in terms of the selector and constructor functions `numer`, `denom`, and `make_rat`. But we haven't yet defined these. What we need is some way to glue together a numerator and a denominator to form a rational number.

Pairs

To enable us to implement the concrete level of our data abstraction, our JavaScript environment provides a compound structure called a *pair*, which can be constructed with the primitive function `pair`. This function takes two arguments and returns a compound data object that contains the two arguments as parts. Given a pair, we can extract the parts using the primitive functions `head` and `tail`. Thus, we can use `pair`, `head`, and `tail` as follows:

```
const x = pair(1, 2);
```

```
head(x);
1

tail(x);
2
```

Notice that a pair is a data object that can be given a name and manipulated, just like a primitive data object. Moreover, `pair` can be used to form pairs whose elements are pairs, and so on:

```
const x = pair(1, 2);

const y = pair(3, 4);

const z = pair(x, y);

head(head(z));
1

head(tail(z));
3
```

In section 2.2 we will see how this ability to combine pairs means that pairs can be used as general-purpose building blocks to create all sorts of complex data structures. The single compound-data primitive *pair*, implemented by the functions `pair`, `head`, and `tail`, is the only glue we need. Data objects constructed from pairs are called *list-structured* data.

Representing rational numbers

Pairs offer a natural way to complete the rational-number system. Simply represent a rational number as a pair of two integers: a numerator and a denominator. Then `make_rat`, `numer`, and `denom` are readily implemented as follows:[2]

```
function make_rat(n, d) { return pair(n, d); }
function numer(x) { return head(x); }
function denom(x) { return tail(x); }
```

Also, in order to display the results of our computations, we can print rational numbers by printing the numerator, a slash, and the denominator. We use the primitive

---

2. Another way to define the selectors and constructor is

```
const make_rat = pair;
const numer = head;
const denom = tail;
```

The first definition associates the name `make_rat` with the value of the expression `pair`, which is the primitive function that constructs pairs. Thus `make_rat` and `pair` are names for the same primitive constructor.

Defining selectors and constructors in this way is efficient: Instead of `make_rat` *calling* `pair`, `make_rat` *is* `pair`, so there is only one function called, not two, when `make_rat` is called. On the other hand, doing this defeats debugging aids that trace function calls or put breakpoints on function calls: You may want to watch `make_rat` being called, but you certainly don't want to watch every call to `pair`.

We have chosen not to use this style of definition in this book.

function `stringify` to turn any value (here a number) into a string. The operator `+` in JavaScript is *overloaded*; it can be applied to two numbers or to two strings, and in the latter case it returns the result of *concatenating* the two strings.[3]

```
function print_rat(x) {
    return display(stringify(numer(x)) + " / " + stringify(denom(x)));
}
```

Now we can try our rational-number functions:[4]

```
const one_half = make_rat(1, 2);

print_rat(one_half);
"1 / 2"

const one_third = make_rat(1, 3);

print_rat(add_rat(one_half, one_third));
"5 / 6"

print_rat(mul_rat(one_half, one_third));
"1 / 6"

print_rat(add_rat(one_third, one_third));
"6 / 9"
```

As the final example shows, our rational-number implementation does not reduce rational numbers to lowest terms. We can remedy this by changing `make_rat`. If we have a `gcd` function like the one in section 1.2.5 that produces the greatest common divisor of two integers, we can use `gcd` to reduce the numerator and the denominator to lowest terms before constructing the pair:

```
function make_rat(n, d) {
    const g = gcd(n, d);
    return pair(n / g, d / g);
}
```

Now we have

```
print_rat(add_rat(one_third, one_third));
"2 / 3"
```

as desired. This modification was accomplished by changing the constructor `make_rat` without changing any of the functions (such as `add_rat` and `mul_rat`) that implement the actual operations.

---

3. In JavaScript, the operator `+` can also be applied to a string and a number and to other operand combinations, but in this book, we choose to apply it either to two numbers or to two strings.

4. The primitive function `display` introduced in exercise 1.22 returns its argument, but in the uses of `print_rat` below, we show only what `print_rat` prints, not what the interpreter prints as the value returned by `print_rat`.

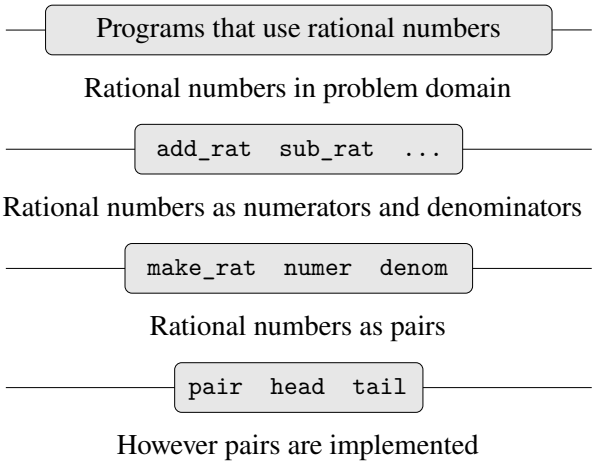


Figure 2.1 Data-abstraction barriers in the rational-number package.

Exercise 2.1

Define a better version of `make_rat` that handles both positive and negative arguments. The function `make_rat` should normalize the sign so that if the rational number is positive, both the numerator and denominator are positive, and if the rational number is negative, only the numerator is negative.

2.1.2 Abstraction Barriers

Before continuing with more examples of compound data and data abstraction, let us consider some of the issues raised by the rational-number example. We defined the rational-number operations in terms of a constructor `make_rat` and selectors `numer` and `denom`. In general, the underlying idea of data abstraction is to identify for each type of data object a basic set of operations in terms of which all manipulations of data objects of that type will be expressed, and then to use only those operations in manipulating the data.

We can envision the structure of the rational-number system as shown in figure 2.1. The horizontal lines represent *abstraction barriers* that isolate different "levels" of the system. At each level, the barrier separates the programs (above) that use the data abstraction from the programs (below) that implement the data abstraction. Programs that use rational numbers manipulate them solely in terms of the functions supplied "for public use" by the rational-number package: `add_rat`, `sub_rat`, `mul_rat`, `div_rat`, and `equal_rat`. These, in turn, are implemented solely in terms of the constructor and selectors `make_rat`, `numer`, and `denom`, which themselves are implemented in terms of pairs. The details of how pairs are implemented are irrelevant to the rest of the rational-number package so long as pairs can be manipulated by the use of `pair`, `head`, and `tail`. In effect, functions at each level are the interfaces that define the abstraction barriers and connect the different levels.

This simple idea has many advantages. One advantage is that it makes programs much easier to maintain and to modify. Any complex data structure can

be represented in a variety of ways with the primitive data structures provided by a programming language. Of course, the choice of representation influences the programs that operate on it; thus, if the representation were to be changed at some later time, all such programs might have to be modified accordingly. This task could be time-consuming and expensive in the case of large programs unless the dependence on the representation were to be confined by design to a very few program modules.

For example, an alternate way to address the problem of reducing rational numbers to lowest terms is to perform the reduction whenever we access the parts of a rational number, rather than when we construct it. This leads to different constructor and selector functions:

```
function make_rat(n, d) {
    return pair(n, d);
}
function numer(x) {
    const g = gcd(head(x), tail(x));
    return head(x) / g;
}
function denom(x) {
    const g = gcd(head(x), tail(x));
    return tail(x) / g;
}
```

The difference between this implementation and the previous one lies in when we compute the `gcd`. If in our typical use of rational numbers we access the numerators and denominators of the same rational numbers many times, it would be preferable to compute the `gcd` when the rational numbers are constructed. If not, we may be better off waiting until access time to compute the `gcd`. In any case, when we change from one representation to the other, the functions `add_rat`, `sub_rat`, and so on do not have to be modified at all.

Constraining the dependence on the representation to a few interface functions helps us design programs as well as modify them, because it allows us to maintain the flexibility to consider alternate implementations. To continue with our simple example, suppose we are designing a rational-number package and we can't decide initially whether to perform the `gcd` at construction time or at selection time. The data-abstraction methodology gives us a way to defer that decision without losing the ability to make progress on the rest of the system.

Exercise 2.2

Consider the problem of representing line segments in a plane. Each segment is represented as a pair of points: a starting point and an ending point. Declare a constructor `make_segment` and selectors `start_segment` and `end_segment` that define the representation of segments in terms of points. Furthermore, a point can be represented as a pair of numbers: the x coordinate and the y coordinate. Accordingly, specify a constructor `make_point` and selectors `x_point` and `y_point` that define this representation. Finally, using your selectors and constructors, declare a function `midpoint_segment` that takes a line segment as argument

and returns its midpoint (the point whose coordinates are the average of the coordinates of the endpoints). To try your functions, you'll need a way to print points:

```
function print_point(p) {
    return display("(" + stringify(x_point(p)) + ", "
                       + stringify(y_point(p)) +        ")");
}
```

Exercise 2.3

Implement a representation for rectangles in a plane. (Hint: You may want to make use of exercise 2.2.) In terms of your constructors and selectors, create functions that compute the perimeter and the area of a given rectangle. Now implement a different representation for rectangles. Can you design your system with suitable abstraction barriers, so that the same perimeter and area functions will work using either representation?

2.1.3 What Is Meant by Data?

We began the rational-number implementation in section 2.1.1 by implementing the rational-number operations `add_rat`, `sub_rat`, and so on in terms of three unspecified functions: `make_rat`, `numer`, and `denom`. At that point, we could think of the operations as being defined in terms of data objects—numerators, denominators, and rational numbers—whose behavior was specified by the latter three functions.

But exactly what is meant by *data*? It is not enough to say "whatever is implemented by the given selectors and constructors." Clearly, not every arbitrary set of three functions can serve as an appropriate basis for the rational-number implementation. We need to guarantee that, if we construct a rational number `x` from a pair of integers `n` and `d`, then extracting the `numer` and the `denom` of `x` and dividing them should yield the same result as dividing `n` by `d`. In other words, `make_rat`, `numer`, and `denom` must satisfy the condition that, for any integer `n` and any nonzero integer `d`, if `x` is `make_rat(n, d)`, then

$$\frac{\texttt{numer(x)}}{\texttt{denom(x)}} = \frac{\texttt{n}}{\texttt{d}}$$

In fact, this is the only condition `make_rat`, `numer`, and `denom` must fulfill in order to form a suitable basis for a rational-number representation. In general, we can think of data as defined by some collection of selectors and constructors, together with specified conditions that these functions must fulfill in order to be a valid representation.[5]

5. Surprisingly, this idea is very difficult to formulate rigorously. There are two approaches to giving such a formulation. One, pioneered by C. A. R. Hoare (1972), is known as the method of *abstract models*. It formalizes the "functions plus conditions" specification as outlined in the rational-number example above. Note that the condition on the rational-number representation was stated in terms of facts about integers (equality and division). In general, abstract models define new kinds of data objects in terms of previously defined types of data objects. Assertions about data objects can therefore be checked by reducing them to assertions about previously defined data objects. Another approach, introduced by Zilles at MIT, by Goguen, Thatcher,

This point of view can serve to define not only "high-level" data objects, such as rational numbers, but lower-level objects as well. Consider the notion of a pair, which we used in order to define our rational numbers. We never actually said what a pair was, only that the language supplied functions `pair`, `head`, and `tail` for operating on pairs. But the only thing we need to know about these three operations is that if we glue two objects together using `pair` we can retrieve the objects using `head` and `tail`. That is, the operations satisfy the condition that, for any objects `x` and `y`, if `z` is `pair(x, y)` then `head(z)` is `x` and `tail(z)` is `y`. Indeed, we mentioned that these three functions are included as primitives in our language. However, any triple of functions that satisfies the above condition can be used as the basis for implementing pairs. This point is illustrated strikingly by the fact that we could implement `pair`, `head`, and `tail` without using any data structures at all but only using functions. Here are the definitions:[6]

```
function pair(x, y) {
    function dispatch(m) {
        return m === 0
               ? x
               : m === 1
               ? y
               : error(m, "argument not 0 or 1 -- pair");
    }
    return dispatch;
}
function head(z) { return z(0); }
function tail(z) { return z(1); }
```

This use of functions corresponds to nothing like our intuitive notion of what data should be. Nevertheless, all we need to do to show that this is a valid way to represent pairs is to verify that these functions satisfy the condition given above.

The subtle point to notice is that the value returned by `pair(x, y)` is a function—namely the internally defined function `dispatch`, which takes one argument and returns either `x` or `y` depending on whether the argument is 0 or 1. Correspondingly, `head(z)` is defined to apply `z` to 0. Hence, if `z` is the function formed by `pair(x, y)`, then `z` applied to 0 will yield `x`. Thus, we have shown that `head(pair(x, y))` yields `x`, as desired. Similarly, `tail(pair(x, y))` applies the function returned by `pair(x, y)` to 1, which returns `y`. Therefore, this functional implementation of pairs is a valid implementation, and if we access pairs using only

---

Wagner, and Wright at IBM (see Thatcher, Wagner, and Wright 1978), and by Guttag at Toronto (see Guttag 1977), is called *algebraic specification*. It regards the "functions" as elements of an abstract algebraic system whose behavior is specified by axioms that correspond to our "conditions," and uses the techniques of abstract algebra to check assertions about data objects. Both methods are surveyed in the paper by Liskov and Zilles (1975).

6. The function `error` introduced in section 1.3.3 takes as optional second argument a string that gets displayed before the first argument—for example, if `m` is 42:

```
Error in line 7: argument not 0 or 1 -- pair: 42
```

`pair`, `head`, and `tail` we cannot distinguish this implementation from one that uses "real" data structures.

The point of exhibiting the functional representation of pairs is not that our language works this way (an efficient implementation of pairs might use JavaScript's native *vector* data structure) but that it could work this way. The functional representation, although obscure, is a perfectly adequate way to represent pairs, since it fulfills the only conditions that pairs need to fulfill. This example also demonstrates that the ability to manipulate functions as objects automatically provides the ability to represent compound data. This may seem a curiosity now, but functional representations of data will play a central role in our programming repertoire. This style of programming is often called *message passing*, and we will be using it as a basic tool in chapter 3 when we address the issues of modeling and simulation.

Exercise 2.4

Here is an alternative functional representation of pairs. For this representation, verify that `head(pair(x, y))` yields `x` for any objects `x` and `y`.

```
function pair(x, y) {
    return m => m(x, y);
}
function head(z) {
    return z((p, q) => p);
}
```

What is the corresponding definition of `tail`? (Hint: To verify that this works, make use of the substitution model of section 1.1.5.)

Exercise 2.5

Show that we can represent pairs of nonnegative integers using only numbers and arithmetic operations if we represent the pair a and b as the integer that is the product $2^a 3^b$. Give the corresponding definitions of the functions `pair`, `head`, and `tail`.

Exercise 2.6

In case representing pairs as functions (exercise 2.4) wasn't mind-boggling enough, consider that, in a language that can manipulate functions, we can get by without numbers (at least insofar as nonnegative integers are concerned) by implementing 0 and the operation of adding 1 as

```
const zero = f => x => x;

function add_1(n) {
    return f => x => f(n(f)(x));
}
```

This representation is known as *Church numerals*, after its inventor, Alonzo Church, the logician who invented the λ calculus.

Define `one` and `two` directly (not in terms of `zero` and `add_1`). (Hint: Use substitution to evaluate `add_1(zero)`). Give a direct definition of the addition function `plus` (not in terms of repeated application of `add_1`).

2.1.4 Extended Exercise: Interval Arithmetic

Alyssa P. Hacker is designing a system to help people solve engineering problems. One feature she wants to provide in her system is the ability to manipulate inexact quantities (such as measured parameters of physical devices) with known precision, so that when computations are done with such approximate quantities the results will be numbers of known precision.

Electrical engineers will be using Alyssa's system to compute electrical quantities. It is sometimes necessary for them to compute the value of a parallel equivalent resistance R_p of two resistors R_1 and R_2 using the formula

$$R_p = \frac{1}{1/R_1 + 1/R_2}$$

Resistance values are usually known only up to some tolerance guaranteed by the manufacturer of the resistor. For example, if you buy a resistor labeled "6.8 ohms with 10% tolerance" you can only be sure that the resistor has a resistance between $6.8 - 0.68 = 6.12$ and $6.8 + 0.68 = 7.48$ ohms. Thus, if you have a 6.8-ohm 10% resistor in parallel with a 4.7-ohm 5% resistor, the resistance of the combination can range from about 2.58 ohms (if the two resistors are at the lower bounds) to about 2.97 ohms (if the two resistors are at the upper bounds).

Alyssa's idea is to implement "interval arithmetic" as a set of arithmetic operations for combining "intervals" (objects that represent the range of possible values of an inexact quantity). The result of adding, subtracting, multiplying, or dividing two intervals is itself an interval, representing the range of the result.

Alyssa postulates the existence of an abstract object called an "interval" that has two endpoints: a lower bound and an upper bound. She also presumes that, given the endpoints of an interval, she can construct the interval using the data constructor `make_interval`. Alyssa first writes a function for adding two intervals. She reasons that the minimum value the sum could be is the sum of the two lower bounds and the maximum value it could be is the sum of the two upper bounds:

```
function add_interval(x, y) {
    return make_interval(lower_bound(x) + lower_bound(y),
                         upper_bound(x) + upper_bound(y));
}
```

Alyssa also works out the product of two intervals by finding the minimum and the maximum of the products of the bounds and using them as the bounds of the resulting interval. (The functions `math_min` and `math_max` are primitives that find the minimum or maximum of any number of arguments.)

```
function mul_interval(x, y) {
    const p1 = lower_bound(x) * lower_bound(y);
    const p2 = lower_bound(x) * upper_bound(y);
    const p3 = upper_bound(x) * lower_bound(y);
    const p4 = upper_bound(x) * upper_bound(y);
    return make_interval(math_min(p1, p2, p3, p4),
                         math_max(p1, p2, p3, p4));
}
```

To divide two intervals, Alyssa multiplies the first by the reciprocal of the second. Note that the bounds of the reciprocal interval are the reciprocal of the upper bound and the reciprocal of the lower bound, in that order.

```
function div_interval(x, y) {
    return mul_interval(x, make_interval(1 / upper_bound(y),
                                         1 / lower_bound(y)));
}
```

Exercise 2.7

Alyssa's program is incomplete because she has not specified the implementation of the interval abstraction. Here is a definition of the interval constructor:

```
function make_interval(x, y) { return pair(x, y); }
```

Define selectors `upper_bound` and `lower_bound` to complete the implementation.

Exercise 2.8

Using reasoning analogous to Alyssa's, describe how the difference of two intervals may be computed. Define a corresponding subtraction function, called `sub_interval`.

Exercise 2.9

The *width* of an interval is half of the difference between its upper and lower bounds. The width is a measure of the uncertainty of the number specified by the interval. For some arithmetic operations the width of the result of combining two intervals is a function only of the widths of the argument intervals, whereas for others the width of the combination is not a function of the widths of the argument intervals. Show that the width of the sum (or difference) of two intervals is a function only of the widths of the intervals being added (or subtracted). Give examples to show that this is not true for multiplication or division.

Exercise 2.10

Ben Bitdiddle, an expert systems programmer, looks over Alyssa's shoulder and comments that it is not clear what it means to divide by an interval that spans zero. Modify Alyssa's program to check for this condition and to signal an error if it occurs.

Exercise 2.11

In passing, Ben also cryptically comments: "By testing the signs of the endpoints of the intervals, it is possible to break `mul_interval` into nine cases, only one of which requires more than two multiplications." Rewrite this function using Ben's suggestion.

After debugging her program, Alyssa shows it to a potential user, who complains that her program solves the wrong problem. He wants a program that can deal with numbers represented as a center value and an additive tolerance; for example, he wants to work with intervals such as 3.5 ± 0.15 rather than $[3.35, 3.65]$. Alyssa

returns to her desk and fixes this problem by supplying an alternate constructor and alternate selectors:

```
function make_center_width(c, w) {
    return make_interval(c - w, c + w);
}
function center(i) {
    return (lower_bound(i) + upper_bound(i)) / 2;
}
function width(i) {
    return (upper_bound(i) - lower_bound(i)) / 2;
}
```

Unfortunately, most of Alyssa's users are engineers. Real engineering situations usually involve measurements with only a small uncertainty, measured as the ratio of the width of the interval to the midpoint of the interval. Engineers usually specify percentage tolerances on the parameters of devices, as in the resistor specifications given earlier.

Exercise 2.12

Define a constructor `make_center_percent` that takes a center and a percentage tolerance and produces the desired interval. You must also define a selector `percent` that produces the percentage tolerance for a given interval. The `center` selector is the same as the one shown above.

Exercise 2.13

Show that under the assumption of small percentage tolerances there is a simple formula for the approximate percentage tolerance of the product of two intervals in terms of the tolerances of the factors. You may simplify the problem by assuming that all numbers are positive.

After considerable work, Alyssa P. Hacker delivers her finished system. Several years later, after she has forgotten all about it, she gets a frenzied call from an irate user, Lem E. Tweakit. It seems that Lem has noticed that the formula for parallel resistors can be written in two algebraically equivalent ways:

$$\frac{R_1 R_2}{R_1 + R_2}$$

and

$$\frac{1}{1/R_1 + 1/R_2}$$

He has written the following two programs, each of which computes the parallel-resistors formula differently:

```
function par1(r1, r2) {
    return div_interval(mul_interval(r1, r2),
                        add_interval(r1, r2));
}
function par2(r1, r2) {
    const one = make_interval(1, 1);
    return div_interval(one,
                        add_interval(div_interval(one, r1),
                                     div_interval(one, r2)));
}
```

Lem complains that Alyssa's program gives different answers for the two ways of computing. This is a serious complaint.

Exercise 2.14

Demonstrate that Lem is right. Investigate the behavior of the system on a variety of arithmetic expressions. Make some intervals A and B, and use them in computing the expressions A/A and A/B. You will get the most insight by using intervals whose width is a small percentage of the center value. Examine the results of the computation in center-percent form (see exercise 2.12).

Exercise 2.15

Eva Lu Ator, another user, has also noticed the different intervals computed by different but algebraically equivalent expressions. She says that a formula to compute with intervals using Alyssa's system will produce tighter error bounds if it can be written in such a form that no name that represents an uncertain number is repeated. Thus, she says, `par2` is a "better" program for parallel resistances than `par1`. Is she right? Why?

Exercise 2.16

Explain, in general, why equivalent algebraic expressions may lead to different answers. Can you devise an interval-arithmetic package that does not have this shortcoming, or is this task impossible? (Warning: This problem is very difficult.)

2.2 Hierarchical Data and the Closure Property

As we have seen, pairs provide a primitive "glue" that we can use to construct compound data objects. Figure 2.2 shows a standard way to visualize a pair—in this case, the pair formed by `pair(1, 2)`. In this representation, which is called *box-and-pointer notation*, each compound object is shown as a *pointer* to a box. The box for a pair has two parts, the left part containing the head of the pair and the right part containing the tail.

We have already seen that `pair` can be used to combine not only numbers but pairs as well. (You made use of this fact, or should have, in doing exercises 2.2 and 2.3.) As a consequence, pairs provide a universal building block from which we can construct all sorts of data structures. Figure 2.3 shows two ways to use pairs to combine the numbers 1, 2, 3, and 4.

The ability to create pairs whose elements are pairs is the essence of list structure's importance as a representational tool. We refer to this ability as the *closure property* of `pair`. In general, an operation for combining data objects satisfies the

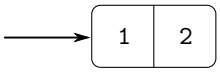


Figure 2.2 Box-and-pointer representation of `pair(1, 2)`.

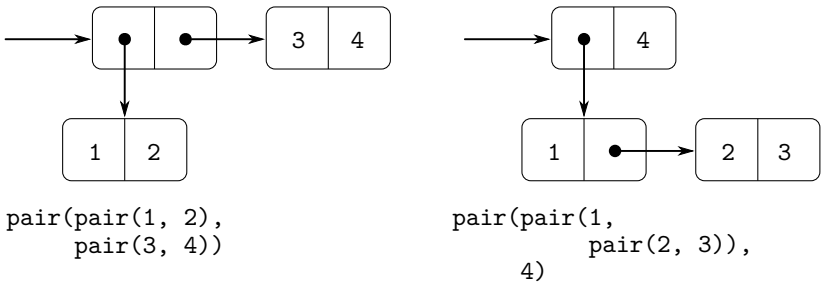


Figure 2.3 Two ways to combine 1, 2, 3, and 4 using pairs.

closure property if the results of combining things with that operation can themselves be combined using the same operation.[7] Closure is the key to power in any means of combination because it permits us to create *hierarchical* structures—structures made up of parts, which themselves are made up of parts, and so on.

From the outset of chapter 1, we've made essential use of closure in dealing with functions, because all but the very simplest programs rely on the fact that the elements of a combination can themselves be combinations. In this section, we take up the consequences of closure for compound data. We describe some conventional techniques for using pairs to represent sequences and trees, and we exhibit a graphics language that illustrates closure in a vivid way.

2.2.1 Representing Sequences

One of the useful structures we can build with pairs is a *sequence*—an ordered collection of data objects. There are, of course, many ways to represent sequences in terms of pairs. One particularly straightforward representation is illustrated in figure 2.4, where the sequence 1, 2, 3, 4 is represented as a chain of pairs. The `head` of each pair is the corresponding item in the chain, and the `tail` of the pair is the next pair in the chain. The `tail` of the final pair signals the end of the sequence, represented in box-and-pointer diagrams as a diagonal line and in programs as JavaScript's primitive value `null`. The entire sequence is constructed by nested `pair` operations:

7. The use of the word "closure" here comes from abstract algebra, where a set of elements is said to be closed under an operation if applying the operation to elements in the set produces an element that is again an element of the set. The programming languages community also (unfortunately) uses the word "closure" to describe a totally unrelated concept: A closure is an implementation technique for representing functions with free names. We do not use the word "closure" in this second sense in this book.

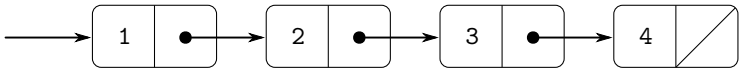


Figure 2.4 The sequence 1, 2, 3, 4 represented as a chain of pairs.

```
pair(1,
     pair(2,
          pair(3,
               pair(4, null))));
```

Such a sequence of pairs, formed by nested `pair` applications, is called a *list*, and our JavaScript environment provides a primitive called `list` to help in constructing lists.[8] The above sequence could be produced by `list(1, 2, 3, 4)`. In general,

`list(`a_1`, `a_2`, ..., `a_n`)`

is equivalent to

`pair(`a_1`, pair(`a_2`, pair(...,  pair(`a_n`, null)...)))`

Our interpreter prints pairs using a textual representation of box-and-pointer diagrams that we call *box notation*. The result of `pair(1, 2)` is printed as `[1, 2]`, and the data object in figure 2.4 is printed as `[1, [2, [3, [4, null]]]]`:

```
const one_through_four = list(1, 2, 3, 4);

one_through_four;
[1, [2, [3, [4, null]]]]
```

We can think of `head` as selecting the first item in the list, and of `tail` as selecting the sublist consisting of all but the first item. Nested applications of `head` and `tail` can be used to extract the second, third, and subsequent items in the list. The constructor `pair` makes a list like the original one, but with an additional item at the beginning.

```
head(one_through_four);
1

tail(one_through_four);
[2, [3, [4, null]]]

head(tail(one_through_four));
2

pair(10, one_through_four);
[10, [1, [2, [3, [4, null]]]]]
```

8. In this book, we use *list* to mean a chain of pairs terminated by the end-of-list marker. In contrast, the term *list structure* refers to any data structure made out of pairs, not just to lists.

```
pair(5, one_through_four);
[5, [1, [2, [3, [4, null]]]]]
```

The value `null`, used to terminate the chain of pairs, can be thought of as a sequence of no elements, the *empty list*.[9]

Box notation is sometimes difficult to read. In this book, when we want to indicate the list nature of a data structure, we will employ the alternative *list notation*: Whenever possible, list notation uses applications of `list` whose evaluation would result in the desired structure. For example, instead of the box notation

```
[1, [[2, 3], [[4, [5, null]], [6, null]]]]
```

we write

```
list(1, [2, 3], list(4, 5), 6)
```

in list notation.[10]

List operations

The use of pairs to represent sequences of elements as lists is accompanied by conventional programming techniques for manipulating lists by successively using `tail` to walk down the lists. For example, the function `list_ref` takes as arguments a list and a number n and returns the nth item of the list. It is customary to number the elements of the list beginning with 0. The method for computing `list_ref` is the following:

- For $n = 0$, `list_ref` should return the `head` of the list.
- Otherwise, `list_ref` should return the $(n-1)$st item of the `tail` of the list.

```
function list_ref(items, n) {
    return n === 0
           ? head(items)
           : list_ref(tail(items), n - 1);
}

const squares = list(1, 4, 9, 16, 25);

list_ref(squares, 3);
16
```

Often we walk down the whole list. To aid in this, our JavaScript environment includes a primitive predicate `is_null`, which tests whether its argument is the empty list. The function `length`, which returns the number of items in a list, illustrates this typical pattern of use:

---

9. The value `null` is used in JavaScript for various purposes, but in this book we shall only use it to represent the empty list.

10. Our JavaScript environment provides a primitive function `display_list` that works like the primitive function `display`, except that it uses list notation instead of box notation.

```
function length(items) {
    return is_null(items)
           ? 0
           : 1 + length(tail(items));
}

const odds = list(1, 3, 5, 7);

length(odds);
4
```

The `length` function implements a simple recursive plan. The reduction step is:

- The `length` of any list is 1 plus the `length` of the `tail` of the list.

This is applied successively until we reach the base case:

- The `length` of the empty list is 0.

We could also compute `length` in an iterative style:

```
function length(items) {
    function length_iter(a, count) {
        return is_null(a)
               ? count
               : length_iter(tail(a), count + 1);
    }
    return length_iter(items, 0);
}
```

Another conventional programming technique is to construct an answer list by adjoining elements to the front of the list with `pair` while walking down a list using `tail`, as in the function `append`, which takes two lists as arguments and combines their elements to make a new list:

```
append(squares, odds);
list(1, 4, 9, 16, 25, 1, 3, 5, 7)

append(odds, squares);
list(1, 3, 5, 7, 1, 4, 9, 16, 25)
```

The function `append` is also implemented using a recursive plan. To `append` lists `list1` and `list2`, do the following:

- If `list1` is the empty list, then the result is just `list2`.
- Otherwise, `append` the `tail` of `list1` and `list2`, and adjoin the `head` of `list1` to the result:

```
function append(list1, list2) {
    return is_null(list1)
           ? list2
           : pair(head(list1), append(tail(list1), list2));
}
```

Exercise 2.17

Define a function `last_pair` that returns the list that contains only the last element of a given (nonempty) list:

```
last_pair(list(23, 72, 149, 34));
list(34)
```

Exercise 2.18

Define a function `reverse` that takes a list as argument and returns a list of the same elements in reverse order:

```
reverse(list(1, 4, 9, 16, 25));
list(25, 16, 9, 4, 1)
```

Exercise 2.19

Consider the change-counting program of section 1.2.2. It would be nice to be able to easily change the currency used by the program, so that we could compute the number of ways to change a British pound, for example. As the program is written, the knowledge of the currency is distributed partly into the function `first_denomination` and partly into the function `count_change` (which knows that there are five kinds of U.S. coins). It would be nicer to be able to supply a list of coins to be used for making change.

We want to rewrite the function `cc` so that its second argument is a list of the values of the coins to use rather than an integer specifying which coins to use. We could then have lists that defined each kind of currency:

```
const us_coins = list(50, 25, 10, 5, 1);
const uk_coins = list(100, 50, 20, 10, 5, 2, 1);
```

We could then call `cc` as follows:

```
cc(100, us_coins);
292
```

To do this will require changing the program `cc` somewhat. It will still have the same form, but it will access its second argument differently, as follows:

```
function cc(amount, coin_values) {
    return amount === 0
           ? 1
           : amount < 0 || no_more(coin_values)
           ? 0
           : cc(amount, except_first_denomination(coin_values)) +
             cc(amount - first_denomination(coin_values), coin_values);
}
```

Define the functions `first_denomination`, `except_first_denomination`, and `no_more` in terms of primitive operations on list structures. Does the order of the list `coin_values` affect the answer produced by `cc`? Why or why not?

Exercise 2.20

In the presence of higher-order functions, it is not strictly necessary for functions to have multiple parameters; one would suffice. If we have a function such as `plus` that

naturally requires two arguments, we could write a variant of the function to which we pass the arguments one at a time. An application of the variant to the first argument could return a function that we can then apply to the second argument, and so on. This practice—called *currying* and named after the American mathematician and logician Haskell Brooks Curry—is quite common in programming languages such as Haskell and OCaml. In JavaScript, a curried version of `plus` looks as follows.

```
function plus_curried(x) {
    return y => x + y;
}
```

Write a function `brooks` that takes a curried function as first argument and as second argument a list of arguments to which the curried function is then applied, one by one, in the given order. For example, the following application of `brooks` should have the same effect as `plus_curried(3)(4)`:

```
brooks(plus_curried, list(3, 4));
7
```

While we are at it, we might as well curry the function `brooks`! Write a function `brooks_curried` that can be applied as follows:

```
brooks_curried(list(plus_curried, 3, 4));
7
```

With this function `brooks_curried`, what are the results of evaluating the following two statements?

```
brooks_curried(list(brooks_curried,
                    list(plus_curried, 3, 4)));

brooks_curried(list(brooks_curried,
                    list(brooks_curried,
                         list(plus_curried, 3, 4))));
```

Mapping over lists

One extremely useful operation is to apply some transformation to each element in a list and generate the list of results. For instance, the following function scales each number in a list by a given factor:

```
function scale_list(items, factor) {
    return is_null(items)
           ? null
           : pair(head(items) * factor,
                  scale_list(tail(items), factor));
}

scale_list(list(1, 2, 3, 4, 5), 10);
[10, [20, [30, [40, [50, null]]]]]
```

We can abstract this general idea and capture it as a common pattern expressed as a higher-order function, just as in section 1.3. The higher-order function here is called `map`. The function `map` takes as arguments a function of one argument and

a list, and returns a list of the results produced by applying the function to each element in the list:

```
function map(fun, items) {
    return is_null(items)
           ? null
           : pair(fun(head(items)),
                  map(fun, tail(items)));
}

map(abs, list(-10, 2.5, -11.6, 17));
[10, [2.5, [11.6, [17, null]]]]

map(x => x * x, list(1, 2, 3, 4));
[1, [4, [9, [16, null]]]]
```

Now we can give a new definition of `scale_list` in terms of `map`:

```
function scale_list(items, factor) {
    return map(x => x * factor, items);
}
```

The function `map` is an important construct, not only because it captures a common pattern, but because it establishes a higher level of abstraction in dealing with lists. In the original definition of `scale_list`, the recursive structure of the program draws attention to the element-by-element processing of the list. Defining `scale_list` in terms of `map` suppresses that level of detail and emphasizes that scaling transforms a list of elements to a list of results. The difference between the two definitions is not that the computer is performing a different process (it isn't) but that we think about the process differently. In effect, `map` helps establish an abstraction barrier that isolates the implementation of functions that transform lists from the details of how the elements of the list are extracted and combined. Like the barriers shown in figure 2.1, this abstraction gives us the flexibility to change the low-level details of how sequences are implemented, while preserving the conceptual framework of operations that transform sequences to sequences. Section 2.2.3 expands on this use of sequences as a framework for organizing programs.

Exercise 2.21

The function `square_list` takes a list of numbers as argument and returns a list of the squares of those numbers.

```
square_list(list(1, 2, 3, 4));
[1, [4, [9, [16, null]]]]
```

Here are two different definitions of `square_list`. Complete both of them by filling in the missing expressions:

```
function square_list(items) {
    return is_null(items)
           ? null
           : pair(⟨??⟩, ⟨??⟩);
}
```

```
function square_list(items) {
   return map(⟨??⟩, ⟨??⟩);
}
```

Exercise 2.22

Louis Reasoner tries to rewrite the first `square_list` function of exercise 2.21 so that it evolves an iterative process:

```
function square_list(items) {
    function iter(things, answer) {
        return is_null(things)
               ? answer
               : iter(tail(things),
                      pair(square(head(things)),
                           answer));
    }
    return iter(items, null);
}
```

Unfortunately, defining `square_list` this way produces the answer list in the reverse order of the one desired. Why?

Louis then tries to fix his bug by interchanging the arguments to `pair`:

```
function square_list(items) {
    function iter(things, answer) {
        return is_null(things)
               ? answer
               : iter(tail(things),
                      pair(answer,
                           square(head(things))));
    }
    return iter(items, null);
}
```

This doesn't work either. Explain.

Exercise 2.23

The function `for_each` is similar to `map`. It takes as arguments a function and a list of elements. However, rather than forming a list of the results, `for_each` just applies the function to each of the elements in turn, from left to right. The values returned by applying the function to the elements are not used at all—`for_each` is used with functions that perform an action, such as printing. For example,

```
for_each(x => display(x), list(57, 321, 88));
57
321
88
```

The value returned by the call to `for_each` (not illustrated above) can be something arbitrary, such as true. Give an implementation of `for_each`.

2.2.2 Hierarchical Structures

The representation of sequences in terms of lists generalizes naturally to represent sequences whose elements may themselves be sequences. For example, we can regard the object `[[1, [2, null]], [3, [4, null]]]` constructed by

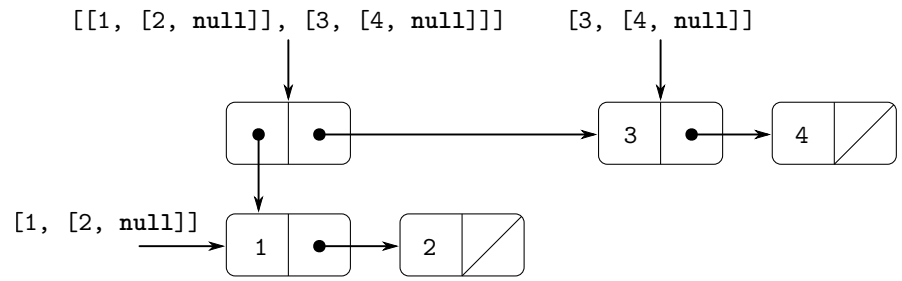


Figure 2.5 Structure formed by `pair(list(1, 2), list(3, 4))`.

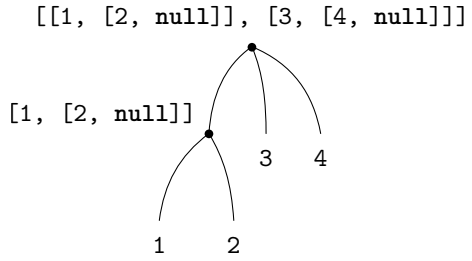


Figure 2.6 The list structure in figure 2.5 viewed as a tree.

```
pair(list(1, 2), list(3, 4));
```

as a list of three items, the first of which is itself a list, `[1, [2, null]]`. Figure 2.5 shows the representation of this structure in terms of pairs.

Another way to think of sequences whose elements are sequences is as *trees*. The elements of the sequence are the branches of the tree, and elements that are themselves sequences are subtrees. Figure 2.6 shows the structure in figure 2.5 viewed as a tree.

Recursion is a natural tool for dealing with tree structures, since we can often reduce operations on trees to operations on their branches, which reduce in turn to operations on the branches of the branches, and so on, until we reach the leaves of the tree. As an example, compare the `length` function of section 2.2.1 with the `count_leaves` function, which returns the total number of leaves of a tree:

```
const x = pair(list(1, 2), list(3, 4));

length(x);
3

count_leaves(x);
4

list(x, x);
list(list(list(1, 2), 3, 4), list(list(1, 2), 3, 4))
```

```
length(list(x, x));
2

count_leaves(list(x, x));
8
```

To implement `count_leaves`, recall the recursive plan for computing `length`:

- The `length` of a list `x` is 1 plus the `length` of the `tail` of `x`.
- The `length` of the empty list is 0.

The function `count_leaves` is similar. The value for the empty list is the same:

- `count_leaves` of the empty list is 0.

But in the reduction step, where we strip off the `head` of the list, we must take into account that the `head` may itself be a tree whose leaves we need to count. Thus, the appropriate reduction step is

- `count_leaves` of a tree `x` is `count_leaves` of the `head` of `x` plus `count_leaves` of the `tail` of `x`.

Finally, by taking `head`s we reach actual leaves, so we need another base case:

- `count_leaves` of a leaf is 1.

To aid in writing recursive functions on trees, our JavaScript environment provides the primitive predicate `is_pair`, which tests whether its argument is a pair. Here is the complete function:[11]

```
function count_leaves(x) {
    return is_null(x)
           ? 0
           : ! is_pair(x)
           ? 1
           : count_leaves(head(x)) + count_leaves(tail(x));
}
```

Exercise 2.24

Suppose we evaluate the expression `list(1, list(2, list(3, 4)))`. Give the result printed by the interpreter, the corresponding box-and-pointer structure, and the interpretation of this as a tree (as in figure 2.6).

11. The order of the two predicates matters, since `null` satisfies `is_null` and also is not a pair.

Exercise 2.25

Give combinations of `heads` and `tails` that will pick 7 from each of the following lists, given in list notation:

```
list(1, 3, list(5, 7), 9)

list(list(7))

list(1, list(2, list(3, list(4, list(5, list(6, 7))))))
```

Exercise 2.26

Suppose we define `x` and `y` to be two lists:

```
const x = list(1, 2, 3);
const y = list(4, 5, 6);
```

What is the result of evaluating each of the following expressions, in box notation and list notation?

```
append(x, y)

pair(x, y)

list(x, y)
```

Exercise 2.27

Modify your `reverse` function of exercise 2.18 to produce a `deep_reverse` function that takes a list as argument and returns as its value the list with its elements reversed and with all sublists deep-reversed as well. For example,

```
const x = list(list(1, 2), list(3, 4));

x;
list(list(1, 2), list(3, 4))

reverse(x);
list(list(3, 4), list(1, 2))

deep_reverse(x);
list(list(4, 3), list(2, 1))
```

Exercise 2.28

Write a function `fringe` that takes as argument a tree (represented as a list) and returns a list whose elements are all the leaves of the tree arranged in left-to-right order. For example,

```
const x = list(list(1, 2), list(3, 4));

fringe(x);
list(1, 2, 3, 4)

fringe(list(x, x));
list(1, 2, 3, 4, 1, 2, 3, 4)
```

Exercise 2.29

A binary mobile consists of two branches, a left branch and a right branch. Each branch is a rod of a certain length, from which hangs either a weight or another binary mobile. We can represent a binary mobile using compound data by constructing it from two branches (for example, using `list`):

```
function make_mobile(left, right) {
    return list(left, right);
}
```

A branch is constructed from a `length` (which must be a number) together with a `structure`, which may be either a number (representing a simple weight) or another mobile:

```
function make_branch(length, structure) {
    return list(length, structure);
}
```

a. Write the corresponding selectors `left_branch` and `right_branch`, which return the branches of a mobile, and `branch_length` and `branch_structure`, which return the components of a branch.

b. Using your selectors, define a function `total_weight` that returns the total weight of a mobile.

c. A mobile is said to be *balanced* if the torque applied by its top-left branch is equal to that applied by its top-right branch (that is, if the length of the left rod multiplied by the weight hanging from that rod is equal to the corresponding product for the right side) and if each of the submobiles hanging off its branches is balanced. Design a predicate that tests whether a binary mobile is balanced.

d. Suppose we change the representation of mobiles so that the constructors are

```
function make_mobile(left, right) {
    return pair(left, right);
}
function make_branch(length, structure) {
    return pair(length, structure);
}
```

How much do you need to change your programs to convert to the new representation?

Mapping over trees

Just as `map` is a powerful abstraction for dealing with sequences, `map` together with recursion is a powerful abstraction for dealing with trees. For instance, the `scale_tree` function, analogous to `scale_list` of section 2.2.1, takes as arguments a numeric factor and a tree whose leaves are numbers. It returns a tree of the same shape, where each number is multiplied by the factor. The recursive plan for `scale_tree` is similar to the one for `count_leaves`:

```
function scale_tree(tree, factor) {
    return is_null(tree)
           ? null
           : ! is_pair(tree)
           ? tree * factor
           : pair(scale_tree(head(tree), factor),
                  scale_tree(tail(tree), factor));
}

scale_tree(list(1, list(2, list(3, 4), 5), list(6, 7)),
           10);
list(10, list(20, list(30, 40), 50), list(60, 70))
```

Another way to implement `scale_tree` is to regard the tree as a sequence of sub-trees and use `map`. We map over the sequence, scaling each sub-tree in turn, and return the list of results. In the base case, where the tree is a leaf, we simply multiply by the factor:

```
function scale_tree(tree, factor) {
    return map(sub_tree => is_pair(sub_tree)
                           ? scale_tree(sub_tree, factor)
                           : sub_tree * factor,
               tree);
}
```

Many tree operations can be implemented by similar combinations of sequence operations and recursion.

Exercise 2.30

Declare a function `square_tree` analogous to the `square_list` function of exercise 2.21. That is, `square_tree` should behave as follows:

```
square_tree(list(1,
                 list(2, list(3, 4), 5),
                 list(6, 7)));
list(1, list(4, list(9, 16), 25), list(36, 49))
```

Declare `square_tree` both directly (i.e., without using any higher-order functions) and also by using `map` and recursion.

Exercise 2.31

Abstract your answer to exercise 2.30 to produce a function `tree_map` with the property that `square_tree` could be declared as

```
function square_tree(tree) { return tree_map(square, tree); }
```

Exercise 2.32

We can represent a set as a list of distinct elements, and we can represent the set of all subsets of the set as a list of lists. For example, if the set is `list(1, 2, 3)`, then the set of all subsets is

```
list(null, list(3), list(2), list(2, 3),
     list(1), list(1, 3), list(1, 2),
     list(1, 2, 3))
```

Complete the following declaration of a function that generates the set of subsets of a set and give a clear explanation of why it works:

```
function subsets(s) {
    if (is_null(s)) {
        return list(null);
    } else {
        const rest = subsets(tail(s));
        return append(rest, map(⟨??⟩, rest));
    }
}
```

2.2.3 Sequences as Conventional Interfaces

In working with compound data, we've stressed how data abstraction permits us to design programs without becoming enmeshed in the details of data representations, and how abstraction preserves for us the flexibility to experiment with alternative representations. In this section, we introduce another powerful design principle for working with data structures—the use of *conventional interfaces*.

In section 1.3 we saw how program abstractions, implemented as higher-order functions, can capture common patterns in programs that deal with numerical data. Our ability to formulate analogous operations for working with compound data depends crucially on the style in which we manipulate our data structures. Consider, for example, the following function, analogous to the `count_leaves` function of section 2.2.2, which takes a tree as argument and computes the sum of the squares of the leaves that are odd:

```
function sum_odd_squares(tree) {
    return is_null(tree)
           ? 0
           : ! is_pair(tree)
           ? is_odd(tree) ? square(tree) : 0
           : sum_odd_squares(head(tree)) +
             sum_odd_squares(tail(tree));
}
```

On the surface, this function is very different from the following one, which constructs a list of all the even Fibonacci numbers Fib(k), where k is less than or equal to a given integer n:

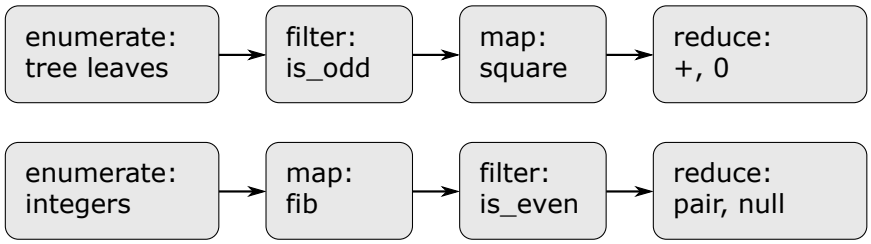


Figure 2.7 The signal-flow plans for the functions `sum_odd_squares` (top) and `even_fibs` (bottom) reveal the commonality between the two programs.

```
function even_fibs(n) {
    function next(k) {
        if (k > n) {
            return null;
        } else {
            const f = fib(k);
            return is_even(f)
                   ? pair(f, next(k + 1))
                   : next(k + 1);
        }
    }
    return next(0);
}
```

Despite the fact that these two functions are structurally very different, a more abstract description of the two computations reveals a great deal of similarity. The first program

- enumerates the leaves of a tree;
- filters them, selecting the odd ones;
- squares each of the selected ones; and
- accumulates the results using +, starting with 0.

The second program

- enumerates the integers from 0 to n;
- computes the Fibonacci number for each integer;
- filters them, selecting the even ones; and
- accumulates the results using `pair`, starting with the empty list.

A signal-processing engineer would find it natural to conceptualize these processes in terms of signals flowing through a cascade of stages, each of which implements part of the program plan, as shown in figure 2.7. In `sum_odd_squares`, we begin with an *enumerator*, which generates a "signal" consisting of the leaves of

a given tree. This signal is passed through a *filter*, which eliminates all but the odd elements. The resulting signal is in turn passed through a *map*, which is a "transducer" that applies the `square` function to each element. The output of the map is then fed to an *accumulator*, which combines the elements using `+`, starting from an initial 0. The plan for `even_fibs` is analogous.

Unfortunately, the two function declarations above fail to exhibit this signal-flow structure. For instance, if we examine the `sum_odd_squares` function, we find that the enumeration is implemented partly by the `is_null` and `is_pair` tests and partly by the tree-recursive structure of the function. Similarly, the accumulation is found partly in the tests and partly in the addition used in the recursion. In general, there are no distinct parts of either function that correspond to the elements in the signal-flow description. Our two functions decompose the computations in a different way, spreading the enumeration over the program and mingling it with the map, the filter, and the accumulation. If we could organize our programs to make the signal-flow structure manifest in the functions we write, this would increase the conceptual clarity of the resulting program.

Sequence Operations

The key to organizing programs so as to more clearly reflect the signal-flow structure is to concentrate on the "signals" that flow from one stage in the process to the next. If we represent these signals as lists, then we can use list operations to implement the processing at each of the stages. For instance, we can implement the mapping stages of the signal-flow diagrams using the `map` function from section 2.2.1:

```
map(square, list(1, 2, 3, 4, 5));
list(1, 4, 9, 16, 25)
```

Filtering a sequence to select only those elements that satisfy a given predicate is accomplished by

```
function filter(predicate, sequence) {
    return is_null(sequence)
           ? null
           : predicate(head(sequence))
           ? pair(head(sequence),
                  filter(predicate, tail(sequence)))
           : filter(predicate, tail(sequence));
}
```

For example,

```
filter(is_odd, list(1, 2, 3, 4, 5));
list(1, 3, 5)
```

Accumulations can be implemented by

```
function accumulate(op, initial, sequence) {
    return is_null(sequence)
           ? initial
           : op(head(sequence),
                accumulate(op, initial, tail(sequence)));
}
```

```
accumulate(plus, 0, list(1, 2, 3, 4, 5));
15

accumulate(times, 1, list(1, 2, 3, 4, 5));
120

accumulate(pair, null, list(1, 2, 3, 4, 5));
list(1, 2, 3, 4, 5)
```

All that remains to implement signal-flow diagrams is to enumerate the sequence of elements to be processed. For `even_fibs`, we need to generate the sequence of integers in a given range, which we can do as follows:

```
function enumerate_interval(low, high) {
    return low > high
           ? null
           : pair(low,
                  enumerate_interval(low + 1, high));
}

enumerate_interval(2, 7);
list(2, 3, 4, 5, 6, 7)
```

To enumerate the leaves of a tree, we can use[12]

```
function enumerate_tree(tree) {
    return is_null(tree)
           ? null
           : ! is_pair(tree)
           ? list(tree)
           : append(enumerate_tree(head(tree)),
                    enumerate_tree(tail(tree)));
}

enumerate_tree(list(1, list(2, list(3, 4)), 5));
list(1, 2, 3, 4, 5)
```

Now we can reformulate `sum_odd_squares` and `even_fibs` as in the signal-flow diagrams. For `sum_odd_squares`, we enumerate the sequence of leaves of the tree, filter this to keep only the odd numbers in the sequence, square each element, and sum the results:

```
function sum_odd_squares(tree) {
    return accumulate(plus,
                      0,
                      map(square,
                          filter(is_odd,
                                 enumerate_tree(tree))));
}
```

12. This is, in fact, precisely the `fringe` function from exercise 2.28. Here we've renamed it to emphasize that it is part of a family of general sequence-manipulation functions.

For `even_fibs`, we enumerate the integers from 0 to n, generate the Fibonacci number for each of these integers, filter the resulting sequence to keep only the even elements, and accumulate the results into a list:

```
function even_fibs(n) {
    return accumulate(pair,
                      null,
                      filter(is_even,
                             map(fib,
                                 enumerate_interval(0, n))));
}
```

The value of expressing programs as sequence operations is that this helps us make program designs that are modular, that is, designs that are constructed by combining relatively independent pieces. We can encourage modular design by providing a library of standard components together with a conventional interface for connecting the components in flexible ways.

Modular construction is a powerful strategy for controlling complexity in engineering design. In real signal-processing applications, for example, designers regularly build systems by cascading elements selected from standardized families of filters and transducers. Similarly, sequence operations provide a library of standard program elements that we can mix and match. For instance, we can reuse pieces from the `sum_odd_squares` and `even_fibs` functions in a program that constructs a list of the squares of the first $n+1$ Fibonacci numbers:

```
function list_fib_squares(n) {
    return accumulate(pair,
                      null,
                      map(square,
                          map(fib,
                              enumerate_interval(0, n))));
}

list_fib_squares(10);
```

list(0, 1, 1, 4, 9, 25, 64, 169, 441, 1156, 3025)

We can rearrange the pieces and use them in computing the product of the squares of the odd integers in a sequence:

```
function product_of_squares_of_odd_elements(sequence) {
    return accumulate(times,
                      1,
                      map(square,
                          filter(is_odd, sequence)));
}

product_of_squares_of_odd_elements(list(1, 2, 3, 4, 5));
```

225

We can also formulate conventional data-processing applications in terms of sequence operations. Suppose we have a sequence of personnel records and we want to find the salary of the highest-paid programmer. Assume that we have a selector

salary that returns the salary of a record, and a predicate is_programmer that tests if a record is for a programmer. Then we can write

```
function salary_of_highest_paid_programmer(records) {
    return accumulate(math_max,
                      0,
                      map(salary,
                          filter(is_programmer, records)));
}
```

These examples give just a hint of the vast range of operations that can be expressed as sequence operations.[13]

Sequences, implemented here as lists, serve as a conventional interface that permits us to combine processing modules. Additionally, when we uniformly represent structures as sequences, we have localized the data-structure dependencies in our programs to a small number of sequence operations. By changing these, we can experiment with alternative representations of sequences, while leaving the overall design of our programs intact. We will exploit this capability in section 3.5, when we generalize the sequence-processing paradigm to admit infinite sequences.

Exercise 2.33

Fill in the missing expressions to complete the following definitions of some basic list-manipulation operations as accumulations:

```
function map(f, sequence) {
    return accumulate((x, y) => ⟨??⟩,
                      null, sequence);
}
function append(seq1, seq2) {
    return accumulate(pair, ⟨??⟩, ⟨??⟩);
}
function length(sequence) {
    return accumulate(⟨??⟩, 0, sequence);
}
```

Exercise 2.34

Evaluating a polynomial in x at a given value of x can be formulated as an accumulation. We evaluate the polynomial

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

using a well-known algorithm called *Horner's rule*, which structures the computation as

$$(\cdots (a_n x + a_{n-1}) x + \cdots + a_1)\, x + a_0$$

---

13. Richard Waters (1979) developed a program that automatically analyzes traditional Fortran programs, viewing them in terms of maps, filters, and accumulations. He found that fully 90 percent of the code in the Fortran Scientific Subroutine Package fits neatly into this paradigm. One of the reasons for the success of Lisp as a programming language is that lists provide a standard medium for expressing ordered collections so that they can be manipulated using higher-order operations. Many modern languages, such as Python, have learned this lesson.

In other words, we start with a_n, multiply by x, add a_{n-1}, multiply by x, and so on, until we reach a_0.[14] Fill in the following template to produce a function that evaluates a polynomial using Horner's rule. Assume that the coefficients of the polynomial are arranged in a sequence, from a_0 through a_n.

```
function horner_eval(x, coefficient_sequence) {
    return accumulate((this_coeff, higher_terms) => ⟨??⟩,
                      0,
                      coefficient_sequence);
}
```

For example, to compute $1 + 3x + 5x^3 + x^5$ at $x = 2$ you would evaluate

```
horner_eval(2, list(1, 3, 0, 5, 0, 1));
```

Exercise 2.35

Redefine `count_leaves` from section 2.2.2 as an accumulation:

```
function count_leaves(t) {
    return accumulate(⟨??⟩, ⟨??⟩, map(⟨??⟩, ⟨??⟩));
}
```

Exercise 2.36

The function `accumulate_n` is similar to `accumulate` except that it takes as its third argument a sequence of sequences, which are all assumed to have the same number of elements. It applies the designated accumulation function to combine all the first elements of the sequences, all the second elements of the sequences, and so on, and returns a sequence of the results. For instance, if `s` is a sequence containing four sequences

```
list(list(1, 2, 3), list(4, 5, 6), list(7, 8, 9), list(10, 11, 12))
```

then the value of `accumulate_n(plus, 0, s)` should be the sequence `list(22, 26, 30)`. Fill in the missing expressions in the following definition of `accumulate_n`:

```
function accumulate_n(op, init, seqs) {
    return is_null(head(seqs))
           ? null
           : pair(accumulate(op, init, ⟨??⟩),
                  accumulate_n(op, init, ⟨??⟩));
}
```

---

14. According to Knuth (1997b), this rule was formulated by W. G. Horner early in the nineteenth century, but the method was actually used by Newton over a hundred years earlier. Horner's rule evaluates the polynomial using fewer additions and multiplications than does the straightforward method of first computing $a_n x^n$, then adding $a_{n-1}x^{n-1}$, and so on. In fact, it is possible to prove that any algorithm for evaluating arbitrary polynomials must use at least as many additions and multiplications as does Horner's rule, and thus Horner's rule is an optimal algorithm for polynomial evaluation. This was proved (for the number of additions) by A. M. Ostrowski in a 1954 paper that essentially founded the modern study of optimal algorithms. The analogous statement for multiplications was proved by V. Y. Pan in 1966. The book by Borodin and Munro (1975) provides an overview of these and other results about optimal algorithms.

Exercise 2.37

Suppose we represent vectors $v = (v_i)$ as sequences of numbers, and matrices $m = (m_{ij})$ as sequences of vectors (the rows of the matrix). For example, the matrix

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 4 & 5 & 6 & 6 \\ 6 & 7 & 8 & 9 \end{bmatrix}$$

is represented as the following sequence:

```
list(list(1, 2, 3, 4),
     list(4, 5, 6, 6),
     list(6, 7, 8, 9))
```

With this representation, we can use sequence operations to concisely express the basic matrix and vector operations. These operations (which are described in any book on matrix algebra) are the following:

`dot_product(`*v*`, `*w*`)`	returns the sum $\sum_i v_i w_i$;
`matrix_times_vector(`*m*`, `*v*`)`	returns the vector t, where $t_i = \sum_j m_{ij} v_j$;
`matrix_times_matrix(`*m*`, `*n*`)`	returns the matrix p, where $p_{ij} = \sum_k m_{ik} n_{kj}$;
`transpose(`*m*`)`	returns the matrix n, where $n_{ij} = m_{ji}$.

We can define the dot product as[15]

```
function dot_product(v, w) {
    return accumulate(plus, 0, accumulate_n(times, 1, list(v, w)));
}
```

Fill in the missing expressions in the following functions for computing the other matrix operations. (The function `accumulate_n` is declared in exercise 2.36.)

```
function matrix_times_vector(m, v) {
    return map(⟨??⟩, m);
}
function transpose(mat) {
    return accumulate_n(⟨??⟩, ⟨??⟩, mat);
}
function matrix_times_matrix(m, n) {
    const cols = transpose(n);
    return map(⟨??⟩, m);
}
```

Exercise 2.38

The `accumulate` function is also known as `fold_right`, because it combines the first element of the sequence with the result of combining all the elements to the right. There is also a `fold_left`, which is similar to `fold_right`, except that it combines elements working in the opposite direction:

15. This definition uses the function `accumulate_n` from exercise 2.36.

```
function fold_left(op, initial, sequence) {
    function iter(result, rest) {
        return is_null(rest)
               ? result
               : iter(op(result, head(rest)),
                      tail(rest));
    }
    return iter(initial, sequence);
}
```

What are the values of

```
fold_right(divide, 1, list(1, 2, 3));

fold_left(divide, 1, list(1, 2, 3));

fold_right(list, null, list(1, 2, 3));

fold_left(list, null, list(1, 2, 3));
```

Give a property that `op` should satisfy to guarantee that `fold_right` and `fold_left` will produce the same values for any sequence.

Exercise 2.39

Complete the following definitions of `reverse` (exercise 2.18) in terms of `fold_right` and `fold_left` from exercise 2.38:

```
function reverse(sequence) {
    return fold_right((x, y) => ⟨??⟩, null, sequence);
}

function reverse(sequence) {
    return fold_left((x, y) => ⟨??⟩, null, sequence);
}
```

Nested Mappings

We can extend the sequence paradigm to include many computations that are commonly expressed using nested loops.[16] Consider this problem: Given a positive integer n, find all ordered pairs of distinct positive integers i and j, where $1 \leq j < i \leq n$, such that $i + j$ is prime. For example, if n is 6, then the pairs are the following:

i	2	3	4	4	5	6	6
j	1	2	1	3	2	1	5
$i+j$	3	5	5	7	7	7	11

A natural way to organize this computation is to generate the sequence of all ordered pairs of positive integers less than or equal to n, filter to select those pairs whose sum

16. This approach to nested mappings was shown to us by David Turner, whose languages KRC and Miranda provide elegant formalisms for dealing with these constructs. The examples in this section (see also exercise 2.42) are adapted from Turner 1981. In section 3.5.3, we'll see how this approach generalizes to infinite sequences.

is prime, and then, for each pair (i, j) that passes through the filter, produce the triple $(i, j, i+j)$.

Here is a way to generate the sequence of pairs: For each integer $i \leq n$, enumerate the integers $j < i$, and for each such i and j generate the pair (i, j). In terms of sequence operations, we map along the sequence `enumerate_interval(1, n)`. For each i in this sequence, we map along the sequence `enumerate_interval(1, i - 1)`. For each j in this latter sequence, we generate the pair `list(i, j)`. This gives us a sequence of pairs for each i. Combining all the sequences for all the i (by accumulating with `append`) produces the required sequence of pairs:[17]

```
accumulate(append,
           null,
           map(i => map(j => list(i, j),
                        enumerate_interval(1, i - 1)),
               enumerate_interval(1, n)));
```

The combination of mapping and accumulating with `append` is so common in this sort of program that we will isolate it as a separate function:

```
function flatmap(f, seq) {
    return accumulate(append, null, map(f, seq));
}
```

Now filter this sequence of pairs to find those whose sum is prime. The filter predicate is called for each element of the sequence; its argument is a pair and it must extract the integers from the pair. Thus, the predicate to apply to each element in the sequence is

```
function is_prime_sum(pair) {
    return is_prime(head(pair) + head(tail(pair)));
}
```

Finally, generate the sequence of results by mapping over the filtered pairs using the following function, which constructs a triple consisting of the two elements of the pair along with their sum:

```
function make_pair_sum(pair) {
    return list(head(pair), head(tail(pair)),
                head(pair) + head(tail(pair)));
}
```

Combining all these steps yields the complete function:

```
function prime_sum_pairs(n) {
    return map(make_pair_sum,
               filter(is_prime_sum,
                      flatmap(i => map(j => list(i, j),
                                       enumerate_interval(1, i - 1)),
                              enumerate_interval(1, n))));
}
```

17. We're representing a pair here as a list of two elements rather than as an ordinary pair. Thus, the "pair" (i, j) is represented as `list(i, j)`, not `pair(i, j)`.

Nested mappings are also useful for sequences other than those that enumerate intervals. Suppose we wish to generate all the permutations of a set S; that is, all the ways of ordering the items in the set. For instance, the permutations of $\{1, 2, 3\}$ are $\{1, 2, 3\}$, $\{1, 3, 2\}$, $\{2, 1, 3\}$, $\{2, 3, 1\}$, $\{3, 1, 2\}$, and $\{3, 2, 1\}$. Here is a plan for generating the permutations of S: For each item x in S, recursively generate the sequence of permutations of $S - x$,[18] and adjoin x to the front of each one. This yields, for each x in S, the sequence of permutations of S that begin with x. Combining these sequences for all x gives all the permutations of S:[19]

```
function permutations(s) {
    return is_null(s)              // empty set?
           ? list(null)            // sequence containing empty set
           : flatmap(x => map(p => pair(x, p),
                              permutations(remove(x, s))),
                     s);
}
```

Notice how this strategy reduces the problem of generating permutations of S to the problem of generating the permutations of sets with fewer elements than S. In the terminal case, we work our way down to the empty list, which represents a set of no elements. For this, we generate `list(null)`, which is a sequence with one item, namely the set with no elements. The `remove` function used in `permutations` returns all the items in a given sequence except for a given item. This can be expressed as a simple filter:

```
function remove(item, sequence) {
    return filter(x => ! (x === item),
                  sequence);
}
```

Exercise 2.40

Write a function `unique_pairs` that, given an integer n, generates the sequence of pairs (i, j) with $1 \leq j < i \leq n$. Use `unique_pairs` to simplify the definition of `prime_sum_pairs` given above.

Exercise 2.41

Write a function to find all ordered triples of distinct positive integers i, j, and k less than or equal to a given integer n that sum to a given integer s.

Exercise 2.42

The "eight-queens puzzle" asks how to place eight queens on a chessboard so that no queen is in check from any other (i.e., no two queens are in the same row, column, or diagonal). One possible solution is shown in figure 2.8. One way to solve the puzzle is to work across the board, placing a queen in each column. Once we have placed $k - 1$ queens, we must

---

18. The set $S - x$ is the set of all elements of S, excluding x.

19. The character sequence // in JavaScript programs is used to introduce *comments*. Everything from // to the end of the line is ignored by the interpreter. In this book we don't use many comments; we try to make our programs self-documenting by using descriptive names.

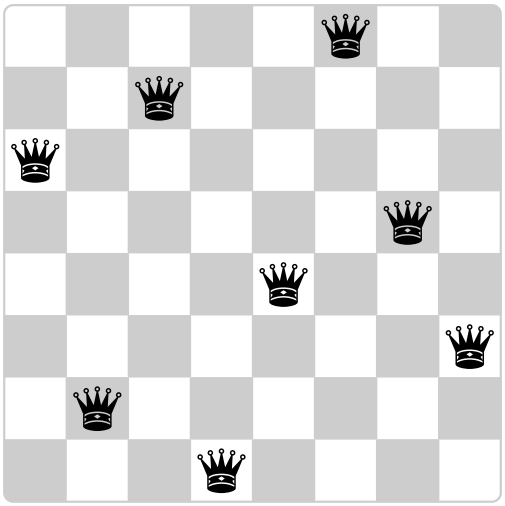


Figure 2.8 A solution to the eight-queens puzzle.

place the kth queen in a position where it does not check any of the queens already on the board. We can formulate this approach recursively: Assume that we have already generated the sequence of all possible ways to place $k-1$ queens in the first $k-1$ columns of the board. For each of these ways, generate an extended set of positions by placing a queen in each row of the kth column. Now filter these, keeping only the positions for which the queen in the kth column is safe with respect to the other queens. This produces the sequence of all ways to place k queens in the first k columns. By continuing this process, we will produce not only one solution, but all solutions to the puzzle.

We implement this solution as a function `queens`, which returns a sequence of all solutions to the problem of placing n queens on an $n \times n$ chessboard. The function `queens` has an internal function `queens_cols` that returns the sequence of all ways to place queens in the first k columns of the board.

```
function queens(board_size) {
    function queen_cols(k) {
        return k === 0
               ? list(empty_board)
               : filter(positions => is_safe(k, positions),
                        flatmap(rest_of_queens =>
                                  map(new_row =>
                                        adjoin_position(new_row, k,
                                                        rest_of_queens),
                                      enumerate_interval(1, board_size)),
                                queen_cols(k - 1)));
    }
    return queen_cols(board_size);
}
```

In this function `rest_of_queens` is a way to place $k-1$ queens in the first $k-1$ columns, and `new_row` is a proposed row in which to place the queen for the kth column. Complete the program by implementing the representation for sets of board positions, including the function `adjoin_position`, which adjoins a new row-column position to a set of positions, and `empty_board`, which represents an empty set of positions. You must also write the

function `is_safe`, which determines for a set of positions whether the queen in the *k*th column is safe with respect to the others. (Note that we need only check whether the new queen is safe—the other queens are already guaranteed safe with respect to each other.)

Exercise 2.43

Louis Reasoner is having a terrible time doing exercise 2.42. His `queens` function seems to work, but it runs extremely slowly. (Louis never does manage to wait long enough for it to solve even the 6×6 case.) When Louis asks Eva Lu Ator for help, she points out that he has interchanged the order of the nested mappings in the `flatmap`, writing it as

```
flatmap(new_row =>
          map(rest_of_queens =>
                adjoin_position(new_row, k, rest_of_queens),
              queen_cols(k - 1)),
        enumerate_interval(1, board_size));
```

Explain why this interchange makes the program run slowly. Estimate how long it will take Louis's program to solve the eight-queens puzzle, assuming that the program in exercise 2.42 solves the puzzle in time *T*.

2.2.4 Example: A Picture Language

This section presents a simple language for drawing pictures that illustrates the power of data abstraction and closure, and also exploits higher-order functions in an essential way. The language is designed to make it easy to experiment with patterns such as the ones in figure 2.9, which are composed of repeated elements that are shifted and scaled.[20] In this language, the data objects being combined are represented as functions rather than as list structure. Just as `pair`, which satisfies the closure property, allowed us to easily build arbitrarily complicated list structure, the operations in this language, which also satisfy the closure property, allow us to easily build arbitrarily complicated patterns.

The picture language

When we began our study of programming in section 1.1, we emphasized the importance of describing a language by focusing on the language's primitives, its means of combination, and its means of abstraction. We'll follow that framework here.

Part of the elegance of this picture language is that there is only one kind of element, called a *painter*. A painter draws an image that is shifted and scaled to fit within a designated parallelogram-shaped frame. For example, there's a primitive painter we'll call `wave` that makes a crude line drawing, as shown in figure 2.10. The

20. The picture language is based on the language Peter Henderson created to construct images like M.C. Escher's "Square Limit" woodcut (see Henderson 1982). The woodcut incorporates a repeated scaled pattern, similar to the arrangements drawn using the `square_limit` function in this section.

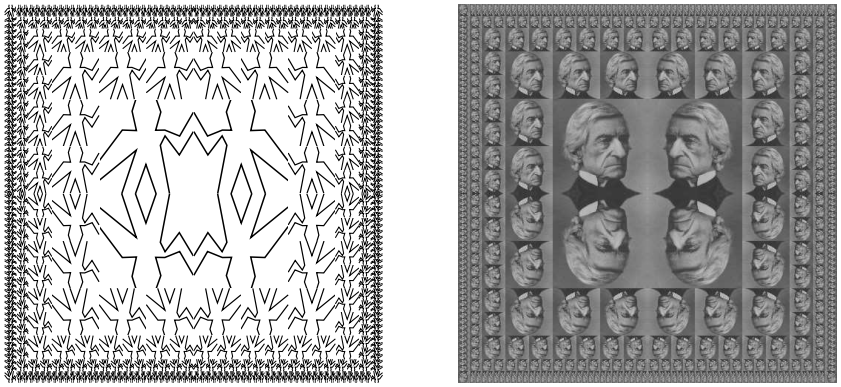


Figure 2.9 Designs generated with the picture language.

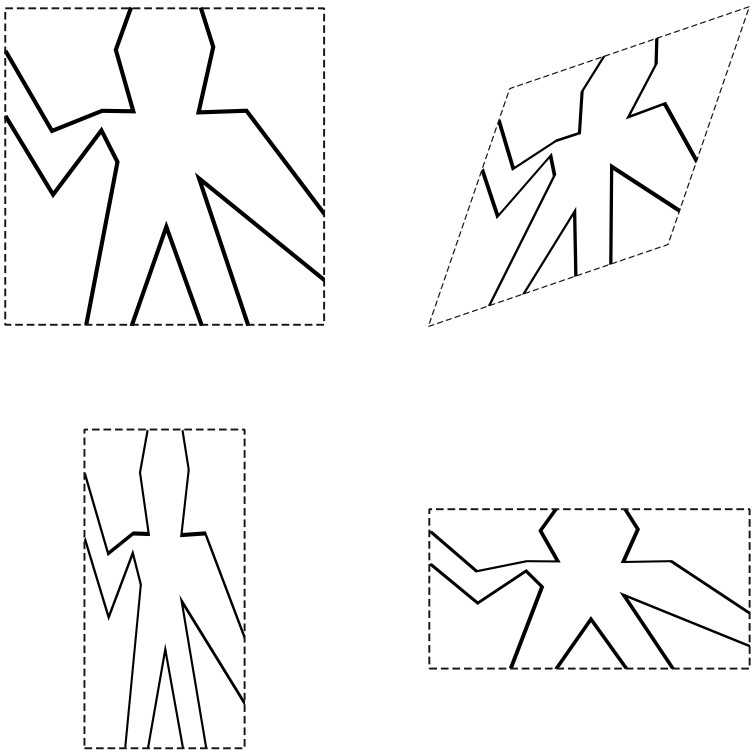


Figure 2.10 Images produced by the `wave` painter, with respect to four different frames. The frames, shown with dashed lines, are not part of the images.

actual shape of the drawing depends on the frame—all four images in figure 2.10 are produced by the same `wave` painter, but with respect to four different frames. Painters can be more elaborate than this: The primitive painter called `rogers` paints

a picture of MIT's founder, William Barton Rogers, as shown in figure 2.11.[21] The four images in figure 2.11 are drawn with respect to the same four frames as the `wave` images in figure 2.10.

To combine images, we use various operations that construct new painters from given painters. For example, the `beside` operation takes two painters and produces a new, compound painter that draws the first painter's image in the left half of the frame and the second painter's image in the right half of the frame. Similarly, `below` takes two painters and produces a compound painter that draws the first painter's image below the second painter's image. Some operations transform a single painter to produce a new painter. For example, `flip_vert` takes a painter and produces a painter that draws its image upside-down, and `flip_horiz` produces a painter that draws the original painter's image left-to-right reversed.

Figure 2.12 shows the drawing of a painter called `wave4` that is built up in two stages starting from `wave`:

---

21. William Barton Rogers (1804–1882) was the founder and first president of MIT. A geologist and talented teacher, he taught at William and Mary College and at the University of Virginia. In 1859 he moved to Boston, where he had more time for research, worked on a plan for establishing a "polytechnic institute," and served as Massachusetts's first State Inspector of Gas Meters.

When MIT was established in 1861, Rogers was elected its first president. Rogers espoused an ideal of "useful learning" that was different from the university education of the time, with its overemphasis on the classics, which, as he wrote, "stand in the way of the broader, higher and more practical instruction and discipline of the natural and social sciences." This education was likewise to be different from narrow trade-school education. In Rogers's words:

> The world-enforced distinction between the practical and the scientific worker is utterly futile, and the whole experience of modern times has demonstrated its utter worthlessness.

Rogers served as president of MIT until 1870, when he resigned due to ill health. In 1878 the second president of MIT, John Runkle, resigned under the pressure of a financial crisis brought on by the Panic of 1873 and strain of fighting off attempts by Harvard to take over MIT. Rogers returned to hold the office of president until 1881.

Rogers collapsed and died while addressing MIT's graduating class at the commencement exercises of 1882. Runkle quoted Rogers's last words in a memorial address delivered that same year:

> "As I stand here today and see what the Institute is, ... I call to mind the beginnings of science. I remember one hundred and fifty years ago Stephen Hales published a pamphlet on the subject of illuminating gas, in which he stated that his researches had demonstrated that 128 grains of bituminous coal—"
>
> "Bituminous coal," these were his last words on earth. Here he bent forward, as if consulting some notes on the table before him, then slowly regaining an erect position, threw up his hands, and was translated from the scene of his earthly labors and triumphs to "the tomorrow of death," where the mysteries of life are solved, and the disembodied spirit finds unending satisfaction in contemplating the new and still unfathomable mysteries of the infinite future.

In the words of Francis A. Walker (MIT's third president):

> All his life he had borne himself most faithfully and heroically, and he died as so good a knight would surely have wished, in harness, at his post, and in the very part and act of public duty.

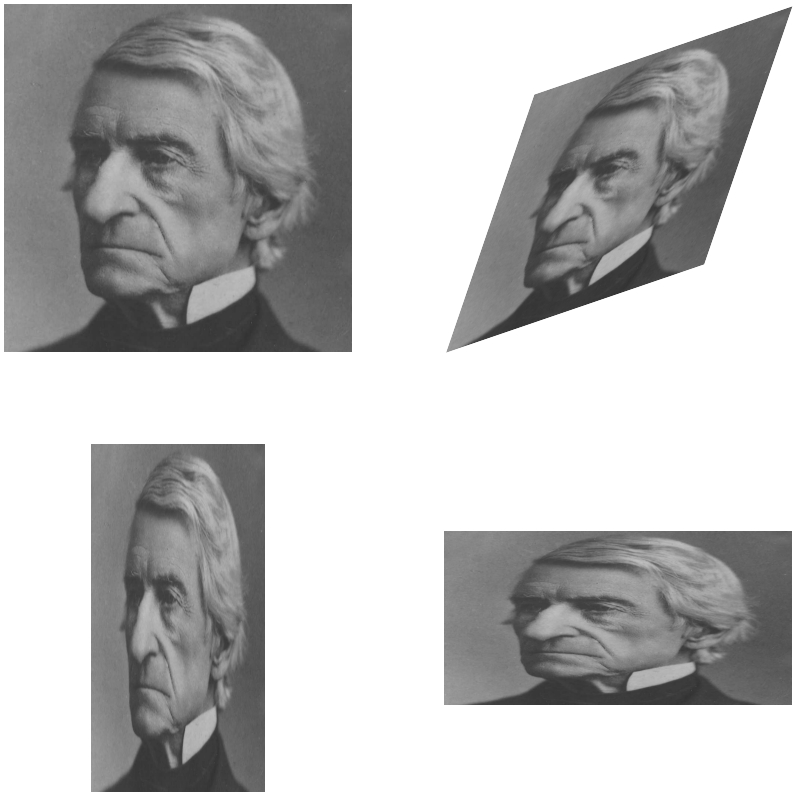


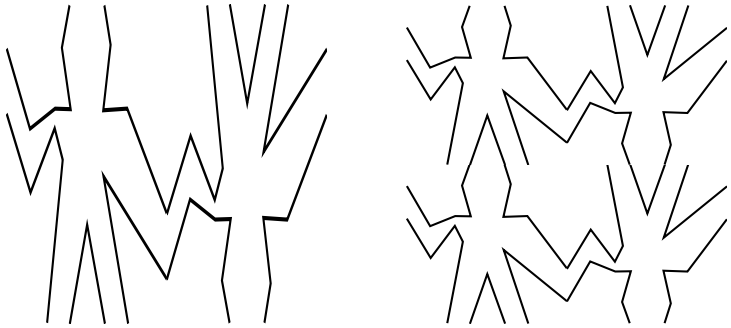
Figure 2.11 Images of William Barton Rogers, founder and first president of MIT, painted with respect to the same four frames as in figure 2.10 (original image courtesy MIT Museum).

```
const wave2 = beside(wave, flip_vert(wave));
const wave4 = below(wave2, wave2);
```

In building up a complex image in this manner we are exploiting the fact that painters are closed under the language's means of combination. The `beside` or `below` of two painters is itself a painter; therefore, we can use it as an element in making more complex painters. As with building up list structure using `pair`, the closure of our data under the means of combination is crucial to the ability to create complex structures while using only a few operations.

Once we can combine painters, we would like to be able to abstract typical patterns of combining painters. We will implement the painter operations as JavaScript functions. This means that we don't need a special abstraction mechanism in the picture language: Since the means of combination are ordinary JavaScript functions, we automatically have the capability to do anything with painter operations that we can do with functions. For example, we can abstract the pattern in `wave4` as

```
function flipped_pairs(painter) {
    const painter2 = beside(painter, flip_vert(painter));
    return below(painter2, painter2);
}
```



```
const wave2 =
    beside(wave, flip_vert(wave));
```

```
const wave4 =
    below(wave2, wave2);
```

Figure 2.12 Creating a complex figure, starting from the `wave` painter of figure 2.10.

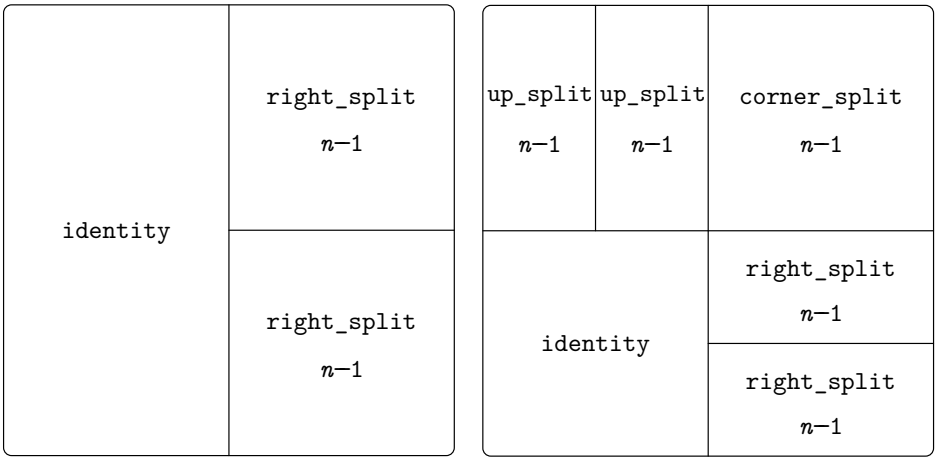


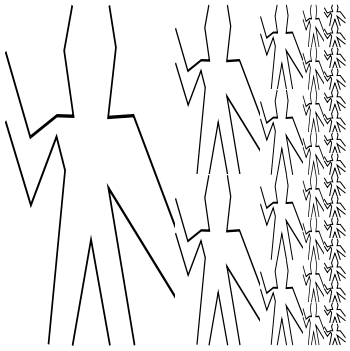
Figure 2.13 Recursive plans for `right_split` and `corner_split`.

and declare `wave4` as an instance of this pattern:

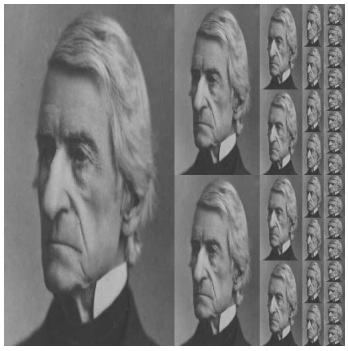
```
const wave4 = flipped_pairs(wave);
```

We can also define recursive operations. Here's one that makes painters split and branch towards the right as shown in figures 2.13 and 2.14:

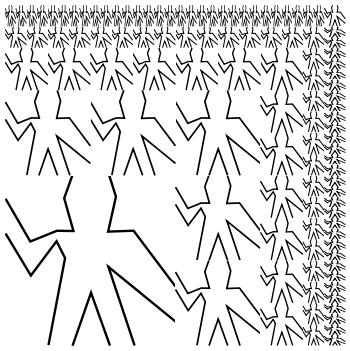
```
function right_split(painter, n) {
    if (n === 0) {
        return painter;
    } else {
        const smaller = right_split(painter, n - 1);
        return beside(painter, below(smaller, smaller));
    }
}
```



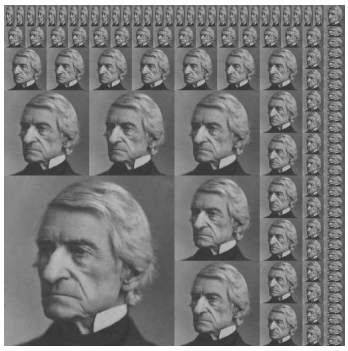
right_split(wave, 4)



right_split(rogers, 4)



corner_split(wave, 4)



corner_split(rogers, 4)

Figure 2.14 The recursive operation `right_split` applied to the painters `wave` and `rogers`. Combining four `corner_split` figures produces symmetric `square_limit` as shown in figure 2.9.

We can produce balanced patterns by branching upwards as well as towards the right (see exercise 2.44 and figures 2.13 and 2.14):

```
function corner_split(painter, n) {
    if (n === 0) {
        return painter;
    } else {
        const up = up_split(painter, n - 1);
        const right = right_split(painter, n - 1);
        const top_left = beside(up, up);
        const bottom_right = below(right, right);
        const corner = corner_split(painter, n - 1);
        return beside(below(painter, top_left),
                      below(bottom_right, corner));
    }
}
```

By placing four copies of a `corner_split` appropriately, we obtain a pattern called `square_limit`, whose application to `wave` and `rogers` is shown in figure 2.9:

```
function square_limit(painter, n) {
    const quarter = corner_split(painter, n);
    const half = beside(flip_horiz(quarter), quarter);
    return below(flip_vert(half), half);
}
```

Exercise 2.44

Declare the function `up_split` used by `corner_split`. It is similar to `right_split`, except that it switches the roles of `below` and `beside`.

Higher-order operations

In addition to abstracting patterns of combining painters, we can work at a higher level, abstracting patterns of combining painter operations. That is, we can view the painter operations as elements to manipulate and can write means of combination for these elements—functions that take painter operations as arguments and create new painter operations.

For example, `flipped_pairs` and `square_limit` each arrange four copies of a painter's image in a square pattern; they differ only in how they orient the copies. One way to abstract this pattern of painter combination is with the following function, which takes four one-argument painter operations and produces a painter operation that transforms a given painter with those four operations and arranges the results in a square.[22] The functions `tl`, `tr`, `bl`, and `br` are the transformations to apply to the top left copy, the top right copy, the bottom left copy, and the bottom right copy, respectively.

```
function square_of_four(tl, tr, bl, br) {
    return painter => {
        const top = beside(tl(painter), tr(painter));
        const bottom = beside(bl(painter), br(painter));
        return below(bottom, top);
    };
}
```

Then `flipped_pairs` can be defined in terms of `square_of_four` as follows:[23]

---

22. In `square_of_four`, we use an extension of the syntax of lambda expressions that was introduced in section 1.3.2: The body of a lambda expression can be a block, not just a return expression. Such a lambda expression has the shape `(`*parameters*`) => {` *statements* `}` or *parameter* `=> {` *statements* `}`.

23. Equivalently, we could write

```
const flipped_pairs = square_of_four(identity, flip_vert,
                                     identity, flip_vert);
```

```
function flipped_pairs(painter) {
    const combine4 = square_of_four(identity, flip_vert,
                                    identity, flip_vert);
    return combine4(painter);
}
```

and `square_limit` can be expressed as[24]

```
function square_limit(painter, n) {
    const combine4 = square_of_four(flip_horiz, identity,
                                    rotate180, flip_vert);
    return combine4(corner_split(painter, n));
}
```

Exercise 2.45

The functions `right_split` and `up_split` can be expressed as instances of a general splitting operation. Declare a function `split` with the property that evaluating

```
const right_split = split(beside, below);
const up_split = split(below, beside);
```

produces functions `right_split` and `up_split` with the same behaviors as the ones already declared.

Frames

Before we can show how to implement painters and their means of combination, we must first consider frames. A frame can be described by three vectors—an origin vector and two edge vectors. The origin vector specifies the offset of the frame's origin from some absolute origin in the plane, and the edge vectors specify the offsets of the frame's corners from its origin. If the edges are perpendicular, the frame will be rectangular. Otherwise the frame will be a more general parallelogram.

Figure 2.15 shows a frame and its associated vectors. In accordance with data abstraction, we need not be specific yet about how frames are represented, other than to say that there is a constructor `make_frame`, which takes three vectors and produces a frame, and three corresponding selectors `origin_frame`, `edge1_frame`, and `edge2_frame` (see exercise 2.47).

We will use coordinates in the unit square ($0 \leq x, y \leq 1$) to specify images. With each frame, we associate a *frame coordinate map*, which will be used to shift and scale images to fit the frame. The map transforms the unit square into the frame by mapping the vector $\mathbf{v} = (x, y)$ to the vector sum

$$\text{Origin(Frame)} + x \cdot \text{Edge}_1\text{ (Frame)} + y \cdot \text{Edge}_2\text{ (Frame)}$$

24. The function `rotate180` rotates a painter by 180 degrees. Instead of `rotate180` we could say `compose(flip_vert, flip_horiz)`, using the `compose` function from exercise 1.42.

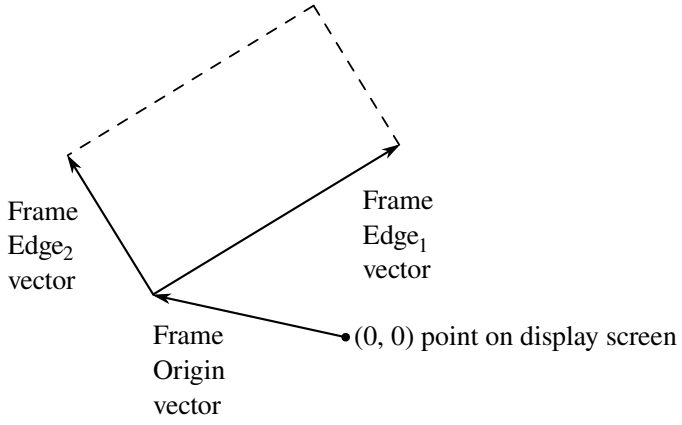


Figure 2.15 A frame is described by three vectors—an origin and two edges.

For example, $(0, 0)$ is mapped to the origin of the frame, $(1, 1)$ to the vertex diagonally opposite the origin, and $(0.5, 0.5)$ to the center of the frame. We can create a frame's coordinate map with the following function:[25]

```
function frame_coord_map(frame) {
    return v => add_vect(origin_frame(frame),
                         add_vect(scale_vect(xcor_vect(v),
                                             edge1_frame(frame)),
                                  scale_vect(ycor_vect(v),
                                             edge2_frame(frame))));
}
```

Observe that applying `frame_coord_map` to a frame returns a function that, given a vector, returns a vector. If the argument vector is in the unit square, the result vector will be in the frame. For example,

```
frame_coord_map(a_frame)(make_vect(0, 0));
```

returns the same vector as

```
origin_frame(a_frame);
```

Exercise 2.46

A two-dimensional vector v running from the origin to a point can be represented as a pair consisting of an x-coordinate and a y-coordinate. Implement a data abstraction for vectors by giving a constructor `make_vect` and corresponding selectors `xcor_vect` and `ycor_vect`. In terms of your selectors and constructor, implement functions `add_vect`,

25. The function `frame_coord_map` uses the vector operations described in exercise 2.46 below, which we assume have been implemented using some representation for vectors. Because of data abstraction, it doesn't matter what this vector representation is, so long as the vector operations behave correctly.

`sub_vect`, and `scale_vect` that perform the operations vector addition, vector subtraction, and multiplying a vector by a scalar:

$$\begin{array}{lcl}(x_1, y_1) + (x_2, y_2) & = & (x_1 + x_2, y_1 + y_2) \\ (x_1, y_1) - (x_2, y_2) & = & (x_1 - x_2, y_1 - y_2) \\ s \cdot (x, y) & = & (sx, sy)\end{array}$$

Exercise 2.47

Here are two possible constructors for frames:

```
function make_frame(origin, edge1, edge2) {
    return list(origin, edge1, edge2);
}

function make_frame(origin, edge1, edge2) {
    return pair(origin, pair(edge1, edge2));
}
```

For each constructor supply the appropriate selectors to produce an implementation for frames.

Painters

A painter is represented as a function that, given a frame as argument, draws a particular image shifted and scaled to fit the frame. That is to say, if `p` is a painter and `f` is a frame, then we produce `p`'s image in `f` by calling `p` with `f` as argument.

The details of how primitive painters are implemented depend on the particular characteristics of the graphics system and the type of image to be drawn. For instance, suppose we have a function `draw_line` that draws a line on the screen between two specified points. Then we can create painters for line drawings, such as the `wave` painter in figure 2.10, from lists of line segments as follows:[26]

```
function segments_to_painter(segment_list) {
    return frame =>
             for_each(segment =>
                        draw_line(
                            frame_coord_map(frame)
                                (start_segment(segment)),
                            frame_coord_map(frame)
                                (end_segment(segment))),
                      segment_list);
}
```

The segments are given using coordinates with respect to the unit square. For each segment in the list, the painter transforms the segment endpoints with the frame coordinate map and draws a line between the transformed points.

---

26. The function `segments_to_painter` uses the representation for line segments described in exercise 2.48 below. It also uses the `for_each` function described in exercise 2.23.

Representing painters as functions erects a powerful abstraction barrier in the picture language. We can create and intermix all sorts of primitive painters, based on a variety of graphics capabilities. The details of their implementation do not matter. Any function can serve as a painter, provided that it takes a frame as argument and draws something scaled to fit the frame.[27]

Exercise 2.48

A directed line segment in the plane can be represented as a pair of vectors—the vector running from the origin to the start-point of the segment, and the vector running from the origin to the end-point of the segment. Use your vector representation from exercise 2.46 to define a representation for segments with a constructor `make_segment` and selectors `start_segment` and `end_segment`.

Exercise 2.49

Use `segments_to_painter` to define the following primitive painters:

a. The painter that draws the outline of the designated frame.

b. The painter that draws an "X" by connecting opposite corners of the frame.

c. The painter that draws a diamond shape by connecting the midpoints of the sides of the frame.

d. The `wave` painter.

Transforming and combining painters

An operation on painters (such as `flip_vert` or `beside`) works by creating a painter that invokes the original painters with respect to frames derived from the argument frame. Thus, for example, `flip_vert` doesn't have to know how a painter works in order to flip it—it just has to know how to turn a frame upside down: The flipped painter just uses the original painter, but in the inverted frame.

Painter operations are based on the function `transform_painter`, which takes as arguments a painter and information on how to transform a frame and produces a new painter. The transformed painter, when called on a frame, transforms the frame and calls the original painter on the transformed frame. The arguments to `transform_painter` are points (represented as vectors) that specify the corners of the new frame: When mapped into the frame, the first point specifies the new frame's origin and the other two specify the ends of its edge vectors. Thus, arguments within the unit square specify a frame contained within the original frame.

---

27. For example, the `rogers` painter of figure 2.11 was constructed from a gray-level image. For each point in a given frame, the `rogers` painter determines the point in the image that is mapped to it under the frame coordinate map, and shades it accordingly. By allowing different types of painters, we are capitalizing on the abstract data idea discussed in section 2.1.3, where we argued that a rational-number representation could be anything at all that satisfies an appropriate condition. Here we're using the fact that a painter can be implemented in any way at all, so long as it draws something in the designated frame. Section 2.1.3 also showed how pairs could be implemented as functions. Painters are our second example of a functional representation for data.

```
function transform_painter(painter, origin, corner1, corner2) {
    return frame => {
             const m = frame_coord_map(frame);
             const new_origin = m(origin);
             return painter(make_frame(
                               new_origin,
                               sub_vect(m(corner1), new_origin),
                               sub_vect(m(corner2), new_origin)));
           };
}
```

Here's how to flip painter images vertically:

```
function flip_vert(painter) {
    return transform_painter(painter,
                             make_vect(0, 1),   // new origin
                             make_vect(1, 1),   // new end of edge1
                             make_vect(0, 0)); // new end of edge2
}
```

Using `transform_painter`, we can easily define new transformations. For example, we can declare a painter that shrinks its image to the upper-right quarter of the frame it is given:

```
function shrink_to_upper_right(painter) {
    return transform_painter(painter,
                             make_vect(0.5, 0.5),
                             make_vect(1, 0.5),
                             make_vect(0.5, 1));
}
```

Other transformations rotate images counterclockwise by 90 degrees[28]

```
function rotate90(painter) {
    return transform_painter(painter,
                             make_vect(1, 0),
                             make_vect(1, 1),
                             make_vect(0, 0));
}
```

or squash images towards the center of the frame:[29]

```
function squash_inwards(painter) {
    return transform_painter(painter,
                             make_vect(0, 0),
                             make_vect(0.65, 0.35),
                             make_vect(0.35, 0.65));
}
```

---

28. The function `rotate90` is a pure rotation only for square frames, because it also stretches and shrinks the image to fit into the rotated frame.

29. The diamond-shaped images in figures 2.10 and 2.11 were created with `squash_inwards` applied to `wave` and `rogers`.

Frame transformation is also the key to defining means of combining two or more painters. The `beside` function, for example, takes two painters, transforms them to paint in the left and right halves of an argument frame respectively, and produces a new, compound painter. When the compound painter is given a frame, it calls the first transformed painter to paint in the left half of the frame and calls the second transformed painter to paint in the right half of the frame:

```
function beside(painter1, painter2) {
    const split_point = make_vect(0.5, 0);
    const paint_left  = transform_painter(painter1,
                                          make_vect(0, 0),
                                          split_point,
                                          make_vect(0, 1));
    const paint_right = transform_painter(painter2,
                                          split_point,
                                          make_vect(1, 0),
                                          make_vect(0.5, 1));
    return frame => {
               paint_left(frame);
               paint_right(frame);
           };
}
```

Observe how the painter data abstraction, and in particular the representation of painters as functions, makes `beside` easy to implement. The `beside` function need not know anything about the details of the component painters other than that each painter will draw something in its designated frame.

Exercise 2.50

Declare the transformation `flip_horiz`, which flips painters horizontally, and transformations that rotate painters counterclockwise by 180 degrees and 270 degrees.

Exercise 2.51

Declare the `below` operation for painters. The function `below` takes two painters as arguments. The resulting painter, given a frame, draws with the first painter in the bottom of the frame and with the second painter in the top. Define `below` in two different ways—first by writing a function that is analogous to the `beside` function given above, and again in terms of `beside` and suitable rotation operations (from exercise 2.50).

Levels of language for robust design

The picture language exploits some of the critical ideas we've introduced about abstraction with functions and data. The fundamental data abstractions, painters, are implemented using functional representations, which enables the language to handle different basic drawing capabilities in a uniform way. The means of combination satisfy the closure property, which permits us to easily build up complex designs. Finally, all the tools for abstracting functions are available to us for abstracting means of combination for painters.

We have also obtained a glimpse of another crucial idea about languages and program design. This is the approach of *stratified design*, the notion that a complex system should be structured as a sequence of levels that are described using a sequence of languages. Each level is constructed by combining parts that are regarded as primitive at that level, and the parts constructed at each level are used as primitives at the next level. The language used at each level of a stratified design has primitives, means of combination, and means of abstraction appropriate to that level of detail.

Stratified design pervades the engineering of complex systems. For example, in computer engineering, resistors and transistors are combined (and described using a language of analog circuits) to produce parts such as and-gates and or-gates, which form the primitives of a language for digital-circuit design.[30] These parts are combined to build processors, bus structures, and memory systems, which are in turn combined to form computers, using languages appropriate to computer architecture. Computers are combined to form distributed systems, using languages appropriate for describing network interconnections, and so on.

As a tiny example of stratification, our picture language uses primitive elements (primitive painters) that specify points and lines to provide the shapes of a painter like `rogers`. The bulk of our description of the picture language focused on combining these primitives, using geometric combiners such as `beside` and `below`. We also worked at a higher level, regarding `beside` and `below` as primitives to be manipulated in a language whose operations, such as `square_of_four`, capture common patterns of combining geometric combiners.

Stratified design helps make programs *robust*, that is, it makes it likely that small changes in a specification will require correspondingly small changes in the program. For instance, suppose we wanted to change the image based on `wave` shown in figure 2.9. We could work at the lowest level to change the detailed appearance of the `wave` element; we could work at the middle level to change the way `corner_split` replicates the `wave`; we could work at the highest level to change how `square_limit` arranges the four copies of the corner. In general, each level of a stratified design provides a different vocabulary for expressing the characteristics of the system, and a different kind of ability to change it.

Exercise 2.52

Make changes to the square limit of `wave` shown in figure 2.9 by working at each of the levels described above. In particular:

a. Add some segments to the primitive `wave` painter of exercise 2.49 (to add a smile, for example).

b. Change the pattern constructed by `corner_split` (for example, by using only one copy of the `up_split` and `right_split` images instead of two).

c. Modify the version of `square_limit` that uses `square_of_four` so as to assemble the corners in a different pattern. (For example, you might make the big Mr. Rogers look outward from each corner of the square.)

30. Section 3.3.4 describes one such language.

2.3 Symbolic Data

All the compound data objects we have used so far were constructed ultimately from numbers. In this section we extend the representational capability of our language by introducing the ability to work with strings of characters as data.

2.3.1 Strings

So far, we have used strings in order to display messages, using the functions `display` and `error` (as for example in exercise 1.22). We can form compound data using strings and have lists such as

```
list("a", "b", "c", "d")
list(23, 45, 17)
list(list("Jakob", 27), list("Lova", 9), list("Luisa", 24))
```

In order to distinguish strings from names, we surround them with double quotation marks. For example, the JavaScript expression `z` denotes the value of the name `z`, whereas the JavaScript expression `"z"` denotes a string that consists of a single character, namely the last letter in the English alphabet in lower case.

Via quotation marks, we can distinguish between strings and names:

```
const a = 1;
const b = 2;

list(a, b);
[1, [2, null]]

list("a", "b");
["a", ["b", null]]

list("a", b);
["a", [2, null]]
```

In section 1.1.6, we introduced `===` and `!==` as primitive predicates on numbers. From now on, we shall allow two strings as operands of `===` and `!==`. The predicate `===` returns true if and only if the two strings are the same, and `!==` returns true if and only if the two strings are not the same.[31] Using `===`, we can implement a useful function called `member`. This takes two arguments: a string and a list of strings or a number and a list of numbers. If the first argument is not contained in the list (i.e., is not `===` to any item in the list), then `member` returns `null`. Otherwise, it returns the sublist of the list beginning with the first occurrence of the string or number:

31. We can consider two strings to be "the same" if they consist of the same characters in the same order. Such a definition skirts a deep issue that we are not yet ready to address: the meaning of "sameness" in a programming language. We will return to this in chapter 3 (section 3.1.3).

```
function member(item, x) {
    return is_null(x)
           ? null
           : item === head(x)
           ? x
           : member(item, tail(x));
}
```

For example, the value of

```
member("apple", list("pear", "banana", "prune"))
```

is `null`, whereas the value of

```
member("apple", list("x", "y", "apple", "pear"))
```

is `list("apple", "pear")`.

Exercise 2.53

What is the result of evaluating each of the following expressions, in box notation and list notation?

```
list("a", "b", "c")
```

```
list(list("george"))
```

```
tail(list(list("x1", "x2"), list("y1", "y2")))
```

```
tail(head(list(list("x1", "x2"), list("y1", "y2"))))
```

```
member("red", list("blue", "shoes", "yellow", "socks"))
```

```
member("red", list("red", "shoes", "blue", "socks"))
```

Exercise 2.54

Two lists are said to be `equal` if they contain equal elements arranged in the same order. For example,

```
equal(list("this", "is", "a", "list"), list("this", "is", "a", "list"))
```

is true, but

```
equal(list("this", "is", "a", "list"), list("this", list("is", "a"), "list"))
```

is false. To be more precise, we can define `equal` recursively in terms of the basic `===` equality of numbers and strings by saying that `a` and `b` are `equal` if they are both strings or both numbers and they are `===`, or if they are both pairs such that `head(a)` is `equal` to `head(b)` and `tail(a)` is `equal` to `tail(b)`. Using this idea, implement `equal` as a function.

Exercise 2.55

The JavaScript interpreter reads the characters after a double quotation mark `"` until it finds another double quotation mark. All characters between the two are part of the string, excluding the double quotation marks themselves. But what if we want a string to contain

double quotation marks? For this purpose, JavaScript also allows *single* quotation marks to delimit strings, as for example in `'say your name aloud'`. Within singly-quoted strings, we can use double quotation marks, and vice versa, so `'say "your name" aloud'` and `"say 'your name' aloud"` are valid strings that have different characters at positions 4 and 14, if we start counting at 0. Depending on the font in use, two single quotation marks might not be easily distinguishable from a double quotation mark. Can you spot which is which and work out the value of the following expression?

```
'"' === ""
```

2.3.2 Example: Symbolic Differentiation

As an illustration of symbol manipulation and a further illustration of data abstraction, consider the design of a function that performs symbolic differentiation of algebraic expressions. We would like the function to take as arguments an algebraic expression and a variable and to return the derivative of the expression with respect to the variable. For example, if the arguments to the function are $ax^2 + bx + c$ and x, the function should return $2ax + b$. Symbolic differentiation is of special historical significance in the programming language Lisp.[32] It was one of the motivating examples behind the development of a computer language for symbol manipulation. Furthermore, it marked the beginning of the line of research that led to the development of powerful systems for symbolic mathematical work, which are today routinely used by applied mathematicians and physicists.

In developing the symbolic-differentiation program, we will follow the same strategy of data abstraction that we followed in developing the rational-number system of section 2.1.1. That is, we will first define a differentiation algorithm that operates on abstract objects such as "sums," "products," and "variables" without worrying about how these are to be represented. Only afterward will we address the representation problem.

The differentiation program with abstract data

To keep things simple, we will consider a very simple symbolic-differentiation program that handles expressions that are built up using only the operations of addition and multiplication with two arguments. Differentiation of any such expression can be carried out by applying the following reduction rules:

$$\begin{aligned}
\frac{dc}{dx} &= 0 \quad \text{for } c \text{ a constant or a variable different from } x \\
\frac{dx}{dx} &= 1 \\
\frac{d(u+v)}{dx} &= \frac{du}{dx} + \frac{dv}{dx} \\
\frac{d(uv)}{dx} &= u\left(\frac{dv}{dx}\right) + v\left(\frac{du}{dx}\right)
\end{aligned}$$

32. The original version of this book used the programming language Scheme, a dialect of Lisp.

Observe that the latter two rules are recursive in nature. That is, to obtain the derivative of a sum we first find the derivatives of the terms and add them. Each of the terms may in turn be an expression that needs to be decomposed. Decomposing into smaller and smaller pieces will eventually produce pieces that are either constants or variables, whose derivatives will be either 0 or 1.

To embody these rules in a function we indulge in a little wishful thinking, as we did in designing the rational-number implementation. If we had a means for representing algebraic expressions, we should be able to tell whether an expression is a sum, a product, a constant, or a variable. We should be able to extract the parts of an expression. For a sum, for example, we want to be able to extract the addend (first term) and the augend (second term). We should also be able to construct expressions from parts. Let us assume that we already have functions to implement the following selectors, constructors, and predicates:

`is_variable(e)`	Is `e` a variable?
`is_same_variable(v1, v2)`	Are `v1` and `v2` the same variable?
`is_sum(e)`	Is `e` a sum?
`addend(e)`	Addend of the sum `e`.
`augend(e)`	Augend of the sum `e`.
`make_sum(a1, a2)`	Construct the sum of `a1` and `a2`.
`is_product(e)`	Is `e` a product?
`multiplier(e)`	Multiplier of the product `e`.
`multiplicand(e)`	Multiplicand of the product `e`.
`make_product(m1, m2)`	Construct the product of `m1` and `m2`.

Using these, and the primitive predicate `is_number`, which identifies numbers, we can express the differentiation rules as the following function:

```
function deriv(exp, variable) {
    return is_number(exp)
           ? 0
           : is_variable(exp)
           ? is_same_variable(exp, variable) ? 1 : 0
           : is_sum(exp)
           ? make_sum(deriv(addend(exp), variable),
                      deriv(augend(exp), variable))
           : is_product(exp)
           ? make_sum(make_product(multiplier(exp),
                                   deriv(multiplicand(exp),
                                         variable)),
                      make_product(deriv(multiplier(exp),
                                         variable),
                                   multiplicand(exp)))
           : error(exp, "unknown expression type -- deriv");
}
```

This `deriv` function incorporates the complete differentiation algorithm. Since it is expressed in terms of abstract data, it will work no matter how we choose to represent algebraic expressions, as long as we design a proper set of selectors and constructors. This is the issue we must address next.

Representing algebraic expressions

We can imagine many ways to use list structure to represent algebraic expressions. For example, we could use lists of symbols that mirror the usual algebraic notation, representing $ax+b$ as `list("a", "*", "x", "+", "b")`. However, it will be more convenient if we reflect the mathematical structure of the expression in the JavaScript value representing it; that is, to represent $ax+b$ as `list("+", list("*", "a", "x"), "b")`. Placing a binary operator in front of its operands is called *prefix notation*, in contrast with the infix notation introduced in section 1.1.1. With prefix notation, our data representation for the differentiation problem is as follows:

- The variables are just strings. They are identified by the primitive predicate `is_string`:

```
function is_variable(x) { return is_string(x); }
```

- Two variables are the same if the strings representing them are equal:

```
function is_same_variable(v1, v2) {
    return is_variable(v1) && is_variable(v2) && v1 === v2;
}
```

- Sums and products are constructed as lists:

```
function make_sum(a1, a2) { return list("+", a1, a2); }
function make_product(m1, m2) { return list("*", m1, m2); }
```

- A sum is a list whose first element is the string `"+"`:

```
function is_sum(x) {
    return is_pair(x) && head(x) === "+";
}
```

- The addend is the second item of the sum list:

```
function addend(s) { return head(tail(s)); }
```

- The augend is the third item of the sum list:

```
function augend(s) { return head(tail(tail(s))); }
```

- A product is a list whose first element is the string `"*"`:

```
function is_product(x) {
    return is_pair(x) && head(x) === "*";
}
```

- The multiplier is the second item of the product list:

```
function multiplier(s) { return head(tail(s)); }
```

- The multiplicand is the third item of the product list:

```
function multiplicand(s) { return head(tail(tail(s))); }
```

Thus, we need only combine these with the algorithm as embodied by `deriv` in order to have a working symbolic-differentiation program. Let us look at some examples of its behavior:

```
deriv(list("+", "x", 3), "x");
list("+", 1, 0)

deriv(list("*", "x", "y"), "x");
list("+", list("*", "x", 0), list("*", 1, "y"))

deriv(list("*", list("*", "x", "y"), list("+", "x", 3)), "x");
list("+", list("*", list("*", "x", "y"), list("+", 1, 0)),
          list("*", list("+", list("*", "x", 0), list("*", 1, "y")),
                    list("+", "x", 3)))
```

The program produces answers that are correct; however, they are unsimplified. It is true that

$$\frac{d(xy)}{dx} = x \cdot 0 + 1 \cdot y$$

but we would like the program to know that $x \cdot 0 = 0$, $1 \cdot y = y$, and $0 + y = y$. The answer for the second example should have been simply `y`. As the third example shows, this becomes a serious issue when the expressions are complex.

Our difficulty is much like the one we encountered with the rational-number implementation: we haven't reduced answers to simplest form. To accomplish the rational-number reduction, we needed to change only the constructors and the selectors of the implementation. We can adopt a similar strategy here. We won't change `deriv` at all. Instead, we will change `make_sum` so that if both summands are numbers, `make_sum` will add them and return their sum. Also, if one of the summands is 0, then `make_sum` will return the other summand.

```
function make_sum(a1, a2) {
    return number_equal(a1, 0)
           ? a2
           : number_equal(a2, 0)
           ? a1
           : is_number(a1) && is_number(a2)
           ? a1 + a2
           : list("+", a1, a2);
}
```

This uses the function `number_equal`, which checks whether an expression is equal to a given number:

```
function number_equal(exp, num) {
    return is_number(exp) && exp === num;
}
```

Similarly, we will change `make_product` to build in the rules that 0 times anything is 0 and 1 times anything is the thing itself:

```
function make_product(m1, m2) {
    return number_equal(m1, 0) || number_equal(m2, 0)
           ? 0
           : number_equal(m1, 1)
           ? m2
           : number_equal(m2, 1)
           ? m1
           : is_number(m1) && is_number(m2)
           ? m1 * m2
           : list("*", m1, m2);
}
```

Here is how this version works on our three examples:

```
deriv(list("+", "x", 3), "x");
```
1

```
deriv(list("*", "x", "y"), "x");
```
"y"

```
deriv(list("*", list("*", "x", "y"), list("+", "x", 3)), "x");
```
list("+", list("", "x", "y"), list("*", "y", list("+", "x", 3)))*

Although this is quite an improvement, the third example shows that there is still a long way to go before we get a program that puts expressions into a form that we might agree is "simplest." The problem of algebraic simplification is complex because, among other reasons, a form that may be simplest for one purpose may not be for another.

Exercise 2.56

Show how to extend the basic differentiator to handle more kinds of expressions. For instance, implement the differentiation rule

$$\frac{d(u^n)}{dx} = nu^{n-1}\left(\frac{du}{dx}\right)$$

by adding a new clause to the `deriv` program and defining appropriate functions `is_exp`, `base`, `exponent`, and `make_exp`. (You may use the string `"**"` to denote exponentiation.) Build in the rules that anything raised to the power 0 is 1 and anything raised to the power 1 is the thing itself.

Exercise 2.57

Extend the differentiation program to handle sums and products of arbitrary numbers of (two or more) terms. Then the last example above could be expressed as

```
deriv(list("*", "x", "y", list("+", "x", 3)), "x");
```

Try to do this by changing only the representation for sums and products, without changing the `deriv` function at all. For example, the `addend` of a sum would be the first term, and the `augend` would be the sum of the rest of the terms.

Exercise 2.58

Suppose we want to modify the differentiation program so that it works with ordinary mathematical notation, in which `"+"` and `"*"` are infix rather than prefix operators. Since the differentiation program is defined in terms of abstract data, we can modify it to work with different representations of expressions solely by changing the predicates, selectors, and constructors that define the representation of the algebraic expressions on which the differentiator is to operate.

a. Show how to do this in order to differentiate algebraic expressions presented in infix form, as in this example:

```
list("x", "+", list(3, "*", list("x", "+", list("y", "+", 2))))
```

To simplify the task, assume that `"+"` and `"*"` always take two arguments and that expressions are fully parenthesized.

b. The problem becomes substantially harder if we allow a notation closer to ordinary infix notation, which omits unnecessary parentheses and assumes that multiplication has higher precedence than addition, as in this example:

```
list("x", "+", 3, "*", list("x", "+", "y", "+", 2))
```

Can you design appropriate predicates, selectors, and constructors for this notation such that our derivative program still works?

2.3.3 Example: Representing Sets

In the previous examples we built representations for two kinds of compound data objects: rational numbers and algebraic expressions. In one of these examples we had the choice of simplifying (reducing) the expressions at either construction time or selection time, but other than that the choice of a representation for these structures in terms of lists was straightforward. When we turn to the representation of sets, the choice of a representation is not so obvious. Indeed, there are a number of possible representations, and they differ significantly from one another in several ways.

Informally, a set is simply a collection of distinct objects. To give a more precise definition we can employ the method of data abstraction. That is, we define "set" by specifying the operations that are to be used on sets. These are `union_set`, `intersection_set`, `is_element_of_set`, and `adjoin_set`. The function `is_element_of_set` is a predicate that determines whether a given element is a member of a set. The function `adjoin_set` takes an object and a set as arguments and returns a set that contains the elements of the original set and also the adjoined element. The function `union_set` computes the union of two sets, which is the set containing each element that appears in either argument. The function `intersection_set` computes the intersection of two sets, which is the set containing only elements that appear in both arguments. From the viewpoint of data abstraction, we are free to

design any representation that implements these operations in a way consistent with the interpretations given above.[33]

Sets as unordered lists

One way to represent a set is as a list of its elements in which no element appears more than once. The empty set is represented by the empty list. In this representation, `is_element_of_set` is similar to the function `member` of section 2.3.1. It uses `equal` instead of `===` so that the set elements need not be just numbers or strings:

```
function is_element_of_set(x, set) {
    return is_null(set)
           ? false
           : equal(x, head(set))
           ? true
           : is_element_of_set(x, tail(set));
}
```

Using this, we can write `adjoin_set`. If the object to be adjoined is already in the set, we just return the set. Otherwise, we use `pair` to add the object to the list that represents the set:

```
function adjoin_set(x, set) {
    return is_element_of_set(x, set)
           ? set
           : pair(x, set);
}
```

For `intersection_set` we can use a recursive strategy. If we know how to form the intersection of `set2` and the `tail` of `set1`, we only need to decide whether to include the `head` of `set1` in this. But this depends on whether `head(set1)` is also in `set2`. Here is the resulting function:

```
function intersection_set(set1, set2) {
    return is_null(set1) || is_null(set2)
           ? null
           : is_element_of_set(head(set1), set2)
           ? pair(head(set1), intersection_set(tail(set1), set2))
           : intersection_set(tail(set1), set2);
}
```

---

33. If we want to be more formal, we can specify "consistent with the interpretations given above" to mean that the operations satisfy a collection of rules such as these:

- For any set `S` and any object `x`, `is_element_of_set(x, adjoin_set(x, S))` is true (informally: "Adjoining an object to a set produces a set that contains the object").
- For any sets `S` and `T` and any object `x`, `is_element_of_set(x, union_set(S, T))` is equal to `is_element_of_set(x, S) || is_element_of_set(x, T)` (informally: "The elements of `union_set(S, T)` are the elements that are in `S` or in `T`").
- For any object `x`, `is_element_of_set(x, null)` is false (informally: "No object is an element of the empty set").

In designing a representation, one of the issues we should be concerned with is efficiency. Consider the number of steps required by our set operations. Since they all use `is_element_of_set`, the speed of this operation has a major impact on the efficiency of the set implementation as a whole. Now, in order to check whether an object is a member of a set, `is_element_of_set` may have to scan the entire set. (In the worst case, the object turns out not to be in the set.) Hence, if the set has n elements, `is_element_of_set` might take up to n steps. Thus, the number of steps required grows as $\Theta(n)$. The number of steps required by `adjoin_set`, which uses this operation, also grows as $\Theta(n)$. For `intersection_set`, which does an `is_element_of_set` check for each element of `set1`, the number of steps required grows as the product of the sizes of the sets involved, or $\Theta(n^2)$ for two sets of size n. The same will be true of `union_set`.

Exercise 2.59

Implement the `union_set` operation for the unordered-list representation of sets.

Exercise 2.60

We specified that a set would be represented as a list with no duplicates. Now suppose we allow duplicates. For instance, the set $\{1, 2, 3\}$ could be represented as the list `list(2, 3, 2, 1, 3, 2, 2)`. Design functions `is_element_of_set`, `adjoin_set`, `union_set`, and `intersection_set` that operate on this representation. How does the efficiency of each compare with the corresponding function for the non-duplicate representation? Are there applications for which you would use this representation in preference to the non-duplicate one?

Sets as ordered lists

One way to speed up our set operations is to change the representation so that the set elements are listed in increasing order. To do this, we need some way to compare two objects so that we can say which is bigger. For example, we could compare strings lexicographically, or we could agree on some method for assigning a unique number to an object and then compare the elements by comparing the corresponding numbers. To keep our discussion simple, we will consider only the case where the set elements are numbers, so that we can compare elements using `>` and `<`. We will represent a set of numbers by listing its elements in increasing order. Whereas our first representation above allowed us to represent the set $\{1, 3, 6, 10\}$ by listing the elements in any order, our new representation allows only the list `list(1, 3, 6, 10)`.

One advantage of ordering shows up in `is_element_of_set`: In checking for the presence of an item, we no longer have to scan the entire set. If we reach a set element that is larger than the item we are looking for, then we know that the item is not in the set:

```
function is_element_of_set(x, set) {
    return is_null(set)
           ? false
           : x === head(set)
           ? true
           : x < head(set)
           ? false
           : //x > head(set)
             is_element_of_set(x, tail(set));
}
```

How many steps does this save? In the worst case, the item we are looking for may be the largest one in the set, so the number of steps is the same as for the unordered representation. On the other hand, if we search for items of many different sizes we can expect that sometimes we will be able to stop searching at a point near the beginning of the list and that other times we will still need to examine most of the list. On the average we should expect to have to examine about half of the items in the set. Thus, the average number of steps required will be about $n/2$. This is still $\Theta(n)$ growth, but it does save us, on the average, a factor of 2 in number of steps over the previous implementation.

We obtain a more impressive speedup with `intersection_set`. In the unordered representation this operation required $\Theta(n^2)$ steps, because we performed a complete scan of `set2` for each element of `set1`. But with the ordered representation, we can use a more clever method. Begin by comparing the initial elements, `x1` and `x2`, of the two sets. If `x1` equals `x2`, then that gives an element of the intersection, and the rest of the intersection is the intersection of the `tail`s of the two sets. Suppose, however, that `x1` is less than `x2`. Since `x2` is the smallest element in `set2`, we can immediately conclude that `x1` cannot appear anywhere in `set2` and hence is not in the intersection. Hence, the intersection is equal to the intersection of `set2` with the `tail` of `set1`. Similarly, if `x2` is less than `x1`, then the intersection is given by the intersection of `set1` with the `tail` of `set2`. Here is the function:

```
function intersection_set(set1, set2) {
    if (is_null(set1) || is_null(set2)) {
        return null;
    } else {
        const x1 = head(set1);
        const x2 = head(set2);
        return x1 === x2
               ? pair(x1, intersection_set(tail(set1), tail(set2)))
               : x1 < x2
               ? intersection_set(tail(set1), set2)
               : //x2 < x1
                 intersection_set(set1, tail(set2));
    }
}
```

To estimate the number of steps required by this process, observe that at each step we reduce the intersection problem to computing intersections of smaller sets—removing the first element from `set1` or `set2` or both. Thus, the number of steps required is at most the sum of the sizes of `set1` and `set2`, rather than the

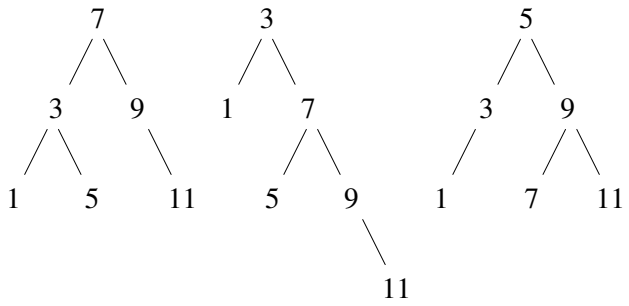


Figure 2.16 Various binary trees that represent the set $\{1, 3, 5, 7, 9, 11\}$.

product of the sizes as with the unordered representation. This is $\Theta(n)$ growth rather than $\Theta(n^2)$—a considerable speedup, even for sets of moderate size.

Exercise 2.61

Give an implementation of `adjoin_set` using the ordered representation. By analogy with `is_element_of_set` show how to take advantage of the ordering to produce a function that requires on the average about half as many steps as with the unordered representation.

Exercise 2.62

Give a $\Theta(n)$ implementation of `union_set` for sets represented as ordered lists.

Sets as binary trees

We can do better than the ordered-list representation by arranging the set elements in the form of a tree. Each node of the tree holds one element of the set, called the "entry" at that node, and a link to each of two other (possibly empty) nodes. The "left" link points to elements smaller than the one at the node, and the "right" link to elements greater than the one at the node. Figure 2.16 shows some trees that represent the set $\{1, 3, 5, 7, 9, 11\}$. The same set may be represented by a tree in a number of different ways. The only thing we require for a valid representation is that all elements in the left subtree be smaller than the node entry and that all elements in the right subtree be larger.

The advantage of the tree representation is this: Suppose we want to check whether a number x is contained in a set. We begin by comparing x with the entry in the top node. If x is less than this, we know that we need only search the left subtree; if x is greater, we need only search the right subtree. Now, if the tree is "balanced," each of these subtrees will be about half the size of the original. Thus, in one step we have reduced the problem of searching a tree of size n to searching a tree of size $n/2$. Since the size of the tree is halved at each step, we should expect that the number of steps needed to search a tree of size n grows as $\Theta(\log n)$.[34] For large sets, this will be a significant speedup over the previous representations.

34. Halving the size of the problem at each step is the distinguishing characteristic of logarithmic growth, as we saw with the fast-exponentiation algorithm of section 1.2.4 and the half-interval search method of section 1.3.3.

We can represent trees by using lists. Each node will be a list of three items: the entry at the node, the left subtree, and the right subtree. A left or a right subtree of the empty list will indicate that there is no subtree connected there. We can describe this representation by the following functions:[35]

```
function entry(tree) { return head(tree); }
function left_branch(tree) { return head(tail(tree)); }
function right_branch(tree) { return head(tail(tail(tree))); }
function make_tree(entry, left, right) {
    return list(entry, left, right);
}
```

Now we can write `is_element_of_set` using the strategy described above:

```
function is_element_of_set(x, set) {
    return is_null(set)
           ? false
           : x === entry(set)
           ? true
           : x < entry(set)
           ? is_element_of_set(x, left_branch(set))
           : //x > entry(set)
             is_element_of_set(x, right_branch(set));
}
```

Adjoining an item to a set is implemented similarly and also requires $\Theta(\log n)$ steps. To adjoin an item `x`, we compare `x` with the node entry to determine whether `x` should be added to the right or to the left branch, and having adjoined `x` to the appropriate branch we piece this newly constructed branch together with the original entry and the other branch. If `x` is equal to the entry, we just return the node. If we are asked to adjoin `x` to an empty tree, we generate a tree that has `x` as the entry and empty right and left branches. Here is the function:

```
function adjoin_set(x, set) {
    return is_null(set)
           ? make_tree(x, null, null)
           : x === entry(set)
           ? set
           : x < entry(set)
           ? make_tree(entry(set),
                       adjoin_set(x, left_branch(set)),
                       right_branch(set))
           : //x > entry(set)
             make_tree(entry(set),
                       left_branch(set),
                       adjoin_set(x, right_branch(set)));
}
```

35. We are representing sets in terms of trees, and trees in terms of lists—in effect, a data abstraction built upon a data abstraction. We can regard the functions `entry`, `left_branch`, `right_branch`, and `make_tree` as a way of isolating the abstraction of a "binary tree" from the particular way we might wish to represent such a tree in terms of list structure.

The above claim that searching the tree can be performed in a logarithmic number of steps rests on the assumption that the tree is "balanced," i.e., that the left and the right subtree of every tree have approximately the same number of elements, so that each subtree contains about half the elements of its parent. But how can we be certain that the trees we construct will be balanced? Even if we start with a balanced tree, adding elements with `adjoin_set` may produce an unbalanced result. Since the position of a newly adjoined element depends on how the element compares with the items already in the set, we can expect that if we add elements "randomly" the tree will tend to be balanced on the average. But this is not a guarantee. For example, if we start with an empty set and adjoin the numbers 1 through 7 in sequence we end up with the highly unbalanced tree shown in figure 2.17. In this tree all the left subtrees are empty, so it has no advantage over a simple ordered list. One way to solve this problem is to define an operation that transforms an arbitrary tree into a balanced tree with the same elements. Then we can perform this transformation after every few `adjoin_set` operations to keep our set in balance. There are also other ways to solve this problem, most of which involve designing new data structures for which searching and insertion both can be done in $\Theta(\log n)$ steps.[36]

Exercise 2.63

Each of the following two functions converts a binary tree to a list.

```
function tree_to_list_1(tree) {
    return is_null(tree)
           ? null
           : append(tree_to_list_1(left_branch(tree)),
                    pair(entry(tree),
                         tree_to_list_1(right_branch(tree))));
}

function tree_to_list_2(tree) {
    function copy_to_list(tree, result_list) {
        return is_null(tree)
               ? result_list
               : copy_to_list(left_branch(tree),
                              pair(entry(tree),
                                   copy_to_list(right_branch(tree),
                                                result_list)));
    }
    return copy_to_list(tree, null);
}
```

a. Do the two functions produce the same result for every tree? If not, how do the results differ? What lists do the two functions produce for the trees in figure 2.16?

b. Do the two functions have the same order of growth in the number of steps required to convert a balanced tree with n elements to a list? If not, which one grows more slowly?

Exercise 2.64

The following function `list_to_tree` converts an ordered list to a balanced binary tree. The helper function `partial_tree` takes as arguments an integer n and list of at least n

36. Examples of such structures include *B-trees* and *red-black trees*. There is a large literature on data structures devoted to this problem. See Cormen, Leiserson, Rivest, and Stein 2022.

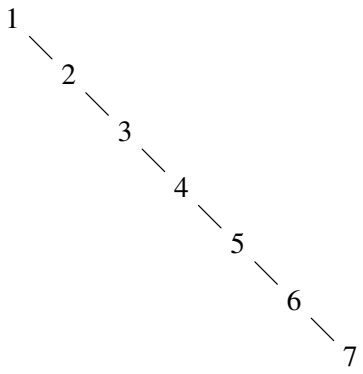


Figure 2.17 Unbalanced tree produced by adjoining 1 through 7 in sequence.

elements and constructs a balanced tree containing the first n elements of the list. The result returned by `partial_tree` is a pair (formed with `pair`) whose `head` is the constructed tree and whose `tail` is the list of elements not included in the tree.

```
function list_to_tree(elements) {
    return head(partial_tree(elements, length(elements)));
}
function partial_tree(elts, n) {
    if (n === 0) {
        return pair(null, elts);
    } else {
        const left_size = math_floor((n - 1) / 2);
        const left_result = partial_tree(elts, left_size);
        const left_tree = head(left_result);
        const non_left_elts = tail(left_result);
        const right_size = n - (left_size + 1);
        const this_entry = head(non_left_elts);
        const right_result = partial_tree(tail(non_left_elts), right_size);
        const right_tree = head(right_result);
        const remaining_elts = tail(right_result);
        return pair(make_tree(this_entry, left_tree, right_tree),
                    remaining_elts);
    }
}
```

a. Write a short paragraph explaining as clearly as you can how `partial_tree` works. Draw the tree produced by `list_to_tree` for the list `list(1, 3, 5, 7, 9, 11)`.

b. What is the order of growth in the number of steps required by `list_to_tree` to convert a list of n elements?

Exercise 2.65

Use the results of exercises 2.63 and 2.64 to give $\Theta(n)$ implementations of `union_set` and `intersection_set` for sets implemented as (balanced) binary trees.[37]

37. Exercises 2.63–2.65 are due to Paul Hilfinger.

Sets and information retrieval

We have examined options for using lists to represent sets and have seen how the choice of representation for a data object can have a large impact on the performance of the programs that use the data. Another reason for concentrating on sets is that the techniques discussed here appear again and again in applications involving information retrieval.

Consider a data base containing a large number of individual records, such as the personnel files for a company or the transactions in an accounting system. A typical data-management system spends a large amount of time accessing or modifying the data in the records and therefore requires an efficient method for accessing records. This is done by identifying a part of each record to serve as an identifying *key*. A key can be anything that uniquely identifies the record. For a personnel file, it might be an employee's ID number. For an accounting system, it might be a transaction number. Whatever the key is, when we define the record as a data structure we should include a `key` selector function that retrieves the key associated with a given record.

Now we represent the data base as a set of records. To locate the record with a given key we use a function `lookup`, which takes as arguments a key and a data base and which returns the record that has that key, or false if there is no such record. The function `lookup` is implemented in almost the same way as `is_element_of_set`. For example, if the set of records is implemented as an unordered list, we could use

```
function lookup(given_key, set_of_records) {
    return is_null(set_of_records)
           ? false
           : equal(given_key, key(head(set_of_records)))
           ? head(set_of_records)
           : lookup(given_key, tail(set_of_records));
}
```

Of course, there are better ways to represent large sets than as unordered lists. Information-retrieval systems in which records have to be "randomly accessed" are typically implemented by a tree-based method, such as the binary-tree representation discussed previously. In designing such a system the methodology of data abstraction can be a great help. The designer can create an initial implementation using a simple, straightforward representation such as unordered lists. This will be unsuitable for the eventual system, but it can be useful in providing a "quick and dirty" data base with which to test the rest of the system. Later on, the data representation can be modified to be more sophisticated. If the data base is accessed in terms of abstract selectors and constructors, this change in representation will not require any changes to the rest of the system.

Exercise 2.66

Implement the `lookup` function for the case where the set of records is structured as a binary tree, ordered by the numerical values of the keys.

2.3.4 Example: Huffman Encoding Trees

This section provides practice in the use of list structure and data abstraction to manipulate sets and trees. The application is to methods for representing data as sequences of ones and zeros (bits). For example, the ASCII standard code used to represent text in computers encodes each character as a sequence of seven bits. Using seven bits allows us to distinguish 2^7, or 128, possible different characters. In general, if we want to distinguish n different symbols, we will need to use $\log_2 n$ bits per symbol. If all our messages are made up of the eight symbols A, B, C, D, E, F, G, and H, we can choose a code with three bits per character, for example

A	000	C	010	E	100	G	110
B	001	D	011	F	101	H	111

With this code, the message

BACADAEAFABBAAAGAH

is encoded as the string of 54 bits

001000010000011000100000101000001001000000000110000111

Codes such as ASCII and the A-through-H code above are known as *fixed-length* codes, because they represent each symbol in the message with the same number of bits. It is sometimes advantageous to use *variable-length* codes, in which different symbols may be represented by different numbers of bits. For example, Morse code does not use the same number of dots and dashes for each letter of the alphabet. In particular, E, the most frequent letter, is represented by a single dot. In general, if our messages are such that some symbols appear very frequently and some very rarely, we can encode data more efficiently (i.e., using fewer bits per message) if we assign shorter codes to the frequent symbols. Consider the following alternative code for the letters A through H:

A	0	C	1010	E	1100	G	1110
B	100	D	1011	F	1101	H	1111

With this code, the same message as above is encoded as the string

100010100101101100011010100100000111001111

This string contains 42 bits, so it saves more than 20% in space in comparison with the fixed-length code shown above.

One of the difficulties of using a variable-length code is knowing when you have reached the end of a symbol in reading a sequence of zeros and ones. Morse code solves this problem by using a special *separator code* (in this case, a pause) after the sequence of dots and dashes for each letter. Another solution is to design the code in such a way that no complete code for any symbol is the beginning (or *prefix*) of the code for another symbol. Such a code is called a *prefix code*. In the example above, A is encoded by 0 and B is encoded by 100, so no other symbol can have a code that begins with 0 or with 100.

In general, we can attain significant savings if we use variable-length prefix codes that take advantage of the relative frequencies of the symbols in the messages

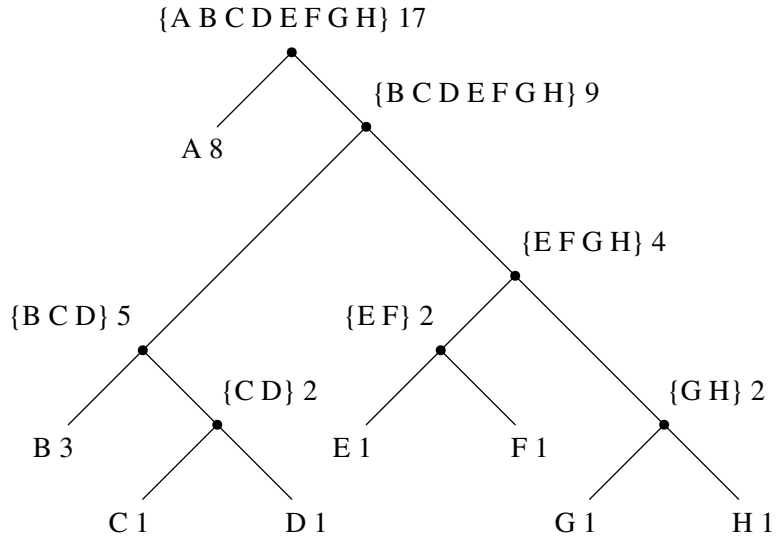


Figure 2.18 A Huffman encoding tree.

to be encoded. One particular scheme for doing this is called the Huffman encoding method, after its discoverer, David Huffman. A Huffman code can be represented as a binary tree whose leaves are the symbols that are encoded. At each non-leaf node of the tree there is a set containing all the symbols in the leaves that lie below the node. In addition, each symbol at a leaf is assigned a weight (which is its relative frequency), and each non-leaf node contains a weight that is the sum of all the weights of the leaves lying below it. The weights are not used in the encoding or the decoding process. We will see below how they are used to help construct the tree.

Figure 2.18 shows the Huffman tree for the A-through-H code given above. The weights at the leaves indicate that the tree was designed for messages in which A appears with relative frequency 8, B with relative frequency 3, and the other letters each with relative frequency 1.

Given a Huffman tree, we can find the encoding of any symbol by starting at the root and moving down until we reach the leaf that holds the symbol. Each time we move down a left branch we add a 0 to the code, and each time we move down a right branch we add a 1. (We decide which branch to follow by testing to see which branch either is the leaf node for the symbol or contains the symbol in its set.) For example, starting from the root of the tree in figure 2.18, we arrive at the leaf for D by following a right branch, then a left branch, then a right branch, then a right branch; hence, the code for D is 1011.

To decode a bit sequence using a Huffman tree, we begin at the root and use the successive zeros and ones of the bit sequence to determine whether to move down the left or the right branch. Each time we come to a leaf, we have generated a new symbol in the message, at which point we start over from the root of the tree to find the next symbol. For example, suppose we are given the tree above and the sequence 10001010. Starting at the root, we move down the right branch (since the first bit of the string is 1), then down the left branch (since the second bit is 0), then down the

left branch (since the third bit is also 0). This brings us to the leaf for B, so the first symbol of the decoded message is B. Now we start again at the root, and we make a left move because the next bit in the string is 0. This brings us to the leaf for A. Then we start again at the root with the rest of the string 1010, so we move right, left, right, left and reach C. Thus, the entire message is BAC.

Generating Huffman trees

Given an "alphabet" of symbols and their relative frequencies, how do we construct the "best" code? (In other words, which tree will encode messages with the fewest bits?) Huffman gave an algorithm for doing this and showed that the resulting code is indeed the best variable-length code for messages where the relative frequency of the symbols matches the frequencies with which the code was constructed. We will not prove this optimality of Huffman codes here, but we will show how Huffman trees are constructed.[38]

The algorithm for generating a Huffman tree is very simple. The idea is to arrange the tree so that the symbols with the lowest frequency appear farthest away from the root. Begin with the set of leaf nodes, containing symbols and their frequencies, as determined by the initial data from which the code is to be constructed. Now find two leaves with the lowest weights and merge them to produce a node that has these two nodes as its left and right branches. The weight of the new node is the sum of the two weights. Remove the two leaves from the original set and replace them by this new node. Now continue this process. At each step, merge two nodes with the smallest weights, removing them from the set and replacing them with a node that has these two as its left and right branches. The process stops when there is only one node left, which is the root of the entire tree. Here is how the Huffman tree of figure 2.18 was generated:

Initial leaves	{(A 8) (B 3) (C 1) (D 1) (E 1) (F 1) (G 1) (H 1)}
Merge	{(A 8) (B 3) ({C D} 2) (E 1) (F 1) (G 1) (H 1)}
Merge	{(A 8) (B 3) ({C D} 2) ({E F} 2) (G 1) (H 1)}
Merge	{(A 8) (B 3) ({C D} 2) ({E F} 2) ({G H} 2)}
Merge	{(A 8) (B 3) ({C D} 2) ({E F G H} 4)}
Merge	{(A 8) ({B C D} 5) ({E F G H} 4)}
Merge	{(A 8) ({B C D E F G H} 9)}
Final merge	{({A B C D E F G H} 17)}

The algorithm does not always specify a unique tree, because there may not be unique smallest-weight nodes at each step. Also, the choice of the order in which the two nodes are merged (i.e., which will be the right branch and which will be the left branch) is arbitrary.

38. See Hamming 1980 for a discussion of the mathematical properties of Huffman codes.

Representing Huffman trees

In the exercises below we will work with a system that uses Huffman trees to encode and decode messages and generates Huffman trees according to the algorithm outlined above. We will begin by discussing how trees are represented.

Leaves of the tree are represented by a list consisting of the string `"leaf"`, the symbol at the leaf, and the weight:

```
function make_leaf(symbol, weight) {
    return list("leaf", symbol, weight);
}
function is_leaf(object) {
    return head(object) === "leaf";
}
function symbol_leaf(x) { return head(tail(x)); }
function weight_leaf(x) { return head(tail(tail(x))); }
```

A general tree will be a list of a string `"code_tree"`, a left branch, a right branch, a set of symbols, and a weight. The set of symbols will be simply a list of the symbols, rather than some more sophisticated set representation. When we make a tree by merging two nodes, we obtain the weight of the tree as the sum of the weights of the nodes, and the set of symbols as the union of the sets of symbols for the nodes. Since our symbol sets are represented as lists, we can form the union by using the `append` function we defined in section 2.2.1:

```
function make_code_tree(left, right) {
    return list("code_tree", left, right,
                append(symbols(left), symbols(right)),
                weight(left) + weight(right));
}
```

If we make a tree in this way, we have the following selectors:

```
function left_branch(tree) { return head(tail(tree)); }
function right_branch(tree) { return head(tail(tail(tree))); }
function symbols(tree) {
    return is_leaf(tree)
           ? list(symbol_leaf(tree))
           : head(tail(tail(tail(tree))));
}
function weight(tree) {
    return is_leaf(tree)
           ? weight_leaf(tree)
           : head(tail(tail(tail(tail(tree)))));
}
```

The functions `symbols` and `weight` must do something slightly different depending on whether they are called with a leaf or a general tree. These are simple examples of *generic functions* (functions that can handle more than one kind of data), which we will have much more to say about in sections 2.4 and 2.5.

The decoding function

The following function implements the decoding algorithm. It takes as arguments a list of zeros and ones, together with a Huffman tree.

```
function decode(bits, tree) {
    function decode_1(bits, current_branch) {
        if (is_null(bits)) {
            return null;
        } else {
            const next_branch = choose_branch(head(bits),
                                              current_branch);
            return is_leaf(next_branch)
                   ? pair(symbol_leaf(next_branch),
                          decode_1(tail(bits), tree))
                   : decode_1(tail(bits), next_branch);
        }
    }
    return decode_1(bits, tree);
}
function choose_branch(bit, branch) {
    return bit === 0
           ? left_branch(branch)
           : bit === 1
           ? right_branch(branch)
           : error(bit, "bad bit -- choose_branch");
}
```

The function `decode_1` takes two arguments: the list of remaining bits and the current position in the tree. It keeps moving "down" the tree, choosing a left or a right branch according to whether the next bit in the list is a zero or a one. (This is done with the function `choose_branch`.) When it reaches a leaf, it returns the symbol at that leaf as the next symbol in the message by adjoining it to the result of decoding the rest of the message, starting at the root of the tree. Note the error check in the final clause of `choose_branch`, which complains if the function finds something other than a zero or a one in the input data.

Sets of weighted elements

In our representation of trees, each non-leaf node contains a set of symbols, which we have represented as a simple list. However, the tree-generating algorithm discussed above requires that we also work with sets of leaves and trees, successively merging the two smallest items. Since we will be required to repeatedly find the smallest item in a set, it is convenient to use an ordered representation for this kind of set.

We will represent a set of leaves and trees as a list of elements, arranged in increasing order of weight. The following `adjoin_set` function for constructing sets is similar to the one described in exercise 2.61; however, items are compared by their weights, and the element being added to the set is never already in it.

```
function adjoin_set(x, set) {
    return is_null(set)
           ? list(x)
           : weight(x) < weight(head(set))
           ? pair(x, set)
           : pair(head(set), adjoin_set(x, tail(set)));
}
```

The following function takes a list of symbol-frequency pairs such as

```
list(list("A", 4), list("B", 2), list("C", 1), list("D", 1))
```

and constructs an initial ordered set of leaves, ready to be merged according to the Huffman algorithm:

```
function make_leaf_set(pairs) {
    if (is_null(pairs)) {
        return null;
    } else {
        const first_pair = head(pairs);
        return adjoin_set(
                   make_leaf(head(first_pair),         // symbol
                             head(tail(first_pair))), // frequency
                   make_leaf_set(tail(pairs)));
    }
}
```

Exercise 2.67

Declare an encoding tree and a sample message:

```
const sample_tree = make_code_tree(make_leaf("A", 4),
                                   make_code_tree(make_leaf("B", 2),
                                                  make_code_tree(
                                                      make_leaf("D", 1),
                                                      make_leaf("C", 1))));
const sample_message = list(0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0);
```

Use the `decode` function to decode the message, and give the result.

Exercise 2.68

The `encode` function takes as arguments a message and a tree and produces the list of bits that gives the encoded message.

```
function encode(message, tree) {
    return is_null(message)
           ? null
           : append(encode_symbol(head(message), tree),
                    encode(tail(message), tree));
}
```

The function `encode_symbol`, which you must write, returns the list of bits that encodes a given symbol according to a given tree. You should design `encode_symbol` so that it

signals an error if the symbol is not in the tree at all. Test your function by encoding the result you obtained in exercise 2.67 with the sample tree and seeing whether it is the same as the original sample message.

Exercise 2.69

The following function takes as its argument a list of symbol-frequency pairs (where no symbol appears in more than one pair) and generates a Huffman encoding tree according to the Huffman algorithm.

```
function generate_huffman_tree(pairs) {
    return successive_merge(make_leaf_set(pairs));
}
```

The function `make_leaf_set` that transforms the list of pairs into an ordered set of leaves is given above. Write the function `successive_merge` using `make_code_tree` to successively merge the smallest-weight elements of the set until there is only one element left, which is the desired Huffman tree. (This function is slightly tricky, but not really complicated. If you find yourself designing a complex function, then you are almost certainly doing something wrong. You can take significant advantage of the fact that we are using an ordered set representation.)

Exercise 2.70

The following eight-symbol alphabet with associated relative frequencies was designed to efficiently encode the lyrics of 1950s rock songs. (Note that the "symbols" of an "alphabet" need not be individual letters.)

A	2	NA	16
BOOM	1	SHA	3
GET	2	YIP	9
JOB	2	WAH	1

Use `generate_huffman_tree` (exercise 2.69) to generate a corresponding Huffman tree, and use `encode` (exercise 2.68) to encode the following message:

Get a job
Sha na na na na na na na na
Get a job
Sha na na na na na na na na
Wah yip yip yip yip yip yip yip yip yip
Sha boom

How many bits are required for the encoding? What is the smallest number of bits that would be needed to encode this song if we used a fixed-length code for the eight-symbol alphabet?

Exercise 2.71

Suppose we have a Huffman tree for an alphabet of n symbols, and that the relative frequencies of the symbols are 1, 2, 4, ..., 2^{n-1}. Sketch the tree for n=5; for n=10. In such a tree (for general n) how may bits are required to encode the most frequent symbol? the least frequent symbol?

Exercise 2.72

Consider the encoding function that you designed in exercise 2.68. What is the order of growth in the number of steps needed to encode a symbol? Be sure to include the number of steps needed to search the symbol list at each node encountered. To answer this question in general is difficult. Consider the special case where the relative frequencies of the n symbols are as described in exercise 2.71, and give the order of growth (as a function of n) of the number of steps needed to encode the most frequent and least frequent symbols in the alphabet.

2.4 Multiple Representations for Abstract Data

We have introduced data abstraction, a methodology for structuring systems in such a way that much of a program can be specified independent of the choices involved in implementing the data objects that the program manipulates. For example, we saw in section 2.1.1 how to separate the task of designing a program that uses rational numbers from the task of implementing rational numbers in terms of the computer language's primitive mechanisms for constructing compound data. The key idea was to erect an abstraction barrier—in this case, the selectors and constructors for rational numbers (`make_rat`, `numer`, `denom`)—that isolates the way rational numbers are used from their underlying representation in terms of list structure. A similar abstraction barrier isolates the details of the functions that perform rational arithmetic (`add_rat`, `sub_rat`, `mul_rat`, and `div_rat`) from the "higher-level" functions that use rational numbers. The resulting program has the structure shown in figure 2.1.

These data-abstraction barriers are powerful tools for controlling complexity. By isolating the underlying representations of data objects, we can divide the task of designing a large program into smaller tasks that can be performed separately. But this kind of data abstraction is not yet powerful enough, because it may not always make sense to speak of "the underlying representation" for a data object.

For one thing, there might be more than one useful representation for a data object, and we might like to design systems that can deal with multiple representations. To take a simple example, complex numbers may be represented in two almost equivalent ways: in rectangular form (real and imaginary parts) and in polar form (magnitude and angle). Sometimes rectangular form is more appropriate and sometimes polar form is more appropriate. Indeed, it is perfectly plausible to imagine a system in which complex numbers are represented in both ways, and in which the functions for manipulating complex numbers work with either representation.

More importantly, programming systems are often designed by many people working over extended periods of time, subject to requirements that change over time. In such an environment, it is simply not possible for everyone to agree in advance on choices of data representation. So in addition to the data-abstraction barriers that isolate representation from use, we need abstraction barriers that isolate different design choices from each other and permit different choices to coexist in a single program. Furthermore, since large programs are often created by combining preexisting modules that were designed in isolation, we need conventions that permit programmers to incorporate modules into larger systems *additively*, that is, without having to redesign or reimplement these modules.

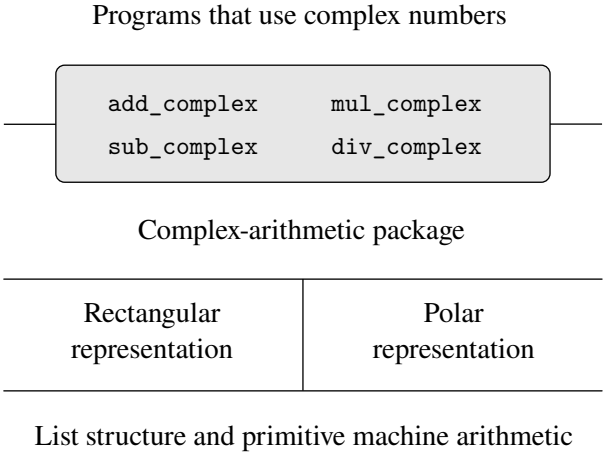


Figure 2.19 Data-abstraction barriers in the complex-number system.

In this section, we will learn how to cope with data that may be represented in different ways by different parts of a program. This requires constructing *generic functions*—functions that can operate on data that may be represented in more than one way. Our main technique for building generic functions will be to work in terms of data objects that have *type tags*, that is, data objects that include explicit information about how they are to be processed. We will also discuss *data-directed* programming, a powerful and convenient implementation strategy for additively assembling systems with generic operations.

We begin with the simple complex-number example. We will see how type tags and data-directed style enable us to design separate rectangular and polar representations for complex numbers while maintaining the notion of an abstract "complex-number" data object. We will accomplish this by defining arithmetic functions for complex numbers (`add_complex`, `sub_complex`, `mul_complex`, and `div_complex`) in terms of generic selectors that access parts of a complex number independent of how the number is represented. The resulting complex-number system, as shown in figure 2.19, contains two different kinds of abstraction barriers. The "horizontal" abstraction barriers play the same role as the ones in figure 2.1. They isolate "higher-level" operations from "lower-level" representations. In addition, there is a "vertical" barrier that gives us the ability to separately design and install alternative representations.

In section 2.5 we will show how to use type tags and data-directed style to develop a generic arithmetic package. This provides functions (`add`, `mul`, and so on) that can be used to manipulate all sorts of "numbers" and can be easily extended when a new kind of number is needed. In section 2.5.3, we'll show how to use generic arithmetic in a system that performs symbolic algebra.

2.4.1 Representations for Complex Numbers

We will develop a system that performs arithmetic operations on complex numbers as a simple but unrealistic example of a program that uses generic operations. We begin by discussing two plausible representations for complex numbers as ordered pairs: rectangular form (real part and imaginary part) and polar form (magnitude and angle).[39] Section 2.4.2 will show how both representations can be made to coexist in a single system through the use of type tags and generic operations.

Like rational numbers, complex numbers are naturally represented as ordered pairs. The set of complex numbers can be thought of as a two-dimensional space with two orthogonal axes, the "real" axis and the "imaginary" axis. (See figure 2.20.) From this point of view, the complex number $z = x + iy$ (where $i^2 = -1$) can be thought of as the point in the plane whose real coordinate is x and whose imaginary coordinate is y. Addition of complex numbers reduces in this representation to addition of coordinates:

$$\begin{aligned} \text{Real-part}(z_1 + z_2) &= \text{Real-part}(z_1) + \text{Real-part}(z_2) \\ \text{Imaginary-part}(z_1 + z_2) &= \text{Imaginary-part}(z_1) + \text{Imaginary-part}(z_2) \end{aligned}$$

When multiplying complex numbers, it is more natural to think in terms of representing a complex number in polar form, as a magnitude and an angle (r and A in figure 2.20). The product of two complex numbers is the vector obtained by stretching one complex number by the length of the other and then rotating it through the angle of the other:

$$\begin{aligned} \text{Magnitude}(z_1 \cdot z_2) &= \text{Magnitude}(z_1) \cdot \text{Magnitude}(z_2) \\ \text{Angle}(z_1 \cdot z_2) &= \text{Angle}(z_1) + \text{Angle}(z_2) \end{aligned}$$

Thus, there are two different representations for complex numbers, which are appropriate for different operations. Yet, from the viewpoint of someone writing a program that uses complex numbers, the principle of data abstraction suggests that all the operations for manipulating complex numbers should be available regardless of which representation is used by the computer. For example, it is often useful to be able to find the magnitude of a complex number that is specified by rectangular coordinates. Similarly, it is often useful to be able to determine the real part of a complex number that is specified by polar coordinates.

To design such a system, we can follow the same data-abstraction strategy we followed in designing the rational-number package in section 2.1.1. Assume that the operations on complex numbers are implemented in terms of four selectors:

---

39. In actual computational systems, rectangular form is preferable to polar form most of the time because of roundoff errors in conversion between rectangular and polar form. This is why the complex-number example is unrealistic. Nevertheless, it provides a clear illustration of the design of a system using generic operations and a good introduction to the more substantial systems to be developed later in this chapter.

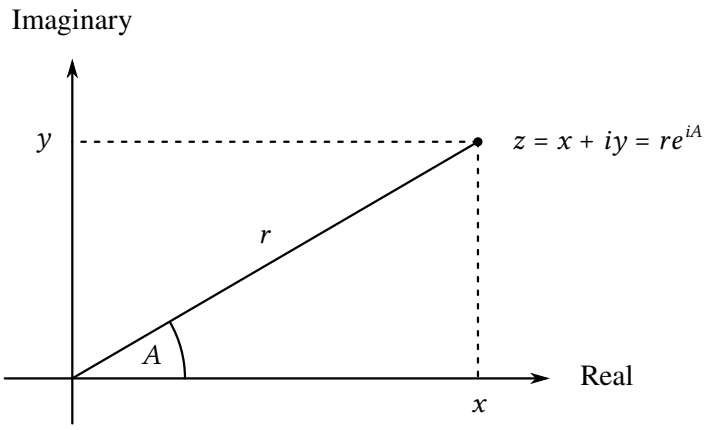


Figure 2.20 Complex numbers as points in the plane.

`real_part`, `imag_part`, `magnitude`, and `angle`. Also assume that we have two functions for constructing complex numbers: `make_from_real_imag` returns a complex number with specified real and imaginary parts, and `make_from_mag_ang` returns a complex number with specified magnitude and angle. These functions have the property that, for any complex number `z`, both

```
make_from_real_imag(real_part(z), imag_part(z));
```

and

```
make_from_mag_ang(magnitude(z), angle(z));
```

produce complex numbers that are equal to `z`.

Using these constructors and selectors, we can implement arithmetic on complex numbers using the "abstract data" specified by the constructors and selectors, just as we did for rational numbers in section 2.1.1. As shown in the formulas above, we can add and subtract complex numbers in terms of real and imaginary parts while multiplying and dividing complex numbers in terms of magnitudes and angles:

```
function add_complex(z1, z2) {
    return make_from_real_imag(real_part(z1) + real_part(z2),
                               imag_part(z1) + imag_part(z2));
}
function sub_complex(z1, z2) {
    return make_from_real_imag(real_part(z1) - real_part(z2),
                               imag_part(z1) - imag_part(z2));
}
function mul_complex(z1, z2) {
    return make_from_mag_ang(magnitude(z1) * magnitude(z2),
                             angle(z1) + angle(z2));
}
function div_complex(z1, z2) {
    return make_from_mag_ang(magnitude(z1) / magnitude(z2),
                             angle(z1) - angle(z2));
}
```

To complete the complex-number package, we must choose a representation and we must implement the constructors and selectors in terms of primitive numbers and primitive list structure. There are two obvious ways to do this: We can represent a complex number in "rectangular form" as a pair (real part, imaginary part) or in "polar form" as a pair (magnitude, angle). Which shall we choose?

In order to make the different choices concrete, imagine that there are two programmers, Ben Bitdiddle and Alyssa P. Hacker, who are independently designing representations for the complex-number system. Ben chooses to represent complex numbers in rectangular form. With this choice, selecting the real and imaginary parts of a complex number is straightforward, as is constructing a complex number with given real and imaginary parts. To find the magnitude and the angle, or to construct a complex number with a given magnitude and angle, he uses the trigonometric relations

$$\begin{array}{rclcrcl} x & = & r\cos A & \qquad & r & = & \sqrt{x^2+y^2} \\ y & = & r\sin A & & A & = & \arctan(y,x) \end{array}$$

which relate the real and imaginary parts (x, y) to the magnitude and the angle (r, A).[40] Ben's representation is therefore given by the following selectors and constructors:

```
function real_part(z) { return head(z); }
function imag_part(z) { return tail(z); }
function magnitude(z) {
    return math_sqrt(square(real_part(z)) + square(imag_part(z)));
}
function angle(z) {
    return math_atan2(imag_part(z), real_part(z));
}
function make_from_real_imag(x, y) { return pair(x, y); }

function make_from_mag_ang(r, a) {
    return pair(r * math_cos(a), r * math_sin(a));
}
```

Alyssa, in contrast, chooses to represent complex numbers in polar form. For her, selecting the magnitude and angle is straightforward, but she has to use the trigonometric relations to obtain the real and imaginary parts. Alyssa's representation is:

```
function real_part(z) {
    return magnitude(z) * math_cos(angle(z));
}
function imag_part(z) {
    return magnitude(z) * math_sin(angle(z));
}
function magnitude(z) { return head(z); }
function angle(z) { return tail(z); }
```

40. The arctangent function referred to here, computed by JavaScript's `math_atan2` function, is defined so as to take two arguments y and x and to return the angle whose tangent is y/x. The signs of the arguments determine the quadrant of the angle.

```
function make_from_real_imag(x, y) {
    return pair(math_sqrt(square(x) + square(y)),
                math_atan2(y, x));
}
function make_from_mag_ang(r, a) { return pair(r, a); }
```

The discipline of data abstraction ensures that the same implementation of `add_complex`, `sub_complex`, `mul_complex`, and `div_complex` will work with either Ben's representation or Alyssa's representation.

2.4.2 Tagged data

One way to view data abstraction is as an application of the "principle of least commitment." In implementing the complex-number system in section 2.4.1, we can use either Ben's rectangular representation or Alyssa's polar representation. The abstraction barrier formed by the selectors and constructors permits us to defer to the last possible moment the choice of a concrete representation for our data objects and thus retain maximum flexibility in our system design.

The principle of least commitment can be carried to even further extremes. If we desire, we can maintain the ambiguity of representation even *after* we have designed the selectors and constructors, and elect to use both Ben's representation *and* Alyssa's representation. If both representations are included in a single system, however, we will need some way to distinguish data in polar form from data in rectangular form. Otherwise, if we were asked, for instance, to find the `magnitude` of the pair $(3, 4)$, we wouldn't know whether to answer 5 (interpreting the number in rectangular form) or 3 (interpreting the number in polar form). A straightforward way to accomplish this distinction is to include a *type tag*—the string `"rectangular"` or `"polar"`—as part of each complex number. Then when we need to manipulate a complex number we can use the tag to decide which selector to apply.

In order to manipulate tagged data, we will assume that we have functions `type_tag` and `contents` that extract from a data object the tag and the actual contents (the polar or rectangular coordinates, in the case of a complex number). We will also postulate a function `attach_tag` that takes a tag and contents and produces a tagged data object. A straightforward way to implement this is to use ordinary list structure:

```
function attach_tag(type_tag, contents) {
    return pair(type_tag, contents);
}
function type_tag(datum) {
    return is_pair(datum)
           ? head(datum)
           : error(datum, "bad tagged datum -- type_tag");
}
function contents(datum) {
    return is_pair(datum)
           ? tail(datum)
           : error(datum, "bad tagged datum -- contents");
}
```

Using type_tag, we can define predicates is_rectangular and is_polar, which recognize rectangular and polar numbers, respectively:

```
function is_rectangular(z) {
    return type_tag(z) === "rectangular";
}
function is_polar(z) {
    return type_tag(z) === "polar";
}
```

With type tags, Ben and Alyssa can now modify their code so that their two different representations can coexist in the same system. Whenever Ben constructs a complex number, he tags it as rectangular. Whenever Alyssa constructs a complex number, she tags it as polar. In addition, Ben and Alyssa must make sure that the names of their functions do not conflict. One way to do this is for Ben to append the suffix rectangular to the name of each of his representation functions and for Alyssa to append polar to the names of hers. Here is Ben's revised rectangular representation from section 2.4.1:

```
function real_part_rectangular(z) { return head(z); }
function imag_part_rectangular(z) { return tail(z); }
function magnitude_rectangular(z) {
    return math_sqrt(square(real_part_rectangular(z)) +
                     square(imag_part_rectangular(z)));
}
function angle_rectangular(z) {
    return math_atan2(imag_part_rectangular(z),
                      real_part_rectangular(z));
}
function make_from_real_imag_rectangular(x, y) {
    return attach_tag("rectangular", pair(x, y));
}
function make_from_mag_ang_rectangular(r, a) {
    return attach_tag("rectangular",
                      pair(r * math_cos(a), r * math_sin(a)));
}
```

and here is Alyssa's revised polar representation:

```
function real_part_polar(z) {
    return magnitude_polar(z) * math_cos(angle_polar(z));
}
function imag_part_polar(z) {
    return magnitude_polar(z) * math_sin(angle_polar(z));
}
function magnitude_polar(z) { return head(z); }
function angle_polar(z) { return tail(z); }
function make_from_real_imag_polar(x, y) {
    return attach_tag("polar",
                      pair(math_sqrt(square(x) + square(y)),
                           math_atan2(y, x)));
}
```

```
function make_from_mag_ang_polar(r, a) {
    return attach_tag("polar", pair(r, a));
}
```

Each generic selector is implemented as a function that checks the tag of its argument and calls the appropriate function for handling data of that type. For example, to obtain the real part of a complex number, `real_part` examines the tag to determine whether to use Ben's `real_part_rectangular` or Alyssa's `real_part_polar`. In either case, we use `contents` to extract the bare, untagged datum and send this to the rectangular or polar function as required:

```
function real_part(z) {
    return is_rectangular(z)
           ? real_part_rectangular(contents(z))
           : is_polar(z)
           ? real_part_polar(contents(z))
           : error(z, "unknown type -- real_part");
}
function imag_part(z) {
    return is_rectangular(z)
           ? imag_part_rectangular(contents(z))
           : is_polar(z)
           ? imag_part_polar(contents(z))
           : error(z, "unknown type -- imag_part");
}
function magnitude(z) {
    return is_rectangular(z)
           ? magnitude_rectangular(contents(z))
           : is_polar(z)
           ? magnitude_polar(contents(z))
           : error(z, "unknown type -- magnitude");
}
function angle(z) {
    return is_rectangular(z)
           ? angle_rectangular(contents(z))
           : is_polar(z)
           ? angle_polar(contents(z))
           : error(z, "unknown type -- angle");
}
```

To implement the complex-number arithmetic operations, we can use the same functions `add_complex`, `sub_complex`, `mul_complex`, and `div_complex` from section 2.4.1, because the selectors they call are generic, and so will work with either representation. For example, the function `add_complex` is still

```
function add_complex(z1, z2) {
   return make_from_real_imag(real_part(z1) + real_part(z2),
                              imag_part(z1) + imag_part(z2));
}
```

Finally, we must choose whether to construct complex numbers using Ben's representation or Alyssa's representation. One reasonable choice is to construct

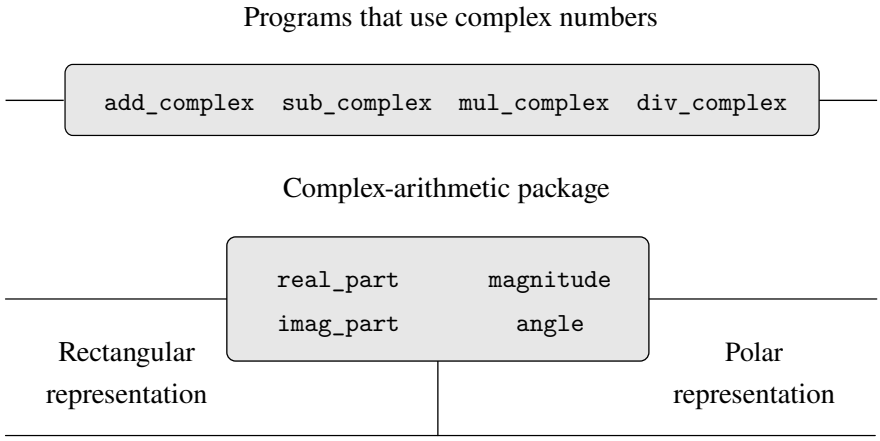


Figure 2.21 Structure of the generic complex-arithmetic system.

rectangular numbers whenever we have real and imaginary parts and to construct polar numbers whenever we have magnitudes and angles:

```
function make_from_real_imag(x, y) {
    return make_from_real_imag_rectangular(x, y);
}
function make_from_mag_ang(r, a) {
    return make_from_mag_ang_polar(r, a);
}
```

The resulting complex-number system has the structure shown in figure 2.21. The system has been decomposed into three relatively independent parts: the complex-number-arithmetic operations, Alyssa's polar implementation, and Ben's rectangular implementation. The polar and rectangular implementations could have been written by Ben and Alyssa working separately, and both of these can be used as underlying representations by a third programmer implementing the complex-arithmetic functions in terms of the abstract constructor/selector interface.

Since each data object is tagged with its type, the selectors operate on the data in a generic manner. That is, each selector is defined to have a behavior that depends upon the particular type of data it is applied to. Notice the general mechanism for interfacing the separate representations: Within a given representation implementation (say, Alyssa's polar package) a complex number is an untyped pair (magnitude, angle). When a generic selector operates on a number of `polar` type, it strips off the tag and passes the contents on to Alyssa's code. Conversely, when Alyssa constructs a number for general use, she tags it with a type so that it can be appropriately recognized by the higher-level functions. This discipline of stripping off and attaching tags as data objects are passed from level to level can be an important organizational strategy, as we shall see in section 2.5.

2.4.3 Data-Directed Programming and Additivity

The general strategy of checking the type of a datum and calling an appropriate function is called *dispatching on type*. This is a powerful strategy for obtaining modularity in system design. On the other hand, implementing the dispatch as in section 2.4.2 has two significant weaknesses. One weakness is that the generic interface functions (`real_part`, `imag_part`, `magnitude`, and `angle`) must know about all the different representations. For instance, suppose we wanted to incorporate a new representation for complex numbers into our complex-number system. We would need to identify this new representation with a type, and then add a clause to each of the generic interface functions to check for the new type and apply the appropriate selector for that representation.

Another weakness of the technique is that even though the individual representations can be designed separately, we must guarantee that no two functions in the entire system have the same name. This is why Ben and Alyssa had to change the names of their original functions from section 2.4.1.

The issue underlying both of these weaknesses is that the technique for implementing generic interfaces is not *additive*. The person implementing the generic selector functions must modify those functions each time a new representation is installed, and the people interfacing the individual representations must modify their code to avoid name conflicts. In each of these cases, the changes that must be made to the code are straightforward, but they must be made nonetheless, and this is a source of inconvenience and error. This is not much of a problem for the complex-number system as it stands, but suppose there were not two but hundreds of different representations for complex numbers. And suppose that there were many generic selectors to be maintained in the abstract-data interface. Suppose, in fact, that no one programmer knew all the interface functions or all the representations. The problem is real and must be addressed in such programs as large-scale data-base-management systems.

What we need is a means for modularizing the system design even further. This is provided by the programming technique known as *data-directed programming*. To understand how data-directed programming works, begin with the observation that whenever we deal with a set of generic operations that are common to a set of different types we are, in effect, dealing with a two-dimensional table that contains the possible operations on one axis and the possible types on the other axis. The entries in the table are the functions that implement each operation for each type of argument presented. In the complex-number system developed in the previous section, the correspondence between operation name, data type, and actual function was spread out among the various conditional clauses in the generic interface functions. But the same information could have been organized in a table, as shown in figure 2.22.

	Types	
Operations	Polar	Rectangular
`real_part`	`real_part_polar`	`real_part_rectangular`
`imag_part`	`imag_part_polar`	`imag_part_rectangular`
`magnitude`	`magnitude_polar`	`magnitude_rectangular`
`angle`	`angle_polar`	`angle_rectangular`

Figure 2.22 Table of operations for the complex-number system.

Data-directed programming is the technique of designing programs to work with such a table directly. Previously, we implemented the mechanism that interfaces the complex-arithmetic code with the two representation packages as a set of functions that each perform an explicit dispatch on type. Here we will implement the interface as a single function that looks up the combination of the operation name and argument type in the table to find the correct function to apply, and then applies it to the contents of the argument. If we do this, then to add a new representation package to the system we need not change any existing functions; we need only add new entries to the table.

To implement this plan, assume that we have two functions, `put` and `get`, for manipulating the operation-and-type table:

- `put(`*op*`,` *type*`,` *item*`)`
  installs the *item* in the table, indexed by the *op* and the *type*.
- `get(`*op*`,` *type*`)`
  looks up the *op*, *type* entry in the table and returns the item found there. If no item is found, `get` returns a unique primitive value that is referred to by the name `undefined` and recognized by the primitive predicate `is_undefined`.[41]

For now, we can assume that `put` and `get` are included in our language. In chapter 3 (section 3.3.3) we will see how to implement these and other operations for manipulating tables.

Here is how data-directed programming can be used in the complex-number system. Ben, who developed the rectangular representation, implements his code just as he did originally. He defines a collection of functions or a *package*, and interfaces these to the rest of the system by adding entries to the table that tell the system how to operate on rectangular numbers. This is accomplished by calling the following function:

41. The name `undefined` is predeclared in any JavaScript implementation and should not be used other than to refer to that primitive value.

```
function install_rectangular_package() {
    // internal functions
    function real_part(z) { return head(z); }
    function imag_part(z) { return tail(z); }
    function make_from_real_imag(x, y) { return pair(x, y); }
    function magnitude(z) {
        return math_sqrt(square(real_part(z)) + square(imag_part(z)));
    }
    function angle(z) {
        return math_atan2(imag_part(z), real_part(z));
    }
    function make_from_mag_ang(r, a) {
        return pair(r * math_cos(a), r * math_sin(a));
    }

    // interface to the rest of the system
    function tag(x) { return attach_tag("rectangular", x); }
    put("real_part", list("rectangular"), real_part);
    put("imag_part", list("rectangular"), imag_part);
    put("magnitude", list("rectangular"), magnitude);
    put("angle", list("rectangular"), angle);
    put("make_from_real_imag", "rectangular",
        (x, y) => tag(make_from_real_imag(x, y)));
    put("make_from_mag_ang", "rectangular",
        (r, a) => tag(make_from_mag_ang(r, a)));
    return "done";
}
```

Notice that the internal functions here are the same functions from section 2.4.1 that Ben wrote when he was working in isolation. No changes are necessary in order to interface them to the rest of the system. Moreover, since these function declarations are internal to the installation function, Ben needn't worry about name conflicts with other functions outside the rectangular package. To interface these to the rest of the system, Ben installs his `real_part` function under the operation name `real_part` and the type `list("rectangular")`, and similarly for the other selectors.[42] The interface also defines the constructors to be used by the external system.[43] These are identical to Ben's internally defined constructors, except that they attach the tag.

---

42. We use the list `list("rectangular")` rather than the string `"rectangular"` to allow for the possibility of operations with multiple arguments, not all of the same type.

43. The type the constructors are installed under needn't be a list because a constructor is always used to make an object of one particular type.

Alyssa's polar package is analogous:

```
function install_polar_package() {
    // internal functions
    function magnitude(z) { return head(z); }
    function angle(z) { return tail(z); }
    function make_from_mag_ang(r, a) { return pair(r, a); }
    function real_part(z) {
        return magnitude(z) * math_cos(angle(z));
    }
    function imag_part(z) {
        return magnitude(z) * math_sin(angle(z));
    }
    function make_from_real_imag(x, y) {
        return pair(math_sqrt(square(x) + square(y)),
                    math_atan2(y, x));
    }

    // interface to the rest of the system
    function tag(x) { return attach_tag("polar", x); }
    put("real_part", list("polar"), real_part);
    put("imag_part", list("polar"), imag_part);
    put("magnitude", list("polar"), magnitude);
    put("angle", list("polar"), angle);
    put("make_from_real_imag", "polar",
        (x, y) => tag(make_from_real_imag(x, y)));
    put("make_from_mag_ang", "polar",
        (r, a) => tag(make_from_mag_ang(r, a)));
    return "done";
}
```

Even though Ben and Alyssa both still use their original functions defined with the same names as each other's (e.g., `real_part`), these declarations are now internal to different functions (see section 1.1.8), so there is no name conflict.

The complex-arithmetic selectors access the table by means of a general "operation" function called `apply_generic`, which applies a generic operation to some arguments. The function `apply_generic` looks in the table under the name of the operation and the types of the arguments and applies the resulting function if one is present:[44]

---

44. The function `apply_generic` uses the function `apply_in_underlying_javascript` given in section 4.1.4 (footnote 18), which takes two arguments, a function and a list, and applies the function, using the elements in the list as arguments. For example,

```
apply_in_underlying_javascript(sum_of_squares, list(1, 3))
```

returns 10.

```
function apply_generic(op, args) {
    const type_tags = map(type_tag, args);
    const fun = get(op, type_tags);
    return ! is_undefined(fun)
           ? apply_in_underlying_javascript(fun, map(contents, args))
           : error(list(op, type_tags),
                   "no method for these types -- apply_generic");
}
```

Using `apply_generic`, we can define our generic selectors as follows:

```
function real_part(z) { return apply_generic("real_part", list(z)); }
function imag_part(z) { return apply_generic("imag_part", list(z)); }
function magnitude(z) { return apply_generic("magnitude", list(z)); }
function angle(z)     { return apply_generic("angle", list(z));     }
```

Observe that these do not change at all if a new representation is added to the system.

We can also extract from the table the constructors to be used by the programs external to the packages in making complex numbers from real and imaginary parts and from magnitudes and angles. As in section 2.4.2, we construct rectangular numbers whenever we have real and imaginary parts, and polar numbers whenever we have magnitudes and angles:

```
function make_from_real_imag(x, y) {
   return get("make_from_real_imag", "rectangular")(x, y);
}
function make_from_mag_ang(r, a) {
   return get("make_from_mag_ang", "polar")(r, a);
}
```

Exercise 2.73

Section 2.3.2 described a program that performs symbolic differentiation:

```
function deriv(exp, variable) {
    return is_number(exp)
           ? 0
           : is_variable(exp)
           ? is_same_variable(exp, variable) ? 1 : 0
           : is_sum(exp)
           ? make_sum(deriv(addend(exp), variable),
                      deriv(augend(exp), variable))
           : is_product(exp)
           ? make_sum(make_product(multiplier(exp),
                                   deriv(multiplicand(exp), variable)),
                      make_product(deriv(multiplier(exp), variable),
                                   multiplicand(exp)))
           // more rules can be added here
           : error(exp, "unknown expression type -- deriv");
}
```

```
deriv(list("*", list("*", "x", "y"), list("+", "x", 4)), "x");
list("+", list("*", list("*", x, y), list("+", 1, 0)),
          list("*", list("+", list("*", x, 0), list("*", 1, y)),
                    list("+",  x, 4)))
```

We can regard this program as performing a dispatch on the type of the expression to be differentiated. In this situation the "type tag" of the datum is the algebraic operator symbol (such as "+") and the operation being performed is `deriv`. We can transform this program into data-directed style by rewriting the basic derivative function as

```
function deriv(exp, variable) {
    return is_number(exp)
           ? 0
           : is_variable(exp)
           ? is_same_variable(exp, variable) ? 1 : 0
           : get("deriv", operator(exp))(operands(exp), variable);
}
function operator(exp) { return head(exp); }
function operands(exp) { return tail(exp); }
```

a. Explain what was done above. Why can't we assimilate the predicates `is_number` and `is_variable` into the data-directed dispatch?

b. Write the functions for derivatives of sums and products, and the auxiliary code required to install them in the table used by the program above.

c. Choose any additional differentiation rule that you like, such as the one for exponents (exercise 2.56), and install it in this data-directed system.

d. In this simple algebraic manipulator the type of an expression is the algebraic operator that binds it together. Suppose, however, we indexed the functions in the opposite way, so that the dispatch line in `deriv` looked like

   ```
   get(operator(exp), "deriv")(operands(exp), variable);
   ```

   What corresponding changes to the derivative system are required?

Exercise 2.74

Insatiable Enterprises, Inc., is a highly decentralized conglomerate company consisting of a large number of independent divisions located all over the world. The company's computer facilities have just been interconnected by means of a clever network-interfacing scheme that makes the entire network appear to any user to be a single computer. Insatiable's president, in her first attempt to exploit the ability of the network to extract administrative information from division files, is dismayed to discover that, although all the division files have been implemented as data structures in JavaScript, the particular data structure used varies from division to division. A meeting of division managers is hastily called to search for a strategy to integrate the files that will satisfy headquarters' needs while preserving the existing autonomy of the divisions.

Show how such a strategy can be implemented with data-directed programming. As an example, suppose that each division's personnel records consist of a single file, which contains a set of records keyed on employees' names. The structure of the set varies from division to division. Furthermore, each employee's record is itself a set (structured differently from division to division) that contains information keyed under identifiers such as `address` and `salary`. In particular:

a. Implement for headquarters a `get_record` function that retrieves a specified employee's record from a specified personnel file. The function should be applicable to any division's file. Explain how the individual divisions' files should be structured. In particular, what type information must be supplied?

b. Implement for headquarters a `get_salary` function that returns the salary information from a given employee's record from any division's personnel file. How should the record be structured in order to make this operation work?

c. Implement for headquarters a `find_employee_record` function. This should search all the divisions' files for the record of a given employee and return the record. Assume that this function takes as arguments an employee's name and a list of all the divisions' files.

d. When Insatiable takes over a new company, what changes must be made in order to incorporate the new personnel information into the central system?

Message passing

The key idea of data-directed programming is to handle generic operations in programs by dealing explicitly with operation-and-type tables, such as the table in figure 2.22. The style of programming we used in section 2.4.2 organized the required dispatching on type by having each operation take care of its own dispatching. In effect, this decomposes the operation-and-type table into rows, with each generic operation function representing a row of the table.

An alternative implementation strategy is to decompose the table into columns and, instead of using "intelligent operations" that dispatch on data types, to work with "intelligent data objects" that dispatch on operation names. We can do this by arranging things so that a data object, such as a rectangular number, is represented as a function that takes as input the required operation name and performs the operation indicated. In such a discipline, `make_from_real_imag` could be written as

```
function make_from_real_imag(x, y) {
    function dispatch(op) {
        return op === "real_part"
               ? x
               : op === "imag_part"
               ? y
               : op === "magnitude"
               ? math_sqrt(square(x) + square(y))
               : op === "angle"
               ? math_atan2(y, x)
               : error(op, "unknown op -- make_from_real_imag");
    }
    return dispatch;
}
```

The corresponding `apply_generic` function, which applies a generic operation to an argument, now simply feeds the operation's name to the data object and lets the object do the work:[45]

```
function apply_generic(op, arg) { return head(arg)(op); }
```

Note that the value returned by `make_from_real_imag` is a function—the internal `dispatch` function. This is the function that is invoked when `apply_generic` requests an operation to be performed.

This style of programming is called *message passing*. The name comes from the image that a data object is an entity that receives the requested operation name as a "message." We have already seen an example of message passing in section 2.1.3, where we saw how `pair`, `head`, and `tail` could be defined with no data objects but only functions. Here we see that message passing is not a mathematical trick but a useful technique for organizing systems with generic operations. In the remainder of this chapter we will continue to use data-directed programming, rather than message passing, to discuss generic arithmetic operations. In chapter 3 we will return to message passing, and we will see that it can be a powerful tool for structuring simulation programs.

Exercise 2.75

Implement the constructor `make_from_mag_ang` in message-passing style. This function should be analogous to the `make_from_real_imag` function given above.

Exercise 2.76

As a large system with generic operations evolves, new types of data objects or new operations may be needed. For each of the three strategies—generic operations with explicit dispatch, data-directed style, and message-passing-style—describe the changes that must be made to a system in order to add new types or new operations. Which organization would be most appropriate for a system in which new types must often be added? Which would be most appropriate for a system in which new operations must often be added?

2.5 Systems with Generic Operations

In the previous section, we saw how to design systems in which data objects can be represented in more than one way. The key idea is to link the code that specifies the data operations to the several representations by means of generic interface functions. Now we will see how to use this same idea not only to define operations that are generic over different representations but also to define operations that are generic over different kinds of arguments. We have already seen several different packages of arithmetic operations: the primitive arithmetic (`+`, `-`, `*`, `/`) built into our language, the rational-number arithmetic (`add_rat`, `sub_rat`, `mul_rat`, `div_rat`) of section 2.1.1, and the complex-number arithmetic that we implemented in section 2.4.3. We will now use data-directed techniques to construct a package of

45. One limitation of this organization is it permits only generic functions of one argument.

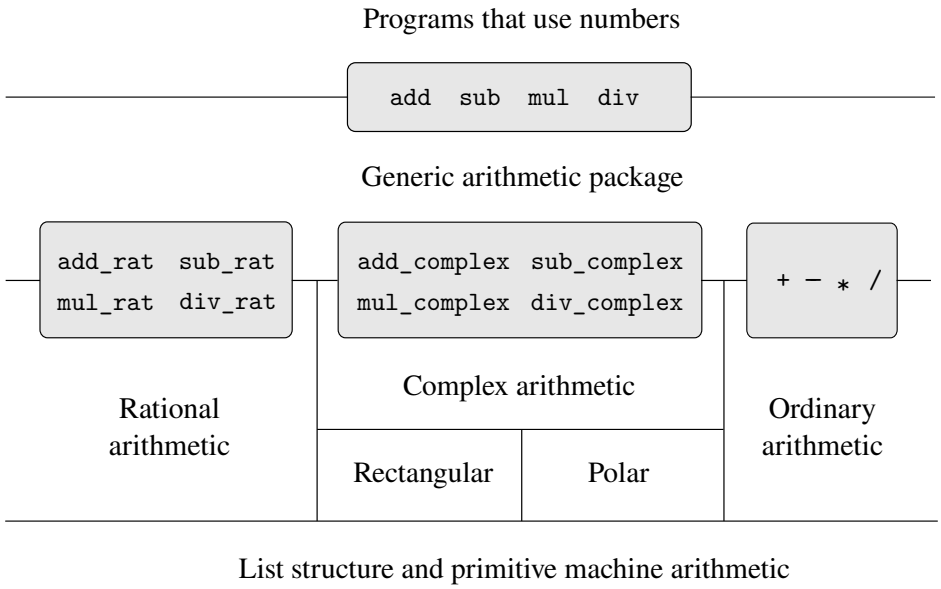


Figure 2.23 Generic arithmetic system.

arithmetic operations that incorporates all the arithmetic packages we have already constructed.

Figure 2.23 shows the structure of the system we shall build. Notice the abstraction barriers. From the perspective of someone using "numbers," there is a single function `add` that operates on whatever numbers are supplied. The function `add` is part of a generic interface that allows the separate ordinary-arithmetic, rational-arithmetic, and complex-arithmetic packages to be accessed uniformly by programs that use numbers. Any individual arithmetic package (such as the complex package) may itself be accessed through generic functions (such as `add_complex`) that combine packages designed for different representations (such as rectangular and polar). Moreover, the structure of the system is additive, so that one can design the individual arithmetic packages separately and combine them to produce a generic arithmetic system.

2.5.1 Generic Arithmetic Operations

The task of designing generic arithmetic operations is analogous to that of designing the generic complex-number operations. We would like, for instance, to have a generic addition function `add` that acts like ordinary primitive addition `+` on ordinary numbers, like `add_rat` on rational numbers, and like `add_complex` on complex numbers. We can implement `add`, and the other generic arithmetic operations, by

following the same strategy we used in section 2.4.3 to implement the generic selectors for complex numbers. We will attach a type tag to each kind of number and cause the generic function to dispatch to an appropriate package according to the data type of its arguments.

The generic arithmetic functions are defined as follows:

```
function add(x, y) { return apply_generic("add", list(x, y)); }
function sub(x, y) { return apply_generic("sub", list(x, y)); }
function mul(x, y) { return apply_generic("mul", list(x, y)); }
function div(x, y) { return apply_generic("div", list(x, y)); }
```

We begin by installing a package for handling *ordinary* numbers, that is, the primitive numbers of our language. We tag these with the string `"javascript_number"`. The arithmetic operations in this package are the primitive arithmetic functions (so there is no need to define extra functions to handle the untagged numbers). Since these operations each take two arguments, they are installed in the table keyed by the list `list("javascript_number", "javascript_number")`:

```
function install_javascript_number_package() {
    function tag(x) {
        return attach_tag("javascript_number", x);
    }
    put("add", list("javascript_number", "javascript_number"),
        (x, y) => tag(x + y));
    put("sub", list("javascript_number", "javascript_number"),
        (x, y) => tag(x - y));
    put("mul", list("javascript_number", "javascript_number"),
        (x, y) => tag(x * y));
    put("div", list("javascript_number", "javascript_number"),
        (x, y) => tag(x / y));
    put("make", "javascript_number",
        x => tag(x));
    return "done";
}
```

Users of the JavaScript-number package will create (tagged) ordinary numbers by means of the function:

```
function make_javascript_number(n) {
    return get("make", "javascript_number")(n);
}
```

Now that the framework of the generic arithmetic system is in place, we can readily include new kinds of numbers. Here is a package that performs rational arithmetic. Notice that, as a benefit of additivity, we can use without modification the rational-number code from section 2.1.1 as the internal functions in the package:

```
function install_rational_package() {
    // internal functions
    function numer(x) { return head(x); }
    function denom(x) { return tail(x); }
    function make_rat(n, d) {
        const g = gcd(n, d);
        return pair(n / g, d / g);
    }
    function add_rat(x, y) {
        return make_rat(numer(x) * denom(y) + numer(y) * denom(x),
                        denom(x) * denom(y));
    }
    function sub_rat(x, y) {
        return make_rat(numer(x) * denom(y) - numer(y) * denom(x),
                        denom(x) * denom(y));
    }
    function mul_rat(x, y) {
        return make_rat(numer(x) * numer(y),
                        denom(x) * denom(y));
    }
    function div_rat(x, y) {
        return make_rat(numer(x) * denom(y),
                        denom(x) * numer(y));
    }
    // interface to rest of the system
    function tag(x) {
        return attach_tag("rational", x);
    }
    put("add", list("rational", "rational"),
        (x, y) => tag(add_rat(x, y)));
    put("sub", list("rational", "rational"),
        (x, y) => tag(sub_rat(x, y)));
    put("mul", list("rational", "rational"),
        (x, y) => tag(mul_rat(x, y)));
    put("div", list("rational", "rational"),
        (x, y) => tag(div_rat(x, y)));
    put("make", "rational",
        (n, d) => tag(make_rat(n, d)));
    return "done";
}

function make_rational(n, d) {
    return get("make", "rational")(n, d);
}
```

We can install a similar package to handle complex numbers, using the tag `"complex"`. In creating the package, we extract from the table the operations `make_from_real_imag` and `make_from_mag_ang` that were defined by the rectangular and polar packages. Additivity permits us to use, as the internal operations, the same `add_complex`, `sub_complex`, `mul_complex`, and `div_complex` functions from section 2.4.1.

```
function install_complex_package() {
    // imported functions from rectangular and polar packages
    function make_from_real_imag(x, y) {
        return get("make_from_real_imag", "rectangular")(x, y);
    }
    function make_from_mag_ang(r, a) {
        return get("make_from_mag_ang", "polar")(r, a);
    }
    // internal functions
    function add_complex(z1, z2) {
        return make_from_real_imag(real_part(z1) + real_part(z2),
                                   imag_part(z1) + imag_part(z2));
    }
    function sub_complex(z1, z2) {
        return make_from_real_imag(real_part(z1) - real_part(z2),
                                   imag_part(z1) - imag_part(z2));
    }
    function mul_complex(z1, z2) {
        return make_from_mag_ang(magnitude(z1) * magnitude(z2),
                                 angle(z1) + angle(z2));
    }
    function div_complex(z1, z2) {
        return make_from_mag_ang(magnitude(z1) / magnitude(z2),
                                 angle(z1) - angle(z2));
    }
    // interface to rest of the system
    function tag(z) { return attach_tag("complex", z); }
    put("add", list("complex", "complex"),
        (z1, z2) => tag(add_complex(z1, z2)));
    put("sub", list("complex", "complex"),
        (z1, z2) => tag(sub_complex(z1, z2)));
    put("mul", list("complex", "complex"),
        (z1, z2) => tag(mul_complex(z1, z2)));
    put("div", list("complex", "complex"),
        (z1, z2) => tag(div_complex(z1, z2)));
    put("make_from_real_imag", "complex",
        (x, y) => tag(make_from_real_imag(x, y)));
    put("make_from_mag_ang", "complex",
        (r, a) => tag(make_from_mag_ang(r, a)));
    return "done";
}
```

Programs outside the complex-number package can construct complex numbers either from real and imaginary parts or from magnitudes and angles. Notice how the underlying functions, originally defined in the rectangular and polar packages, are exported to the complex package, and exported from there to the outside world.

```
function make_complex_from_real_imag(x, y){
   return get("make_from_real_imag", "complex")(x, y);
}
function make_complex_from_mag_ang(r, a){
   return get("make_from_mag_ang", "complex")(r, a);
}
```

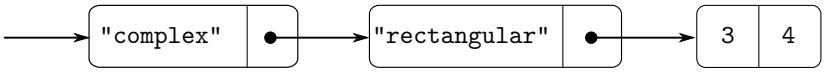


Figure 2.24 Representation of $3+4i$ in rectangular form.

What we have here is a two-level tag system. A typical complex number, such as $3+4i$ in rectangular form, would be represented as shown in figure 2.24. The outer tag (`"complex"`) is used to direct the number to the complex package. Once within the complex package, the next tag (`"rectangular"`) is used to direct the number to the rectangular package. In a large and complicated system there might be many levels, each interfaced with the next by means of generic operations. As a data object is passed "downward," the outer tag that is used to direct it to the appropriate package is stripped off (by applying `contents`) and the next level of tag (if any) becomes visible to be used for further dispatching.

In the above packages, we used `add_rat`, `add_complex`, and the other arithmetic functions exactly as originally written. Once these declarations are internal to different installation functions, however, they no longer need names that are distinct from each other: we could simply name them `add`, `sub`, `mul`, and `div` in both packages.

Exercise 2.77

Louis Reasoner tries to evaluate the expression `magnitude(z)` where `z` is the object shown in figure 2.24. To his surprise, instead of the answer 5 he gets an error message from `apply_generic`, saying there is no method for the operation `magnitude` on the types `list("complex")`. He shows this interaction to Alyssa P. Hacker, who says "The problem is that the complex-number selectors were never defined for `"complex"` numbers, just for `"polar"` and `"rectangular"` numbers. All you have to do to make this work is add the following to the `complex` package:"

```
put("real_part", list("complex"), real_part);
put("imag_part", list("complex"), imag_part);
put("magnitude", list("complex"), magnitude);
put("angle",     list("complex"), angle);
```

Describe in detail why this works. As an example, trace through all the functions called in evaluating the expression `magnitude(z)` where `z` is the object shown in figure 2.24. In particular, how many times is `apply_generic` invoked? What function is dispatched to in each case?

Exercise 2.78

The internal functions in the `javascript_number` package are essentially nothing more than calls to the primitive functions `+`, `-`, etc. It was not possible to use the primitives of the language directly because our type-tag system requires that each data object have a type attached to it. In fact, however, all JavaScript implementations do have a type system, which they use internally. Primitive predicates such as `is_string` and `is_number` determine whether data objects have particular types. Modify the definitions of `type_tag`, `contents`, and `attach_tag` from section 2.4.2 so that our generic system takes advantage of JavaScript's internal type system. That is to say, the system should work as before except that ordinary numbers should be represented simply as JavaScript numbers rather than as pairs whose `head` is the string `"javascript_number"`.

Exercise 2.79

Define a generic equality predicate `is_equal` that tests the equality of two numbers, and install it in the generic arithmetic package. This operation should work for ordinary numbers, rational numbers, and complex numbers.

Exercise 2.80

Define a generic predicate `is_equal_to_zero` that tests if its argument is zero, and install it in the generic arithmetic package. This operation should work for ordinary numbers, rational numbers, and complex numbers.

2.5.2 Combining Data of Different Types

We have seen how to define a unified arithmetic system that encompasses ordinary numbers, complex numbers, rational numbers, and any other type of number we might decide to invent, but we have ignored an important issue. The operations we have defined so far treat the different data types as being completely independent. Thus, there are separate packages for adding, say, two ordinary numbers, or two complex numbers. What we have not yet considered is the fact that it is meaningful to define operations that cross the type boundaries, such as the addition of a complex number to an ordinary number. We have gone to great pains to introduce barriers between parts of our programs so that they can be developed and understood separately. We would like to introduce the cross-type operations in some carefully controlled way, so that we can support them without seriously violating our module boundaries.

One way to handle cross-type operations is to design a different function for each possible combination of types for which the operation is valid. For example, we could extend the complex-number package so that it provides a function for adding complex numbers to ordinary numbers and installs this in the table using the tag `list("complex", "javascript_number")`:[46]

```
// to be included in the complex package
function add_complex_to_javascript_num(z, x) {
   return make_complex_from_real_imag(real_part(z) + x, imag_part(z));
}
put("add", list("complex", "javascript_number"),
    (z, x) => tag(add_complex_to_javascript_num(z, x)));
```

This technique works, but it is cumbersome. With such a system, the cost of introducing a new type is not just the construction of the package of functions for that type but also the construction and installation of the functions that implement the cross-type operations. This can easily be much more code than is needed to define the operations on the type itself. The method also undermines our ability to combine separate packages additively, or least to limit the extent to which the implementors of the individual packages need to take account of other packages. For instance, in the example above, it seems reasonable that handling mixed operations on complex numbers and ordinary numbers should be the responsibility of the complex-number

46. We also have to supply an almost identical function to handle the types `list("javascript_number", "complex")`.

package. Combining rational numbers and complex numbers, however, might be done by the complex package, by the rational package, or by some third package that uses operations extracted from these two packages. Formulating coherent policies on the division of responsibility among packages can be an overwhelming task in designing systems with many packages and many cross-type operations.

Coercion

In the general situation of completely unrelated operations acting on completely unrelated types, implementing explicit cross-type operations, cumbersome though it may be, is the best that one can hope for. Fortunately, we can usually do better by taking advantage of additional structure that may be latent in our type system. Often the different data types are not completely independent, and there may be ways by which objects of one type may be viewed as being of another type. This process is called *coercion*. For example, if we are asked to arithmetically combine an ordinary number with a complex number, we can view the ordinary number as a complex number whose imaginary part is zero. This transforms the problem to that of combining two complex numbers, which can be handled in the ordinary way by the complex-arithmetic package.

In general, we can implement this idea by designing coercion functions that transform an object of one type into an equivalent object of another type. Here is a typical coercion function, which transforms a given ordinary number to a complex number with that real part and zero imaginary part:

```
function javascript_number_to_complex(n) {
    return make_complex_from_real_imag(contents(n), 0);
}
```

We install these coercion functions in a special coercion table, indexed under the names of the two types:

```
put_coercion("javascript_number", "complex",
             javascript_number_to_complex);
```

(We assume that there are `put_coercion` and `get_coercion` functions available for manipulating this table.) Generally some of the slots in the table will be empty, because it is not generally possible to coerce an arbitrary data object of each type into all other types. For example, there is no way to coerce an arbitrary complex number to an ordinary number, so there will be no general `complex_to_javascript_number` function included in the table.

Once the coercion table has been set up, we can handle coercion in a uniform manner by modifying the `apply_generic` function of section 2.4.3. When asked to apply an operation, we first check whether the operation is defined for the arguments' types, just as before. If so, we dispatch to the function found in the operation-and-type table. Otherwise, we try coercion. For simplicity, we consider only the case where there are two arguments.[47] We check the coercion table to see if objects of the first type can be coerced to the second type. If so, we coerce the first argument

47. See exercise 2.82 for generalizations.

and try the operation again. If objects of the first type cannot in general be coerced to the second type, we try the coercion the other way around to see if there is a way to coerce the second argument to the type of the first argument. Finally, if there is no known way to coerce either type to the other type, we give up. Here is the function:

```
function apply_generic(op, args) {
    const type_tags = map(type_tag, args);
    const fun = get(op, type_tags);
    if (! is_undefined(fun)) {
        return apply(fun, map(contents, args));
    } else {
        if (length(args) === 2) {
            const type1 = head(type_tags);
            const type2 = head(tail(type_tags));
            const a1 = head(args);
            const a2 = head(tail(args));
            const t1_to_t2 = get_coercion(type1, type2);
            const t2_to_t1 = get_coercion(type2, type1);
            return ! is_undefined(t1_to_t2)
                   ? apply_generic(op, list(t1_to_t2(a1), a2))
                   : ! is_undefined(t2_to_t1)
                   ? apply_generic(op, list(a1, t2_to_t1(a2)))
                   : error(list(op, type_tags),
                           "no method for these types");
        } else {
            return error(list(op, type_tags),
                         "no method for these types");
        }
    }
}
```

This coercion scheme has many advantages over the method of defining explicit cross-type operations, as outlined above. Although we still need to write coercion functions to relate the types (possibly n^2 functions for a system with n types), we need to write only one function for each pair of types rather than a different function for each collection of types and each generic operation.[48] What we are counting on here is the fact that the appropriate transformation between types depends only on the types themselves, not on the operation to be applied.

48. If we are clever, we can usually get by with fewer than n^2 coercion functions. For instance, if we know how to convert from type 1 to type 2 and from type 2 to type 3, then we can use this knowledge to convert from type 1 to type 3. This can greatly decrease the number of coercion functions we need to supply explicitly when we add a new type to the system. If we are willing to build the required amount of sophistication into our system, we can have it search the "graph" of relations among types and automatically generate those coercion functions that can be inferred from the ones that are supplied explicitly.

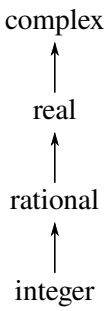


Figure 2.25 A tower of types.

On the other hand, there may be applications for which our coercion scheme is not general enough. Even when neither of the objects to be combined can be converted to the type of the other it may still be possible to perform the operation by converting both objects to a third type. In order to deal with such complexity and still preserve modularity in our programs, it is usually necessary to build systems that take advantage of still further structure in the relations among types, as we discuss next.

Hierarchies of types

The coercion scheme presented above relied on the existence of natural relations between pairs of types. Often there is more "global" structure in how the different types relate to each other. For instance, suppose we are building a generic arithmetic system to handle integers, rational numbers, real numbers, and complex numbers. In such a system, it is quite natural to regard an integer as a special kind of rational number, which is in turn a special kind of real number, which is in turn a special kind of complex number. What we actually have is a so-called *hierarchy of types*, in which, for example, integers are a *subtype* of rational numbers (i.e., any operation that can be applied to a rational number can automatically be applied to an integer). Conversely, we say that rational numbers form a *supertype* of integers. The particular hierarchy we have here is of a very simple kind, in which each type has at most one supertype and at most one subtype. Such a structure, called a *tower*, is illustrated in figure 2.25.

If we have a tower structure, then we can greatly simplify the problem of adding a new type to the hierarchy, for we need only specify how the new type is embedded in the next supertype above it and how it is the supertype of the type below it. For example, if we want to add an integer to a complex number, we need not explicitly define a special coercion function `integer_to_complex`. Instead, we define how an integer can be transformed into a rational number, how a rational number is

transformed into a real number, and how a real number is transformed into a complex number. We then allow the system to transform the integer into a complex number through these steps and then add the two complex numbers.

We can redesign our `apply_generic` function in the following way: For each type, we need to supply a `raise` function, which "raises" objects of that type one level in the tower. Then when the system is required to operate on objects of different types it can successively raise the lower types until all the objects are at the same level in the tower. (Exercises 2.83 and 2.84 concern the details of implementing such a strategy.)

Another advantage of a tower is that we can easily implement the notion that every type "inherits" all operations defined on a supertype. For instance, if we do not supply a special function for finding the real part of an integer, we should nevertheless expect that `real_part` will be defined for integers by virtue of the fact that integers are a subtype of complex numbers. In a tower, we can arrange for this to happen in a uniform way by modifying `apply_generic`. If the required operation is not directly defined for the type of the object given, we raise the object to its supertype and try again. We thus crawl up the tower, transforming our argument as we go, until we either find a level at which the desired operation can be performed or hit the top (in which case we give up).

Yet another advantage of a tower over a more general hierarchy is that it gives us a simple way to "lower" a data object to the simplest representation. For example, if we add $2+3i$ to $4-3i$, it would be nice to obtain the answer as the integer 6 rather than as the complex number $6+0i$. Exercise 2.85 discusses a way to implement such a lowering operation. (The trick is that we need a general way to distinguish those objects that can be lowered, such as $6+0i$, from those that cannot, such as $6+2i$.)

Inadequacies of hierarchies

If the data types in our system can be naturally arranged in a tower, this greatly simplifies the problems of dealing with generic operations on different types, as we have seen. Unfortunately, this is usually not the case. Figure 2.26 illustrates a more complex arrangement of mixed types, this one showing relations among different types of geometric figures. We see that, in general, a type may have more than one subtype. Triangles and quadrilaterals, for instance, are both subtypes of polygons. In addition, a type may have more than one supertype. For example, an isosceles right triangle may be regarded either as an isosceles triangle or as a right triangle. This multiple-supertypes issue is particularly thorny, since it means that there is no unique way to "raise" a type in the hierarchy. Finding the "correct" supertype in which to apply an operation to an object may involve considerable searching through the entire type network on the part of a function such as `apply_generic`. Since there generally are multiple subtypes for a type, there is a similar problem in coercing a

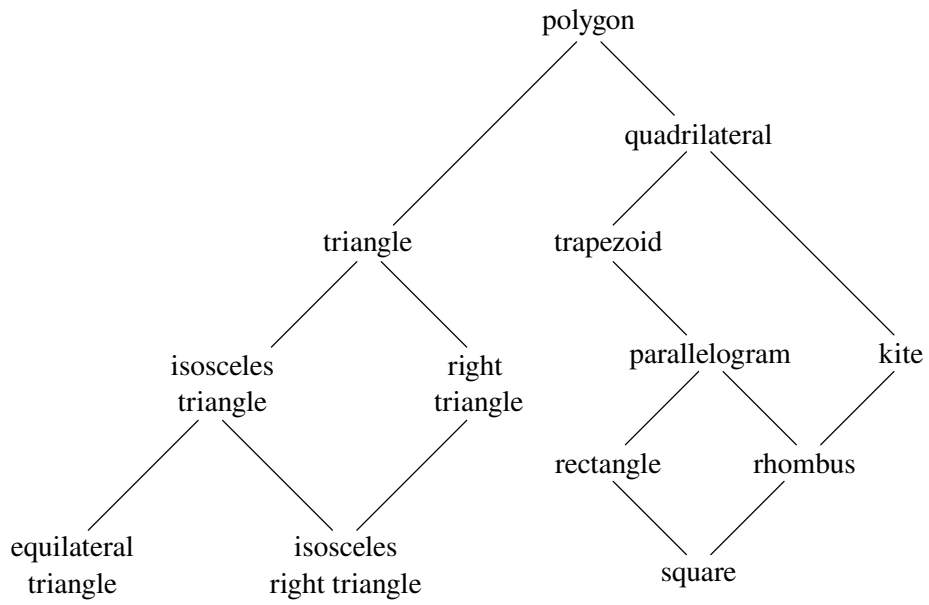


Figure 2.26 Relations among types of geometric figures.

value "down" the type hierarchy. Dealing with large numbers of interrelated types while still preserving modularity in the design of large systems is very difficult, and is an area of much current research.[49]

Exercise 2.81

Louis Reasoner has noticed that `apply_generic` may try to coerce the arguments to each other's type even if they already have the same type. Therefore, he reasons, we need to put functions in the coercion table to "coerce" arguments of each type to their own type. For example, in addition to the `javascript_number_to_complex` coercion shown above, he would do:

---

49. This statement, which also appears in the first edition of this book, is just as true now as it was when we wrote it in 1984. Developing a useful, general framework for expressing the relations among different types of entities (what philosophers call "ontology") seems intractably difficult. The main difference between the confusion that existed in 1984 and the confusion that exists now is that now a variety of inadequate ontological theories have been embodied in a plethora of correspondingly inadequate programming languages. For example, much of the complexity of object-oriented programming languages—and the subtle and confusing differences among contemporary object-oriented languages—centers on the treatment of generic operations on interrelated types. Our own discussion of computational objects in chapter 3 avoids these issues entirely. Readers familiar with object-oriented programming will notice that we have much to say in chapter 3 about local state, but we do not even mention "classes" or "inheritance." In fact, we suspect that these problems cannot be adequately addressed in terms of computer-language design alone, without also drawing on work in knowledge representation and automated reasoning.

```
function javascript_number_to_javascript_number(n) { return n; }
function complex_to_complex(n) { return n; }
put_coercion("javascript_number", "javascript_number",
             javascript_number_to_javascript_number);
put_coercion("complex", "complex", complex_to_complex);
```

a. With Louis's coercion functions installed, what happens if `apply_generic` is called with two arguments of type `"complex"` or two arguments of type `"javascript_number"` for an operation that is not found in the table for those types? For example, assume that we've defined a generic exponentiation operation:

```
function exp(x, y) {
    return apply_generic("exp", list(x, y));
}
```

and have put a function for exponentiation in the JavaScript-number package but not in any other package:

```
// following added to JavaScript-number package
put("exp", list("javascript_number", "javascript_number"),
    (x, y) => tag(math_exp(x, y))); // using primitive math_exp
```

What happens if we call `exp` with two complex numbers as arguments?

b. Is Louis correct that something had to be done about coercion with arguments of the same type, or does `apply_generic` work correctly as is?

c. Modify `apply_generic` so that it doesn't try coercion if the two arguments have the same type.

Exercise 2.82

Show how to generalize `apply_generic` to handle coercion in the general case of multiple arguments. One strategy is to attempt to coerce all the arguments to the type of the first argument, then to the type of the second argument, and so on. Give an example of a situation where this strategy (and likewise the two-argument version given above) is not sufficiently general. (Hint: Consider the case where there are some suitable mixed-type operations present in the table that will not be tried.)

Exercise 2.83

Suppose you are designing a generic arithmetic system for dealing with the tower of types shown in figure 2.25: integer, rational, real, complex. For each type (except complex), design a function that raises objects of that type one level in the tower. Show how to install a generic `raise` operation that will work for each type (except complex).

Exercise 2.84

Using the `raise` operation of exercise 2.83, modify the `apply_generic` function so that it coerces its arguments to have the same type by the method of successive raising, as discussed in this section. You will need to devise a way to test which of two types is higher in the tower. Do this in a manner that is "compatible" with the rest of the system and will not lead to problems in adding new levels to the tower.

Exercise 2.85

This section mentioned a method for "simplifying" a data object by lowering it in the tower of types as far as possible. Design a function `drop` that accomplishes this for the tower described in exercise 2.83. The key is to decide, in some general way, whether an object can be lowered. For example, the complex number $1.5 + 0i$ can be lowered as far as `"real"`, the complex number $1 + 0i$ can be lowered as far as `"integer"`, and the complex number $2 + 3i$ cannot be lowered at all. Here is a plan for determining whether an object can be lowered: Begin by defining a generic operation `project` that "pushes" an object down in the tower. For example, projecting a complex number would involve throwing away the imaginary part. Then a number can be dropped if, when we `project` it and `raise` the result back to the type we started with, we end up with something equal to what we started with. Show how to implement this idea in detail, by writing a `drop` function that drops an object as far as possible. You will need to design the various projection operations[50] and install `project` as a generic operation in the system. You will also need to make use of a generic equality predicate, such as described in exercise 2.79. Finally, use `drop` to rewrite `apply_generic` from exercise 2.84 so that it "simplifies" its answers.

Exercise 2.86

Suppose we want to handle complex numbers whose real parts, imaginary parts, magnitudes, and angles can be either ordinary numbers, rational numbers, or other numbers we might wish to add to the system. Describe and implement the changes to the system needed to accommodate this. You will have to define operations such as `sine` and `cosine` that are generic over ordinary numbers and rational numbers.

2.5.3 Example: Symbolic Algebra

The manipulation of symbolic algebraic expressions is a complex process that illustrates many of the hardest problems that occur in the design of large-scale systems. An algebraic expression, in general, can be viewed as a hierarchical structure, a tree of operators applied to operands. We can construct algebraic expressions by starting with a set of primitive objects, such as constants and variables, and combining these by means of algebraic operators, such as addition and multiplication. As in other languages, we form abstractions that enable us to refer to compound objects in simple terms. Typical abstractions in symbolic algebra are ideas such as linear combination, polynomial, rational function, or trigonometric function. We can regard these as compound "types," which are often useful for directing the processing of expressions. For example, we could describe the expression

$$x^2\ \sin(y^2+1) + x\ \cos 2y + \cos(y^3 - 2y^2)$$

as a polynomial in x with coefficients that are trigonometric functions of polynomials in y whose coefficients are integers.

We will not attempt to develop a complete algebraic-manipulation system here. Such systems are exceedingly complex programs, embodying deep algebraic knowledge and elegant algorithms. What we will do is look at a simple but important

50. A real number can be projected to an integer using the `math_round` primitive, which returns the closest integer to its argument.

part of algebraic manipulation: the arithmetic of polynomials. We will illustrate the kinds of decisions the designer of such a system faces, and how to apply the ideas of abstract data and generic operations to help organize this effort.

Arithmetic on polynomials

Our first task in designing a system for performing arithmetic on polynomials is to decide just what a polynomial is. Polynomials are normally defined relative to certain variables (the *indeterminates* of the polynomial). For simplicity, we will restrict ourselves to polynomials having just one indeterminate *(univariate polynomials)*.[51] We will define a polynomial to be a sum of terms, each of which is either a coefficient, a power of the indeterminate, or a product of a coefficient and a power of the indeterminate. A coefficient is defined as an algebraic expression that is not dependent upon the indeterminate of the polynomial. For example,

$$5x^2 + 3x + 7$$

is a simple polynomial in x, and

$$(y^2 + 1)x^3 + (2y)x + 1$$

is a polynomial in x whose coefficients are polynomials in y.

Already we are skirting some thorny issues. Is the first of these polynomials the same as the polynomial $5y^2 + 3y + 7$, or not? A reasonable answer might be "yes, if we are considering a polynomial purely as a mathematical function, but no, if we are considering a polynomial to be a syntactic form." The second polynomial is algebraically equivalent to a polynomial in y whose coefficients are polynomials in x. Should our system recognize this, or not? Furthermore, there are other ways to represent a polynomial—for example, as a product of factors, or (for a univariate polynomial) as the set of roots, or as a listing of the values of the polynomial at a specified set of points.[52] We can finesse these questions by deciding that in our algebraic-manipulation system a "polynomial" will be a particular syntactic form, not its underlying mathematical meaning.

Now we must consider how to go about doing arithmetic on polynomials. In this simple system, we will consider only addition and multiplication. Moreover, we will insist that two polynomials to be combined must have the same indeterminate.

We will approach the design of our system by following the familiar discipline of data abstraction. We will represent polynomials using a data structure called a *poly*, which consists of a variable and a collection of terms. We assume that we

---

51. On the other hand, we will allow polynomials whose coefficients are themselves polynomials in other variables. This will give us essentially the same representational power as a full multivariate system, although it does lead to coercion problems, as discussed below.

52. For univariate polynomials, giving the value of a polynomial at a given set of points can be a particularly good representation. This makes polynomial arithmetic extremely simple. To obtain, for example, the sum of two polynomials represented in this way, we need only add the values of the polynomials at corresponding points. To transform back to a more familiar representation, we can use the Lagrange interpolation formula, which shows how to recover the coefficients of a polynomial of degree n given the values of the polynomial at $n + 1$ points.

have selectors `variable` and `term_list` that extract those parts from a poly and a constructor `make_poly` that assembles a poly from a given variable and a term list. A variable will be just a string, so we can use the `is_same_variable` function of section 2.3.2 to compare variables. The following functions define addition and multiplication of polys:

```
function add_poly(p1, p2) {
    return is_same_variable(variable(p1), variable(p2))
           ? make_poly(variable(p1),
                       add_terms(term_list(p1), term_list(p2)))
           : error(list(p1, p2), "polys not in same var -- add_poly");
}
function mul_poly(p1, p2) {
    return is_same_variable(variable(p1), variable(p2))
           ? make_poly(variable(p1),
                       mul_terms(term_list(p1), term_list(p2)))
           : error(list(p1, p2), "polys not in same var -- mul_poly");
}
```

To incorporate polynomials into our generic arithmetic system, we need to supply them with type tags. We'll use the tag `"polynomial"`, and install appropriate operations on tagged polynomials in the operation table. We'll embed all our code in an installation function for the polynomial package, similar to the installation functions in section 2.5.1:

```
function install_polynomial_package() {
    // internal functions
    // representation of poly
    function make_poly(variable, term_list) {
        return pair(variable, term_list);
    }
    function variable(p) { return head(p); }
    function term_list(p) { return tail(p); }
    ⟨functions is_same_variable and is_variable from section 2.3.2⟩

    // representation of terms and term lists
    ⟨functions adjoin_term...coeff from text below⟩

    function add_poly(p1, p2) { ... }
    ⟨functions used by add_poly⟩
    function mul_poly(p1, p2) { ... }
    ⟨functions used by mul_poly⟩

    // interface to rest of the system
    function tag(p) { return attach_tag("polynomial", p); }
    put("add", list("polynomial", "polynomial"),
        (p1, p2) => tag(add_poly(p1, p2)));
    put("mul", list("polynomial", "polynomial"),
        (p1, p2) => tag(mul_poly(p1, p2)));
    put("make", "polynomial",
        (variable, terms) => tag(make_poly(variable, terms)));
    return "done";
}
```

Polynomial addition is performed termwise. Terms of the same order (i.e., with the same power of the indeterminate) must be combined. This is done by forming a new term of the same order whose coefficient is the sum of the coefficients of the addends. Terms in one addend for which there are no terms of the same order in the other addend are simply accumulated into the sum polynomial being constructed.

In order to manipulate term lists, we will assume that we have a constructor `the_empty_termlist` that returns an empty term list and a constructor `adjoin_term` that adjoins a new term to a term list. We will also assume that we have a predicate `is_empty_termlist` that tells if a given term list is empty, a selector `first_term` that extracts the highest-order term from a term list, and a selector `rest_terms` that returns all but the highest-order term. To manipulate terms, we will suppose that we have a constructor `make_term` that constructs a term with given order and coefficient, and selectors `order` and `coeff` that return, respectively, the order and the coefficient of the term. These operations allow us to consider both terms and term lists as data abstractions, whose concrete representations we can worry about separately.

Here is the function that constructs the term list for the sum of two polynomials;[53] note that we slightly extend the syntax of conditional statements described in section 1.3.2 by admitting another conditional statement in place of the block following `else`:

```
function add_terms(L1, L2) {
    if (is_empty_termlist(L1)) {
        return L2;
    } else if (is_empty_termlist(L2)) {
        return L1;
    } else {
        const t1 = first_term(L1);
        const t2 = first_term(L2);
        return order(t1) > order(t2)
               ? adjoin_term(t1, add_terms(rest_terms(L1), L2))
               : order(t1) < order(t2)
               ? adjoin_term(t2, add_terms(L1, rest_terms(L2)))
               : adjoin_term(make_term(order(t1),
                                       add(coeff(t1), coeff(t2))),
                             add_terms(rest_terms(L1),
                                       rest_terms(L2)));
    }
}
```

The most important point to note here is that we used the generic addition function `add` to add together the coefficients of the terms being combined. This has powerful consequences, as we will see below.

---

53. This operation is very much like the ordered `union_set` operation we developed in exercise 2.62. In fact, if we think of the terms of the polynomial as a set ordered according to the power of the indeterminate, then the program that produces the term list for a sum is almost identical to `union_set`.

In order to multiply two term lists, we multiply each term of the first list by all the terms of the other list, repeatedly using `mul_term_by_all_terms`, which multiplies a given term by all terms in a given term list. The resulting term lists (one for each term of the first list) are accumulated into a sum. Multiplying two terms forms a term whose order is the sum of the orders of the factors and whose coefficient is the product of the coefficients of the factors:

```
function mul_terms(L1, L2) {
    return is_empty_termlist(L1)
           ? the_empty_termlist
           : add_terms(mul_term_by_all_terms(
                                first_term(L1), L2),
                       mul_terms(rest_terms(L1), L2));
}
function mul_term_by_all_terms(t1, L) {
    if (is_empty_termlist(L)) {
        return the_empty_termlist;
    } else {
        const t2 = first_term(L);
        return adjoin_term(
                   make_term(order(t1) + order(t2),
                             mul(coeff(t1), coeff(t2))),
                   mul_term_by_all_terms(t1, rest_terms(L)));
    }
}
```

This is really all there is to polynomial addition and multiplication. Notice that, since we operate on terms using the generic functions `add` and `mul`, our polynomial package is automatically able to handle any type of coefficient that is known about by the generic arithmetic package. If we include a coercion mechanism such as one of those discussed in section 2.5.2, then we also are automatically able to handle operations on polynomials of different coefficient types, such as

$$\left[3x^2 + (2+3i)x + 7\right] \cdot \left[x^4 + \tfrac{2}{3}x^2 + (5+3i)\right]$$

Because we installed the polynomial addition and multiplication functions `add_poly` and `mul_poly` in the generic arithmetic system as the `add` and `mul` operations for type `polynomial`, our system is also automatically able to handle polynomial operations such as

$$\left[(y+1)x^2 + (y^2+1)x + (y-1)\right] \cdot \left[(y-2)x + (y^3+7)\right]$$

The reason is that when the system tries to combine coefficients, it will dispatch through `add` and `mul`. Since the coefficients are themselves polynomials (in y), these will be combined using `add_poly` and `mul_poly`. The result is a kind of "data-directed recursion" in which, for example, a call to `mul_poly` will result in recursive calls to `mul_poly` in order to multiply the coefficients. If the coefficients of the coefficients were themselves polynomials (as might be used to represent polynomials in three variables), the data direction would ensure that the system would follow

through another level of recursive calls, and so on through as many levels as the structure of the data dictates.[54]

Representing term lists

Finally, we must confront the job of implementing a good representation for term lists. A term list is, in effect, a set of coefficients keyed by the order of the term. Hence, any of the methods for representing sets, as discussed in section 2.3.3, can be applied to this task. On the other hand, our functions `add_terms` and `mul_terms` always access term lists sequentially from highest to lowest order. Thus, we will use some kind of ordered list representation.

How should we structure the list that represents a term list? One consideration is the "density" of the polynomials we intend to manipulate. A polynomial is said to be *dense* if it has nonzero coefficients in terms of most orders. If it has many zero terms it is said to be *sparse*. For example,

$$A: \quad x^5 + 2x^4 + 3x^2 - 2x - 5$$

is a dense polynomial, whereas

$$B: \quad x^{100} + 2x^2 + 1$$

is sparse.

The term list of a dense polynomial is most efficiently represented as a list of the coefficients. For example, the polynomial A above would be nicely represented as `list(1, 2, 0, 3, -2, -5)`. The order of a term in this representation is the length of the sublist beginning with that term's coefficient, decremented by 1.[55] This would be a terrible representation for a sparse polynomial such as B: There would be a giant list of zeros punctuated by a few lonely nonzero terms. A more reasonable representation of the term list of a sparse polynomial is as a list of the nonzero terms, where each term is a list containing the order of the term and the coefficient for that order. In such a scheme, polynomial B is efficiently represented as `list(list(100, 1), list(2, 2), list(0, 1))`. As most polynomial manipulations are performed on sparse polynomials, we will use this method. We will assume that term lists are represented as lists of terms, arranged from highest-order

54. To make this work completely smoothly, we should also add to our generic arithmetic system the ability to coerce a "number" to a polynomial by regarding it as a polynomial of degree zero whose coefficient is the number. This is necessary if we are going to perform operations such as

$$\left[x^2 + (y+1)x + 5\right] + \left[x^2 + 2x + 1\right]$$

which requires adding the coefficient $y+1$ to the coefficient 2.

55. In these polynomial examples, we assume that we have implemented the generic arithmetic system using the type mechanism suggested in exercise 2.78. Thus, coefficients that are ordinary numbers will be represented as the numbers themselves rather than as pairs whose `head` is the string `"javascript_number"`.

to lowest-order term. Once we have made this decision, implementing the selectors and constructors for terms and term lists is straightforward:[56]

```
function adjoin_term(term, term_list) {
    return is_equal_to_zero(coeff(term))
           ? term_list
           : pair(term, term_list);
}

const the_empty_termlist = null;
function first_term(term_list) { return head(term_list); }
function rest_terms(term_list) { return tail(term_list); }
function is_empty_termlist(term_list) { return is_null(term_list); }

function make_term(order, coeff) { return list(order, coeff); }
function order(term) { return head(term); }
function coeff(term) { return head(tail(term)); }
```

where `is_equal_to_zero` is as defined in exercise 2.80. (See also exercise 2.87 below.)

Users of the polynomial package will create (tagged) polynomials by means of the function:

```
function make_polynomial(variable, terms) {
    return get("make", "polynomial")(variable, terms);
}
```

Exercise 2.87

Install `is_equal_to_zero` for polynomials in the generic arithmetic package. This will allow `adjoin_term` to work for polynomials with coefficients that are themselves polynomials.

Exercise 2.88

Extend the polynomial system to include subtraction of polynomials. (Hint: You may find it helpful to define a generic negation operation.)

Exercise 2.89

Declare functions that implement the term-list representation described above as appropriate for dense polynomials.

---

56. Although we are assuming that term lists are ordered, we have implemented `adjoin_term` to simply adjoin the new term to the front of the existing term list. We can get away with this so long as we guarantee that the functions (such as `add_terms`) that use `adjoin_term` always call it with a higher-order term than appears in the list. If we did not want to make such a guarantee, we could have implemented `adjoin_term` to be similar to the `adjoin_set` constructor for the ordered-list representation of sets (exercise 2.61).

Exercise 2.90

Suppose we want to have a polynomial system that is efficient for both sparse and dense polynomials. One way to do this is to allow both kinds of term-list representations in our system. The situation is analogous to the complex-number example of section 2.4, where we allowed both rectangular and polar representations. To do this we must distinguish different types of term lists and make the operations on term lists generic. Redesign the polynomial system to implement this generalization. This is a major effort, not a local change.

Exercise 2.91

A univariate polynomial can be divided by another one to produce a polynomial quotient and a polynomial remainder. For example,

$$\frac{x^5-1}{x^2-1} \quad = \quad x^3+x, \text{ remainder } x-1$$

Division can be performed via long division. That is, divide the highest-order term of the dividend by the highest-order term of the divisor. The result is the first term of the quotient. Next, multiply the result by the divisor, subtract that from the dividend, and produce the rest of the answer by recursively dividing the difference by the divisor. Stop when the order of the divisor exceeds the order of the dividend and declare the dividend to be the remainder. Also, if the dividend ever becomes zero, return zero as both quotient and remainder.

We can design a `div_poly` function on the model of `add_poly` and `mul_poly`. The function checks to see if the two polys have the same variable. If so, `div_poly` strips off the variable and passes the problem to `div_terms`, which performs the division operation on term lists. The function `div_poly` finally reattaches the variable to the result supplied by `div_terms`. It is convenient to design `div_terms` to compute both the quotient and the remainder of a division. The function `div_terms` can take two term lists as arguments and return a list of the quotient term list and the remainder term list.

Complete the following definition of `div_terms` by filling in the missing parts. Use this to implement `div_poly`, which takes two polys as arguments and returns a list of the quotient and remainder polys.

```
function div_terms(L1, L2) {
    if (is_empty_termlist(L1)) {
        return list(the_empty_termlist, the_empty_termlist);
    } else {
        const t1 = first_term(L1);
        const t2 = first_term(L2);
        if (order(t2) > order(t1)) {
            return list(the_empty_termlist, L1);
        } else {
            const new_c = div(coeff(t1), coeff(t2));
            const new_o = order(t1) - order(t2);
            const rest_of_result = ⟨compute rest of result recursively⟩;
            ⟨form and return complete result⟩
        }
    }
}
```

Hierarchies of types in symbolic algebra

Our polynomial system illustrates how objects of one type (polynomials) may in fact be complex objects that have objects of many different types as parts. This poses no real difficulty in defining generic operations. We need only install appropriate generic operations for performing the necessary manipulations of the parts of the compound types. In fact, we saw that polynomials form a kind of "recursive data abstraction," in that parts of a polynomial may themselves be polynomials. Our generic operations and our data-directed programming style can handle this complication without much trouble.

On the other hand, polynomial algebra is a system for which the data types cannot be naturally arranged in a tower. For instance, it is possible to have polynomials in x whose coefficients are polynomials in y. It is also possible to have polynomials in y whose coefficients are polynomials in x. Neither of these types is "above" the other in any natural way, yet it is often necessary to add together elements from each set. There are several ways to do this. One possibility is to convert one polynomial to the type of the other by expanding and rearranging terms so that both polynomials have the same principal variable. One can impose a towerlike structure on this by ordering the variables and thus always converting any polynomial to a "canonical form" with the highest-priority variable dominant and the lower-priority variables buried in the coefficients. This strategy works fairly well, except that the conversion may expand a polynomial unnecessarily, making it hard to read and perhaps less efficient to work with. The tower strategy is certainly not natural for this domain or for any domain where the user can invent new types dynamically using old types in various combining forms, such as trigonometric functions, power series, and integrals.

It should not be surprising that controlling coercion is a serious problem in the design of large-scale algebraic-manipulation systems. Much of the complexity of such systems is concerned with relationships among diverse types. Indeed, it is fair to say that we do not yet completely understand coercion. In fact, we do not yet completely understand the concept of a data type. Nevertheless, what we know provides us with powerful structuring and modularity principles to support the design of large systems.

Exercise 2.92

By imposing an ordering on variables, extend the polynomial package so that addition and multiplication of polynomials works for polynomials in different variables. (This is not easy!)

Extended exercise: Rational functions

We can extend our generic arithmetic system to include *rational functions*. These are "fractions" whose numerator and denominator are polynomials, such as

$$\frac{x+1}{x^3-1}$$

The system should be able to add, subtract, multiply, and divide rational functions, and to perform such computations as

$$\frac{x+1}{x^3-1}+\frac{x}{x^2-1} \quad = \quad \frac{x^3+2x^2+3x+1}{x^4+x^3-x-1}$$

(Here the sum has been simplified by removing common factors. Ordinary "cross multiplication" would have produced a fourth-degree polynomial over a fifth-degree polynomial.)

If we modify our rational-arithmetic package so that it uses generic operations, then it will do what we want, except for the problem of reducing fractions to lowest terms.

Exercise 2.93

Modify the rational-arithmetic package to use generic operations, but change `make_rat` so that it does not attempt to reduce fractions to lowest terms. Test your system by calling `make_rational` on two polynomials to produce a rational function

```
const p1 = make_polynomial("x", list(make_term(2, 1), make_term(0, 1)));
const p2 = make_polynomial("x", list(make_term(3, 1), make_term(0, 1)));
const rf = make_rational(p2, p1);
```

Now add `rf` to itself, using `add`. You will observe that this addition function does not reduce fractions to lowest terms.

We can reduce polynomial fractions to lowest terms using the same idea we used with integers: modifying `make_rat` to divide both the numerator and the denominator by their greatest common divisor. The notion of "greatest common divisor" makes sense for polynomials. In fact, we can compute the GCD of two polynomials using essentially the same Euclid's Algorithm that works for integers.[57] The integer version is

```
function gcd(a, b) {
    return b === 0
           ? a
           : gcd(b, a % b);
}
```

57. The fact that Euclid's Algorithm works for polynomials is formalized in algebra by saying that polynomials form a kind of algebraic domain called a *Euclidean ring*. A Euclidean ring is a domain that admits addition, subtraction, and commutative multiplication, together with a way of assigning to each element x of the ring a positive integer "measure" $m(x)$ with the properties that $m(xy) \geq m(x)$ for any nonzero x and y and that, given any x and y, there exists a q such that $y = qx + r$ and either $r = 0$ or $m(r) < m(x)$. From an abstract point of view, this is what is needed to prove that Euclid's Algorithm works. For the domain of integers, the measure m of an integer is the absolute value of the integer itself. For the domain of polynomials, the measure of a polynomial is its degree.

Using this, we could make the obvious modification to define a GCD operation that works on term lists:

```
function gcd_terms(a, b) {
    return is_empty_termlist(b)
           ? a
           : gcd_terms(b, remainder_terms(a, b));
}
```

where `remainder_terms` picks out the remainder component of the list returned by the term-list division operation `div_terms` that was implemented in exercise 2.91.

Exercise 2.94

Using `div_terms`, implement the function `remainder_terms` and use this to define `gcd_terms` as above. Now write a function `gcd_poly` that computes the polynomial GCD of two polys. (The function should signal an error if the two polys are not in the same variable.) Install in the system a generic operation `greatest_common_divisor` that reduces to `gcd_poly` for polynomials and to ordinary `gcd` for ordinary numbers. As a test, try

```
const p1 = make_polynomial("x", list(make_term(4, 1), make_term(3, -1),
                                     make_term(2, -2), make_term(1, 2)));
const p2 = make_polynomial("x", list(make_term(3, 1), make_term(1, -1)));
greatest_common_divisor(p1, p2);
```

and check your result by hand.

Exercise 2.95

Define P_1, P_2, and P_3 to be the polynomials

P_1: $x^2 - 2x + 1$

P_2: $11x^2 + 7$

P_3: $13x + 5$

Now define Q_1 to be the product of P_1 and P_2 and Q_2 to be the product of P_1 and P_3, and use `greatest_common_divisor` (exercise 2.94) to compute the GCD of Q_1 and Q_2. Note that the answer is not the same as P_1. This example introduces noninteger operations into the computation, causing difficulties with the GCD algorithm.[58] To understand what is happening, try tracing `gcd_terms` while computing the GCD or try performing the division by hand.

We can solve the problem exhibited in exercise 2.95 if we use the following modification of the GCD algorithm (which really works only in the case of polynomials with integer coefficients). Before performing any polynomial division in the GCD computation, we multiply the dividend by an integer constant factor, chosen to guarantee that no fractions will arise during the division process. Our answer will thus differ from the actual GCD by an integer constant factor, but this does not matter

58. In JavaScript, division of integers can produce limited-precision decimal numbers, and thus we may fail to get a valid divisor.

in the case of reducing rational functions to lowest terms; the GCD will be used to divide both the numerator and denominator, so the integer constant factor will cancel out.

More precisely, if P and Q are polynomials, let O_1 be the order of P (i.e., the order of the largest term of P) and let O_2 be the order of Q. Let c be the leading coefficient of Q. Then it can be shown that, if we multiply P by the *integerizing factor* $c^{1+O_1-O_2}$, the resulting polynomial can be divided by Q by using the `div_terms` algorithm without introducing any fractions. The operation of multiplying the dividend by this constant and then dividing is sometimes called the *pseudodivision* of P by Q. The remainder of the division is called the *pseudoremainder*.

Exercise 2.96

a. Implement the function `pseudoremainder_terms`, which is just like `remainder_terms` except that it multiplies the dividend by the integerizing factor described above before calling `div_terms`. Modify `gcd_terms` to use `pseudoremainder_terms`, and verify that `greatest_common_divisor` now produces an answer with integer coefficients on the example in exercise 2.95.

b. The GCD now has integer coefficients, but they are larger than those of P_1. Modify `gcd_terms` so that it removes common factors from the coefficients of the answer by dividing all the coefficients by their (integer) greatest common divisor.

Thus, here is how to reduce a rational function to lowest terms:

- Compute the GCD of the numerator and denominator, using the version of `gcd_terms` from exercise 2.96.
- When you obtain the GCD, multiply both numerator and denominator by the same integerizing factor before dividing through by the GCD, so that division by the GCD will not introduce any noninteger coefficients. As the factor you can use the leading coefficient of the GCD raised to the power $1 + O_1 - O_2$, where O_2 is the order of the GCD and O_1 is the maximum of the orders of the numerator and denominator. This will ensure that dividing the numerator and denominator by the GCD will not introduce any fractions.
- The result of this operation will be a numerator and denominator with integer coefficients. The coefficients will normally be very large because of all of the integerizing factors, so the last step is to remove the redundant factors by computing the (integer) greatest common divisor of all the coefficients of the numerator and the denominator and dividing through by this factor.

Exercise 2.97

a. Implement this algorithm as a function `reduce_terms` that takes two term lists `n` and `d` as arguments and returns a list `nn`, `dd`, which are `n` and `d` reduced to lowest terms via the algorithm given above. Also write a function `reduce_poly`, analogous to `add_poly`, that checks to see if the two polys have the same variable. If so, `reduce_poly` strips off the variable and passes the problem to `reduce_terms`, then reattaches the variable to the two term lists supplied by `reduce_terms`.

b. Define a function analogous to `reduce_terms` that does what the original `make_rat` did for integers:

```
function reduce_integers(n, d) {
    const g = gcd(n, d);
    return list(n / g, d / g);
}
```

and define `reduce` as a generic operation that calls `apply_generic` to dispatch either to `reduce_poly` (for `polynomial` arguments) or to `reduce_integers` (for `javascript_number` arguments). You can now easily make the rational-arithmetic package reduce fractions to lowest terms by having `make_rat` call `reduce` before combining the given numerator and denominator to form a rational number. The system now handles rational expressions in either integers or polynomials. To test your program, try the example at the beginning of this extended exercise:

```
const p1 = make_polynomial("x", list(make_term(1, 1), make_term(0, 1)));
const p2 = make_polynomial("x", list(make_term(3, 1), make_term(0, -1)));
const p3 = make_polynomial("x", list(make_term(1, 1)));
const p4 = make_polynomial("x", list(make_term(2, 1), make_term(0, -1)));

const rf1 = make_rational(p1, p2);
const rf2 = make_rational(p3, p4);

add(rf1, rf2);
```

See if you get the correct answer, correctly reduced to lowest terms.

The GCD computation is at the heart of any system that does operations on rational functions. The algorithm used above, although mathematically straightforward, is extremely slow. The slowness is due partly to the large number of division operations and partly to the enormous size of the intermediate coefficients generated by the pseudodivisions. One of the active areas in the development of algebraic-manipulation systems is the design of better algorithms for computing polynomial GCDs.[59]

59. One extremely efficient and elegant method for computing polynomial GCDs was discovered by Richard Zippel (1979). The method is a probabilistic algorithm, as is the fast test for primality that we discussed in chapter 1. Zippel's book (1993) describes this method, together with other ways to compute polynomial GCDs.

3 Modularity, Objects, and State

Μεταβάλλον αναπαύεται
(Even while it changes, it stands still.)

—Heraclitus

Plus ça change, plus c'est la même chose.

—Alphonse Karr

The preceding chapters introduced the basic elements from which programs are made. We saw how primitive functions and primitive data are combined to construct compound entities, and we learned that abstraction is vital in helping us to cope with the complexity of large systems. But these tools are not sufficient for designing programs. Effective program synthesis also requires organizational principles that can guide us in formulating the overall design of a program. In particular, we need strategies to help us structure large systems so that they will be *modular*, that is, so that they can be divided "naturally" into coherent parts that can be separately developed and maintained.

One powerful design strategy, which is particularly appropriate to the construction of programs for modeling physical systems, is to base the structure of our programs on the structure of the system being modeled. For each object in the system, we construct a corresponding computational object. For each system action, we define a symbolic operation in our computational model. Our hope in using this strategy is that extending the model to accommodate new objects or new actions will require no strategic changes to the program, only the addition of the new symbolic analogs of those objects or actions. If we have been successful in our system organization, then to add a new feature or debug an old one we will have to work on only a localized part of the system.

To a large extent, then, the way we organize a large program is dictated by our perception of the system to be modeled. In this chapter we will investigate two prominent organizational strategies arising from two rather different "world views" of the structure of systems. The first organizational strategy concentrates on *objects*, viewing a large system as a collection of distinct objects whose behaviors may change over time. An alternative organizational strategy concentrates on the *streams* of information that flow in the system, much as an electrical engineer views a signal-processing system.

Both the object-based approach and the stream-processing approach raise significant linguistic issues in programming. With objects, we must be concerned with how a computational object can change and yet maintain its identity. This will force us to abandon our old substitution model of computation (section 1.1.5) in

favor of a more mechanistic but less theoretically tractable *environment model* of computation. The difficulties of dealing with objects, change, and identity are a fundamental consequence of the need to grapple with time in our computational models. These difficulties become even greater when we allow the possibility of concurrent execution of programs. The stream approach can be most fully exploited when we decouple simulated time in our model from the order of the events that take place in the computer during evaluation. We will accomplish this using a technique known as *delayed evaluation*.

3.1 Assignment and Local State

We ordinarily view the world as populated by independent objects, each of which has a state that changes over time. An object is said to "have state" if its behavior is influenced by its history. A bank account, for example, has state in that the answer to the question "Can I withdraw $100?" depends upon the history of deposit and withdrawal transactions. We can characterize an object's state by one or more *state variables*, which among them maintain enough information about history to determine the object's current behavior. In a simple banking system, we could characterize the state of an account by a current balance rather than by remembering the entire history of account transactions.

In a system composed of many objects, the objects are rarely completely independent. Each may influence the states of others through interactions, which serve to couple the state variables of one object to those of other objects. Indeed, the view that a system is composed of separate objects is most useful when the state variables of the system can be grouped into closely coupled subsystems that are only loosely coupled to other subsystems.

This view of a system can be a powerful framework for organizing computational models of the system. For such a model to be modular, it should be decomposed into computational objects that model the actual objects in the system. Each computational object must have its own *local state variables* describing the actual object's state. Since the states of objects in the system being modeled change over time, the state variables of the corresponding computational objects must also change. If we choose to model the flow of time in the system by the elapsed time in the computer, then we must have a way to construct computational objects whose behaviors change as our programs run. In particular, if we wish to model state variables by ordinary symbolic names in the programming language, then the language must provide an *assignment operation* to enable us to change the value associated with a name.

3.1.1 Local State Variables

To illustrate what we mean by having a computational object with time-varying state, let us model the situation of withdrawing money from a bank account. We will do this using a function `withdraw`, which takes as argument an `amount` to be withdrawn. If there is enough money in the account to accommodate the withdrawal, then `withdraw` should return the balance remaining after the withdrawal. Otherwise, `withdraw` should return the message *Insufficient funds.* For example, if we begin

with $100 in the account, we should obtain the following sequence of responses using `withdraw`:

```
withdraw(25);
75

withdraw(25);
50

withdraw(60);
"Insufficient funds"

withdraw(15);
35
```

Observe that the expression `withdraw(25)`, evaluated twice, yields different values. This is a new kind of behavior for a function. Until now, all our JavaScript functions could be viewed as specifications for computing mathematical functions. A call to a function computed the value of the function applied to the given arguments, and two calls to the same function with the same arguments always produced the same result.[1]

So far, all our names have been *immutable*. When a function was applied, the values that its parameters referred to never changed, and once a declaration was evaluated, the declared name never changed its value. To implement functions like `withdraw`, we introduce *variable declarations*, which use the keyword **`let`**, in addition to constant declarations, which use the keyword **`const`**. We can declare a variable `balance` to indicate the balance of money in the account and define `withdraw` as a function that accesses `balance`. The `withdraw` function checks to see if `balance` is at least as large as the requested `amount`. If so, `withdraw` decrements `balance` by `amount` and returns the new value of `balance`. Otherwise, `withdraw` returns the *Insufficient funds* message. Here are the declarations of `balance` and `withdraw`:

```
let balance = 100;

function withdraw(amount) {
    if (balance >= amount) {
        balance = balance - amount;
        return balance;
    } else {
        return "Insufficient funds";
    }
}
```

1. Actually, this is not quite true. One exception was the random-number generator in section 1.2.6. Another exception involved the operation/type tables we introduced in section 2.4.3, where the values of two calls to `get` with the same arguments depended on intervening calls to `put`. On the other hand, until we introduce assignment, we have no way to create such functions ourselves.

Decrementing `balance` is accomplished by the expression statement

```
balance = balance - amount;
```

The syntax of *assignment* expressions is

name `=` *new-value*

Here *name* has been declared with `let` or as a function parameter and *new-value* is any expression. The assignment changes *name* so that its value is the result obtained by evaluating *new-value*. In the case at hand, we are changing `balance` so that its new value will be the result of subtracting `amount` from the previous value of `balance`.[2]

The function `withdraw` also uses a *sequence of statements* to cause two statements to be evaluated in the case where the `if` test is true: first decrementing `balance` and then returning the value of `balance`. In general, executing a sequence

$stmt_1$ $stmt_2 \ldots stmt_n$

causes the statements $stmt_1$ through $stmt_n$ to be evaluated in sequence.[3]

Although `withdraw` works as desired, the variable `balance` presents a problem. As specified above, `balance` is a name defined in the program environment and is freely accessible to be examined or modified by any function. It would be much better if we could somehow make `balance` internal to `withdraw`, so that `withdraw` would be the only function that could access `balance` directly and any other function could access `balance` only indirectly (through calls to `withdraw`). This would more accurately model the notion that `balance` is a local state variable used by `withdraw` to keep track of the state of the account.

---

2. The value of an assignment is the value being assigned to the name. Assignment expression statements look similar to and should not be confused with constant and variable declarations of the form

`const` *name* `=` *value*`;`

and

`let` *name* `=` *value*`;`

in which a newly declared *name* is associated with a *value*. Assignment expressions look similar to and should not be confused with expressions of the form

$expression_1$ `===` $expression_2$

which evaluate to `true` if $expression_1$ evaluates to the same value as $expression_2$ and to `false` otherwise.

3. We have already used sequences implicitly in our programs, because in JavaScript the body block of a function can contain a sequence of function declarations followed by a return statement, not just a single return statement, as discussed in section 1.1.8.

We can make `balance` internal to `withdraw` by rewriting the definition as follows:

```
function make_withdraw_balance_100() {
    let balance = 100;
    return amount => {
                if (balance >= amount) {
                    balance = balance - amount;
                    return balance;
                } else {
                    return "Insufficient funds";
                }
            };
}
const new_withdraw = make_withdraw_balance_100();
```

What we have done here is use `let` to establish an environment with a local variable `balance`, bound to the initial value 100. Within this local environment, we use a lambda expression[4] to create a function that takes `amount` as an argument and behaves like our previous `withdraw` function. This function—returned as the result of evaluating the body of the `make_withdraw_balance_100` function—behaves in precisely the same way as `withdraw`, but its variable `balance` is not accessible by any other function.[5]

Combining assignments with variable declarations is the general programming technique we will use for constructing computational objects with local state. Unfortunately, using this technique raises a serious problem: When we first introduced functions, we also introduced the substitution model of evaluation (section 1.1.5) to provide an interpretation of what function application means. We said that applying a function whose body is a return statement should be interpreted as evaluating the return expression of the function with the parameters replaced by their values. For functions with more complex bodies, we need to evaluate the whole body with the parameters replaced by their values. The trouble is that, as soon as we introduce assignment into our language, substitution is no longer an adequate model of function application. (We will see why this is so in section 3.1.3.) As a consequence, we technically have at this point no way to understand why the `new_withdraw` function behaves as claimed above. In order to really understand a function such as `new_withdraw`, we will need to develop a new model of function application.

---

4. Blocks as bodies of lambda expressions were introduced in section 2.2.4.

5. In programming-language jargon, the variable `balance` is said to be *encapsulated* within the `new_withdraw` function. Encapsulation reflects the general system-design principle known as the *hiding principle*: One can make a system more modular and robust by protecting parts of the system from each other; that is, by providing information access only to those parts of the system that have a "need to know."

In section 3.2 we will introduce such a model, together with an explanation of assignments and variable declarations. First, however, we examine some variations on the theme established by `new_withdraw`.

Parameters of functions as well as names declared with `let` are variables. The following function, `make_withdraw`, creates "withdrawal processors." The parameter `balance` in `make_withdraw` specifies the initial amount of money in the account.[6]

```
function make_withdraw(balance) {
    return amount => {
                if (balance >= amount) {
                    balance = balance - amount;
                    return balance;
                } else {
                    return "Insufficient funds";
                }
            };
}
```

The function `make_withdraw` can be used as follows to create two objects `W1` and `W2`:

```
const W1 = make_withdraw(100);
const W2 = make_withdraw(100);

W1(50);
50

W2(70);
30

W2(40);
"Insufficient funds"

W1(40);
10
```

Observe that `W1` and `W2` are completely independent objects, each with its own local state variable `balance`. Withdrawals from one do not affect the other.

We can also create objects that handle deposits as well as withdrawals, and thus we can represent simple bank accounts. Here is a function that returns a "bank-account object" with a specified initial balance:

---

6. In contrast with `make_withdraw_balance_100` above, we do not have to use `let` to make `balance` a local variable, since parameters are already local. This will be clearer after the discussion of the environment model of evaluation in section 3.2. (See also exercise 3.10.)

```
function make_account(balance) {
    function withdraw(amount) {
        if (balance >= amount) {
            balance = balance - amount;
            return balance;
        } else {
            return "Insufficient funds";
        }
    }
    function deposit(amount) {
        balance = balance + amount;
        return balance;
    }
    function dispatch(m) {
        return m === "withdraw"
               ? withdraw
               : m === "deposit"
               ? deposit
               : error(m, "unknown request -- make_account");
    }
    return dispatch;
}
```

Each call to `make_account` sets up an environment with a local state variable `balance`. Within this environment, `make_account` defines functions `deposit` and `withdraw` that access `balance` and an additional function `dispatch` that takes a "message" as input and returns one of the two local functions. The `dispatch` function itself is returned as the value that represents the bank-account object. This is precisely the *message-passing* style of programming that we saw in section 2.4.3, although here we are using it in conjunction with the ability to modify local variables.

The function `make_account` can be used as follows:

```
const acc = make_account(100);

acc("withdraw")(50);
50

acc("withdraw")(60);
"Insufficient funds"

acc("deposit")(40);
90

acc("withdraw")(60);
30
```

Each call to `acc` returns the locally defined `deposit` or `withdraw` function, which is then applied to the specified `amount`. As was the case with `make_withdraw`, another call to `make_account`

```
const acc2 = make_account(100);
```

will produce a completely separate account object, which maintains its own local `balance`.

Exercise 3.1

An *accumulator* is a function that is called repeatedly with a single numeric argument and accumulates its arguments into a sum. Each time it is called, it returns the currently accumulated sum. Write a function `make_accumulator` that generates accumulators, each maintaining an independent sum. The input to `make_accumulator` should specify the initial value of the sum; for example

```
const a = make_accumulator(5);

a(10);
15

a(10);
25
```

Exercise 3.2

In software-testing applications, it is useful to be able to count the number of times a given function is called during the course of a computation. Write a function `make_monitored` that takes as input a function, `f`, that itself takes one input. The result returned by `make_monitored` is a third function, say `mf`, that keeps track of the number of times it has been called by maintaining an internal counter. If the input to `mf` is the string `"how many calls"`, then `mf` returns the value of the counter. If the input is the string `"reset count"`, then `mf` resets the counter to zero. For any other input, `mf` returns the result of calling `f` on that input and increments the counter. For instance, we could make a monitored version of the `sqrt` function:

```
const s = make_monitored(math_sqrt);

s(100);
10

s("how many calls");
1
```

Exercise 3.3

Modify the `make_account` function so that it creates password-protected accounts. That is, `make_account` should take a string as an additional argument, as in

```
const acc = make_account(100, "secret password");
```

The resulting account object should process a request only if it is accompanied by the password with which the account was created, and should otherwise return a complaint:

```
acc("secret password", "withdraw")(40);
60

acc("some other password", "deposit")(40);
"Incorrect password"
```

Exercise 3.4

Modify the `make_account` function of exercise 3.3 by adding another local state variable so that, if an account is accessed more than seven consecutive times with an incorrect password, it invokes the function `call_the_cops`.

3.1.2 The Benefits of Introducing Assignment

As we shall see, introducing assignment into our programming language leads us into a thicket of difficult conceptual issues. Nevertheless, viewing systems as collections of objects with local state is a powerful technique for maintaining a modular design. As a simple example, consider the design of a function `rand` that, whenever it is called, returns an integer chosen at random.

It is not at all clear what is meant by "chosen at random." What we presumably want is for successive calls to `rand` to produce a sequence of numbers that has statistical properties of uniform distribution. We will not discuss methods for generating suitable sequences here. Rather, let us assume that we have a function `rand_update` that has the property that if we start with a given number x_1 and form

```
x2 = rand_update(x1);
x3 = rand_update(x2);
```

then the sequence of values $x_1, x_2, x_3, \ldots$, will have the desired statistical properties.[7]

We can implement `rand` as a function with a local state variable `x` that is initialized to some fixed value `random_init`. Each call to `rand` computes `rand_update` of the current value of `x`, returns this as the random number, and also stores this as the new value of `x`.

```
function make_rand() {
   let x = random_init;
   return () => {
               x = rand_update(x);
               return x;
          };
}
const rand = make_rand();
```

7. One common way to implement `rand_update` is to use the rule that x is updated to $ax+b$ modulo m, where a, b, and m are appropriately chosen integers. Chapter 3 of Knuth 1997b includes an extensive discussion of techniques for generating sequences of random numbers and establishing their statistical properties. Notice that the `rand_update` function computes a mathematical function: Given the same input twice, it produces the same output. Therefore, the number sequence produced by `rand_update` certainly is not "random," if by "random" we insist that each number in the sequence is unrelated to the preceding number. The relation between "real randomness" and so-called *pseudo-random* sequences, which are produced by well-determined computations and yet have suitable statistical properties, is a complex question involving difficult issues in mathematics and philosophy. Kolmogorov, Solomonoff, and Chaitin have made great progress in clarifying these issues; a discussion can be found in Chaitin 1975.

Of course, we could generate the same sequence of random numbers without using assignment by simply calling `rand_update` directly. However, this would mean that any part of our program that used random numbers would have to explicitly remember the current value of `x` to be passed as an argument to `rand_update`. To realize what an annoyance this would be, consider using random numbers to implement a technique called *Monte Carlo simulation*.

The Monte Carlo method consists of choosing sample experiments at random from a large set and then making deductions on the basis of the probabilities estimated from tabulating the results of those experiments. For example, we can approximate π using the fact that $6/\pi^2$ is the probability that two integers chosen at random will have no factors in common; that is, that their greatest common divisor will be 1.[8] To obtain the approximation to π, we perform a large number of experiments. In each experiment we choose two integers at random and perform a test to see if their GCD is 1. The fraction of times that the test is passed gives us our estimate of $6/\pi^2$, and from this we obtain our approximation to π.

The heart of our program is a function `monte_carlo`, which takes as arguments the number of times to try an experiment, together with the experiment, represented as a no-argument function that will return either true or false each time it is run. The function `monte_carlo` runs the experiment for the designated number of trials and returns a number telling the fraction of the trials in which the experiment was found to be true.

```
function estimate_pi(trials) {
    return math_sqrt(6 / monte_carlo(trials, dirichlet_test));
}
function dirichlet_test() {
    return gcd(rand(), rand()) === 1;
}
function monte_carlo(trials, experiment) {
    function iter(trials_remaining, trials_passed) {
        return trials_remaining === 0
               ? trials_passed / trials
               : experiment()
               ? iter(trials_remaining - 1, trials_passed + 1)
               : iter(trials_remaining - 1, trials_passed);
    }
    return iter(trials, 0);
}
```

Now let us try the same computation using `rand_update` directly rather than `rand`, the way we would be forced to proceed if we did not use assignment to model local state:

---

8. This theorem is due to G. Lejeune Dirichlet. See section 4.5.2 of Knuth 1997b for a discussion and a proof.

```
function estimate_pi(trials) {
    return math_sqrt(6 / random_gcd_test(trials, random_init));
}
function random_gcd_test(trials, initial_x) {
    function iter(trials_remaining, trials_passed, x) {
        const x1 = rand_update(x);
        const x2 = rand_update(x1);
        return trials_remaining === 0
               ? trials_passed / trials
               : gcd(x1, x2) === 1
               ? iter(trials_remaining - 1, trials_passed + 1, x2)
               : iter(trials_remaining - 1, trials_passed, x2);
    }
    return iter(trials, 0, initial_x);
}
```

While the program is still simple, it betrays some painful breaches of modularity. In our first version of the program, using `rand`, we can express the Monte Carlo method directly as a general `monte_carlo` function that takes as an argument an arbitrary `experiment` function. In our second version of the program, with no local state for the random-number generator, `random_gcd_test` must explicitly manipulate the random numbers `x1` and `x2` and recycle `x2` through the iterative loop as the new input to `rand_update`. This explicit handling of the random numbers intertwines the structure of accumulating test results with the fact that our particular experiment uses two random numbers, whereas other Monte Carlo experiments might use one random number or three. Even the top-level function `estimate_pi` has to be concerned with supplying an initial random number. The fact that the random-number generator's insides are leaking out into other parts of the program makes it difficult for us to isolate the Monte Carlo idea so that it can be applied to other tasks. In the first version of the program, assignment encapsulates the state of the random-number generator within the `rand` function, so that the details of random-number generation remain independent of the rest of the program.

The general phenomenon illustrated by the Monte Carlo example is this: From the point of view of one part of a complex process, the other parts appear to change with time. They have hidden time-varying local state. If we wish to write computer programs whose structure reflects this decomposition, we make computational objects (such as bank accounts and random-number generators) whose behavior changes with time. We model state with local state variables, and we model the changes of state with assignments to those variables.

It is tempting to conclude this discussion by saying that, by introducing assignment and the technique of hiding state in local variables, we are able to structure systems in a more modular fashion than if all state had to be manipulated explicitly, by passing additional parameters. Unfortunately, as we shall see, the story is not so simple.

Exercise 3.5

Monte Carlo integration is a method of estimating definite integrals by means of Monte Carlo simulation. Consider computing the area of a region of space described by a predicate $P(x, y)$ that is true for points (x, y) in the region and false for points not in the region. For example, the region contained within a circle of radius 3 centered at $(5, 7)$ is described by the predicate that tests whether $(x-5)^2 + (y-7)^2 \leq 3^2$. To estimate the area of the region described by such a predicate, begin by choosing a rectangle that contains the region. For example, a rectangle with diagonally opposite corners at $(2, 4)$ and $(8, 10)$ contains the circle above. The desired integral is the area of that portion of the rectangle that lies in the region. We can estimate the integral by picking, at random, points (x, y) that lie in the rectangle, and testing $P(x, y)$ for each point to determine whether the point lies in the region. If we try this with many points, then the fraction of points that fall in the region should give an estimate of the proportion of the rectangle that lies in the region. Hence, multiplying this fraction by the area of the entire rectangle should produce an estimate of the integral.

Implement Monte Carlo integration as a function `estimate_integral` that takes as arguments a predicate `P`, upper and lower bounds `x1`, `x2`, `y1`, and `y2` for the rectangle, and the number of trials to perform in order to produce the estimate. Your function should use the same `monte_carlo` function that was used above to estimate π. Use your `estimate_integral` to produce an estimate of π by measuring the area of a unit circle.

You will find it useful to have a function that returns a number chosen at random from a given range. The following `random_in_range` function implements this in terms of the `math_random` function used in section 1.2.6, which returns a nonnegative number less than 1.

```
function random_in_range(low, high) {
    const range = high - low;
    return low + math_random() * range;
}
```

Exercise 3.6

It is useful to be able to reset a random-number generator to produce a sequence starting from a given value. Design a new `rand` function that is called with an argument that is either the string `"generate"` or the string `"reset"` and behaves as follows: `rand("generate")` produces a new random number; `rand("reset")(`*new-value*`)` resets the internal state variable to the designated *new-value*. Thus, by resetting the state, one can generate repeatable sequences. These are very handy to have when testing and debugging programs that use random numbers.

3.1.3 The Costs of Introducing Assignment

As we have seen, assignment enables us to model objects that have local state. However, this advantage comes at a price. Our programming language can no longer be interpreted in terms of the substitution model of function application that we introduced in section 1.1.5. Moreover, no simple model with "nice" mathematical properties can be an adequate framework for dealing with objects and assignment in programming languages.

So long as we do not use assignments, two evaluations of the same function with the same arguments will produce the same result, so that functions can be viewed as computing mathematical functions. Programming without any use of assignments, as we did throughout the first two chapters of this book, is accordingly known as *functional programming*.

To understand how assignment complicates matters, consider a simplified version of the `make_withdraw` function of section 3.1.1 that does not bother to check for an insufficient amount:

```
function make_simplified_withdraw(balance) {
    return amount => {
                balance = balance - amount;
                return balance;
            };
}

const W = make_simplified_withdraw(25);

W(20);
5

W(10);
-5
```

Compare this function with the following `make_decrementer` function, which does not use assignment:

```
function make_decrementer(balance) {
    return amount => balance - amount;
}
```

The function `make_decrementer` returns a function that subtracts its input from a designated amount `balance`, but there is no accumulated effect over successive calls, as with `make_simplified_withdraw`:

```
const D = make_decrementer(25);

D(20);
5

D(10);
15
```

We can use the substitution model to explain how `make_decrementer` works. For instance, let us analyze the evaluation of the expression

```
make_decrementer(25)(20)
```

We first simplify the function expression of the application by substituting 25 for `balance` in the body of `make_decrementer`. This reduces the expression to

```
(amount => 25 - amount)(20)
```

Now we apply the function by substituting 20 for `amount` in the body of the lambda expression:

```
25 - 20
```

The final answer is 5.

Observe, however, what happens if we attempt a similar substitution analysis with `make_simplified_withdraw`:

```
make_simplified_withdraw(25)(20)
```

We first simplify the function expression by substituting 25 for `balance` in the body of `make_simplified_withdraw`. This reduces the expression to[9]

```
(amount => {
     balance = 25 - amount;
     return 25;
 })(20)
```

Now we apply the function by substituting 20 for `amount` in the body of the lambda expression:

```
balance = 25 - 20;
return 25;
```

If we adhered to the substitution model, we would have to say that the meaning of the function application is to first set `balance` to 5 and then return 25 as the value of the expression. This gets the wrong answer. In order to get the correct answer, we would have to somehow distinguish the first occurrence of `balance` (before the effect of the assignment) from the second occurrence of `balance` (after the effect of the assignment), and the substitution model cannot do this.

The trouble here is that substitution is based ultimately on the notion that the names in our language are essentially symbols for values. This worked well for constants. But a variable, whose value can change with assignment, cannot simply be a name for a value. A variable somehow refers to a place where a value can be stored, and the value stored at this place can change. In section 3.2 we will see how environments play this role of "place" in our computational model.

Sameness and change

The issue surfacing here is more profound than the mere breakdown of a particular model of computation. As soon as we introduce change into our computational models, many notions that were previously straightforward become problematical. Consider the concept of two things being "the same."

Suppose we call `make_decrementer` twice with the same argument to create two functions:

```
const D1 = make_decrementer(25);
const D2 = make_decrementer(25);
```

Are `D1` and `D2` the same? An acceptable answer is yes, because `D1` and `D2` have the same computational behavior—each is a function that subtracts its input from 25. In fact, `D1` could be substituted for `D2` in any computation without changing the result.

---

9. We don't substitute for the occurrence of `balance` in the assignment because the name in an assignment is not evaluated. If we did substitute for it, we would get `25 = 25 - amount;`, which makes no sense.

Contrast this with making two calls to `make_simplified_withdraw`:

```
const W1 = make_simplified_withdraw(25);
const W2 = make_simplified_withdraw(25);
```

Are `W1` and `W2` the same? Surely not, because calls to `W1` and `W2` have distinct effects, as shown by the following sequence of interactions:

```
W1(20);
5

W1(20);
-15

W2(20);
5
```

Even though `W1` and `W2` are "equal" in the sense that they are both created by evaluating the same expression, `make_simplified_withdraw(25)`, it is not true that `W1` could be substituted for `W2` in any expression without changing the result of evaluating the expression.

A language that supports the concept that "equals can be substituted for equals" in an expression without changing the value of the expression is said to be *referentially transparent*. Referential transparency is violated when we include assignment in our computer language. This makes it tricky to determine when we can simplify expressions by substituting equivalent expressions. Consequently, reasoning about programs that use assignment becomes drastically more difficult.

Once we forgo referential transparency, the notion of what it means for computational objects to be "the same" becomes difficult to capture in a formal way. Indeed, the meaning of "same" in the real world that our programs model is hardly clear in itself. In general, we can determine that two apparently identical objects are indeed "the same one" only by modifying one object and then observing whether the other object has changed in the same way. But how can we tell if an object has "changed" other than by observing the "same" object twice and seeing whether some property of the object differs from one observation to the next? Thus, we cannot determine "change" without some a priori notion of "sameness," and we cannot determine sameness without observing the effects of change.

As an example of how this issue arises in programming, consider the situation where Peter and Paul have a bank account with $100 in it. There is a substantial difference between modeling this as

```
const peter_acc = make_account(100);
const paul_acc = make_account(100);
```

and modeling it as

```
const peter_acc = make_account(100);
const paul_acc = peter_acc;
```

In the first situation, the two bank accounts are distinct. Transactions made by Peter will not affect Paul's account, and vice versa. In the second situation, however, we have defined `paul_acc` to be *the same thing* as `peter_acc`. In effect, Peter and Paul

now have a joint bank account, and if Peter makes a withdrawal from `peter_acc` Paul will observe less money in `paul_acc`. These two similar but distinct situations can cause confusion in building computational models. With the shared account, in particular, it can be especially confusing that there is one object (the bank account) that has two different names (`peter_acc` and `paul_acc`); if we are searching for all the places in our program where `paul_acc` can be changed, we must remember to look also at things that change `peter_acc`.[10]

With reference to the above remarks on "sameness" and "change," observe that if Peter and Paul could only examine their bank balances, and could not perform operations that changed the balance, then the issue of whether the two accounts are distinct would be moot. In general, so long as we never modify data objects, we can regard a compound data object to be precisely the totality of its pieces. For example, a rational number is determined by giving its numerator and its denominator. But this view is no longer valid in the presence of change, where a compound data object has an "identity" that is something different from the pieces of which it is composed. A bank account is still "the same" bank account even if we change the balance by making a withdrawal; conversely, we could have two different bank accounts with the same state information. This complication is a consequence, not of our programming language, but of our perception of a bank account as an object. We do not, for example, ordinarily regard a rational number as a changeable object with identity, such that we could change the numerator and still have "the same" rational number.

Pitfalls of imperative programming

In contrast to functional programming, programming that makes extensive use of assignment is known as *imperative programming*. In addition to raising complications about computational models, programs written in imperative style are susceptible to bugs that cannot occur in functional programs. For example, recall the iterative factorial program from section 1.2.1 (here using a conditional statement instead of a conditional expression):

```
function factorial(n) {
    function iter(product, counter) {
        if (counter > n) {
            return product;
        } else {
            return iter(counter * product,
                        counter + 1);
        }
    }
    return iter(1, 1);
}
```

10. The phenomenon of a single computational object being accessed by more than one name is known as *aliasing*. The joint bank account situation illustrates a very simple example of an alias. In section 3.3 we will see much more complex examples, such as "distinct" compound data structures that share parts. Bugs can occur in our programs if we forget that a change to an object may also, as a "side effect," change a "different" object because the two "different" objects are actually a single object appearing under different aliases. These so-called *side-effect bugs* are so difficult to locate and to analyze that some people have proposed that programming languages be designed in such a way as to not allow side effects or aliasing (Lampson et al. 1981; Morris, Schmidt, and Wadler 1980).

Instead of passing arguments in the internal iterative loop, we could adopt a more imperative style by using explicit assignment to update the values of the variables `product` and `counter`:

```
function factorial(n) {
    let product = 1;
    let counter = 1;
    function iter() {
        if (counter > n) {
            return product;
        } else {
            product = counter * product;
            counter = counter + 1;
            return iter();
        }
    }
    return iter();
}
```

This does not change the results produced by the program, but it does introduce a subtle trap. How do we decide the order of the assignments? As it happens, the program is correct as written. But writing the assignments in the opposite order

```
counter = counter + 1;
product = counter * product;
```

would have produced a different, incorrect result. In general, programming with assignment forces us to carefully consider the relative orders of the assignments to make sure that each statement is using the correct version of the variables that have been changed. This issue simply does not arise in functional programs.[11]

The complexity of imperative programs becomes even worse if we consider applications in which several processes execute concurrently. We will return to this in section 3.4. First, however, we will address the issue of providing a computational model for expressions that involve assignment, and explore the uses of objects with local state in designing simulations.

Exercise 3.7

Consider the bank account objects created by `make_account`, with the password modification described in exercise 3.3. Suppose that our banking system requires the ability to make joint accounts. Define a function `make_joint` that accomplishes this. The function `make_joint` should take three arguments. The first is a password-protected account. The second argument must match the password with which the account was defined in order for the `make_joint` operation to proceed. The third argument is a new password. The function

11. In view of this, it is ironic that introductory programming is most often taught in a highly imperative style. This may be a vestige of a belief, common throughout the 1960s and 1970s, that programs that call functions must inherently be less efficient than programs that perform assignments. (Steele (1977) debunks this argument.) Alternatively it may reflect a view that step-by-step assignment is easier for beginners to visualize than function call. Whatever the reason, it often saddles beginning programmers with "should I set this variable before or after that one" concerns that can complicate programming and obscure the important ideas.

`make_joint` is to create an additional access to the original account using the new password. For example, if `peter_acc` is a bank account with password `"open sesame"`, then

```
const paul_acc = make_joint(peter_acc, "open sesame", "rosebud");
```

will allow one to make transactions on `peter_acc` using the name `paul_acc` and the password `"rosebud"`. You may wish to modify your solution to exercise 3.3 to accommodate this new feature.

Exercise 3.8

When we defined the evaluation model in section 1.1.3, we said that the first step in evaluating an expression is to evaluate its subexpressions. But we never specified the order in which the subexpressions should be evaluated (e.g., left to right or right to left). When we introduce assignment, the order in which the operands of an operator combination are evaluated can make a difference to the result. Define a simple function `f` such that evaluating `f(0) + f(1)` will return 0 if the operands of `+` are evaluated from left to right but will return 1 if the operands are evaluated from right to left.

3.2 The Environment Model of Evaluation

When we introduced compound functions in chapter 1, we used the substitution model of evaluation (section 1.1.5) to define what is meant by applying a function to arguments:

- To apply a compound function to arguments, evaluate the return expression of the function (more generally, the body) with each parameter replaced by the corresponding argument.

Once we admit assignment into our programming language, such a definition is no longer adequate. In particular, section 3.1.3 argued that, in the presence of assignment, a name cannot be considered to be merely representing a value. Rather, a name must somehow designate a "place" in which values can be stored. In our new model of evaluation, these places will be maintained in structures called *environments*.

An environment is a sequence of *frames*. Each frame is a table (possibly empty) of *bindings*, which associate names with their corresponding values. (A single frame may contain at most one binding for any name.) Each frame also has a pointer to its *enclosing environment*, unless, for the purposes of discussion, the frame is considered to be *global*. The *value of a name* with respect to an environment is the value given by the binding of the name in the first frame in the environment that contains a binding for that name. If no frame in the sequence specifies a binding for the name, then the name is said to be *unbound* in the environment.

Figure 3.1 shows a simple environment structure consisting of three frames, labeled I, II, and III. In the diagram, A, B, C, and D are pointers to environments. C and D point to the same environment. The names `z` and `x` are bound in frame II, while `y` and `x` are bound in frame I. The value of `x` in environment D is 3. The value of `x` with respect to environment B is also 3. This is determined as follows: We examine the first frame in the sequence (frame III) and do not find a binding for `x`, so we proceed to the enclosing environment D and find the binding in frame I. On the other hand, the value of `x` in environment A is 7, because the first frame in the

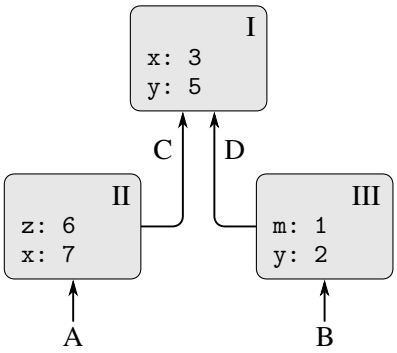


Figure 3.1 A simple environment structure.

sequence (frame II) contains a binding of `x` to 7. With respect to environment A, the binding of `x` to 7 in frame II is said to *shadow* the binding of `x` to 3 in frame I.

The environment is crucial to the evaluation process, because it determines the context in which an expression should be evaluated. Indeed, one could say that expressions in a programming language do not, in themselves, have any meaning. Rather, an expression acquires a meaning only with respect to some environment in which it is evaluated. Even the interpretation of an expression as straightforward as `display(1)` depends on an understanding that one is operating in a context in which the name `display` refers to the primitive function that displays a value. Thus, in our model of evaluation we will always speak of evaluating an expression with respect to some environment. To describe interactions with the interpreter, we will suppose that there is a global environment, consisting of a single frame (with no enclosing environment) that includes values for the names associated with the primitive functions. For example, the idea that `display` is the name for the primitive display function is captured by saying that the name `display` is bound in the global environment to the primitive display function.

Before we evaluate a program, we extend the global environment with a new frame, the *program frame*, resulting in the *program environment*. We will add the names that are declared at the top level of the program, outside of any block, to this frame. The given program is then evaluated with respect to the program environment.

3.2.1 The Rules for Evaluation

The overall specification of how the interpreter evaluates a function application remains the same as when we first introduced it in section 1.1.4:

- To evaluate an application:
  1. Evaluate the subexpressions of the application.[12]

12. Assignment introduces a subtlety into step 1 of the evaluation rule. As shown in exercise 3.8, the presence of assignment allows us to write expressions that will produce different values depending on the order in which the subexpressions in a combination are evaluated. To eliminate such ambiguities, JavaScript specifies left-to-right evaluation of the subexpressions of combinations and of the argument expressions of applications.

2. Apply the value of the function subexpression to the values of the argument subexpressions.

The environment model of evaluation replaces the substitution model in specifying what it means to apply a compound function to arguments.

In the environment model of evaluation, a function is always a pair consisting of some code and a pointer to an environment. Functions are created in one way only: by evaluating a lambda expression. This produces a function whose code is obtained from the text of the lambda expression and whose environment is the environment in which the lambda expression was evaluated to produce the function. For example, consider the function declaration

```
function square(x) {
    return x * x;
}
```

evaluated in the program environment. The function declaration syntax is equivalent to an underlying implicit lambda expression. It would have been equivalent to have used[13]

```
const square = x => x * x;
```

which evaluates `x => x * x` and binds `square` to the resulting value, all in the program environment.

Figure 3.2 shows the result of evaluating this declaration statement. The global environment encloses the program environment. To reduce clutter, after this figure we will not display the global environment (as it is always the same), but we are reminded of its existence by the pointer from the program environment upward. The function object is a pair whose code specifies that the function has one parameter, namely `x`, and a function body `return x * x;`. The environment part of the function is a pointer to the program environment, since that is the environment in which the lambda expression was evaluated to produce the function. A new binding, which associates the function object with the name `square`, has been added to the program frame.

In general, `const`, `function`, and `let` add bindings to frames. Assignment is forbidden on constants, so our environment model needs to distinguish names that refer to constants from names that refer to variables. We indicate that a name is a constant by writing an equal sign after the colon that follows the name. We consider function declarations as equivalent to constant declarations;[14] observe the equal signs after the colons in figure 3.2.

Now that we have seen how functions are created, we can describe how functions are applied. The environment model specifies: To apply a function to arguments, create a new environment containing a frame that binds the parameters to the values of

13. Footnote 55 in chapter 1 mentions subtle differences between the two in full JavaScript, which we will ignore in this book.

14. We mentioned in footnote 55 in chapter 1 that the full JavaScript language allows assignment to names that are declared with function declarations.

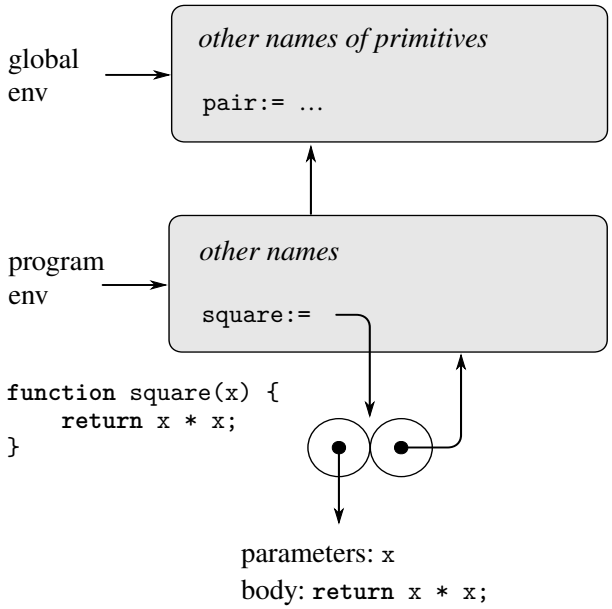


Figure 3.2 Environment structure produced by evaluating `function square(x) { return x * x; }` in the program environment.

the arguments. The enclosing environment of this frame is the environment specified by the function. Now, within this new environment, evaluate the function body.

To show how this rule is followed, figure 3.3 illustrates the environment structure created by evaluating the expression `square(5)` in the program environment, where `square` is the function generated in figure 3.2. Applying the function results in the creation of a new environment, labeled E1 in the figure, that begins with a frame in which `x`, the parameter for the function, is bound to the argument 5. Note that name `x` in environment E1 is followed by a colon with no equal sign, which indicates that the parameter `x` is treated as a variable.[15] The pointer leading upward from this frame shows that the frame's enclosing environment is the program environment. The program environment is chosen here, because this is the environment that is indicated as part of the `square` function object. Within E1, we evaluate the body of the function, `return x * x;`. Since the value of `x` in E1 is 5, the result is `5 * 5`, or 25.

The environment model of function application can be summarized by two rules:

- A function object is applied to a set of arguments by constructing a frame, binding the parameters of the function to the arguments of the call, and then evaluating the body of the function in the context of the new environment constructed. The new frame has as its enclosing environment the environment part of the function object being applied. The result of the application is the result of evaluating the return

15. This example does not make use of the fact that the parameter `x` is a variable, but recall the function `make_withdraw` in section 3.1.1, which relied on its parameter being a variable.

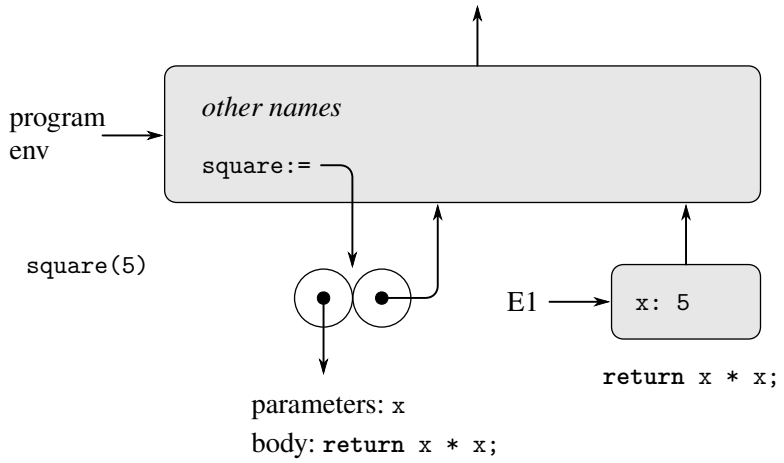


Figure 3.3 Environment created by evaluating `square(5)` in the program environment.

expression of the first return statement encountered while evaluating the function body.

- A function is created by evaluating a lambda expression relative to a given environment. The resulting function object is a pair consisting of the text of the lambda expression and a pointer to the environment in which the function was created.

Finally, we specify the behavior of assignment, the operation that forced us to introduce the environment model in the first place. Evaluating the expression *name* = *value* in some environment locates the binding of the name in the environment. That is, one finds the first frame in the environment that contains a binding for the name. If the binding is a variable binding—indicated in the frame by just `:` after the name—that binding is changed to reflect the new value of the variable. Otherwise, if the binding in the frame is a constant binding—indicated in the frame by `:=` after the name—the assignment signals an `"assignment to constant"` error. If the name is unbound in the environment, then the assignment signals a `"variable undeclared"` error.

These evaluation rules, though considerably more complex than the substitution model, are still reasonably straightforward. Moreover, the evaluation model, though abstract, provides a correct description of how the interpreter evaluates expressions. In chapter 4 we shall see how this model can serve as a blueprint for implementing a working interpreter. The following sections elaborate the details of the model by analyzing some illustrative programs.

3.2.2 Applying Simple Functions

When we introduced the substitution model in section 1.1.5 we showed how the application `f(5)` evaluates to 136, given the following function declarations:

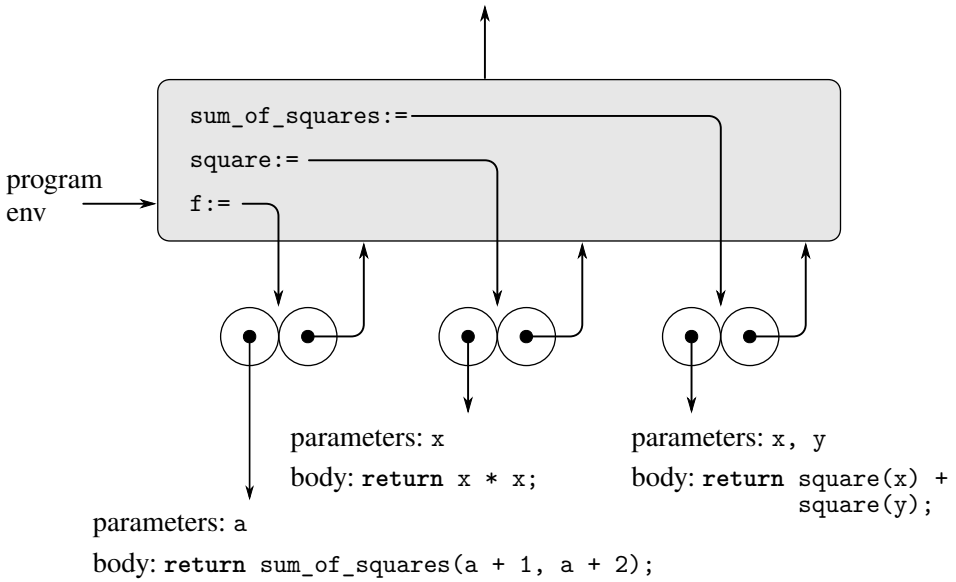


Figure 3.4 Function objects in the program frame.

```
function square(x) {
    return x * x;
}
function sum_of_squares(x, y) {
    return square(x) + square(y);
}
function f(a) {
    return sum_of_squares(a + 1, a * 2);
}
```

We can analyze the same example using the environment model. Figure 3.4 shows the three function objects created by evaluating the definitions of `f`, `square`, and `sum_of_squares` in the program environment. Each function object consists of some code, together with a pointer to the program environment.

In figure 3.5 we see the environment structure created by evaluating the expression `f(5)`. The call to `f` creates a new environment, E1, beginning with a frame in which `a`, the parameter of `f`, is bound to the argument 5. In E1, we evaluate the body of `f`:

```
return sum_of_squares(a + 1, a * 2);
```

To evaluate the return statement, we first evaluate the subexpressions of the return expression. The first subexpression, `sum_of_squares`, has a value that is a function object. (Notice how this value is found: We first look in the first frame of E1, which contains no binding for `sum_of_squares`. Then we proceed to the enclosing environment, i.e., the program environment, and find the binding shown in figure 3.4.) The

other two subexpressions are evaluated by applying the primitive operations + and * to evaluate the two combinations `a + 1` and `a * 2` to obtain 6 and 10, respectively.

Now we apply the function object `sum_of_squares` to the arguments 6 and 10. This results in a new environment, E2, in which the parameters `x` and `y` are bound to the arguments. Within E2 we evaluate the statement

```
return square(x) + square(y);
```

This leads us to evaluate `square(x)`, where `square` is found in the program frame and `x` is 6. Once again, we set up a new environment, E3, in which `x` is bound to 6, and within this we evaluate the body of `square`, which is `return x * x;`. Also as part of applying `sum_of_squares`, we must evaluate the subexpression `square(y)`, where `y` is 10. This second call to `square` creates another environment, E4, in which `x`, the parameter of `square`, is bound to 10. And within E4 we must evaluate `return x * x;`.

The important point to observe is that each call to `square` creates a new environment containing a binding for `x`. We can see here how the different frames serve to keep separate the different local variables all named `x`. Notice that each frame created by `square` points to the program environment, since this is the environment indicated by the `square` function object.

After the subexpressions are evaluated, the results are returned. The values generated by the two calls to `square` are added by `sum_of_squares`, and this result is returned by `f`. Since our focus here is on the environment structures, we will not dwell on how these returned values are passed from call to call; however, this is also an important aspect of the evaluation process, and we will return to it in detail in chapter 5.

Exercise 3.9

In section 1.2.1 we used the substitution model to analyze two functions for computing factorials, a recursive version

```
function factorial(n) {
    return n === 1
           ? 1
           : n * factorial(n - 1);
}
```

and an iterative version

```
function factorial(n) {
    return fact_iter(1, 1, n);
}
function fact_iter(product, counter, max_count) {
    return counter > max_count
           ? product
           : fact_iter(counter * product,
                       counter + 1,
                       max_count);
}
```

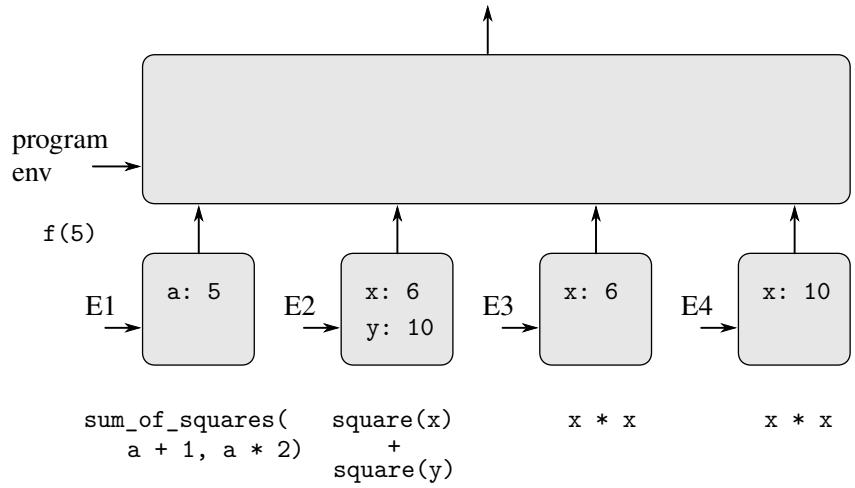


Figure 3.5 Environments created by evaluating `f(5)` using the functions in figure 3.4.

Show the environment structures created by evaluating `factorial(6)` using each version of the `factorial` function.[16]

3.2.3 Frames as the Repository of Local State

We can turn to the environment model to see how functions and assignment can be used to represent objects with local state. As an example, consider the "withdrawal processor" from section 3.1.1 created by calling the function

```
function make_withdraw(balance) {
    return amount => {
               if (balance >= amount) {
                   balance = balance - amount;
                   return balance;
               } else {
                   return "insufficient funds";
               }
           };
}
```

Let us describe the evaluation of

```
const W1 = make_withdraw(100);
```

followed by

```
W1(50);
50
```

16. The environment model will not clarify our claim in section 1.2.1 that the interpreter can execute a function such as `fact_iter` in a constant amount of space using tail recursion. We will discuss tail recursion when we deal with the control structure of the interpreter in section 5.4.

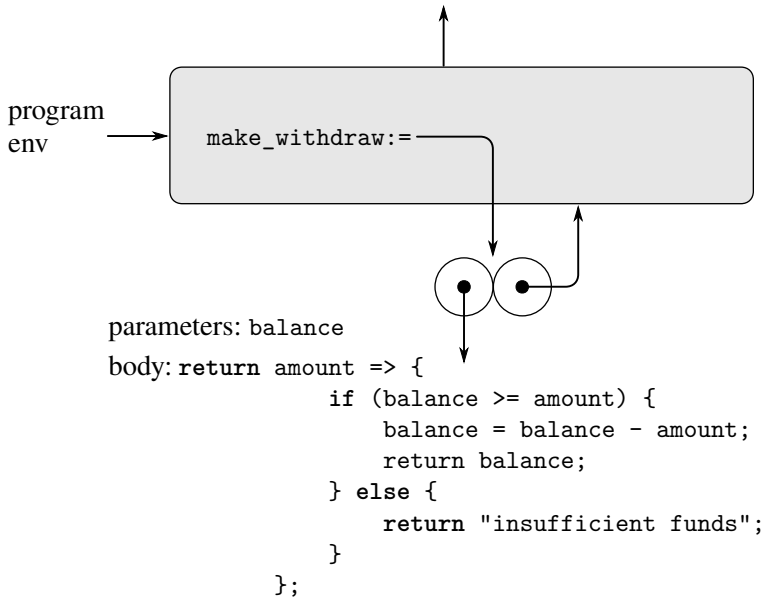


Figure 3.6 Result of defining `make_withdraw` in the program environment.

Figure 3.6 shows the result of declaring the `make_withdraw` function in the program environment. This produces a function object that contains a pointer to the program environment. So far, this is no different from the examples we have already seen, except that the return expression in the body of the function is itself a lambda expression.

The interesting part of the computation happens when we apply the function `make_withdraw` to an argument:

```
const W1 = make_withdraw(100);
```

We begin, as usual, by setting up an environment E1 in which the parameter `balance` is bound to the argument 100. Within this environment, we evaluate the body of `make_withdraw`, namely the return statement whose return expression is a lambda expression. The evaluation of this lambda expression constructs a new function object, whose code is as specified by the lambda expression and whose environment is E1, the environment in which the lambda expression was evaluated to produce the function. The resulting function object is the value returned by the call to `make_withdraw`. This is bound to `W1` in the program environment, since the constant declaration itself is being evaluated in the program environment. Figure 3.7 shows the resulting environment structure.

Now we can analyze what happens when `W1` is applied to an argument:

```
W1(50);
50
```

We begin by constructing a frame in which `amount`, the parameter of `W1`, is bound to the argument 50. The crucial point to observe is that this frame has as its enclosing

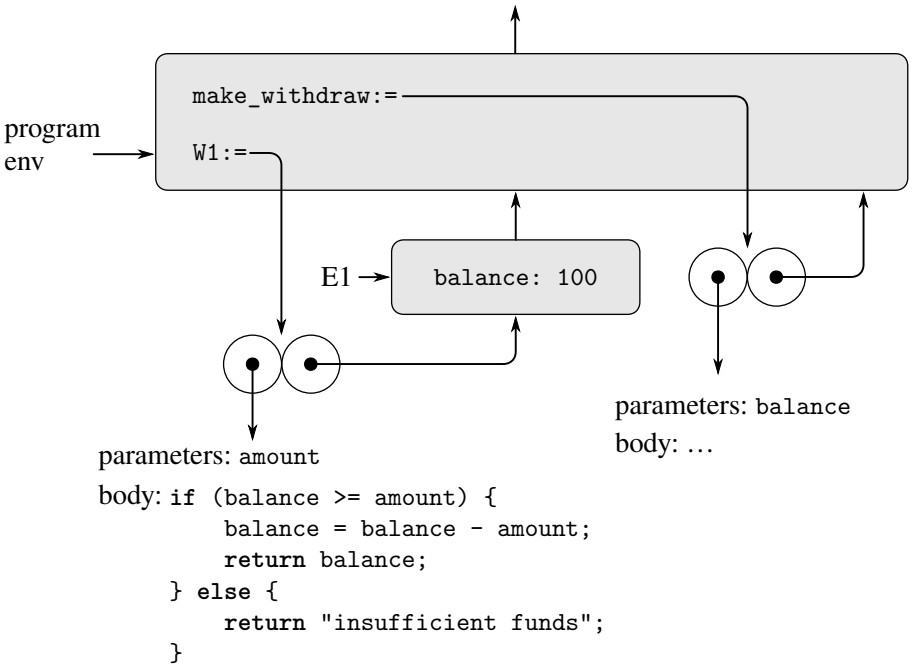


Figure 3.7 Result of evaluating `const W1 = make_withdraw(100);`.

environment not the program environment, but rather the environment E1, because this is the environment that is specified by the `W1` function object. Within this new environment, we evaluate the body of the function:

```
if (balance >= amount) {
    balance = balance - amount;
    return balance;
} else {
    return "insufficient funds";
}
```

The resulting environment structure is shown in figure 3.8. The expression being evaluated references both `amount` and `balance`. The variable `amount` will be found in the first frame in the environment, and `balance` will be found by following the enclosing-environment pointer to E1.

When the assignment is executed, the binding of `balance` in E1 is changed. At the completion of the call to `W1`, `balance` is 50, and the frame that contains `balance` is still pointed to by the function object `W1`. The frame that binds `amount` (in which we executed the code that changed `balance`) is no longer relevant, since the function call that constructed it has terminated, and there are no pointers to that frame from other parts of the environment. The next time `W1` is called, this will build a new frame that binds `amount` and whose enclosing environment is E1. We see that E1 serves as the "place" that holds the local state variable for the function object `W1`. Figure 3.9 shows the situation after the call to `W1`.

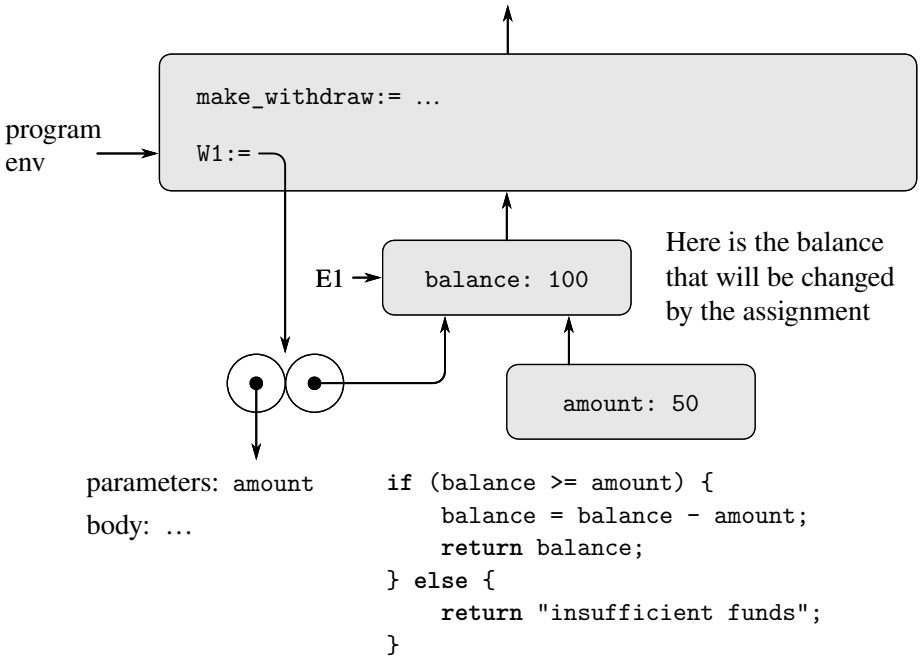


Figure 3.8 Environments created by applying the function object `W1`.

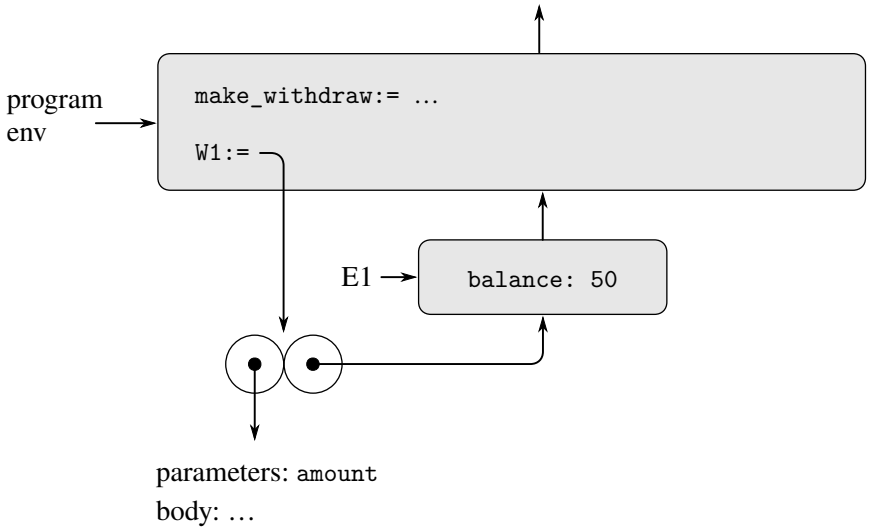


Figure 3.9 Environments after the call to `W1`.

Observe what happens when we create a second "withdraw" object by making another call to `make_withdraw`:

```
const W2 = make_withdraw(100);
```

This produces the environment structure of figure 3.10, which shows that `W2` is a function object, that is, a pair with some code and an environment. The environment E2 for `W2` was created by the call to `make_withdraw`. It contains a frame with its

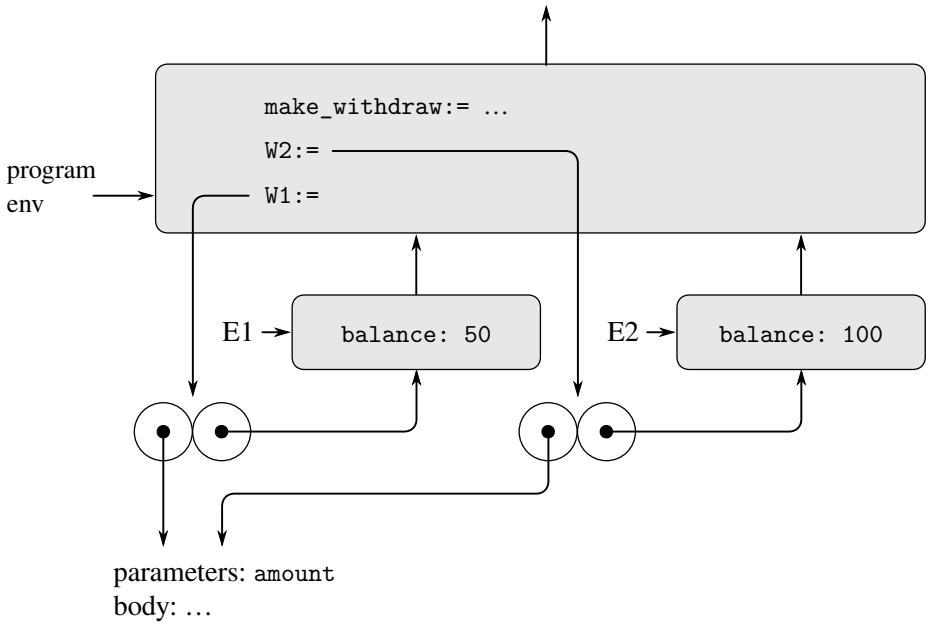


Figure 3.10 Using `const W2 = make_withdraw(100);` to create a second object.

own local binding for `balance`. On the other hand, `W1` and `W2` have the same code: the code specified by the lambda expression in the body of `make_withdraw`.[17] We see here why `W1` and `W2` behave as independent objects. Calls to `W1` reference the state variable `balance` stored in E1, whereas calls to `W2` reference the `balance` stored in E2. Thus, changes to the local state of one object do not affect the other object.

Exercise 3.10

In the `make_withdraw` function the local variable `balance` is created as a parameter of `make_withdraw`. We could also create the local state variable separately, using what we might call an *immediately invoked lambda expression* as follows:

```
function make_withdraw(initial_amount) {
    return (balance =>
              amount => {
                  if (balance >= amount) {
                      balance = balance - amount;
                      return balance;
                   } else {
                      return "insufficient funds";
                   }
              })(initial_amount);
}
```

The outer lambda expression is invoked immediately after it is evaluated. Its only purpose is to create a local variable `balance` and initialize it to `initial_amount`. Use the environment

17. Whether `W1` and `W2` share the same physical code stored in the computer, or whether they each keep a copy of the code, is a detail of the implementation. For the interpreter we implement in chapter 4, the code is in fact shared.

model to analyze this alternate version of `make_withdraw`, drawing figures like the ones above to illustrate the interactions

```
const W1 = make_withdraw(100);

W1(50);

const W2 = make_withdraw(100);
```

Show that the two versions of `make_withdraw` create objects with the same behavior. How do the environment structures differ for the two versions?

3.2.4 Internal Declarations

In this section we handle the evaluation of function bodies or other blocks (such as the branches of conditional statements) that contain declarations. Each block opens a new scope for names declared in the block. In order to evaluate a block in a given environment, we extend that environment by a new frame that contains all names declared directly (that is, outside of nested blocks) in the body of the block and then evaluate the body in the newly constructed environment.

Section 1.1.8 introduced the idea that functions can have internal declarations, thus leading to a block structure as in the following function to compute square roots:

```
function sqrt(x) {
    function is_good_enough(guess) {
        return abs(square(guess) - x) < 0.001;
    }
    function improve(guess) {
        return average(guess, x / guess);
    }
    function sqrt_iter(guess){
        return is_good_enough(guess)
               ? guess
               : sqrt_iter(improve(guess));
    }
    return sqrt_iter(1);
}
```

Now we can use the environment model to see why these internal declarations behave as desired. Figure 3.11 shows the point in the evaluation of the expression `sqrt(2)` where the internal function `is_good_enough` has been called for the first time with `guess` equal to 1.

Observe the structure of the environment. The name `sqrt` is bound in the program environment to a function object whose associated environment is the program environment. When `sqrt` was called, a new environment, E1, was formed, subordinate to the program environment, in which the parameter `x` is bound to 2. The body of `sqrt` was then evaluated in E1. That body is a block with local function declarations and therefore E1 was extended with a new frame for those declarations, resulting in the new environment E2. The body of the block was then evaluated in E2. Since the first statement in the body is

```
function is_good_enough(guess) {
    return abs(square(guess) - x) < 0.001;
}
```

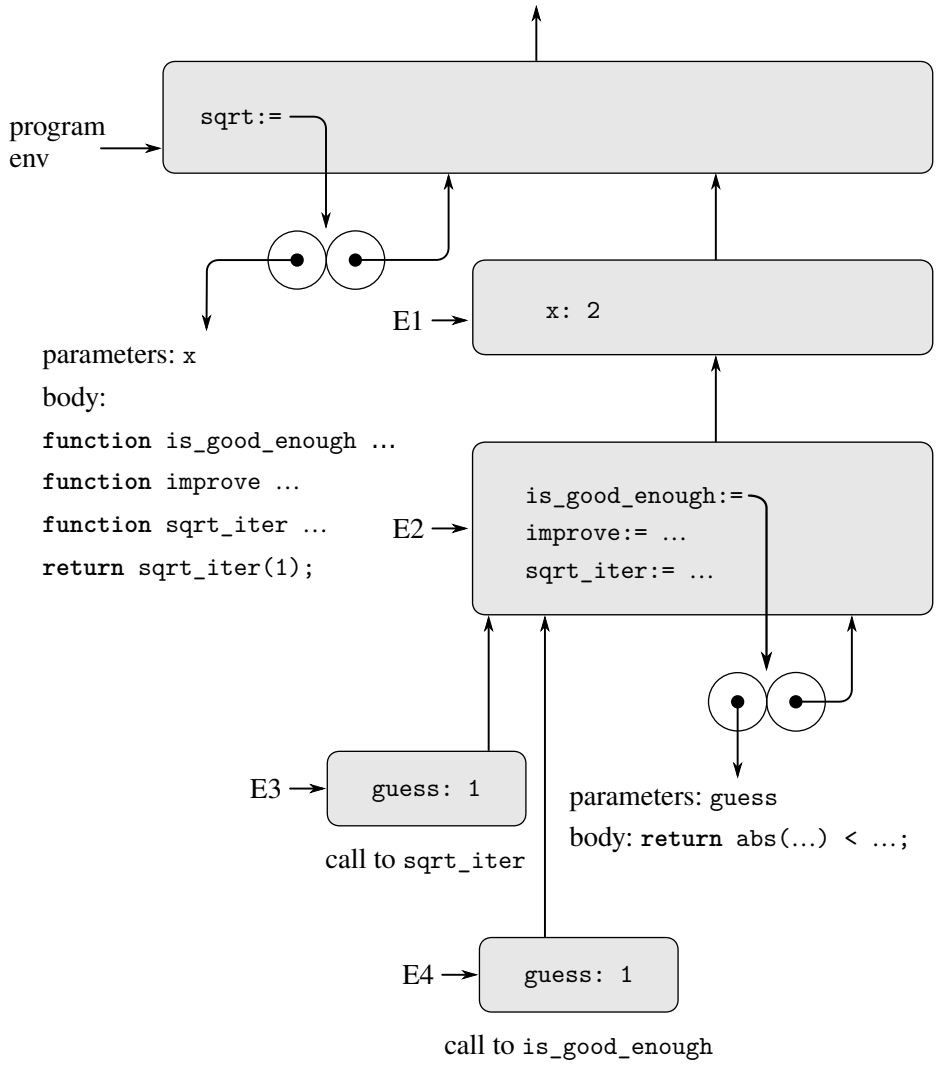


Figure 3.11 The `sqrt` function with internal declarations.

evaluating this declaration created the function `is_good_enough` in the environment E2. To be more precise, the name `is_good_enough` in the first frame of E2 was bound to a function object whose associated environment is E2. Similarly, `improve` and `sqrt_iter` were defined as functions in E2. For conciseness, figure 3.11 shows only the function object for `is_good_enough`.

After the local functions were defined, the expression `sqrt_iter(1)` was evaluated, still in environment E2. So the function object bound to `sqrt_iter` in E2 was called with 1 as an argument. This created an environment E3 in which `guess`, the parameter of `sqrt_iter`, is bound to 1. The function `sqrt_iter` in turn called `is_good_enough` with the value of `guess` (from E3) as the argument for `is_good_enough`. This set up another environment, E4, in which `guess` (the parameter of `is_good_enough`) is bound to 1. Although `sqrt_iter` and `is_good_enough` both

have a parameter named `guess`, these are two distinct local variables located in different frames. Also, E3 and E4 both have E2 as their enclosing environment, because the `sqrt_iter` and `is_good_enough` functions both have E2 as their environment part. One consequence of this is that the name `x` that appears in the body of `is_good_enough` will reference the binding of `x` that appears in E1, namely the value of `x` with which the original `sqrt` function was called.

The environment model thus explains the two key properties that make local function declarations a useful technique for modularizing programs:

- The names of the local functions do not interfere with names external to the enclosing function, because the local function names will be bound in the frame that the block creates when it is evaluated, rather than being bound in the program environment.
- The local functions can access the arguments of the enclosing function, simply by using parameter names as free names. This is because the body of the local function is evaluated in an environment that is subordinate to the evaluation environment for the enclosing function.

Exercise 3.11

In section 3.2.3, we saw how the environment model described the behavior of functions with local state. Now we have seen how internal declarations work. A typical message-passing function contains both of these aspects. Consider the bank account function of section 3.1.1:

```
function make_account(balance) {
    function withdraw(amount) {
        if (balance >= amount) {
            balance = balance - amount;
            return balance;
        } else {
            return "Insufficient funds";
        }
    }
    function deposit(amount) {
        balance = balance + amount;
        return balance;
    }
    function dispatch(m) {
        return m === "withdraw"
               ? withdraw
               : m === "deposit"
               ? deposit
               : error(m, "Unknown request: make_account");
    }
    return dispatch;
}
```

Show the environment structure generated by the sequence of interactions

```
const acc = make_account(50);
```

```
acc("deposit")(40);
90

acc("withdraw")(60);
30
```

Where is the local state for `acc` kept? Suppose we define another account

```
const acc2 = make_account(100);
```

How are the local states for the two accounts kept distinct? Which parts of the environment structure are shared between `acc` and `acc2`?

More about blocks

As we saw, the scope of the names declared in `sqrt` is the whole body of `sqrt`. This explains why *mutual recursion* works, as in this (quite wasteful) way of checking whether a nonnegative integer is even.

```
function f(x) {
            function is_even(n) {
            return n === 0
            ? true
            : is_odd(n - 1);
            }
            function is_odd(n) {
            return n === 0
            ? false
            : is_even(n - 1);
            }
            return is_even(x);
            }
```

At the time when `is_even` is called during a call to `f`, the environment diagram looks like the one in figure 3.11 when `sqrt_iter` is called. The functions `is_even` and `is_odd` are bound in E2 to function objects that point to E2 as the environment in which to evaluate calls to those functions. Thus `is_odd` in the body of `is_even` refers to the right function. Although `is_odd` is defined after `is_even`, this is no different from how in the body of `sqrt_iter` the name `improve` and the name `sqrt_iter` itself refer to the right functions.

Equipped with a way to handle declarations within blocks, we can revisit declarations of names at the top level. In section 3.2.1, we saw that the names declared at the top level are added to the program frame. A better explanation is that the whole program is placed in an implicit block, which is evaluated in the global environment. The treatment of blocks described above then handles the top level: The global environment is extended by a frame that contains the bindings of all names declared in the implicit block. That frame is the program frame and the resulting environment is the program environment.

We said that a block's body is evaluated in an environment that contains all names declared directly in the body of the block. A locally declared name is put into the environment when the block is entered, but without an associated value. The evaluation of its declaration during evaluation of the block body then assigns to the

name the result of evaluating the expression to the right of the =, as if the declaration were an assignment. Since the addition of the name to the environment is separate from the evaluation of the declaration, and the whole block is in the scope of the name, an erroneous program could attempt to access the value of a name before its declaration is evaluated; the evaluation of an unassigned name signals an error.[18]

3.2.5 CSE Machine

The environment model as presented so far focuses on how functions can refer to their parameters, locally declared names, and names that are declared outside the function. We achieve this by evaluating statements and expressions with respect to a *current environment*. It does not specify how we keep track of environments as computation proceeds. For example, when we evaluate an expression `f(x) + y`, we need to evaluate `x` in the current environment, establish as the new current environment the environment of `f` extended by a binding of its parameter to the value of `x`, and evaluate the body of `f` in this extended environment. But what environment should we use for evaluating `y` after `f` returns? In this section, we extend the

Evaluating arithmetic expressions

Exercise 3.8 shows that the presence of assignments makes the result of a program depend on the order in which the operands of an operator combination are evaluated. To remove ambiguities that arise from this, the JavaScript standard specifies left-to-right evaluation of operands. As an example, consider the evaluation of the arithmetic expression statement

```
1 + (2 * 3);
```

The expression is decomposed into its operands `1` and `2 * 3`, followed by the *instruction* to add their results.

3.3 Modeling with Mutable Data

Chapter 2 dealt with compound data as a means for constructing computational objects that have several parts, in order to model real-world objects that have several aspects. In that chapter we introduced the discipline of data abstraction, according to which data structures are specified in terms of constructors, which create data objects, and selectors, which access the parts of compound data objects. But we now know that there is another aspect of data that chapter 2 did not address. The desire to model systems composed of objects that have changing state leads us to the need to modify compound data objects, as well as to construct and select from them. In order to model compound objects with changing state, we will design data abstractions to include, in addition to selectors and constructors, operations called *mutators*, which modify data objects. For instance, modeling a banking system requires us to change

18. This explains why the program in footnote 57 of chapter 1 goes wrong. The time between creating the binding for a name and evaluating the declaration of the name is called the *temporal dead zone* (TDZ).

account balances. Thus, a data structure for representing bank accounts might admit an operation

`set_balance(`*account*`, `*new-value*`)`

that changes the balance of the designated account to the designated new value. Data objects for which mutators are defined are known as *mutable data objects*.

Chapter 2 introduced pairs as a general-purpose "glue" for synthesizing compound data. We begin this section by defining basic mutators for pairs, so that pairs can serve as building blocks for constructing mutable data objects. These mutators greatly enhance the representational power of pairs, enabling us to build data structures other than the sequences and trees that we worked with in section 2.2. We also present some examples of simulations in which complex systems are modeled as collections of objects with local state.

3.3.1 Mutable List Structure

The basic operations on pairs—`pair`, `head`, and `tail`—can be used to construct list structure and to select parts from list structure, but they are incapable of modifying list structure. The same is true of the list operations we have used so far, such as `append` and `list`, since these can be defined in terms of `pair`, `head`, and `tail`. To modify list structures we need new operations.

The primitive mutators for pairs are `set_head` and `set_tail`. The function `set_head` takes two arguments, the first of which must be a pair. It modifies this pair, replacing the `head` pointer by a pointer to the second argument of `set_head`.[19]

As an example, suppose that `x` is bound to `list(list("a", "b"), "c", "d")` and `y` to `list("e", "f")` as illustrated in figure 3.12. Evaluating the expression `set_head(x, y)` modifies the pair to which `x` is bound, replacing its `head` by the value of `y`. The result of the operation is shown in figure 3.13. The structure `x` has been modified and is now equivalent to `list(list("e", "f"), "c", "d")`. The pairs representing the list `list("a", "b")`, identified by the pointer that was replaced, are now detached from the original structure.[20]

Compare figure 3.13 with figure 3.14, which illustrates the result of executing

```
const z = pair(y, tail(x));
```

with `x` and `y` bound to the original lists of figure 3.12. The name `z` is now bound to a new pair created by the `pair` operation; the list to which `x` is bound is unchanged.

---

19. The functions `set_head` and `set_tail` return the value `undefined`. They should be used only for their effect.

20. We see from this that mutation operations on lists can create "garbage" that is not part of any accessible structure. We will see in section 5.3.2 that JavaScript memory-management systems include a *garbage collector*, which identifies and recycles the memory space used by unneeded pairs.

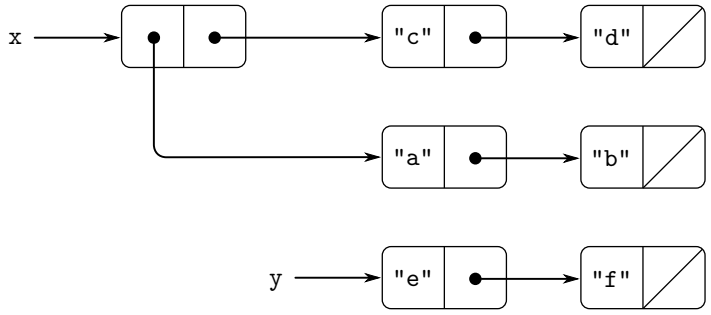


Figure 3.12 Lists x: `list(list("a", "b"), "c", "d")` and y: `list("e", "f")`.

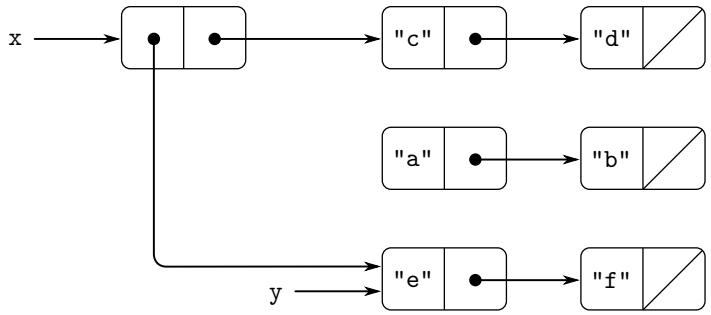


Figure 3.13 Effect of `set_head(x, y)` on the lists in figure 3.12.

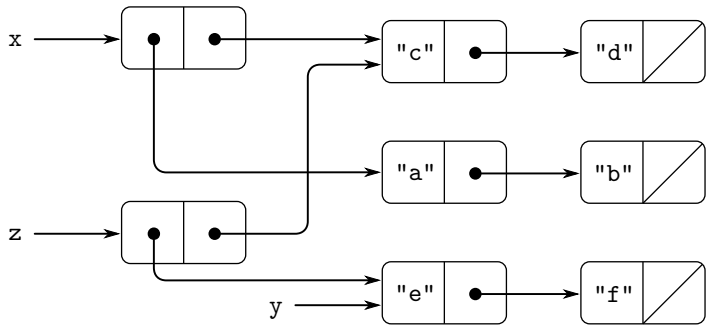


Figure 3.14 Effect of `const z = pair(y, tail(x));` on the lists in figure 3.12.

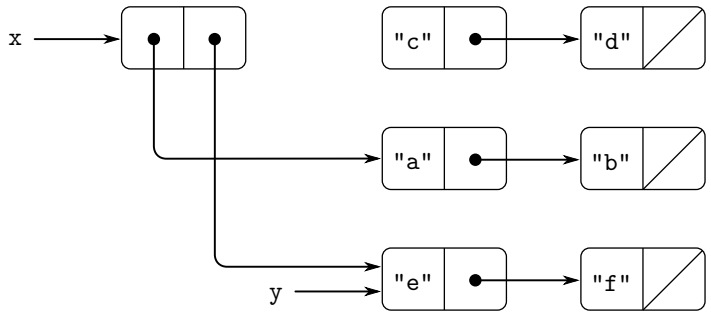


Figure 3.15 Effect of `set_tail(x, y)` on the lists in figure 3.12.

The `set_tail` operation is similar to `set_head`. The only difference is that the `tail` pointer of the pair, rather than the `head` pointer, is replaced. The effect of executing `set_tail(x, y)` on the lists of figure 3.12 is shown in figure 3.15. Here the `tail` pointer of `x` has been replaced by the pointer to `list("e", "f")`. Also, the list `list("c", "d")`, which used to be the `tail` of `x`, is now detached from the structure.

The function `pair` builds new list structure by creating new pairs, whereas `set_head` and `set_tail` modify existing pairs. Indeed, we could implement `pair` in terms of the two mutators, together with a function `get_new_pair`, which returns a new pair that is not part of any existing list structure. We obtain the new pair, set its `head` and `tail` pointers to the designated objects, and return the new pair as the result of the `pair`.[21]

```
function pair(x, y) {
    const fresh = get_new_pair();
    set_head(fresh, x);
    set_tail(fresh, y);
    return fresh;
}
```

Exercise 3.12

The following function for appending lists was introduced in section 2.2.1:

```
function append(x, y) {
    return is_null(x)
           ? y
           : pair(head(x), append(tail(x), y));
}
```

The function `append` forms a new list by successively adjoining the elements of `x` to the front of `y`. The function `append_mutator` is similar to `append`, but it is a mutator rather than a constructor. It appends the lists by splicing them together, modifying the final pair of `x` so that its `tail` is now `y`. (It is an error to call `append_mutator` with an empty `x`.)

```
function append_mutator(x, y) {
    set_tail(last_pair(x), y);
    return x;
}
```

Here `last_pair` is a function that returns the last pair in its argument:

```
function last_pair(x) {
    return is_null(tail(x))
           ? x
           : last_pair(tail(x));
}
```

Consider the interaction

```
const x = list("a", "b");
```

21. Section 5.3.1 will show how a memory-management system can implement `get_new_pair`.

```
const y = list("c", "d");

const z = append(x, y);

z;
["a", ["b", ["c", ["d", null]]]]

tail(x);
response

const w = append_mutator(x, y);

w;
["a", ["b", ["c", ["d", null]]]]

tail(x);
response
```

What are the missing *response*s? Draw box-and-pointer diagrams to explain your answer.

Exercise 3.13

Consider the following `make_cycle` function, which uses the `last_pair` function defined in exercise 3.12:

```
function make_cycle(x) {
    set_tail(last_pair(x), x);
    return x;
}
```

Draw a box-and-pointer diagram that shows the structure `z` created by

```
const z = make_cycle(list("a", "b", "c"));
```

What happens if we try to compute `last_pair(z)`?

Exercise 3.14

The following function is quite useful, although obscure:

```
function mystery(x) {
    function loop(x, y) {
        if (is_null(x)) {
            return y;
        } else {
            const temp = tail(x);
            set_tail(x, y);
            return loop(temp, x);
        }
    }
    return loop(x, null);
}
```

The function `loop` uses the "temporary" name `temp` to hold the old value of the `tail` of `x`, since the `set_tail` on the next line destroys the `tail`. Explain what `mystery` does in general. Suppose `v` is defined by

```
const v = list("a", "b", "c", "d");
```

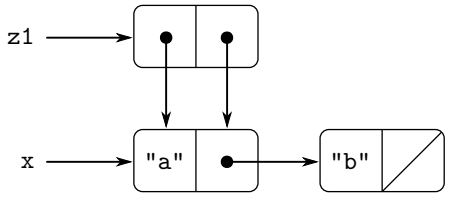


Figure 3.16 The list `z1` formed by `pair(x, x)`.

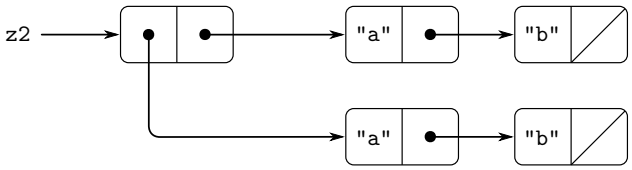


Figure 3.17 The list `z2` formed by `pair(list("a", "b"), list("a", "b"))`.

Draw the box-and-pointer diagram that represents the list to which `v` is bound. Suppose that we now evaluate

```
const w = mystery(v);
```

Draw box-and-pointer diagrams that show the structures `v` and `w` after evaluating this program. What would be printed as the values of `v` and `w`?

Sharing and identity

We mentioned in section 3.1.3 the theoretical issues of "sameness" and "change" raised by the introduction of assignment. These issues arise in practice when individual pairs are *shared* among different data objects. For example, consider the structure formed by

```
const x = list("a", "b");
const z1 = pair(x, x);
```

As shown in figure 3.16, `z1` is a pair whose `head` and `tail` both point to the same pair `x`. This sharing of `x` by the `head` and `tail` of `z1` is a consequence of the straightforward way in which `pair` is implemented. In general, using `pair` to construct lists will result in an interlinked structure of pairs in which many individual pairs are shared by many different structures.

In contrast to figure 3.16, figure 3.17 shows the structure created by

```
const z2 = pair(list("a", "b"), list("a", "b"));
```

In this structure, the pairs in the two `list("a", "b")` lists are distinct, although they contain the same strings.[22]

22. The two pairs are distinct because each call to `pair` returns a new pair. The strings are "the same" in the sense that they are primitive data (just like numbers) that are composed of the same characters in the same order. Since JavaScript provides no way to mutate a string, any

When thought of as a list, z1 and z2 both represent "the same" list:

```
list(list("a", "b"), "a", "b")
```

In general, sharing is completely undetectable if we operate on lists using only pair, head, and tail. However, if we allow mutators on list structure, sharing becomes significant. As an example of the difference that sharing can make, consider the following function, which modifies the head of the structure to which it is applied:

```
function set_to_wow(x) {
    set_head(head(x), "wow");
    return x;
}
```

Even though z1 and z2 are "the same" structure, applying set_to_wow to them yields different results. With z1, altering the head also changes the tail, because in z1 the head and the tail are the same pair. With z2, the head and tail are distinct, so set_to_wow modifies only the head:

```
z1;
[["a", ["b", null]], ["a", ["b", null]]]

set_to_wow(z1);
[["wow", ["b", null]], ["wow", ["b", null]]]

z2;
[["a", ["b", null]], ["a", ["b", null]]]

set_to_wow(z2);
[["wow", ["b", null]], ["a", ["b", null]]]
```

One way to detect sharing in list structures is to use the primitive predicate ===, which we introduced in section 1.1.6 to test whether two numbers are equal and extended in section 2.3.1 to test whether two strings are equal. When applied to two nonprimitive values, x === y tests whether x and y are the same object (that is, whether x and y are equal as pointers). Thus, with z1 and z2 as defined in figure 3.16 and 3.17, head(z1) === tail(z1) is true and head(z2) === tail(z2) is false.

As will be seen in the following sections, we can exploit sharing to greatly extend the repertoire of data structures that can be represented by pairs. On the other hand, sharing can also be dangerous, since modifications made to structures will also affect other structures that happen to share the modified parts. The mutation operations set_head and set_tail should be used with care; unless we have a good

---

sharing that the designers of a JavaScript interpreter might decide to implement for strings is undetectable. We consider primitive data such as numbers, booleans, and strings to be *identical* if and only if they are *indistinguishable*.

understanding of how our data objects are shared, mutation can have unanticipated results.[23]

Exercise 3.15

Draw box-and-pointer diagrams to explain the effect of `set_to_wow` on the structures `z1` and `z2` above.

Exercise 3.16

Ben Bitdiddle decides to write a function to count the number of pairs in any list structure. "It's easy," he reasons. "The number of pairs in any structure is the number in the `head` plus the number in the `tail` plus one more to count the current pair." So Ben writes the following function

```
function count_pairs(x) {
    return ! is_pair(x)
           ? 0
           : count_pairs(head(x)) +
             count_pairs(tail(x)) +
             1;
}
```

Show that this function is not correct. In particular, draw box-and-pointer diagrams representing list structures made up of exactly three pairs for which Ben's function would return 3; return 4; return 7; never return at all.

Exercise 3.17

Devise a correct version of the `count_pairs` function of exercise 3.16 that returns the number of distinct pairs in any structure. (Hint: Traverse the structure, maintaining an auxiliary data structure that is used to keep track of which pairs have already been counted.)

Exercise 3.18

Write a function that examines a list and determines whether it contains a cycle, that is, whether a program that tried to find the end of the list by taking successive `tail`s would go into an infinite loop. Exercise 3.13 constructed such lists.

---

23. The subtleties of dealing with sharing of mutable data objects reflect the underlying issues of "sameness" and "change" that were raised in section 3.1.3. We mentioned there that admitting change to our language requires that a compound object must have an "identity" that is something different from the pieces from which it is composed. In JavaScript, we consider this "identity" to be the quality that is tested by `===`, i.e., by equality of pointers. Since in most JavaScript implementations a pointer is essentially a memory address, we are "solving the problem" of defining the identity of objects by stipulating that a data object "itself" is the information stored in some particular set of memory locations in the computer. This suffices for simple JavaScript programs, but is hardly a general way to resolve the issue of "sameness" in computational models.

Exercise 3.19

Redo exercise 3.18 using an algorithm that takes only a constant amount of space. (This requires a very clever idea.)

Mutation is just assignment

When we introduced compound data, we observed in section 2.1.3 that pairs can be represented purely in terms of functions:

```
function pair(x, y) {
    function dispatch(m) {
        return m === "head"
               ? x
               : m === "tail"
               ? y
               : error(m, "undefined operation -- pair");
    }
    return dispatch;
}
function head(z) { return z("head"); }
function tail(z) { return z("tail"); }
```

The same observation is true for mutable data. We can implement mutable data objects as functions using assignment and local state. For instance, we can extend the above pair implementation to handle `set_head` and `set_tail` in a manner analogous to the way we implemented bank accounts using `make_account` in section 3.1.1:

```
function pair(x, y) {
    function set_x(v) { x = v; }
    function set_y(v) { y = v; }
    return m => m === "head"
                ? x
                : m === "tail"
                ? y
                : m === "set_head"
                ? set_x
                : m === "set_tail"
                ? set_y
                : error(m, "undefined operation -- pair");
}
function head(z) { return z("head"); }
function tail(z) { return z("tail"); }
function set_head(z, new_value) {
    z("set_head")(new_value);
    return z;
}
function set_tail(z, new_value) {
    z("set_tail")(new_value);
    return z;
}
```

Operation	Resulting Queue
`const q = make_queue();`	
`insert_queue(q, "a");`	`a`
`insert_queue(q, "b");`	`a b`
`delete_queue(q);`	`b`
`insert_queue(q, "c");`	`b c`
`insert_queue(q, "d");`	`b c d`
`delete_queue(q);`	`c d`

Figure 3.18 Queue operations.

Assignment is all that is needed, theoretically, to account for the behavior of mutable data. As soon as we admit assignment to our language, we raise all the issues, not only of assignment, but of mutable data in general.[24]

Exercise 3.20

Draw environment diagrams to illustrate the evaluation of the sequence of statements

```
const x = pair(1, 2);
const z = pair(x, x);
set_head(tail(z), 17);

head(x);
17
```

using the functional implementation of pairs given above. (Compare exercise 3.11.)

3.3.2 Representing Queues

The mutators `set_head` and `set_tail` enable us to use pairs to construct data structures that cannot be built with `pair`, `head`, and `tail` alone. This section shows how to use pairs to represent a data structure called a queue. Section 3.3.3 will show how to represent data structures called tables.

A *queue* is a sequence in which items are inserted at one end (called the *rear* of the queue) and deleted from the other end (the *front*). Figure 3.18 shows an initially empty queue in which the items `a` and `b` are inserted. Then `a` is removed, `c` and `d` are inserted, and `b` is removed. Because items are always removed in the order in which they are inserted, a queue is sometimes called a *FIFO* (first in, first out) buffer.

24. On the other hand, from the viewpoint of implementation, assignment requires us to modify the environment, which is itself a mutable data structure. Thus, assignment and mutation are equipotent: Each can be implemented in terms of the other.

In terms of data abstraction, we can regard a queue as defined by the following set of operations:

- a constructor:
  `make_queue()`
  returns an empty queue (a queue containing no items).

- a predicate:
  `is_empty_queue(`*queue*`)`
  tests if the queue is empty.

- a selector:
  `front_queue(`*queue*`)`
  returns the object at the front of the queue, signaling an error if the queue is empty; it does not modify the queue.

- two mutators:
  `insert_queue(`*queue*`, `*item*`)`
  inserts the item at the rear of the queue and returns the modified queue as its value.

  `delete_queue(`*queue*`)`
  removes the item at the front of the queue and returns the modified queue as its value, signaling an error if the queue is empty before the deletion.

Because a queue is a sequence of items, we could certainly represent it as an ordinary list; the front of the queue would be the `head` of the list, inserting an item in the queue would amount to appending a new element at the end of the list, and deleting an item from the queue would just be taking the `tail` of the list. However, this representation is inefficient, because in order to insert an item we must scan the list until we reach the end. Since the only method we have for scanning a list is by successive `tail` operations, this scanning requires $\Theta(n)$ steps for a list of n items. A simple modification to the list representation overcomes this disadvantage by allowing the queue operations to be implemented so that they require $\Theta(1)$ steps; that is, so that the number of steps needed is independent of the length of the queue.

The difficulty with the list representation arises from the need to scan to find the end of the list. The reason we need to scan is that, although the standard way of representing a list as a chain of pairs readily provides us with a pointer to the beginning of the list, it gives us no easily accessible pointer to the end. The modification that avoids the drawback is to represent the queue as a list, together with an additional pointer that indicates the final pair in the list. That way, when we go to insert an item, we can consult the rear pointer and so avoid scanning the list.

A queue is represented, then, as a pair of pointers, `front_ptr` and `rear_ptr`, which indicate, respectively, the first and last pairs in an ordinary list. Since we would like the queue to be an identifiable object, we can use `pair` to combine the two pointers. Thus, the queue itself will be the `pair` of the two pointers. Figure 3.19 illustrates this representation.

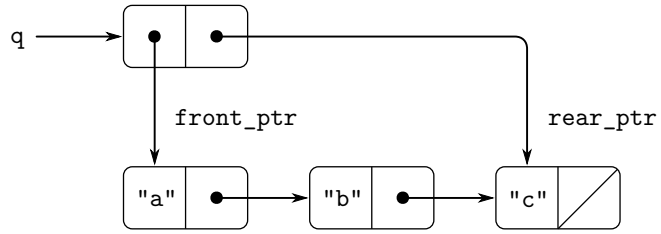


Figure 3.19 Implementation of a queue as a list with front and rear pointers.

To define the queue operations we use the following functions, which enable us to select and to modify the front and rear pointers of a queue:

```
function front_ptr(queue) { return head(queue); }
function rear_ptr(queue) { return tail(queue); }
function set_front_ptr(queue, item) { set_head(queue, item); }
function set_rear_ptr(queue, item) { set_tail(queue, item); }
```

Now we can implement the actual queue operations. We will consider a queue to be empty if its front pointer is the empty list:

```
function is_empty_queue(queue) { return is_null(front_ptr(queue)); }
```

The `make_queue` constructor returns, as an initially empty queue, a pair whose `head` and `tail` are both the empty list:

```
function make_queue() { return pair(null, null); }
```

To select the item at the front of the queue, we return the `head` of the pair indicated by the front pointer:

```
function front_queue(queue) {
    return is_empty_queue(queue)
           ? error(queue, "front_queue called with an empty queue")
           : head(front_ptr(queue));
}
```

To insert an item in a queue, we follow the method whose result is indicated in figure 3.20. We first create a new pair whose `head` is the item to be inserted and whose `tail` is the empty list. If the queue was initially empty, we set the front and rear pointers of the queue to this new pair. Otherwise, we modify the final pair in the queue to point to the new pair, and also set the rear pointer to the new pair.

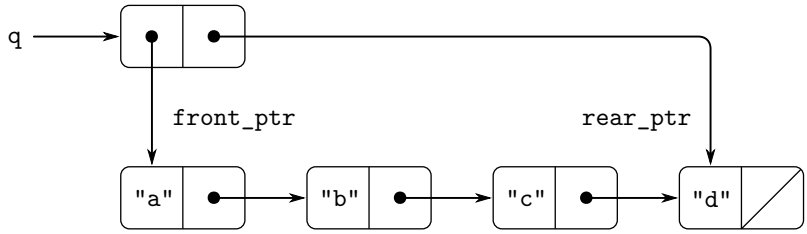


Figure 3.20 Result of using `insert_queue(q, "d")` on the queue of figure 3.19.

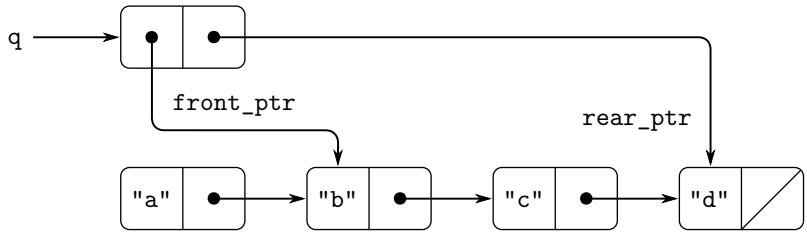


Figure 3.21 Result of using `delete_queue(q)` on the queue of figure 3.20.

```
function insert_queue(queue, item) {
    const new_pair = pair(item, null);
    if (is_empty_queue(queue)) {
        set_front_ptr(queue, new_pair);
        set_rear_ptr(queue, new_pair);
    } else {
        set_tail(rear_ptr(queue), new_pair);
        set_rear_ptr(queue, new_pair);
    }
    return queue;
}
```

To delete the item at the front of the queue, we merely modify the front pointer so that it now points at the second item in the queue, which can be found by following the `tail` pointer of the first item (see figure 3.21):[25]

```
function delete_queue(queue) {
    if (is_empty_queue(queue)) {
        error(queue, "delete_queue called with an empty queue");
    } else {
        set_front_ptr(queue, tail(front_ptr(queue)));
        return queue;
    }
}
```

25. If the first item is the final item in the queue, the front pointer will be the empty list after the deletion, which will mark the queue as empty; we needn't worry about updating the rear pointer, which will still point to the deleted item, because `is_empty_queue` looks only at the front pointer.

Exercise 3.21

Ben Bitdiddle decides to test the queue implementation described above. He types in the functions to the JavaScript interpreter and proceeds to try them out:

```
const q1 = make_queue();

insert_queue(q1, "a");
[["a", null], ["a", null]]

insert_queue(q1, "b");
[["a", ["b", null]], ["b", null]]

delete_queue(q1);
[["b", null], ["b", null]]

delete_queue(q1);
[null, ["b", null]]
```

"It's all wrong!" he complains. "The interpreter's response shows that the last item is inserted into the queue twice. And when I delete both items, the second `b` is still there, so the queue isn't empty, even though it's supposed to be." Eva Lu Ator suggests that Ben has misunderstood what is happening. "It's not that the items are going into the queue twice," she explains. "It's just that the standard JavaScript printer doesn't know how to make sense of the queue representation. If you want to see the queue printed correctly, you'll have to define your own print function for queues." Explain what Eva Lu is talking about. In particular, show why Ben's examples produce the printed results that they do. Define a function `print_queue` that takes a queue as input and prints the sequence of items in the queue.

Exercise 3.22

Instead of representing a queue as a pair of pointers, we can build a queue as a function with local state. The local state will consist of pointers to the beginning and the end of an ordinary list. Thus, the `make_queue` function will have the form

```
function make_queue() {
    let front_ptr = ...;
    let rear_ptr = ...;
    ⟨declarations of internal functions⟩
    function dispatch(m) {...}
    return dispatch;
}
```

Complete the definition of `make_queue` and provide implementations of the queue operations using this representation.

Exercise 3.23

A *deque* ("double-ended queue") is a sequence in which items can be inserted and deleted either at the front or at the rear. Operations on deques are the constructor `make_deque`, the predicate `is_empty_deque`, selectors `front_deque` and `rear_deque`, and mutators

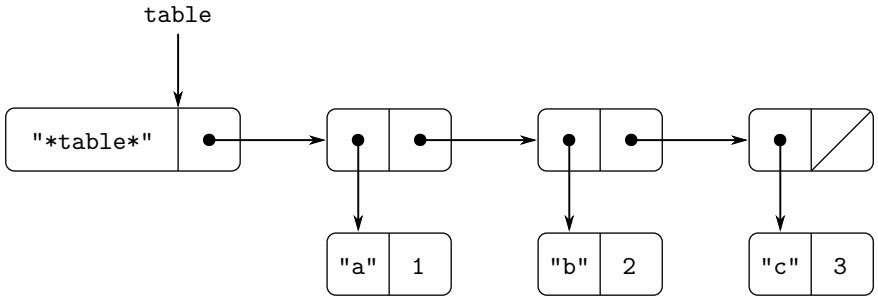


Figure 3.22 A table represented as a headed list.

`front_insert_deque`, `front_delete_deque`, `rear_insert_deque`, and `rear_delete_deque`. Show how to represent deques using pairs, and give implementations of the operations.[26] All operations should be accomplished in $\Theta(1)$ steps.

3.3.3 Representing Tables

When we studied various ways of representing sets in chapter 2, we mentioned in section 2.3.3 the task of maintaining a table of records indexed by identifying keys. In the implementation of data-directed programming in section 2.4.3, we made extensive use of two-dimensional tables, in which information is stored and retrieved using two keys. Here we see how to build tables as mutable list structures.

We first consider a one-dimensional table, in which each value is stored under a single key. We implement the table as a list of records, each of which is implemented as a pair consisting of a key and the associated value. The records are glued together to form a list by pairs whose `head`s point to successive records. These gluing pairs are called the *backbone* of the table. In order to have a place that we can change when we add a new record to the table, we build the table as a *headed list*. A headed list has a special backbone pair at the beginning, which holds a dummy "record"—in this case the arbitrarily chosen string `"*table*"`. Figure 3.22 shows the box-and-pointer diagram for the table

```
a: 1
b: 2
c: 3
```

To extract information from a table we use the `lookup` function, which takes a key as argument and returns the associated value (or `undefined` if there is no value stored under that key). The function `lookup` is defined in terms of the `assoc` operation, which expects a key and a list of records as arguments. Note that `assoc` never sees the dummy record. The function `assoc` returns the record that has the given key as its `head`.[27] The function `lookup` then checks to see that the resulting

26. Be careful not to make the interpreter try to print a structure that contains cycles. (See exercise 3.13.)

27. Because `assoc` uses `equal`, it can recognize keys that are strings, numbers, or list structure.

record returned by `assoc` is not `undefined`, and returns the value (the `tail`) of the record.

```
function lookup(key, table) {
    const record = assoc(key, tail(table));
    return is_undefined(record)
           ? undefined
           : tail(record);
}
function assoc(key, records) {
    return is_null(records)
           ? undefined
           : equal(key, head(head(records)))
           ? head(records)
           : assoc(key, tail(records));
}
```

To insert a value in a table under a specified key, we first use `assoc` to see if there is already a record in the table with this key. If not, we form a new record by `pair`ing the key with the value, and insert this at the head of the table's list of records, after the dummy record. If there already is a record with this key, we set the `tail` of this record to the designated new value. The header of the table provides us with a fixed location to modify in order to insert the new record.[28]

```
function insert(key, value, table) {
    const record = assoc(key, tail(table));
    if (is_undefined(record)) {
        set_tail(table,
                 pair(pair(key, value), tail(table)));
    } else {
        set_tail(record, value);
    }
    return "ok";
}
```

To construct a new table, we simply create a list containing just the string `"*table*"`:

```
function make_table() {
    return list("*table*");
}
```

Two-dimensional tables

In a two-dimensional table, each value is indexed by two keys. We can construct such a table as a one-dimensional table in which each key identifies a subtable. Figure 3.23 shows the box-and-pointer diagram for the table

28. Thus, the first backbone pair is the object that represents the table "itself"; that is, a pointer to the table is a pointer to this pair. This same backbone pair always starts the table. If we did not arrange things in this way, `insert` would have to return a new value for the start of the table when it added a new record.

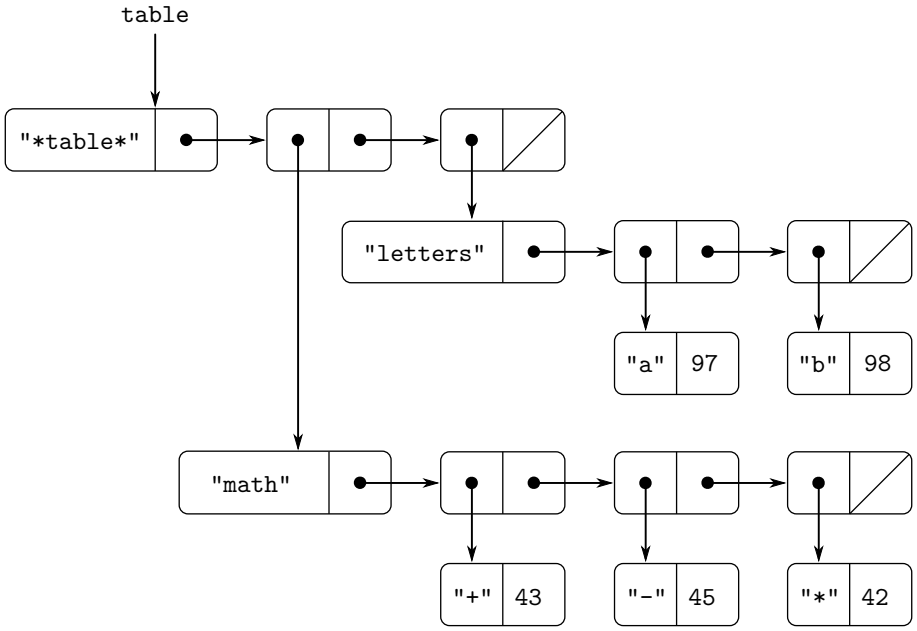


Figure 3.23 A two-dimensional table.

```
"math":
    "+":  43
    "-":  45
    "*":  42
"letters":
    "a":  97
    "b":  98
```

which has two subtables. (The subtables don't need a special header string, since the key that identifies the subtable serves this purpose.)

When we look up an item, we use the first key to identify the correct subtable. Then we use the second key to identify the record within the subtable.

```
function lookup(key_1, key_2, table) {
    const subtable = assoc(key_1, tail(table));
    if (is_undefined(subtable)) {
        return undefined;
    } else {
        const record = assoc(key_2, tail(subtable));
        return is_undefined(record)
               ? undefined
               : tail(record);
    }
}
```

To insert a new item under a pair of keys, we use `assoc` to see if there is a subtable stored under the first key. If not, we build a new subtable containing the single record (`key_2`, `value`) and insert it into the table under the first key. If a

subtable already exists for the first key, we insert the new record into this subtable, using the insertion method for one-dimensional tables described above:

```
function insert(key_1, key_2, value, table) {
    const subtable = assoc(key_1, tail(table));
    if (is_undefined(subtable)) {
        set_tail(table,
                 pair(list(key_1, pair(key_2, value)), tail(table)));
    } else {
        const record = assoc(key_2, tail(subtable));
        if (is_undefined(record)) {
            set_tail(subtable,
                     pair(pair(key_2, value), tail(subtable)));
        } else {
            set_tail(record, value);
        }
    }
    return "ok";
}
```

Creating local tables

The `lookup` and `insert` operations defined above take the table as an argument. This enables us to use programs that access more than one table. Another way to deal with multiple tables is to have separate `lookup` and `insert` functions for each table. We can do this by representing a table procedurally, as an object that maintains an internal table as part of its local state. When sent an appropriate message, this "table object" supplies the function with which to operate on the internal table. Here is a generator for two-dimensional tables represented in this fashion:

```
function make_table() {
    const local_table = list("*table*");
    function lookup(key_1, key_2) {
        const subtable = assoc(key_1, tail(local_table));
        if (is_undefined(subtable)) {
            return undefined;
        } else {
            const record = assoc(key_2, tail(subtable));
            return is_undefined(record)
                   ? undefined
                   : tail(record);
        }
    }
    function insert(key_1, key_2, value) {
        const subtable = assoc(key_1, tail(local_table));
        if (is_undefined(subtable)) {
            set_tail(local_table,
                     pair(list(key_1, pair(key_2, value)),
                          tail(local_table)));
        } else {
            const record = assoc(key_2, tail(subtable));
            if (is_undefined(record)) {
                set_tail(subtable,
                         pair(pair(key_2, value), tail(subtable)));
            } else {
                set_tail(record, value);
            }
        }
    }
    function dispatch(m) {
        return m === "lookup"
               ? lookup
               : m === "insert"
               ? insert
               : error(m, "unknown operation -- table");
    }
    return dispatch;
}
```

Using `make_table`, we could implement the `get` and `put` operations used in section 2.4.3 for data-directed programming, as follows:

```
const operation_table = make_table();
const get = operation_table("lookup");
const put = operation_table("insert");
```

The function `get` takes as arguments two keys, and `put` takes as arguments two keys and a value. Both operations access the same local table, which is encapsulated within the object created by the call to `make_table`.

Exercise 3.24

In the table implementations above, the keys are tested for equality using `equal` (called by `assoc`). This is not always the appropriate test. For instance, we might have a table with numeric keys in which we don't need an exact match to the number we're looking

up, but only a number within some tolerance of it. Design a table constructor `make_table` that takes as an argument a `same_key` function that will be used to test "equality" of keys. The function `make_table` should return a `dispatch` function that can be used to access appropriate `lookup` and `insert` functions for a local table.

Exercise 3.25

Generalizing one- and two-dimensional tables, show how to implement a table in which values are stored under an arbitrary number of keys and different values may be stored under different numbers of keys. The `lookup` and `insert` functions should take as input a list of keys used to access the table.

Exercise 3.26

To search a table as implemented above, one needs to scan through the list of records. This is basically the unordered list representation of section 2.3.3. For large tables, it may be more efficient to structure the table in a different manner. Describe a table implementation where the (key, value) records are organized using a binary tree, assuming that keys can be ordered in some way (e.g., numerically or alphabetically). (Compare exercise 2.66 of chapter 2.)

Exercise 3.27

Memoization (also called *tabulation*) is a technique that enables a function to record, in a local table, values that have previously been computed. This technique can make a vast difference in the performance of a program. A memoized function maintains a table in which values of previous calls are stored using as keys the arguments that produced the values. When the memoized function is asked to compute a value, it first checks the table to see if the value is already there and, if so, just returns that value. Otherwise, it computes the new value in the ordinary way and stores this in the table. As an example of memoization, recall from section 1.2.2 the exponential process for computing Fibonacci numbers:

```
function fib(n) {
    return n === 0
           ? 0
           : n === 1
           ? 1
           : fib(n - 1) + fib(n - 2);
}
```

The memoized version of the same function is

```
const memo_fib = memoize(n => n === 0
                              ? 0
                              : n === 1
                              ? 1
                              : memo_fib(n - 1) +
                                memo_fib(n - 2)
                         );
```

where the memoizer is defined as

```
function memoize(f) {
    const table = make_table();
    return x => {
                const previously_computed_result =
                    lookup(x, table);
                if (is_undefined(previously_computed_result)) {
                    const result = f(x);
                    insert(x, result, table);
                    return result;
                } else {
                    return previously_computed_result;
                }
            };
}
```

Draw an environment diagram to analyze the computation of `memo_fib(3)`. Explain why `memo_fib` computes the nth Fibonacci number in a number of steps proportional to n. Would the scheme still work if we had simply defined `memo_fib` to be `memoize(fib)`?

3.3.4 A Simulator for Digital Circuits

Designing complex digital systems, such as computers, is an important engineering activity. Digital systems are constructed by interconnecting simple elements. Although the behavior of these individual elements is simple, networks of them can have very complex behavior. Computer simulation of proposed circuit designs is an important tool used by digital systems engineers. In this section we design a system for performing digital logic simulations. This system typifies a kind of program called an *event-driven simulation*, in which actions ("events") trigger further events that happen at a later time, which in turn trigger more events, and so on.

Our computational model of a circuit will be composed of objects that correspond to the elementary components from which the circuit is constructed. There are *wires*, which carry *digital signals*. A digital signal may at any moment have only one of two possible values, 0 and 1. There are also various types of digital *function boxes*, which connect wires carrying input signals to other output wires. Such boxes produce output signals computed from their input signals. The output signal is delayed by a time that depends on the type of the function box. For example, an *inverter* is a primitive function box that inverts its input. If the input signal to an inverter changes to 0, then one *inverter-delay* later the inverter will change its output signal to 1. If the input signal to an inverter changes to 1, then one *inverter-delay* later the inverter will change its output signal to 0. We draw an inverter symbolically as in figure 3.24. An *and-gate*, also shown in figure 3.24, is a primitive function box with two inputs and one output. It drives its output signal to a value that is the *logical and* of the inputs. That is, if both of its input signals become 1, then one *and-gate-delay* time later the and-gate will force its output signal to be 1; otherwise the output will be 0. An *or-gate* is a similar two-input primitive function box that drives its output signal to a value that is the *logical or* of the inputs. That is, the output will become 1 if at least one of the input signals is 1; otherwise the output will become 0.

We can connect primitive functions together to construct more complex functions. To accomplish this we wire the outputs of some function boxes to the inputs of other function boxes. For example, the *half-adder* circuit shown in figure 3.25 consists of

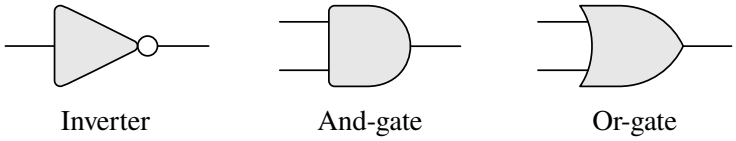


Figure 3.24 Primitive functions in the digital logic simulator.

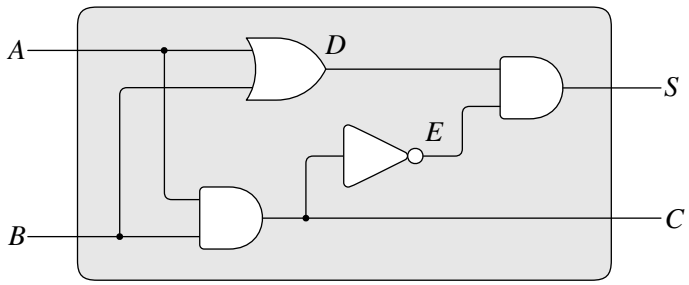


Figure 3.25 A half-adder circuit.

an or-gate, two and-gates, and an inverter. It takes two input signals, *A* and *B*, and has two output signals, *S* and *C*. *S* will become 1 whenever precisely one of *A* and *B* is 1, and *C* will become 1 whenever *A* and *B* are both 1. We can see from the figure that, because of the delays involved, the outputs may be generated at different times. Many of the difficulties in the design of digital circuits arise from this fact.

We will now build a program for modeling the digital logic circuits we wish to study. The program will construct computational objects modeling the wires, which will "hold" the signals. Function boxes will be modeled by functions that enforce the correct relationships among the signals.

One basic element of our simulation will be a function `make_wire`, which constructs wires. For example, we can construct six wires as follows:

```
const a = make_wire();
const b = make_wire();
const c = make_wire();
const d = make_wire();
const e = make_wire();
const s = make_wire();
```

We attach a function box to a set of wires by calling a function that constructs that kind of box. The arguments to the constructor function are the wires to be attached to the box. For example, given that we can construct and-gates, or-gates, and inverters, we can wire together the half-adder shown in figure 3.25:

```
or_gate(a, b, d);
"ok"

and_gate(a, b, c);
"ok"
```

```
inverter(c, e);
"ok"

and_gate(d, e, s);
"ok"
```

Better yet, we can explicitly name this operation by defining a function `half_adder` that constructs this circuit, given the four external wires to be attached to the half-adder:

```
function half_adder(a, b, s, c) {
    const d = make_wire();
    const e = make_wire();
    or_gate(a, b, d);
    and_gate(a, b, c);
    inverter(c, e);
    and_gate(d, e, s);
    return "ok";
}
```

The advantage of making this definition is that we can use `half_adder` itself as a building block in creating more complex circuits. Figure 3.26, for example, shows a *full-adder* composed of two half-adders and an or-gate.[29] We can construct a full-adder as follows:

```
function full_adder(a, b, c_in, sum, c_out) {
    const s = make_wire();
    const c1 = make_wire();
    const c2 = make_wire();
    half_adder(b, c_in, s, c1);
    half_adder(a, s, sum, c2);
    or_gate(c1, c2, c_out);
    return "ok";
}
```

Having defined `full_adder` as a function, we can now use it as a building block for creating still more complex circuits. (For example, see exercise 3.30.)

In essence, our simulator provides us with the tools to construct a language of circuits. If we adopt the general perspective on languages with which we approached the study of JavaScript in section 1.1, we can say that the primitive function boxes form the primitive elements of the language, that wiring boxes together provides a means of combination, and that specifying wiring patterns as functions serves as a means of abstraction.

---

29. A full-adder is a basic circuit element used in adding two binary numbers. Here A and B are the bits at corresponding positions in the two numbers to be added, and C_{in} is the carry bit from the addition one place to the right. The circuit generates SUM, which is the sum bit in the corresponding position, and C_{out}, which is the carry bit to be propagated to the left.

Primitive function boxes

The primitive function boxes implement the "forces" by which a change in the signal on one wire influences the signals on other wires. To build function boxes, we use the following operations on wires:

- `get_signal(`*wire*`)`
  returns the current value of the signal on the wire.
- `set_signal(`*wire*`,` *new-value*`)`:
  changes the value of the signal on the wire to the new value.
- `add_action(`*wire*`,` *function-of-no-arguments*`)`:
  asserts that the designated function should be run whenever the signal on the wire changes value. Such functions are the vehicles by which changes in the signal value on the wire are communicated to other wires.

In addition, we will make use of a function `after_delay` that takes a time delay and a function to be run and executes the given function after the given delay.

Using these functions, we can define the primitive digital logic functions. To connect an input to an output through an inverter, we use `add_action` to associate with the input wire a function that will be run whenever the signal on the input wire changes value. The function computes the `logical_not` of the input signal, and then, after one `inverter_delay`, sets the output signal to be this new value:

```
function inverter(input, output) {
    function invert_input() {
        const new_value = logical_not(get_signal(input));
        after_delay(inverter_delay,
                    () => set_signal(output, new_value));
    }
    add_action(input, invert_input);
    return "ok";
}
function logical_not(s) {
    return s === 0
           ? 1
           : s === 1
           ? 0
           : error(s, "invalid signal");
}
```

An and-gate is a little more complex. The action function must be run if either of the inputs to the gate changes. It computes the `logical_and` (using a function analogous to `logical_not`) of the values of the signals on the input wires and sets up a change to the new value to occur on the output wire after one `and_gate_delay`.

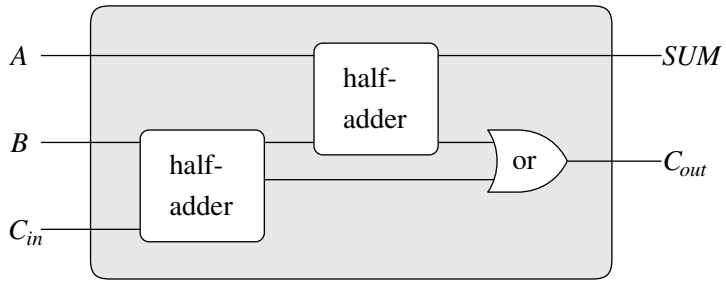


Figure 3.26 A full-adder circuit.

```
function and_gate(a1, a2, output) {
    function and_action_function() {
        const new_value = logical_and(get_signal(a1),
                                      get_signal(a2));
        after_delay(and_gate_delay,
                    () => set_signal(output, new_value));
    }
    add_action(a1, and_action_function);
    add_action(a2, and_action_function);
    return "ok";
}
```

Exercise 3.28

Define an or-gate as a primitive function box. Your `or_gate` constructor should be similar to `and_gate`.

Exercise 3.29

Another way to construct an or-gate is as a compound digital logic device, built from and-gates and inverters. Define a function `or_gate` that accomplishes this. What is the delay time of the or-gate in terms of `and_gate_delay` and `inverter_delay`?

Exercise 3.30

Figure 3.27 shows a *ripple-carry adder* formed by stringing together n full-adders. This is the simplest form of parallel adder for adding two n-bit binary numbers. The inputs A_1, A_2, A_3, ..., A_n and B_1, B_2, B_3, ..., B_n are the two binary numbers to be added (each A_k and B_k is a 0 or a 1). The circuit generates S_1, S_2, S_3, ..., S_n, the n bits of the sum, and C, the carry from the addition. Write a function `ripple_carry_adder` that generates this circuit. The function should take as arguments three lists of n wires each—the A_k, the B_k, and the S_k—and also another wire C. The major drawback of the ripple-carry adder is the need to wait for the carry signals to propagate. What is the delay needed to obtain the complete output from an n-bit ripple-carry adder, expressed in terms of the delays for and-gates, or-gates, and inverters?

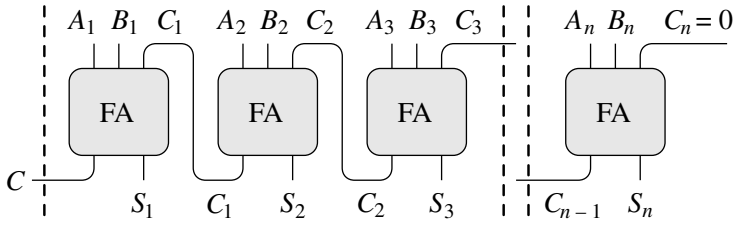


Figure 3.27 A ripple-carry adder for n-bit numbers.

Representing wires

A wire in our simulation will be a computational object with two local state variables: a `signal_value` (initially taken to be 0) and a collection of `action_functions` to be run when the signal changes value. We implement the wire, using message-passing style, as a collection of local functions together with a `dispatch` function that selects the appropriate local operation, just as we did with the simple bank-account object in section 3.1.1:

```
function make_wire() {
    let signal_value = 0;
    let action_functions = null;
    function set_my_signal(new_value) {
        if (signal_value !== new_value) {
            signal_value = new_value;
            return call_each(action_functions);
        } else {
            return "done";
        }
    }
    function accept_action_function(fun) {
        action_functions = pair(fun, action_functions);
        fun();
    }
    function dispatch(m) {
        return m === "get_signal"
               ? signal_value
               : m === "set_signal"
               ? set_my_signal
               : m === "add_action"
               ? accept_action_function
               : error(m, "unknown operation -- wire");
    }
    return dispatch;
}
```

The local function `set_my_signal` tests whether the new signal value changes the signal on the wire. If so, it runs each of the action functions, using the following function `call_each`, which calls each of the items in a list of no-argument functions:

```
function call_each(functions) {
    if (is_null(functions)) {
        return "done";
    } else {
        head(functions)();
        return call_each(tail(functions));
    }
}
```

The local function `accept_action_function` adds the given function to the list of functions to be run, and then runs the new function once. (See exercise 3.31.)

With the local `dispatch` function set up as specified, we can provide the following functions to access the local operations on wires:[30]

```
function get_signal(wire) {
    return wire("get_signal");
}
function set_signal(wire, new_value) {
    return wire("set_signal")(new_value);
}
function add_action(wire, action_function) {
    return wire("add_action")(action_function);
}
```

Wires, which have time-varying signals and may be incrementally attached to devices, are typical of mutable objects. We have modeled them as functions with local state variables that are modified by assignment. When a new wire is created, a new set of state variables is allocated (by the `let` statements in `make_wire`) and a new `dispatch` function is constructed and returned, capturing the environment with the new state variables.

The wires are shared among the various devices that have been connected to them. Thus, a change made by an interaction with one device will affect all the other devices attached to the wire. The wire communicates the change to its neighbors by calling the action functions provided to it when the connections were established.

The agenda

The only thing needed to complete the simulator is `after_delay`. The idea here is that we maintain a data structure, called an *agenda*, that contains a schedule of things to do. The following operations are defined for agendas:

---

30. These functions are simply syntactic sugar that allow us to use ordinary functional syntax to access the local functions of objects. It is striking that we can interchange the role of "functions" and "data" in such a simple way. For example, if we write `wire("get_signal")` we think of `wire` as a function that is called with the message `"get_signal"` as input. Alternatively, writing `get_signal(wire)` encourages us to think of `wire` as a data object that is the input to a function `get_signal`. The truth of the matter is that, in a language in which we can deal with functions as objects, there is no fundamental difference between "functions" and "data," and we can choose our syntactic sugar to allow us to program in whatever style we choose.

- `make_agenda()`:
  returns a new empty agenda.
- `is_empty_agenda(`*agenda*`)`
  is true if the specified agenda is empty.
- `first_agenda_item(`*agenda*`)`
  returns the first item on the agenda.
- `remove_first_agenda_item(`*agenda*`)`
  modifies the agenda by removing the first item.
- `add_to_agenda(`*time*`, `*action*`, `*agenda*`)`
  modifies the agenda by adding the given action function to be run at the specified time.
- `current_time(`*agenda*`)`
  returns the current simulation time.

The particular agenda that we use is denoted by `the_agenda`. The function `after_delay` adds new elements to `the_agenda`:

```
function after_delay(delay, action) {
    add_to_agenda(delay + current_time(the_agenda),
                  action,
                  the_agenda);
}
```

The simulation is driven by the function `propagate`, which executes each function on `the_agenda` in sequence. In general, as the simulation runs, new items will be added to the agenda, and `propagate` will continue the simulation as long as there are items on the agenda:

```
function propagate() {
    if (is_empty_agenda(the_agenda)) {
        return "done";
    } else {
        const first_item = first_agenda_item(the_agenda);
        first_item();
        remove_first_agenda_item(the_agenda);
        return propagate();
    }
}
```

A sample simulation

The following function, which places a “probe” on a wire, shows the simulator in action. The probe tells the wire that, whenever its signal changes value, it should print the new signal value, together with the current time and a name that identifies the wire.

```
function probe(name, wire) {
    add_action(wire,
               () => display(name + " " +
                             stringify(current_time(the_agenda)) +
                             ", new value = " +
                             stringify(get_signal(wire))));
}
```

We begin by initializing the agenda and specifying delays for the primitive function boxes:

```
const the_agenda = make_agenda();
const inverter_delay = 2;
const and_gate_delay = 3;
const or_gate_delay = 5;
```

Now we define four wires, placing probes on two of them:

```
const input_1 = make_wire();
const input_2 = make_wire();
const sum = make_wire();
const carry = make_wire();

probe("sum", sum);
"sum 0, new value = 0"

probe("carry", carry);
"carry 0, new value = 0"
```

Next we connect the wires in a half-adder circuit (as in figure 3.25), set the signal on `input_1` to 1, and run the simulation:

```
half_adder(input_1, input_2, sum, carry);
"ok"

set_signal(input_1, 1);
"done"

propagate();
"sum 8, new value = 1"
"done"
```

The `sum` signal changes to 1 at time 8. We are now eight time units from the beginning of the simulation. At this point, we can set the signal on `input_2` to 1 and allow the values to propagate:

```
set_signal(input_2, 1);
"done"

propagate();
"carry 11, new value = 1"
"sum 16, new value = 0"
"done"
```

The `carry` changes to 1 at time 11 and the `sum` changes to 0 at time 16.

Exercise 3.31

The internal function `accept_action_function` defined in `make_wire` specifies that when a new action function is added to a wire, the function is immediately run. Explain why this initialization is necessary. In particular, trace through the half-adder example in the paragraphs above and say how the system's response would differ if we had defined `accept_action_function` as

```
function accept_action_function(fun) {
   action_functions = pair(fun, action_functions);
}
```

Implementing the agenda

Finally, we give details of the agenda data structure, which holds the functions that are scheduled for future execution.

The agenda is made up of *time segments*. Each time segment is a pair consisting of a number (the time) and a queue (see exercise 3.32) that holds the functions that are scheduled to be run during that time segment.

```
function make_time_segment(time, queue) {
   return pair(time, queue);
}
function segment_time(s) { return head(s); }
function segment_queue(s) { return tail(s); }
```

We will operate on the time-segment queues using the queue operations described in section 3.3.2.

The agenda itself is a one-dimensional table of time segments. It differs from the tables described in section 3.3.3 in that the segments will be sorted in order of increasing time. In addition, we store the *current time* (i.e., the time of the last action that was processed) at the head of the agenda. A newly constructed agenda has no time segments and has a current time of 0:[31]

```
function make_agenda() { return list(0); }
function current_time(agenda) { return head(agenda); }
function set_current_time(agenda, time) {
   set_head(agenda, time);
}
function segments(agenda) { return tail(agenda); }
function set_segments(agenda, segs) {
   set_tail(agenda, segs);
}
function first_segment(agenda) { return head(segments(agenda)); }
function rest_segments(agenda) { return tail(segments(agenda)); }
```

An agenda is empty if it has no time segments:

```
function is_empty_agenda(agenda) {
   return is_null(segments(agenda));
}
```

To add an action to an agenda, we first check if the agenda is empty. If so, we create a time segment for the action and install this in the agenda. Otherwise, we scan the agenda, examining the time of each segment. If we find a segment for our

31. The agenda is a headed list, like the tables in section 3.3.3, but since the list is headed by the time, we do not need an additional dummy header (such as the `"*table*"` string used with tables).

appointed time, we add the action to the associated queue. If we reach a time later than the one to which we are appointed, we insert a new time segment into the agenda just before it. If we reach the end of the agenda, we must create a new time segment at the end.

```
function add_to_agenda(time, action, agenda) {
   function belongs_before(segs) {
      return is_null(segs) || time < segment_time(head(segs));
   }
   function make_new_time_segment(time, action) {
      const q = make_queue();
      insert_queue(q, action);
      return make_time_segment(time, q);
   }
   function add_to_segments(segs) {
      if (segment_time(head(segs)) === time) {
          insert_queue(segment_queue(head(segs)), action);
      } else {
          const rest = tail(segs);
          if (belongs_before(rest)) {
              set_tail(segs, pair(make_new_time_segment(time, action),
                                  tail(segs)));
          } else {
              add_to_segments(rest);
          }
      }
   }
   const segs = segments(agenda);
   if (belongs_before(segs)) {
       set_segments(agenda,
                    pair(make_new_time_segment(time, action), segs));
   } else {
       add_to_segments(segs);
   }
}
```

The function that removes the first item from the agenda deletes the item at the front of the queue in the first time segment. If this deletion makes the time segment empty, we remove it from the list of segments:[32]

```
function remove_first_agenda_item(agenda) {
   const q = segment_queue(first_segment(agenda));
   delete_queue(q);
   if (is_empty_queue(q)) {
       set_segments(agenda, rest_segments(agenda));
   } else {}
}
```

32. Observe that the conditional statement in this function has an empty block as its alternative statement. Such a "one-armed conditional statement" is used to decide whether to do something, rather than to select between two statements.

The first agenda item is found at the head of the queue in the first time segment. Whenever we extract an item, we also update the current time:[33]

```
function first_agenda_item(agenda) {
    if (is_empty_agenda(agenda)) {
        error("agenda is empty -- first_agenda_item");
    } else {
        const first_seg = first_segment(agenda);
        set_current_time(agenda, segment_time(first_seg));
        return front_queue(segment_queue(first_seg));
    }
}
```

Exercise 3.32

The functions to be run during each time segment of the agenda are kept in a queue. Thus, the functions for each segment are called in the order in which they were added to the agenda (first in, first out). Explain why this order must be used. In particular, trace the behavior of an and-gate whose inputs change from 0,1 to 1,0 in the same segment and say how the behavior would differ if we stored a segment's functions in an ordinary list, adding and removing functions only at the front (last in, first out).

3.3.5 Propagation of Constraints

Computer programs are traditionally organized as one-directional computations, which perform operations on prespecified arguments to produce desired outputs. On the other hand, we often model systems in terms of relations among quantities. For example, a mathematical model of a mechanical structure might include the information that the deflection d of a metal rod is related to the force F on the rod, the length L of the rod, the cross-sectional area A, and the elastic modulus E via the equation

$$dAE = FL$$

Such an equation is not one-directional. Given any four of the quantities, we can use it to compute the fifth. Yet translating the equation into a traditional computer language would force us to choose one of the quantities to be computed in terms of the other four. Thus, a function for computing the area A could not be used to compute the deflection d, even though the computations of A and d arise from the same equation.[34]

---

33. In this way, the current time will always be the time of the action most recently processed. Storing this time at the head of the agenda ensures that it will still be available even if the associated time segment has been deleted.

34. Constraint propagation first appeared in the incredibly forward-looking SKETCHPAD system of Ivan Sutherland (1963). A beautiful constraint-propagation system based on the Smalltalk language was developed by Alan Borning (1977) at Xerox Palo Alto Research Center. Sussman, Stallman, and Steele applied constraint propagation to electrical circuit analysis (Sussman and Stallman 1975; Sussman and Steele 1980). TK!Solver (Konopasek and Jayaraman 1984) is an extensive modeling environment based on constraints.

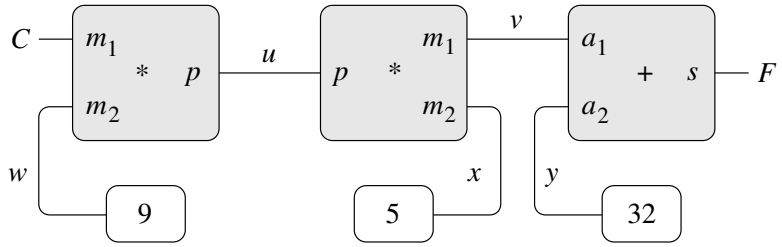


Figure 3.28 The relation $9C = 5(F - 32)$ expressed as a constraint network.

In this section, we sketch the design of a language that enables us to work in terms of relations themselves. The primitive elements of the language are *primitive constraints*, which state that certain relations hold between quantities. For example, `adder(a, b, c)` specifies that the quantities a, b, and c must be related by the equation $a + b = c$, `multiplier(x, y, z)` expresses the constraint $xy = z$, and `constant(3.14, x)` says that the value of x must be 3.14.

Our language provides a means of combining primitive constraints in order to express more complex relations. We combine constraints by constructing *constraint networks*, in which constraints are joined by *connectors*. A connector is an object that "holds" a value that may participate in one or more constraints. For example, we know that the relationship between Fahrenheit and Celsius temperatures is

$$9C = 5(F - 32)$$

Such a constraint can be thought of as a network consisting of primitive adder, multiplier, and constant constraints (figure 3.28). In the figure, we see on the left a multiplier box with three terminals, labeled m_1, m_2, and p. These connect the multiplier to the rest of the network as follows: The m_1 terminal is linked to a connector C, which will hold the Celsius temperature. The m_2 terminal is linked to a connector w, which is also linked to a constant box that holds 9. The p terminal, which the multiplier box constrains to be the product of m_1 and m_2, is linked to the p terminal of another multiplier box, whose m_2 is connected to a constant 5 and whose m_1 is connected to one of the terms in a sum.

Computation by such a network proceeds as follows: When a connector is given a value (by the user or by a constraint box to which it is linked), it awakens all of its associated constraints (except for the constraint that just awakened it) to inform them that it has a value. Each awakened constraint box then polls its connectors to see if there is enough information to determine a value for a connector. If so, the box sets that connector, which then awakens all of its associated constraints, and so on. For instance, in conversion between Celsius and Fahrenheit, w, x, and y are immediately set by the constant boxes to 9, 5, and 32, respectively. The connectors awaken the multipliers and the adder, which determine that there is not enough information to proceed. If the user (or some other part of the network) sets C to a value (say 25), the leftmost multiplier will be awakened, and it will set u to $25 \cdot 9 = 225$. Then u awakens the second multiplier, which sets v to 45, and v awakens the adder, which sets F to 77.

Using the constraint system

To use the constraint system to carry out the temperature computation outlined above, we first call the constructor `make_connector` to create two connectors, `C` and `F`, and then link them in an appropriate network:

```
const C = make_connector();
const F = make_connector();
celsius_fahrenheit_converter(C, F);
"ok"
```

The function that creates the network is defined as follows:

```
function celsius_fahrenheit_converter(c, f) {
    const u = make_connector();
    const v = make_connector();
    const w = make_connector();
    const x = make_connector();
    const y = make_connector();
    multiplier(c, w, u);
    multiplier(v, x, u);
    adder(v, y, f);
    constant(9, w);
    constant(5, x);
    constant(32, y);
    return "ok";
}
```

This function creates the internal connectors `u`, `v`, `w`, `x`, and `y`, and links them as shown in figure 3.28 using the primitive constraint constructors `adder`, `multiplier`, and `constant`. Just as with the digital-circuit simulator of section 3.3.4, expressing these combinations of primitive elements in terms of functions automatically provides our language with a means of abstraction for compound objects.

To watch the network in action, we can place probes on the connectors `C` and `F`, using a `probe` function similar to the one we used to monitor wires in section 3.3.4. Placing a probe on a connector will cause a message to be printed whenever the connector is given a value:

```
probe("Celsius temp", C);
probe("Fahrenheit temp", F);
```

Next we set the value of `C` to 25. (The third argument to `set_value` tells `C` that this directive comes from the `user`.)

```
set_value(C, 25, "user");
"Probe: Celsius temp = 25"
"Probe: Fahrenheit temp = 77"
"done"
```

The probe on `C` awakens and reports the value. `C` also propagates its value through the network as described above. This sets `F` to 77, which is reported by the probe on `F`.

Now we can try to set `F` to a new value, say 212:

```
set_value(F, 212, "user");
"Error! Contradiction: (77, 212)"
```

The connector complains that it has sensed a contradiction: Its value is 77, and someone is trying to set it to 212. If we really want to reuse the network with new values, we can tell C to forget its old value:

```
forget_value(C, "user");
"Probe: Celsius temp = ?"
"Probe: Fahrenheit temp = ?"
"done"
```

C finds that the "user", who set its value originally, is now retracting that value, so C agrees to lose its value, as shown by the probe, and informs the rest of the network of this fact. This information eventually propagates to F, which now finds that it has no reason for continuing to believe that its own value is 77. Thus, F also gives up its value, as shown by the probe.

Now that F has no value, we are free to set it to 212:

```
set_value(F, 212, "user");
"Probe: Fahrenheit temp = 212"
"Probe: Celsius temp = 100"
"done"
```

This new value, when propagated through the network, forces C to have a value of 100, and this is registered by the probe on C. Notice that the very same network is being used to compute C given F and to compute F given C. This nondirectionality of computation is the distinguishing feature of constraint-based systems.

Implementing the constraint system

The constraint system is implemented via procedural objects with local state, in a manner very similar to the digital-circuit simulator of section 3.3.4. Although the primitive objects of the constraint system are somewhat more complex, the overall system is simpler, since there is no concern about agendas and logic delays.

The basic operations on connectors are the following:

- `has_value(`*connector*`)`
  tells whether the connector has a value.
- `get_value(`*connector*`)`
  returns the connector's current value.
- `set_value(`*connector*`,` *new-value*`,` *informant*`)`
  indicates that the informant is requesting the connector to set its value to the new value.
- `forget_value(`*connector*`,` *retractor*`)`
  tells the connector that the retractor is requesting it to forget its value.
- `connect(`*connector*`,` *new-constraint*`)`
  tells the connector to participate in the new constraint.

The connectors communicate with the constraints by means of the functions `inform_about_value`, which tells the given constraint that the connector has a value, and `inform_about_no_value`, which tells the constraint that the connector has lost its value.

`Adder` constructs an adder constraint among summand connectors `a1` and `a2` and a `sum` connector. An adder is implemented as a function with local state (the function `me` below):

```
function adder(a1, a2, sum) {
    function process_new_value() {
        if (has_value(a1) && has_value(a2)) {
            set_value(sum, get_value(a1) + get_value(a2), me);
        } else if (has_value(a1) && has_value(sum)) {
            set_value(a2, get_value(sum) - get_value(a1), me);
        } else if (has_value(a2) && has_value(sum)) {
            set_value(a1, get_value(sum) - get_value(a2), me);
        } else {}
    }
    function process_forget_value() {
        forget_value(sum, me);
        forget_value(a1, me);
        forget_value(a2, me);
        process_new_value();
    }
    function me(request) {
        if (request === "I have a value.") {
            process_new_value();
        } else if (request === "I lost my value.") {
            process_forget_value();
        } else {
            error(request, "unknown request -- adder");
        }
    }
    connect(a1, me);
    connect(a2, me);
    connect(sum, me);
    return me;
}
```

The function `adder` connects the new adder to the designated connectors and returns it as its value. The function `me`, which represents the adder, acts as a dispatch to the local functions. The following "syntax interfaces" (see footnote 30 in section 3.3.4) are used in conjunction with the dispatch:

```
function inform_about_value(constraint) {
    return constraint("I have a value.");
}
function inform_about_no_value(constraint) {
    return constraint("I lost my value.");
}
```

The adder's local function `process_new_value` is called when the adder is informed that one of its connectors has a value. The adder first checks to see if both `a1` and `a2` have values. If so, it tells `sum` to set its value to the sum of the two addends. The `informant` argument to `set_value` is `me`, which is the adder object itself. If `a1` and `a2` do not both have values, then the adder checks to see if perhaps `a1` and `sum` have values. If so, it sets `a2` to the difference of these two. Finally, if `a2` and `sum` have values, this gives the adder enough information to set `a1`. If the adder is told that one of its connectors has lost a value, it requests that all of its connectors now lose their values. (Only those values that were set by this adder are actually lost.) Then it runs `process_new_value`. The reason for this last step is that one or more connectors may still have a value (that is, a connector may have had a value that was not originally set by the adder), and these values may need to be propagated back through the adder.

A multiplier is very similar to an adder. It will set its `product` to 0 if either of the factors is 0, even if the other factor is not known.

```
function multiplier(m1, m2, product) {
    function process_new_value() {
        if ((has_value(m1) && get_value(m1) === 0)
         || (has_value(m2) && get_value(m2) === 0)) {
            set_value(product, 0, me);
        } else if (has_value(m1) && has_value(m2)) {
            set_value(product, get_value(m1) * get_value(m2), me);
        } else if (has_value(product) && has_value(m1)) {
            set_value(m2, get_value(product) / get_value(m1), me);
        } else if (has_value(product) && has_value(m2)) {
            set_value(m1, get_value(product) / get_value(m2), me);
        } else {}
    }
    function process_forget_value() {
        forget_value(product, me);
        forget_value(m1, me);
        forget_value(m2, me);
        process_new_value();
    }
    function me(request) {
        if (request === "I have a value.") {
            process_new_value();
        } else if (request === "I lost my value.") {
            process_forget_value();
        } else {
            error(request, "unknown request -- multiplier");
        }
    }
    connect(m1, me);
    connect(m2, me);
    connect(product, me);
    return me;
}
```

A `constant` constructor simply sets the value of the designated connector. Any `"I have a value."` or `"I lost my value."` message sent to the constant box will produce an error.

```
function constant(value, connector) {
    function me(request) {
        error(request, "unknown request -- constant");
    }
    connect(connector, me);
    set_value(connector, value, me);
    return me;
}
```

Finally, a probe prints a message about the setting or unsetting of the designated connector:

```
function probe(name, connector) {
    function print_probe(value) {
        display("Probe: " + name + " = " + stringify(value));
    }
    function process_new_value() {
        print_probe(get_value(connector));
    }
    function process_forget_value() {
        print_probe("?");
    }
    function me(request) {
        return request === "I have a value."
               ? process_new_value()
               : request === "I lost my value."
               ? process_forget_value()
               : error(request, "unknown request -- probe");
    }
    connect(connector, me);
    return me;
}
```

Representing connectors

A connector is represented as a procedural object with local state variables `value`, the current value of the connector; `informant`, the object that set the connector's value; and `constraints`, a list of the constraints in which the connector participates.

```
function make_connector() {
    let value = false;
    let informant = false;
    let constraints = null;
    function set_my_value(newval, setter) {
        if (!has_value(me)) {
            value = newval;
            informant = setter;
            return for_each_except(setter,
                                   inform_about_value,
                                   constraints);
        } else if (value !== newval) {
            error(list(value, newval), "contradiction");
        } else {
            return "ignored";
        }
    }
    function forget_my_value(retractor) {
        if (retractor === informant) {
            informant = false;
            return for_each_except(retractor,
                                   inform_about_no_value,
                                   constraints);
        } else {
            return "ignored";
        }
    }
    function connect(new_constraint) {
        if (is_null(member(new_constraint, constraints))) {
            constraints = pair(new_constraint, constraints);
        } else {}
        if (has_value(me)) {
            inform_about_value(new_constraint);
        } else {}
        return "done";
    }
    function me(request) {
        if (request === "has_value") {
            return informant !== false;
        } else if (request === "value") {
            return value;
        } else if (request === "set_value") {
            return set_my_value;
        } else if (request === "forget") {
            return forget_my_value;
        } else if (request === "connect") {
            return connect;
        } else {
            error(request, "unknown operation -- connector");
        }
    }
    return me;
}
```

The connector's local function `set_my_value` is called when there is a request to set the connector's value. If the connector does not currently have a value, it will set its value and remember as `informant` the constraint that requested the value to be set.[35] Then the connector will notify all of its participating constraints except the constraint that requested the value to be set. This is accomplished using the following iterator, which applies a designated function to all items in a list except a given one:

```
function for_each_except(exception, fun, list) {
    function loop(items) {
        if (is_null(items)) {
            return "done";
        } else if (head(items) === exception) {
            return loop(tail(items));
        } else {
            fun(head(items));
            return loop(tail(items));
        }
    }
    return loop(list);
}
```

If a connector is asked to forget its value, it runs `forget_my_value`, a local function that first checks to make sure that the request is coming from the same object that set the value originally. If so, the connector informs its associated constraints about the loss of the value.

The local function `connect` adds the designated new constraint to the list of constraints if it is not already in that list.[36] Then, if the connector has a value, it informs the new constraint of this fact.

The connector's function `me` serves as a dispatch to the other internal functions and also represents the connector as an object. The following functions provide a syntax interface for the dispatch:

```
function has_value(connector) {
    return connector("has_value");
}
function get_value(connector) {
    return connector("value");
}
function set_value(connector, new_value, informant) {
    return connector("set_value")(new_value, informant);
}
function forget_value(connector, retractor) {
    return connector("forget")(retractor);
}
function connect(connector, new_constraint) {
    return connector("connect")(new_constraint);
}
```

35. The `setter` might not be a constraint. In our temperature example, we used `"user"` as the `setter`.

36. We can use the function `member` from section 2.3.1 to test whether `new_constraint` is already in `constraints`, although `member` was introduced as being limited to numbers and strings, because we extended `===` to pointer equality in section 3.3.1.

Exercise 3.33

Using primitive multiplier, adder, and constant constraints, define a function `averager` that takes three connectors `a`, `b`, and `c` as inputs and establishes the constraint that the value of `c` is the average of the values of `a` and `b`.

Exercise 3.34

Louis Reasoner wants to build a squarer, a constraint device with two terminals such that the value of connector `b` on the second terminal will always be the square of the value `a` on the first terminal. He proposes the following simple device made from a multiplier:

```
function squarer(a, b) {
    return multiplier(a, a, b);
}
```

There is a serious flaw in this idea. Explain.

Exercise 3.35

Ben Bitdiddle tells Louis that one way to avoid the trouble in exercise 3.34 is to define a squarer as a new primitive constraint. Fill in the missing portions in Ben's outline for a function to implement such a constraint:

```
function squarer(a, b) {
    function process_new_value() {
        if (has_value(b)) {
            if (get_value(b) < 0) {
                error(get_value(b), "square less than 0 -- squarer");
            } else {
                alternative1
            }
        } else {
            alternative2
        }
    }
    function process_forget_value() {
        body1
    }
    function me(request) {
        body2
    }
    statements
    return me;
}
```

Exercise 3.36

Suppose we evaluate the following sequence of statements in the program environment:

```
const a = make_connector();
const b = make_connector();
set_value(a, 10, "user");
```

At some time during evaluation of the `set_value`, the following expression from the connector's local function is evaluated:

```
for_each_except(setter, inform_about_value, constraints);
```

Draw an environment diagram showing the environment in which the above expression is evaluated.

Exercise 3.37

The `celsius_fahrenheit_converter` function is cumbersome when compared with a more expression-oriented style of definition, such as

```
function celsius_fahrenheit_converter(x) {
    return cplus(cmul(cdiv(cv(9), cv(5)), x), cv(32));
}

const C = make_connector();
const F = celsius_fahrenheit_converter(C);
```

Here `cplus`, `cmul`, etc. are the "constraint" versions of the arithmetic operations. For example, `cplus` takes two connectors as arguments and returns a connector that is related to these by an adder constraint:

```
function cplus(x, y) {
    const z = make_connector();
    adder(x, y, z);
    return z;
}
```

Define analogous functions `cminus`, `cmul`, `cdiv`, and `cv` (constant value) that enable us to define compound constraints as in the converter example above.[37]

3.4 Concurrency: Time Is of the Essence

We've seen the power of computational objects with local state as tools for modeling. Yet, as section 3.1.3 warned, this power extracts a price: the loss of referential transparency, giving rise to a thicket of questions about sameness and change, and the

---

37. The expression-oriented format is convenient because it avoids the need to name the intermediate expressions in a computation. Our original formulation of the constraint language is cumbersome in the same way that many languages are cumbersome when dealing with operations on compound data. For example, if we wanted to compute the product $(a+b)\cdot(c+d)$, where the variables represent vectors, we could work in "imperative style," using functions that set the values of designated vector arguments but do not themselves return vectors as values:

```
v_sum(a, b, temp1);
v_sum(c, d, temp2);
v_prod(temp1, temp2, answer);
```

Alternatively, we could deal with expressions, using functions that return vectors as values, and thus avoid explicitly mentioning `temp1` and `temp2`:

```
const answer = v_prod(v_sum(a, b), v_sum(c, d));
```

Since JavaScript allows us to return compound objects as values of functions, we can transform our imperative-style constraint language into an expression-oriented style as shown in this exercise. Given the advantage of the expression-oriented format, one might ask if there is any reason to have implemented the system in imperative style, as we did in this section. One reason is that the non-expression-oriented constraint language provides a handle on constraint objects (e.g., the value of the `adder` function) as well as on connector objects. This is useful if we wish to extend the system with new operations that communicate with constraints directly rather than only indirectly via operations on connectors. Although it is easy to implement the expression-oriented style in terms of the imperative implementation, it is very difficult to do the converse.

need to abandon the substitution model of evaluation in favor of the more intricate environment model.

The central issue lurking beneath the complexity of state, sameness, and change is that by introducing assignment we are forced to admit *time* into our computational models. Before we introduced assignment, all our programs were timeless, in the sense that any expression that has a value always has the same value. In contrast, recall the example of modeling withdrawals from a bank account and returning the resulting balance, introduced at the beginning of section 3.1.1:

```
withdraw(25);
75

withdraw(25);
50
```

Here successive evaluations of the same expression yield different values. This behavior arises from the fact that the execution of assignments (in this case, assignments to the variable `balance`) delineates *moments in time* when values change. The result of evaluating an expression depends not only on the expression itself, but also on whether the evaluation occurs before or after these moments. Building models in terms of computational objects with local state forces us to confront time as an essential concept in programming.

We can go further in structuring computational models to match our perception of the physical world. Objects in the world do not change one at a time in sequence. Rather we perceive them as acting *concurrently*—all at once. So it is often natural to model systems as collections of *threads* (sequences of computational steps) that execute concurrently.[38] Just as we can make our programs modular by organizing models in terms of objects with separate local state, it is often appropriate to divide computational models into parts that evolve separately and concurrently. Even if the programs are to be executed on a sequential computer, the practice of writing programs as if they were to be executed concurrently forces the programmer to avoid inessential timing constraints and thus makes programs more modular.

In addition to making programs more modular, concurrent computation can provide a speed advantage over sequential computation. Sequential computers execute only one operation at a time, so the amount of time it takes to perform a task is proportional to the total number of operations performed.[39] However, if it is possible to decompose a problem into pieces that are relatively independent and need to communicate only rarely, it may be possible to allocate pieces to separate computing processors, producing a speed advantage proportional to the number of processors available.

---

38. Such sequential threads are called "processes" throughout the book, but in this section we use the term "thread" to emphasize their access to shared memory.

39. Most real processors actually execute a few operations at a time, following a strategy called *pipelining*. Although this technique greatly improves the effective utilization of the hardware, it is used only to speed up the execution of a sequential instruction stream, while retaining the behavior of the sequential program.

Unfortunately, the complexities introduced by assignment become even more problematic in the presence of concurrency. The fact of concurrent execution, either because the world operates in parallel or because our computers do, entails additional complexity in our understanding of time.

3.4.1 The Nature of Time in Concurrent Systems

On the surface, time seems straightforward. It is an ordering imposed on events.[40] For any events A and B, either A occurs before B, A and B are simultaneous, or A occurs after B. For instance, returning to the bank account example, suppose that Peter withdraws \$10 and Paul withdraws \$25 from a joint account that initially contains \$100, leaving \$65 in the account. Depending on the order of the two withdrawals, the sequence of balances in the account is either $\$100 \rightarrow \$90 \rightarrow \$65$ or $\$100 \rightarrow \$75 \rightarrow \$65$. In a computer implementation of the banking system, this changing sequence of balances could be modeled by successive assignments to a variable `balance`.

In complex situations, however, such a view can be problematic. Suppose that Peter and Paul, and other people besides, are accessing the same bank account through a network of banking machines distributed all over the world. The actual sequence of balances in the account will depend critically on the detailed timing of the accesses and the details of the communication among the machines.

This indeterminacy in the order of events can pose serious problems in the design of concurrent systems. For instance, suppose that the withdrawals made by Peter and Paul are implemented as two separate threads sharing a common variable `balance`, each thread specified by the function given in section 3.1.1:

```
function withdraw(amount) {
    if (balance >= amount) {
        balance = balance - amount;
        return balance;
    } else {
        return "Insufficient funds";
    }
}
```

If the two threads operate independently, then Peter might test the balance and attempt to withdraw a legitimate amount. However, Paul might withdraw some funds in between the time that Peter checks the balance and the time Peter completes the withdrawal, thus invalidating Peter's test.

Things can be worse still. Consider the statement

```
balance = balance - amount;
```

executed as part of each withdrawal process. This consists of three steps: (1) accessing the value of the `balance` variable; (2) computing the new balance; (3) setting `balance` to this new value. If Peter and Paul's withdrawals execute this statement

40. To quote some graffiti seen on a building wall in Cambridge, Massachusetts: "Time is a device that was invented to keep everything from happening at once."

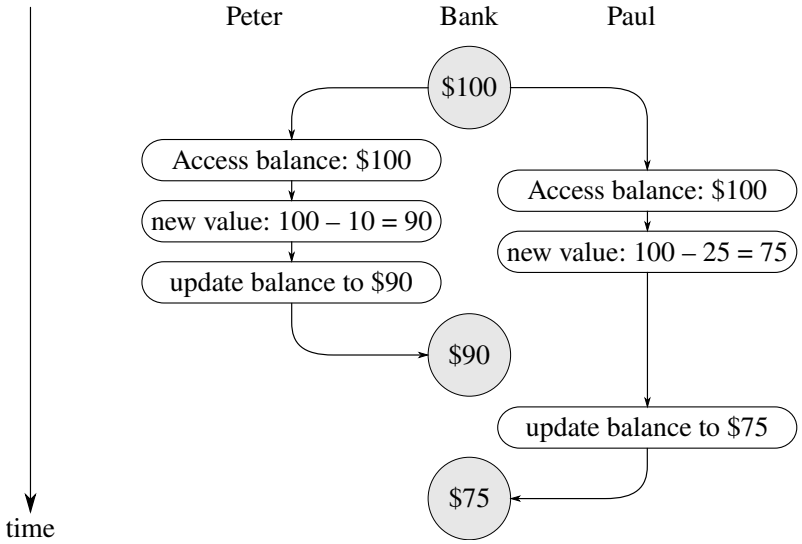


Figure 3.29 Timing diagram showing how interleaving the order of events in two banking withdrawals can lead to an incorrect final balance.

concurrently, then the two withdrawals might interleave the order in which they access `balance` and set it to the new value.

The timing diagram in figure 3.29 depicts an order of events where `balance` starts at 100, Peter withdraws 10, Paul withdraws 25, and yet the final value of `balance` is 75. As shown in the diagram, the reason for this anomaly is that Paul's assignment of 75 to `balance` is made under the assumption that the value of `balance` to be decremented is 100. That assumption, however, became invalid when Peter changed `balance` to 90. This is a catastrophic failure for the banking system, because the total amount of money in the system is not conserved. Before the transactions, the total amount of money was $100. Afterwards, Peter has $10, Paul has $25, and the bank has $75.[41]

The general phenomenon illustrated here is that several threads may share a common state variable. What makes this complicated is that more than one thread may be trying to manipulate the shared state at the same time. For the bank account example, during each transaction, each customer should be able to act as if the other customers did not exist. When customers change the balance in a way that depends

41. An even worse failure for this system could occur if the two assignments attempt to change the balance simultaneously, in which case the actual data appearing in memory might end up being a random combination of the information being written by the two threads. Most computers have interlocks on the primitive memory-write operations, which protect against such simultaneous access. Even this seemingly simple kind of protection, however, raises implementation challenges in the design of multiprocessing computers, where elaborate *cache-coherence* protocols are required to ensure that the various processors will maintain a consistent view of memory contents, despite the fact that data may be replicated ("cached") among the different processors to increase the speed of memory access.

on the balance, they must be able to assume that, just before the moment of change, the balance is still what they thought it was.

Correct behavior of concurrent programs

The above example typifies the subtle bugs that can creep into concurrent programs. The root of this complexity lies in the assignments to variables that are shared among the different threads. We already know that we must be careful in writing programs that use assignment, because the results of a computation depend on the order in which the assignments occur.[42] With concurrent threads we must be especially careful about assignments, because we may not be able to control the order of the assignments made by the different threads. If several such changes might be made concurrently (as with two depositors accessing a joint account) we need some way to ensure that our system behaves correctly. For example, in the case of withdrawals from a joint bank account, we must ensure that money is conserved. To make concurrent programs behave correctly, we may have to place some restrictions on concurrent execution.

One possible restriction on concurrency would stipulate that no two operations that change any shared state variables can occur at the same time. This is an extremely stringent requirement. For distributed banking, it would require the system designer to ensure that only one transaction could proceed at a time. This would be both inefficient and overly conservative. Figure 3.30 shows Peter and Paul sharing a bank account, where Paul has a private account as well. The diagram illustrates two withdrawals from the shared account (one by Peter and one by Paul) and a deposit to Paul's private account.[43] The two withdrawals from the shared account must not be concurrent (since both access and update the same account), and Paul's deposit and withdrawal must not be concurrent (since both access and update the amount in Paul's wallet). But there should be no problem permitting Paul's deposit to his private account to proceed concurrently with Peter's withdrawal from the shared account.

A less stringent restriction on concurrency would ensure that a concurrent system produces the same result as if the threads had run sequentially in some order. There are two important aspects to this requirement. First, it does not require the threads to actually run sequentially, but only to produce results that are the same *as if* they had run sequentially. For the example in figure 3.30, the designer of the bank account system can safely allow Paul's deposit and Peter's withdrawal to happen concurrently, because the net result will be the same as if the two operations had happened sequentially. Second, there may be more than one possible "correct" result produced by a concurrent program, because we require only that the result be the same as for *some* sequential order. For example, suppose that Peter and Paul's joint account starts out with $100, and Peter deposits $40 while Paul concurrently withdraws half the

42. The factorial program in section 3.1.3 illustrates this for a single sequential thread.

43. The columns show the contents of Peter's wallet, the joint account (in Bank1), Paul's wallet, and Paul's private account (in Bank2), before and after each withdrawal (W) and deposit (D). Peter withdraws $10 from Bank1; Paul deposits $5 in Bank2, then withdraws $25 from Bank1.

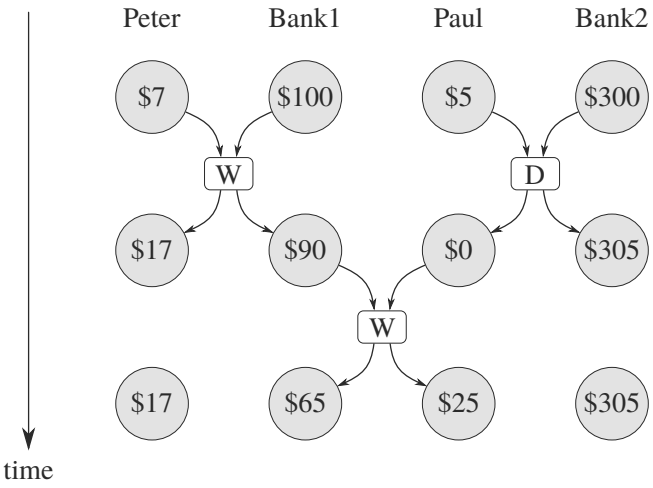


Figure 3.30 Concurrent deposits and withdrawals from a joint account in Bank1 and a private account in Bank2.

money in the account. Then sequential execution could result in the account balance being either \$70 or \$90 (see exercise 3.38).[44]

There are still weaker requirements for correct execution of concurrent programs. A program for simulating diffusion (say, the flow of heat in an object) might consist of a large number of threads, each one representing a small volume of space, that update their values concurrently. Each thread repeatedly changes its value to the average of its own value and its neighbors' values. This algorithm converges to the right answer independent of the order in which the operations are done; there is no need for any restrictions on concurrent use of the shared values.

Exercise 3.38

Suppose that Peter, Paul, and Mary share a joint bank account that initially contains \$100. Concurrently, Peter deposits \$10, Paul withdraws \$20, and Mary withdraws half the money in the account, by executing the following commands:

```
Peter:  balance = balance + 10
Paul:   balance = balance - 20
Mary:   balance = balance - (balance / 2)
```

a. List all the different possible values for `balance` after these three transactions have been completed, assuming that the banking system forces the three threads to run sequentially in some order.

---

44. A more formal way to express this idea is to say that concurrent programs are inherently *nondeterministic*. That is, they are described not by single-valued functions, but by functions whose results are sets of possible values. In section 4.3 we will study a language for expressing nondeterministic computations.

b. What are some other values that could be produced if the system allows the threads to be interleaved? Draw timing diagrams like the one in figure 3.29 to explain how these values can occur.

3.4.2 Mechanisms for Controlling Concurrency

We've seen that the difficulty in dealing with concurrent threads is rooted in the need to consider the interleaving of the order of events in the different threads. For example, suppose we have two threads, one with three ordered events (a, b, c) and one with three ordered events (x, y, z). If the two threads run concurrently, with no constraints on how their execution is interleaved, then there are 20 different possible orderings for the events that are consistent with the individual orderings for the two threads:

(a, b, c, x, y, z)	(a, x, b, y, c, z)	(x, a, b, c, y, z)	(x, a, y, z, b, c)
(a, b, x, c, y, z)	(a, x, b, y, z, c)	(x, a, b, y, c, z)	(x, y, a, b, c, z)
(a, b, x, y, c, z)	(a, x, y, b, c, z)	(x, a, b, y, z, c)	(x, y, a, b, z, c)
(a, b, x, y, z, c)	(a, x, y, b, z, c)	(x, a, y, b, c, z)	(x, y, a, z, b, c)
(a, x, b, c, y, z)	(a, x, y, z, b, c)	(x, a, y, b, z, c)	(x, y, z, a, b, c)

As programmers designing this system, we would have to consider the effects of each of these 20 orderings and check that each behavior is acceptable. Such an approach rapidly becomes unwieldy as the numbers of threads and events increase.

A more practical approach to the design of concurrent systems is to devise general mechanisms that allow us to constrain the interleaving of concurrent threads so that we can be sure that the program behavior is correct. Many mechanisms have been developed for this purpose. In this section, we describe one of them, the *serializer*.

Serializing access to shared state

Serialization implements the following idea: Threads will execute concurrently, but there will be certain collections of functions that cannot be executed concurrently. More precisely, serialization creates distinguished sets of functions such that only one execution of a function in each serialized set is permitted to happen at a time. If some function in the set is being executed, then a thread that attempts to execute any function in the set will be forced to wait until the first execution has finished.

We can use serialization to control access to shared variables. For example, if we want to update a shared variable based on the previous value of that variable, we put the access to the previous value of the variable and the assignment of the new value to the variable in the same function. We then ensure that no other function that assigns to the variable can run concurrently with this function by serializing all of these functions with the same serializer. This guarantees that the value of the variable cannot be changed between an access and the corresponding assignment.

Serializers

To make the above mechanism more concrete, suppose that we have extended JavaScript to include a function called `concurrent_execute`:

`concurrent_execute(`f_1`,` f_2`, ...,` f_k`)`

Each f must be a function of no arguments. The function `concurrent_execute` creates a separate thread for each f, which applies f (to no arguments). These threads all run concurrently.[45]

As an example of how this is used, consider

```
let x = 10;
concurrent_execute(() => { x = x * x; },
                   () => { x = x + 1; });
```

This creates two concurrent threads—T_1, which sets `x` to `x` times `x`, and T_2, which increments `x`. After execution is complete, `x` will be left with one of five possible values, depending on the interleaving of the events of T_1 and T_2:

101: T_1 sets `x` to 100 and then T_2 increments `x` to 101.

121: T_2 increments `x` to 11 and then T_1 sets `x` to `x` times `x`.

110: T_2 changes `x` from 10 to 11 between the two times that T_1 accesses the value of `x` during the evaluation of `x * x`.

11: T_2 accesses `x`, then T_1 sets `x` to 100, then T_2 sets `x`.

100: T_1 accesses `x` (twice), then T_2 sets `x` to 11, then T_1 sets `x`.

We can constrain the concurrency by using serialized functions, which are created by *serializers*. Serializers are constructed by `make_serializer`, whose implementation is given below. A serializer takes a function as argument and returns a serialized function that behaves like the original function. All calls to a given serializer return serialized functions in the same set.

Thus, in contrast to the example above, executing

```
let x = 10;
const s = make_serializer();
concurrent_execute(s(() => { x = x * x; }),
                   s(() => { x = x + 1; }));
```

can produce only two possible values for `x`, 101 or 121. The other possibilities are eliminated, because the execution of T_1 and T_2 cannot be interleaved.

Here is a version of the `make_account` function from section 3.1.1, where the deposits and withdrawals have been serialized:

---

45. The function `concurrent_execute` is not part of the JavaScript standard, but the examples in this section can be implemented in ECMAScript 2020.

```
function make_account(balance) {
    function withdraw(amount) {
        if (balance > amount) {
            balance = balance - amount;
            return balance;
        } else {
            return "Insufficient funds";
        }
    }
    function deposit(amount) {
        balance = balance + amount;
        return balance;
    }
    const protect = make_serializer();
    function dispatch(m) {
        return m === "withdraw"
               ? protect(withdraw)
               : m === "deposit"
               ? protect(deposit)
               : m === "balance"
               ? balance
               : error(m, "unknown request -- make_account");
    }
    return dispatch;
}
```

With this implementation, two threads cannot be withdrawing from or depositing into a single account concurrently. This eliminates the source of the error illustrated in figure 3.29, where Peter changes the account balance between the times when Paul accesses the balance to compute the new value and when Paul actually performs the assignment. On the other hand, each account has its own serializer, so that deposits and withdrawals for different accounts can proceed concurrently.

Exercise 3.39

Which of the five possibilities in the concurrent execution shown above remain if we instead serialize execution as follows:

```
let x = 10;
const s = make_serializer();
concurrent_execute(  () => { x = s(() => x * x)(); },
                   s(() => { x = x + 1;            }));
```

Exercise 3.40

Give all possible values of `x` that can result from executing

```
let x = 10;
concurrent_execute(() => { x = x * x; },
                   () => { x = x * x * x; });
```

Which of these possibilities remain if we instead use serialized functions:

```
let x = 10;
const s = make_serializer();
concurrent_execute(s(() => { x = x * x;     }),
                   s(() => { x = x * x * x; }));
```

Exercise 3.41

Ben Bitdiddle worries that it would be better to implement the bank account as follows (where the commented line has been changed):

```
function make_account(balance) {
    function withdraw(amount) {
        if (balance > amount) {
            balance = balance - amount;
            return balance;
        } else {
            return "Insufficient funds";
        }
    }
    function deposit(amount) {
        balance = balance + amount;
        return balance;
    }
    const protect = make_serializer();
    function dispatch(m) {
        return m === "withdraw"
               ? protect(withdraw)
               : m === "deposit"
               ? protect(deposit)
               : m === "balance"
               ? protect(() => balance)(undefined) // serialized
               : error(m, "unknown request -- make_account");
    }
    return dispatch;
}
```

because allowing unserialized access to the bank balance can result in anomalous behavior. Do you agree? Is there any scenario that demonstrates Ben's concern?

Exercise 3.42

Ben Bitdiddle suggests that it's a waste of time to create a new serialized function in response to every `withdraw` and `deposit` message. He says that `make_account` could be changed so that the calls to `protect` are done outside the `dispatch` function. That is, an account would return the same serialized function (which was created at the same time as the account) each time it is asked for a withdrawal function.

```
function make_account(balance) {
    function withdraw(amount) {
        if (balance > amount) {
            balance = balance - amount;
            return balance;
        } else {
            return "Insufficient funds";
        }
    }
    function deposit(amount) {
        balance = balance + amount;
        return balance;
    }
    const protect = make_serializer();
    const protect_withdraw = protect(withdraw);
    const protect_deposit = protect(deposit);
    function dispatch(m) {
        return m === "withdraw"
               ? protect_withdraw
               : m === "deposit"
               ? protect_deposit
               : m === "balance"
               ? balance
               : error(m, "unknown request -- make_account");
    }
    return dispatch;
}
```

Is this a safe change to make? In particular, is there any difference in what concurrency is allowed by these two versions of `make_account` ?

Complexity of using multiple shared resources

Serializers provide a powerful abstraction that helps isolate the complexities of concurrent programs so that they can be dealt with carefully and (hopefully) correctly. However, while using serializers is relatively straightforward when there is only a single shared resource (such as a single bank account), concurrent programming can be treacherously difficult when there are multiple shared resources.

To illustrate one of the difficulties that can arise, suppose we wish to swap the balances in two bank accounts. We access each account to find the balance, compute the difference between the balances, withdraw this difference from one account, and deposit it in the other account. We could implement this as follows:[46]

```
function exchange(account1, account2) {
    const difference = account1("balance") - account2("balance");
    account1("withdraw")(difference);
    account2("deposit")(difference);
}
```

46. We have simplified `exchange` by exploiting the fact that our `deposit` message accepts negative amounts. (This is a serious bug in our banking system!)

This function works well when only a single thread is trying to do the exchange. Suppose, however, that Peter and Paul both have access to accounts a_1, a_2, and a_3, and that Peter exchanges a_1 and a_2 while Paul concurrently exchanges a_1 and a_3. Even with account deposits and withdrawals serialized for individual accounts (as in the `make_account` function shown above in this section), `exchange` can still produce incorrect results. For example, Peter might compute the difference in the balances for a_1 and a_2, but then Paul might change the balance in a_1 before Peter is able to complete the exchange.[47] For correct behavior, we must arrange for the `exchange` function to lock out any other concurrent accesses to the accounts during the entire time of the exchange.

One way we can accomplish this is by using both accounts' serializers to serialize the entire `exchange` function. To do this, we will arrange for access to an account's serializer. Note that we are deliberately breaking the modularity of the bank-account object by exposing the serializer. The following version of `make_account` is identical to the original version given in section 3.1.1, except that a serializer is provided to protect the balance variable, and the serializer is exported via message passing:

```
function make_account_and_serializer(balance) {
    function withdraw(amount) {
        if (balance > amount) {
            balance = balance - amount;
            return balance;
        } else {
            return "Insufficient funds";
        }
    }
    function deposit(amount) {
        balance = balance + amount;
        return balance;
    }
    const balance_serializer = make_serializer();
    return m => m === "withdraw"
                ? withdraw
                : m === "deposit"
                ? deposit
                : m === "balance"
                ? balance
                : m === "serializer"
                ? balance_serializer
                : error(m, "unknown request -- make_account");
}
```

47. If the account balances start out as \$10, \$20, and \$30, then after any number of concurrent exchanges, the balances should still be \$10, \$20, and \$30 in some order. Serializing the deposits to individual accounts is not sufficient to guarantee this. See exercise 3.43.

We can use this to do serialized deposits and withdrawals. However, unlike our earlier serialized account, it is now the responsibility of each user of bank-account objects to explicitly manage the serialization, for example as follows:[48]

```
function deposit(account, amount) {
    const s = account("serializer");
    const d = account("deposit");
    s(d)(amount);
}
```

Exporting the serializer in this way gives us enough flexibility to implement a serialized exchange program. We simply serialize the original `exchange` function with the serializers for both accounts:

```
function serialized_exchange(account1, account2) {
    const serializer1 = account1("serializer");
    const serializer2 = account2("serializer");
    serializer1(serializer2(exchange))(account1, account2);
}
```

Exercise 3.43

Suppose that the balances in three accounts start out as $10, $20, and $30, and that multiple threads run, exchanging the balances in the accounts. Argue that if the threads are run sequentially, after any number of concurrent exchanges, the account balances should be $10, $20, and $30 in some order. Draw a timing diagram like the one in figure 3.29 to show how this condition can be violated if the exchanges are implemented using the first version of the account-exchange program in this section. On the other hand, argue that even with this `exchange` program, the sum of the balances in the accounts will be preserved. Draw a timing diagram to show how even this condition would be violated if we did not serialize the transactions on individual accounts.

Exercise 3.44

Consider the problem of transferring an amount from one account to another. Ben Bitdiddle claims that this can be accomplished with the following function, even if there are multiple people concurrently transferring money among multiple accounts, using any account mechanism that serializes deposit and withdrawal transactions, for example, the version of `make_account` in the text above.

```
function transfer(from_account, to_account, amount) {
    from_account("withdraw")(amount);
    to_account("deposit")(amount);
}
```

---

48. Exercise 3.45 investigates why deposits and withdrawals are no longer automatically serialized by the account.

Louis Reasoner claims that there is a problem here, and that we need to use a more sophisticated method, such as the one required for dealing with the exchange problem. Is Louis right? If not, what is the essential difference between the transfer problem and the exchange problem? (You should assume that the balance in `from_account` is at least `amount`.)

Exercise 3.45

Louis Reasoner thinks our bank-account system is unnecessarily complex and error-prone now that deposits and withdrawals aren't automatically serialized. He suggests that `make_account_and_serializer` should have exported the serializer (for use by such functions as `serialized_exchange`) in addition to (rather than instead of) using it to serialize accounts and deposits as `make_account` did. He proposes to redefine accounts as follows:

```
function make_account_and_serializer(balance) {
    function withdraw(amount) {
        if (balance > amount) {
            balance = balance - amount;
            return balance;
        } else {
            return "Insufficient funds";
        }
    }
    function deposit(amount) {
        balance = balance + amount;
        return balance;
    }
    const balance_serializer = make_serializer();
    return m => m === "withdraw"
                ? balance_serializer(withdraw)
                : m === "deposit"
                ? balance_serializer(deposit)
                : m === "balance"
                ? balance
                : m === "serializer"
                ? balance_serializer
                : error(m, "unknown request -- make_account");
}
```

Then deposits are handled as with the original `make_account`:

```
function deposit(account, amount) {
    account("deposit")(amount);
}
```

Explain what is wrong with Louis's reasoning. In particular, consider what happens when `serialized_exchange` is called.

Implementing serializers

We implement serializers in terms of a more primitive synchronization mechanism called a *mutex*. A mutex is an object that supports two operations—the mutex can be *acquired*, and the mutex can be *released*. Once a mutex has been acquired, no other acquire operations on that mutex may proceed until the mutex is released.[49] In our implementation, each serializer has an associated mutex. Given a function `f`, the serializer returns a function that acquires the mutex, runs `f`, and then releases the mutex. This ensures that only one of the functions produced by the serializer can be running at once, which is precisely the serialization property that we need to guarantee. To apply serializers to functions that take an arbitrary number of arguments, we use JavaScript's *rest* parameter and *spread* syntax. The `...` in front of the parameter `args` collects the rest (here all) of the arguments of any call of the function into a *vector* data structure. The `...` in front of `args` in the application `f(...args)` spreads the elements of `args` so that they become separate arguments of `f`.

```
function make_serializer() {
    const mutex = make_mutex();
    return f => {
                function serialized_f(...args) {
                    mutex("acquire");
                    const val = f(...args);
                    mutex("release");
                    return val;
                }
                return serialized_f;
            };
}
```

The mutex is a mutable object (here we'll use a one-element list, which we'll refer to as a *cell*) that can hold the value true or false. When the value is false, the mutex is available to be acquired. When the value is true, the mutex is unavailable, and any thread that attempts to acquire the mutex must wait.

Our mutex constructor `make_mutex` begins by initializing the cell contents to false. To acquire the mutex, we test the cell. If the mutex is available, we set the cell contents to true and proceed. Otherwise, we wait in a loop, attempting to acquire

---

49. The term "mutex" is an abbreviation for *mutual exclusion*. The general problem of arranging a mechanism that permits concurrent threads to safely share resources is called the mutual exclusion problem. Our mutex is a simple variant of the *semaphore* mechanism (see exercise 3.47), which was introduced in the "THE" Multiprogramming System developed at the Technological University of Eindhoven and named for the university's initials in Dutch (Dijkstra 1968a). The acquire and release operations were originally called P and V, from the Dutch words *passeren* (to pass) and *vrijgeven* (to release), in reference to the semaphores used on railroad systems. Dijkstra's classic exposition (1968b) was one of the first to clearly present the issues of concurrency control, and showed how to use semaphores to handle a variety of concurrency problems.

over and over again, until we find that the mutex is available.[50] To release the mutex, we set the cell contents to false.

```
function make_mutex() {
    const cell = list(false);
    function the_mutex(m) {
        return m === "acquire"
               ? test_and_set(cell)
                 ? the_mutex("acquire") // retry
                 : true
               : m === "release"
               ? clear(cell)
               : error(m, "unknown request -- mutex");
    }
    return the_mutex;
}
function clear(cell) {
    set_head(cell, false);
}
```

The function `test_and_set` tests the cell and returns the result of the test. In addition, if the test was false, `test_and_set` sets the cell contents to true before returning false. We can express this behavior as the following function:

```
function test_and_set(cell) {
    if (head(cell)) {
        return true;
    } else {
        set_head(cell, true);
        return false;
    }
}
```

However, this implementation of `test_and_set` does not suffice as it stands. There is a crucial subtlety here, which is the essential place where concurrency control enters the system: The `test_and_set` operation must be performed *atomically*. That is, we must guarantee that, once a thread has tested the cell and found it to be false, the cell contents will actually be set to true before any other thread can test the cell. If we do not make this guarantee, then the mutex can fail in a way similar to the bank-account failure in figure 3.29. (See exercise 3.46.)

The actual implementation of `test_and_set` depends on the details of how our system runs concurrent threads. For example, we might be executing concurrent threads on a sequential processor using a time-slicing mechanism that cycles through the threads, permitting each thread to run for a short time before interrupting it and moving on to the next thread. In that case, `test_and_set` can work by disabling

50. In most time-shared operating systems, threads that are blocked by a mutex do not waste time "busy-waiting" as above. Instead, the system schedules another thread to run while the first is waiting, and the blocked thread is awakened when the mutex becomes available.

time slicing during the testing and setting. Alternatively, multiprocessing computers provide instructions that support atomic operations directly in hardware.[51]

Exercise 3.46

Suppose that we implement `test_and_set` using an ordinary function as shown in the text, without attempting to make the operation atomic. Draw a timing diagram like the one in figure 3.29 to demonstrate how the mutex implementation can fail by allowing two threads to acquire the mutex at the same time.

Exercise 3.47

A semaphore (of size n) is a generalization of a mutex. Like a mutex, a semaphore supports acquire and release operations, but it is more general in that up to n threads can acquire it concurrently. Additional threads that attempt to acquire the semaphore must wait for release operations. Give implementations of semaphores

a. in terms of mutexes

b. in terms of atomic `test_and_set` operations.

Deadlock

Now that we have seen how to implement serializers, we can see that account exchanging still has a problem, even with the `serialized_exchange` function above. Imagine that Peter attempts to exchange a_1 with a_2 while Paul concurrently attempts to exchange a_2 with a_1. Suppose that Peter's thread reaches the point where it has entered a serialized function protecting a_1 and, just after that, Paul's thread enters a serialized function protecting a_2. Now Peter cannot proceed (to enter a serialized function protecting a_2) until Paul exits the serialized function protecting a_2. Similarly, Paul cannot proceed until Peter exits the serialized function protecting a_1. Each thread is stalled forever, waiting for the other. This situation is called a *deadlock*. Deadlock is always a danger in systems that provide concurrent access to multiple shared resources.

One way to avoid the deadlock in this situation is to give each account a unique identification number and rewrite `serialized_exchange` so that a thread will always attempt to enter a function protecting the lowest-numbered account first. Although this method works well for the exchange problem, there are other situations

51. There are many variants of such instructions—including test-and-set, test-and-clear, swap, compare-and-exchange, load-reserve, and store-conditional—whose design must be carefully matched to the machine's processor–memory interface. One issue that arises here is to determine what happens if two threads attempt to acquire the same resource at exactly the same time by using such an instruction. This requires some mechanism for making a decision about which thread gets control. Such a mechanism is called an *arbiter*. Arbiters usually boil down to some sort of hardware device. Unfortunately, it is possible to prove that one cannot physically construct a fair arbiter that works 100% of the time unless one allows the arbiter an arbitrarily long time to make its decision. The fundamental phenomenon here was originally observed by the fourteenth-century French philosopher Jean Buridan in his commentary on Aristotle's *De caelo*. Buridan argued that a perfectly rational dog placed between two equally attractive sources of food will starve to death, because it is incapable of deciding which to go to first.

that require more sophisticated deadlock-avoidance techniques, or where deadlock cannot be avoided at all. (See exercises 3.48 and 3.49.)[52]

Exercise 3.48

Explain in detail why the deadlock-avoidance method described above, (i.e., the accounts are numbered, and each thread attempts to acquire the smaller-numbered account first) avoids deadlock in the exchange problem. Rewrite `serialized_exchange` to incorporate this idea. (You will also need to modify `make_account` so that each account is created with a number, which can be accessed by sending an appropriate message.)

Exercise 3.49

Give a scenario where the deadlock-avoidance mechanism described above does not work. (Hint: In the exchange problem, each thread knows in advance which accounts it will need to get access to. Consider a situation where a thread must get access to some shared resources before it can know which additional shared resources it will require.)

Concurrency, time, and communication

We've seen how programming concurrent systems requires controlling the ordering of events when different threads access shared state, and we've seen how to achieve this control through judicious use of serializers. But the problems of concurrency lie deeper than this, because, from a fundamental point of view, it's not always clear what is meant by "shared state."

Mechanisms such as `test_and_set` require threads to examine a global shared flag at arbitrary times. This is problematic and inefficient to implement in modern high-speed processors, where due to optimization techniques such as pipelining and cached memory, the contents of memory may not be in a consistent state at every instant. In some multiprocessing systems, therefore, the serializer paradigm is being supplanted by other approaches to concurrency control.[53]

The problematic aspects of shared state also arise in large, distributed systems. For instance, imagine a distributed banking system where individual branch banks maintain local values for bank balances and periodically compare these with values maintained by other branches. In such a system the value of "the account balance"

---

52. The general technique for avoiding deadlock by numbering the shared resources and acquiring them in order is due to Havender (1968). Situations where deadlock cannot be avoided require *deadlock-recovery* methods, which entail having threads "back out" of the deadlocked state and try again. Deadlock-recovery mechanisms are widely used in data-base-management systems, a topic that is treated in detail in Gray and Reuter 1993.

53. One such alternative to serialization is called *barrier synchronization*. The programmer permits concurrent threads to execute as they please, but establishes certain synchronization points ("barriers") through which no thread can proceed until all the threads have reached the barrier. Some processors provide machine instructions that permit programmers to establish synchronization points at places where consistency is required. The PowerPC™, for example, includes for this purpose two instructions called SYNC and EIEIO (Enforced In-order Execution of Input/Output).

would be undetermined, except right after synchronization. If Peter deposits money in an account he holds jointly with Paul, when should we say that the account balance has changed—when the balance in the local branch changes, or not until after the synchronization? And if Paul accesses the account from a different branch, what are the reasonable constraints to place on the banking system such that the behavior is "correct"? The only thing that might matter for correctness is the behavior observed by Peter and Paul individually and the "state" of the account immediately after synchronization. Questions about the "real" account balance or the order of events between synchronizations may be irrelevant or meaningless.[54]

The basic phenomenon here is that synchronizing different threads, establishing shared state, or imposing an order on events requires communication among the threads. In essence, any notion of time in concurrency control must be intimately tied to communication.[55] It is intriguing that a similar connection between time and communication also arises in the Theory of Relativity, where the speed of light (the fastest signal that can be used to synchronize events) is a fundamental constant relating time and space. The complexities we encounter in dealing with time and state in our computational models may in fact mirror a fundamental complexity of the physical universe.

3.5 Streams

We've gained a good understanding of assignment as a tool in modeling, as well as an appreciation of the complex problems that assignment raises. It is time to ask whether we could have gone about things in a different way, so as to avoid some of these problems. In this section, we explore an alternative approach to modeling state, based on data structures called *streams*. As we shall see, streams can mitigate some of the complexity of modeling state.

Let's step back and review where this complexity comes from. In an attempt to model real-world phenomena, we made some apparently reasonable decisions: We modeled real-world objects with local state by computational objects with local variables. We identified time variation in the real world with time variation in the computer. We implemented the time variation of the states of the model objects in the computer with assignments to the local variables of the model objects.

Is there another approach? Can we avoid identifying time in the computer with time in the modeled world? Must we make the model change with time in order to

---

54. This may seem like a strange point of view, but there are systems that work this way. International charges to credit-card accounts, for example, are normally cleared on a per-country basis, and the charges made in different countries are periodically reconciled. Thus the account balance may be different in different countries.

55. For distributed systems, this perspective was pursued by Lamport (1978), who showed how to use communication to establish "global clocks" that can be used to establish orderings on events in distributed systems.

model phenomena in a changing world? Think about the issue in terms of mathematical functions. We can describe the time-varying behavior of a quantity x as a function of time $x(t)$. If we concentrate on x instant by instant, we think of it as a changing quantity. Yet if we concentrate on the entire time history of values, we do not emphasize change—the function itself does not change.[56]

If time is measured in discrete steps, then we can model a time function as a (possibly infinite) sequence. In this section, we will see how to model change in terms of sequences that represent the time histories of the systems being modeled. To accomplish this, we introduce new data structures called *streams*. From an abstract point of view, a stream is simply a sequence. However, we will find that the straightforward implementation of streams as lists (as in section 2.2.1) doesn't fully reveal the power of stream processing. As an alternative, we introduce the technique of *delayed evaluation*, which enables us to represent very large (even infinite) sequences as streams.

Stream processing lets us model systems that have state without ever using assignment or mutable data. This has important implications, both theoretical and practical, because we can build models that avoid the drawbacks inherent in introducing assignment. On the other hand, the stream framework raises difficulties of its own, and the question of which modeling technique leads to more modular and more easily maintained systems remains open.

3.5.1 Streams Are Delayed Lists

As we saw in section 2.2.3, sequences can serve as standard interfaces for combining program modules. We formulated powerful abstractions for manipulating sequences, such as `map`, `filter`, and `accumulate`, that capture a wide variety of operations in a manner that is both succinct and elegant.

Unfortunately, if we represent sequences as lists, this elegance is bought at the price of severe inefficiency with respect to both the time and space required by our computations. When we represent manipulations on sequences as transformations of lists, our programs must construct and copy data structures (which may be huge) at every step of a process.

To see why this is true, let us compare two programs for computing the sum of all the prime numbers in an interval. The first program is written in standard iterative style:[57]

---

56. Physicists sometimes adopt this view by introducing the "world lines" of particles as a device for reasoning about motion. We've also already mentioned (section 2.2.3) that this is the natural way to think about signal-processing systems. We will explore applications of streams to signal processing in section 3.5.3.

57. Assume that we have a predicate `is_prime` (e.g., as in section 1.2.6) that tests for primality.

```
function sum_primes(a, b) {
    function iter(count, accum) {
        return count > b
               ? accum
               : is_prime(count)
               ? iter(count + 1, count + accum)
               : iter(count + 1, accum);
    }
    return iter(a, 0);
}
```

The second program performs the same computation using the sequence operations of section 2.2.3:

```
function sum_primes(a, b) {
    return accumulate((x, y) => x + y,
                      0,
                      filter(is_prime,
                             enumerate_interval(a, b)));
}
```

In carrying out the computation, the first program needs to store only the sum being accumulated. In contrast, the filter in the second program cannot do any testing until `enumerate_interval` has constructed a complete list of the numbers in the interval. The filter generates another list, which in turn is passed to `accumulate` before being collapsed to form a sum. Such large intermediate storage is not needed by the first program, which we can think of as enumerating the interval incrementally, adding each prime to the sum as it is generated.

The inefficiency in using lists becomes painfully apparent if we use the sequence paradigm to compute the second prime in the interval from 10,000 to 1,000,000 by evaluating the expression

```
head(tail(filter(is_prime,
                 enumerate_interval(10000, 1000000))));
```

This expression does find the second prime, but the computational overhead is outrageous. We construct a list of almost a million integers, filter this list by testing each element for primality, and then ignore almost all of the result. In a more traditional programming style, we would interleave the enumeration and the filtering, and stop when we reached the second prime.

Streams are a clever idea that allows one to use sequence manipulations without incurring the costs of manipulating sequences as lists. With streams we can achieve the best of both worlds: We can formulate programs elegantly as sequence manipulations, while attaining the efficiency of incremental computation. The basic idea is to arrange to construct a stream only partially, and to pass the partial construction to the program that consumes the stream. If the consumer attempts to access a part of the stream that has not yet been constructed, the stream will automatically construct just enough more of itself to produce the required part, thus preserving the illusion that the entire stream exists. In other words, although we will write programs as if we were processing complete sequences, we design our stream implementation to automatically and transparently interleave the construction of the stream with its use.

To accomplish this, we will construct streams using pairs, with the first item of the stream in the head of the pair. However, rather than placing the value of the rest of the stream into the tail of the pair, we will put there a "promise" to compute the rest if it is ever requested. If we have a data item `h` and a stream `t`, we construct a stream whose head is `h` and whose tail is `t` by evaluating `pair(h, () => t)`—the tail `t` of a stream is "wrapped" in a function of no arguments, so that its evaluation will be *delayed*. The empty stream is `null`, the same as the empty list.

To access the first data item of a nonempty stream, we simply select the `head` of the pair, as with a list. But to access the tail of a stream, we need to evaluate the delayed expression. For convenience, we define

```
function stream_tail(stream) {
    return tail(stream)();
}
```

This selects the tail of the pair and applies the function found there to obtain the next pair of the stream (or `null` if the tail of the stream is empty)—in effect, *forcing* the function in the tail of the pair to fulfill its promise.

We can make and use streams, in just the same way as we can make and use lists, to represent aggregate data arranged in a sequence. In particular, we can build stream analogs of the list operations from chapter 2, such as `list_ref`, `map`, and `for_each`:[58]

```
function stream_ref(s, n) {
    return n === 0
           ? head(s)
           : stream_ref(stream_tail(s), n - 1);
}
function stream_map(f, s) {
    return is_null(s)
           ? null
           : pair(f(head(s)),
                  () => stream_map(f, stream_tail(s)));
}
function stream_for_each(fun, s) {
    if (is_null(s)) {
        return true;
    } else {
        fun(head(s));
        return stream_for_each(fun, stream_tail(s));
    }
}
```

58. This should bother you. The fact that we are defining such similar functions for streams and lists indicates that we are missing some underlying abstraction. Unfortunately, in order to exploit this abstraction, we will need to exert finer control over the process of evaluation than we can at present. We will discuss this point further at the end of section 3.5.4. In section 4.2, we'll develop a framework that unifies lists and streams.

The function `stream_for_each` is useful for viewing streams:

```
function display_stream(s) {
    return stream_for_each(display, s);
}
```

To make the stream implementation automatically and transparently interleave the construction of a stream with its use, we have arranged for the tail of a stream to be evaluated when it is accessed by the `stream_tail` function rather than when the stream is constructed by `pair`. This implementation choice is reminiscent of our discussion of rational numbers in section 2.1.2, where we saw that we can choose to implement rational numbers so that the reduction of numerator and denominator to lowest terms is performed either at construction time or at selection time. The two rational-number implementations produce the same data abstraction, but the choice has an effect on efficiency. There is a similar relationship between streams and ordinary lists. As a data abstraction, streams are the same as lists. The difference is the time at which the elements are evaluated. With ordinary lists, both the `head` and the `tail` are evaluated at construction time. With streams, the `tail` is evaluated at selection time.

Streams in action

To see how this data structure behaves, let us analyze the "outrageous" prime computation we saw above, reformulated in terms of streams:

```
head(stream_tail(stream_filter(
                    is_prime,
                    stream_enumerate_interval(10000, 1000000))));
```

We will see that it does indeed work efficiently.

We begin by calling `stream_enumerate_interval` with the arguments 10,000 and 1,000,000. The function `stream_enumerate_interval` is the stream analog of `enumerate_interval` (section 2.2.3):

```
function stream_enumerate_interval(low, high) {
    return low > high
           ? null
           : pair(low,
                  () => stream_enumerate_interval(low + 1, high));
}
```

and thus the result returned by `stream_enumerate_interval`, formed by the `pair`, is[59]

```
pair(10000, () => stream_enumerate_interval(10001, 1000000));
```

59. The numbers shown here do not really appear in the delayed expression. What actually appears is the original expression, in an environment in which the variables are bound to the appropriate numbers. For example, `low + 1` with `low` bound to 10,000 actually appears where `10001` is shown.

That is, stream_enumerate_interval returns a stream represented as a pair whose head is 10,000 and whose tail is a promise to enumerate more of the interval if so requested. This stream is now filtered for primes, using the stream analog of the filter function (section 2.2.3):

```
function stream_filter(pred, stream) {
    return is_null(stream)
           ? null
           : pred(head(stream))
           ? pair(head(stream),
                  () => stream_filter(pred, stream_tail(stream)))
           : stream_filter(pred, stream_tail(stream));
}
```

The function stream_filter tests the head of the stream (which is 10,000). Since this is not prime, stream_filter examines the tail of its input stream. The call to stream_tail forces evaluation of the delayed stream_enumerate_interval, which now returns

```
pair(10001, () => stream_enumerate_interval(10002, 1000000));
```

The function stream_filter now looks at the head of this stream, 10,001, sees that this is not prime either, forces another stream_tail, and so on, until stream_enumerate_interval yields the prime 10,007, whereupon stream_filter, according to its definition, returns

```
pair(head(stream),
     () => stream_filter(pred, stream_tail(stream)));
```

which in this case is

```
pair(10007,
     () => stream_filter(
               is_prime,
               pair(10008,
                    () => stream_enumerate_interval(10009, 1000000))));
```

This result is now passed to stream_tail in our original expression. This forces the delayed stream_filter, which in turn keeps forcing the delayed stream_enumerate_interval until it finds the next prime, which is 10,009. Finally, the result passed to head in our original expression is

```
pair(10009,
     () => stream_filter(
               is_prime,
               pair(10010,
                    () => stream_enumerate_interval(10011, 1000000))));
```

The function head returns 10,009, and the computation is complete. Only as many integers were tested for primality as were necessary to find the second prime, and the interval was enumerated only as far as was necessary to feed the prime filter.

In general, we can think of delayed evaluation as "demand-driven" programming, whereby each stage in the stream process is activated only enough to satisfy the next stage. What we have done is to decouple the actual order of events in the computation from the apparent structure of our functions. We write functions as if the streams existed "all at once" when, in reality, the computation is performed incrementally, as in traditional programming styles.

An optimization

When we construct stream pairs, we delay the evaluation of their tail expressions by wrapping these expressions in a function. We force their evaluation when needed, by applying the function.

This implementation suffices for streams to work as advertised, but there is an important optimization that we shall consider where needed. In many applications, we end up forcing the same delayed object many times. This can lead to serious inefficiency in recursive programs involving streams. (See exercise 3.57.) The solution is to build delayed objects so that the first time they are forced, they store the value that is computed. Subsequent forcings will simply return the stored value without repeating the computation. In other words, we implement the construction of stream pairs as a memoized function similar to the one described in exercise 3.27. One way to accomplish this is to use the following function, which takes as argument a function (of no arguments) and returns a memoized version of the function. The first time the memoized function is run, it saves the computed result. On subsequent evaluations, it simply returns the result.[60]

```
function memo(fun) {
    let already_run = false;
    let result = undefined;
    return () => {
                if (!already_run) {
                    result = fun();
                    already_run = true;
                    return result;
                } else {
                    return result;
                }
            };
}
```

We can make use of `memo` whenever we construct a stream pair. For example, instead of

60. There are many possible implementations of streams other than the one described in this section. Delayed evaluation, which is the key to making streams practical, was inherent in Algol 60's *call-by-name* parameter-passing method. The use of this mechanism to implement streams was first described by Landin (1965). Delayed evaluation for streams was introduced into Lisp by Friedman and Wise (1976). In their implementation, `cons` (the Lisp equivalent of our `pair` function) always delays evaluating its arguments, so that lists automatically behave as streams. The memoizing optimization is also known as *call-by-need*. The Algol community would refer to our original delayed objects as *call-by-name thunks* and to the optimized versions as *call-by-need thunks*.

```
function stream_map(f, s) {
    return is_null(s)
           ? null
           : pair(f(head(s)),
                  () => stream_map(f, stream_tail(s)));
}
```

we can define an optimized function `stream_map` as follows:

```
function stream_map_optimized(f, s) {
    return is_null(s)
           ? null
           : pair(f(head(s)),
                  memo(() =>
                         stream_map_optimized(f, stream_tail(s))));
}
```

Exercise 3.50

Declare a function `stream_map_2` that takes a binary function and two streams as arguments and returns a stream whose elements are the results of applying the function pairwise to the corresponding elements of the argument streams.

```
function stream_map_2(f, s1, s2) {
    ...
}
```

Similar to `stream_map_optimized`, declare a function `stream_map_2_optimized` by modifying your `stream_map_2` such that the result stream employs memoization.

Exercise 3.51

Note that our primitive function `display` returns its argument after displaying it. What does the interpreter print in response to evaluating each statement in the following sequence?[61]

```
let x = stream_map(display, stream_enumerate_interval(0, 10));
stream_ref(x, 5);
stream_ref(x, 7);
```

What does the interpreter print if `stream_map_optimized` is used instead of `stream_map`?

```
let x = stream_map_optimized(display, stream_enumerate_interval(0, 10));
stream_ref(x, 5);
stream_ref(x, 7);
```

---

61. Exercises such as 3.51 and 3.52 are valuable for testing our understanding of how delayed evaluation works. On the other hand, intermixing delayed evaluation with printing—and, even worse, with assignment—is extremely confusing, and instructors of courses on computer languages have traditionally tormented their students with examination questions such as the ones in this section. Needless to say, writing programs that depend on such subtleties is odious programming style. Part of the power of stream processing is that it lets us ignore the order in which events actually happen in our programs. Unfortunately, this is precisely what we cannot afford to do in the presence of assignment, which forces us to be concerned with time and change.

Exercise 3.52

Consider the sequence of statements

```
let sum = 0;
function accum(x) {
    sum = x + sum;
    return sum;
}
const seq = stream_map(accum, stream_enumerate_interval(1, 20));
const y = stream_filter(is_even, seq);
const z = stream_filter(x => x % 5 === 0, seq);
stream_ref(y, 7);
display_stream(z);
```

What is the value of `sum` after each of the above statements is evaluated? What is the printed response to evaluating the `stream_ref` and `display_stream` expressions? Would these responses differ if we had applied the function `memo` on every tail of every constructed stream pair, as suggested in the optimization above? Explain.

3.5.2 Infinite Streams

We have seen how to support the illusion of manipulating streams as complete entities even though, in actuality, we compute only as much of the stream as we need to access. We can exploit this technique to represent sequences efficiently as streams, even if the sequences are very long. What is more striking, we can use streams to represent sequences that are infinitely long. For instance, consider the following definition of the stream of positive integers:

```
function integers_starting_from(n) {
    return pair(n, () => integers_starting_from(n + 1));
}

const integers = integers_starting_from(1);
```

This makes sense because `integers` will be a pair whose `head` is 1 and whose `tail` is a promise to produce the integers beginning with 2. This is an infinitely long stream, but in any given time we can examine only a finite portion of it. Thus, our programs will never know that the entire infinite stream is not there.

Using `integers` we can define other infinite streams, such as the stream of integers that are not divisible by 7:

```
function is_divisible(x, y) { return x % y === 0; }

const no_sevens = stream_filter(x => ! is_divisible(x, 7),
                                integers);
```

Then we can find integers not divisible by 7 simply by accessing elements of this stream:

```
stream_ref(no_sevens, 100);
```
117

In analogy with `integers`, we can define the infinite stream of Fibonacci numbers:

```
function fibgen(a, b) {
    return pair(a, () => fibgen(b, a + b));
}

const fibs = fibgen(0, 1);
```

The constant `fibs` is a pair whose `head` is 0 and whose `tail` is a promise to evaluate `fibgen(1, 1)`. When we evaluate this delayed `fibgen(1, 1)`, it will produce a pair whose `head` is 1 and whose `tail` is a promise to evaluate `fibgen(1, 2)`, and so on.

For a look at a more exciting infinite stream, we can generalize the `no_sevens` example to construct the infinite stream of prime numbers, using a *sieving* method.[62] We start with the integers beginning with 2, which is the first prime. To get the rest of the primes, we start by filtering the multiples of 2 from the rest of the integers. This leaves a stream beginning with 3, which is the next prime. Now we filter the multiples of 3 from the rest of this stream. This leaves a stream beginning with 5, which is the next prime, and so on. In other words, we construct the primes by a sieving process, described as follows: To sieve a stream S, form a stream whose first element is the first element of S and the rest of which is obtained by filtering all multiples of the first element of S out of the rest of S and sieving the result. This process is readily described in terms of stream operations:

```
function sieve(stream) {
    return pair(head(stream),
                () => sieve(stream_filter(
                                x => ! is_divisible(x, head(stream)),
                                stream_tail(stream))));
}
const primes = sieve(integers_starting_from(2));
```

Now to find a particular prime we need only ask for it:

```
stream_ref(primes, 50);
```
233

---

62. This method is reminiscent of the ancient *sieve of Eratosthenes*, named for Eratosthenes, a third-century BCE Alexandrian Greek mathematician. His sieve finds the primes by repeatedly crossing off the multiples of each prime in turn; although ancient, it has formed the basis for special-purpose hardware "sieves" that, until the 1970s, were the most powerful tools in existence for locating large primes. Since then, however, these methods have been superseded by outgrowths of the probabilistic techniques discussed in section 1.2.6. But the stream-based method given here is not actually Eratosthenes's sieve: as Melissa E. O'Neill shows in O'Neill 2009, it tests each candidate for divisibility by the primes found so far—a form of *trial division*—rather than crossing off multiples, and is substantially less efficient than the sieve of Eratosthenes.

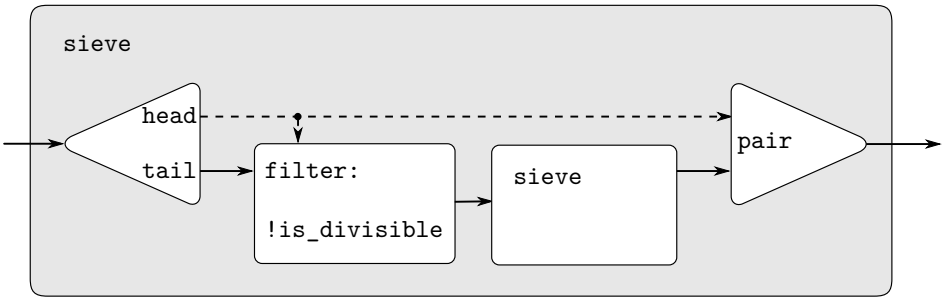


Figure 3.31 The prime sieve viewed as a signal-processing system. Each solid line represents a stream of values being transmitted. The dashed line from the `head` to the `pair` and the `filter` indicates that this is a single value rather than a stream.

It is interesting to contemplate the signal-processing system set up by `sieve`, shown in the "Henderson diagram" in figure 3.31.[63] The input stream feeds into an "unpairer" that separates the first element of the stream from the rest of the stream. The first element is used to construct a divisibility filter, through which the rest is passed, and the output of the filter is fed to another sieve box. Then the original first element is adjoined to the output of the internal sieve to form the output stream. Thus, not only is the stream infinite, but the signal processor is also infinite, because the sieve contains a sieve within it.

Defining streams implicitly

The `integers` and `fibs` streams above were defined by specifying "generating" functions that explicitly compute the stream elements one by one. An alternative way to specify streams is to take advantage of delayed evaluation to define streams implicitly. For example, the following statement defines the stream `ones` to be an infinite stream of ones:

```
const ones = pair(1, () => ones);
```

This works much like the declaration of a recursive function: `ones` is a pair whose `head` is 1 and whose `tail` is a promise to evaluate `ones`. Evaluating the `tail` gives us again a 1 and a promise to evaluate `ones`, and so on.

We can do more interesting things by manipulating streams with operations such as `add_streams`, which produces the elementwise sum of two given streams:[64]

```
function add_streams(s1, s2) {
    return stream_map_2((x1, x2) => x1 + x2, s1, s2);
}
```

Now we can define the integers as follows:

```
const integers = pair(1, () => add_streams(ones, integers));
```

63. We have named these figures after Peter Henderson, who was the first person to show us diagrams of this sort as a way of thinking about stream processing.

64. This uses the function `stream_map_2` from exercise 3.50.

This defines `integers` to be a stream whose first element is 1 and the rest of which is the sum of `ones` and `integers`. Thus, the second element of `integers` is 1 plus the first element of `integers`, or 2; the third element of `integers` is 1 plus the second element of `integers`, or 3; and so on. This definition works because, at any point, enough of the `integers` stream has been generated so that we can feed it back into the definition to produce the next integer.

We can define the Fibonacci numbers in the same style:

```
const fibs = pair(0,
                  () => pair(1,
                             () => add_streams(stream_tail(fibs),
                                               fibs)));
```

This definition says that `fibs` is a stream beginning with 0 and 1, such that the rest of the stream can be generated by adding `fibs` to itself shifted by one place:

		1	1	2	3	5	8	13	21	...	= `stream_tail(fibs)`
		0	1	1	2	3	5	8	13	...	= `fibs`
0	1	1	2	3	5	8	13	21	34	...	= `fibs`

The function `scale_stream` is also useful in formulating such stream definitions. This multiplies each item in a stream by a given constant:

```
function scale_stream(stream, factor) {
    return stream_map(x => x * factor,
                      stream);
}
```

For example,

```
const double = pair(1, () => scale_stream(double, 2));
```

produces the stream of powers of 2: $1, 2, 4, 8, 16, 32, \ldots$.

An alternate definition of the stream of primes can be given by starting with the integers and filtering them by testing for primality. We will need the first prime, 2, to get started:

```
const primes = pair(2,
                    () => stream_filter(is_prime,
                                        integers_starting_from(3)));
```

This definition is not so straightforward as it appears, because we will test whether a number n is prime by checking whether n is divisible by a prime (not by just any integer) less than or equal to $\sqrt{n}$:

```
function is_prime(n) {
    function iter(ps) {
        return square(head(ps)) > n
               ? true
               : is_divisible(n, head(ps))
               ? false
               : iter(stream_tail(ps));
    }
    return iter(primes);
}
```

This is a recursive definition, since `primes` is defined in terms of the `is_prime` predicate, which itself uses the `primes` stream. The reason this function works is that, at any point, enough of the `primes` stream has been generated to test the primality of the numbers we need to check next. That is, for every n we test for primality, either n is not prime (in which case there is a prime already generated that divides it) or n is prime (in which case there is a prime already generated—i.e., a prime less than n—that is greater than $\sqrt{n}$).[65]

Exercise 3.53

Without running the program, describe the elements of the stream defined by

```
const s = pair(1, () => add_streams(s, s));
```

Exercise 3.54

Define a function `mul_streams`, analogous to `add_streams`, that produces the elementwise product of its two input streams. Use this together with the stream of `integers` to complete the following definition of the stream whose nth element (counting from 0) is $n+1$ factorial:

```
const factorials = pair(1, () => mul_streams(⟨??⟩, ⟨??⟩));
```

Exercise 3.55

Define a function `partial_sums` that takes as argument a stream S and returns the stream whose elements are $S_0, S_0+S_1, S_0+S_1+S_2, \ldots$. For example, `partial_sums(integers)` should be the stream $1, 3, 6, 10, 15, \ldots$.

Exercise 3.56

A famous problem, first raised by R. Hamming, is to enumerate, in ascending order with no repetitions, all positive integers with no prime factors other than 2, 3, or 5. One obvious way to do this is to simply test each integer in turn to see whether it has any factors other than 2, 3, and 5. But this is very inefficient, since, as the integers get larger, fewer and fewer of them fit the requirement. As an alternative, let us call the required stream of numbers `S` and notice the following facts about it.

- `S` begins with 1.
- The elements of `scale_stream(S, 2)` are also elements of `S`.
- The same is true for `scale_stream(S, 3)` and `scale_stream(S, 5)`.
- These are all the elements of `S`.

Now all we have to do is combine elements from these sources. For this we define a function `merge` that combines two ordered streams into one ordered result stream, eliminating repetitions:

---

65. This last point is very subtle and relies on the fact that $p_{n+1} \leq p_n^2$. (Here, p_k denotes the kth prime.) Estimates such as these are very difficult to establish. The ancient proof by Euclid that there are an infinite number of primes shows that $p_{n+1} \leq p_1 p_2 \cdots p_n + 1$, and no substantially better result was proved until 1851, when the Russian mathematician P. L. Chebyshev established that $p_{n+1} \leq 2p_n$ for all n. This result, originally conjectured in 1845, is known as *Bertrand's hypothesis*. A proof can be found in section 22.3 of Hardy and Wright 1960.

```
function merge(s1, s2) {
    if (is_null(s1)) {
        return s2;
    } else if (is_null(s2)) {
        return s1;
    } else {
        const s1head = head(s1);
        const s2head = head(s2);
        return s1head < s2head
               ? pair(s1head, () => merge(stream_tail(s1), s2))
               : s1head > s2head
               ? pair(s2head, () => merge(s1, stream_tail(s2)))
               : pair(s1head, () => merge(stream_tail(s1), stream_tail(s2)));
    }
}
```

Then the required stream may be constructed with `merge`, as follows:

```
const S = pair(1, () => merge(⟨??⟩, ⟨??⟩));
```

Fill in the missing expressions in the places marked ⟨??⟩ above.

Exercise 3.57

How many additions are performed when we compute the *n*th Fibonacci number using the declaration of `fibs` based on the `add_streams` function? Show that this number is exponentially greater than the number of additions performed if `add_streams` had used the function `stream_map_2_optimized` described in exercise 3.50, and if we had declared `fibs` as follows:

```
const fibs = pair(0,
                  memo(() => pair(1,
                                  memo(() => add_streams(stream_tail(fibs),
                                                         fibs)))));
```

Exercise 3.58

Give an interpretation of the stream computed by the function

```
function expand(num, den, radix) {
    return pair(math_trunc((num * radix) / den),
                () => expand((num * radix) % den, den, radix));
}
```

where `math_trunc` discards the fractional part of its argument, here the remainder of the division. What are the successive elements produced by `expand(1, 7, 10)`? What is produced by `expand(3, 8, 10)`?

Exercise 3.59

In section 2.5.3 we saw how to implement a polynomial arithmetic system representing polynomials as lists of terms. In a similar way, we can work with *power series*, such as

$$\begin{aligned} e^x &= 1 + x + \frac{x^2}{2} + \frac{x^3}{3\cdot 2} + \frac{x^4}{4\cdot 3\cdot 2} + \cdots, \\ \cos x &= 1 - \frac{x^2}{2} + \frac{x^4}{4\cdot 3\cdot 2} - \cdots, \\ \sin x &= x - \frac{x^3}{3\cdot 2} + \frac{x^5}{5\cdot 4\cdot 3\cdot 2} - \cdots, \end{aligned}$$

represented as infinite streams. We will represent the series $a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \cdots$ as the stream whose elements are the coefficients $a_0, a_1, a_2, a_3, \ldots$.

a. The integral of the series $a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \cdots$ is the series

$$c + a_0 x + \tfrac{1}{2} a_1 x^2 + \tfrac{1}{3} a_2 x^3 + \tfrac{1}{4} a_3 x^4 + \cdots$$

where c is any constant. Define a function `integrate_series` that takes as input a stream $a_0, a_1, a_2, \ldots$ representing a power series and returns the stream $a_0, \frac{1}{2}a_1, \frac{1}{3}a_2, \ldots$ of coefficients of the nonconstant terms of the integral of the series. (Since the result has no constant term, it doesn't represent a power series; when we use `integrate_series`, we will use `pair` to adjoin the appropriate constant to the beginning of the stream.)

b. The function $x \mapsto e^x$ is its own derivative. This implies that e^x and the integral of e^x are the same series, except for the constant term, which is $e^0 = 1$. Accordingly, we can generate the series for e^x as

```
const exp_series = pair(1, () => integrate_series(exp_series));
```

Show how to generate the series for sine and cosine, starting from the facts that the derivative of sine is cosine and the derivative of cosine is the negative of sine:

```
const cosine_series = pair(1, ⟨??⟩);
const sine_series = pair(0, ⟨??⟩);
```

Exercise 3.60

With power series represented as streams of coefficients as in exercise 3.59, adding series is implemented by `add_streams`. Complete the declaration of the following function for multiplying series:

```
function mul_series(s1, s2) {
    return pair(⟨??⟩, () => add_streams(⟨??⟩, ⟨??⟩));
}
```

You can test your function by verifying that $sin^2 x + cos^2 x = 1$, using the series from exercise 3.59.

Exercise 3.61

Let S be a power series (exercise 3.59) whose constant term is 1. Suppose we want to find the power series $1/S$, that is, the series X such that $S \cdot X = 1$. Write $S = 1 + S_R$ where S_R is the part of S after the constant term. Then we can solve for X as follows:

$$\begin{aligned} S \cdot X &= 1 \\ (1 + S_R) \cdot X &= 1 \\ X + S_R \cdot X &= 1 \\ X &= 1 - S_R \cdot X \end{aligned}$$

In other words, X is the power series whose constant term is 1 and whose higher-order terms are given by the negative of S_R times X. Use this idea to write a function `invert_unit_series` that computes $1/S$ for a power series S with constant term 1. You will need to use `mul_series` from exercise 3.60.

Exercise 3.62

Use the results of exercises 3.60 and 3.61 to define a function `div_series` that divides two power series. The function `div_series` should work for any two series, provided that the denominator series begins with a nonzero constant term. (If the denominator has a zero constant term, then `div_series` should signal an error.) Show how to use `div_series` together with the result of exercise 3.59 to generate the power series for tangent.

3.5.3 Exploiting the Stream Paradigm

Streams with delayed evaluation can be a powerful modeling tool, providing many of the benefits of local state and assignment. Moreover, they avoid some of the theoretical tangles that accompany the introduction of assignment into a programming language.

The stream approach can be illuminating because it allows us to build systems with different module boundaries than systems organized around assignment to state variables. For example, we can think of an entire time series (or signal) as a focus of interest, rather than the values of the state variables at individual moments. This makes it convenient to combine and compare components of state from different moments.

Formulating iterations as stream processes

In section 1.2.1, we introduced iterative processes, which proceed by updating state variables. We know now that we can represent state as a "timeless" stream of values rather than as a set of variables to be updated. Let's adopt this perspective in revisiting the square-root function from section 1.1.7. Recall that the idea is to generate a sequence of better and better guesses for the square root of x by applying over and over again the function that improves guesses:

```
function sqrt_improve(guess, x) {
    return average(guess, x / guess);
}
```

In our original `sqrt` function, we made these guesses be the successive values of a state variable. Instead we can generate the infinite stream of guesses, starting with an initial guess of 1:

```
function sqrt_stream(x) {
   return pair(1, () => stream_map(guess => sqrt_improve(guess, x),
                                   sqrt_stream(x)));
}

display_stream(sqrt_stream(2));
1
1.5
1.4166666666666665
1.4142156862745097
1.4142135623746899
...
```

We can generate more and more terms of the stream to get better and better guesses. If we like, we can write a function that keeps generating terms until the answer is good enough. (See exercise 3.64.)

Another iteration that we can treat in the same way is to generate an approximation to π, based upon the alternating series that we saw in section 1.3.1:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots$$

We first generate the stream of summands of the series (the reciprocals of the odd integers, with alternating signs). Then we take the stream of sums of more and more terms (using the `partial_sums` function of exercise 3.55) and scale the result by 4:

```
function pi_summands(n) {
   return pair(1 / n, () => stream_map(x => - x, pi_summands(n + 2)));
}
const pi_stream = scale_stream(partial_sums(pi_summands(1)), 4);

display_stream(pi_stream);
4
2.666666666666667
3.466666666666667
2.8952380952380956
3.3396825396825403
2.9760461760461765
3.2837384837384844
3.017071817071818
...
```

This gives us a stream of better and better approximations to π, although the approximations converge rather slowly. Eight terms of the sequence bound the value of π between 3.284 and 3.017.

So far, our use of the stream of states approach is not much different from updating state variables. But streams give us an opportunity to do some interesting tricks. For example, we can transform a stream with a *sequence accelerator* that converts a sequence of approximations to a new sequence that converges to the same value as the original, only faster.

One such accelerator, due to the eighteenth-century Swiss mathematician Leonhard Euler, works well with sequences that are partial sums of alternating series (series of terms with alternating signs). In Euler's technique, if S_n is the nth term of the original sum sequence, then the accelerated sequence has terms

$$S_{n+1} - \frac{(S_{n+1} - S_n)^2}{S_{n-1} - 2S_n + S_{n+1}}$$

Thus, if the original sequence is represented as a stream of values, the transformed sequence is given by

```
function euler_transform(s) {
    const s0 = stream_ref(s, 0);     // S_{n-1}
    const s1 = stream_ref(s, 1);     // S_n
    const s2 = stream_ref(s, 2);     // S_{n+1}
    return pair(s2 - square(s2 - s1) / (s0 + (-2) * s1 + s2),
                memo(() => euler_transform(stream_tail(s))));
}
```

Note that we make use of the memoization optimization of section 3.5.1, because in the following we will rely on repeated evaluation of the resulting stream.

We can demonstrate Euler acceleration with our sequence of approximations to π:

```
display_stream(euler_transform(pi_stream));
3.166666666666667
3.1333333333333337
3.1452380952380956
3.13968253968254
3.1427128427128435
3.1408813408813416
3.142071817071818
3.1412548236077655
...
```

Even better, we can accelerate the accelerated sequence, and recursively accelerate that, and so on. Namely, we create a stream of streams (a structure we'll call a *tableau*) in which each stream is the transform of the preceding one:

```
function make_tableau(transform, s) {
    return pair(s, () => make_tableau(transform, transform(s)));
}
```

The tableau has the form

$$\begin{array}{cccccc} s_{00} & s_{01} & s_{02} & s_{03} & s_{04} & \ldots \\ & s_{10} & s_{11} & s_{12} & s_{13} & \ldots \\ & & s_{20} & s_{21} & s_{22} & \ldots \\ & & & & \ldots & \end{array}$$

Finally, we form a sequence by taking the first term in each row of the tableau:

```
function accelerated_sequence(transform, s) {
    return stream_map(head, make_tableau(transform, s));
}
```

We can demonstrate this kind of "super-acceleration" of the π sequence:

```
display_stream(accelerated_sequence(euler_transform, pi_stream));
4
3.166666666666667
3.142105263157895
3.141599357319005
3.1415927140337785
3.1415926539752927
3.1415926535911765
3.141592653589778
...
```

The result is impressive. Taking eight terms of the sequence yields the correct value of π to 14 decimal places. If we had used only the original π sequence, we would need to compute on the order of 10^{13} terms (i.e., expanding the series far enough so that the individual terms are less then 10^{-13}) to get that much accuracy!

We could have implemented these acceleration techniques without using streams. But the stream formulation is particularly elegant and convenient because the entire sequence of states is available to us as a data structure that can be manipulated with a uniform set of operations.

Exercise 3.63

Louis Reasoner is not happy with the performance of the stream produced by the `sqrt_stream` function and tries to optimize it using memoization:

```
function sqrt_stream_optimized(x) {
    return pair(1,
                memo(() => stream_map(guess =>
                                      sqrt_improve(guess, x),
                                    sqrt_stream_optimized(x))));
}
```

Alyssa P. Hacker instead proposes

```
function sqrt_stream_optimized_2(x) {
    const guesses = pair(1,
                         memo(() => stream_map(guess =>
                                               sqrt_improve(guess, x),
                                             guesses)));
    return guesses;
}
```

and claims that Louis's version is considerably less efficient than hers, because it performs redundant computation. Explain Alyssa's answer. Would Alyssa's approach without memoization be more efficient than the original `sqrt_stream`?

Exercise 3.64

Write a function `stream_limit` that takes as arguments a stream and a number (the tolerance). It should examine the stream until it finds two successive elements that differ in absolute value by less than the tolerance, and return the second of the two elements. Using this, we could compute square roots up to a given tolerance by

```
function sqrt(x, tolerance) {
    return stream_limit(sqrt_stream(x), tolerance);
}
```

Exercise 3.65

Use the series

$$\ln 2 = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \cdots$$

to compute three sequences of approximations to the natural logarithm of 2, in the same way we did above for π. How rapidly do these sequences converge?

Infinite streams of pairs

In section 2.2.3, we saw how the sequence paradigm handles traditional nested loops as processes defined on sequences of pairs. If we generalize this technique to infinite streams, then we can write programs that are not easily represented as loops, because the "looping" must range over an infinite set.

For example, suppose we want to generalize the `prime_sum_pairs` function of section 2.2.3 to produce the stream of pairs of *all* integers (i, j) with $i \leq j$ such that $i + j$ is prime. If `int_pairs` is the sequence of all pairs of integers (i, j) with $i \leq j$, then our required stream is simply[66]

```
stream_filter(p => is_prime(head(p) + head(tail(p))),
              int_pairs);
```

Our problem, then, is to produce the stream `int_pairs`. More generally, suppose we have two streams $S = (S_i)$ and $T = (T_j)$, and imagine the infinite rectangular array

$$\begin{array}{llll} (S_0, T_0) & (S_0, T_1) & (S_0, T_2) & \ldots \\ (S_1, T_0) & (S_1, T_1) & (S_1, T_2) & \ldots \\ (S_2, T_0) & (S_2, T_1) & (S_2, T_2) & \ldots \\ \ldots & & & \end{array}$$

We wish to generate a stream that contains all the pairs in the array that lie on or above the diagonal, i.e., the pairs

$$\begin{array}{llll} (S_0, T_0) & (S_0, T_1) & (S_0, T_2) & \ldots \\ & (S_1, T_1) & (S_1, T_2) & \ldots \\ & & (S_2, T_2) & \ldots \\ & & & \ldots \end{array}$$

(If we take both S and T to be the stream of integers, then this will be our desired stream `int_pairs`.)

Call the general stream of pairs `pairs(S, T)`, and consider it to be composed of three parts: the pair (S_0, T_0), the rest of the pairs in the first row, and the remaining pairs:[67]

66. As in section 2.2.3, we represent a pair of integers as a list rather than a pair.

67. See exercise 3.68 for some insight into why we chose this decomposition.

(S_0, T_0)	(S_0, T_1)	(S_0, T_2)	...
	(S_1, T_1)	(S_1, T_2)	...
		(S_2, T_2)	...
			...

Observe that the third piece in this decomposition (pairs that are not in the first row) is (recursively) the pairs formed from `stream_tail(S)` and `stream_tail(T)`. Also note that the second piece (the rest of the first row) is

```
stream_map(x => list(head(s), x),
           stream_tail(t));
```

Thus we can form our stream of pairs as follows:

```
function pairs(s, t) {
    return pair(list(head(s), head(t)),
                () => combine-in-some-way(
                          stream_map(x => list(head(s), x),
                                     stream_tail(t)),
                          pairs(stream_tail(s), stream_tail(t))));
}
```

In order to complete the function, we must choose some way to combine the two inner streams. One idea is to use the stream analog of the `append` function from section 2.2.1:

```
function stream_append(s1, s2) {
    return is_null(s1)
           ? s2
           : pair(head(s1),
                  () => stream_append(stream_tail(s1), s2));
}
```

This is unsuitable for infinite streams, however, because it takes all the elements from the first stream before incorporating the second stream. In particular, if we try to generate all pairs of positive integers using

```
pairs(integers, integers);
```

our stream of results will first try to run through all pairs with the first integer equal to 1, and hence will never produce pairs with any other value of the first integer.

To handle infinite streams, we need to devise an order of combination that ensures that every element will eventually be reached if we let our program run long enough. An elegant way to accomplish this is with the following `interleave` function:[68]

68. The precise statement of the required property on the order of combination is as follows: There should be a function f of two arguments such that the pair corresponding to element i of the first stream and element j of the second stream will appear as element number $f(i, j)$ of the output stream. The trick of using `interleave` to accomplish this was shown to us by David Turner, who employed it in the language KRC (Turner 1981).

```
function interleave(s1, s2) {
    return is_null(s1)
           ? s2
           : pair(head(s1),
                  () => interleave(s2, stream_tail(s1)));
}
```

Since `interleave` takes elements alternately from the two streams, every element of the second stream will eventually find its way into the interleaved stream, even if the first stream is infinite.

We can thus generate the required stream of pairs as

```
function pairs(s, t) {
    return pair(list(head(s), head(t)),
                () => interleave(stream_map(x => list(head(s), x),
                                            stream_tail(t)),
                                 pairs(stream_tail(s),
                                       stream_tail(t))));
}
```

Exercise 3.66

Examine the stream `pairs(integers, integers)`. Can you make any general comments about the order in which the pairs are placed into the stream? For example, approximately how many pairs precede the pair (1,100)? the pair (99,100)? the pair (100,100)? (If you can make precise mathematical statements here, all the better. But feel free to give more qualitative answers if you find yourself getting bogged down.)

Exercise 3.67

Modify the `pairs` function so that `pairs(integers, integers)` will produce the stream of *all* pairs of integers (i,j) (without the condition $i \leq j$). Hint: You will need to mix in an additional stream.

Exercise 3.68

Louis Reasoner thinks that building a stream of pairs from three parts is unnecessarily complicated. Instead of separating the pair (S_0, T_0) from the rest of the pairs in the first row, he proposes to work with the whole first row, as follows:

```
function pairs(s, t) {
    return interleave(stream_map(x => list(head(s), x),
                                 t),
                      pair(stream_tail(s), stream_tail(t)));
}
```

Does this work? Consider what happens if we evaluate `pairs(integers, integers)` using Louis's definition of `pairs`.

Exercise 3.69

Write a function `triples` that takes three infinite streams, S, T, and U, and produces the stream of triples (S_i, T_j, U_k) such that $i \leq j \leq k$. Use `triples` to generate the stream of all Pythagorean triples of positive integers, i.e., the triples (i,j,k) such that $i \leq j$ and $i^2 + j^2 = k^2$.

Exercise 3.70

It would be nice to be able to generate streams in which the pairs appear in some useful order, rather than in the order that results from an *ad hoc* interleaving process. We can use a technique similar to the `merge` function of exercise 3.56, if we define a way to say that one pair of integers is "less than" another. One way to do this is to define a "weighting function" $W(i,j)$ and stipulate that (i_1,j_1) is less than (i_2,j_2) if $W(i_1,j_1) < W(i_2,j_2)$. Write a function `merge_weighted` that is like `merge`, except that `merge_weighted` takes an additional argument `weight`, which is a function that computes the weight of a pair, and is used to determine the order in which elements should appear in the resulting merged stream.[69] Using this, generalize `pairs` to a function `weighted_pairs` that takes two streams, together with a function that computes a weighting function, and generates the stream of pairs, ordered according to weight. Use your function to generate

a. the stream of all pairs of positive integers (i,j) with $i \leq j$ ordered according to the sum $i+j$

b. the stream of all pairs of positive integers (i,j) with $i \leq j$, where neither i nor j is divisible by 2, 3, or 5, and the pairs are ordered according to the sum $2i+3j+5ij$.

Exercise 3.71

Numbers that can be expressed as the sum of two cubes in more than one way are sometimes called *Ramanujan numbers*, in honor of the mathematician Srinivasa Ramanujan.[70] Ordered streams of pairs provide an elegant solution to the problem of computing these numbers. To find a number that can be written as the sum of two cubes in two different ways, we need only generate the stream of pairs of integers (i,j) weighted according to the sum i^3+j^3 (see exercise 3.70), then search the stream for two consecutive pairs with the same weight. Write a function to generate the Ramanujan numbers. The first such number is 1,729. What are the next five?

Exercise 3.72

In a similar way to exercise 3.71 generate a stream of all numbers that can be written as the sum of two squares in three different ways (showing how they can be so written).

Streams as signals

We began our discussion of streams by describing them as computational analogs of the "signals" in signal-processing systems. In fact, we can use streams to model

---

69. We will require that the weighting function be such that the weight of a pair increases as we move out along a row or down along a column of the array of pairs.

70. To quote from G. H. Hardy's obituary of Ramanujan (Hardy 1921): "It was Mr. Littlewood (I believe) who remarked that 'every positive integer was one of his friends.' I remember once going to see him when he was lying ill at Putney. I had ridden in taxi-cab No. 1729, and remarked that the number seemed to me a rather dull one, and that I hoped it was not an unfavorable omen. 'No,' he replied, 'it is a very interesting number; it is the smallest number expressible as the sum of two cubes in two different ways.'" The trick of using weighted pairs to generate the Ramanujan numbers was shown to us by Charles Leiserson.

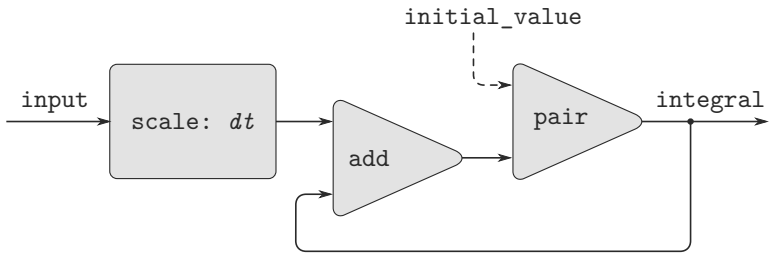


Figure 3.32 The `integral` function viewed as a signal-processing system.

signal-processing systems in a very direct way, representing the values of a signal at successive time intervals as consecutive elements of a stream. For instance, we can implement an *integrator* or *summer* that, for an input stream $x = (x_i)$, an initial value C, and a small increment dt, accumulates the sum

$$S_i \quad = \quad C + \sum_{j=1}^{i} x_j \, dt$$

and returns the stream of values $S = (S_i)$. The following `integral` function is reminiscent of the "implicit style" definition of the stream of integers (section 3.5.2):

```
function integral(integrand, initial_value, dt) {
    const integ = pair(initial_value,
                       () => add_streams(scale_stream(integrand, dt),
                                         integ));
    return integ;
}
```

Figure 3.32 is a picture of a signal-processing system that corresponds to the `integral` function. The input stream is scaled by dt and passed through an adder, whose output is passed back through the same adder. The self-reference in the definition of `integ` is reflected in the figure by the feedback loop that connects the output of the adder to one of the inputs.

Exercise 3.73

We can model electrical circuits using streams to represent the values of currents or voltages at a sequence of times. For instance, suppose we have an *RC circuit* consisting of a resistor of resistance R and a capacitor of capacitance C in series. The voltage response v of the circuit to an injected current i is determined by the formula in figure 3.33, whose structure is shown by the accompanying signal-flow diagram.

Write a function `RC` that models this circuit. `RC` should take as inputs the values of R, C, and dt and should return a function that takes as inputs a stream representing the current i and an initial value for the capacitor voltage v_0 and produces as output the stream of voltages v. For example, you should be able to use `RC` to model an RC circuit with $R = 5$ ohms, $C = 1$ farad, and a 0.5-second time step by evaluating `const RC1 = RC(5, 1, 0.5)`. This defines `RC1` as a function that takes a stream representing the time sequence of currents and an initial capacitor voltage and produces the output stream of voltages.

Exercise 3.74

Alyssa P. Hacker is designing a system to process signals coming from physical sensors. One important feature she wishes to produce is a signal that describes the *zero crossings* of the input signal. That is, the resulting signal should be +1 whenever the input signal changes from negative to positive, –1 whenever the input signal changes from positive to negative, and 0 otherwise. (Assume that the sign of a 0 input is positive.) For example, a typical input signal with its associated zero-crossing signal would be

```
... 1  2  1.5  1  0.5  -0.1  -2  -3  -2  -0.5  0.2  3  4 ...
...  0  0    0  0    0     -1  0   0   0     0    1  0  0 ...
```

In Alyssa's system, the signal from the sensor is represented as a stream `sense_data` and the stream `zero_crossings` is the corresponding stream of zero crossings. Alyssa first writes a function `sign_change_detector` that takes two values as arguments and compares the signs of the values to produce an appropriate 0, 1, or –1. She then constructs her zero-crossing stream as follows:

```
function make_zero_crossings(input_stream, last_value) {
    return pair(sign_change_detector(head(input_stream), last_value),
                () => make_zero_crossings(stream_tail(input_stream),
                                          head(input_stream)));
}
const zero_crossings = make_zero_crossings(sense_data, 0);
```

Alyssa's boss, Eva Lu Ator, walks by and suggests that this program is approximately equivalent to the following one, which uses the function `stream_map_2` from exercise 3.50:

```
const zero_crossings = stream_map_2(sign_change_detector,
                                    sense_data,
                                    expression);
```

Complete the program by supplying the indicated *expression*.

Exercise 3.75

Unfortunately, Alyssa's zero-crossing detector in exercise 3.74 proves to be insufficient, because the noisy signal from the sensor leads to spurious zero crossings. Lem E. Tweakit, a hardware specialist, suggests that Alyssa smooth the signal to filter out the noise before extracting the zero crossings. Alyssa takes his advice and decides to extract the zero crossings from the signal constructed by averaging each value of the sense data with the previous value. She explains the problem to her assistant, Louis Reasoner, who attempts to implement the idea, altering Alyssa's program as follows:

```
function make_zero_crossings(input_stream, last_value) {
    const avpt = (head(input_stream) + last_value) / 2;
    return pair(sign_change_detector(avpt, last_value),
                () => make_zero_crossings(stream_tail(input_stream),
                                          avpt));
}
```

This does not correctly implement Alyssa's plan. Find the bug that Louis has installed and fix it without changing the structure of the program. (Hint: You will need to increase the number of arguments to `make_zero_crossings`.)

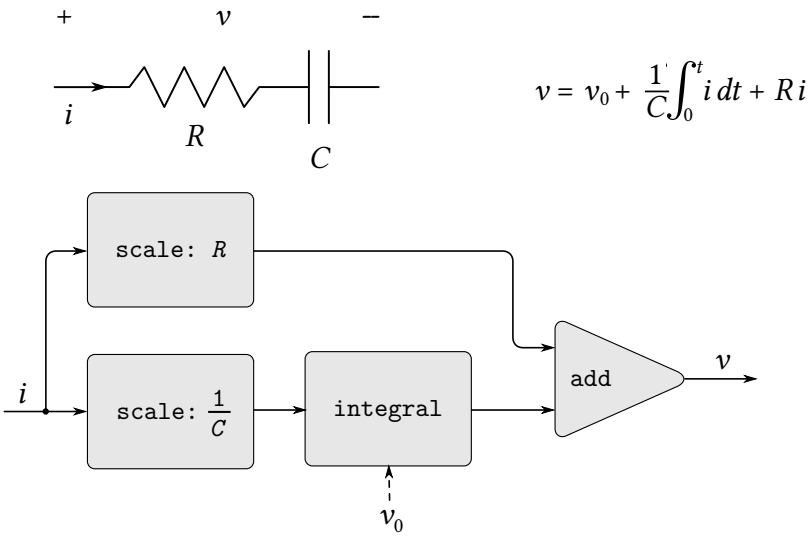


Figure 3.33 An RC circuit and the associated signal-flow diagram.

Exercise 3.76

Eva Lu Ator has a criticism of Louis's approach in exercise 3.75. The program he wrote is not modular, because it intermixes the operation of smoothing with the zero-crossing extraction. For example, the extractor should not have to be changed if Alyssa finds a better way to condition her input signal. Help Louis by writing a function `smooth` that takes a stream as input and produces a stream in which each element is the average of two successive input stream elements. Then use `smooth` as a component to implement the zero-crossing detector in a more modular style.

3.5.4 Streams and Delayed Evaluation

The `integral` function at the end of the preceding section shows how we can use streams to model signal-processing systems that contain feedback loops. The feedback loop for the adder shown in figure 3.32 is modeled by the fact that `integral`'s internal stream `integ` is defined in terms of itself:

```
const integ = pair(initial_value,
                   () => add_streams(scale_stream(integrand, dt),
                                     integ));
```

The interpreter's ability to deal with such an implicit definition depends on the delay resulting from wrapping the call to `add_streams` in a lambda expression. Without this delay, the interpreter could not construct `integ` before evaluating the call to `add_streams`, which would require that `integ` already be defined. In general, such a delay is crucial for using streams to model signal-processing systems that contain loops. Without a delay, our models would have to be formulated so that the inputs to any signal-processing component would be fully evaluated before the output could be produced. This would outlaw loops.

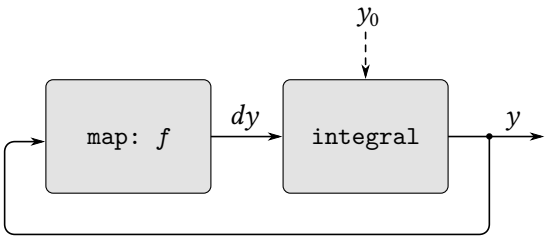


Figure 3.34 An "analog computer circuit" that solves the equation $dy/dt = f(y)$.

Unfortunately, stream models of systems with loops may require uses of a delay beyond the stream programming pattern seen so far. For instance, figure 3.34 shows a signal-processing system for solving the differential equation $dy/dt = f(y)$ where f is a given function. The figure shows a mapping component, which applies f to its input signal, linked in a feedback loop to an integrator in a manner very similar to that of the analog computer circuits that are actually used to solve such equations.

Assuming we are given an initial value y_0 for y, we could try to model this system using the function

```
function solve(f, y0, dt) {
    const y = integral(dy, y0, dt);
    const dy = stream_map(f, y);
    return y;
}
```

This function does not work, because in the first line of `solve` the call to `integral` requires that the input `dy` be defined, which does not happen until the second line of `solve`.

On the other hand, the intent of our definition does make sense, because we can, in principle, begin to generate the `y` stream without knowing `dy`. Indeed, `integral` and many other stream operations can generate part of the answer given only partial information about the arguments. For `integral`, the first element of the output stream is the specified `initial_value`. Thus, we can generate the first element of the output stream without evaluating the integrand `dy`. Once we know the first element of `y`, the `stream_map` in the second line of `solve` can begin working to generate the first element of `dy`, which will produce the next element of `y`, and so on.

To take advantage of this idea, we will redefine `integral` to expect the integrand stream to be a *delayed argument*. The function `integral` will force the integrand to be evaluated only when it is required to generate more than the first element of the output stream:

```
function integral(delayed_integrand, initial_value, dt) {
    const integ =
        pair(initial_value,
             () => {
                 const integrand = delayed_integrand();
                 return add_streams(scale_stream(integrand, dt),
                                    integ);
             });
    return integ;
}
```

Now we can implement our `solve` function by delaying the evaluation of `dy` in the declaration of `y`:

```
function solve(f, y0, dt) {
    const y = integral(() => dy, y0, dt);
    const dy = stream_map(f, y);
    return y;
}
```

In general, every caller of `integral` must now delay the integrand argument. We can demonstrate that the `solve` function works by approximating $e \approx 2.718$ by computing the value at $y = 1$ of the solution to the differential equation $dy/dt = y$ with initial condition $y(0) = 1$:[71]

```
stream_ref(solve(y => y, 1, 0.001), 1000);
```
2.716923932235896

Exercise 3.77

The `integral` function used above was analogous to the "implicit" definition of the infinite stream of integers in section 3.5.2. Alternatively, we can give a definition of `integral` that is more like `integers-starting-from` (also in section 3.5.2):

```
function integral(integrand, initial_value, dt) {
    return pair(initial_value,
                is_null(integrand)
                ? null
                : integral(stream_tail(integrand),
                           dt * head(integrand) + initial_value,
                           dt));
}
```

When used in systems with loops, this function has the same problem as does our original version of `integral`. Modify the function so that it expects the `integrand` as a delayed argument and hence can be used in the `solve` function shown above.

---

71. To complete in reasonable time, this calculation requires the use of the memoization optimization from section 3.5.1 in `integral` and in the function `add_streams` used in `integral` (using the function `stream_map_2_optimized` as suggested in exercise 3.57).

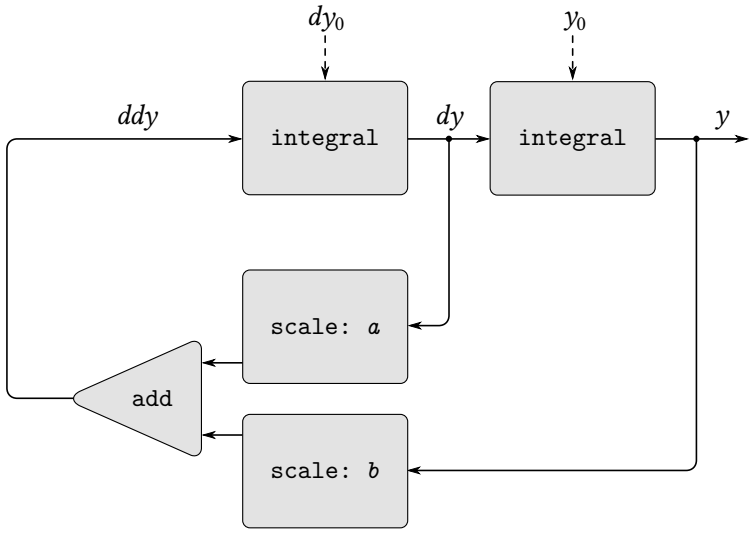


Figure 3.35 Signal-flow diagram for the solution to a second-order linear differential equation.

Exercise 3.78

Consider the problem of designing a signal-processing system to study the homogeneous second-order linear differential equation

$$\frac{d^2y}{dt^2} - a\frac{dy}{dt} - by = 0$$

The output stream, modeling y, is generated by a network that contains a loop. This is because the value of d^2y/dt^2 depends upon the values of y and dy/dt and both of these are determined by integrating d^2y/dt^2. The diagram we would like to encode is shown in figure 3.35. Write a function `solve_2nd` that takes as arguments the constants a, b, and dt and the initial values y_0 and dy_0 for y and dy/dt and generates the stream of successive values of y.

Exercise 3.79

Generalize the `solve_2nd` function of exercise 3.78 so that it can be used to solve general second-order differential equations $d^2y/dt^2 = f(dy/dt, y)$.

Exercise 3.80

A *series RLC circuit* consists of a resistor, a capacitor, and an inductor connected in series, as shown in figure 3.36. If R, L, and C are the resistance, inductance, and capacitance, then the relations between voltage (v) and current (i) for the three components are described by

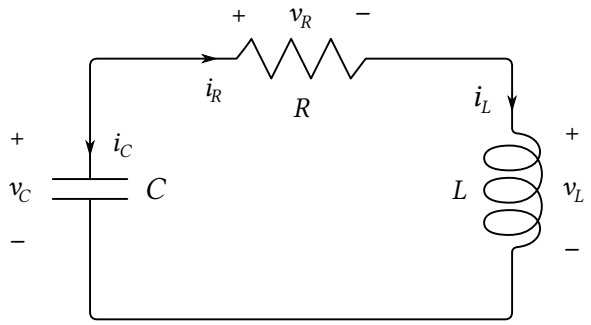


Figure 3.36 A series RLC circuit.

the equations

$$\begin{aligned} v_R &= i_R R \\ v_L &= L\frac{di_L}{dt} \\ i_C &= C\frac{dv_C}{dt} \end{aligned}$$

and the circuit connections dictate the relations

$$\begin{aligned} i_R &= i_L = -i_C \\ v_C &= v_L + v_R \end{aligned}$$

Combining these equations shows that the state of the circuit (summarized by v_C, the voltage across the capacitor, and i_L, the current in the inductor) is described by the pair of differential equations

$$\begin{aligned} \frac{dv_C}{dt} &= -\frac{i_L}{C} \\ \frac{di_L}{dt} &= \frac{1}{L}v_C - \frac{R}{L}i_L \end{aligned}$$

The signal-flow diagram representing this system of differential equations is shown in figure 3.37.

Write a function `RLC` that takes as arguments the parameters R, L, and C of the circuit and the time increment dt. In a manner similar to that of the `RC` function of exercise 3.73, `RLC` should produce a function that takes the initial values of the state variables, v_{C_0} and i_{L_0}, and produces a pair (using `pair`) of the streams of states v_C and i_L. Using `RLC`, generate the pair of streams that models the behavior of a series RLC circuit with $R = 1$ ohm, $C = 0.2$ farad, $L = 1$ henry, $dt = 0.1$ second, and initial values $i_{L_0} = 0$ amps and $v_{C_0} = 10$ volts.

Normal-order evaluation

The examples in this section illustrate how delayed evaluation provides great programming flexibility, but the same examples also show how this can make our programs more complex. Our new `integral` function, for instance, gives us the power to model systems with loops, but we must now remember that `integral` should be called with a delayed integrand, and every function that uses `integral` must be aware of this. In effect, we have created two classes of functions: ordinary functions and functions that take delayed arguments. In general, creating separate classes of functions forces us to create separate classes of higher-order functions as well.[72]

One way to avoid the need for two different classes of functions is to make all functions take delayed arguments. We could adopt a model of evaluation in which all arguments to functions are automatically delayed and arguments are forced only when they are actually needed (for example, when they are required by a primitive operation). This would transform our language to use normal-order evaluation, which we first described when we introduced the substitution model for evaluation in section 1.1.5. Converting to normal-order evaluation provides a uniform and elegant way to simplify the use of delayed evaluation, and this would be a natural strategy to adopt if we were concerned only with stream processing. In section 4.2, after we have studied the evaluator, we will see how to transform our language in just this way. Unfortunately, including delays in function calls wreaks havoc with our ability to design programs that depend on the order of events, such as programs that use assignment, mutate data, or perform input or output. Even a single delay in the tail of a pair can cause great confusion, as illustrated by exercises 3.51 and 3.52. As far as anyone knows, mutability and delayed evaluation do not mix well in programming languages.

---

72. This is a small reflection, in JavaScript, of the difficulties that early statically typed languages such as Pascal had in coping with higher-order functions. In these languages, the programmer had to specify the data types of the arguments and the result of each function: number, logical value, sequence, and so on. Consequently, we could not express an abstraction such as "map a given function `fun` over all the elements in a sequence" by a single higher-order function such as `stream_map`. Rather, we would need a different mapping function for each different combination of argument and result data types that might be specified for a `fun`. Maintaining a practical notion of "data type" in the presence of higher-order functions raises many difficult issues. One way of dealing with this problem is illustrated by the language ML (Gordon, Milner, and Wadsworth 1979), whose "parametrically polymorphic data types" include templates for higher-order transformations between data types. Moreover, data types for most functions in ML are never explicitly declared by the programmer. Instead, ML includes a *type-inferencing* mechanism that uses information in the environment to deduce the data types for newly defined functions. Today, statically typed programming languages have evolved to typically support some form of type inference as well as parametric polymorphism, with varying degrees of power. Haskell couples an expressive type system with powerful type inference.

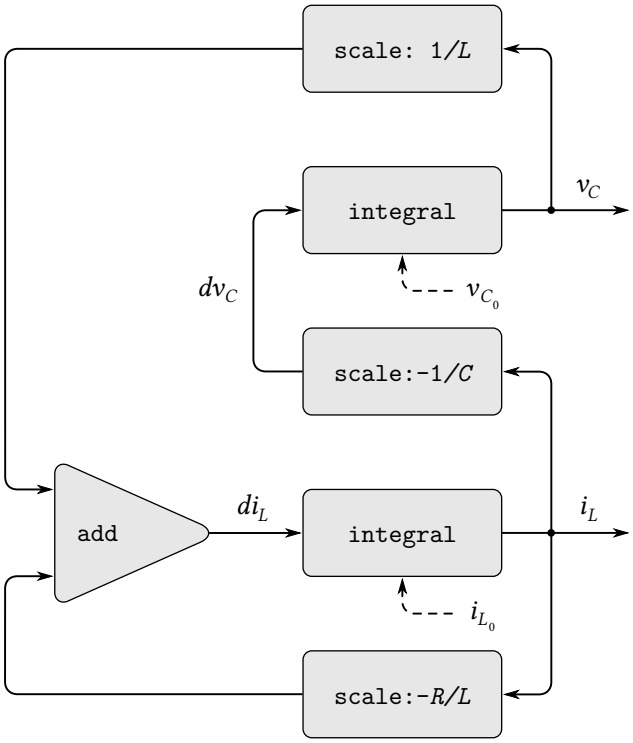


Figure 3.37 A signal-flow diagram for the solution to a series RLC circuit.

3.5.5 Modularity of Functional Programs and Modularity of Objects

As we saw in section 3.1.2, one of the major benefits of introducing assignment is that we can increase the modularity of our systems by encapsulating, or "hiding," parts of the state of a large system within local variables. Stream models can provide an equivalent modularity without the use of assignment. As an illustration, we can reimplement the Monte Carlo estimation of π, which we examined in section 3.1.2, from a stream-processing point of view.

The key modularity issue was that we wished to hide the internal state of a random-number generator from programs that used random numbers. We began with a function `rand_update`, whose successive values furnished our supply of random numbers, and used this to produce a random-number generator:

```
function make_rand() {
    let x = random_init;
    return () => {
                x = rand_update(x);
                return x;
            };
}
const rand = make_rand();
```

In the stream formulation there is no random-number generator *per se*, just a stream of random numbers produced by successive calls to `rand_update`:

```
const random_numbers =
    pair(random_init,
         () => stream_map(rand_update, random_numbers));
```

We use this to construct the stream of outcomes of the Cesàro experiment performed on consecutive pairs in the `random_numbers` stream:

```
function map_successive_pairs(f, s) {
    return pair(f(head(s), head(stream_tail(s))),
                () => map_successive_pairs(
                          f,
                          stream_tail(stream_tail(s))));
}
const dirichlet_stream =
    map_successive_pairs((r1, r2) => gcd(r1, r2) === 1,
                         random_numbers);
```

The `dirichlet_stream` is now fed to a `monte_carlo` function, which produces a stream of estimates of probabilities. The results are then converted into a stream of estimates of π. This version of the program doesn't need a parameter telling how many trials to perform. Better estimates of π (from performing more experiments) are obtained by looking farther into the `pi` stream:

```
function monte_carlo(experiment_stream, passed, failed) {
    function next(passed, failed) {
        return pair(passed / (passed + failed),
                    () => monte_carlo(stream_tail(experiment_stream),
                                      passed, failed));
    }
    return head(experiment_stream)
           ? next(passed + 1, failed)
           : next(passed, failed + 1);
}
const pi = stream_map(p => math_sqrt(6 / p),
                      monte_carlo(dirichlet_stream, 0, 0));
```

There is considerable modularity in this approach, because we still can formulate a general `monte_carlo` function that can deal with arbitrary experiments. Yet there is no assignment or local state.

Exercise 3.81

Exercise 3.6 discussed generalizing the random-number generator to allow one to reset the random-number sequence so as to produce repeatable sequences of "random" numbers. Produce a stream formulation of this same generator that operates on an input stream of requests to `"generate"` a new random number or to `"reset"` the sequence to a specified value and that produces the desired stream of random numbers. Don't use assignment in your solution.

Exercise 3.82

Redo exercise 3.5 on Monte Carlo integration in terms of streams. The stream version of `estimate_integral` will not have an argument telling how many trials to perform. Instead, it will produce a stream of estimates based on successively more trials.

A functional-programming view of time

Let us now return to the issues of objects and state that were raised at the beginning of this chapter and examine them in a new light. We introduced assignment and mutable objects to provide a mechanism for modular construction of programs that model systems with state. We constructed computational objects with local state variables and used assignment to modify these variables. We modeled the temporal behavior of the objects in the world by the temporal behavior of the corresponding computational objects.

Now we have seen that streams provide an alternative way to model objects with local state. We can model a changing quantity, such as the local state of some object, using a stream that represents the time history of successive states. In essence, we represent time explicitly, using streams, so that we decouple time in our simulated world from the sequence of events that take place during evaluation. Indeed, because of the presence of delayed evaluation there may be little relation between simulated time in the model and the order of events during the evaluation.

In order to contrast these two approaches to modeling, let us reconsider the implementation of a "withdrawal processor" that monitors the balance in a bank account. In section 3.1.3 we implemented a simplified version of such a processor:

```
function make_simplified_withdraw(balance) {
    return amount => {
                balance = balance - amount;
                return balance;
            };
}
```

Calls to `make_simplified_withdraw` produce computational objects, each with a local state variable `balance` that is decremented by successive calls to the object. The object takes an `amount` as an argument and returns the new balance. We can imagine the user of a bank account typing a sequence of inputs to such an object and observing the sequence of returned values shown on a display screen.

Alternatively, we can model a withdrawal processor as a function that takes as input a balance and a stream of amounts to withdraw and produces the stream of successive balances in the account:

```
function stream_withdraw(balance, amount_stream) {
    return pair(balance,
                () => stream_withdraw(balance - head(amount_stream),
                                      stream_tail(amount_stream)));
}
```

The function `stream_withdraw` implements a well-defined mathematical function whose output is fully determined by its input. Suppose, however, that the input `amount_stream` is the stream of successive values typed by the user and that the resulting stream of balances is displayed. Then, from the perspective of the user who is typing values and watching results, the stream process has the same behavior as the object created by `make_simplified_withdraw`. However, with the stream version, there is no assignment, no local state variable, and consequently none of the theoretical difficulties that we encountered in section 3.1.3. Yet the system has state!

This is really remarkable. Even though `stream_withdraw` implements a well-defined mathematical function whose behavior does not change, the user's perception here is one of interacting with a system that has a changing state. One way to resolve this paradox is to realize that it is the user's temporal existence that imposes state on the system. If the user could step back from the interaction and think in terms of streams of balances rather than individual transactions, the system would appear stateless.[73]

From the point of view of one part of a complex process, the other parts appear to change with time. They have hidden time-varying local state. If we wish to write programs that model this kind of natural decomposition in our world (as we see it from our viewpoint as a part of that world) with structures in our computer, we make computational objects that are not functional—they must change with time. We model state with local state variables, and we model the changes of state with assignments to those variables. By doing this we make the time of execution of a computation model time in the world that we are part of, and thus we get "objects" in our computer.

Modeling with objects is powerful and intuitive, largely because this matches the perception of interacting with a world of which we are part. However, as we've seen repeatedly throughout this chapter, these models raise thorny problems of constraining the order of events and of synchronizing multiple processes. The possibility of avoiding these problems has stimulated the development of *functional programming languages*, which do not include any provision for assignment or mutable data. In such a language, all functions implement well-defined mathematical functions of their arguments, whose behavior does not change. The functional approach is extremely attractive for dealing with concurrent systems.[74]

On the other hand, if we look closely, we can see time-related problems creeping into functional models as well. One particularly troublesome area arises when we wish to design interactive systems, especially ones that model interactions between independent entities. For instance, consider once more the implementation of a banking system that permits joint bank accounts. In a conventional system using assignment and objects, we would model the fact that Peter and Paul share an account by having both Peter and Paul send their transaction requests to the same bank-account object, as we saw in section 3.1.3. From the stream point of view, where there are no "objects" *per se*, we have already indicated that a bank account can be modeled as a process that operates on a stream of transaction requests to produce a stream of responses. Accordingly, we could model the fact that Peter and Paul have a joint bank account by merging Peter's stream of transaction requests with

---

73. Similarly in physics, when we observe a moving particle, we say that the position (state) of the particle is changing. However, from the perspective of the particle's world line in space-time there is no change involved.

74. John Backus, the inventor of Fortran, gave high visibility to functional programming when he was awarded the ACM Turing award in 1978. His acceptance speech (Backus 1978) strongly advocated the functional approach. A good overview of functional programming is given in Henderson 1980 and in Darlington, Henderson, and Turner 1982.

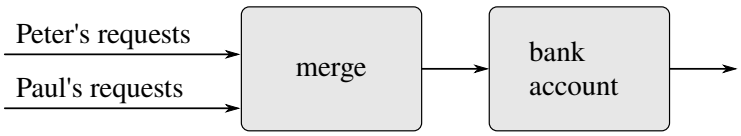


Figure 3.38 A joint bank account, modeled by merging two streams of transaction requests.

Paul's stream of requests and feeding the result to the bank-account stream process, as shown in figure 3.38.

The trouble with this formulation is in the notion of *merge*. It will not do to merge the two streams by simply taking alternately one request from Peter and one request from Paul. Suppose Paul accesses the account only very rarely. We could hardly force Peter to wait for Paul to access the account before he could issue a second transaction. However such a merge is implemented, it must interleave the two transaction streams in some way that is constrained by "real time" as perceived by Peter and Paul, in the sense that, if Peter and Paul meet, they can agree that certain transactions were processed before the meeting, and other transactions were processed after the meeting.[75] This is precisely the same constraint that we had to deal with in section 3.4.1, where we found the need to introduce explicit synchronization to ensure a "correct" order of events in concurrent processing of objects with state. Thus, in an attempt to support the functional style, the need to merge inputs from different agents reintroduces the same problems that the functional style was meant to eliminate.

We began this chapter with the goal of building computational models whose structure matches our perception of the real world we are trying to model. We can model the world as a collection of separate, time-bound, interacting objects with state, or we can model the world as a single, timeless, stateless unity. Each view has powerful advantages, but neither view alone is completely satisfactory. A grand unification has yet to emerge.[76]

---

75. Observe that, for any two streams, there is in general more than one acceptable order of interleaving. Thus, technically, "merge" is a relation rather than a function—the answer is not a deterministic function of the inputs. We already mentioned (footnote 44) that nondeterminism is essential when dealing with concurrency. The merge relation illustrates the same essential nondeterminism, from the functional perspective. In section 4.3, we will look at nondeterminism from yet another point of view.

76. The object model approximates the world by dividing it into separate pieces. The functional model does not modularize along object boundaries. The object model is useful when the unshared state of the "objects" is much larger than the state that they share. An example of a place where the object viewpoint fails is quantum mechanics, where thinking of things as individual particles leads to paradoxes and confusions. Unifying the object view with the functional view may have little to do with programming, but rather with fundamental epistemological issues.

4 Metalinguistic Abstraction

> . . . It's in words that the magic is—Abracadabra, Open Sesame, and the rest—but the magic words in one story aren't magical in the next. The real magic is to understand which words work, and when, and for what; the trick is to learn the trick.
> . . . And those words are made from the letters of our alphabet: a couple-dozen squiggles we can draw with the pen. This is the key! And the treasure, too, if we can only get our hands on it! It's as if—as if the key to the treasure *is* the treasure!
>
> —John Barth, *Chimera*

In our study of program design, we have seen that expert programmers control the complexity of their designs with the same general techniques used by designers of all complex systems. They combine primitive elements to form compound objects, they abstract compound objects to form higher-level building blocks, and they preserve modularity by adopting appropriate large-scale views of system structure. In illustrating these techniques, we have used JavaScript as a language for describing processes and for constructing computational data objects and processes to model complex phenomena in the real world. However, as we confront increasingly complex problems, we will find that JavaScript, or indeed any fixed programming language, is not sufficient for our needs. We must constantly turn to new languages in order to express our ideas more effectively. Establishing new languages is a powerful strategy for controlling complexity in engineering design; we can often enhance our ability to deal with a complex problem by adopting a new language that enables us to describe (and hence to think about) the problem in a different way, using primitives, means of combination, and means of abstraction that are particularly well suited to the problem at hand.[1]

---

1. The same idea is pervasive throughout all of engineering. For example, electrical engineers use many different languages for describing circuits. Two of these are the language of electrical *networks* and the language of electrical *systems*. The network language emphasizes the physical modeling of devices in terms of discrete electrical elements. The primitive objects of the network language are primitive electrical components such as resistors, capacitors, inductors, and transistors, which are characterized in terms of physical variables called voltage and current. When describing circuits in the network language, the engineer is concerned with the physical characteristics of a design. In contrast, the primitive objects of the system language are signal-processing modules such as filters and amplifiers. Only the functional behavior of the modules is relevant, and signals are manipulated without concern for their physical realization as voltages and currents. The system language is erected on the network language, in the sense that the elements of signal-processing systems are constructed from electrical networks. Here, however, the concerns are with the large-scale organization of electrical devices to solve a given application problem; the physical feasibility of the parts is assumed. This layered collection of languages is another example of the stratified design technique illustrated by the picture language of section 2.2.4.

Programming is endowed with a multitude of languages. There are physical languages, such as the machine languages for particular computers. These languages are concerned with the representation of data and control in terms of individual bits of storage and primitive machine instructions. The machine-language programmer is concerned with using the given hardware to erect systems and utilities for the efficient implementation of resource-limited computations. High-level languages, erected on a machine-language substrate, hide concerns about the representation of data as collections of bits and the representation of programs as sequences of primitive instructions. These languages have means of combination and abstraction, such as function declaration, that are appropriate to the larger-scale organization of systems.

Metalinguistic abstraction—establishing new languages—plays an important role in all branches of engineering design. It is particularly important to computer programming, because in programming not only can we formulate new languages but we can also implement these languages by constructing evaluators. An *evaluator* (or *interpreter*) for a programming language is a function that, when applied to a statement or expression of the language, performs the actions required to evaluate that statement or expression. It is no exaggeration to regard this as the most fundamental idea in programming:

> The evaluator, which determines the meaning of statements and expressions in a programming language, is just another program.

To appreciate this point is to change our images of ourselves as programmers. We come to see ourselves as designers of languages, rather than only users of languages designed by others.

In fact, we can regard almost any program as the evaluator for some language. For instance, the polynomial manipulation system of section 2.5.3 embodies the rules of polynomial arithmetic and implements them in terms of operations on list-structured data. If we augment this system with functions to read and print polynomial expressions, we have the core of a special-purpose language for dealing with problems in symbolic mathematics. The digital-logic simulator of section 3.3.4 and the constraint propagator of section 3.3.5 are legitimate languages in their own right, each with its own primitives, means of combination, and means of abstraction. Seen from this perspective, the technology for coping with large-scale computer systems merges with the technology for building new computer languages, and computer science itself becomes no more (and no less) than the discipline of constructing appropriate descriptive languages.

We now embark on a tour of the technology by which languages are established in terms of other languages. In this chapter we shall use JavaScript as a base, implementing evaluators as JavaScript functions. We will take the first step in understanding how languages are implemented by building an evaluator for JavaScript itself. The language implemented by our evaluator will be a subset of JavaScript. Although the evaluator described in this chapter is written for a particular subset of JavaScript, it contains the essential structure of an evaluator for any language designed for writing

programs for a sequential machine. (In fact, most language processors contain, deep within them, a little evaluator.) The evaluator has been simplified for the purposes of illustration and discussion, and some features have been left out that would be important to include in a production-quality JavaScript system. Nevertheless, this simple evaluator is adequate to execute most of the programs in this book.[2]

An important advantage of making the evaluator accessible as a JavaScript program is that we can implement alternative evaluation rules by describing these as modifications to the evaluator program. One place where we can use this power to good effect is to gain extra control over the ways in which computational models embody the notion of time, which was so central to the discussion in chapter 3. There, we mitigated some of the complexities of state and assignment by using streams to decouple the representation of time in the world from time in the computer. Our stream programs, however, were sometimes cumbersome, because they were constrained by the applicative-order evaluation of JavaScript. In section 4.2, we'll change the underlying language to provide for a more elegant approach, by modifying the evaluator to provide for *normal-order evaluation*.

Section 4.3 implements a more ambitious linguistic change, whereby statements and expressions have many values, rather than just a single value. In this language of *nondeterministic computing*, it is natural to express processes that generate all possible values for statements and expressions and then search for those values that satisfy certain constraints. In terms of models of computation and time, this is like having time branch into a set of "possible futures" and then searching for appropriate time lines. With our nondeterministic evaluator, keeping track of multiple values and performing searches are handled automatically by the underlying mechanism of the language.

In section 4.4 we implement a *logic-programming* language in which knowledge is expressed in terms of relations, rather than in terms of computations with inputs and outputs. Even though this makes the language drastically different from JavaScript, or indeed from any conventional language, we will see that the logic-programming evaluator shares the essential structure of the JavaScript evaluator.

4.1 The Metacircular Evaluator

Our evaluator for JavaScript will be implemented as a JavaScript program. It may seem circular to think about evaluating JavaScript programs using an evaluator that is itself implemented in JavaScript. However, evaluation is a process, so it is appropriate to describe the evaluation process using JavaScript, which, after all, is our tool for

2. The most important features that our evaluator leaves out are mechanisms for handling errors and supporting debugging. For a more extensive discussion of evaluators, see Friedman, Wand, and Haynes 1992, which gives an exposition of programming languages that proceeds via a sequence of evaluators written in the Scheme dialect of Lisp.

describing processes.[3] An evaluator that is written in the same language that it evaluates is said to be *metacircular*.

The metacircular evaluator is essentially a JavaScript formulation of the environment model of evaluation described in section 3.2. Recall that the model specifies the evaluation of function application in two basic steps:

1. To evaluate a function application, evaluate the subexpressions and then apply the value of the function subexpression to the values of the argument subexpressions.
2. To apply a compound function to a set of arguments, evaluate the body of the function in a new environment. To construct this environment, extend the environment part of the function object by a frame in which the parameters of the function are bound to the arguments to which the function is applied.

These two rules describe the essence of the evaluation process, a basic cycle in which statements and expressions to be evaluated in environments are reduced to functions to be applied to arguments, which in turn are reduced to new statements and expressions to be evaluated in new environments, and so on, until we get down to names, whose values are looked up in the environment, and to operators and primitive functions, which are applied directly (see figure 4.1).[4] This evaluation cycle will be embodied by the interplay between the two critical functions in the evaluator, `evaluate` and `apply`, which are described in section 4.1.1 (see figure 4.1).

The implementation of the evaluator will depend upon functions that define the *syntax* of the statements and expressions to be evaluated. We will use data abstraction

---

3. Even so, there will remain important aspects of the evaluation process that are not elucidated by our evaluator. The most important of these are the detailed mechanisms by which functions call other functions and return values to their callers. We will address these issues in chapter 5, where we take a closer look at the evaluation process by implementing the evaluator as a simple register machine.

4. If we grant ourselves the ability to apply primitives, then what remains for us to implement in the evaluator? The job of the evaluator is not to specify the primitives of the language, but rather to provide the connective tissue—the means of combination and the means of abstraction—that binds a collection of primitives to form a language. Specifically:

- The evaluator enables us to deal with nested expressions. For example, although simply applying primitives would suffice for evaluating the expression `2 * 6`, it is not adequate for handling `2 * (1 + 5)`. As far as the operator `*` is concerned, its arguments must be numbers, and it would choke if we passed it the expression `1 + 5` as an argument. One important role of the evaluator is to choreograph composition so that `1 + 5` is reduced to 6 before being passed as an argument to `*`.
- The evaluator allows us to use names. For example, the addition operator has no way to deal with expressions such as `x + 1`. We need an evaluator to keep track of names and obtain their values before invoking the operators.
- The evaluator allows us to define compound functions. This involves knowing how to use these functions in evaluating expressions and providing a mechanism that enables functions to accept arguments.
- The evaluator provides the other syntactic forms of the language such as conditionals and blocks.

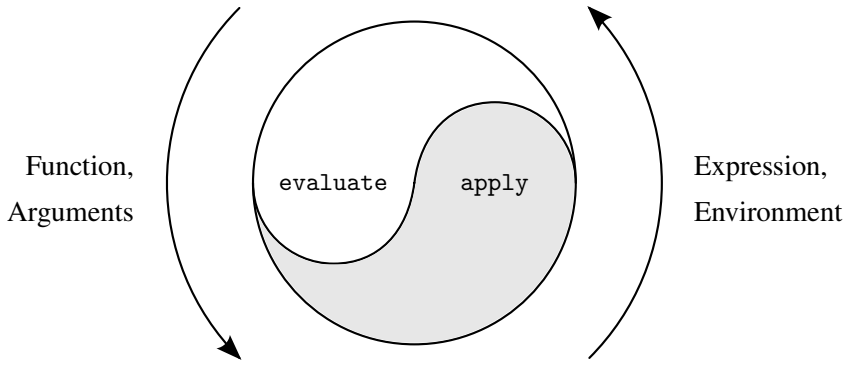


Figure 4.1 The `evaluate`–`apply` cycle exposes the essence of a computer language.

to make the evaluator independent of the representation of the language. For example, rather than committing to a choice that an assignment is to be represented by a string beginning with a name followed by `=`, we use an abstract predicate `is_assignment` to test for an assignment, and we use abstract selectors `assignment_symbol` and `assignment_value_expression` to access the parts of an assignment. The data abstraction layers presented in section 4.1.2 will allow the evaluator to remain independent of concrete syntactic issues, such as the keywords of the interpreted language, and of the choice of data structures that represent the program components. There are also operations, described in section 4.1.3, that specify the representation of functions and environments. For example, `make_function` constructs compound functions, `lookup_symbol_value` accesses the values of names, and `apply_primitive_function` applies a primitive function to a given list of arguments.

4.1.1 The Core of the Evaluator

The evaluation process can be described as the interplay between two functions: `evaluate` and `apply`.

The function `evaluate`

The function `evaluate` takes as arguments a program *component*—a statement or expression[5]—and an environment. It classifies the component and directs its evaluation. The function `evaluate` is structured as a case analysis of the syntactic type of the component to be evaluated. In order to keep the function general, we express the determination of the type of a component abstractly, making no commitment

5. There is no need to distinguish between statements and expressions in our evaluator. For example, we do not differentiate between expressions and expression statements; we represent them identically and consequently they are handled in the same way by the `evaluate` function. Similarly, our evaluator does not enforce JavaScript's syntactic restriction that statements cannot appear inside expressions other than lambda expressions.

to any particular representation for the various types of components. Each type of component has a *syntax predicate* that tests for it and an abstract means for selecting its parts. This *abstract syntax* makes it easy to see how we can change the syntax of the language by using the same evaluator, but with a different collection of syntax functions.

Primitive expressions

- For literal expressions, such as numbers, `evaluate` returns their value.
- The function `evaluate` must look up names in the environment to find their values.

Combinations

- For a function application, `evaluate` must recursively evaluate the function expression and the argument expressions of the application. The resulting function and arguments are passed to `apply`, which handles the actual function application.
- An operator combination is transformed into a function application and then evaluated.

Syntactic forms

- A conditional expression or statement requires special processing of its parts, so as to evaluate the consequent if the predicate is true, and otherwise to evaluate the alternative.
- A lambda expression must be transformed into an applicable function by packaging together the parameters and body specified by the lambda expression with the environment of the evaluation.
- A sequence of statements requires evaluating its components in the order in which they appear.
- A block requires evaluating its body in a new environment that reflects all names declared within the block.
- A return statement must produce a value that becomes the result of the function call that gave rise to the evaluation of the return statement.
- A function declaration is transformed into a constant declaration and then evaluated.
- A constant or variable declaration or an assignment must call `evaluate` recursively to compute the new value to be associated with the name being declared or assigned. The environment must be modified to reflect the new value of the name.

Here is the declaration of `evaluate`:

```
function evaluate(component, env) {
    return is_literal(component)
           ? literal_value(component)
           : is_name(component)
           ? lookup_symbol_value(symbol_of_name(component), env)
           : is_application(component)
           ? apply(evaluate(function_expression(component), env),
                   list_of_values(arg_expressions(component), env))
           : is_operator_combination(component)
           ? evaluate(operator_combination_to_application(component),
                      env)
           : is_conditional(component)
           ? eval_conditional(component, env)
           : is_lambda_expression(component)
           ? make_function(lambda_parameter_symbols(component),
                           lambda_body(component), env)
           : is_sequence(component)
           ? eval_sequence(sequence_statements(component), env)
           : is_block(component)
           ? eval_block(component, env)
           : is_return_statement(component)
           ? eval_return_statement(component, env)
           : is_function_declaration(component)
           ? evaluate(function_decl_to_constant_decl(component), env)
           : is_declaration(component)
           ? eval_declaration(component, env)
           : is_assignment(component)
           ? eval_assignment(component, env)
           : error(component, "unknown syntax -- evaluate");
}
```

For clarity, `evaluate` has been implemented as a case analysis using conditional expressions. The disadvantage of this is that our function handles only a few distinguishable types of statements and expressions, and no new ones can be defined without editing the declaration of `evaluate`. In most interpreter implementations, dispatching on the type of a component is done in a data-directed style. This allows a user to add new types of components that `evaluate` can distinguish, without modifying the declaration of `evaluate` itself. (See exercise 4.3.)

The representation of names is handled by the syntax abstractions. Internally, the evaluator uses strings to represent names, and we refer to such strings as *symbols*. The function `symbol_of_name` used in `evaluate` extracts from a name the symbol by which it is represented.

Apply

The function `apply` takes two arguments, a function and a list of arguments to which the function should be applied. The function `apply` classifies functions into two kinds: It calls `apply_primitive_function` to apply primitives; it applies compound functions by evaluating the block that makes up the body of the function. The environment for the evaluation of the body of a compound function is constructed by extending the base environment carried by the function to include a frame that binds the parameters of the function to the arguments to which the function is to be applied. Here is the declaration of `apply`:

```
function apply(fun, args) {
    if (is_primitive_function(fun)) {
        return apply_primitive_function(fun, args);
    } else if (is_compound_function(fun)) {
        const result = evaluate(function_body(fun),
                                extend_environment(
                                    function_parameters(fun),
                                    args,
                                    function_environment(fun)));
        return is_return_value(result)
               ? return_value_content(result)
               : undefined;
    } else {
        error(fun, "unknown function type -- apply");
    }
}
```

In order to return a value, a JavaScript function needs to evaluate a return statement. If a function terminates without evaluating a return statement, the value `undefined` is returned. To distinguish the two cases, the evaluation of a return statement will wrap the result of evaluating its return expression into a *return value*. If the evaluation of the function body yields such a return value, the content of the return value is retrieved; otherwise the value `undefined` is returned.[6]

Function arguments

When `evaluate` processes a function application, it uses `list_of_values` to produce the list of arguments to which the function is to be applied. The function

6. This test is a deferred operation, and thus our evaluator will give rise to a recursive process even if the interpreted program should give rise to an iterative process according to the description in section 1.2.1. In other words, our metacircular evaluator implementation of JavaScript is not tail-recursive. Sections 5.4.2 and 5.5.3 show how to achieve tail recursion using a register machine.

`list_of_values` takes as an argument the argument expressions of the application. It evaluates each argument expression and returns a list of the corresponding values:[7]

```
function list_of_values(exps, env) {
    return map(arg => evaluate(arg, env), exps);
}
```

Conditionals

The function `eval_conditional` evaluates the predicate part of a conditional component in the given environment. If the result is true, the consequent is evaluated, otherwise the alternative is evaluated:

```
function eval_conditional(component, env) {
    return is_truthy(evaluate(conditional_predicate(component), env))
           ? evaluate(conditional_consequent(component), env)
           : evaluate(conditional_alternative(component), env);
}
```

Note that the evaluator does not need to distinguish between conditional expressions and conditional statements.

The use of `is_truthy` in `eval_conditional` highlights the issue of the connection between an implemented language and an implementation language. The `conditional_predicate` is evaluated in the language being implemented and thus yields a value in that language. The interpreter predicate `is_truthy` translates that value into a value that can be tested by the conditional expression in the implementation language: The metacircular representation of truth might not be the same as that of the underlying JavaScript.[8]

Sequences

The function `eval_sequence` is used by `evaluate` to evaluate a sequence of statements at the top level or in a block. It takes as arguments a sequence of statements

---

7. We chose to implement `list_of_values` using the higher-order function `map`, and we will use `map` in other places as well. However, the evaluator can be implemented without any use of higher-order functions (and thus could be written in a language that doesn't have higher-order functions), even though the language that it supports will include higher-order functions. For example, `list_of_values` can be written without `map` as follows:

```
function list_of_values(exps, env) {
    return is_null(exps)
           ? null
           : pair(evaluate(head(exps), env),
                  list_of_values(tail(exps), env));
}
```

8. In this case, the language being implemented and the implementation language are the same. Contemplation of the meaning of `is_truthy` here yields expansion of consciousness without the abuse of substance.

and an environment, and evaluates the statements in the order in which they occur. The value returned is the value of the final statement, except that if the evaluation of any statement in the sequence yields a return value, that value is returned and the subsequent statements are ignored.[9]

```
function eval_sequence(stmts, env) {
    if (is_empty_sequence(stmts)) {
        return undefined;
    } else if (is_last_statement(stmts)) {
        return evaluate(first_statement(stmts), env);
    } else {
        const first_stmt_value =
            evaluate(first_statement(stmts), env);
        if (is_return_value(first_stmt_value)) {
            return first_stmt_value;
        } else {
            return eval_sequence(rest_statements(stmts), env);
        }
    }
}
```

Blocks

The function `eval_block` handles blocks. The variables and constants (including functions) declared in the block have the whole block as their scope and thus are "scanned out" before the body of the block is evaluated. The body of the block is evaluated with respect to an environment that extends the current environment by a frame that binds each local name to a special value, `"*unassigned*"`. This string serves as a placeholder, before the evaluation of the declaration assigns the name its proper value. An attempt to access the value of the name before its declaration is evaluated leads to an error at run time (see exercise 4.12), as stated in footnote 57 in chapter 1.

```
function eval_block(component, env) {
    const body = block_body(component);
    const locals = scan_out_declarations(body);
    const unassigneds = list_of_unassigned(locals);
    return evaluate(body, extend_environment(locals,
                                             unassigneds,
                                             env));
}
function list_of_unassigned(symbols) {
    return map(symbol => "*unassigned*", symbols);
}
```

The function `scan_out_declarations` collects a list of all symbols representing names declared in the body. It uses `declaration_symbol` to retrieve the symbol that represents the name from the declaration statements it finds.

---

9. The treatment of return statements in `eval_sequence` reflects the proper result of evaluating function applications in JavaScript, but the evaluator presented here does not comply with the ECMAScript specification for the value of a program that consists of a sequence of statements *outside of* any function body. Exercise 4.8 addresses this issue.

```
function scan_out_declarations(component) {
    return is_sequence(component)
           ? accumulate(append,
                        null,
                        map(scan_out_declarations,
                            sequence_statements(component)))
           : is_declaration(component)
           ? list(declaration_symbol(component))
           : null;
}
```

We ignore declarations that are nested in another block, because the evaluation of that block will take care of them. The function `scan_out_declarations` looks for declarations only in sequences because declarations in conditional statements, function declarations, and lambda expressions are always in a nested block.

Return statements

The function `eval_return_statement` is used to evaluate return statements. As seen in `apply` and the evaluation of sequences, the result of evaluation of a return statement needs to be identifiable so that the evaluation of a function body can return immediately, even if there are statements after the return statement. For this purpose, the evaluation of a return statement wraps the result of evaluating the return expression in a return value object.[10]

```
function eval_return_statement(component, env) {
    return make_return_value(evaluate(return_expression(component),
                                      env));
}
```

Assignments and declarations

The function `eval_assignment` handles assignments to names. (To simplify the presentation of our evaluator, we are allowing assignment not just to variables but also—erroneously—to constants. Exercise 4.11 explains how we could distinguish constants from variables and prevent assignment to constants.) The function `eval_assignment` calls `evaluate` on the value expression to find the value to be assigned and calls `assignment_symbol` to retrieve the symbol that represents the name from the assignment. The function `eval_assignment` transmits the symbol and the value to `assign_symbol_value` to be installed in the designated environment. The evaluation of an assignment returns the value that was assigned.

```
function eval_assignment(component, env) {
    const value = evaluate(assignment_value_expression(component),
                           env);
    assign_symbol_value(assignment_symbol(component), value, env);
    return value;
}
```

10. The application of the function `make_return_value` to the result of evaluating the return expression creates a deferred operation, in addition to the deferred operation created by `apply`. See footnote 6 for details.

Constant and variable declarations are both recognized by the `is_declaration` syntax predicate. They are treated in a manner similar to assignments, because `eval_block` has already bound their symbols to `"*unassigned*"` in the current environment. Their evaluation replaces `"*unassigned*"` with the result of evaluating the value expression.

```
function eval_declaration(component, env) {
    assign_symbol_value(
        declaration_symbol(component),
        evaluate(declaration_value_expression(component), env),
        env);
    return undefined;
}
```

The result of evaluating the body of a function is determined by return statements, and therefore the return value `undefined` in `eval_declaration` only matters when the declaration occurs at the top level, outside of any function body. Here we use the return value `undefined` to simplify the presentation; exercise 4.8 describes the real result of evaluating top-level components in JavaScript.

Exercise 4.1

Notice that we cannot tell whether the metacircular evaluator evaluates argument expressions from left to right or from right to left. Its evaluation order is inherited from the underlying JavaScript: If the arguments to `pair` in `map` are evaluated from left to right, then `list_of_values` will evaluate argument expressions from left to right; and if the arguments to `pair` are evaluated from right to left, then `list_of_values` will evaluate argument expressions from right to left.

Write a version of `list_of_values` that evaluates argument expressions from left to right regardless of the order of evaluation in the underlying JavaScript. Also write a version of `list_of_values` that evaluates argument expressions from right to left.

4.1.2 Representing Components

Programmers write programs as text, i.e. sequences of characters, entered in a programming environment or a text editor. To run our evaluator, we need to start with a representation of this program text as a JavaScript value. In section 2.3.1 we introduced strings to represent text. We would like to evaluate programs such as `"const size = 2; 5 * size;"` from section 1.1.2. Unfortunately, such program text does not provide enough structure to the evaluator. In this example, the program parts `"size = 2"` and `"5 * size"` look similar, but carry very different meanings. Abstract syntax functions such as `declaration_value_expression` would be difficult and error-prone to implement by examining the program text. In this section, we therefore introduce a function `parse` that translates program text to a *tagged-list representation*, reminiscent of the tagged data of section 2.4.2. For example, the application of `parse` to the program string above produces a data structure that

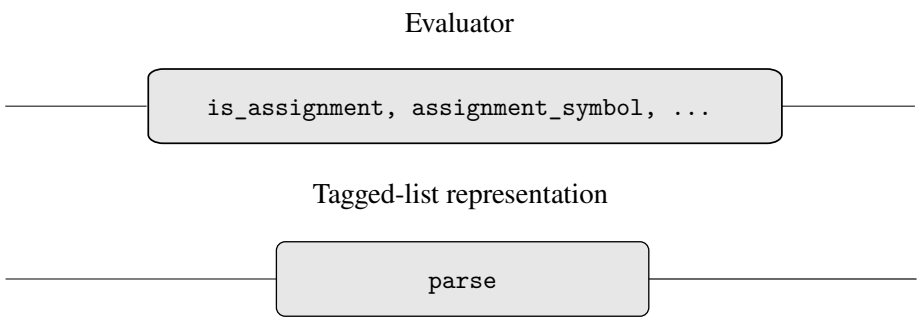


Figure 4.2 Syntax abstraction in the evaluator.

reflects the structure of the program: a sequence consisting of a constant declaration associating the name `size` with the value 2 and a multiplication.

```
parse("const size = 2; 5 * size;");
list("sequence",
     list(list("constant_declaration",
               list("name", "size"), list("literal", 2)),
          list("binary_operator_combination", "*",
               list("literal", 5), list("name", "size"))))
```

The syntax functions used by the evaluator access the tagged-list representation produced by `parse`.

The evaluator is reminiscent of the symbolic differentiation program discussed in section 2.3.2. Both programs operate on symbolic data. In both programs, the result of operating on an object is determined by operating recursively on the pieces of the object and combining the results in a way that depends on the type of the object. In both programs we used data abstraction to decouple the general rules of operation from the details of how the objects are represented. In the differentiation program this meant that the same differentiation function could deal with algebraic expressions in prefix form, in infix form, or in some other form. For the evaluator, this means that the syntax of the language being evaluated is determined solely by `parse` and the functions that classify and extract pieces of the tagged lists produced by `parse`.

Figure 4.2 depicts the abstraction barrier formed by the syntax predicates and selectors, which interface the evaluator to the tagged-list representation of programs, which in turn is separated from the string representation by `parse`. Below we describe the parsing of program components and list the corresponding syntax predicates and selectors, as well as constructors if they are needed.

Literal expression

Literal expressions are parsed into tagged lists with tag `"literal"` and the actual value.

≪ *literal-expression* ≫ = `list("literal",` *value*`)`

where *value* is the JavaScript value represented by the *literal-expression* string. Here ≪ *literal-expression* ≫ denotes the result of parsing the string *literal-expression*.

```
parse("1;");
list("literal", 1)

parse("'hello world';");
list("literal", "hello world")

parse("null;");
list("literal", null)
```

The syntax predicate for literal expressions is `is_literal`.

```
function is_literal(component) {
    return is_tagged_list(component, "literal");
}
```

It is defined in terms of the function `is_tagged_list`, which identifies lists that begin with a designated string:

```
function is_tagged_list(component, the_tag) {
    return is_pair(component) && head(component) === the_tag;
}
```

The second element of the list that results from parsing a literal expression is its actual JavaScript value. The selector for retrieving the value is `literal_value`.

```
function literal_value(component) {
    return head(tail(component));
}

literal_value(parse("null;"));
null
```

In the rest of this section, we just list the syntax predicates and selectors, and omit their declarations if they just access the obvious list elements.

We provide a constructor for literals, which will come in handy:

```
function make_literal(value) {
    return list("literal", value);
}
```

Names

The tagged-list representation for names includes the tag `"name"` as first element and the string representing the name as second element.

≪ *name* ≫ = `list("name",` *symbol*`)`

where *symbol* is a string that contains the characters that make up the *name* as written in the program. The syntax predicate for names is `is_name`. The symbol is accessed using the selector `symbol_of_name`. We provide a constructor for names, to be used by `operator_combination_to_application`:

```
function make_name(symbol) {
    return list("name", symbol);
}
```

Expression statements

We do not need to distinguish between expressions and expression statements. Consequently, `parse` can ignore the difference between the two kinds of components:

≪ *expression*; ≫ = ≪ *expression* ≫

Function applications

Function applications are parsed as follows:

≪ *fun-expr*(*arg-expr*$_1$, ..., *arg-expr*$_n$) ≫ =
    list("application",
        ≪ *fun-expr* ≫,
        list(≪ *arg-expr*$_1$ ≫, ..., ≪ *arg-expr*$_n$ ≫))

We declare `is_application` as the syntax predicate and `function_expression` and `arg_expressions` as the selectors. We add a constructor for function applications, to be used by `operator_combination_to_application`:

```
function make_application(function_expression, argument_expressions) {
    return list("application",
                function_expression, argument_expressions);
}
```

Conditionals

Conditional expressions are parsed as follows:

≪ *predicate* ? *consequent-expression* : *alternative-expression* ≫ =
        list("conditional_expression",
            ≪ *predicate* ≫,
            ≪ *consequent-expression* ≫,
            ≪ *alternative-expression* ≫)

Similarly, conditional statements are parsed as follows:

≪ if (*predicate*) *consequent-block* else *alternative-block* ≫ =
        list("conditional_statement",
            ≪ *predicate* ≫,
            ≪ *consequent-block* ≫,
            ≪ *alternative-block* ≫)

The syntax predicate `is_conditional` returns true for both kinds of conditionals, and the selectors `conditional_predicate`, `conditional_consequent`, and `conditional_alternative` can be applied to both kinds.

Lambda expressions

A lambda expression whose body is an expression is parsed as if the body consisted of a block containing a single return statement whose return expression is the body of the lambda expression.

```
≪ (name1, ..., namen) => expression ≫ =
    ≪ (name1, ..., namen) => { return expression; } ≫
```

A lambda expression whose body is a block is parsed as follows:

```
≪ (name1, ..., namen) => block ≫ =
     list("lambda_expression",
          list(≪ name1 ≫, ..., ≪ namen ≫),
          ≪ block ≫)
```

The syntax predicate is `is_lambda_expression` and the selector for the body of the lambda expression is `lambda_body`. The selector for the parameters, called `lambda_parameter_symbols`, additionally extracts the symbols from the names.

```
function lambda_parameter_symbols(component) {
    return map(symbol_of_name, head(tail(component)));
}
```

The function `function_decl_to_constant_decl` needs a constructor for lambda expressions:

```
function make_lambda_expression(parameters, body) {
    return list("lambda_expression", parameters, body);
}
```

Sequences

A sequence statement packages a sequence of statements into a single statement. A sequence of statements is parsed as follows:

```
≪ statement1 ··· statementn ≫ =
     list("sequence", list(≪ statement1 ≫, ..., ≪ statementn ≫))
```

The syntax predicate is `is_sequence` and the selector is `sequence_statements`. We retrieve the first of a list of statements using `first_statement` and the remaining statements using `rest_statements`. We test whether the list is empty using the predicate `is_empty_sequence` and whether it contains only one element using the predicate `is_last_statement`.[11]

```
function first_statement(stmts) { return head(stmts); }
function rest_statements(stmts) { return tail(stmts); }
function is_empty_sequence(stmts) { return is_null(stmts); }
function is_last_statement(stmts) { return is_null(tail(stmts)); }
```

11. These selectors for a list of statements are not intended as a data abstraction. They are introduced as mnemonic names for the basic list operations in order to make it easier to understand the explicit-control evaluator in section 5.4.

Blocks

Blocks are parsed as follows:[12]

≪ { *statements* } ≫ = list("block", ≪ *statements* ≫)

Here *statements* refers to a sequence of statements, as shown above. The syntax predicate is `is_block` and the selector is `block_body`.

Return statements

Return statements are parsed as follows:

≪ **return** *expression*; ≫ = list("return_statement", ≪ *expression* ≫)

The syntax predicate and selector are, respectively, `is_return_statement` and `return_expression`.

Assignments

Assignments are parsed as follows:

≪ *name* = *expression* ≫ = list("assignment", ≪ *name* ≫, ≪ *expression* ≫)

The syntax predicate is `is_assignment` and the selectors are `assignment_symbol` and `assignment_value_expression`. The symbol is wrapped in a tagged list representing the name, and thus `assignment_symbol` needs to unwrap it.

```
function assignment_symbol(component) {
    return symbol_of_name(head(tail(component))));
}
```

Constant, variable, and function declarations

Constant and variable declarations are parsed as follows:

```
≪ const name = expression; ≫ =
     list("constant_declaration", ≪ name ≫, ≪ expression ≫)

≪ let name = expression; ≫ =
     list("variable_declaration", ≪ name ≫, ≪ expression ≫)
```

The selectors `declaration_symbol` and `declaration_value_expression` apply to both kinds.

```
function declaration_symbol(component) {
    return symbol_of_name(head(tail(component)));
}
function declaration_value_expression(component) {
    return head(tail(tail(component)));
}
```

12. A parser implementation may decide to represent a block by just its statement sequence if none of the statements of the sequence are declarations, or to represent a sequence with only one statement by just that statement. The language processors in this chapter and in chapter 5 do not depend on these decisions.

The function `function_decl_to_constant_decl` needs a constructor for constant declarations:

```
function make_constant_declaration(name, value_expression) {
    return list("constant_declaration", name, value_expression);
}
```

Function declarations are parsed as follows:

```
≪ function name(name1, ... namen) block ≫ =
    list("function_declaration",
         ≪ name ≫,
         list(≪ name1 ≫, ..., ≪ namen ≫),
         ≪ block ≫)
```

The syntax predicate `is_function_declaration` recognizes these. The selectors are `function_declaration_name`, `function_declaration_parameters`, and `function_declaration_body`.

The syntax predicate `is_declaration` returns true for all three kinds of declarations.

```
function is_declaration(component) {
    return is_tagged_list(component, "constant_declaration") ||
           is_tagged_list(component, "variable_declaration") ||
           is_tagged_list(component, "function_declaration");
}
```

Derived components

Some syntactic forms in our language can be defined in terms of components involving other syntactic forms, rather than being implemented directly. One example is function declaration, which `evaluate` transforms into a constant declaration whose value expression is a lambda expression.[13]

```
function function_decl_to_constant_decl(component) {
    return make_constant_declaration(
               function_declaration_name(component),
               make_lambda_expression(
                   function_declaration_parameters(component),
                   function_declaration_body(component)));
}
```

Implementing the evaluation of function declarations in this way simplifies the evaluator because it reduces the number of syntactic forms for which the evaluation process must be explicitly specified.

Similarly, we define operator combinations in terms of function applications. Operator combinations are unary or binary and carry their operator symbol as second element in the tagged-list representation:

---

13. In actual JavaScript, there are subtle differences between the two forms; see footnote 55 in chapter 1. Exercise 4.17 addresses these differences.

≪ *unary-operator expression* ≫ =
    list("unary_operator_combination",
         "*unary-operator*",
         list(≪ *expression* ≫))

where *unary-operator* is `!` (for logical negation) or `-unary` (for numeric negation), and

≪ *expression*$_1$ *binary-operator expression*$_2$ ≫ =
    list("binary_operator_combination",
         "*binary-operator*",
         list(≪ *expression*$_1$ ≫, ≪ *expression*$_2$ ≫))

where *binary-operator* is `+`, `-`, `*`, `/`, `%`, `===`, `!==`, `>`, `<`, `>=` or `<=`. The syntax predicates are `is_operator_combination`, `is_unary_operator_combination`, and `is_binary_operator_combination`, and the selectors are `operator_symbol`, `first_operand`, and `second_operand`.

The evaluator uses `operator_combination_to_application` to transform an operator combination into a function application whose function expression is the name of the operator:

```
function operator_combination_to_application(component) {
    const operator = operator_symbol(component);
    return is_unary_operator_combination(component)
           ? make_application(make_name(operator),
                              list(first_operand(component)))
           : make_application(make_name(operator),
                              list(first_operand(component),
                                   second_operand(component)));
}
```

Components (such as function declarations and operator combinations) that we choose to implement as syntactic transformations are called *derived components*. Logical composition operations are also derived components (see exercise 4.4).

Exercise 4.2

The inverse of `parse` is called `unparse`. It takes as argument a tagged list as produced by `parse` and returns a string that adheres to JavaScript notation.

a. Write a function `unparse` by following the structure of `evaluate` (without the environment parameter), but producing a string that represents the given component, rather than evaluating it. Recall from section 3.3.4 that the operator `+` can be applied to two strings to concatenate them and that the primitive function `stringify` turns values such as 1.5, true, `null` and `undefined` into strings. Take care to respect operator precedences by surrounding the strings that result from unparsing operator combinations with parentheses (always or whenever necessary).

b. Your `unparse` function will come in handy when solving later exercises in this section. Improve `unparse` by adding `" "` (space) and `"\n"` (newline) characters to the result string, to follow the indentation style used in the JavaScript programs of this book. Adding such whitespace characters to (or removing them from) a program text in order to make the text easier to read is called *pretty-printing*.

Exercise 4.3

Rewrite `evaluate` so that the dispatch is done in data-directed style. Compare this with the data-directed differentiation function of exercise 2.73. (You may use the tag of the tagged-list representation as the type of the components.)

Exercise 4.4

Recall from section 1.1.6 that the logical composition operations `&&` and `||` are syntactic sugar for conditional expressions: The logical conjunction *expression*$_1$ `&&` *expression*$_2$ is syntactic sugar for *expression*$_1$ `?` *expression*$_2$ `: false`, and the logical disjunction *expression*$_1$ `||` *expression*$_2$ is syntactic sugar for *expression*$_1$ `? true :` *expression*$_2$. They are parsed as follows:

```
≪ expression1 logical-operation expression2 ≫ =
    list("logical_composition",
         "logical-operation",
         list(≪ expression1 ≫, ≪ expression2 ≫))
```

where *logical-operation* is `&&` or `||`. Install `&&` and `||` as new syntactic forms for the evaluator by declaring appropriate syntax functions and evaluation functions `eval_and` and `eval_or`. Alternatively, show how to implement `&&` and `||` as derived components.

Exercise 4.5

a. In JavaScript, lambda expressions must not have duplicate parameters. The evaluator in section 4.1.1 does not check for this.

   - Modify the evaluator so that any attempt to apply a function with duplicate parameters signals an error.
   - Implement a `verify` function that checks whether any lambda expression in a given program contains duplicate parameters. With such a function, we could check the entire program before we pass it to `evaluate`.

   In order to implement this check in an evaluator for JavaScript, which of these two approaches would you prefer? Why?

b. In JavaScript, the parameters of a lambda expression must be distinct from the names declared *directly* in the body block of the lambda expression (as opposed to in an inner block). Use your preferred approach above to check for this as well.

Exercise 4.6

The language Scheme includes a variant of `let` called `let*`. We could approximate the behavior of `let*` in JavaScript by stipulating that a `let*` declaration implicitly introduces a new block whose body includes the declaration and all subsequent statements of the statement sequence in which the declaration occurs. For example, the program

```
let* x = 3;
let* y = x + 2;
let* z = x + y + 5;
display(x * z);
```

displays 39 and could be seen as a shorthand for

```
{
  let x = 3;
  {
    let y = x + 2;
    {
      let z = x + y + 5;
      display(x * z);
    }
  }
}
```

a. Write a program in such an extended JavaScript language that behaves differently when some occurrences of the keyword `let` are replaced with `let*`.

b. Introduce `let*` as a new syntactic form by designing a suitable tagged-list representation and writing a parse rule. Declare a syntax predicate and selectors for the tagged-list representation.

c. Assuming that `parse` implements your new rule, write a `let_star_to_nested_let` function that transforms any occurrence of `let*` in a given program as described above. We could then evaluate a program `p` in the extended language by running `evaluate(let_star_to_nested_let(p))`.

d. As an alternative, consider implementing `let*` by adding to `evaluate` a clause that recognizes the new syntactic form and calls a function `eval_let_star_declaration`. Why does this approach not work?

Exercise 4.7

JavaScript supports *while loops* that execute a given statement repeatedly. Specifically,

`while` (*predicate*) { *body* }

evaluates the *predicate*, and if the result is true, evaluates the *body* and then evaluates the whole while loop again. Once the *predicate* evaluates to false, the while loop terminates.

For example, recall the imperative-style version of the iterative factorial function from section 3.1.3:

```
function factorial(n) {
    let product = 1;
    let counter = 1;
    function iter() {
        if (counter > n) {
            return product;
        } else {
            product = counter * product;
            counter = counter + 1;
            return iter();
        }
    }
    return iter();
}
```

We can formulate the same algorithm using a while loop as follows:

```
function factorial(n) {
    let product = 1;
    let counter = 1;
    while (counter <= n) {
        product = counter * product;
        counter = counter + 1;
    }
    return product;
}
```

While loops are parsed as follows:

```
≪ while (predicate) block ≫ =
        list("while_loop", ≪ predicate ≫, ≪ block ≫)
```

a. Declare a syntax predicate and selectors to handle while loops.

b. Declare a function `while_loop` that takes as arguments a predicate and a body—each represented by a function of no arguments—and simulates the behavior of the while loop. The `factorial` function would then look as follows:

```
function factorial(n) {
    let product = 1;
    let counter = 1;
    while_loop(() => counter <= n,
               () => {
                   product = counter * product;
                   counter = counter + 1;
               });
    return product;
}
```

Your function `while_loop` should generate an iterative process (see section 1.2.1).

c. Install while loops as a derived component by defining a transformation function `while_to_application` that makes use of your function `while_loop`.

d. What problem arises with this approach for implementing while loops, when the programmer decides within the body of the loop to return from the function that contains the loop?

e. Change your approach to address the problem. How about directly installing while loops for the evaluator, using a function `eval_while`?

f. Following this direct approach, implement a `break;` statement that immediately terminates the loop in which it is evaluated.

g. Implement a `continue;` statement that terminates only the loop iteration in which it is evaluated, and continues with evaluating the while loop predicate.

Exercise 4.8

The result of evaluating the body of a function is determined by its return statements. Following up on footnote 9 and the evaluation of declarations in section 4.1.1, this exercise addresses the question of what should be the result of evaluating a JavaScript program that consists of a sequence of statements (declarations, blocks, expression statements, and conditional statements) *outside of* any function body.

For such a program, JavaScript statically distinguishes between *value-producing* and *non-value-producing statements*. (Here "statically" means that we can make the distinction by *inspecting* the program rather than by running it.) All declarations are non-value-producing, and all expression statements and conditional statements are value-producing. The value of an expression statement is the value of the expression. The value of a conditional statement is the value of the branch that gets executed, or the value `undefined` if that branch is not value-producing. A block is value-producing if its body (sequence of statements) is value-producing, and then its value is the value of its body. A sequence is value-producing if any of its component statements is value-producing, and then its value is the value of its *last* value-producing component statement. Finally, if the whole program is not value-producing, its value is the value `undefined`.

a. According to this specification, what are the values of the following four programs?

```
1; 2; 3;

1; { if (true) {} else { 2; } }

1; const x = 2;

1; { let x = 2; { x = x + 3; } }
```

b. Modify the evaluator to adhere to this specification.

4.1.3 Evaluator Data Structures

In addition to defining the representation of components, the evaluator implementation must also define the data structures that the evaluator manipulates internally, as part of the execution of a program, such as the representation of functions and environments and the representation of true and false.

Testing of predicates

In order to limit the predicate expressions of conditionals to proper predicates (expressions that evaluate to a boolean value) as we do throughout this book, we insist here that the function `is_truthy` gets applied only to boolean values, and we accept only the boolean value `true` to be truthy. The opposite of `is_truthy` is called `is_falsy`.[14]

---

14. Conditionals in full JavaScript accept *any* value, not just a boolean, as the result of evaluating the "predicate" expression. JavaScript's notion of truthiness and falsiness is captured by the following variants of `is_truthy` and `is_falsy`:

```
function is_truthy(x) { return ! is_falsy(x); }
function is_falsy(x) {
    return (is_boolean(x) && !x )                  ||
           (is_number(x) && (x === 0 || x !== x )) ||
           (is_string(x) && x === "")              ||
           is_null(x)                              ||
           is_undefined(x);
}
```

The test `x !== x` is not a typo; the only JavaScript value for which `x !== x` yields true is the value `NaN` ("Not a Number"), which is considered to be a falsy number (also not a typo), along with 0. The numerical value `NaN` is the result of certain arithmetic border cases such as `0 / 0`.

The terms "truthy" and "falsy" were coined by Douglas Crockford, one of whose books (Crockford 2008) inspired this JavaScript adaptation.

```
function is_truthy(x) {
    return is_boolean(x)
           ? x
           : error(x, "boolean expected, received");
}
function is_falsy(x) { return ! is_truthy(x); }
```

Representing functions

To handle primitives, we assume that we have available the following functions:

- `apply_primitive_function`(*fun*, *args*)
  applies the given primitive function to the argument values in the list *args* and returns the result of the application.
- `is_primitive_function`(*fun*)
  tests whether *fun* is a primitive function.

These mechanisms for handling primitives are further described in section 4.1.4.

Compound functions are constructed from parameters, function bodies, and environments using the constructor `make_function`:

```
function make_function(parameters, body, env) {
    return list("compound_function", parameters, body, env);
}
function is_compound_function(f) {
    return is_tagged_list(f, "compound_function");
}
function function_parameters(f) { return list_ref(f, 1); }
function function_body(f) { return list_ref(f, 2); }
function function_environment(f) { return list_ref(f, 3); }
```

Representing return values

We saw in section 4.1.1 that the evaluation of a sequence terminates when a return statement is encountered, and that the evaluation of a function application needs to return the value `undefined` if the evaluation of the function body does not encounter a return statement. In order to recognize that a value resulted from a return statement, we introduce *return values* as evaluator data structures.

```
function make_return_value(content) {
    return list("return_value", content);
}
function is_return_value(value) {
    return is_tagged_list(value, "return_value");
}
function return_value_content(value) {
    return head(tail(value));
}
```

Operations on Environments

The evaluator needs operations for manipulating environments. As explained in section 3.2, an environment is a sequence of frames, where each frame is a table of bindings that associate symbols with their corresponding values. We use the following operations for manipulating environments:

- `lookup_symbol_value`(*symbol*, *env*)
  returns the value that is bound to *symbol* in the environment *env*, or signals an error if *symbol* is unbound.
- `extend_environment`(*symbols*, *values*, *base-env*)
  returns a new environment, consisting of a new frame in which the symbols in the list *symbols* are bound to the corresponding elements in the list *values*, where the enclosing environment is the environment *base-env*.
- `assign_symbol_value`(*symbol*, *value*, *env*)
  finds the innermost frame of *env* in which *symbol* is bound, and changes that frame so that *symbol* is now bound to *value*, or signals an error if *symbol* is unbound.

To implement these operations we represent an environment as a list of frames. The enclosing environment of an environment is the `tail` of the list. The empty environment is simply the empty list.

```
function enclosing_environment(env) { return tail(env); }
function first_frame(env) { return head(env); }
const the_empty_environment = null;
```

Each frame of an environment is represented as a pair of lists: a list of the names bound in that frame and a list of the associated values.[15]

```
function make_frame(symbols, values) { return pair(symbols, values); }
function frame_symbols(frame) { return head(frame); }
function frame_values(frame) { return tail(frame); }
```

To extend an environment by a new frame that associates symbols with values, we make a frame consisting of the list of symbols and the list of values, and we adjoin this to the environment. We signal an error if the number of symbols does not match the number of values.

```
function extend_environment(symbols, vals, base_env) {
    return length(symbols) === length(vals)
           ? pair(make_frame(symbols, vals), base_env)
           : error(pair(symbols, vals),
                   length(symbols) < length(vals)
                   ? "too many arguments supplied"
                   : "too few arguments supplied");
}
```

15. Frames are not really a data abstraction: The function `assign_symbol_value` below uses `set_head` to directly modify the values in a frame. The purpose of the frame functions is to make the environment-manipulation functions easy to read.

This is used by `apply` in section 4.1.1 to bind the parameters of a function to its arguments.

To look up a symbol in an environment, we scan the list of symbols in the first frame. If we find the desired symbol, we return the corresponding element in the list of values. If we do not find the symbol in the current frame, we search the enclosing environment, and so on. If we reach the empty environment, we signal an `"unbound name"` error.

```
function lookup_symbol_value(symbol, env) {
    function env_loop(env) {
        function scan(symbols, vals) {
            return is_null(symbols)
                   ? env_loop(enclosing_environment(env))
                   : symbol === head(symbols)
                   ? head(vals)
                   : scan(tail(symbols), tail(vals));
        }
        if (env === the_empty_environment) {
            error(symbol, "unbound name");
        } else {
            const frame = first_frame(env);
            return scan(frame_symbols(frame), frame_values(frame));
        }
    }
    return env_loop(env);
}
```

To assign a new value to a symbol in a specified environment, we scan for the symbol, just as in `lookup_symbol_value`, and change the corresponding value when we find it.

```
function assign_symbol_value(symbol, val, env) {
    function env_loop(env) {
        function scan(symbols, vals) {
            return is_null(symbols)
                   ? env_loop(enclosing_environment(env))
                   : symbol === head(symbols)
                   ? set_head(vals, val)
                   : scan(tail(symbols), tail(vals));
        }
        if (env === the_empty_environment) {
            error(symbol, "unbound name -- assignment");
        } else {
            const frame = first_frame(env);
            return scan(frame_symbols(frame), frame_values(frame));
        }
    }
    return env_loop(env);
}
```

The method described here is only one of many plausible ways to represent environments. Since we used data abstraction to isolate the rest of the evaluator from the

detailed choice of representation, we could change the environment representation if we wanted to. (See exercise 4.9.) In a production-quality JavaScript system, the speed of the evaluator's environment operations—especially that of symbol lookup—has a major impact on the performance of the system. The representation described here, although conceptually simple, is not efficient and would not ordinarily be used in a production system.[16]

Exercise 4.9

Instead of representing a frame as a pair of lists, we can represent a frame as a list of bindings, where each binding is a symbol-value pair. Rewrite the environment operations to use this alternative representation.

Exercise 4.10

The functions `lookup_symbol_value` and `assign_symbol_value` can be expressed in terms of a more abstract function for traversing the environment structure. Define an abstraction that captures the common pattern and redefine the two functions in terms of this abstraction.

Exercise 4.11

Our language distinguishes constants from variables by using different keywords—`const` and `let`—and prevents assignment to constants. However, our interpreter does not make use of this distinction; the function `assign_symbol_value` will happily assign a new value to a given symbol, regardless whether it is declared as a constant or a variable. Correct this flaw by calling the function `error` whenever an attempt is made to use a constant on the left-hand side of an assignment. You may proceed as follows:

- Introduce predicates `is_constant_declaration` and `is_variable_declaration` that allow you to distinguish the two kinds. As shown in section 4.1.2, `parse` distinguishes them by using the tags `"constant_declaration"` and `"variable_declaration"`.
- Change `scan_out_declarations` and (if necessary) `extend_environment` such that constants are distinguishable from variables in the frames in which they are bound.
- Change `assign_symbol_value` such that it checks whether the given symbol has been declared as a variable or as a constant, and in the latter case signals an error that assignment operations are not allowed on constants.
- Change `eval_declaration` such that when it encounters a constant declaration, it calls a new function, `assign_constant_value`, which does not perform the check that you introduced in `assign_symbol_value`.
- If necessary, change `apply` to ensure that assignment to function parameters remains possible.

---

16. The drawback of this representation (as well as the variant in exercise 4.9) is that the evaluator may have to search through many frames in order to find the binding for a given variable. (Such an approach is referred to as *deep binding*.) One way to avoid this inefficiency is to make use of a strategy called *lexical addressing*, which will be discussed in section 5.5.6.

Exercise 4.12

a. JavaScript's specification requires an implementation to signal a runtime error upon an attempt to access the value of a name before its declaration is evaluated (see the end of section 3.2.4). To achieve this behavior in the evaluator, change `lookup_symbol_value` to signal an error if the value it finds is `"*unassigned*"`.

b. Similarly, we must not assign a new value to a variable if we have not evaluated its `let` declaration yet. Change the evaluation of assignment such that assignment to a variable declared with `let` signals an error in this case.

Exercise 4.13

Prior to ECMAScript 2015's strict mode that we are using in this book, JavaScript variables worked quite differently from Scheme variables, which would have made this adaptation to JavaScript considerably less compelling.

a. Before ECMAScript 2015, the only way to declare a local variable in JavaScript was using the keyword `var` instead of the keyword `let`. The scope of variables declared with `var` is the entire body of the immediately surrounding function declaration or lambda expression, rather than just the immediately enclosing block. Modify `scan_out_declarations` and `eval_block` such that names declared with `const` and `let` follow the scoping rules of `var`.

b. When not in strict mode, JavaScript permits undeclared names to appear to the left of the `=` in assignments. Such an assignment adds the new binding to the global environment. Modify the function `assign_symbol_value` to make assignment behave this way. The strict mode, which forbids such assignments, was introduced in JavaScript in order to make programs more secure. What security issue is addressed by preventing assignment from adding bindings to the global environment?

4.1.4 Running the Evaluator as a Program

Given the evaluator, we have in our hands a description (expressed in JavaScript) of the process by which JavaScript statements and expressions are evaluated. One advantage of expressing the evaluator as a program is that we can run the program. This gives us, running within JavaScript, a working model of how JavaScript itself evaluates expressions. This can serve as a framework for experimenting with evaluation rules, as we shall do later in this chapter.

Our evaluator program reduces expressions ultimately to the application of primitive functions. Therefore, all that we need to run the evaluator is to create a mechanism that calls on the underlying JavaScript system to model the application of primitive functions.

There must be a binding for each primitive function name and operator, so that when `evaluate` evaluates the function expression of an application of a primitive, it will find an object to pass to `apply`. We thus set up a global environment that associates unique objects with the names of the primitive functions and operators that can appear in the expressions we will be evaluating. The global environment also includes bindings for `undefined` and other names, so that they can be used as constants in expressions to be evaluated.

```
function setup_environment() {
    return extend_environment(append(primitive_function_symbols,
                                     primitive_constant_symbols),
                              append(primitive_function_objects,
                                     primitive_constant_values),
                              the_empty_environment);
}

const the_global_environment = setup_environment();
```

It does not matter how we represent primitive function objects, so long as `apply` can identify and apply them using the functions `is_primitive_function` and `apply_primitive_function`. We have chosen to represent a primitive function as a list beginning with the string `"primitive"` and containing a function in the underlying JavaScript that implements that primitive.

```
function is_primitive_function(fun) {
    return is_tagged_list(fun, "primitive");
}
function primitive_implementation(fun) { return head(tail(fun)); }
```

The function `setup_environment` will get the primitive names and implementation functions from a list:[17]

```
const primitive_functions = list(list("head",    head            ),
                                 list("tail",    tail            ),
                                 list("pair",    pair            ),
                                 list("is_null", is_null         ),
                                 list("+",       (x, y) => x + y ),
                                 ⟨more primitive functions⟩
                                );
const primitive_function_symbols =
    map(f => head(f), primitive_functions);
const primitive_function_objects =
    map(f => list("primitive", head(tail(f))),
        primitive_functions);
```

Similar to primitive functions, we define other primitive constants that are installed in the global environment by the function `setup_environment`.

---

17. Any function defined in the underlying JavaScript can be used as a primitive for the metacircular evaluator. The name of a primitive installed in the evaluator need not be the same as the name of its implementation in the underlying JavaScript; the names are the same here because the metacircular evaluator implements JavaScript itself. Thus, for example, we could put `list("first", head)` or `list("square", x => x * x)` in the list of `primitive_functions`.

```
const primitive_constants = list(list("undefined", undefined),
                                 list("math_PI",   math_PI)
                                 ⟨more primitive constants⟩
                                );
const primitive_constant_symbols =
    map(c => head(c), primitive_constants);
const primitive_constant_values =
    map(c => head(tail(c)), primitive_constants);
```

To apply a primitive function, we simply apply the implementation function to the arguments, using the underlying JavaScript system:[18]

```
function apply_primitive_function(fun, arglist) {
    return apply_in_underlying_javascript(
               primitive_implementation(fun), arglist);
}
```

For convenience in running the metacircular evaluator, we provide a *driver loop* that models the read-evaluate-print loop of the underlying JavaScript system. It prints a *prompt* and reads an input program as a string. It transforms the program string into a tagged-list representation of the statement as described in section 4.1.2—a process called parsing and accomplished by the primitive function `parse`. We precede each printed result by an *output prompt* so as to distinguish the value of the program from other output that may be printed. The driver loop gets the program environment of the previous program as argument. As described at the end of section 3.2.4, the driver loop treats the program as if it were in a block: It scans out the declarations, extends the given environment by a frame containing a binding of each name to `"*unassigned*"`, and evaluates the program with respect to the extended environment, which is then passed as argument to the next iteration of the driver loop.

---

18. JavaScript's `apply` method expects the function arguments in a *vector*. (Vectors are called "arrays" in JavaScript.) Thus, the `arglist` is transformed into a vector—here using a while loop (see exercise 4.7):

```
function apply_in_underlying_javascript(prim, arglist) {
    const arg_vector = [];               // empty vector
    let i = 0;
    while (!is_null(arglist)) {
        arg_vector[i] = head(arglist);   // store value at index i
        i = i + 1;
        arglist = tail(arglist);
    }
    return prim.apply(prim, arg_vector); // apply is accessed via prim
}
```

We also made use of `apply_in_underlying_javascript` to declare the function `apply_generic` in section 2.4.3.

```
const input_prompt = "M-evaluate input: ";
const output_prompt = "M-evaluate value: ";

function driver_loop(env) {
    const input = user_read(input_prompt);
    if (is_null(input)) {
       display("evaluator terminated");
    } else {
       const program = parse(input);
       const locals = scan_out_declarations(program);
       const unassigneds = list_of_unassigned(locals);
       const program_env = extend_environment(locals, unassigneds, env);
       const output = evaluate(program, program_env);
       user_print(output_prompt, output);
       return driver_loop(program_env);
    }
}
```

We use JavaScript's `prompt` function to request and read the input string from the user:

```
function user_read(prompt_string) {
    return prompt(prompt_string);
}
```

The function `prompt` returns **`null`** when the user cancels the input. We use a special printing function `user_print`, to avoid printing the environment part of a compound function, which may be a very long list (or may even contain cycles).

```
function user_print(string, object) {
    function prepare(object) {
        return is_compound_function(object)
               ? "< compound-function >"
               : is_primitive_function(object)
               ? "< primitive-function >"
               : is_pair(object)
               ? pair(prepare(head(object)),
                      prepare(tail(object)))
               : object;
    }
    display(string + " " + stringify(prepare(object)));
}
```

Now all we need to do to run the evaluator is to initialize the global environment and start the driver loop. Here is a sample interaction:

```
const the_global_environment = setup_environment();
driver_loop(the_global_environment);
```

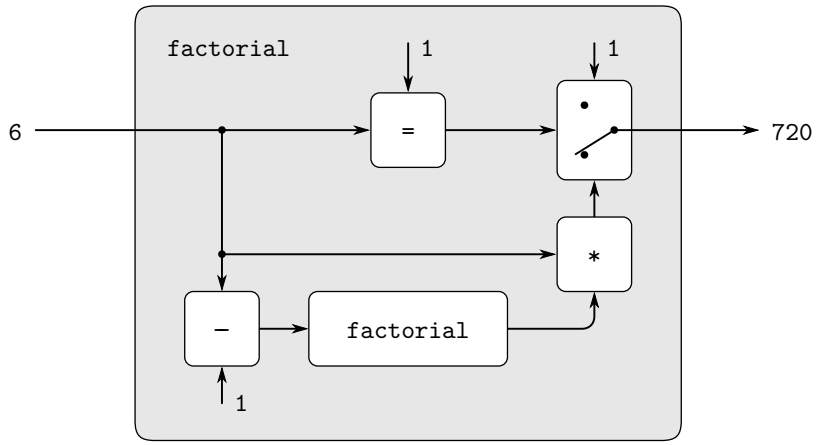


Figure 4.3 The factorial program, viewed as an abstract machine.

```
M-evaluate input:
function append(xs, ys) {
    return is_null(xs)
           ? ys
           : pair(head(xs), append(tail(xs), ys));
}

M-evaluate value:
undefined

M-evaluate input:
append(list("a", "b", "c"), list("d", "e", "f"));

M-evaluate value:
["a", ["b", ["c", ["d", ["e", ["f", null]]]]]]
```

Exercise 4.14

Eva Lu Ator and Louis Reasoner are each experimenting with the metacircular evaluator. Eva types in the definition of `map`, and runs some test programs that use it. They work fine. Louis, in contrast, has installed the system version of `map` as a primitive for the metacircular evaluator. When he tries it, things go terribly wrong. Explain why Louis's `map` fails even though Eva's works.

4.1.5 Data as Programs

In thinking about a JavaScript program that evaluates JavaScript statements and expressions, an analogy might be helpful. One operational view of the meaning of a program is that a program is a description of an abstract (perhaps infinitely large) machine. For example, consider the familiar program to compute factorials:

```
function factorial(n) {
    return n === 1
           ? 1
           : factorial(n - 1) * n;
}
```

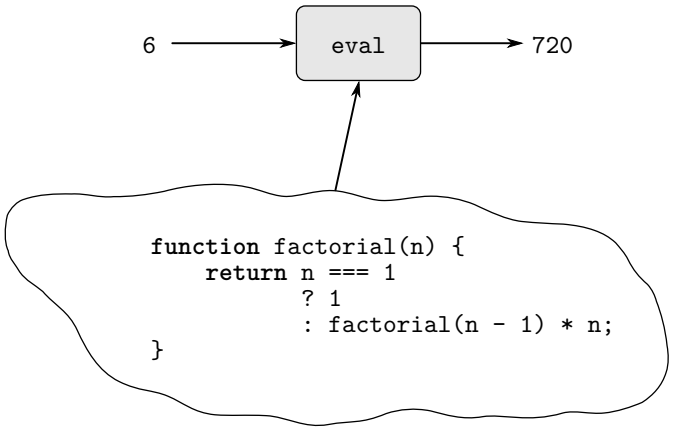


Figure 4.4 The evaluator emulating a factorial machine.

We may regard this program as the description of a machine containing parts that decrement, multiply, and test for equality, together with a two-position switch and another factorial machine. (The factorial machine is infinite because it contains another factorial machine within it.) Figure 4.3 is a flow diagram for the factorial machine, showing how the parts are wired together.

In a similar way, we can regard the evaluator as a very special machine that takes as input a description of a machine. Given this input, the evaluator configures itself to emulate the machine described. For example, if we feed our evaluator the definition of `factorial`, as shown in figure 4.4, the evaluator will be able to compute factorials.

From this perspective, our evaluator is seen to be a *universal machine*. It mimics other machines when these are described as JavaScript programs.[19] This is striking. Try to imagine an analogous evaluator for electrical circuits. This would be a circuit

19. The fact that the machines are described in JavaScript is inessential. If we give our evaluator a JavaScript program that behaves as an evaluator for some other language, say C, the JavaScript evaluator will emulate the C evaluator, which in turn can emulate any machine described as a C program. Similarly, writing a JavaScript evaluator in C produces a C program that can execute any JavaScript program. The deep idea here is that any evaluator can emulate any other. Thus, the notion of "what can in principle be computed" (ignoring practicalities of time and memory required) is independent of the language or the computer, and instead reflects an underlying notion of *computability*. This was first demonstrated in a clear way by Alan M. Turing (1912–1954), whose 1936 paper laid the foundations for theoretical computer science. In the paper, Turing presented a simple computational model—now known as a *Turing machine*—and argued that any "effective process" can be formulated as a program for such a machine. (This argument is known as the *Church–Turing thesis*.) Turing then implemented a universal machine, i.e., a Turing machine that behaves as an evaluator for Turing-machine programs. He used this framework to demonstrate that there are well-posed problems that cannot be computed by Turing machines (see exercise 4.15), and so by implication cannot be formulated as "effective processes." Turing went on to make fundamental contributions to practical computer science as well. For example, he invented the idea of structuring programs using general-purpose subroutines. See Hodges 1983 for a biography of Turing.

that takes as input a signal encoding the plans for some other circuit, such as a filter. Given this input, the circuit evaluator would then behave like a filter with the same description. Such a universal electrical circuit is almost unimaginably complex. It is remarkable that the program evaluator is a rather simple program.[20]

Another striking aspect of the evaluator is that it acts as a bridge between the data objects that are manipulated by our programming language and the programming language itself. Imagine that the evaluator program (implemented in JavaScript) is running, and that a user is typing programs to the evaluator and observing the results. From the perspective of the user, an input program such as `x * x;` is a program in the programming language, which the evaluator should execute. From the perspective of the evaluator, however, the program is simply a string or—after parsing—a tagged-list representation that is to be manipulated according to a well-defined set of rules.

That the user's programs are the evaluator's data need not be a source of confusion. In fact, it is sometimes convenient to ignore this distinction, and to give the user the ability to explicitly evaluate a string as a JavaScript statement, using JavaScript's primitive function `eval` that takes as argument a string. It parses the string and—provided that it is syntactically correct—evaluates the resulting representation in the environment in which `eval` is applied. Thus,

```
eval("5 * 5;");
```

and

```
evaluate(parse("5 * 5;"), the_global_environment);
```

will both return 25.[21]

Exercise 4.15

Given a one-argument function `f` and an object `a`, `f` is said to "halt" on `a` if evaluating the expression `f(a)` returns a value (as opposed to terminating with an error message or running forever). Show that it is impossible to write a function `halts` that correctly determines whether `f` halts on `a` for any function `f` and object `a`. Use the following reasoning: If you had such a function `halts`, you could implement the following program:

```
function run_forever() { return run_forever(); }
function strange(f) {
    return halts(f, f)
           ? run_forever()
           : "halted";
}
```

---

20. Some people find it counterintuitive that an evaluator, which is implemented by a relatively simple function, can emulate programs that are more complex than the evaluator itself. The existence of a universal evaluator machine is a deep and wonderful property of computation. *Recursion theory*, a branch of mathematical logic, is concerned with logical limits of computation. Douglas Hofstadter's beautiful book *Gödel, Escher, Bach* (1979) explores some of these ideas.

21. Note that `eval` may not be available in the JavaScript environment that you are using, or its use may be restricted for security reasons.

Now consider evaluating the expression `strange(strange)` and show that any possible outcome (either halting or running forever) violates the intended behavior of `halts`.[22]

4.1.6 Internal Declarations

In JavaScript, the scope of a declaration is the entire block that immediately surrounds the declaration, not just the portion of the block starting at the point where the declaration occurs. This section takes a closer look at this design choice.

Let us revisit the pair of mutually recursive functions `is_even` and `is_odd` from Section 3.2.4, declared locally in the body of a function `f`.

```
function f(x) {
    function is_even(n) {
        return n === 0
               ? true
               : is_odd(n - 1);
    }
    function is_odd(n) {
        return n === 0
               ? false
               : is_even(n - 1);
    }
    return is_even(x);
}
```

Our intention here is that the name `is_odd` in the body of the function `is_even` should refer to the function `is_odd` that is declared after `is_even`. The scope of the name `is_odd` is the entire body block of `f`, not just the portion of the body of `f` starting at the point where the declaration of `is_odd` occurs. Indeed, when we consider that `is_odd` is itself defined in terms of `is_even`—so that `is_even` and `is_odd` are mutually recursive functions—we see that the only satisfactory interpretation of the two declarations is to regard them as if the names `is_even` and `is_odd` were being added to the environment simultaneously. More generally, in block structure, the scope of a local name is the entire block in which the declaration is evaluated.

---

22. Although we stipulated that `halts` is given a function object, notice that this reasoning still applies even if `halts` can gain access to the function's text and its environment. This is Turing's celebrated *Halting Theorem*, which gave the first clear example of a *noncomputable* problem, i.e., a well-posed task that cannot be carried out as a computational function.

The evaluation of blocks in the metacircular evaluator of section 4.1.1 achieves such a simultaneous scope for local names by scanning out the declarations in the block and extending the current environment with a frame containing bindings for all the declared names before evaluating the declarations. Thus the new environment in which the block body is evaluated already contains bindings for `is_even` and `is_odd`, and any occurrence of one of these names refers to the correct binding. Once their declarations are evaluated, these names are bound to their declared values, namely function objects that have the extended environment as their environment part. Thus, for example, by the time `is_even` gets applied in the body of `f`, its environment already contains the correct binding for the symbol `is_odd`, and the evaluation of the name `is_odd` in the body of `is_even` retrieves the correct value.

Exercise 4.16

Consider the function `f_3` of section 1.3.2:

```
function f_3(x, y) {
    const a = 1 + x * y;
    const b = 1 - y;
    return x * square(a) + y * b + a * b;
}
```

a. Draw a diagram of the environment in effect during evaluation of the return expression of `f_3`.

b. When evaluating a function application, the evaluator creates two frames: one for the parameters and one for the names declared *directly* in the function's body block, as opposed to in an inner block. Since all these names have the same scope, an implementation could combine the two frames. Change the evaluator such that the evaluation of the body block does not create a new frame. You may assume that this will not result in duplicate names in the frame (exercise 4.5 justifies this).

Exercise 4.17

Eva Lu Ator is writing programs in which function declarations and other statements are interleaved. She needs to make sure that the declarations are evaluated before the functions are applied. She complains: "Why can't the evaluator take care of this chore, and hoist all function declarations to the beginning of the block in which they appear? Function declarations outside of blocks should be hoisted to the beginning of the program."

a. Modify the evaluator following Eva's suggestion.

b. The designers of JavaScript decided to follow Eva's approach. Discuss this decision.

c. In addition, the designers of JavaScript decided to allow the name declared by a function declaration to be reassigned using assignment. Modify your solution accordingly and discuss this decision.

Exercise 4.18

Recursive functions are obtained in a roundabout way in our interpreter: First declare the name that will refer to the recursive function and assign to it the special value `"*unassigned*"`; then define the recursive function in the scope of that name; and finally

assign the defined function to the name. By the time the recursive function gets applied, any occurrences of the name in the body properly refer to the recursive function. Amazingly, it is possible to specify recursive functions without using declarations or assignment. The following program computes 10 factorial by applying a recursive factorial function:[23]

```
(n => (fact => fact(fact, n))
      ((ft, k) => k === 1
                  ? 1
                  : k * ft(ft, k - 1)))(10);
```

a. Check (by evaluating the expression) that this really does compute factorials. Devise an analogous expression for computing Fibonacci numbers.

b. Consider the function `f` given above:

```
function f(x) {
    function is_even(n) {
        return n === 0
               ? true
               : is_odd(n - 1);
    }
    function is_odd(n) {
        return n === 0
               ? false
               : is_even(n - 1);
    }
    return is_even(x);
}
```

Fill in the missing expressions to complete an alternative declaration of `f`, which has no internal function declarations:

```
function f(x) {
    return ((is_even, is_odd) => is_even(is_even, is_odd, x))
           ((is_ev, is_od, n) => n === 0 ? true : is_od(⟨??⟩, ⟨??⟩, ⟨??⟩),
            (is_ev, is_od, n) => n === 0 ? false : is_ev(⟨??⟩, ⟨??⟩, ⟨??⟩)));
}
```

Sequential Declaration Processing

The design of our evaluator of section 4.1.1 imposes a runtime burden on the evaluation of blocks: It needs to scan the body of the block for locally declared names, extend the current environment with a new frame that binds those names, and evaluate the block body in this extended environment. Alternatively, the evaluation of a block could extend the current environment with an empty frame. The evaluation of each declaration in the block body would then add a new binding to that frame. To implement this design, we first simplify `eval_block`:

---

23. This example illustrates a programming trick for formulating recursive functions without using assignment. The most general trick of this sort is the *Y operator*, which can be used to give a "pure λ-calculus" implementation of recursion. (See Stoy 1977 for details on the lambda calculus, and Gabriel 1988 for an exposition of the *Y* operator in the language Scheme.)

```
function eval_block(component, env) {
    const body = block_body(component);
    return evaluate(body, extend_environment(null, null, env);
}
```

The function `eval_declaration` can no longer assume that the environment already has a binding for the name. Instead of using `assign_symbol_value` to change an existing binding, it calls a new function, `add_binding_to_frame`, to add to the first frame of the environment a binding of the name to the value of the value expression.

```
function eval_declaration(component, env) {
    add_binding_to_frame(
        declaration_symbol(component),
        evaluate(declaration_value_expression(component), env),
        first_frame(env));
    return undefined;
}
function add_binding_to_frame(symbol, value, frame) {
    set_head(frame, pair(symbol, head(frame)));
    set_tail(frame, pair(value, tail(frame)));
}
```

With sequential declaration processing, the scope of a declaration is no longer the entire block that immediately surrounds the declaration, but rather just the portion of the block starting at the point where the declaration occurs. Although we no longer have simultaneous scope, sequential declaration processing will evaluate calls to the function `f` at the beginning of this section correctly, but for an "accidental" reason: Since the declarations of the internal functions come first, no calls to these functions will be evaluated until all of them have been declared. Hence, `is_odd` will have been declared by the time `is_even` is executed. In fact, sequential declaration processing will give the same result as our scanning-out-names evaluator in section 4.1.1 for any function in which the internal declarations come first in a body and evaluation of the value expressions for the declared names doesn't actually use any of the declared names. Exercise 4.19 shows an example of a function that doesn't obey these restrictions, so that the alternative evaluator isn't equivalent to our scanning-out-names evaluator.

Sequential declaration processing is more efficient and easier to implement than scanning out names. However, with sequential processing, the declaration to which a name refers may depend on the order in which the statements in a block are evaluated. In exercise 4.19, we see that views may differ as to whether that is desirable.

Exercise 4.19

Ben Bitdiddle, Alyssa P. Hacker, and Eva Lu Ator are arguing about the desired result of evaluating the program

```
const a = 1;
function f(x) {
    const b = a + x;
    const a = 5;
    return a + b;
}
f(10);
```

Ben asserts that the result should be obtained using the sequential processing of declarations: `b` is declared to be 11, then `a` is declared to be 5, so the result is 16. Alyssa objects that mutual recursion requires the simultaneous scope rule for internal function declarations, and that it is unreasonable to treat function names differently from other names. Thus, she argues for the mechanism implemented in section 4.1.1. This would lead to `a` being unassigned at the time that the value for `b` is to be computed. Hence, in Alyssa's view the function should produce an error. Eva has a third opinion. She says that if the declarations of `a` and `b` are truly meant to be simultaneous, then the value 5 for `a` should be used in evaluating `b`. Hence, in Eva's view `a` should be 5, `b` should be 15, and the result should be 20. Which (if any) of these viewpoints do you support? Can you devise a way to implement internal declarations so that they behave as Eva prefers?[24]

4.1.7 Separating Syntactic Analysis from Execution

The evaluator implemented above is simple, but it is very inefficient, because the syntactic analysis of components is interleaved with their execution. Thus if a program is executed many times, its syntax is analyzed many times. Consider, for example, evaluating `factorial(4)` using the following definition of `factorial`:

```
function factorial(n) {
    return n === 1
           ? 1
           : factorial(n - 1) * n;
}
```

Each time `factorial` is called, the evaluator must determine that the body is a conditional expression and extract the predicate. Only then can it evaluate the predicate and dispatch on its value. Each time it evaluates the expression `factorial(n - 1) * n`, or the subexpressions `factorial(n - 1)` and `n - 1`, the evaluator must perform the case analysis in `evaluate` to determine that the expression is an application, and must extract its function expression and argument expressions. This analysis is expensive. Performing it repeatedly is wasteful.

---

24. The designers of JavaScript support Alyssa on the following grounds: Eva is in principle correct—the declarations should be regarded as simultaneous. But it seems difficult to implement a general, efficient mechanism that does what Eva requires. In the absence of such a mechanism, it is better to generate an error in the difficult cases of simultaneous declarations (Alyssa's notion) than to produce an incorrect answer (as Ben would have it).

We can transform the evaluator to be significantly more efficient by arranging things so that syntactic analysis is performed only once.[25] We split `evaluate`, which takes a component and an environment, into two parts. The function `analyze` takes only the component. It performs the syntactic analysis and returns a new function, the *execution function*, that encapsulates the work to be done in executing the analyzed component. The execution function takes an environment as its argument and completes the evaluation. This saves work because `analyze` will be called only once on a component, while the execution function may be called many times.

With the separation into analysis and execution, `evaluate` now becomes

```
function evaluate(component, env) {
    return analyze(component)(env);
}
```

The result of calling `analyze` is the execution function to be applied to the environment. The `analyze` function is the same case analysis as performed by the original `evaluate` of section 4.1.1, except that the functions to which we dispatch perform only analysis, not full evaluation:

```
function analyze(component) {
    return is_literal(component)
           ? analyze_literal(component)
           : is_name(component)
           ? analyze_name(component)
           : is_application(component)
           ? analyze_application(component)
           : is_operator_combination(component)
           ? analyze(operator_combination_to_application(component))
           : is_conditional(component)
           ? analyze_conditional(component)
           : is_lambda_expression(component)
           ? analyze_lambda_expression(component)
           : is_sequence(component)
           ? analyze_sequence(sequence_statements(component))
           : is_block(component)
           ? analyze_block(component)
           : is_return_statement(component)
           ? analyze_return_statement(component)
           : is_function_declaration(component)
           ? analyze(function_decl_to_constant_decl(component))
           : is_declaration(component)
           ? analyze_declaration(component)
           : is_assignment(component)
           ? analyze_assignment(component)
           : error(component, "unknown syntax -- analyze");
}
```

25. This technique is an integral part of the compilation process, which we shall discuss in chapter 5. Jonathan Rees wrote a Scheme interpreter like this in about 1982 for the T project (Rees and Adams 1982). Marc Feeley 1986 (see also Feeley and Lapalme 1987) independently invented this technique in his master's thesis.

Here is the simplest syntactic analysis function, which handles literal expressions. It returns an execution function that ignores its environment argument and just returns the value of the literal:

```
function analyze_literal(component) {
    return env => literal_value(component);
}
```

Looking up the value of a name must still be done in the execution phase, since this depends upon knowing the environment.[26]

```
function analyze_name(component) {
    return env => lookup_symbol_value(symbol_of_name(component), env);
}
```

To analyze an application, we analyze the function expression and argument expressions and construct an execution function that calls the execution function of the function expression (to obtain the actual function to be applied) and the argument-expression execution functions (to obtain the actual arguments). We then pass these to `execute_application`, which is the analog of `apply` in section 4.1.1. The function `execute_application` differs from `apply` in that the function body for a compound function has already been analyzed, so there is no need to do further analysis. Instead, we just call the execution function for the body on the extended environment.

```
function analyze_application(component) {
    const ffun = analyze(function_expression(component));
    const afuns = map(analyze, arg_expressions(component));
    return env => execute_application(ffun(env),
                                      map(afun => afun(env), afuns));
}
function execute_application(fun, args) {
    if (is_primitive_function(fun)) {
        return apply_primitive_function(fun, args);
    } else if (is_compound_function(fun)) {
        const result = function_body(fun)
                       (extend_environment(function_parameters(fun),
                                           args,
                                           function_environment(fun)));
        return is_return_value(result)
               ? return_value_content(result)
               : undefined;
    } else {
        error(fun, "unknown function type -- execute_application");
    }
}
```

26. There is, however, an important part of the search for a name that *can* be done as part of the syntactic analysis. As we will show in section 5.5.6, one can determine the position in the environment structure where the value of the variable will be found, thus obviating the need to scan the environment for the entry that matches the variable.

For conditionals, we extract and analyze the predicate, consequent, and alternative at analysis time.

```
function analyze_conditional(component) {
    const pfun = analyze(conditional_predicate(component));
    const cfun = analyze(conditional_consequent(component));
    const afun = analyze(conditional_alternative(component));
    return env => is_truthy(pfun(env)) ? cfun(env) : afun(env);
}
```

Analyzing a lambda expression also achieves a major gain in efficiency: We analyze the lambda body only once, even though functions resulting from evaluation of the lambda expression may be applied many times.

```
function analyze_lambda_expression(component) {
    const params = lambda_parameter_symbols(component);
    const bfun = analyze(lambda_body(component));
    return env => make_function(params, bfun, env);
}
```

Analysis of a sequence of statements is more involved.[27] Each statement in the sequence is analyzed, yielding an execution function. These execution functions are combined to produce an execution function that takes an environment as argument and sequentially calls each individual execution function with the environment as argument.

```
function analyze_sequence(stmts) {
    function sequentially(fun1, fun2) {
        return env => {
                    const fun1_val = fun1(env);
                    return is_return_value(fun1_val)
                           ? fun1_val
                           : fun2(env);
                };
    }
    function loop(first_fun, rest_funs) {
        return is_null(rest_funs)
               ? first_fun
               : loop(sequentially(first_fun, head(rest_funs)),
                      tail(rest_funs));
    }
    const funs = map(analyze, stmts);
    return is_null(funs)
           ? env => undefined
           : loop(head(funs), tail(funs));
}
```

27. See exercise 4.21 for some insight into the processing of sequences.

The body of a block is scanned only once for local declarations. The bindings are installed in the environment when the execution function for the block is called.

```
function analyze_block(component) {
    const body = block_body(component);
    const bfun = analyze(body);
    const locals = scan_out_declarations(body);
    const unassigneds = list_of_unassigned(locals);
    return env => bfun(extend_environment(locals, unassigneds, env));
}
```

For return statements, we analyze the return expression. The execution function for the return statement simply calls the execution function for the return expression and wraps the result in a return value.

```
function analyze_return_statement(component) {
    const rfun = analyze(return_expression(component));
    return env => make_return_value(rfun(env));
}
```

The function `analyze_assignment` must defer actually setting the variable until the execution, when the environment has been supplied. However, the fact that the assignment-value expression can be analyzed (recursively) during analysis is a major gain in efficiency, because the assignment-value expression will now be analyzed only once. The same holds true for constant and variable declarations.

```
function analyze_assignment(component) {
    const symbol = assignment_symbol(component);
    const vfun = analyze(assignment_value_expression(component));
    return env => {
                const value = vfun(env);
                assign_symbol_value(symbol, value, env);
                return value;
            };
}
function analyze_declaration(component) {
    const symbol = declaration_symbol(component);
    const vfun = analyze(declaration_value_expression(component));
    return env => {
                assign_symbol_value(symbol, vfun(env), env);
                return undefined;
            };
}
```

Our new evaluator uses the same data structures, syntax functions, and runtime support functions as in sections 4.1.2, 4.1.3, and 4.1.4.

Exercise 4.20

Extend the evaluator in this section to support while loops. (See exercise 4.7.)

Exercise 4.21

Alyssa P. Hacker doesn't understand why `analyze_sequence` needs to be so complicated. All the other analysis functions are straightforward transformations of the corresponding evaluation functions (or `evaluate` clauses) in section 4.1.1. She expected `analyze_sequence` to look like this:

```
function analyze_sequence(stmts) {
    function execute_sequence(funs, env) {
        if (is_null(funs)) {
            return undefined;
        } else if (is_null(tail(funs))) {
            return head(funs)(env);
        } else {
            const head_val = head(funs)(env);
            return is_return_value(head_val)
                   ? head_val
                   : execute_sequence(tail(funs), env);
        }
    }
    const funs = map(analyze, stmts);
    return env => execute_sequence(funs, env);
}
```

Eva Lu Ator explains to Alyssa that the version in the text does more of the work of evaluating a sequence at analysis time. Alyssa's sequence-execution function, rather than having the calls to the individual execution functions built in, loops through the functions in order to call them: In effect, although the individual statements in the sequence have been analyzed, the sequence itself has not been.

Compare the two versions of `analyze_sequence`. For example, consider the common case (typical of function bodies) where the sequence has just one statement. What work will the execution function produced by Alyssa's program do? What about the execution function produced by the program in the text above? How do the two versions compare for a sequence with two expressions?

Exercise 4.22

Design and carry out some experiments to compare the speed of the original metacircular evaluator with the version in this section. Use your results to estimate the fraction of time that is spent in analysis versus execution for various functions.

4.2 Lazy Evaluation

Now that we have an evaluator expressed as a JavaScript program, we can experiment with alternative choices in language design simply by modifying the evaluator. Indeed, new languages are often invented by first writing an evaluator that embeds the new language within an existing high-level language. For example, if we wish to discuss some aspect of a proposed modification to JavaScript with another member of the JavaScript community, we can supply an evaluator that embodies the change. The recipient can then experiment with the new evaluator and send back comments as further modifications. Not only does the high-level implementation base make it easier to test and debug the evaluator; in addition, the embedding enables the designer

to snarf[28] features from the underlying language, just as our embedded JavaScript evaluator uses primitives and control structure from the underlying JavaScript. Only later (if ever) need the designer go to the trouble of building a complete implementation in a low-level language or in hardware. In this section and the next we explore some variations on JavaScript that provide significant additional expressive power.

4.2.1 Normal Order and Applicative Order

In section 1.1, where we began our discussion of models of evaluation, we noted that JavaScript is an *applicative-order* language, namely, that all the arguments to JavaScript functions are evaluated when the function is applied. In contrast, *normal-order* languages delay evaluation of function arguments until the actual argument values are needed. Delaying evaluation of function arguments until the last possible moment (e.g., until they are required by a primitive operation) is called *lazy evaluation*.[29] Consider the function

```
function try_me(a, b) {
    return a === 0 ? 1 : b;
}
```

Evaluating `try_me(0, head(null));` signals an error in JavaScript. With lazy evaluation, there would be no error. Evaluating the statement would return 1, because the argument `head(null)` would never be evaluated.

An example that exploits lazy evaluation is the declaration of a function `unless`

```
function unless(condition, usual_value, exceptional_value) {
    return condition ? exceptional_value : usual_value;
}
```

that can be used in statements such as

```
unless(is_null(xs), head(xs), display("error: xs should not be null"));
```

This won't work in an applicative-order language because both the usual value and the exceptional value will be evaluated before `unless` is called (compare exercise 1.6). An advantage of lazy evaluation is that some functions, such as `unless`, can do useful computation even if evaluation of some of their arguments would produce errors or would not terminate.

---

28. Snarf: "To grab, especially a large document or file for the purpose of using it either with or without the owner's permission." Snarf down: "To snarf, sometimes with the connotation of absorbing, processing, or understanding." (These definitions were snarfed from Steele et al. 1983. See also Raymond 1996.)

29. The difference between the "lazy" terminology and the "normal-order" terminology is somewhat fuzzy. Generally, "lazy" refers to the mechanisms of particular evaluators, while "normal-order" refers to the semantics of languages, independent of any particular evaluation strategy. But this is not a hard-and-fast distinction, and the two terminologies are often used interchangeably.

If the body of a function is entered before an argument has been evaluated we say that the function is *non-strict* in that argument. If the argument is evaluated before the body of the function is entered we say that the function is *strict* in that argument.[30] In a purely applicative-order language, all functions are strict in each argument. In a purely normal-order language, all compound functions are non-strict in each argument, and primitive functions may be either strict or non-strict. There are also languages (see exercise 4.29) that give programmers detailed control over the strictness of the functions they define.

A striking example of a function that can usefully be made non-strict is `pair` (or, in general, almost any constructor for data structures). One can do useful computation, combining elements to form data structures and operating on the resulting data structures, even if the values of the elements are not known. It makes perfect sense, for instance, to compute the length of a list without knowing the values of the individual elements in the list. We will exploit this idea in section 4.2.3 to implement the streams of chapter 3 as lists formed of non-strict pairs.

Exercise 4.23

Suppose that (in ordinary applicative-order JavaScript) we define `unless` as shown above and then define `factorial` in terms of `unless` as

```
function factorial(n) {
    return unless(n === 1,
                  n * factorial(n - 1),
                  1);
}
```

What happens if we attempt to evaluate `factorial(5)`? Will our functions work in a normal-order language?

Exercise 4.24

Ben Bitdiddle and Alyssa P. Hacker disagree over the importance of lazy evaluation for implementing things such as `unless`. Ben points out that it's possible to implement `unless` in applicative order as a syntactic form. Alyssa counters that, if one did that, `unless` would be merely syntax, not a function that could be used in conjunction with higher-order functions. Fill in the details on both sides of the argument. Show how to implement `unless` as a derived component (like operator combination), by catching in `evaluate` applications whose function expression is the name `unless`. Give an example of a situation where it might be useful to have `unless` available as a function, rather than as a syntactic form.

4.2.2 An Interpreter with Lazy Evaluation

In this section we will implement a normal-order language that is the same as JavaScript except that compound functions are non-strict in each argument. Primitive functions will still be strict. It is not difficult to modify the evaluator of section 4.1.1

30. The "strict" versus "non-strict" terminology means essentially the same as "applicative-order" versus "normal-order," except that it refers to individual functions and arguments rather than to the language as a whole. At a conference on programming languages you might hear someone say, "The normal-order language Hassle has certain strict primitives. Other functions take their arguments by lazy evaluation."

so that the language it interprets behaves this way. Almost all the required changes center around function application.

The basic idea is that, when applying a function, the interpreter must determine which arguments are to be evaluated and which are to be delayed. The delayed arguments are not evaluated; instead, they are transformed into objects called *thunk*s.[31] The thunk must contain the information required to produce the value of the argument when it is needed, as if it had been evaluated at the time of the application. Thus, the thunk must contain the argument expression and the environment in which the function application is being evaluated.

The process of evaluating the expression in a thunk is called *forcing*.[32] In general, a thunk will be forced only when its value is needed: when it is passed to a primitive function that will use the value of the thunk; when it is the value of a predicate of a conditional; and when it is the value of a function expression that is about to be applied as a function. One design choice we have available is whether or not to *memoize* thunks, similar to the optimization for streams in section 3.5.1. With memoization, the first time a thunk is forced, it stores the value that is computed. Subsequent forcings simply return the stored value without repeating the computation. We'll make our interpreter memoize, because this is more efficient for many applications. There are tricky considerations here, however.[33]

Modifying the evaluator

The main difference between the lazy evaluator and the one in section 4.1 is in the handling of function applications in `evaluate` and `apply`.

The `is_application` clause of `evaluate` becomes

```
: is_application(component)
? apply(actual_value(function_expression(component), env),
        arg_expressions(component), env)
```

This is almost the same as the `is_application` clause of `evaluate` in section 4.1.1. For lazy evaluation, however, we call `apply` with the argument expressions, rather than the arguments produced by evaluating them. Since we will need the environment to construct thunks if the arguments are to be delayed, we must pass this as well. We

---

31. The word *thunk* was invented by an informal working group that was discussing the implementation of call-by-name in Algol 60. They observed that most of the analysis of ("thinking about") the expression could be done at compile time; thus, at run time, the expression would already have been "thunk" about (Ingerman et al. 1960).

32. This is analogous to the forcing of the delayed objects that were introduced in chapter 3 to represent streams. The critical difference between what we are doing here and what we did in chapter 3 is that we are building delaying and forcing into the evaluator, and thus making this uniform and automatic throughout the language.

33. Lazy evaluation combined with memoization is sometimes referred to as *call-by-need* argument passing, in contrast to *call-by-name* argument passing. (Call-by-name, introduced in Algol 60, is similar to non-memoized lazy evaluation.) As language designers, we can build our evaluator to memoize, not to memoize, or leave this an option for programmers (exercise 4.29). As you might expect from chapter 3, these choices raise issues that become both subtle and confusing in the presence of assignments. (See exercises 4.25 and 4.27.) An excellent article by Clinger (1982) attempts to clarify the multiple dimensions of confusion that arise here.

still evaluate the function expression, because `apply` needs the actual function to be applied in order to dispatch on its type (primitive versus compound) and apply it.

Whenever we need the actual value of an expression, we use

```
function actual_value(exp, env) {
  return force_it(evaluate(exp, env));
}
```

instead of just `evaluate`, so that if the expression's value is a thunk, it will be forced.

Our new version of `apply` is also almost the same as the version in section 4.1.1. The difference is that `evaluate` has passed in unevaluated argument expressions: For primitive functions (which are strict), we evaluate all the arguments before applying the primitive; for compound functions (which are non-strict) we delay all the arguments before applying the function.

```
function apply(fun, args, env) {
  if (is_primitive_function(fun)) {
      return apply_primitive_function(
                 fun,
                 list_of_arg_values(args, env));              // changed
  } else if (is_compound_function(fun)) {
      const result = evaluate(
                        function_body(fun),
                        extend_environment(
                            function_parameters(fun),
                            list_of_delayed_args(args, env), // changed
                        function_environment(fun)));
      return is_return_value(result)
             ? return_value_content(result)
             : undefined;
  } else {
      error(fun, "unknown function type -- apply");
  }
}
```

The functions that process the arguments are just like `list_of_values` from section 4.1.1, except that `list_of_delayed_args` delays the arguments instead of evaluating them, and `list_of_arg_values` uses `actual_value` instead of `evaluate`:

```
function list_of_arg_values(exps, env) {
  return map(exp => actual_value(exp, env), exps);
}
function list_of_delayed_args(exps, env) {
  return map(exp => delay_it(exp, env), exps);
}
```

The other place we must change the evaluator is in the handling of conditionals, where we must use `actual_value` instead of `evaluate` to get the value of the predicate expression before testing whether it is true or false:

```
function eval_conditional(component, env) {
  return is_truthy(actual_value(conditional_predicate(component), env))
         ? evaluate(conditional_consequent(component), env)
         : evaluate(conditional_alternative(component), env);
}
```

Finally, we must change the `driver_loop` function (from section 4.1.4) to use `actual_value` instead of `evaluate`, so that if a delayed value is propagated back to the read-evaluate-print loop, it will be forced before being printed. We also change the prompts to indicate that this is the lazy evaluator:

```
const input_prompt = "L-evaluate input: ";
const output_prompt = "L-evaluate value: ";

function driver_loop(env) {
  const input = user_read(input_prompt);
  if (is_null(input)) {
      display("evaluator terminated");
  } else {
      const program = parse(input);
      const locals = scan_out_declarations(program);
      const unassigneds = list_of_unassigned(locals);
      const program_env = extend_environment(locals, unassigneds, env);
      const output = actual_value(program, program_env);
      user_print(output_prompt, output);
      return driver_loop(program_env);
  }
}
```

With these changes made, we can start the evaluator and test it. The successful evaluation of the `try_me` expression discussed in section 4.2.1 indicates that the interpreter is performing lazy evaluation:

```
const the_global_environment = setup_environment();
driver_loop(the_global_environment);

L-evaluate input:
function try_me(a, b) {
  return a === 0 ? 1 : b;
}

L-evaluate value:
undefined

L-evaluate input:
try_me(0, head(null));

L-evaluate value:
1
```

Representing thunks

Our evaluator must arrange to create thunks when functions are applied to arguments and to force these thunks later. A thunk must package an expression together with the environment, so that the argument can be produced later. To force the thunk, we simply extract the expression and environment from the thunk and evaluate the expression in the environment. We use `actual_value` rather than `evaluate` so that in case the value of the expression is itself a thunk, we will force that, and so on, until we reach something that is not a thunk:

```
function force_it(obj) {
    return is_thunk(obj)
           ? actual_value(thunk_exp(obj), thunk_env(obj))
           : obj;
}
```

One easy way to package an expression with an environment is to make a list containing the expression and the environment. Thus, we create a thunk as follows:

```
function delay_it(exp, env) {
    return list("thunk", exp, env);
}
function is_thunk(obj) {
    return is_tagged_list(obj, "thunk");
}
function thunk_exp(thunk) { return head(tail(thunk)); }
function thunk_env(thunk) { return head(tail(tail(thunk))); }
```

Actually, what we want for our interpreter is not quite this, but rather thunks that have been memoized. When a thunk is forced, we will turn it into an evaluated thunk by replacing the stored expression with its value and changing the `thunk` tag so that it can be recognized as already evaluated.[34]

```
function is_evaluated_thunk(obj) {
    return is_tagged_list(obj, "evaluated_thunk");
}
function thunk_value(evaluated_thunk) {
    return head(tail(evaluated_thunk));
}
```

---

34. Notice that we also erase the `env` from the thunk once the expression's value has been computed. This makes no difference in the values returned by the interpreter. It does help save space, however, because removing the reference from the thunk to the `env` once it is no longer needed allows this structure to be *garbage-collected* and its space recycled, as we will discuss in section 5.3.

Similarly, we could have allowed unneeded environments in the memoized delayed objects of section 3.5.1 to be garbage-collected, by having `memo` do something like `fun = null;` to discard the function `fun` (which includes the environment in which the lambda expression that makes up the tail of the stream was evaluated) after storing its value.

```
function force_it(obj) {
    if (is_thunk(obj)) {
        const result = actual_value(thunk_exp(obj), thunk_env(obj));
        set_head(obj, "evaluated_thunk");
        set_head(tail(obj), result);   // replace exp with its value
        set_tail(tail(obj), null);     // forget unneeded env
        return result;
    } else if (is_evaluated_thunk(obj)) {
        return thunk_value(obj);
    } else {
        return obj;
    }
}
```

Notice that the same `delay_it` function works both with and without memoization.

Exercise 4.25

Suppose we type in the following declarations to the lazy evaluator:

```
let count = 0;
function id(x) {
    count = count + 1;
    return x;
}
```

Give the missing values in the following sequence of interactions, and explain your answers.[35]

```
const w = id(id(10));

L-evaluate input:
count;

L-evaluate value:
⟨response⟩

L-evaluate input:
w;

L-evaluate value:
⟨response⟩

L-evaluate input:
count;

L-evaluate value:
⟨response⟩
```

35. This exercise demonstrates that the interaction between lazy evaluation and side effects can be very confusing. This is just what you might expect from the discussion in chapter 3.

Exercise 4.26

The function `evaluate` uses `actual_value` rather than `evaluate` to evaluate the function expression before passing it to `apply`, in order to force the value of the function expression. Give an example that demonstrates the need for this forcing.

Exercise 4.27

Exhibit a program that you would expect to run much more slowly without memoization than with memoization. Also, consider the following interaction, where the `id` function is defined as in exercise 4.25 and `count` starts at 0:

```
function square(x) {
    return x * x;
}

L-evaluate input:
square(id(10));

L-evaluate value:
⟨response⟩

L-evaluate input:
count;

L-evaluate value:
⟨response⟩
```

Give the responses both when the evaluator memoizes and when it does not.

Exercise 4.28

Cy D. Fect, a reformed C programmer, is worried that some side effects may never take place, because the lazy evaluator doesn't force the statements in a sequence. Since the value of a statement in a sequence may not be used (the statement may be there only for its effect, such as assigning to a variable or printing), there may be no subsequent use of this value (e.g., as an argument to a primitive function) that will cause it to be forced. Cy thus thinks that when evaluating sequences, we must force all statements in the sequence. He proposes to modify `evaluate_sequence` from section 4.1.1 to use `actual_value` rather than `evaluate`:

```
function eval_sequence(stmts, env) {
    if (is_empty_sequence(stmts)) {
        return undefined;
    } else if (is_last_statement(stmts)) {
        return actual_value(first_statement(stmts), env);
    } else {
        const first_stmt_value =
            actual_value(first_statement(stmts), env);
        if (is_return_value(first_stmt_value)) {
            return first_stmt_value;
        } else {
            return eval_sequence(rest_statements(stmts), env);
        }
    }
}
```

a. Ben Bitdiddle thinks Cy is wrong. He shows Cy the `for_each` function described in exercise 2.23, which gives an important example of a sequence with side effects:

```
function for_each(fun, items) {
    if (is_null(items)){
        return "done";
    } else {
        fun(head(items));
        for_each(fun, tail(items));
    }
}
```

He claims that the evaluator in the text (with the original `eval_sequence`) handles this correctly:

```
L-evaluate input:
for_each(display, list(57, 321, 88));

57
321
88
L-evaluate value:
"done"
```

Explain why Ben is right about the behavior of `for_each`.

b. Cy agrees that Ben is right about the `for_each` example, but says that that's not the kind of program he was thinking about when he proposed his change to `eval_sequence`. He declares the following two functions in the lazy evaluator:

```
function f1(x) {
    x = pair(x, list(2));
    return x;
}

function f2(x) {
    function f(e) {
        e;
        return x;
    }
    return f(x = pair(x, list(2)));
}
```

What are the values of `f1(1)` and `f2(1)` with the original `eval_sequence`? What would the values be with Cy's proposed change to `eval_sequence`?

c. Cy also points out that changing `eval_sequence` as he proposes does not affect the behavior of the example in part a. Explain why this is true.

d. How do you think sequences ought to be treated in the lazy evaluator? Do you like Cy's approach, the approach in the text, or some other approach?

Exercise 4.29

The approach taken in this section is somewhat unpleasant, because it makes an incompatible change to JavaScript. It might be nicer to implement lazy evaluation as an *upward-compatible extension*, that is, so that ordinary JavaScript programs will work as before. We can do this by introducing optional parameter declaration as a new syntactic form inside function declarations to let the user control whether or not arguments are to be delayed. While we're at it, we may as well also give the user the choice between delaying with and without memoization. For example, the declaration

```
function f(a, b, c, d) {
    parameters("strict", "lazy", "strict", "lazy_memo");
    ...
}
```

would define `f` to be a function of four arguments, where the first and third arguments are evaluated when the function is called, the second argument is delayed, and the fourth argument is both delayed and memoized. You can assume that the parameter declaration is always the first statement in the body of a function declaration, and if it is omitted, all parameters are strict. Thus, ordinary function declaration will produce the same behavior as ordinary JavaScript, while adding the `"lazy_memo"` declaration to each parameter of every compound function will produce the behavior of the lazy evaluator defined in this section. Design and implement the changes required to produce such an extension to JavaScript. The `parse` function will treat parameter declarations as function applications, so you need to modify `apply` to dispatch to your implementation of the new syntactic form. You must also arrange for `evaluate` or `apply` to determine when arguments are to be delayed, and to force or delay arguments accordingly, and you must arrange for forcing to memoize or not, as appropriate.

4.2.3 Streams as Lazy Lists

In section 3.5.1, we showed how to implement streams as delayed lists. We used a lambda expression to construct a "promise" to compute the tail of a stream, without actually fulfilling that promise until later. We were forced to create streams as a new kind of data object similar but not identical to lists, and this required us to reimplement many ordinary list operations (`map`, `append`, and so on) for use with streams.

With lazy evaluation, streams and lists can be identical, so there is no need for separate list and stream operations. All we need to do is to arrange matters so that `pair` is non-strict. One way to accomplish this is to extend the lazy evaluator to allow for non-strict primitives, and to implement `pair` as one of these. An easier way is to recall (section 2.1.3) that there is no fundamental need to implement `pair` as a primitive at all. Instead, we can represent pairs as functions:[36]

---

36. This is the functional representation described in exercise 2.4. Essentially any functional representation (e.g., a message-passing implementation) would do as well. Notice that we can install these definitions in the lazy evaluator simply by typing them at the driver loop. If we had originally included `pair`, `head`, and `tail` as primitives in the global environment, they will be redefined. (Also see exercises 4.31 and 4.32.)

```
function pair(x, y) {
    return m => m(x, y);
}
function head(z) {
    return z((p, q) => p);
}
function tail(z) {
    return z((p, q) => q);
}
```

In terms of these basic operations, the standard definitions of the list operations will work with infinite lists (streams) as well as finite ones, and the stream operations can be implemented as list operations. Here are some examples:

```
function list_ref(items, n) {
    return n === 0
           ? head(items)
           : list_ref(tail(items), n - 1);
}
function map(fun, items) {
    return is_null(items)
           ? null
           : pair(fun(head(items)),
                  map(fun, tail(items)));
}
function scale_list(items, factor) {
    return map(x => x * factor, items);
}
function add_lists(list1, list2) {
    return is_null(list1)
           ? list2
           : is_null(list2)
           ? list1
           : pair(head(list1) + head(list2),
                  add_lists(tail(list1), tail(list2)));
}
const ones = pair(1, ones);
const integers = pair(1, add_lists(ones, integers));

L-evaluate input:
list_ref(integers, 17);

L-evaluate value:
18
```

Note that these lazy lists are even lazier than the streams of chapter 3: The head of the list, as well as the tail, is delayed.[37] In fact, even accessing the `head` or `tail`

37. This permits us to create delayed versions of more general kinds of list structures, not just sequences. Hughes 1990 discusses some applications of "lazy trees."

of a lazy pair need not force the value of a list element. The value will be forced only when it is really needed—e.g., for use as the argument of a primitive, or to be printed as an answer.

Lazy pairs also help with the problem that arose with streams in section 3.5.4, where we found that formulating stream models of systems with loops may require us to sprinkle our programs with additional lambda expressions for delays, beyond the ones required to construct a stream pair. With lazy evaluation, all arguments to functions are delayed uniformly. For instance, we can implement functions to integrate lists and solve differential equations as we originally intended in section 3.5.4:

```
function integral(integrand, initial_value, dt) {
    const int = pair(initial_value,
                     add_lists(scale_list(integrand, dt),
                               int));
    return int;
}
function solve(f, y0, dt) {
    const y = integral(dy, y0, dt);
    const dy = map(f, y);
    return y;
}

L-evaluate input:
list_ref(solve(x => x, 1, 0.001), 1000);

L-evaluate value:
2.716924
```

Exercise 4.30

Give some examples that illustrate the difference between the streams of chapter 3 and the "lazier" lazy lists described in this section. How can you take advantage of this extra laziness?

Exercise 4.31

Ben Bitdiddle tests the lazy list implementation given above by evaluating the expression

```
head(list("a", "b", "c"));
```

To his surprise, this produces an error. After some thought, he realizes that the "lists" obtained from the primitive `list` function are different from the lists manipulated by the new definitions of `pair`, `head`, and `tail`. Modify the evaluator such that applications of the primitive `list` function typed at the driver loop will produce true lazy lists.

Exercise 4.32

Modify the driver loop for the evaluator so that lazy pairs and lists will print in some reasonable way. (What are you going to do about infinite lists?) You may also need to modify the representation of lazy pairs so that the evaluator can identify them in order to print them.

4.3 Nondeterministic Computing

In this section, we extend the JavaScript evaluator to support a programming paradigm called *nondeterministic computing* by building into the evaluator a facility to support automatic search. This is a much more profound change to the language than the introduction of lazy evaluation in section 4.2.

Nondeterministic computing, like stream processing, is useful for "generate and test" applications. Consider the task of starting with two lists of positive integers and finding a pair of integers—one from the first list and one from the second list—whose sum is prime. We saw how to handle this with finite sequence operations in section 2.2.3 and with infinite streams in section 3.5.3. Our approach was to generate the sequence of all possible pairs and filter these to select the pairs whose sum is prime. Whether we actually generate the entire sequence of pairs first as in chapter 2, or interleave the generating and filtering as in chapter 3, is immaterial to the essential image of how the computation is organized.

The nondeterministic approach evokes a different image. Imagine simply that we choose (in some way) a number from the first list and a number from the second list and require (using some mechanism) that their sum be prime. This is expressed by the following function:

```
function prime_sum_pair(list1, list2) {
    const a = an_element_of(list1);
    const b = an_element_of(list2);
    require(is_prime(a + b));
    return list(a, b);
}
```

It might seem as if this function merely restates the problem, rather than specifying a way to solve it. Nevertheless, this is a legitimate nondeterministic program.[38]

The key idea here is that components in a nondeterministic language can have more than one possible value. For instance, `an_element_of` might return any element of the given list. Our nondeterministic program evaluator will work by automatically choosing a possible value and keeping track of the choice. If a subsequent requirement is not met, the evaluator will try a different choice, and it will keep trying new choices until the evaluation succeeds, or until we run out of choices. Just as the lazy evaluator freed the programmer from the details of how values are delayed and forced, the nondeterministic program evaluator will free the programmer from the details of how choices are made.

---

38. We assume that we have previously defined a function `is_prime` that tests whether numbers are prime. Even with `is_prime` defined, the `prime_sum_pair` function may look suspiciously like the unhelpful "pseudo-JavaScript" attempt to define the square-root function, which we described at the beginning of section 1.1.7. In fact, a square-root function along those lines can actually be formulated as a nondeterministic program. By incorporating a search mechanism into the evaluator, we are eroding the distinction between purely declarative descriptions and imperative specifications of how to compute answers. We'll go even farther in this direction in section 4.4.

It is instructive to contrast the different images of time evoked by nondeterministic evaluation and stream processing. Stream processing uses lazy evaluation to decouple the time when the stream of possible answers is assembled from the time when the actual stream elements are produced. The evaluator supports the illusion that all the possible answers are laid out before us in a timeless sequence. With nondeterministic evaluation, a component represents the exploration of a set of possible worlds, each determined by a set of choices. Some of the possible worlds lead to dead ends, while others have useful values. The nondeterministic program evaluator supports the illusion that time branches, and that our programs have different possible execution histories. When we reach a dead end, we can revisit a previous choice point and proceed along a different branch.

The nondeterministic program evaluator implemented below is called the `amb` evaluator because it is based on a new syntactic form called `amb`. We can type the above declaration of `prime_sum_pair` at the `amb` evaluator driver loop (along with declarations of `is_prime`, `an_element_of`, and `require`) and run the function as follows:

```
amb-evaluate input:
prime_sum_pair(list(1, 3, 5, 8), list(20, 35, 110));

Starting a new problem
amb-evaluate value:
[3, [20, null]]
```

The value returned was obtained after the evaluator repeatedly chose elements from each of the lists, until a successful choice was made.

Section 4.3.1 introduces `amb` and explains how it supports nondeterminism through the evaluator's automatic search mechanism. Section 4.3.2 presents examples of nondeterministic programs, and section 4.3.3 gives the details of how to implement the `amb` evaluator by modifying the ordinary JavaScript evaluator.

4.3.1 Search and `amb`

To extend JavaScript to support nondeterminism, we introduce a new syntactic form called `amb`.[39] The expression `amb(`e_1, $e_2, \ldots, e_n$`)` returns the value of one of the n expressions e_i "ambiguously." For example, the expression

```
list(amb(1, 2, 3), amb("a", "b"));
```

can have six possible values:

```
list(1, "a") list(1, "b") list(2, "a")
list(2, "b") list(3, "a") list(3, "b")
```

An `amb` expression with a single choice produces an ordinary (single) value.

An `amb` expression with no choices—the expression `amb()`—is an expression with no acceptable values. Operationally, we can think of `amb()` as an expression that when evaluated causes the computation to "fail": The computation aborts and no

39. The idea of `amb` for nondeterministic programming was first described in 1961 by John McCarthy (see McCarthy 1967).

value is produced. Using this idea, we can express the requirement that a particular predicate expression p must be true as follows:

```
function require(p) {
    if (! p) {
        amb();
    } else {}
}
```

With amb and require, we can implement the an_element_of function used above:

```
function an_element_of(items) {
    require(! is_null(items));
    return amb(head(items), an_element_of(tail(items)));
}
```

An application of an_element_of fails if the list is empty. Otherwise it ambiguously returns either the first element of the list or an element chosen from the rest of the list.

We can also express infinite ranges of choices. The following function potentially returns any integer greater than or equal to some given *n*:

```
function an_integer_starting_from(n) {
    return amb(n, an_integer_starting_from(n + 1));
}
```

This is like the stream function integers_starting_from described in section 3.5.2, but with an important difference: The stream function returns an object that represents the sequence of all integers beginning with *n*, whereas the amb function returns a single integer.[40]

Abstractly, we can imagine that evaluating an amb expression causes time to split into branches, where the computation continues on each branch with one of the possible values of the expression. We say that amb represents a *nondeterministic choice point*. If we had a machine with a sufficient number of processors that could be dynamically allocated, we could implement the search in a straightforward way. Execution would proceed as in a sequential machine, until an amb expression is encountered. At this point, more processors would be allocated and initialized to continue all of the parallel executions implied by the choice. Each processor would proceed sequentially as if it were the only choice, until it either terminates by encountering a failure, or it further subdivides, or it finishes.[41]

---

40. In actuality, the distinction between nondeterministically returning a single choice and returning all choices depends somewhat on our point of view. From the perspective of the code that uses the value, the nondeterministic choice returns a single value. From the perspective of the programmer designing the code, the nondeterministic choice potentially returns all possible values, and the computation branches so that each value is investigated separately.

41. One might object that this is a hopelessly inefficient mechanism. It might require millions of processors to solve some easily stated problem this way, and most of the time most of those processors would be idle. This objection should be taken in the context of history. Memory used to be considered just such an expensive commodity. In 1965 a megabyte of RAM cost about $400,000. Now every personal computer has many gigabytes of RAM, and most of the time most of that RAM is unused. It is hard to underestimate the cost of mass-produced electronics.

On the other hand, if we have a machine that can execute only one process (or a few concurrent processes), we must consider the alternatives sequentially. One could imagine modifying an evaluator to pick at random a branch to follow whenever it encounters a choice point. Random choice, however, can easily lead to failing values. We might try running the evaluator over and over, making random choices and hoping to find a non-failing value, but it is better to *systematically search* all possible execution paths. The `amb` evaluator that we will develop and work with in this section implements a systematic search as follows: When the evaluator encounters an application of `amb`, it initially selects the first alternative. This selection may itself lead to a further choice. The evaluator will always initially choose the first alternative at each choice point. If a choice results in a failure, then the evaluator automagically[42] *backtracks* to the most recent choice point and tries the next alternative. If it runs out of alternatives at any choice point, the evaluator will back up to the previous choice point and resume from there. This process leads to a search strategy known as *depth-first search* or *chronological backtracking*.[43]

---

42. Automagically: "Automatically, but in a way which, for some reason (typically because it is too complicated, or too ugly, or perhaps even too trivial), the speaker doesn't feel like explaining." (Steele 1983, Raymond 1996)

43. The integration of automatic search strategies into programming languages has had a long and checkered history. The first suggestions that nondeterministic algorithms might be elegantly encoded in a programming language with search and automatic backtracking came from Robert Floyd (1967). Carl Hewitt (1969) invented a programming language called Planner that explicitly supported automatic chronological backtracking, providing for a built-in depth-first search strategy. Sussman, Winograd, and Charniak (1971) implemented a subset of this language, called MicroPlanner, which was used to support work in problem solving and robot planning. Similar ideas, arising from logic and theorem proving, led to the genesis in Edinburgh and Marseille of the elegant language Prolog (which we will discuss in section 4.4). After sufficient frustration with automatic search, McDermott and Sussman (1972) developed a language called Conniver, which included mechanisms for placing the search strategy under programmer control. This proved unwieldy, however, and Sussman and Stallman (1975) found a more tractable approach while investigating methods of symbolic analysis for electrical circuits. They developed a nonchronological backtracking scheme that was based on tracing out the logical dependencies connecting facts, a technique that has come to be known as *dependency-directed backtracking*. Although their method was complex, it produced reasonably efficient programs because it did little redundant search. Doyle (1979) and McAllester (1978, 1980) generalized and clarified the methods of Stallman and Sussman, developing a new paradigm for formulating search that is now called *truth maintenance*. Many problem-solving systems use some form of truth-maintenance system as a substrate. See Forbus and de Kleer 1993 for a discussion of elegant ways to build truth-maintenance systems and applications using truth maintenance. Zabih, McAllester, and Chapman 1987 describes a nondeterministic extension to Scheme that is based on `amb`; it is similar to the interpreter described in this section, but more sophisticated, because it uses dependency-directed backtracking rather than chronological backtracking. Winston 1992 gives an introduction to both kinds of backtracking.

Driver loop

The driver loop for the amb evaluator has some unusual properties. It reads a program and prints the value of the first non-failing execution, as in the prime_sum_pair example shown above. If we want to see the value of the next successful execution, we can ask the interpreter to backtrack and attempt to generate a second non-failing execution. This is signaled by typing retry. If any other input except retry is given, the interpreter will start a new problem, discarding the unexplored alternatives in the previous problem. Here is a sample interaction:

```
amb-evaluate input:
prime_sum_pair(list(1, 3, 5, 8), list(20, 35, 110));

Starting a new problem
amb-evaluate value:
[3, [20, null]]

amb-evaluate input:
retry

amb-evaluate value:
[3, [110, null]]

amb-evaluate input:
retry

amb-evaluate value:
[8, [35, null]]

amb-evaluate input:
retry

There are no more values of
prime_sum_pair([1, [3, [5, [8, null]]]], [20, [35, [110, null]]])

amb-evaluate input:
prime_sum_pair(list(19, 27, 30), list(11, 36, 58));

Starting a new problem
amb-evaluate value:
[30, [11, null]]
```

Exercise 4.33

Write a function an_integer_between that returns an integer between two given bounds. This can be used to implement a function that finds Pythagorean triples, i.e., triples of integers (i, j, k) between the given bounds such that $i \leq j$ and $i^2 + j^2 = k^2$, as follows:

```
function a_pythogorean_triple_between(low, high) {
    const i = an_integer_between(low, high);
    const j = an_integer_between(i, high);
    const k = an_integer_between(j, high);
    require(i * i + j * j === k * k);
    return list(i, j, k);
}
```

Exercise 4.34

Exercise 3.69 discussed how to generate the stream of *all* Pythagorean triples, with no upper bound on the size of the integers to be searched. Explain why simply replacing `an_integer_between` by `an_integer_starting_from` in the function in exercise 4.33 is not an adequate way to generate arbitrary Pythagorean triples. Write a function that actually will accomplish this. (That is, write a function for which repeatedly typing `retry` would in principle eventually generate all Pythagorean triples.)

Exercise 4.35

Ben Bitdiddle claims that the following method for generating Pythagorean triples is more efficient than the one in exercise 4.33. Is he correct? (Hint: Consider the number of possibilities that must be explored.)

```
function a_pythagorean_triple_between(low, high) {
    const i = an_integer_between(low, high);
    const hsq = high * high;
    const j = an_integer_between(i, high);
    const ksq = i * i + j * j;
    require(hsq >= ksq);
    const k = math_sqrt(ksq);
    require(is_integer(k));
    return list(i, j, k);
}
```

4.3.2 Examples of Nondeterministic Programs

Section 4.3.3 describes the implementation of the `amb` evaluator. First, however, we give some examples of how it can be used. The advantage of nondeterministic programming is that we can suppress the details of how search is carried out, thereby expressing our programs at a higher level of abstraction.

Logic Puzzles

The following puzzle (adapted from Dinesman 1968) is typical of a large class of simple logic puzzles:

> The software company Gargle is expanding, and Alyssa, Ben, Cy, Lem, and Louis are moving into a row of five private offices in a new building. Alyssa does not move into the last office. Ben does not move into the first office. Cy takes neither the first nor the last office. Lem moves into an office after Ben's. Louis's office is not next to Cy's. Cy's office is not next to Ben's. Who moves into which office?

We can determine who moves into which office in a straightforward way by enumerating all the possibilities and imposing the given restrictions:[44]

```
function office_move() {
    const alyssa = amb(1, 2, 3, 4, 5);
    const ben = amb(1, 2, 3, 4, 5);
    const cy = amb(1, 2, 3, 4, 5);
    const lem = amb(1, 2, 3, 4, 5);
    const louis = amb(1, 2, 3, 4, 5);
    require(distinct(list(alyssa, ben, cy, lem, louis)));
    require(alyssa !== 5);
    require(ben !== 1);
    require(cy !== 5);
    require(cy !== 1);
    require(lem > ben);
    require(math_abs(louis - cy) !== 1);
    require(math_abs(cy - ben) !== 1);
    return list(list("alyssa", alyssa),
                list("ben", ben),
                list("cy", cy),
                list("lem", lem),
                list("louis", louis));
}
```

Evaluating the expression `office_move()` produces the result

```
list(list("alyssa", 3), list("ben", 2), list("cy", 4),
     list("lem", 5), list("louis", 1))
```

Although this simple function works, it is very slow. Exercises 4.37 and 4.38 discuss some possible improvements.

Exercise 4.36

Modify the office-move function to omit the requirement that Louis's office is not next to Cy's. How many solutions are there to this modified puzzle?

Exercise 4.37

Does the order of the restrictions in the office-move function affect the answer? Does it affect the time to find an answer? If you think it matters, demonstrate a faster program obtained from the given one by reordering the restrictions. If you think it does not matter, argue your case.

---

44. Our program uses the following function to determine if the elements of a list are distinct:

```
function distinct(items) {
    return is_null(items)
           ? true
           : is_null(tail(items))
           ? true
           : is_null(member(head(items), tail(items)))
           ? distinct(tail(items))
           : false;
}
```

Exercise 4.38

In the office move problem, how many sets of assignments are there of people to offices, both before and after the requirement that office assignments be distinct? It is very inefficient to generate all possible assignments of people to offices and then leave it to backtracking to eliminate them. For example, most of the restrictions depend on only one or two of the person-office names, and can thus be imposed before offices have been selected for all the people. Write and demonstrate a much more efficient nondeterministic function that solves this problem based upon generating only those possibilities that are not already ruled out by previous restrictions.

Exercise 4.39

Write an ordinary JavaScript program to solve the office move puzzle.

Exercise 4.40

Solve the following "Liars" puzzle (adapted from Phillips 1934):

> Alyssa, Cy, Eva, Lem, and Louis meet for a business lunch at SoSoService. Their meals arrive one after the other, a considerable time after they placed their orders. To entertain Ben, who expects them back at the office for a meeting, they decide to each make one true statement and one false statement about their orders:
>
> - Alyssa: "Lem's meal arrived second. Mine arrived third."
> - Cy: "Mine arrived first. Eva's arrived second."
> - Eva: "Mine arrived third, and poor Cy's arrived last."
> - Lem: "Mine arrived second. Louis's arrived fourth."
> - Louis: "Mine arrived fourth. Alyssa's meal arrived first."
>
> What was the real order in which the five diners received their meals?

Exercise 4.41

Use the `amb` evaluator to solve the following puzzle (adapted from Phillips 1961):

> Alyssa, Ben, Cy, Eva, and Louis each pick a different chapter of SICP JS and solve all the exercises in that chapter. Louis solves the exercises in the "Functions" chapter, Alyssa the ones in the "Data" chapter, and Cy the ones in the "State" chapter. They decide to check each other's work, and Alyssa volunteers to check the exercises in the "Meta" chapter. The exercises in the "Register Machines" chapter are solved by Ben and checked by Louis. The person who checks the exercises in the "Functions" chapter solves the exercises that are checked by Eva. Who checks the exercises in the "Data" chapter?

Try to write the program so that it runs efficiently (see exercise 4.38). Also determine how many solutions there are if we are not told that Alyssa checks the exercises in the "Meta" chapter.

Exercise 4.42

Exercise 2.42 described the "eight-queens puzzle" of placing queens on a chessboard so that no two attack each other. Write a nondeterministic program to solve this puzzle.

Parsing natural language

Programs designed to accept natural language as input usually start by attempting to *parse* the input, that is, to match the input against some grammatical structure. For example, we might try to recognize simple sentences consisting of an article followed by a noun followed by a verb, such as "The cat eats." To accomplish such an analysis, we must be able to identify the parts of speech of individual words. We could start with some lists that classify various words:[45]

```
const nouns = list("noun", "student", "professor", "cat", "class");

const verbs = list("verb", "studies", "lectures", "eats", "sleeps");

const articles = list("article", "the", "a");
```

We also need a *grammar*, that is, a set of rules describing how grammatical elements are composed from simpler elements. A very simple grammar might stipulate that a sentence always consists of two pieces—a noun phrase followed by a verb—and that a noun phrase consists of an article followed by a noun. With this grammar, the sentence "The cat eats" is parsed as follows:

```
list("sentence",
     list("noun-phrase", list("article", "the"), list("noun", "cat"),
     list("verb", "eats"))
```

We can generate such a parse with a simple program that has separate functions for each of the grammatical rules. To parse a sentence, we identify its two constituent pieces and return a list of these two elements, tagged with the symbol `sentence`:

```
function parse_sentence() {
    return list("sentence",
                parse_noun_phrase(),
                parse_word(verbs));
}
```

A noun phrase, similarly, is parsed by finding an article followed by a noun:

```
function parse_noun_phrase() {
    return list("noun-phrase",
                parse_word(articles),
                parse_word(nouns));
}
```

At the lowest level, parsing boils down to repeatedly checking that the next not-yet-parsed word is a member of the list of words for the required part of speech. To implement this, we maintain a global variable `not_yet_parsed`, which is the input that has not yet been parsed. Each time we check a word, we require that `not_yet_parsed` must be nonempty and that it should begin with a word from the

45. Here we use the convention that the first element of each list designates the part of speech for the rest of the words in the list.

designated list. If so, we remove that word from `not_yet_parsed` and return the word together with its part of speech (which is found at the head of the list):[46]

```
function parse_word(word_list) {
    require(! is_null(not_yet_parsed));
    require(! is_null(member(head(not_yet_parsed), tail(word_list))));
    const found_word = head(not_yet_parsed);
    not_yet_parsed = tail(not_yet_parsed);
    return list(head(word_list), found_word);
}
```

To start the parsing, all we need to do is set `not_yet_parsed` to be the entire input, try to parse a sentence, and check that nothing is left over:

```
let not_yet_parsed = null;

function parse_input(input) {
    not_yet_parsed = input;
    const sent = parse_sentence();
    require(is_null(not_yet_parsed));
    return sent;
}
```

We can now try the parser and verify that it works for our simple test sentence:

```
amb-evaluate input:
parse_input(list("the",  "cat",  "eats"));

Starting a new problem
amb-evaluate value:
list("sentence",
     list("noun-phrase", list("article", "the"), list("noun", "cat")),
     list("verb", "eats"))
```

The `amb` evaluator is useful here because it is convenient to express the parsing constraints with the aid of `require`. Automatic search and backtracking really pay off, however, when we consider more complex grammars where there are choices for how the units can be decomposed.

Let's add to our grammar a list of prepositions:

```
const prepositions = list("prep", "for", "to", "in", "by", "with");
```

and define a prepositional phrase (e.g., "for the cat") to be a preposition followed by a noun phrase:

```
function parse_prepositional_phrase() {
    return list("prep-phrase",
                parse_word(prepositions),
                parse_noun_phrase());
}
```

46. Notice that `parse_word` uses assignment to modify the not-yet-parsed input list. For this to work, our `amb` evaluator must undo the effects of assignments when it backtracks.

Now we can define a sentence to be a noun phrase followed by a verb phrase, where a verb phrase can be either a verb or a verb phrase extended by a prepositional phrase:[47]

```
function parse_sentence() {
    return list("sentence",
                parse_noun_phrase(),
                parse_verb_phrase());
}
function parse_verb_phrase() {
    function maybe_extend(verb_phrase) {
        return amb(verb_phrase,
                   maybe_extend(list("verb-phrase",
                                     verb_phrase,
                                     parse_prepositional_phrase())));
    }
    return maybe_extend(parse_word(verbs));
}
```

While we're at it, we can also elaborate the definition of noun phrases to permit such things as "a cat in the class." What we used to call a noun phrase, we'll now call a simple noun phrase, and a noun phrase will now be either a simple noun phrase or a noun phrase extended by a prepositional phrase:

```
function parse_simple_noun_phrase() {
    return list("simple-noun-phrase",
                parse_word(articles),
                parse_word(nouns));
}
function parse_noun_phrase() {
    function maybe_extend(noun_phrase) {
        return amb(noun_phrase,
                   maybe_extend(list("noun-phrase",
                                     noun_phrase,
                                     parse_prepositional_phrase())));
    }
    return maybe_extend(parse_simple_noun_phrase());
}
```

Our new grammar lets us parse more complex sentences. For example

```
parse_input(list("the", "student", "with", "the", "cat",
                 "sleeps", "in", "the", "class"));
```

produces

47. Observe that this definition is recursive—a verb may be followed by any number of prepositional phrases.

```
list("sentence",
     list("noun-phrase",
          list("simple-noun-phrase",
               list("article", "the"), list("noun", "student")),
          list("prep-phrase", list("prep", "with"),
               list("simple-noun-phrase",
                    list("article", "the"),
                    list("noun", "cat")))),
     list("verb-phrase",
          list("verb", "sleeps"),
          list("prep-phrase", list("prep", "in"),
               list("simple-noun-phrase",
                    list("article", "the"),
                    list("noun", "class")))))
```

Observe that a given input may have more than one legal parse. In the sentence "The professor lectures to the student with the cat," it may be that the professor is lecturing with the cat, or that the student has the cat. Our nondeterministic program finds both possibilities:

```
parse_input(list("the", "professor", "lectures",
                 "to", "the", "student", "with", "the", "cat"));
```

produces

```
list("sentence",
     list("simple-noun-phrase",
          list("article", "the"), list("noun", "professor")),
     list("verb-phrase",
          list("verb-phrase",
               list("verb", "lectures"),
               list("prep-phrase", list("prep", "to"),
                    list("simple-noun-phrase",
                    list("article", "the"),
                    list("noun", "student")))),
          list("prep-phrase", list("prep", "with"),
               list("simple-noun-phrase",
                    list("article", "the"),
                    list("noun", "cat")))))
```

Asking the evaluator to retry yields

```
list("sentence",
     list("simple-noun-phrase",
          list("article", "the"), list("noun", "professor")),
     list("verb-phrase",
          list("verb", "lectures"),
          list("prep-phrase", list("prep", "to"),
               list("noun-phrase",
                    list("simple-noun-phrase",
                         list("article", "the"),
                         list("noun", "student")),
                    list("prep-phrase", list("prep", "with"),
                         list("simple-noun-phrase",
                              list("article", "the"),
                              list("noun", "cat")))))))
```

Exercise 4.43

With the grammar given above, the following sentence can be parsed in five different ways: "The professor lectures to the student in the class with the cat." Give the five parses and explain the differences in shades of meaning among them.

Exercise 4.44

The evaluators in sections 4.1 and 4.2 do not determine what order argument expressions are evaluated in. We will see that the `amb` evaluator evaluates them from left to right. Explain why our parsing program wouldn't work if the argument expressions were evaluated in some other order.

Exercise 4.45

Louis Reasoner suggests that, since a verb phrase is either a verb or a verb phrase followed by a prepositional phrase, it would be much more straightforward to declare the function `parse_verb_phrase` as follows (and similarly for noun phrases):

```
function parse_verb_phrase() {
    return amb(parse_word(verbs),
               list("verb-phrase",
                    parse_verb_phrase(),
                    parse_prepositional_phrase()));
}
```

Does this work? Does the program's behavior change if we interchange the order of expressions in the `amb`?

Exercise 4.46

Extend the grammar given above to handle more complex sentences. For example, you could extend noun phrases and verb phrases to include adjectives and adverbs, or you could handle compound sentences.[48]

Exercise 4.47

Alyssa P. Hacker is more interested in generating interesting sentences than in parsing them. She reasons that by simply changing the function `parse_word` so that it ignores the "input sentence" and instead always succeeds and generates an appropriate word, we can use the programs we had built for parsing to do generation instead. Implement Alyssa's idea, and show the first half-dozen or so sentences generated.[49]

---

48. This kind of grammar can become arbitrarily complex, but it is only a toy as far as real language understanding is concerned. Real natural-language understanding by computer requires an elaborate mixture of syntactic analysis and interpretation of meaning. On the other hand, even toy parsers can be useful in supporting flexible command languages for programs such as information-retrieval systems. Winston 1992 discusses computational approaches to real language understanding and also the applications of simple grammars to command languages.

49. Although Alyssa's idea works just fine (and is surprisingly simple), the sentences that it generates are a bit boring—they don't sample the possible sentences of this language in a very interesting way. In fact, the grammar is highly recursive in many places, and Alyssa's technique "falls into" one of these recursions and gets stuck. See exercise 4.48 for a way to deal with this.

4.3.3 Implementing the `amb` Evaluator

The evaluation of an ordinary JavaScript program may return a value, may never terminate, or may signal an error. In nondeterministic JavaScript the evaluation of a program may in addition result in the discovery of a dead end, in which case evaluation must backtrack to a previous choice point. The interpretation of nondeterministic JavaScript is complicated by this extra case.

We will construct the `amb` evaluator for nondeterministic JavaScript by modifying the analyzing evaluator of section 4.1.7.[50] As in the analyzing evaluator, evaluation of a component is accomplished by calling an execution function produced by analysis of that component. The difference between the interpretation of ordinary JavaScript and the interpretation of nondeterministic JavaScript will be entirely in the execution functions.

Execution functions and continuations

Recall that the execution functions for the ordinary evaluator take one argument: the environment of execution. In contrast, the execution functions in the `amb` evaluator take three arguments: the environment, and two functions called *continuation functions*. The evaluation of a component will finish by calling one of these two continuations: If the evaluation results in a value, the *success continuation* is called with that value; if the evaluation results in the discovery of a dead end, the *failure continuation* is called. Constructing and calling appropriate continuations is the mechanism by which the nondeterministic evaluator implements backtracking.

It is the job of the success continuation to receive a value and proceed with the computation. Along with that value, the success continuation is passed another failure continuation, which is to be called subsequently if the use of that value leads to a dead end.

It is the job of the failure continuation to try another branch of the nondeterministic process. The essence of the nondeterministic language is in the fact that components may represent choices among alternatives. The evaluation of such a component must proceed with one of the indicated alternative choices, even though it is not known in advance which choices will lead to acceptable results. To deal with this, the evaluator picks one of the alternatives and passes this value to the success continuation. Together with this value, the evaluator constructs and passes along a failure continuation that can be called later to choose a different alternative.

A failure is triggered during evaluation (that is, a failure continuation is called) when a user program explicitly rejects the current line of attack (for example, a call to `require` may result in execution of `amb()`, an expression that always fails—see

---

50. We chose to implement the lazy evaluator in section 4.2 as a modification of the ordinary metacircular evaluator of section 4.1.1. In contrast, we will base the `amb` evaluator on the analyzing evaluator of section 4.1.7, because the execution functions in that evaluator provide a convenient framework for implementing backtracking.

section 4.3.1). The failure continuation in hand at that point will cause the most recent choice point to choose another alternative. If there are no more alternatives to be considered at that choice point, a failure at an earlier choice point is triggered, and so on. Failure continuations are also invoked by the driver loop in response to a `retry` request, to find another value of the program.

In addition, if a side-effect operation (such as assignment to a variable) occurs on a branch of the process resulting from a choice, it may be necessary, when the process finds a dead end, to undo the side effect before making a new choice. This is accomplished by having the side-effect operation produce a failure continuation that undoes the side effect and propagates the failure.

In summary, failure continuations are constructed by

- `amb` expressions—to provide a mechanism to make alternative choices if the current choice made by the `amb` expression leads to a dead end;
- the top-level driver—to provide a mechanism to report failure when the choices are exhausted;
- assignments—to intercept failures and undo assignments during backtracking.

Failures are initiated only when a dead end is encountered. This occurs

- if the user program executes `amb()`;
- if the user types `retry` at the top-level driver.

Failure continuations are also called during processing of a failure:

- When the failure continuation created by an assignment finishes undoing a side effect, it calls the failure continuation it intercepted, in order to propagate the failure back to the choice point that led to this assignment or to the top level.
- When the failure continuation for an `amb` runs out of choices, it calls the failure continuation that was originally given to the `amb`, in order to propagate the failure back to the previous choice point or to the top level.

Structure of the evaluator

The syntax- and data-representation functions for the `amb` evaluator, and also the basic `analyze` function, are identical to those in the evaluator of section 4.1.7, except for the fact that we need additional syntax functions to recognize the `amb` syntactic form:

```
function is_amb(component) {
    return is_tagged_list(component, "application") &&
           is_name(function_expression(component)) &&
           symbol_of_name(function_expression(component)) === "amb";
}
function amb_choices(component) {
    return arg_expressions(component);
}
```

We continue to use the parse function of section 4.1.2, which doesn't support `amb` as a syntactic form and instead treats `amb(...)` as a function application. The function `is_amb` ensures that whenever the name `amb` appears as the function expression of an application, the evaluator treats the "application" as a nondeterministic choice point.[51]

We must also add to the dispatch in `analyze` a clause that will recognize such expressions and generate an appropriate execution function:

```
...
: is_amb(component)
? analyze_amb(component)
: is_application(component)
...
```

The top-level function `ambeval` (similar to the version of `evaluate` given in section 4.1.7) analyzes the given component and applies the resulting execution function to the given environment, together with two given continuations:

```
function ambeval(component, env, succeed, fail) {
    return analyze(component)(env, succeed, fail);
}
```

A success continuation is a function of two arguments: the value just obtained and another failure continuation to be used if that value leads to a subsequent failure. A failure continuation is a function of no arguments. So the general form of an execution function is

```
(env, succeed, fail) => {
    // succeed is (value, fail) => ...
    // fail is () => ...
    ...
}
```

For example, executing

```
ambeval(component,
        the_global_environment,
        (value, fail) => value,
        () => "failed");
```

will attempt to evaluate the given component and will return either the component's value (if the evaluation succeeds) or the string `"failed"` (if the evaluation fails). The call to `ambeval` in the driver loop shown below uses much more complicated continuation functions, which continue the loop and support the `retry` request.

Most of the complexity of the `amb` evaluator results from the mechanics of passing the continuations around as the execution functions call each other. In going through the following code, you should compare each of the execution functions with the corresponding function for the ordinary evaluator given in section 4.1.7.

---

51. With this treatment, `amb` is no longer a name with proper scoping. To avoid confusion, we must refrain from declaring `amb` as a name in our nondeterministic programs.

Simple expressions

The execution functions for the simplest kinds of expressions are essentially the same as those for the ordinary evaluator, except for the need to manage the continuations. The execution functions simply succeed with the value of the expression, passing along the failure continuation that was passed to them.

```
function analyze_literal(component) {
    return (env, succeed, fail) =>
             succeed(literal_value(component), fail);
}

function analyze_name(component) {
    return (env, succeed, fail) =>
             succeed(lookup_symbol_value(symbol_of_name(component),
                                         env),
                     fail);
}

function analyze_lambda_expression(component) {
    const params = lambda_parameter_symbols(component);
    const bfun = analyze(lambda_body(component));
    return (env, succeed, fail) =>
             succeed(make_function(params, bfun, env),
                     fail);
}
```

Notice that looking up a name always "succeeds." If `lookup_symbol_value` fails to find the name, it signals an error, as usual. Such a "failure" indicates a program bug—a reference to an unbound name; it is not an indication that we should try another nondeterministic choice instead of the one that is currently being tried.

Conditionals and sequences

Conditionals are also handled in a similar way as in the ordinary evaluator. The execution function generated by `analyze_conditional` invokes the predicate execution function `pfun` with a success continuation that checks whether the predicate value is true and goes on to execute either the consequent or the alternative. If the execution of `pfun` fails, the original failure continuation for the conditional expression is called.

```
function analyze_conditional(component) {
    const pfun = analyze(conditional_predicate(component));
    const cfun = analyze(conditional_consequent(component));
    const afun = analyze(conditional_alternative(component));
    return (env, succeed, fail) =>
             pfun(env,
                  // success continuation for evaluating the predicate
                  // to obtain pred_value
                  (pred_value, fail2) =>
                    is_truthy(pred_value)
                    ? cfun(env, succeed, fail2)
                    : afun(env, succeed, fail2),
                  // failure continuation for evaluating the predicate
                  fail);
}
```

Sequences are also handled in the same way as in the previous evaluator, except for the machinations in the subfunction `sequentially` that are required for passing the continuations. Namely, to sequentially execute `a` and then `b`, we call `a` with a success continuation that calls `b`.

```
function analyze_sequence(stmts) {
    function sequentially(a, b) {
        return (env, succeed, fail) =>
                 a(env,
                   // success continuation for calling a
                   (a_value, fail2) =>
                     is_return_value(a_value)
                     ? succeed(a_value, fail2)
                     : b(env, succeed, fail2),
                   // failure continuation for calling a
                   fail);
    }
    function loop(first_fun, rest_funs) {
        return is_null(rest_funs)
               ? first_fun
               : loop(sequentially(first_fun, head(rest_funs)),
                      tail(rest_funs));
    }
    const funs = map(analyze, stmts);
    return is_null(funs)
           ? env => undefined
           : loop(head(funs), tail(funs));
}
```

Declarations and assignments

Declarations are another case where we must go to some trouble to manage the continuations, because it is necessary to evaluate the declaration-value expression before actually declaring the new name. To accomplish this, the declaration-value execution function `vfun` is called with the environment, a success continuation, and the failure continuation. If the execution of `vfun` succeeds, obtaining a value `val` for the declared name, the name is declared and the success is propagated:

```
function analyze_declaration(component) {
    const symbol = declaration_symbol(component);
    const vfun = analyze(declaration_value_expression(component));
    return (env, succeed, fail) =>
             vfun(env,
                  (val, fail2) => {
                      assign_symbol_value(symbol, val, env);
                      return succeed(undefined, fail2);
                  },
                  fail);
}
```

Assignments are more interesting. This is the first place where we really use the continuations, rather than just passing them around. The execution function for assignments starts out like the one for declarations. It first attempts to obtain the new

value to be assigned to the name. If this evaluation of vfun fails, the assignment fails.

If vfun succeeds, however, and we go on to make the assignment, we must consider the possibility that this branch of the computation might later fail, which will require us to backtrack out of the assignment. Thus, we must arrange to undo the assignment as part of the backtracking process.[52]

This is accomplished by giving vfun a success continuation (marked with the comment "*1*" below) that saves the old value of the variable before assigning the new value to the variable and proceeding from the assignment. The failure continuation that is passed along with the value of the assignment (marked with the comment "*2*" below) restores the old value of the variable before continuing the failure. That is, a successful assignment provides a failure continuation that will intercept a subsequent failure; whatever failure would otherwise have called fail2 calls this function instead, to undo the assignment before actually calling fail2.

```
function analyze_assignment(component) {
    const symbol = assignment_symbol(component);
    const vfun = analyze(assignment_value_expression(component));
    return (env, succeed, fail) =>
             vfun(env,
                  (val, fail2) => {              // *1*
                      const old_value = lookup_symbol_value(symbol,
                                                             env);
                      assign_symbol_value(symbol, val, env);
                      return succeed(val,
                                     () => {     // *2*
                                         assign_symbol_value(symbol,
                                                             old_value,
                                                             env);
                                         return fail2();
                                     });
                  },
                  fail);
}
```

Return statements and blocks

Analyzing return statements is straightforward. The return expression is analyzed to produce an execution function. The execution function for the return statement calls that execution function with a success continuation that wraps the return value in a return value object and passes it to the original success continuation.

---

52. We didn't worry about undoing declarations, since we assume that a name can't be used prior to the evaluation of its declaration, so its previous value doesn't matter.

```
function analyze_return_statement(component) {
    const rfun = analyze(return_expression(component));
    return (env, succeed, fail) =>
             rfun(env,
                  (val, fail2) =>
                    succeed(make_return_value(val), fail2),
                  fail);
}
```

The execution function for blocks calls the body's execution function on an extended environment, without changing success or failure continuations.

```
function analyze_block(component) {
    const body = block_body(component);
    const locals = scan_out_declarations(body);
    const unassigneds = list_of_unassigned(locals);
    const bfun = analyze(body);
    return (env, succeed, fail) =>
             bfun(extend_environment(locals, unassigneds, env),
                  succeed,
                  fail);
}
```

Function applications

The execution function for applications contains no new ideas except for the technical complexity of managing the continuations. This complexity arises in `analyze_application`, due to the need to keep track of the success and failure continuations as we evaluate the argument expressions. We use a function `get_args` to evaluate the list of argument expressions, rather than a simple `map` as in the ordinary evaluator.

```
function analyze_application(component) {
    const ffun = analyze(function_expression(component));
    const afuns = map(analyze, arg_expressions(component));
    return (env, succeed, fail) =>
             ffun(env,
                  (fun, fail2) =>
                    get_args(afuns,
                             env,
                             (args, fail3) =>
                               execute_application(fun,
                                                   args,
                                                   succeed,
                                                   fail3),
                             fail2),
                  fail);
}
```

In `get_args`, notice how walking down the list of `afun` execution functions and constructing the resulting list of `args` is accomplished by calling each `afun` in the list with a success continuation that recursively calls `get_args`. Each of these recursive

calls to get_args has a success continuation whose value is the new list resulting from using pair to adjoin the newly obtained argument to the list of accumulated arguments:

```
function get_args(afuns, env, succeed, fail) {
    return is_null(afuns)
           ? succeed(null, fail)
           : head(afuns)(env,
                         // success continuation for this afun
                         (arg, fail2) =>
                           get_args(tail(afuns),
                                    env,
                                    // success continuation for
                                    // recursive call to get_args
                                    (args, fail3) =>
                                      succeed(pair(arg, args),
                                              fail3),
                                    fail2),
                         fail);
}
```

The actual function application, which is performed by execute_application, is accomplished in the same way as for the ordinary evaluator, except for the need to manage the continuations.

```
function execute_application(fun, args, succeed, fail) {
    return is_primitive_function(fun)
           ? succeed(apply_primitive_function(fun, args),
                     fail)
           : is_compound_function(fun)
           ? function_body(fun)(
                 extend_environment(function_parameters(fun),
                                    args,
                                    function_environment(fun)),
                 (body_result, fail2) =>
                   succeed(is_return_value(body_result)
                           ? return_value_content(body_result)
                           : undefined,
                           fail2),
                 fail)
           : error(fun, "unknown function type - execute_application");
}
```

Evaluating `amb` expressions

The `amb` syntactic form is the key element in the nondeterministic language. Here we see the essence of the interpretation process and the reason for keeping track of the continuations. The execution function for `amb` defines a loop `try_next` that cycles through the execution functions for all the possible values of the `amb` expression. Each execution function is called with a failure continuation that will try the next one. When there are no more alternatives to try, the entire `amb` expression fails.

```
function analyze_amb(component) {
    const cfuns = map(analyze, amb_choices(component));
    return (env, succeed, fail) => {
               function try_next(choices) {
                   return is_null(choices)
                          ? fail()
                          : head(choices)(env,
                                          succeed,
                                          () =>
                                            try_next(tail(choices)));
               }
               return try_next(cfuns);
           };
}
```

Driver loop

The driver loop for the `amb` evaluator is complex, due to the mechanism that permits the user to retry in evaluating a program. The driver uses a function called `internal_loop`, which takes as argument a function `retry`. The intent is that calling `retry` should go on to the next untried alternative in the nondeterministic evaluation. The function `internal_loop` either calls `retry` in response to the user typing `retry` at the driver loop, or else starts a new evaluation by calling `ambeval`.

The failure continuation for this call to `ambeval` informs the user that there are no more values and reinvokes the driver loop.

The success continuation for the call to `ambeval` is more subtle. We print the obtained value and then reinvoke the internal loop with a `retry` function that will be able to try the next alternative. This `next_alternative` function is the second argument that was passed to the success continuation. Ordinarily, we think of this second argument as a failure continuation to be used if the current evaluation branch later fails. In this case, however, we have completed a successful evaluation, so we can invoke the "failure" alternative branch in order to search for additional successful evaluations.

```
const input_prompt = "amb-evaluate input:";
const output_prompt =  "amb-evaluate value:";

function driver_loop(env) {
    function internal_loop(retry) {
        const input = user_read(input_prompt);
        if (is_null(input)) {
            display("evaluator terminated");
        } else if (input === "retry") {
            return retry();
        } else {
            display("Starting a new problem");
            const program = parse(input);
            const locals = scan_out_declarations(program);
            const unassigneds = list_of_unassigned(locals);
            const program_env = extend_environment(
                                    locals, unassigneds, env);
            return ambeval(
                       program,
                       program_env,
                       // ambeval success
                       (val, next_alternative) => {
                           user_print(output_prompt, val);
                           return internal_loop(next_alternative);
                       },
                       // ambeval failure
                       () => {
                           display("There are no more values of");
                           display(input);
                           return driver_loop(program_env);
                       });
        }
    }
    return internal_loop(() => {
                             display("There is no current problem");
                             return driver_loop(env);
                         });
}
```

The initial call to `internal_loop` uses a `retry` function that complains that there is no current problem and restarts the driver loop. This is the behavior that will happen if the user types `retry` when there is no evaluation in progress.

We start the driver loop as usual, by setting up the global environment and passing it as the enclosing environment for the first iteration of `driver_loop`.

```
const the_global_environment = setup_environment();
driver_loop(the_global_environment);
```

Exercise 4.48

Implement a new syntactic form `ramb` that is like `amb` except that it searches alternatives in a random order, rather than from left to right. Show how this can help with Alyssa's problem in exercise 4.47.

Exercise 4.49

Change the implementation of assignment so that it is not undone upon failure. For example, we can choose two distinct elements from a list and count the number of trials required to make a successful choice as follows:

```
let count = 0;

let x = an_element_of("a", "b", "c");
let y = an_element_of("a", "b", "c");
count = count + 1;
require(x !== y);
list(x, y, count);
Starting a new problem
amb-evaluate value:
["a", ["b", [2, null]]]

amb-evaluate input:
retry

amb-evaluate value:
["a", ["c", [3, null]]]
```

What values would have been displayed if we had used the original meaning of assignment rather than permanent assignment?

Exercise 4.50

We shall horribly abuse the syntax for conditional statements, by implementing a construct of the following form:

`if (evaluation_succeeds_take) { `*statement*` } else { `*alternative*` }`

The construct permits the user to catch the failure of a statement. It evaluates the statement as usual and returns as usual if the evaluation succeeds. If the evaluation fails, however, the given alternative statement is evaluated, as in the following example:

```
amb-evaluate input:
if (evaluation_succeeds_take) {
    const x = an_element_of(list(1, 3, 5));
    require(is_even(x));
    x;
} else {
    "all odd";
}

Starting a new problem
amb-evaluate value:
"all odd"
```

```
amb-evaluate input:
if (evaluation_succeeds_take) {
    const x = an_element_of(list(1, 3, 5, 8));
    require(is_even(x));
    x;
} else {
    "all odd";
}

Starting a new problem
amb-evaluate value:
8
```

Implement this construct by extending the amb evaluator. Hint: The function is_amb shows how to abuse the existing JavaScript syntax in order to implement a new syntactic form.

Exercise 4.51

With the new kind of assignment as described in exercise 4.49 and the construct

```
if (evaluation_succeeds_take) { ... } else { ... }
```

as in exercise 4.50, what will be the result of evaluating

```
let pairs = null;
if (evaluation_succeeds_take) {
    const p = prime_sum_pair(list(1, 3, 5, 8), list(20, 35, 110));
    pairs = pair(p, pairs); // using permanent assignment
    amb();
} else {
    pairs;
}
```

Exercise 4.52

If we had not realized that require could be implemented as an ordinary function that uses amb, to be defined by the user as part of a nondeterministic program, we would have had to implement it as a syntactic form. This would require syntax functions

```
function is_require(component) {
    return is_tagged_list(component, "require");
}
function require_predicate(component) { return head(tail(component)); }
```

and a new clause in the dispatch in analyze

```
: is_require(component)
? analyze_require(component)
```

as well the function analyze_require that handles require expressions. Complete the following definition of analyze_require.

```
function analyze_require(component) {
    const pfun = analyze(require_predicate(component));
    return (env, succeed, fail) =>
             pfun(env,
                  (pred_value, fail2) =>
                    ⟨??⟩
                    ? ⟨??⟩
                    : succeed("ok", fail2),
                  fail);
}
```

4.4 Logic Programming

In chapter 1 we stressed that computer science deals with imperative (how to) knowledge, whereas mathematics deals with declarative (what is) knowledge. Indeed, programming languages require that the programmer express knowledge in a form that indicates the step-by-step methods for solving particular problems. On the other hand, high-level languages provide, as part of the language implementation, a substantial amount of methodological knowledge that frees the user from concern with numerous details of how a specified computation will progress.

Most programming languages, including JavaScript, are organized around computing the values of mathematical functions. Expression-oriented languages (such as Lisp, C, Python, and JavaScript) capitalize on the "pun" that an expression that describes the value of a function may also be interpreted as a means of computing that value. Because of this, most programming languages are strongly biased toward unidirectional computations (computations with well-defined inputs and outputs). There are, however, radically different programming languages that relax this bias. We saw one such example in section 3.3.5, where the objects of computation were arithmetic constraints. In a constraint system the direction and the order of computation are not so well specified; in carrying out a computation the system must therefore provide more detailed "how to" knowledge than would be the case with an ordinary arithmetic computation. This does not mean, however, that the user is released altogether from the responsibility of providing imperative knowledge. There are many constraint networks that implement the same set of constraints, and the user must choose from the set of mathematically equivalent networks a suitable network to specify a particular computation.

The nondeterministic program evaluator of section 4.3 also moves away from the view that programming is about constructing algorithms for computing unidirectional functions. In a nondeterministic language, expressions can have more than one value, and, as a result, the computation is dealing with relations rather than with single-valued functions. Logic programming extends this idea by combining a relational vision of programming with a powerful kind of symbolic pattern matching called *unification*.[53]

---

53. Logic programming has grown out of a long history of research in automatic theorem proving. Early theorem-proving programs could accomplish very little, because they exhaustively searched the space of possible proofs. The major breakthrough that made such a search plausible was the discovery in the early 1960s of the *unification algorithm* and the *resolution principle*

This approach, when it works, can be a very powerful way to write programs. Part of the power comes from the fact that a single "what is" fact can be used to solve a number of different problems that would have different "how to" components. As an example, consider the `append` operation, which takes two lists as arguments and combines their elements to form a single list. In a procedural language such as JavaScript, we could define `append` in terms of the basic list constructor `pair`, as we did in section 2.2.1:

```
function append(x, y) {
    return is_null(x)
           ? y
           : pair(head(x), append(tail(x), y));
}
```

This function can be regarded as a translation into JavaScript of the following two rules, the first of which covers the case where the first list is empty and the second of which handles the case of a nonempty list, which is a `pair` of two parts:

- For any list `y`, the empty list and `y` `append` to form `y`.
- For any `u`, `v`, `y`, and `z`, `pair(u, v)` and `y` `append` to form `pair(u, z)` if `v` and `y` `append` to form `z`.[54]

Using the `append` function, we can answer questions such as

> Find the `append` of `list("a", "b")` and `list("c", "d")`.

But the same two rules are also sufficient for answering the following sorts of questions, which the function can't answer:

> Find a list `y` that `append`s with `list("a", "b")` to produce `list("a", "b", "c", "d")`.
>
> Find all `x` and `y` that `append` to form `list("a", "b", "c", "d")`.

(Robinson 1965). Resolution was used, for example, by Green and Raphael (1968) (see also Green 1969) as the basis for a deductive question-answering system. During most of this period, researchers concentrated on algorithms that are guaranteed to find a proof if one exists. Such algorithms were difficult to control and to direct toward a proof. Hewitt (1969) recognized the possibility of merging the control structure of a programming language with the operations of a logic-manipulation system, leading to the work in automatic search mentioned in section 4.3.1 (footnote 43). At the same time that this was being done, Colmerauer, in Marseille, was developing rule-based systems for manipulating natural language (see Colmerauer et al. 1973). He invented a programming language called Prolog for representing those rules. Kowalski (1973; 1979) in Edinburgh, recognized that execution of a Prolog program could be interpreted as proving theorems (using a proof technique called linear Horn-clause resolution). The merging of the last two strands led to the logic-programming movement. Thus, in assigning credit for the development of logic programming, the French can point to Prolog's genesis at the University of Marseille, while the British can highlight the work at the University of Edinburgh. According to people at MIT, logic programming was developed by these groups in an attempt to figure out what Hewitt was talking about in his brilliant but impenetrable Ph.D. thesis. For a history of logic programming, see Robinson 1983.

54. To see the correspondence between the rules and the function, let `x` in the function (where `x` is nonempty) correspond to `pair(u, v)` in the rule. Then `z` in the rule corresponds to the `append` of `tail(x)` and `y`.

In a logic programming language, the programmer writes an `append` "function" by stating the two rules about `append` given above. "How to" knowledge is provided automatically by the interpreter to allow this single pair of rules to be used to answer all three types of questions about `append`.[55]

Contemporary logic programming languages (including the one we implement here) have substantial deficiencies, in that their general "how to" methods can lead them into spurious infinite loops or other undesirable behavior. Logic programming is an active field of research in computer science.[56]

Earlier in this chapter we explored the technology of implementing interpreters and described the elements that are essential to an interpreter for a JavaScript-like language (indeed, to an interpreter for any conventional language). Now we will apply these ideas to discuss an interpreter for a logic programming language. We call this language the *query language*, because it is very useful for retrieving information from data bases by formulating *queries*, or questions, expressed in the language. Even though the query language is very different from JavaScript, we will find it convenient to describe the language in terms of the same general framework we have been using all along: as a collection of primitive elements, together with means of combination that enable us to combine simple elements to create more complex elements and means of abstraction that enable us to regard complex elements as single conceptual units. An interpreter for a logic programming language is considerably more complex than an interpreter for a language like JavaScript. Nevertheless, we will see that our query-language interpreter contains many of the same elements found in the interpreter of section 4.1. In particular, there will be an "evaluate" part that classifies expressions according to type and an "apply" part that implements the language's abstraction mechanism (functions in the case of JavaScript, and *rules* in the case of logic programming). Also, a central role is played in the implementation by a frame data structure, which determines the correspondence between symbols and their associated values. One additional interesting aspect of our query-language implementation is that we make substantial use of streams, which were introduced in chapter 3.

---

55. This certainly does not relieve the user of the entire problem of how to compute the answer. There are many different mathematically equivalent sets of rules for formulating the `append` relation, only some of which can be turned into effective devices for computing in any direction. In addition, sometimes "what is" information gives no clue "how to" compute an answer. For example, consider the problem of computing the y such that $y^2 = x$.

56. Interest in logic programming peaked during the early 1980s when the Japanese government began an ambitious project aimed at building superfast computers optimized to run logic programming languages. The speed of such computers was to be measured in LIPS (Logical Inferences Per Second) rather than the usual FLOPS (FLoating-point Operations Per Second). Although the project succeeded in developing hardware and software as originally planned, the international computer industry moved in a different direction. See Feigenbaum and Shrobe 1993 for an overview evaluation of the Japanese project. The logic programming community has also moved on to consider relational programming based on techniques other than simple pattern matching, such as the ability to deal with numerical constraints such as the ones illustrated in the constraint-propagation system of section 3.3.5.

4.4.1 Deductive Information Retrieval

Logic programming excels in providing interfaces to data bases for information retrieval. The query language we shall implement in this chapter is designed to be used in this way.

In order to illustrate what the query system does, we will show how it can be used to manage the data base of personnel records for Gargle, a thriving high-technology company in the Boston area. The language provides pattern-directed access to personnel information and can also take advantage of general rules in order to make logical deductions.

A sample data base

The personnel data base for Gargle contains *assertions* about company personnel. Here is the information about Ben Bitdiddle, the resident computer wizard:

```
address(list("Bitdiddle", "Ben"),
        list("Slumerville", list("Ridge", "Road"), 10))
job(list("Bitdiddle", "Ben"), list("computer", "wizard"))
salary(list("Bitdiddle", "Ben"), 122000)
```

Assertions look just like function applications in JavaScript, but they actually represent information in the data base. The first symbols—here `address`, `job` and `salary`—describe the *kind of information* contained in the respective assertion, and the "arguments" are lists or primitive values such as strings and numbers. The first symbols do not need to be declared, as do constants or variables in JavaScript; their scope is global.

As resident wizard, Ben is in charge of the company's computer division, and he supervises two programmers and one technician. Here is the information about them:

```
address(list("Hacker", "Alyssa", "P"),
        list("Cambridge", list("Mass", "Ave"), 78))
job(list("Hacker", "Alyssa", "P"), list("computer", "programmer"))
salary(list("Hacker", "Alyssa", "P"), 81000)
supervisor(list("Hacker", "Alyssa", "P"), list("Bitdiddle", "Ben"))

address(list("Fect", "Cy", "D"),
        list("Cambridge", list("Ames", "Street"), 3))
job(list("Fect", "Cy", "D"), list("computer", "programmer"))
salary(list("Fect", "Cy", "D"), 70000)
supervisor(list("Fect", "Cy", "D"), list("Bitdiddle", "Ben"))

address(list("Tweakit", "Lem", "E"),
        list("Boston", list("Bay", "State", "Road"), 22))
job(list("Tweakit", "Lem", "E"), list("computer", "technician"))
salary(list("Tweakit", "Lem", "E"), 51000)
supervisor(list("Tweakit", "Lem", "E"), list("Bitdiddle", "Ben"))
```

There is also a programmer trainee, who is supervised by Alyssa:

```
address(list("Reasoner", "Louis"),
        list("Slumerville", list("Pine", "Tree", "Road"), 80))
job(list("Reasoner", "Louis"),
         list("computer", "programmer", "trainee"))
salary(list("Reasoner", "Louis"), 62000)
supervisor(list("Reasoner", "Louis"), list("Hacker", "Alyssa", "P"))
```

All these people are in the computer division, as indicated by the word `"computer"` as the first item in their job descriptions.

Ben is a high-level employee. His supervisor is the company's big wheel himself:

```
supervisor(list("Bitdiddle", "Ben"), list("Warbucks", "Oliver"))

address(list("Warbucks", "Oliver"),
        list("Swellesley", list("Top", "Heap", "Road")))
job(list("Warbucks", "Oliver"), list("administration", "big", "wheel"))
salary(list("Warbucks", "Oliver"), 314159)
```

Besides the computer division supervised by Ben, the company has an accounting division, consisting of a chief accountant and his assistant:

```
address(list("Scrooge", "Eben"),
        list("Weston", list("Shady", "Lane"), 10))
job(list("Scrooge", "Eben"), list("accounting", "chief", "accountant"))
salary(list("Scrooge", "Eben"), 141421)
supervisor(list("Scrooge", "Eben"), list("Warbucks", "Oliver"))

address(list("Cratchit", "Robert"),
        list("Allston", list("N", "Harvard", "Street"), 16))
job(list("Cratchit", "Robert"), list("accounting", "scrivener"))
salary(list("Cratchit", "Robert"), 26100)
supervisor(list("Cratchit", "Robert"), list("Scrooge", "Eben"))
```

There is also an administrative assistant for the big wheel:

```
address(list("Aull", "DeWitt"),
        list("Slumerville", list("Onion", "Square"), 5))
job(list("Aull", "DeWitt"), list("administration", "assistant"))
salary(list("Aull", "DeWitt"), 42195)
supervisor(list("Aull", "DeWitt"), list("Warbucks", "Oliver"))
```

The data base also contains assertions about which kinds of jobs can be done by people holding other kinds of jobs. For instance, a computer wizard can do the jobs of both a computer programmer and a computer technician:

```
can_do_job(list("computer", "wizard"),
           list("computer", "programmer"))
can_do_job(list("computer", "wizard"),
           list("computer", "technician"))
```

A computer programmer could fill in for a trainee:

```
can_do_job(list("computer", "programmer"),
           list("computer", "programmer", "trainee"))
```

Also, as is well known,

```
can_do_job(list("administration", "assistant"),
           list("administration", "big", "wheel"))
```

Simple queries

The query language allows users to retrieve information from the data base by posing queries in response to the system's prompt. For example, to find all computer programmers one can say

```
Query input:
job($x, list("computer", "programmer"))
```

The system will respond with the following items:

```
Query results:
job(list("Hacker", "Alyssa", "P"), list("computer", "programmer"))
job(list("Fect", "Cy", "D"), list("computer", "programmer"))
```

The input query specifies that we are looking for entries in the data base that match a certain *pattern*. In this example, the pattern specifies `job` as the kind of information that we are looking for. The first item can be anything, and the second is the literal list `list("computer", "programmer")`. The "anything" that can be the first item in the matching assertion is specified by a *pattern variable*, `$x`. As pattern variables, we use JavaScript names that start with a dollar sign. We will see below why it is useful to specify names for pattern variables rather than just putting a single symbol such as `$` into patterns to represent "anything." The system responds to a simple query by showing all entries in the data base that match the specified pattern.

A pattern can have more than one variable. For example, the query

```
address($x, $y)
```

will list all the employees' addresses.

A pattern can have no variables, in which case the query simply determines whether that pattern is an entry in the data base. If so, there will be one match; if not, there will be no matches.

The same pattern variable can appear more than once in a query, specifying that the same "anything" must appear in each position. This is why variables have names. For example,

```
supervisor($x, $x)
```

finds all people who supervise themselves (though there are no such assertions in our sample data base).

The query

```
job($x, list("computer", $type))
```

matches all job entries whose second item is a two-element list whose first item is `"computer"`:

```
job(list("Bitdiddle", "Ben"), list("computer", "wizard"))
job(list("Hacker", "Alyssa", "P"), list("computer", "programmer"))
job(list("Fect", "Cy", "D"), list("computer", "programmer"))
job(list("Tweakit", "Lem", "E"), list("computer", "technician"))
```

This same pattern does *not* match

```
job(list("Reasoner", "Louis"),
    list("computer", "programmer", "trainee"))
```

because the second item in the assertion is a list of three elements, and the pattern's second item specifies that there should be two elements. If we wanted to change the pattern so that the second item could be any list beginning with `"computer"`, we could specify

```
job($x, pair("computer", $type))
```

For example,

```
pair("computer", $type)
```

matches the data

```
list("computer", "programmer", "trainee")
```

with `$type` as `list("programmer", "trainee")`. It also matches the data

```
list("computer", "programmer")
```

with `$type` as `list("programmer")`, and matches the data

```
list("computer")
```

with `$type` as the empty list, `null`.

We can describe the query language's processing of simple queries as follows:

- The system finds all assignments to variables in the query pattern that *satisfy* the pattern—that is, all sets of values for the variables such that if the pattern variables are *instantiated with* (replaced by) the values, the result is in the data base.
- The system responds to the query by listing all instantiations of the query pattern with the variable assignments that satisfy it.

Note that if the pattern has no variables, the query reduces to a determination of whether that pattern is in the data base. If so, the empty assignment, which assigns no values to variables, satisfies that pattern for that data base.

Exercise 4.53

Give simple queries that retrieve the following information from the data base:

a. all people supervised by Ben Bitdiddle;

b. the names and jobs of all people in the accounting division;

c. the names and addresses of all people who live in Slumerville.

Compound queries

Simple queries form the primitive operations of the query language. In order to form compound operations, the query language provides means of combination. One thing that makes the query language a logic programming language is that the means of combination mirror the means of combination used in forming logical expressions: `and`, `or`, and `not`.

We can use `and` as follows to find the addresses of all the computer programmers:

```
and(job($person, list("computer", "programmer")),
    address($person, $where))
```

The resulting output is

```
and(job(list("Hacker", "Alyssa", "P"), list("computer", "programmer")),
    address(list("Hacker", "Alyssa", "P"),
            list("Cambridge", list("Mass", "Ave"), 78)))

and(job(list("Fect", "Cy", "D"), list("computer", "programmer")),
    address(list("Fect", "Cy", "D"),
            list("Cambridge", list("Ames", "Street"), 3)))
```

In general,

`and(`*query*$_1$`,` *query*$_2$`, ...,` *query*$_n$`)`

is satisfied by all sets of values for the pattern variables that simultaneously satisfy *query*$_1, \ldots,$ *query*$_n$.

As for simple queries, the system processes a compound query by finding all assignments to the pattern variables that satisfy the query, then displaying instantiations of the query with those values.

Another means of constructing compound queries is through `or`. For example,

```
or(supervisor($x, list("Bitdiddle", "Ben")),
   supervisor($x, list("Hacker", "Alyssa", "P")))
```

will find all employees supervised by Ben Bitdiddle or Alyssa P. Hacker:

```
or(supervisor(list("Hacker", "Alyssa", "P"),
              list("Bitdiddle", "Ben")),
   supervisor(list("Hacker", "Alyssa", "P"),
              list("Hacker", "Alyssa", "P")))

or(supervisor(list("Fect", "Cy", "D"),
              list("Bitdiddle", "Ben")),
   supervisor(list("Fect", "Cy", "D"),
              list("Hacker", "Alyssa", "P")))

or(supervisor(list("Tweakit", "Lem", "E"),
              list("Bitdiddle", "Ben")),
   supervisor(list("Tweakit", "Lem", "E"),
              list("Hacker", "Alyssa", "P")))

or(supervisor(list("Reasoner", "Louis"),
              list("Bitdiddle", "Ben")),
   supervisor(list("Reasoner", "Louis"),
              list("Hacker", "Alyssa", "P")))
```

In general,

`or(`*query*$_1$`, `*query*$_2$`, ..., `*query*$_n$`)`

is satisfied by all sets of values for the pattern variables that satisfy at least one of *query*$_1$ ... *query*$_n$.

Compound queries can also be formed with `not`. For example,

```
and(supervisor($x, list("Bitdiddle", "Ben")),
    not(job($x, list("computer", "programmer"))))
```

finds all people supervised by Ben Bitdiddle who are not computer programmers. In general,

`not(`*query*$_1$`)`

is satisfied by all assignments to the pattern variables that do not satisfy *query*$_1$.[57]

The final combining form starts with `javascript_predicate` and contains a JavaScript predicate. In general,

`javascript_predicate(`*predicate*`)`

will be satisfied by assignments to the pattern variables in the *predicate* for which the instantiated *predicate* is true. For example, to find all people whose salary is greater than $50,000 we could write[58]

```
and(salary($person, $amount), javascript_predicate($amount > 50000))
```

Exercise 4.54

Formulate compound queries that retrieve the following information:

a. the names of all people who are supervised by Ben Bitdiddle, together with their addresses;

b. all people whose salary is less than Ben Bitdiddle's, together with their salary and Ben Bitdiddle's salary;

c. all people who are supervised by someone who is not in the computer division, together with the supervisor's name and job.

---

57. Actually, this description of `not` is valid only for simple cases. The real behavior of `not` is more complex. We will examine `not`'s peculiarities in sections 4.4.2 and 4.4.3.

58. A query should use `javascript_predicate` only to perform an operation not provided in the query language. In particular, `javascript_predicate` should not be used to test equality (since that is what the matching in the query language is designed to do) or inequality (since that can be done with the `same` rule shown below).

Rules

In addition to primitive queries and compound queries, the query language provides means for abstracting queries. These are given by *rules*. The rule

```
rule(lives_near($person_1, $person_2),
     and(address($person_1, pair($town, $rest_1)),
         address($person_2, pair($town, $rest_2)),
         not(same($person_1, $person_2))))
```

specifies that two people live near each other if they live in the same town. The final `not` clause prevents the rule from saying that all people live near themselves. The `same` relation is defined by a very simple rule:[59]

```
rule(same($x, $x))
```

The following rule declares that a person is a "wheel" in an organization if he supervises someone who is in turn a supervisor:

```
rule(wheel($person),
     and(supervisor($middle_manager, $person),
         supervisor($x, $middle_manager)))
```

The general form of a rule is

`rule(`*conclusion*`,` *body*`)`

where *conclusion* is a pattern and *body* is any query.[60] We can think of a rule as representing a large (even infinite) set of assertions, namely all instantiations of the rule conclusion with variable assignments that satisfy the rule body. When we described simple queries (patterns), we said that an assignment to variables satisfies a pattern if the instantiated pattern is in the data base. But the pattern needn't be explicitly in the data base as an assertion. It can be an implicit assertion implied by a rule. For example, the query

---

59. Notice that we do not need `same` in order to make two things be the same: We just use the same pattern variable for each—in effect, we have one thing instead of two things in the first place. For example, see `$town` in the `lives_near` rule and `$middle_manager` in the `wheel` rule below. The `same` relation is useful when we want to force two things to be different, such as `$person_1` and `$person_2` in the `lives_near` rule. Although using the same pattern variable in two parts of a query forces the same value to appear in both places, using different pattern variables does not force different values to appear. (The values assigned to different pattern variables may be the same or different.)

60. We will also allow rules without bodies, as in `same`, and we will interpret such a rule to mean that the rule conclusion is satisfied by any values of the variables.

```
lives_near($x, list("Bitdiddle", "Ben"))
```

results in

```
lives_near(list("Reasoner", "Louis"), list("Bitdiddle", "Ben"))
lives_near(list("Aull", "DeWitt"), list("Bitdiddle", "Ben"))
```

To find all computer programmers who live near Ben Bitdiddle, we can ask

```
and(job($x, list("computer", "programmer")),
    lives_near($x, list("Bitdiddle", "Ben")))
```

As in the case of compound functions, rules can be used as parts of other rules (as we saw with the `lives_near` rule above) or even be defined recursively. For instance, the rule

```
rule(outranked_by($staff_person, $boss),
     or(supervisor($staff_person, $boss),
        and(supervisor($staff_person, $middle_manager),
            outranked_by($middle_manager, $boss))))
```

says that a staff person is outranked by a boss in the organization if the boss is the person's supervisor or (recursively) if the person's supervisor is outranked by the boss.

Exercise 4.55

Define a rule that says that person 1 can replace person 2 if either person 1 does the same job as person 2 or someone who does person 1's job can also do person 2's job, and if person 1 and person 2 are not the same person. Using your rule, give queries that find the following:

a. all people who can replace Cy D. Fect;

b. all people who can replace someone who is being paid more than they are, together with the two salaries.

Exercise 4.56

Define a rule that says that a person is a "big shot" in a division if the person works in the division but does not have a supervisor who works in the division.

Exercise 4.57

Ben Bitdiddle has missed one meeting too many. Fearing that his habit of forgetting meetings could cost him his job, Ben decides to do something about it. He adds all the weekly meetings of the firm to the Gargle data base by asserting the following:

```
meeting("accounting", list("Monday", "9am"))
meeting("administration", list("Monday", "10am"))
meeting("computer", list("Wednesday", "3pm"))
meeting("administration", list("Friday", "1pm"))
```

Each of the above assertions is for a meeting of an entire division. Ben also adds an entry for the company-wide meeting that spans all the divisions. All of the company's employees attend this meeting.

```
meeting("whole-company", list("Wednesday", "4pm"))
```

a. On Friday morning, Ben wants to query the data base for all the meetings that occur that day. What query should he use?

b. Alyssa P. Hacker is unimpressed. She thinks it would be much more useful to be able to ask for her meetings by specifying her name. So she designs a rule that says that a person's meetings include all `"whole-company"` meetings plus all meetings of that person's division. Fill in the body of Alyssa's rule.

```
rule(meeting_time($person, $day_and_time),
     rule-body)
```

c. Alyssa arrives at work on Wednesday morning and wonders what meetings she has to attend that day. Having defined the above rule, what query should she make to find this out?

Exercise 4.58

By giving the query

```
lives_near($person, list("Hacker", "Alyssa", "P"))
```

Alyssa P. Hacker is able to find people who live near her, with whom she can ride to work. On the other hand, when she tries to find all pairs of people who live near each other by querying

```
lives_near($person_1, $person_2)
```

she notices that each pair of people who live near each other is listed twice; for example,

```
lives_near(list("Hacker", "Alyssa", "P"), list("Fect", "Cy", "D"))
lives_near(list("Fect", "Cy", "D"), list("Hacker", "Alyssa", "P"))
```

Why does this happen? Is there a way to find a list of people who live near each other, in which each pair appears only once? Explain.

Logic as programs

We can regard a rule as a kind of logical implication: *If* an assignment of values to pattern variables satisfies the body, *then* it satisfies the conclusion. Consequently, we can regard the query language as having the ability to perform *logical deductions* based upon the rules. As an example, consider the `append` operation described at the beginning of section 4.4. As we said, `append` can be characterized by the following two rules:

- For any `y`, the empty list and `y` `append` to form `y`.
- For any `u`, `v`, `y`, and `z`, `pair(u, v)` and `y` `append` to form `pair(u, z)` if `v` and `y` `append` to form `z`.

To express this in our query language, we define two rules for a relation

```
append_to_form(x, y, z)
```

which we can interpret to mean "`x` and `y` `append` to form `z`":

```
rule(append_to_form(null, $y, $y))

rule(append_to_form(pair($u, $v), $y, pair($u, $z)),
     append_to_form($v, $y, $z))
```

The first rule has no body, which means that the conclusion holds for any value of `$y`. Note how the second rule makes use of `pair` to name the head and tail of a list.

Given these two rules, we can formulate queries that compute the `append` of two lists:

```
Query input:
append_to_form(list("a", "b"), list("c", "d"), $z)

Query results:
append_to_form(list("a", "b"), list("c", "d"), list("a", "b", "c", "d"))
```

What is more striking, we can use the same rules to ask the question "Which list, when appended to `list("a", "b")`, yields `list("a", "b", "c", "d")`?" This is done as follows:

```
Query input:
append_to_form(list("a", "b"), $y, list("a", "b", "c", "d"))

Query results:
append_to_form(list("a", "b"), list("c", "d"), list("a", "b", "c", "d"))
```

We can ask for all pairs of lists that `append` to form `list("a", "b", "c", "d")`:

```
Query input:
append_to_form($x, $y, list("a", "b", "c", "d"))

Query results:
append_to_form(null, list("a", "b", "c", "d"), list("a", "b", "c", "d"))
append_to_form(list("a"), list("b", "c", "d"), list("a", "b", "c", "d"))
append_to_form(list("a", "b"), list("c", "d"), list("a", "b", "c", "d"))
append_to_form(list("a", "b", "c"), list("d"), list("a", "b", "c", "d"))
append_to_form(list("a", "b", "c", "d"), null, list("a", "b", "c", "d"))
```

The query system may seem to exhibit quite a bit of intelligence in using the rules to deduce the answers to the queries above. Actually, as we will see in the next section, the system is following a well-determined algorithm in unraveling the rules. Unfortunately, although the system works impressively in the `append` case, the general methods may break down in more complex cases, as we will see in section 4.4.3.

Exercise 4.59

The following rules implement a `next_to_in` relation that finds adjacent elements of a list:

```
rule(next_to_in($x, $y, pair($x, pair($y, $u))))

rule(next_to_in($x, $y, pair($v, $z)),
     next_to_in($x, $y, $z))
```

What will the response be to the following queries?

```
next_to_in($x, $y, list(1, list(2, 3), 4))

next_to_in($x, 1, list(2, 1, 3, 1))
```

Exercise 4.60

Define rules to implement the `last_pair` operation of exercise 2.17, which returns a list containing the last element of a nonempty list. Check your rules on the following queries:

- `last_pair(list(3), $x)`
- `last_pair(list(1, 2, 3), $x)`
- `last_pair(list(2, $x), list(3))`

Do your rules work correctly on queries such as `last_pair($x, list(3))`?

Exercise 4.61

The following data base (see Genesis 4) traces the genealogy of the descendants of Ada back to Adam, by way of Cain:

```
son("Adam", "Cain")
son("Cain", "Enoch")
son("Enoch", "Irad")
son("Irad", "Mehujael")
son("Mehujael", "Methushael")
son("Methushael", "Lamech")
wife("Lamech", "Ada")
son("Ada", "Jabal")
son("Ada", "Jubal")
```

Formulate rules such as "If *S* is the son of *F*, and *F* is the son of *G*, then *S* is the grandson of *G*" and "If *W* is the wife of *M*, and *S* is the son of *W*, then *S* is the son of *M*" (which was supposedly more true in biblical times than today) that will enable the query system to find the grandson of Cain; the sons of Lamech; the grandsons of Methushael. (See exercise 4.67 for some rules to deduce more complicated relationships.)

4.4.2 How the Query System Works

In section 4.4.4 we will present an implementation of the query interpreter as a collection of functions. In this section we give an overview that explains the general structure of the system independent of low-level implementation details. After describing the implementation of the interpreter, we will be in a position to understand some of its limitations and some of the subtle ways in which the query language's logical operations differ from the operations of mathematical logic.

It should be apparent that the query evaluator must perform some kind of search in order to match queries against facts and rules in the data base. One way to do this would be to implement the query system as a nondeterministic program, using the `amb` evaluator of section 4.3 (see exercise 4.75). Another possibility is to manage the search with the aid of streams. Our implementation follows this second approach.

The query system is organized around two central operations, called *pattern matching* and *unification*. We first describe pattern matching and explain how this

operation, together with the organization of information in terms of streams of frames, enables us to implement both simple and compound queries. We next discuss unification, a generalization of pattern matching needed to implement rules. Finally, we show how the entire query interpreter fits together through a function that classifies queries in a manner analogous to the way `evaluate` classifies expressions for the interpreter described in section 4.1.

Pattern matching

A *pattern matcher* is a program that tests whether some datum fits a specified pattern. For example, the datum `list(list("a", "b"), "c", list("a", "b"))` matches the pattern `list($x, "c", $x)` with the pattern variable `$x` bound to `list("a", "b")`. The same data list matches the pattern `list($x, $y, $z)` with `$x` and `$z` both bound to `list("a", "b")` and `$y` bound to `"c"`. It also matches the pattern `list(list($x, $y), "c", list($x, $y))` with `$x` bound to `"a"` and `$y` bound to `"b"`. However, it does not match the pattern `list($x, "a", $y)`, since that pattern specifies a list whose second element is the string `"a"`.

The pattern matcher used by the query system takes as inputs a pattern, a datum, and a *frame* that specifies bindings for various pattern variables. It checks whether the datum matches the pattern in a way that is consistent with the bindings already in the frame. If so, it returns the given frame augmented by any bindings that may have been determined by the match. Otherwise, it indicates that the match has failed.

Using the pattern `list($x, $y, $x)` to match `list("a", "b", "a")` given an empty frame, for example, will return a frame specifying that `$x` is bound to `"a"` and `$y` is bound to `"b"`. Trying the match with the same pattern, the same datum, and a frame specifying that `$y` is bound to `"a"` will fail. Trying the match with the same pattern, the same datum, and a frame in which `$y` is bound to `"b"` and `$x` is unbound will return the given frame augmented by a binding of `$x` to `"a"`.

The pattern matcher is all the mechanism that is needed to process simple queries that don't involve rules. For instance, to process the query

```
job($x, list("computer", "programmer"))
```

we scan through all assertions in the data base and select those that match the pattern with respect to an initially empty frame. For each match we find, we use the frame returned by the match to instantiate the pattern with a value for `$x`.

Streams of frames

The testing of patterns against frames is organized through the use of streams. Given a single frame, the matching process runs through the data-base entries one by one. For each data-base entry, the matcher generates either a special symbol indicating that the match has failed or an extension to the frame. The results for all the data-base entries are collected into a stream, which is passed through a filter to weed out the

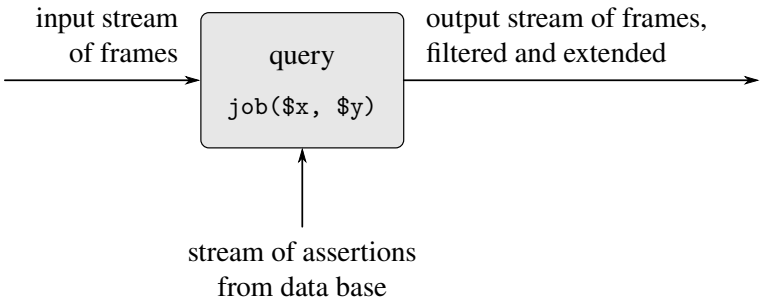


Figure 4.5 A query processes a stream of frames.

failures. The result is a stream of all the frames that extend the given frame via a match to some assertion in the data base.[61]

In our system, a query takes an input stream of frames and performs the above matching operation for every frame in the stream, as indicated in figure 4.5. That is, for each frame in the input stream, the query generates a new stream consisting of all extensions to that frame by matches to assertions in the data base. All these streams are then combined to form one huge stream, which contains all possible extensions of every frame in the input stream. This stream is the output of the query.

To answer a simple query, we use the query with an input stream consisting of a single empty frame. The resulting output stream contains all extensions to the empty frame (that is, all answers to our query). This stream of frames is then used to generate a stream of copies of the original query pattern with the variables instantiated by the values in each frame, and this is the stream that is finally printed.

Compound queries

The real elegance of the stream-of-frames implementation is evident when we deal with compound queries. The processing of compound queries makes use of the ability of our matcher to demand that a match be consistent with a specified frame. For example, to handle the `and` of two queries, such as

```
and(can_do_job($x, list("computer", "programmer", "trainee")),
    job($person, $x))
```

(informally, "Find all people who can do the job of a computer programmer trainee"), we first find all entries that match the pattern

```
can_do_job($x, list("computer", "programmer", "trainee"))
```

---

61. Because matching is generally very expensive, we would like to avoid applying the full matcher to every element of the data base. This is usually arranged by breaking up the process into a fast, coarse match and the final match. The coarse match filters the data base to produce a small set of candidates for the final match. With care, we can arrange our data base so that some of the work of coarse matching can be done when the data base is constructed rather then when we want to select the candidates. This is called *indexing* the data base. There is a vast technology built around data-base-indexing schemes. Our implementation, described in section 4.4.4, contains a simpleminded form of such an optimization.

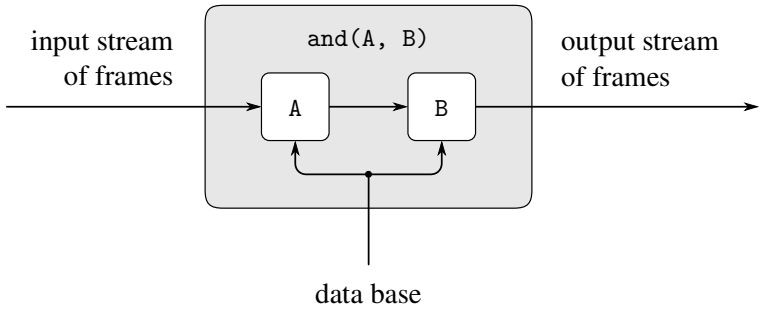


Figure 4.6 The `and` combination of two queries is produced by operating on the stream of frames in series.

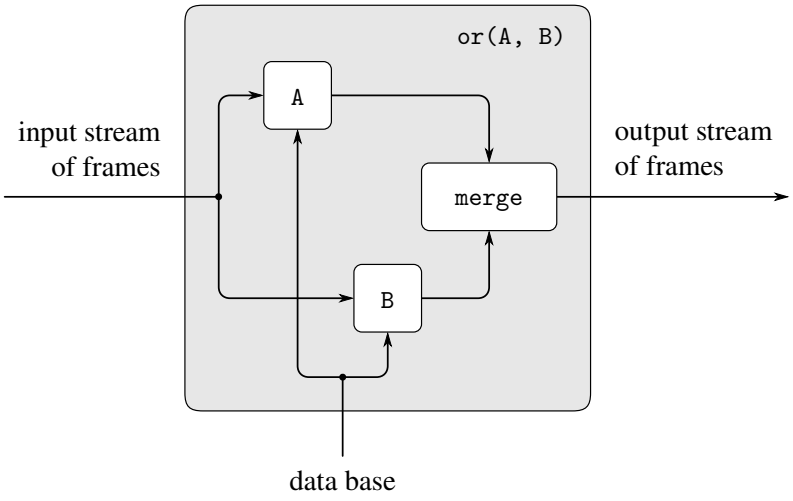


Figure 4.7 The `or` combination of two queries is produced by operating on the stream of frames in parallel and merging the results.

This produces a stream of frames, each of which contains a binding for `$x`. Then for each frame in the stream we find all entries that match

```
job($person, $x)
```

in a way that is consistent with the given binding for `$x`. Each such match will produce a frame containing bindings for `$x` and `$person`. The `and` of two queries can be viewed as a series combination of the two component queries, as shown in figure 4.6. The frames that pass through the first query filter are filtered and further extended by the second query.

Figure 4.7 shows the analogous method for computing the `or` of two queries as a parallel combination of the two component queries. The input stream of frames is extended separately by each query. The two resulting streams are then merged to produce the final output stream.

Even from this high-level description, it is apparent that the processing of compound queries can be slow. For example, since a query may produce more than one output frame for each input frame, and each query in an `and` gets its input frames

from the previous query, an `and` query could, in the worst case, have to perform a number of matches that is exponential in the number of queries (see exercise 4.73).[62] Though systems for handling only simple queries are quite practical, dealing with complex queries is extremely difficult.[63]

From the stream-of-frames viewpoint, the `not` of some query acts as a filter that removes all frames for which the query can be satisfied. For instance, given the pattern

```
not(job($x, list("computer", "programmer")))
```

we attempt, for each frame in the input stream, to produce extension frames that satisfy `job($x, list("computer", "programmer"))`. We remove from the input stream all frames for which such extensions exist. The result is a stream consisting of only those frames in which the binding for `$x` does not satisfy `job($x, list("computer", "programmer"))`. For example, in processing the query

```
and(supervisor($x, $y),
    not(job($x, list("computer", "programmer"))))
```

the first clause will generate frames with bindings for `$x` and `$y`. The `not` clause will then filter these by removing all frames in which the binding for `$x` satisfies the restriction that `$x` is a computer programmer.[64]

The `javascript_predicate` syntactic form is implemented as a similar filter on frame streams. We use each frame in the stream to instantiate any variables in the pattern, then apply the JavaScript predicate. We remove from the input stream all frames for which the predicate fails.

Unification

In order to handle rules in the query language, we must be able to find the rules whose conclusions match a given query pattern. Rule conclusions are like assertions except that they can contain variables, so we will need a generalization of pattern matching—called *unification*—in which both the "pattern" and the "datum" may contain variables.

A unifier takes two patterns, each containing constants and variables, and determines whether it is possible to assign values to the variables that will make the two patterns equal. If so, it returns a frame containing these bindings. For example, unifying `list($x, "a", $y)` and `list($y, $z, "a")` will specify a frame in which `$x`, `$y`, and `$z` must all be bound to `"a"`. On the other hand, unifying `list($x, $y, "a")` and `list($x, "b", $y)` will fail, because there is no value for `$y` that can make the two patterns equal. (For the second elements of the patterns

---

62. But this kind of exponential explosion is not common in `and` queries because the added conditions tend to reduce rather than expand the number of frames produced.

63. There is a large literature on data-base-management systems that is concerned with how to handle complex queries efficiently.

64. There is a subtle difference between this filter implementation of `not` and the usual meaning of `not` in mathematical logic. See section 4.4.3.

to be equal, $y would have to be "b"; however, for the third elements to be equal, $y would have to be "a".) The unifier used in the query system, like the pattern matcher, takes a frame as input and performs unifications that are consistent with this frame.

The unification algorithm is the most technically difficult part of the query system. With complex patterns, performing unification may seem to require deduction. To unify

```
list($x, $x)
```

and

```
list(list("a", $y, "c"), list("a", "b", $z))
```

for example, the algorithm must infer that $x should be list("a", "b", "c"), $y should be "b", and $z should be "c". We may think of this process as solving a set of equations among the pattern components. In general, these are simultaneous equations, which may require substantial manipulation to solve.[65] For example, unifying list($x, $x) and list(list("a", $y, "c"), list("a", "b", $z)) may be thought of as specifying the simultaneous equations

```
$x  =  list("a", $y, "c")
$x  =  list("a", "b", $z)
```

These equations imply that

```
list("a", $y, "c")  =  list("a", "b", $z)
```

which in turn implies that

```
"a"  =  "a",    $y  =  "b",    "c"  =  $z
```

and hence that

```
$x  =  list("a", "b", "c")
```

In a successful pattern match, all pattern variables become bound, and the values to which they are bound contain only constants. This is also true of all the examples of unification we have seen so far. In general, however, a successful unification may not completely determine the variable values; some variables may remain unbound and others may be bound to values that contain variables.

Consider the unification of list($x, "a") and list(list("b", $y), $z). We can deduce that $x = list("b", $y) and "a" = $z, but we cannot further solve for $x or $y. The unification doesn't fail, since it is certainly possible to make the two patterns equal by assigning values to $x and $y. Since this match in no way restricts the values $y can take on, no binding for $y is put into the result frame. The match does, however, restrict the value of $x. Whatever value $y has, $x must be list("b", $y). A binding of $x to the pattern list("b", $y) is thus put into the frame. If a value for $y is later determined and added to the frame (by a pattern

65. In one-sided pattern matching, all the equations that contain pattern variables are explicit and already solved for the unknown (the pattern variable).

match or unification that is required to be consistent with this frame), the previously bound `$x` will refer to this value.[66]

Applying rules

Unification is the key to the component of the query system that makes inferences from rules. To see how this is accomplished, consider processing a query that involves applying a rule, such as

```
lives_near($x, list("Hacker", "Alyssa", "P"))
```

To process this query, we first use the ordinary pattern-match function described above to see if there are any assertions in the data base that match this pattern. (There will not be any in this case, since our data base includes no direct assertions about who lives near whom.) The next step is to attempt to unify the query pattern with the conclusion of each rule. We find that the pattern unifies with the conclusion of the rule

```
rule(lives_near($person_1, $person_2),
     and(address($person_1, pair($town, $rest_1)),
         address($person_2, list($town, $rest_2)),
         not(same($person_1, $person_2))))
```

resulting in a frame specifying that `$x` should be bound to (have the same value as) `$person_1` and that `$person_2` is bound to `list("Hacker", "Alyssa", "P")`. Now, relative to this frame, we evaluate the compound query given by the body of the rule. Successful matches will extend this frame by providing a binding for `$person_1`, and consequently a value for `$x`, which we can use to instantiate the original query pattern.

In general, the query evaluator uses the following method to apply a rule when trying to establish a query pattern in a frame that specifies bindings for some of the pattern variables:

- Unify the query with the conclusion of the rule to form, if successful, an extension of the original frame.
- Relative to the extended frame, evaluate the query formed by the body of the rule.

Notice how similar this is to the method for applying a function in the `evaluate`/`apply` evaluator for JavaScript:

- Bind the function's parameters to its arguments to form a frame that extends the original function environment.
- Relative to the extended environment, evaluate the expression formed by the body of the function.

---

66. Another way to think of unification is that it generates the most general pattern that is a specialization of the two input patterns. This means that the unification of `list($x, "a")` and `list(list("b", $y), $z)` is `list(list("b", $y), "a")` and that the unification of `list($x, "a", $y)` and `list($y, $z, "a")`, discussed above, is `list("a", "a", "a")`. For our implementation, it is more convenient to think of the result of unification as a frame rather than a pattern.

The similarity between the two evaluators should come as no surprise. Just as function definitions are the means of abstraction in JavaScript, rule definitions are the means of abstraction in the query language. In each case, we unwind the abstraction by creating appropriate bindings and evaluating the rule or function body relative to these.

Simple queries

We saw earlier in this section how to evaluate simple queries in the absence of rules. Now that we have seen how to apply rules, we can describe how to evaluate simple queries by using both rules and assertions.

Given the query pattern and a stream of frames, we produce, for each frame in the input stream, two streams:

- a stream of extended frames obtained by matching the pattern against all assertions in the data base (using the pattern matcher), and
- a stream of extended frames obtained by applying all possible rules (using the unifier).[67]

Appending these two streams produces a stream that consists of all the ways that the given pattern can be satisfied consistent with the original frame. These streams (one for each frame in the input stream) are now all combined to form one large stream, which therefore consists of all the ways that any of the frames in the original input stream can be extended to produce a match with the given pattern.

The query evaluator and the driver loop

Despite the complexity of the underlying matching operations, the system is organized much like an evaluator for any language. The function that coordinates the matching operations is called `evaluate_query`, and it plays a role analogous to that of the `evaluate` function for JavaScript. The function `evaluate_query` takes as inputs a query and a stream of frames. Its output is a stream of frames, corresponding to successful matches to the query pattern, that extend some frame in the input stream, as indicated in figure 4.5. Like `evaluate`, `evaluate_query` classifies the different types of expressions (queries) and dispatches to an appropriate function for each. There is a function for each syntactic form (`and`, `or`, `not`, and `javascript_predicate`) and one for simple queries.

The driver loop, which is analogous to the `driver_loop` function for the other evaluators in this chapter, reads queries typed by the user. For each query, it calls `evaluate_query` with the query and a stream that consists of a single empty frame. This will produce the stream of all possible matches (all possible extensions to the empty frame). For each frame in the resulting stream, it instantiates the original

---

67. Since unification is a generalization of matching, we could simplify the system by using the unifier to produce both streams. Treating the easy case with the simple matcher, however, illustrates how matching (as opposed to full-blown unification) can be useful in its own right.

query using the values of the variables found in the frame. This stream of instantiated queries is then printed.[68]

The driver also checks for the special command `assert`, which signals that the input is not a query but rather an assertion or rule to be added to the data base. For instance,

```
assert(job(list("Bitdiddle", "Ben"), list("computer", "wizard")))

assert(rule(wheel($person),
            and(supervisor($middle_manager, $person),
                supervisor($x, $middle_manager))))
```

4.4.3 Is Logic Programming Mathematical Logic?

The means of combination used in the query language may at first seem identical to the operations `and`, `or`, and `not` of mathematical logic, and the application of query-language rules is in fact accomplished through a legitimate method of inference.[69] This identification of the query language with mathematical logic is not really valid, though, because the query language provides a *control structure* that interprets the logical statements procedurally. We can often take advantage of this control structure. For example, to find all of the supervisors of programmers we could formulate a query in either of two logically equivalent forms:

```
and(job($x, list("computer", "programmer")),
    supervisor($x, $y))
```

or

```
and(supervisor($x, $y),
    job($x, list("computer", "programmer")))
```

If a company has many more supervisors than programmers, it is better to use the first form rather than the second, because the data base must be scanned for each intermediate result (frame) produced by the first clause of the `and`.

The aim of logic programming is to provide the programmer with techniques for decomposing a computational problem into two separate problems: "what" is to be computed, and "how" this should be computed. This is accomplished by selecting a subset of the statements of mathematical logic that is powerful enough to be able to describe anything one might want to compute, yet weak enough to have a controllable procedural interpretation. The intention here is that, on the one hand, a program

---

68. The reason we use streams (rather than lists) of frames is that the recursive application of rules can generate infinite numbers of values that satisfy a query. The delayed evaluation embodied in streams is crucial here: The system will print responses one by one as they are generated, regardless of whether there are a finite or infinite number of responses.

69. That a particular method of inference is legitimate is not a trivial assertion. One must prove that if one starts with true premises, only true conclusions can be derived. The method of inference represented by rule applications is *modus ponens*, the familiar method of inference that says that if A is true and A *implies* B is true, then we may conclude that B is true.

specified in a logic programming language should be an effective program that can be carried out by a computer. Control ("how" to compute) is effected by using the order of evaluation of the language. We should be able to arrange the order of clauses and the order of subgoals within each clause so that the computation is done in an order deemed to be effective and efficient. At the same time, we should be able to view the result of the computation ("what" to compute) as a simple consequence of the laws of logic.

Our query language can be regarded as just such a procedurally interpretable subset of mathematical logic. An assertion represents a simple fact (an atomic proposition). A rule represents the implication that the rule conclusion holds for those cases where the rule body holds. A rule has a natural procedural interpretation: To establish the conclusion of the rule, establish the body of the rule. Rules, therefore, specify computations. However, because rules can also be regarded as statements of mathematical logic, we can justify any "inference" accomplished by a logic program by asserting that the same result could be obtained by working entirely within mathematical logic.[70]

Infinite loops

A consequence of the procedural interpretation of logic programs is that it is possible to construct hopelessly inefficient programs for solving certain problems. An extreme case of inefficiency occurs when the system falls into infinite loops in making deductions. As a simple example, suppose we are setting up a data base of famous marriages, including

```
assert(married("Minnie", "Mickey"))
```

If we now ask

```
married("Mickey", $who)
```

we will get no response, because the system doesn't know that if A is married to B, then B is married to A. So we assert the rule

```
assert(rule(married($x, $y),
            married($y, $x)))
```

and again query

---

70. We must qualify this statement by agreeing that, in speaking of the "inference" accomplished by a logic program, we assume that the computation terminates. Unfortunately, even this qualified statement is false for our implementation of the query language (and also false for programs in Prolog and most other current logic programming languages) because of our use of `not` and `javascript_predicate`. As we will describe below, the `not` implemented in the query language is not always consistent with the `not` of mathematical logic, and `javascript_predicate` introduces additional complications. We could implement a language consistent with mathematical logic by simply removing `not` and `javascript_predicate` from the language and agreeing to write programs using only simple queries, `and`, and `or`. However, this would greatly restrict the expressive power of the language. One of the major concerns of research in logic programming was to find ways to achieve more consistency with mathematical logic without unduly sacrificing expressive power.

```
married("Mickey", $who)
```

Unfortunately, this will drive the system into an infinite loop, as follows:

- The system finds that the `married` rule is applicable; that is, the rule conclusion `married($x, $y)` unifies with the query pattern `married("Mickey", $who)` to produce a frame in which `$x` is bound to `"Mickey"` and `$y` is bound to `$who`. So the interpreter proceeds to evaluate the rule body `married($y, $x)` in this frame—in effect, to process the query `married($who, "Mickey")`.
- One answer, `married("Minnie", "Mickey")`, appears directly as an assertion in the data base.
- The `married` rule is also applicable, so the interpreter again evaluates the rule body, which this time is equivalent to `married("Mickey", $who)`.

The system is now in an infinite loop. Indeed, whether the system will find the simple answer `married("Minnie", "Mickey")` before it goes into the loop depends on implementation details concerning the order in which the system checks the items in the data base. This is a very simple example of the kinds of loops that can occur. Collections of interrelated rules can lead to loops that are much harder to anticipate, and the appearance of a loop can depend on the order of clauses in an `and` (see exercise 4.62) or on low-level details concerning the order in which the system processes queries.[71]

Problems with `not`

Another quirk in the query system concerns `not`. Given the data base of section 4.4.1, consider the following two queries:

```
and(supervisor($x, $y),
    not(job($x, list("computer", "programmer"))))

and(not(job($x, list("computer", "programmer"))),
    supervisor($x, $y))
```

These two queries do not produce the same result. The first query begins by finding all entries in the data base that match `supervisor($x, $y)`, and then filters the resulting frames by removing the ones in which the value of `$x` satisfies `job($x, list("computer", "programmer"))`. The second query begins by filtering the

71. This is not a problem of the logic but one of the procedural interpretation of the logic provided by our interpreter. We could write an interpreter that would not fall into a loop here. For example, we could enumerate all the proofs derivable from our assertions and our rules in a breadth-first rather than a depth-first order. However, such a system makes it more difficult to take advantage of the order of deductions in our programs. One attempt to build sophisticated control into such a program is described in de Kleer et al. 1977. Another technique, which does not lead to such serious control problems, is to put in special knowledge, such as detectors for particular kinds of loops (exercise 4.65). However, there can be no general scheme for reliably preventing a system from going down infinite paths in performing deductions. Imagine a diabolical rule of the form "To show $P(x)$ is true, show that $P(f(x))$ is true," for some suitably chosen function f.

incoming frames to remove those that can satisfy `job($x, list("computer", "programmer"))`. Since the only incoming frame is empty, it checks the data base for patterns that satisfy `job($x, list("computer", "programmer"))`. Since there generally are entries of this form, the `not` clause filters out the empty frame and returns an empty stream of frames. Consequently, the entire compound query returns an empty stream.

The trouble is that our implementation of `not` really is meant to serve as a filter on values for the variables. If a `not` clause is processed with a frame in which some of the variables remain unbound (as does `$x` in the example above), the system will produce unexpected results. Similar problems occur with the use of `javascript_predicate`—the JavaScript predicate can't work if some of its variables are unbound. See exercise 4.74.

There is also a much more serious way in which the `not` of the query language differs from the `not` of mathematical logic. In logic, we interpret the statement "not *P*" to mean that *P* is not true. In the query system, however, "not *P*" means that *P* is not deducible from the knowledge in the data base. For example, given the personnel data base of section 4.4.1, the system would happily deduce all sorts of `not` statements, such as that Ben Bitdiddle is not a baseball fan, that it is not raining outside, and that $2+2$ is not 4.[72] In other words, the `not` of logic programming languages reflects the so-called *closed world assumption* that all relevant information has been included in the data base.[73]

Exercise 4.62

Louis Reasoner mistakenly deletes the `outranked_by` rule (section 4.4.1) from the data base. When he realizes this, he quickly reinstalls it. Unfortunately, he makes a slight change in the rule, and types it in as

```
rule(outranked_by($staff_person, $boss),
     or(supervisor($staff_person, $boss),
        and(outranked_by($middle_manager, $boss),
            supervisor($staff_person, $middle_manager))))
```

Just after Louis types this information into the system, DeWitt Aull comes by to find out who outranks Ben Bitdiddle. He issues the query

```
outanked_by(list("Bitdiddle", "Ben"), $who)
```

After answering, the system goes into an infinite loop. Explain why.

---

72. Consider the query `not(baseball_fan(list("Bitdiddle", "Ben")))`. The system finds that `baseball_fan(list("Bitdiddle", "Ben"))` is not in the data base, so the empty frame does not satisfy the pattern and is not filtered out of the initial stream of frames. The result of the query is thus the empty frame, which is used to instantiate the input query to produce `not(baseball_fan(list("Bitdiddle", "Ben")))`.

73. A discussion and justification of this treatment of `not` can be found in the article "Negation as Failure" by Clark (1978).

Exercise 4.63

Cy D. Fect, looking forward to the day when he will rise in the organization, gives a query to find all the wheels (using the `wheel` rule of section 4.4.1):

```
wheel($who)
```

To his surprise, the system responds

```
Query results:
wheel(list("Warbucks", "Oliver"))
wheel(list("Bitdiddle", "Ben"))
wheel(list("Warbucks", "Oliver"))
wheel(list("Warbucks", "Oliver"))
wheel(list("Warbucks", "Oliver"))
```

Why is Oliver Warbucks listed four times?

Exercise 4.64

Ben has been generalizing the query system to provide statistics about the company. For example, to find the total salaries of all the computer programmers one will be able to say

```
sum($amount,
    and(job($x, list("computer", "programmer")),
        salary($x, $amount)))
```

In general, Ben's new system allows expressions of the form

```
accumulation_function(variable,
                      query-pattern)
```

where `accumulation_function` can be things like `sum`, `average`, or `maximum`. Ben reasons that it should be a cinch to implement this. He will simply feed the query pattern to `evaluate_query`. This will produce a stream of frames. He will then pass this stream through a mapping function that extracts the value of the designated variable from each frame in the stream and feed the resulting stream of values to the accumulation function. Just as Ben completes the implementation and is about to try it out, Cy walks by, still puzzling over the `wheel` query result in exercise 4.63. When Cy shows Ben the system's response, Ben groans, "Oh, no, my simple accumulation scheme won't work!"

What has Ben just realized? Outline a method he can use to salvage the situation.

Exercise 4.65

Devise a way to install a loop detector in the query system so as to avoid the kinds of simple loops illustrated in the text and in exercise 4.62. The general idea is that the system should maintain some sort of history of its current chain of deductions and should not begin processing a query that it is already working on. Describe what kind of information (patterns and frames) is included in this history, and how the check should be made. (After you study the details of the query-system implementation in section 4.4.4, you may want to modify the system to include your loop detector.)

Exercise 4.66

Define rules to implement the `reverse` operation of exercise 2.18, which returns a list containing the same elements as a given list in reverse order. (Hint: Use `append_to_form`.) Can your rules answer both the query `reverse(list(1, 2, 3), $x)` and the query `reverse($x, list(1, 2, 3))`?

Exercise 4.67

Let us modify the data base and the rules of exercise 4.61 to add "great" to a grandson relationship. This should enable the system to deduce that Irad is the great-grandson of Adam, or that Jabal and Jubal are the great-great-great-great-great-grandsons of Adam.

a. Change the assertions in the data base such that there is only one kind of relationship information, namely `related`. The first item then describes the relationship. Thus, instead of `son("Adam", "Cain")`, you would write `related("son", "Adam", "Cain")`. Represent the fact about Irad, for example, as

```
related(list("great", "grandson"), "Adam", "Irad")
```

b. Write rules that determine if a list ends in the word `"grandson"`.

c. Use this to express a rule that allows one to derive the relationship

```
list(pair("great", $rel), $x, $y)
```

where `$rel` is a list ending in `"grandson"`.

d. Check your rules on the queries `related(list("great", "grandson"), $g, $ggs)` and `related($relationship, "Adam", "Irad")`.

4.4.4 Implementing the Query System

Section 4.4.2 described how the query system works. Now we fill in the details by presenting a complete implementation of the system.

4.4.4.1 The Driver Loop

The driver loop for the query system repeatedly reads input expressions. If the expression is a rule or assertion to be added to the data base, then the information is added. Otherwise the expression is assumed to be a query. The driver passes this query to `evaluate_query` together with an initial frame stream consisting of a single empty frame. The result of the evaluation is a stream of frames generated by satisfying the query with variable values found in the data base. These frames are used to form a new stream consisting of copies of the original query in which the variables are instantiated with values supplied by the stream of frames, and this final stream is displayed:

```
const input_prompt = "Query input:";
const output_prompt = "Query results:";

function query_driver_loop() {
    const input = user_read(input_prompt) + ";";
    if (is_null(input)) {
        display("evaluator terminated");
    } else {
        const expression = parse(input);
        const query = convert_to_query_syntax(expression);
        if (is_assertion(query)) {
            add_rule_or_assertion(assertion_body(query));
            display("Assertion added to data base.");
        } else {
            display(output_prompt);
            display_stream(
              stream_map(
                 frame =>
                   unparse(instantiate_expression(expression, frame)),
                 evaluate_query(query, singleton_stream(null))));
        }
        return query_driver_loop();
    }
}
```

Here, as in the other evaluators in this chapter, we use `parse` to transform a component of the query language given as a string into a JavaScript syntax representation. (We append a semicolon to the input expression string because `parse` expects a statement.) Then we further transform the syntax representation to a conceptual level appropriate for the query system using `convert_to_query_syntax`, which is declared in section 4.4.4.7 along with the predicate `is_assertion` and the selector `assertion_body`. The function `add_rule_or_assertion` is declared in section 4.4.4.5. The frames resulting from query evaluation are used to instantiate the syntax representation, and the result is unparsed into a string for display. The functions `instantiate_expression` and `unparse` are declared in section 4.4.4.7.

4.4.4.2 The Evaluator

The `evaluate_query` function, called by the `query_driver_loop`, is the basic evaluator of the query system. It takes as inputs a query and a stream of frames, and it returns a stream of extended frames. It identifies syntactic forms by a data-directed dispatch using `get` and `put`, just as we did in implementing generic operations in chapter 2. Any query that is not identified as a syntactic form is assumed to be a simple query, to be processed by `simple_query`.

```
function evaluate_query(query, frame_stream) {
   const qfun = get(type(query), "evaluate_query");
   return is_undefined(qfun)
          ? simple_query(query, frame_stream)
          : qfun(contents(query), frame_stream);
}
```

The functions `type` and `contents`, defined in section 4.4.4.7, implement the abstract syntax of the syntactic forms.

Simple queries

The `simple_query` function handles simple queries. It takes as arguments a simple query (a pattern) together with a stream of frames, and it returns the stream formed by extending each frame by all data-base matches of the query.

```
function simple_query(query_pattern, frame_stream) {
   return stream_flatmap(
              frame =>
                stream_append_delayed(
                    find_assertions(query_pattern, frame),
                    () => apply_rules(query_pattern, frame)),
              frame_stream);
}
```

For each frame in the input stream, we use `find_assertions` (section 4.4.4.3) to match the pattern against all assertions in the data base, producing a stream of extended frames, and we use `apply_rules` (section 4.4.4.4) to apply all possible rules, producing another stream of extended frames. These two streams are combined (using `stream_append_delayed`, section 4.4.4.6) to make a stream of all the ways that the given pattern can be satisfied consistent with the original frame (see exercise 4.68). The streams for the individual input frames are combined using `stream_flatmap` (section 4.4.4.6) to form one large stream of all the ways that any of the frames in the original input stream can be extended to produce a match with the given pattern.

Compound queries

We handle `and` queries as illustrated in figure 4.6 with the `conjoin` function, which takes as inputs the conjuncts and the frame stream and returns the stream of extended frames. First, `conjoin` processes the stream of frames to find the stream of all possible frame extensions that satisfy the first query in the conjunction. Then, using this as the new frame stream, it recursively applies `conjoin` to the rest of the queries.

```
function conjoin(conjuncts, frame_stream) {
   return is_empty_conjunction(conjuncts)
          ? frame_stream
          : conjoin(rest_conjuncts(conjuncts),
                    evaluate_query(first_conjunct(conjuncts),
                                   frame_stream));
}
```

The statement

```
put("and", "evaluate_query", conjoin);
```

sets up `evaluate_query` to dispatch to `conjoin` when an `and` is encountered.

We handle `or` queries similarly, as shown in figure 4.7. The output streams for the various disjuncts of the `or` are computed separately and merged using the `interleave_delayed` function from section 4.4.4.6. (See exercises 4.68 and 4.69.)

```
function disjoin(disjuncts, frame_stream) {
   return is_empty_disjunction(disjuncts)
          ? null
          : interleave_delayed(
               evaluate_query(first_disjunct(disjuncts), frame_stream),
               () => disjoin(rest_disjuncts(disjuncts), frame_stream));
}
put("or", "evaluate_query", disjoin);
```

The predicates and selectors for the representation of conjuncts and disjuncts are given in section 4.4.4.7.

Filters

The `not` syntactic form is handled by the method outlined in section 4.4.2. We attempt to extend each frame in the input stream to satisfy the query being negated, and we include a given frame in the output stream only if it cannot be extended.

```
function negate(exps, frame_stream) {
   return stream_flatmap(
              frame =>
                is_null(evaluate_query(negated_query(exps),
                                       singleton_stream(frame)))
                ? singleton_stream(frame)
                : null,
              frame_stream);
}
put("not", "evaluate_query", negate);
```

The `javascript_predicate` syntactic form is a filter similar to `not`. Each frame in the stream is used to instantiate the variables in the predicate, the instantiated predicate is evaluated, and the frames for which the predicate evaluates to false are filtered out of the input stream. The instantiated predicate is evaluated using `evaluate` from section 4.1 with `the_global_environment` and thus can handle any JavaScript expression, as long as all pattern variables are instantiated prior to evaluation.

```
function javascript_predicate(exps, frame_stream) {
   return stream_flatmap(
              frame =>
                evaluate(instantiate_expression(
                             javascript_predicate_expression(exps),
                             frame),
                         the_global_environment)
                ? singleton_stream(frame)
                : null,
              frame_stream);
}
put("javascript_predicate", "evaluate_query", javascript_predicate);
```

The `always_true` syntactic form provides for a query that is always satisfied. It ignores its contents (normally empty) and simply passes through all the frames in the input stream. The `rule_body` selector (section 4.4.4.7) uses `always_true` to provide bodies for rules that were defined without bodies (that is, rules whose bodies are always satisfied).

```
function always_true(ignore, frame_stream) {
    return frame_stream;
}
put("always_true", "evaluate_query", always_true);
```

The selectors that define the syntax of `not` and `javascript_predicate` are given in section 4.4.4.7.

4.4.4.3 Finding Assertions by Pattern Matching

The function `find_assertions`, called by `simple_query` (section 4.4.4.2), takes as input a pattern and a frame. It returns a stream of frames, each extending the given one by a data-base match of the given pattern. It uses `fetch_assertions` (section 4.4.4.5) to get a stream of all the assertions in the data base that should be checked for a match against the pattern and the frame. The reason for `fetch_assertions` here is that we can often apply simple tests that will eliminate many of the entries in the data base from the pool of candidates for a successful match. The system would still work if we eliminated `fetch_assertions` and simply checked a stream of all assertions in the data base, but the computation would be less efficient because we would need to make many more calls to the matcher.

```
function find_assertions(pattern, frame) {
    return stream_flatmap(
                datum => check_an_assertion(datum, pattern, frame),
                fetch_assertions(pattern, frame));
}
```

The function `check_an_assertion` takes as arguments a data object (an assertion), a pattern, and a frame and returns either a one-element stream containing the extended frame or `null` if the match fails.

```
function check_an_assertion(assertion, query_pat, query_frame) {
    const match_result = pattern_match(query_pat, assertion,
                                       query_frame);
    return match_result === "failed"
           ? null
           : singleton_stream(match_result);
}
```

The basic pattern matcher returns either the string `"failed"` or an extension of the given frame. The basic idea of the matcher is to check the pattern against the data, element by element, accumulating bindings for the pattern variables. If the pattern and the data object are the same, the match succeeds and we return the frame of bindings accumulated so far. Otherwise, if the pattern is a variable (checked by the function `is_variable` declared in section 4.4.4.7) we extend the current frame by

binding the variable to the data, so long as this is consistent with the bindings already in the frame. If the pattern and the data are both pairs, we (recursively) match the head of the pattern against the head of the data to produce a frame; in this frame we then match the tail of the pattern against the tail of the data. If none of these cases are applicable, the match fails and we return the string `"failed"`.

```
function pattern_match(pattern, data, frame) {
    return frame === "failed"
           ? "failed"
           : equal(pattern, data)
           ? frame
           : is_variable(pattern)
           ? extend_if_consistent(pattern, data, frame)
           : is_pair(pattern) && is_pair(data)
           ? pattern_match(tail(pattern),
                           tail(data),
                           pattern_match(head(pattern),
                                         head(data),
                                         frame))
           : "failed";
}
```

Here is the function that extends a frame by adding a new binding, if this is consistent with the bindings already in the frame:

```
function extend_if_consistent(variable, data, frame) {
    const binding = binding_in_frame(variable, frame);
    return is_undefined(binding)
           ? extend(variable, data, frame)
           : pattern_match(binding_value(binding), data, frame);
}
```

If there is no binding for the variable in the frame, we simply add the binding of the variable to the data. Otherwise we match, in the frame, the data against the value of the variable in the frame. If the stored value contains only constants, as it must if it was stored during pattern matching by `extend_if_consistent`, then the match simply tests whether the stored and new values are the same. If so, it returns the unmodified frame; if not, it returns a failure indication. The stored value may, however, contain pattern variables if it was stored during unification (see section 4.4.4.4). The recursive match of the stored pattern against the new data will add or check bindings for the variables in this pattern. For example, suppose we have a frame in which `$x` is bound to `list("f", $y)` and `$y` is unbound, and we wish to augment this frame by a binding of `$x` to `list("f", "b")`. We look up `$x` and find that it is bound to `list("f", $y)`. This leads us to match `list("f", $y)` against the proposed new value `list("f", "b")` in the same frame. Eventually this match extends the frame by adding a binding of `$y` to `"b"`. The variable `$x` remains bound to `list("f", $y)`. We never modify a stored binding and we never store more than one binding for a given variable.

The functions used by `extend_if_consistent` to manipulate bindings are defined in section 4.4.4.8.

4.4.4.4 Rules and Unification

The function `apply_rules` is the rule analog of `find_assertions` (section 4.4.4.3). It takes as input a pattern and a frame, and it forms a stream of extension frames by applying rules from the data base. The function `stream_flatmap` maps `apply_a_rule` down the stream of possibly applicable rules (selected by `fetch_rules`, section 4.4.4.5) and combines the resulting streams of frames.

```
function apply_rules(pattern, frame) {
    return stream_flatmap(rule => apply_a_rule(rule, pattern, frame),
                          fetch_rules(pattern, frame));
}
```

The function `apply_a_rule` applies a rule using the method outlined in section 4.4.2. It first augments its argument frame by unifying the rule conclusion with the pattern in the given frame. If this succeeds, it evaluates the rule body in this new frame.

Before any of this happens, however, the program renames all the variables in the rule with unique new names. The reason for this is to prevent the variables for different rule applications from becoming confused with each other. For instance, if two rules both use a variable named `$x`, then each one may add a binding for `$x` to the frame when it is applied. These two `$x`'s have nothing to do with each other, and we should not be fooled into thinking that the two bindings must be consistent. Rather than rename variables, we could devise a more clever environment structure; however, the renaming approach we have chosen here is the most straightforward, even if not the most efficient. (See exercise 4.76.) Here is the `apply_a_rule` function:

```
function apply_a_rule(rule, query_pattern, query_frame) {
    const clean_rule = rename_variables_in(rule);
    const unify_result = unify_match(query_pattern,
                                     conclusion(clean_rule),
                                     query_frame);
    return unify_result === "failed"
           ? null
           : evaluate_query(rule_body(clean_rule),
                            singleton_stream(unify_result));
}
```

The selectors `rule_body` and `conclusion` that extract parts of a rule are defined in section 4.4.4.7.

We generate unique variable names by associating a unique identifier (such as a number) with each rule application and combining this identifier with the original variable names. For example, if the rule-application identifier is 7, we might change each `$x` in the rule to `$x_7` and each `$y` in the rule to `$y_7`. (The functions `make_new_variable` and `new_rule_application_id` are included with the syntax functions in section 4.4.4.7.)

```
function rename_variables_in(rule) {
    const rule_application_id = new_rule_application_id();
    function tree_walk(exp) {
        return is_variable(exp)
               ? make_new_variable(exp, rule_application_id)
               : is_pair(exp)
               ? pair(tree_walk(head(exp)),
                      tree_walk(tail(exp)))
               : exp;
    }
    return tree_walk(rule);
}
```

The unification algorithm is implemented as a function that takes as inputs two patterns and a frame and returns either the extended frame or the string `"failed"`. The unifier is like the pattern matcher except that it is symmetrical—variables are allowed on both sides of the match. The function `unify_match` is basically the same as `pattern_match`, except that there is an extra clause (marked "***" below) to handle the case where the object on the right side of the match is a variable.

```
function unify_match(p1, p2, frame) {
    return frame === "failed"
           ? "failed"
           : equal(p1, p2)
           ? frame
           : is_variable(p1)
           ? extend_if_possible(p1, p2, frame)
           : is_variable(p2)                    // ***
           ? extend_if_possible(p2, p1, frame)  // ***
           : is_pair(p1) && is_pair(p2)
           ? unify_match(tail(p1),
                         tail(p2),
                         unify_match(head(p1),
                                     head(p2),
                                     frame))
           : "failed";
}
```

In unification, as in one-sided pattern matching, we want to accept a proposed extension of the frame only if it is consistent with existing bindings. The function `extend_if_possible` used in unification is the same as the function `extend_if_consistent` used in pattern matching except for two special checks, marked "***" in the program below. In the first case, if the variable we are trying to match is not bound, but the value we are trying to match it with is itself a (different) variable, it is necessary to check to see if the value is bound, and if so, to match its value. If both parties to the match are unbound, we may bind either to the other.

The second check deals with attempts to bind a variable to a pattern that includes that variable. Such a situation can occur whenever a variable is repeated in both patterns. Consider, for example, unifying the two patterns `list($x, $x)` and `list($y,` ⟨*expression involving* `$y`⟩`)` in a frame where both `$x` and `$y` are unbound. First `$x` is matched against `$y`, making a binding of `$x` to `$y`. Next, the same `$x` is matched against the given expression involving `$y`. Since `$x` is already bound to `$y`, this results in matching `$y` against the expression. If we think of the unifier as finding a set of values for the pattern variables that make the patterns the same, then these patterns imply instructions to find a `$y` such that `$y` is equal to the expression involving `$y`. We reject such bindings; these cases are recognized by the predicate `depends_on`.[74] On the other hand, we do not want to reject attempts to bind a variable to itself. For example, consider unifying `list($x, $x)` and `list($y, $y)`. The second attempt to bind `$x` to `$y` matches `$y` (the stored value of `$x`) against `$y` (the new value of `$x`). This is taken care of by the `equal` clause of `unify_match`.

---

74. In general, unifying `$y` with an expression involving `$y` would require our being able to find a fixed point of the equation `$y` = ⟨*expression involving* `$y`⟩. It is sometimes possible to syntactically form an expression that appears to be the solution. For example, `$y = list("f", $y)` seems to have the fixed point `list("f", list("f", list("f", ... )))`, which we can produce by beginning with the expression `list("f", $y)` and repeatedly substituting `list("f", $y)` for `$y`. Unfortunately, not every such equation has a meaningful fixed point. The issues that arise here are similar to the issues of manipulating infinite series in mathematics. For example, we know that 2 is the solution to the equation $y = 1 + y/2$. Beginning with the expression $1 + y/2$ and repeatedly substituting $1 + y/2$ for y gives

$$2 = y = 1 + y/2 = 1 + (1 + y/2)/2 = 1 + 1/2 + y/4 = \cdots ,$$

which leads to

$$2 = 1 + 1/2 + 1/4 + 1/8 + \cdots .$$

However, if we try the same manipulation beginning with the observation that –1 is the solution to the equation $y = 1 + 2y$, we obtain

$$-1 = y = 1 + 2y = 1 + 2(1 + 2y) = 1 + 2 + 4y = \cdots ,$$

which leads to

$$-1 = 1 + 2 + 4 + 8 + \cdots .$$

Although the formal manipulations used in deriving these two equations are identical, the first result is a valid assertion about infinite series but the second is not. Similarly, for our unification results, reasoning with an arbitrary syntactically constructed expression may lead to errors.

Nevertheless, most logic programming systems today allow cyclic references, by accepting the cyclic data structure as the result of the match. This is justified theoretically using *rational trees* (Jaffar and Stuckey 1986). Accepting a cyclic data structure allows self-referential data, such as an employee data structure that refers to the employer, which in turn refers to the employee.

```
function extend_if_possible(variable, value, frame) {
    const binding = binding_in_frame(variable, frame);
    if (! is_undefined(binding)) {
        return unify_match(binding_value(binding),
                           value, frame);
    } else if (is_variable(value)) {                 // ***
        const binding = binding_in_frame(value, frame);
        return ! is_undefined(binding)
               ? unify_match(variable,
                             binding_value(binding),
                             frame)
               : extend(variable, value, frame);
    } else if (depends_on(value, variable, frame)) { // ***
        return "failed";
    } else {
        return extend(variable, value, frame);
    }
}
```

The function `depends_on` is a predicate that tests whether an expression proposed to be the value of a pattern variable depends on the variable. This must be done relative to the current frame because the expression may contain occurrences of a variable that already has a value that depends on our test variable. The structure of `depends_on` is a simple recursive tree walk in which we substitute for the values of variables whenever necessary.

```
function depends_on(expression, variable, frame) {
    function tree_walk(e) {
        if (is_variable(e)) {
            if (equal(variable, e)) {
                return true;
            } else {
                const b = binding_in_frame(e, frame);
                return is_undefined(b)
                       ? false
                       : tree_walk(binding_value(b));
            }
        } else {
            return is_pair(e)
                   ? tree_walk(head(e)) || tree_walk(tail(e))
                   : false;
        }
    }
    return tree_walk(expression);
}
```

4.4.4.5 Maintaining the Data Base

One important problem in designing logic programming languages is that of arranging things so that as few irrelevant data-base entries as possible will be examined in checking a given pattern. For this purpose, we will represent an assertion as a list whose head is a string that represents the kind of information of the assertion. We store the assertions in separate streams, one for each kind of information, in a table indexed by the kind. To fetch an assertion that may match a pattern, we return (to be tested using the matcher) all the stored assertions that have the same head (the same kind of information). Cleverer methods could also take advantage of information in the frame. We avoid building our criteria for indexing into the program; instead we call on predicates and selectors that embody our criteria.

```
function fetch_assertions(pattern, frame) {
    return get_indexed_assertions(pattern);
}
function get_indexed_assertions(pattern) {
    return get_stream(index_key_of(pattern), "assertion-stream");
}
```

The function `get_stream` looks up a stream in the table and returns an empty stream if nothing is stored there.

```
function get_stream(key1, key2) {
    const s = get(key1, key2);
    return is_undefined(s) ? null : s;
}
```

Rules are stored similarly, using the head of the rule conclusion. A pattern can match rules whose conclusions have the same head. Thus, when fetching rules that might match a pattern we fetch all rules whose conclusions have the same head as the pattern.

```
function fetch_rules(pattern, frame) {
    return get_indexed_rules(pattern);
}
function get_indexed_rules(pattern) {
    return get_stream(index_key_of(pattern), "rule-stream");
}
```

The function `add_rule_or_assertion` is used by `query_driver_loop` to add assertions and rules to the data base. Each item is stored in the index.

```
function add_rule_or_assertion(assertion) {
    return is_rule(assertion)
           ? add_rule(assertion)
           : add_assertion(assertion);
}
function add_assertion(assertion) {
    store_assertion_in_index(assertion);
    return "ok";
}
function add_rule(rule) {
    store_rule_in_index(rule);
    return "ok";
}
```

To actually store an assertion or a rule, we store it in the appropriate stream.

```
function store_assertion_in_index(assertion) {
    const key = index_key_of(assertion);
    const current_assertion_stream =
                get_stream(key, "assertion-stream");
    put(key, "assertion-stream",
        pair(assertion, () => current_assertion_stream));
}
function store_rule_in_index(rule) {
    const pattern = conclusion(rule);
    const key = index_key_of(pattern);
    const current_rule_stream =
                get_stream(key, "rule-stream");
    put(key, "rule-stream",
        pair(rule, () => current_rule_stream));
}
```

The key under which a pattern (an assertion or rule conclusion) is stored in the table is the string it starts with.

```
function index_key_of(pattern) { return head(pattern); }
```

4.4.4.6 Stream Operations

The query system uses a few stream operations that were not presented in chapter 3.

The functions `stream_append_delayed` and `interleave_delayed` are just like `stream_append` and `interleave` (section 3.5.3), except that they take a delayed argument (like the `integral` function in section 3.5.4). This postpones looping in some cases (see exercise 4.68).

```
function stream_append_delayed(s1, delayed_s2) {
    return is_null(s1)
           ? delayed_s2()
           : pair(head(s1),
                  () => stream_append_delayed(stream_tail(s1),
                                              delayed_s2));
}
function interleave_delayed(s1, delayed_s2) {
    return is_null(s1)
           ? delayed_s2()
           : pair(head(s1),
                  () => interleave_delayed(delayed_s2(),
                                           () => stream_tail(s1)));
}
```

The function `stream_flatmap`, which is used throughout the query evaluator to map a function over a stream of frames and combine the resulting streams of frames, is the stream analog of the `flatmap` function introduced for ordinary lists in section 2.2.3. Unlike ordinary `flatmap`, however, we accumulate the streams with an interleaving process, rather than simply appending them (see exercises 4.69 and 4.70).

```
function stream_flatmap(fun, s) {
    return flatten_stream(stream_map(fun, s));
}
function flatten_stream(stream) {
    return is_null(stream)
           ? null
           : interleave_delayed(
                 head(stream),
                 () => flatten_stream(stream_tail(stream)));
}
```

The evaluator also uses the following simple function to generate a stream consisting of a single element:

```
function singleton_stream(x) {
    return pair(x, () => null);
}
```

4.4.4.7 Query Syntax Functions and Instantiation

We saw in section 4.4.4.1 that the driver loop first transforms an input string into the JavaScript syntax representation. The input is designed to look like a JavaScript expression so that we can use the `parse` function from section 4.1.2 and also to support JavaScript notation in `javascript_predicate`. For example,

```
parse('job($x, list("computer", "wizard"));');
```

yields

```
list("application",
     list("name", "job"),
     list(list("name", "$x"),
          list("application",
               list("name", "list"),
               list(list("literal", "computer"),
                    list("literal", "wizard")))))
```

The tag `"application"` indicates that syntactically, the query would be treated as a function application in JavaScipt. The function `unparse` transforms the syntax back into a string:

```
unparse(parse('job($x, list("computer", "wizard"));'));
'job($x, list("computer", "wizard"))'
```

In the query processor, we assumed a query-language-specific representation of assertions, rules, and queries. The function `convert_to_query_syntax` transforms the syntax representation into that representation. Using the same example,

```
convert_to_query_syntax(parse('job($x, list("computer", "wizard"));'));
```

yields

```
list("job", list("name", "$x"), list("computer", "wizard"))
```

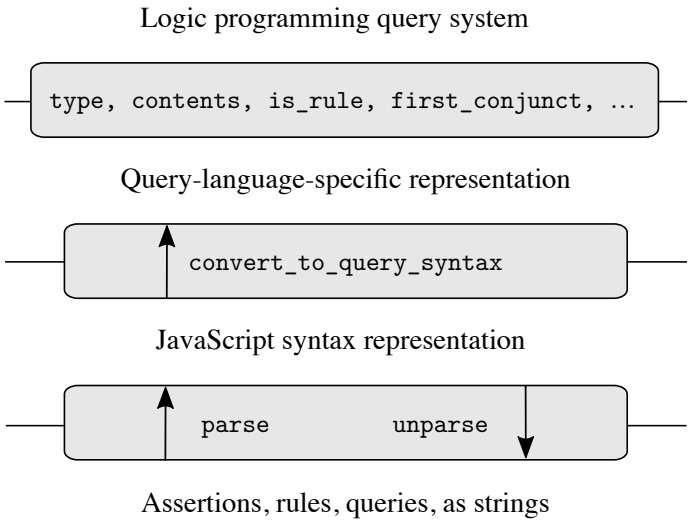


Figure 4.8 Syntax abstraction in the query system.

Query-system functions such as `add_rule_or_assertion` in section 4.4.4.5 and `evaluate_query` in section 4.4.4.2 operate on the query-language-specific representation using selectors and predicates such as `type`, `contents`, `is_rule`, and `first_conjunct` declared below. Figure 4.8 depicts the three abstraction barriers used by the query system and how the transformation functions `parse`, `unparse`, and `convert_to_query_syntax` bridge them.

Handling pattern variables

The predicate `is_variable` is used on the query-language-specific representation during query processing and on the JavaScript syntax representation during instantiation to identify names that start with a dollar sign. We assume there is a function `char_at` that returns a string containing only the character of the given string at the given position.[75]

```
function is_variable(exp) {
    return is_name(exp) && char_at(symbol_of_name(exp), 0) === "$";
}
```

Unique variables are constructed during rule application (in section 4.4.4.4) by means of the following functions. The unique identifier for a rule application is a number, which is incremented each time a rule is applied.[76]

---

75. The actual way to get the string that contains the first character of a string `s` in JavaScript is `s.charAt(0)`.

76. Creating new variables with string concatenation and identifying variables by checking their first character during query processing is somewhat wasteful. A more efficient solution would mark pattern variables with a separate tag in the query-language-specific representation and use pair construction rather than string concatenation to create new variables. We chose the less efficient solution to simplify the presentation.

```
let rule_counter = 0;

function new_rule_application_id() {
   rule_counter = rule_counter + 1;
   return rule_counter;
}
function make_new_variable(variable, rule_application_id) {
   return make_name(symbol_of_name(variable) + "_" +
                    stringify(rule_application_id));
}
```

The function `convert_to_query_syntax`

The function `convert_to_query_syntax` recursively transforms the JavaScript syntax representation into the query-language-specific representation by simplifying assertions, rules, and queries such that the symbol of a name in a function expression of an application becomes a tag, except that if the symbol is `"pair"` or `"list"`, an (untagged) JavaScript pair or list is built. This means that `convert_to_query_syntax` interprets applications of the constructors `pair` and `list` during the transformation, and processing functions such as `pattern_match` of section 4.4.4.3 and `unify_match` of section 4.4.4.4 can operate directly on the intended pairs and lists rather than on the syntax representation generated by the parser. The (one-element) "argument" list of `javascript_predicate` remains unprocessed, as explained below. A variable remains unchanged, and a literal is simplified to the primitive value it contains.

```
function convert_to_query_syntax(exp) {
   if (is_application(exp)) {
     const function_symbol = symbol_of_name(function_expression(exp));
     if (function_symbol === "javascript_predicate") {
       return pair(function_symbol, arg_expressions(exp));
     } else {
       const processed_args = map(convert_to_query_syntax,
                                  arg_expressions(exp));
       return function_symbol === "pair"
              ? pair(head(processed_args), head(tail(processed_args)))
              : function_symbol === "list"
              ? processed_args
              : pair(function_symbol, processed_args);
     }
   } else if (is_variable(exp)) {
     return exp;
   } else { // exp is literal
     return literal_value(exp);
   }
}
```

An exception to this processing is `javascript_predicate`. Since the instantiated JavaScript syntax representation of its predicate expression is passed to `evaluate` of section 4.1.1, the original syntax representation coming from `parse`

needs to remain intact in the query-language-specific representation of the expression. In this example of section 4.4.1

```
and(salary($person, $amount), javascript_predicate($amount > 50000))
```

`convert_to_query_syntax` produces a data structure in which a JavaScript syntax representation is embedded in a query-language-specific representation:

```
list("and",
     list("salary", list("name", "$person"), list("name", "$amount")),
     list("javascript_predicate",
          list("binary_operator_combination",
               ">",
               list("name", "$amount"),
               list("literal", 50000))))
```

In order to evaluate the `javascript_predicate` subexpression of that processed query, the function `javascript_predicate` in section 4.4.4.2 calls the function `instantiate_expression` (below) on the embedded JavaScript syntax representation of `$amount > 50000` to replace the variable `list("name", "$amount")` by a literal, for example `list("literal", 70000)`, that represents the primitive value to which `$amount` is bound, here 70000. The JavaScript evaluator can evaluate the instantiated predicate, which now represents `70000 > 50000`.

Instantiating an expression

The function `javascript_predicate` of section 4.4.4.2 and the driver loop of section 4.4.4.1 call `instantiate_expression` on an expression to obtain a copy in which any variable in the expression is replaced by its value in a given frame. The input and result expressions use the JavaScript syntax representation, so any value that results from instantiating a variable needs to be converted from its form in the binding to the JavaScript syntax representation.

```
function instantiate_expression(expression, frame) {
   return is_variable(expression)
          ? convert(instantiate_term(expression, frame))
          : is_pair(expression)
          ? pair(instantiate_expression(head(expression), frame),
                 instantiate_expression(tail(expression), frame))
          : expression;
}
```

The function `instantiate_term` takes a variable, pair, or primitive value as first argument and a frame as second argument and recursively replaces the variables in the first argument by their values in the frame until a primitive value or an unbound variable is reached. When the process encounters a pair, a new pair is constructed whose parts are the instantiated versions of the original parts. For example, if `$x` is bound to the pair [`$y`, 5] in a frame f as the result of unification, and `$y` is in turn bound to 3, the result of applying `instantiate_term` to `list("name", "$x")` and f is the pair [3, 5].

```
function instantiate_term(term, frame) {
    if (is_variable(term)) {
        const binding = binding_in_frame(term, frame);
        return is_undefined(binding)
               ? term  // leave unbound variable as is
               : instantiate_term(binding_value(binding), frame);
    } else if (is_pair(term)) {
        return pair(instantiate_term(head(term), frame),
                    instantiate_term(tail(term), frame));
    } else { // term is a primitive value
        return term;
    }
}
```

The function `convert` constructs a JavaScript syntax representation for a variable, pair, or primitive value returned by `instantiate_term`. A pair in the original becomes an application of JavaScript's pair constructor and a primitive value becomes a literal.

```
function convert(term) {
    return is_variable(term)
           ? term
           : is_pair(term)
           ? make_application(make_name("pair"),
                              list(convert(head(term)),
                                   convert(tail(term))))
           : // term is a primitive value
             make_literal(term);
}
```

To illustrate these three functions, consider what happens when the query

```
job($x, list("computer", "wizard"))
```

whose JavaScript syntax representation is given at the beginning of section 4.4.4.7, is processed by the driver loop. Let's say a frame *g* of the result stream binds the variable `$x` to the pair `["Bitdiddle", $y]` and the variable `$y` to the pair `["Ben", null]`. Then

```
instantiate_term(list("name", "$x"), g)
```

returns the list

```
list("Bitdiddle", "Ben")
```

which `convert` transforms into

```
list("application",
     list("name", "pair"),
     list(list("literal", "Bitdiddle"),
          list("application",
               list("name", "pair"),
               list(list("literal", "Ben"),
                    list("literal", null)))))
```

The result of `instantiate_expression` applied to the JavaScript syntax representation of the query and the frame *g* is:

```
list("application",
     list("name", "job"),
     list(list("application",
               list("name", "pair"),
               list(list("literal", "Bitdiddle"),
                    list("application",
                         list("name", "pair"),
                         list(list("literal", "Ben"),
                              list("literal", null))))),
          list("application",
               list("name", "list"),
               list(list("literal", "computer"),
                    list("literal", "wizard")))))
```

The driver loop unparses this representation and displays it as:

```
'job(list("Bitdiddle", "Ben"), list("computer", "wizard"))'
```

The function `unparse`

The function `unparse` transforms a component given in the JavaScript syntax representation into a string by applying the syntax rules of section 4.1.2. We describe `unparse` only for those kinds of expressions that appear in the examples of section 4.4.1, leaving statements and the remaining kinds of expressions as exercise 4.2. A literal is transformed by `stringify`ing its value, and a name is transformed into its symbol. An application is formatted by unparsing the function expression, which we can assume to be a name here, followed by the comma-separated argument expression strings enclosed in parentheses. Binary operator combinations are formatted using infix notation.

```
function unparse(exp) {
    return is_literal(exp)
           ? stringify(literal_value(exp))
           : is_name(exp)
           ? symbol_of_name(exp)
           : is_list_construction(exp)
           ? unparse(make_application(make_name("list"),
                                      element_expressions(exp)))
           : is_application(exp) && is_name(function_expression(exp))
           ? symbol_of_name(function_expression(exp)) +
                "(" +
                comma_separated(map(unparse, arg_expressions(exp))) +
                ")"
           : is_binary_operator_combination(exp)
           ? "(" + unparse(first_operand(exp)) +
             " " + operator_symbol(exp) +
             " " + unparse(second_operand(exp)) +
             ")"
           ⟨unparsing other kinds of JavaScript components⟩
           : error(exp, "unknown syntax -- unparse");
}
```

```
function comma_separated(strings) {
   return accumulate((s, acc) => s + (acc === "" ? "" : ", " + acc),
                     "",
                     strings);
}
```

The function `unparse` would work fine without the clause

```
: is_list_construction(exp)
? unparse(make_application(make_name("list"),
                           element_expressions(exp)))
```

but the output string would be unnecessarily verbose in cases where pattern variables are instantiated by lists. In the example above, where processing the query

```
job($x, list("computer", "wizard"))
```

yields a frame that binds `$x` to `["Bitdiddle", ["Ben", null]]`, `unparse` produces

```
'job(list("Bitdiddle", "Ben"), list("computer", "wizard"))'
```

However, without the clause it would produce

```
'job(pair("Bitdiddle", pair("Ben", null)), list("computer", "wizard"))'
```

which explicitly constructs the two pairs that make up the first list. To achieve the more concise formatting used throughout section 4.4.1, we inserted the clause to check if the expression constructs a list, in which case we format it as a single application of `list` to the list of element expressions that we extract from the expression. A list construction is the literal **`null`** or an application of `pair` whose second argument is itself a list construction.

```
function is_list_construction(exp) {
   return (is_literal(exp) && is_null(literal_value(exp))) ||
          (is_application(exp) && is_name(function_expression(exp)) &&
           symbol_of_name(function_expression(exp)) === "pair" &&
           is_list_construction(head(tail(arg_expressions(exp)))));
}
```

Extracting the element expressions from a given list construction amounts to collecting the first arguments of applications of `pair` until the literal **`null`** is reached.

```
function element_expressions(list_constr) {
   return is_literal(list_constr)
          ? null // list_constr is literal null
          :      // list_constr is application of pair
            pair(head(arg_expressions(list_constr)),
                 element_expressions(
                     head(tail(arg_expressions(list_constr)))));
}
```

Predicates and selectors for the query-language-specific representation

The functions `type` and `contents`, used by `evaluate_query` (section 4.4.4.2), specify that a syntactic form of a query-language-specific representation is identified by the string in its head. They are the same as the `type_tag` and `contents` functions in section 2.4.2, except for the error message.

```
function type(exp) {
   return is_pair(exp)
          ? head(exp)
          : error(exp, "unknown expression type");
}
function contents(exp) {
   return is_pair(exp)
          ? tail(exp)
          : error(exp, "unknown expression contents");
}
```

The following functions, used by `query_driver_loop` (in section 4.4.4.1), specify that rules and assertions are added to the data base by an `assert` command, which the function `convert_to_query_syntax` transforms into a pair of the form `["assert",` *rule-or-assertion*`]`:

```
function is_assertion(exp) {
   return type(exp) === "assert";
}
function assertion_body(exp) { return head(contents(exp)); }
```

Here are the declarations of the predicates and selectors for the `and`, `or`, `not`, and `javascript_predicate` syntactic forms (section 4.4.4.2):

```
function is_empty_conjunction(exps) { return is_null(exps); }
function first_conjunct(exps) { return head(exps); }
function rest_conjuncts(exps) { return tail(exps); }

function is_empty_disjunction(exps) { return is_null(exps); }
function first_disjunct(exps) { return head(exps); }
function rest_disjuncts(exps) { return tail(exps); }

function negated_query(exps) { return head(exps); }

function javascript_predicate_expression(exps) { return head(exps); }
```

The following three functions define the query-language-specific representation of rules:

```
function is_rule(assertion) {
    return is_tagged_list(assertion, "rule");
}
function conclusion(rule) { return head(tail(rule)); }
function rule_body(rule) {
    return is_null(tail(tail(rule)))
           ? list("always_true")
           : head(tail(tail(rule)));
}
```

4.4.4.8 Frames and Bindings

Frames are represented as lists of bindings, which are variable-value pairs:

```
function make_binding(variable, value) {
    return pair(variable, value);
}
function binding_variable(binding) {
    return head(binding);
}
function binding_value(binding) {
    return tail(binding);
}
function binding_in_frame(variable, frame) {
    return assoc(variable, frame);
}
function extend(variable, value, frame) {
    return pair(make_binding(variable, value), frame);
}
```

Exercise 4.68

Louis Reasoner wonders why the `simple_query` and `disjoin` functions (section 4.4.4.2) are implemented using delayed expressions rather than being defined as follows:

```
function simple_query(query_pattern, frame_stream) {
    return stream_flatmap(
               frame =>
                 stream_append(find_assertions(query_pattern, frame),
                               apply_rules(query_pattern, frame)),
               frame_stream);
}
function disjoin(disjuncts, frame_stream) {
    return is_empty_disjunction(disjuncts)
           ? null
           : interleave(
                 evaluate_query(first_disjunct(disjuncts), frame_stream),
                 disjoin(rest_disjuncts(disjuncts), frame_stream));
}
```

Can you give examples of queries where these simpler definitions would lead to undesirable behavior?

Exercise 4.69

Why do `disjoin` and `stream_flatmap` interleave the streams rather than simply append them? Give examples that illustrate why interleaving works better. (Hint: Why did we use `interleave` in section 3.5.3?)

Exercise 4.70

Why does `flatten_stream` use a delayed expression in its body? What would be wrong with defining it as follows:

```
function flatten_stream(stream) {
    return is_null(stream)
           ? null
           : interleave(head(stream),
                        flatten_stream(stream_tail(stream)));
}
```

Exercise 4.71

Alyssa P. Hacker proposes to use a simpler version of `stream_flatmap` in `negate`, `javascript_predicate`, and `find_assertions`. She observes that the function that is mapped over the frame stream in these cases always produces either the empty stream or a singleton stream, so no interleaving is needed when combining these streams.

a. Fill in the missing expressions in Alyssa's program.

```
function simple_stream_flatmap(fun, s) {
    return simple_flatten(stream_map(fun, s));
}
function simple_flatten(stream) {
    return stream_map(⟨??⟩,
                      stream_filter(⟨??⟩, stream));
}
```

b. Does the query system's behavior change if we change it in this way?

Exercise 4.72

Implement for the query language a syntactic form called `unique`. Applictions of `unique` should succeed if there is precisely one item in the data base satisfying a specified query. For example,

```
unique(job($x, list("computer", "wizard")))
```

should print the one-item stream

```
unique(job(list("Bitdiddle", "Ben"), list("computer", "wizard")))
```

since Ben is the only computer wizard, and

```
unique(job($x, list("computer", "programmer")))
```

should print the empty stream, since there is more than one computer programmer. Moreover,

```
and(job($x, $j), unique(job($anyone, $j)))
```

should list all the jobs that are filled by only one person, and the people who fill them.

There are two parts to implementing `unique`. The first is to write a function that handles this syntactic form, and the second is to make `evaluate_query` dispatch to that function. The second part is trivial, since `evaluate_query` does its dispatching in a data-directed way. If your function is called `uniquely_asserted`, all you need to do is

```
put("unique", "evaluate_query", uniquely_asserted);
```

and `evaluate_query` will dispatch to this function for every query whose `type` (head) is the string `"unique"`.

The real problem is to write the function `uniquely_asserted`. This should take as input the `contents` (tail) of the `unique` query, together with a stream of frames. For each frame in the stream, it should use `evaluate_query` to find the stream of all extensions to the frame that satisfy the given query. Any stream that does not have exactly one item in it should be eliminated. The remaining streams should be passed back to be accumulated into one big stream that is the result of the `unique` query. This is similar to the implementation of the `not` syntactic form.

Test your implementation by forming a query that lists all people who supervise precisely one person.

Exercise 4.73

Our implementation of `and` as a series combination of queries (figure 4.6) is elegant, but it is inefficient because in processing the second query of the `and` we must scan the data base for each frame produced by the first query. If the data base has N elements, and a typical query produces a number of output frames proportional to N (say N/k), then scanning the data base for each frame produced by the first query will require N^2/k calls to the pattern matcher. Another approach would be to process the two clauses of the `and` separately, then look for all pairs of output frames that are compatible. If each query produces N/k output frames, then this means that we must perform N^2/k^2 compatibility checks—a factor of k fewer than the number of matches required in our current method.

Devise an implementation of `and` that uses this strategy. You must implement a function that takes two frames as inputs, checks whether the bindings in the frames are compatible, and, if so, produces a frame that merges the two sets of bindings. This operation is similar to unification.

Exercise 4.74

In section 4.4.3 we saw that `not` and `javascript_predicate` can cause the query language to give "wrong" answers if these filtering operations are applied to frames in which variables are unbound. Devise a way to fix this shortcoming. One idea is to perform the filtering in a "delayed" manner by appending to the frame a "promise" to filter that is fulfilled only when enough variables have been bound to make the operation possible. We could wait to perform filtering until all other operations have been performed. However, for efficiency's sake, we would like to perform filtering as soon as possible so as to cut down on the number of intermediate frames generated.

Exercise 4.75

Redesign the query language as a nondeterministic program to be implemented using the evaluator of section 4.3, rather than as a stream process. In this approach, each query will produce a single answer (rather than the stream of all answers) and the user can type `retry` to see more answers. You should find that much of the mechanism we built in this section is subsumed by nondeterministic search and backtracking. You will probably also find, however, that your new query language has subtle differences in behavior from the one implemented here. Can you find examples that illustrate this difference?

Exercise 4.76

When we implemented the JavaScript evaluator in section 4.1, we saw how to use local environments to avoid name conflicts between the parameters of functions. For example, in evaluating

```
function square(x) {
    return x * x;
}
function sum_of_squares(x, y) {
    return square(x) + square(y);
}
sum_of_squares(3, 4);
```

there is no confusion between the `x` in `square` and the `x` in `sum_of_squares`, because we evaluate the body of each function in an environment that is specially constructed to contain bindings for the local names. In the query system, we used a different strategy to avoid name conflicts in applying rules. Each time we apply a rule we rename the variables with new names that are guaranteed to be unique. The analogous strategy for the JavaScript evaluator would be to do away with local environments and simply rename the variables in the body of a function each time we apply the function.

Implement for the query language a rule-application method that uses environments rather than renaming. See if you can build on your environment structure to create constructs in the query language for dealing with large systems, such as the rule analog of block-structured functions. Can you relate any of this to the problem of making deductions in a context (e.g., "If I supposed that *P* were true, then I would be able to deduce *A* and *B*.") as a method of problem solving? (This problem is open-ended.)

5 Computing with Register Machines

> My aim is to show that the heavenly machine is not a kind of divine, live being, but a kind of clockwork (and he who believes that a clock has soul attributes the maker's glory to the work), insofar as nearly all the manifold motions are caused by a most simple and material force, just as all motions of the clock are caused by a single weight.
>
> —Johannes Kepler (letter to Herwart von Hohenburg, 1605)

We began this book by studying processes and by describing processes in terms of functions written in JavaScript. To explain the meanings of these functions, we used a succession of models of evaluation: the substitution model of chapter 1, the environment model of chapter 3, and the metacircular evaluator of chapter 4. Our examination of the metacircular evaluator, in particular, dispelled much of the mystery of how JavaScript-like languages are interpreted. But even the metacircular evaluator leaves important questions unanswered, because it fails to elucidate the mechanisms of control in a JavaScript system. For instance, the evaluator does not explain how the evaluation of a subexpression manages to return a value to the expression that uses this value. Also, the evaluator does not explain how some recursive functions can generate iterative processes (that is, be evaluated using constant space) whereas other recursive functions will generate recursive processes.[1] This chapter addresses both of these issues.

We will describe processes in terms of the step-by-step operation of a traditional computer. Such a computer, or *register machine*, sequentially executes *instructions* that manipulate the contents of a fixed set of storage elements called *registers*. A typical register-machine instruction applies a primitive operation to the contents of some registers and assigns the result to another register. Our descriptions of processes executed by register machines will look very much like "machine-language" programs for traditional computers. However, instead of focusing on the machine language of any particular computer, we will examine several JavaScript functions and design a specific register machine to execute each function. Thus, we will approach our task from the perspective of a hardware architect rather than that of a machine-language computer programmer. In designing register machines, we will develop mechanisms for implementing important programming constructs such as recursion. We will also present a language for describing designs for register machines. In section 5.2 we will implement a JavaScript program that uses these descriptions to simulate the machines we design.

Most of the primitive operations of our register machines are very simple. For example, an operation might add the numbers fetched from two registers, producing

---

1. With our metacircular evaluator, a recursive function always gives rise to a recursive process, even when the process should be iterative according to the distinction of section 1.2.1. See footnote 6 in section 4.1.1.

a result to be stored into a third register. Such an operation can be performed by easily described hardware. In order to deal with list structure, however, we will also use the memory operations `head`, `tail`, and `pair`, which require an elaborate storage-allocation mechanism. In section 5.3 we study their implementation in terms of more elementary operations.

In section 5.4, after we have accumulated experience formulating simple functions as register machines, we will design a machine that carries out the algorithm described by the metacircular evaluator of section 4.1. This will fill in the gap in our understanding of how JavaScript programs are interpreted, by providing an explicit model for the mechanisms of control in the evaluator. In section 5.5 we will study a simple compiler that translates JavaScript programs into sequences of instructions that can be executed directly with the registers and operations of the evaluator register machine.

5.1 Designing Register Machines

To design a register machine, we must design its *data paths* (registers and operations) and the *controller* that sequences these operations. To illustrate the design of a simple register machine, let us examine Euclid's Algorithm, which is used to compute the greatest common divisor (GCD) of two integers. As we saw in section 1.2.5, Euclid's Algorithm can be carried out by an iterative process, as specified by the following function:

```
function gcd(a, b) {
    return b === 0 ? a : gcd(b, a % b);
}
```

A machine to carry out this algorithm must keep track of two numbers, a and b, so let us assume that these numbers are stored in two registers with those names. The basic operations required are testing whether the contents of register `b` is zero and computing the remainder of the contents of register `a` divided by the contents of register `b`. The remainder operation is a complex process, but assume for the moment that we have a primitive device that computes remainders. On each cycle of the GCD algorithm, the contents of register `a` must be replaced by the contents of register `b`, and the contents of `b` must be replaced by the remainder of the old contents of `a` divided by the old contents of `b`. It would be convenient if these replacements could be done simultaneously, but in our model of register machines we will assume that only one register can be assigned a new value at each step. To accomplish the replacements, our machine will use a third "temporary" register, which we call `t`. (First the remainder will be placed in `t`, then the contents of `b` will be placed in `a`, and finally the remainder stored in `t` will be placed in `b`.)

We can illustrate the registers and operations required for this machine by using the data-path diagram shown in figure 5.1. In this diagram, the registers (`a`, `b`, and `t`) are represented by rectangles. Each way to assign a value to a register is indicated by an arrow with a button—drawn as ⊗— behind the head, pointing from the source of data to the register. When pushed, the button allows the value at the source to "flow" into the designated register. The label next to each button is the name we will use to refer to the button. The names are arbitrary, and can be chosen to have mnemonic

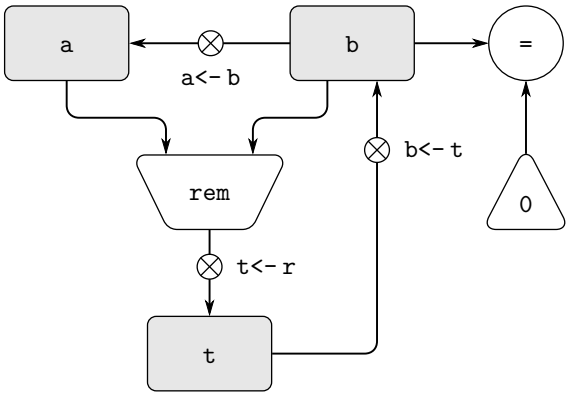


Figure 5.1 Data paths for a GCD machine.

value (for example, `a<-b` denotes pushing the button that assigns the contents of register `b` to register `a`). The source of data for a register can be another register (as in the `a<-b` assignment), an operation result (as in the `t<-r` assignment), or a constant (a built-in value that cannot be changed, represented in a data-path diagram by a triangle containing the constant).

An operation that computes a value from constants and the contents of registers is represented in a data-path diagram by a trapezoid containing a name for the operation. For example, the box marked `rem` in figure 5.1 represents an operation that computes the remainder of the contents of the registers `a` and `b` to which it is attached. Arrows (without buttons) point from the input registers and constants to the box, and arrows connect the operation's output value to registers. A test is represented by a circle containing a name for the test. For example, our GCD machine has an operation that tests whether the contents of register `b` is zero. A test also has arrows from its input registers and constants, but it has no output arrows; its value is used by the controller rather than by the data paths. Overall, the data-path diagram shows the registers and operations that are required for the machine and how they must be connected. If we view the arrows as wires and the $\otimes$ buttons as switches, the data-path diagram is very like the wiring diagram for a machine that could be constructed from electrical components.

In order for the data paths to actually compute GCDs, the buttons must be pushed in the correct sequence. We will describe this sequence in terms of a controller diagram, as illustrated in figure 5.2. The elements of the controller diagram indicate how the data-path components should be operated. The rectangular boxes in the controller diagram identify data-path buttons to be pushed, and the arrows describe the sequencing from one step to the next. The diamond in the diagram represents a decision. One of the two sequencing arrows will be followed, depending on the value of the data-path test identified in the diamond. We can interpret the controller in terms of a physical analogy: Think of the diagram as a maze in which a marble is rolling. When the marble rolls into a box, it pushes the data-path button that is named by the box. When the marble rolls into a decision node (such as the test for `b = 0`), it leaves the node on the path determined by the result of the indicated test.

Taken together, the data paths and the controller completely describe a machine for computing GCDs. We start the controller (the rolling marble) at the place marked `start`, after placing numbers in registers `a` and `b`. When the controller reaches `done`, we will find the value of the GCD in register `a`.

Exercise 5.1

Design a register machine to compute factorials using the iterative algorithm specified by the following function. Draw data-path and controller diagrams for this machine.

```
function factorial(n) {
    function iter(product, counter) {
        return counter > n
               ? product
               : iter(counter * product,
                      counter + 1);
   }
   return iter(1, 1);
}
```

5.1.1 A Language for Describing Register Machines

Data-path and controller diagrams are adequate for representing simple machines such as GCD, but they are unwieldy for describing large machines such as a JavaScript interpreter. To make it possible to deal with complex machines, we will create a language that presents, in textual form, all the information given by the data-path and controller diagrams. We will start with a notation that directly mirrors the diagrams.

We define the data paths of a machine by describing the registers and the operations. To describe a register, we give it a name and specify the buttons that control assignment to it. We give each of these buttons a name and specify the source of the data that enters the register under the button's control. (The source is a register, a constant, or an operation.) To describe an operation, we give it a name and specify its inputs (registers or constants).

We define the controller of a machine as a sequence of *instructions* together with *labels* that identify *entry points* in the sequence. An instruction is one of the following:

- The name of a data-path button to push to assign a value to a register. (This corresponds to a box in the controller diagram.)
- A `test` instruction, which performs a specified test.
- A conditional branch (`branch` instruction) to a location indicated by a controller label, based on the result of the previous test. (The test and branch together correspond to a diamond in the controller diagram.) If the test is false, the controller should continue with the next instruction in the sequence. Otherwise, the controller should continue with the instruction after the label.
- An unconditional branch (`go_to` instruction) naming a controller label at which to continue execution.

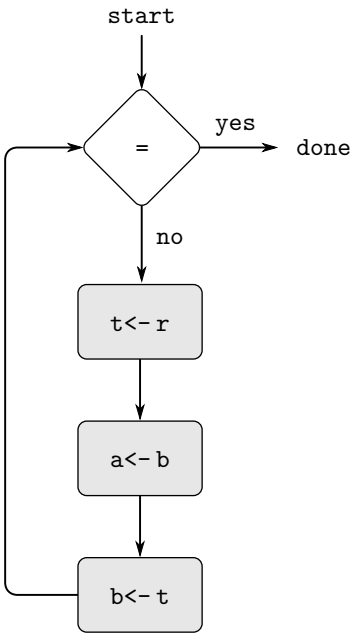


Figure 5.2 Controller for a GCD machine.

The machine starts at the beginning of the controller instruction sequence and stops when execution reaches the end of the sequence. Except when a branch changes the flow of control, instructions are executed in the order in which they are listed.

Figure 5.3 shows the GCD machine described in this way. This example only hints at the generality of these descriptions, since the GCD machine is a very simple case: Each register has only one button, and each button and test is used only once in the controller.

Unfortunately, it is difficult to read such a description. In order to understand the controller instructions we must constantly refer back to the definitions of the button names and the operation names, and to understand what the buttons do we may have to refer to the definitions of the operation names. We will thus transform our notation to combine the information from the data-path and controller descriptions so that we see it all together.

To obtain this form of description, we will replace the arbitrary button and operation names by the definitions of their behavior. That is, instead of saying (in the controller) "Push button `t<-r`" and separately saying (in the data paths) "Button `t<-r` assigns the value of the `rem` operation to register `t`" and "The `rem` operation's inputs are the contents of registers `a` and `b`," we will say (in the controller) "Push the button that assigns to register `t` the value of the `rem` operation on the contents of registers `a` and `b`." Similarly, instead of saying (in the controller) "Perform the = test" and separately saying (in the data paths) "The = test operates on the contents of register `b` and the constant 0," we will say "Perform the = test on the contents of register `b` and the constant 0." We will omit the data-path description, leaving only the controller sequence. Thus, the GCD machine is described as follows:

```
controller(
  list(
    "test_b",
      test(list(op("="), reg("b"), constant(0))),
      branch(label("gcd_done")),
      assign("t", list(op("rem"), reg("a"), reg("b"))),
      assign("a", reg("b")),
      assign("b", reg("t")),
      go_to(label("test_b")),
    "gcd_done"))
```

This form of description is easier to read than the kind illustrated in figure 5.3, but it also has disadvantages:

- It is more verbose for large machines, because complete descriptions of the data-path elements are repeated whenever the elements are mentioned in the controller instruction sequence. (This is not a problem in the GCD example, because each operation and button is used only once.) Moreover, repeating the data-path descriptions obscures the actual data-path structure of the machine; it is not obvious for a large machine how many registers, operations, and buttons there are and how they are interconnected.
- Because the controller instructions in a machine definition look like JavaScript expressions, it is easy to forget that they are not arbitrary JavaScript expressions. They can notate only legal machine operations. For example, operations can operate directly only on constants and the contents of registers, not on the results of other operations.

In spite of these disadvantages, we will use this register-machine language throughout this chapter, because we will be more concerned with understanding controllers than with understanding the elements and connections in data paths. We should keep in mind, however, that data-path design is crucial in designing real machines.

Exercise 5.2

Use the register-machine language to describe the iterative factorial machine of exercise 5.1.

Actions

Let us modify the GCD machine so that we can type in the numbers whose GCD we want and get the answer printed. We will not discuss how to make a machine that can read and print, but will assume (as we do when we use `prompt` and `display` in JavaScript) that they are available as primitive operations.[2]

The operation `prompt` is like the operations we have been using in that it produces a value that can be stored in a register. But `prompt` does not take inputs from any registers; its value depends on something that happens outside the parts of the

2. This assumption glosses over a great deal of complexity. Implementation of reading and printing requires significant effort, for example to handle character encodings for different languages.

```
data_paths(
  registers(
    list(
      pair(name("a"),
           buttons(name("a<-b"), source(register("b")))),
      pair(name("b"),
           buttons(name("b<-t"), source(register("t")))),
      pair(name("t"),
           buttons(name("t<-r"), source(operation("rem"))))))),
  operations(
    list(
      pair(name("rem"),
           inputs(register("a"), register("b"))),
      pair(name("="),
           inputs(register("b"), constant(0))))));

controller(
  list(
    "test_b",                      // label
      test("="),                   // test
      branch(label("gcd_done")),   // conditional branch
      "t<-r",                      // button push
      "a<-b",                      // button push
      "b<-t",                      // button push
      go_to(label("test_b")),      // unconditional branch
    "gcd_done"));                  // label
```

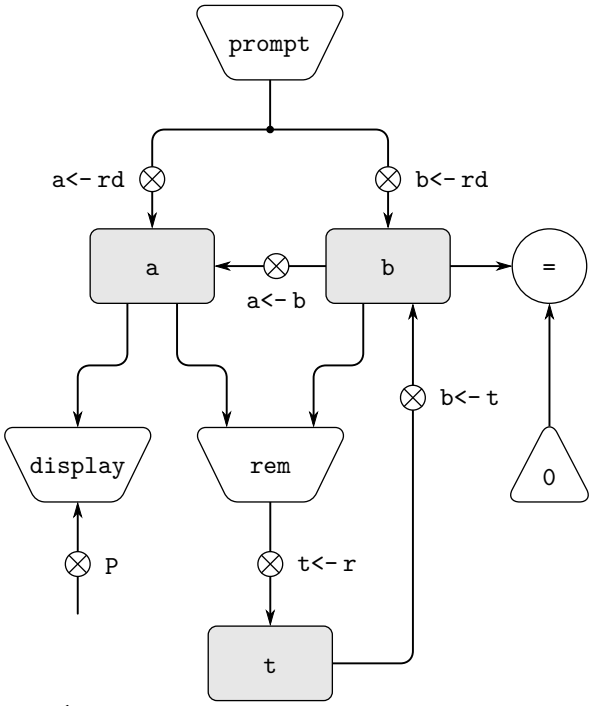
Figure 5.3 A specification of the GCD machine.

machine we are designing. We will allow our machine's operations to have such behavior, and thus will draw and notate the use of `prompt` just as we do any other operation that computes a value.

The operation `display`, on the other hand, differs from the operations we have been using in a fundamental way: It does not produce an output value to be stored in a register. Though it has an effect, this effect is not on a part of the machine we are designing. We will refer to this kind of operation as an *action*. We will represent an action in a data-path diagram just as we represent an operation that computes a value—as a trapezoid that contains the name of the action. Arrows point to the action box from any inputs (registers or constants). We also associate a button with the action. Pushing the button makes the action happen. To make a controller push an action button we use a new kind of instruction called `perform`. Thus, the action of printing the contents of register `a` is represented in a controller sequence by the instruction

```
perform(list(op("display"), reg("a")))
```

Figure 5.4 shows the data paths and controller for the new GCD machine. Instead of having the machine stop after printing the answer, we have made it start over, so that it repeatedly reads a pair of numbers, computes their GCD, and prints the result. This structure is like the driver loops we used in the interpreters of chapter 4.




```
controller(
  list(
    "gcd_loop",
      assign("a", list(op("prompt"))),
      assign("b", list(op("prompt"))),
    "test_b",
      test(list(op("="), reg("b"), constant(0))),
      branch(label("gcd_done")),
      assign("t", list(op("rem"), reg("a"), reg("b"))),
      assign("a", reg("b")),
      assign("b", reg("t")),
      go_to(label("test_b")),
    "gcd_done",
      perform(list(op("display"), reg("a"))),
      go_to(label("gcd_loop"))))
```

Figure 5.4 A GCD machine that reads inputs and prints results.

5.1.2 Abstraction in Machine Design

We will often define a machine to include "primitive" operations that are actually very complex. For example, in sections 5.4 and 5.5 we will treat JavaScript's environment manipulations as primitive. Such abstraction is valuable because it allows us to ignore the details of parts of a machine so that we can concentrate on other aspects of the design. The fact that we have swept a lot of complexity under the rug, however, does not mean that a machine design is unrealistic. We can always replace the complex "primitives" by simpler primitive operations.

Consider the GCD machine. The machine has an instruction that computes the remainder of the contents of registers `a` and `b` and assigns the result to register `t`. If we want to construct the GCD machine without using a primitive remainder operation, we must specify how to compute remainders in terms of simpler operations, such as subtraction. Indeed, we can write a JavaScript function that finds remainders in this way:

```
function remainder(n, d) {
    return n < d
           ? n
           : remainder(n - d, d);
}
```

We can thus replace the remainder operation in the GCD machine's data paths with a subtraction operation and a comparison test. Figure 5.5 shows the data paths and controller for the elaborated machine. The instruction

```
assign("t", list(op("rem"), reg("a"), reg("b")))
```

in the GCD controller definition is replaced by a sequence of instructions that contains a loop, as shown in figure 5.6.

Exercise 5.3

Design a machine to compute square roots using Newton's method, as described in section 1.1.7 and implemented with the following code in section 1.1.8:

```
function sqrt(x) {
    function is_good_enough(guess) {
        return math_abs(square(guess) - x) < 0.001;
    }
    function improve(guess) {
        return average(guess, x / guess);
    }
    function sqrt_iter(guess) {
        return is_good_enough(guess)
               ? guess
               : sqrt_iter(improve(guess));
    }
    return  sqrt_iter(1);
}
```

Begin by assuming that `is_good_enough` and `improve` operations are available as primitives. Then show how to expand these in terms of arithmetic operations. Describe each version of the `sqrt` machine design by drawing a data-path diagram and writing a controller definition in the register-machine language.

5.1.3 Subroutines

When designing a machine to perform a computation, we would often prefer to arrange for components to be shared by different parts of the computation rather than duplicate the components. Consider a machine that includes two GCD computations—one that finds the GCD of the contents of registers `a` and `b` and one that finds

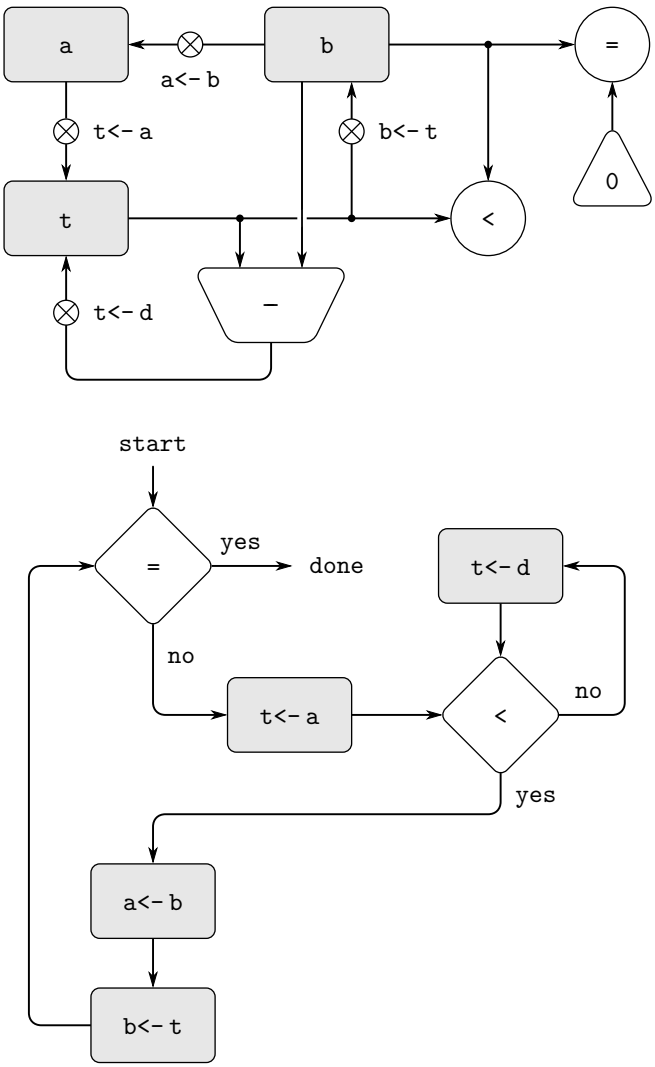


Figure 5.5 Data paths and controller for the elaborated GCD machine.

the GCD of the contents of registers `c` and `d`. We might start by assuming we have a primitive `gcd` operation, then expand the two instances of `gcd` in terms of more primitive operations. Figure 5.7 shows just the GCD portions of the resulting machine's data paths, without showing how they connect to the rest of the machine. The figure also shows the corresponding portions of the machine's controller sequence.

This machine has two remainder operation boxes and two boxes for testing equality. If the duplicated components are complicated, as is the remainder box, this will not be an economical way to build the machine. We can avoid duplicating the data-path components by using the same components for both GCD computations, provided that doing so will not affect the rest of the larger machine's computation. If the values in registers `a` and `b` are not needed by the time the controller gets to `gcd_2` (or if these values can be moved to other registers for safekeeping), we can

```
controller(
  list(
    "test_b",
      test(list(op("="), reg("b"), constant(0))),
      branch(label("gcd_done")),
      assign("t", reg("a")),
    "rem_loop",
      test(list(op("<"), reg("t"), reg("b"))),
      branch(label("rem_done")),
      assign("t", list(op("-"), reg("t"), reg("b"))),
      go_to(label("rem_loop")),
    "rem_done",
      assign("a", reg("b")),
      assign("b", reg("t")),
      go_to(label("test_b")),
    "gcd_done"))
```

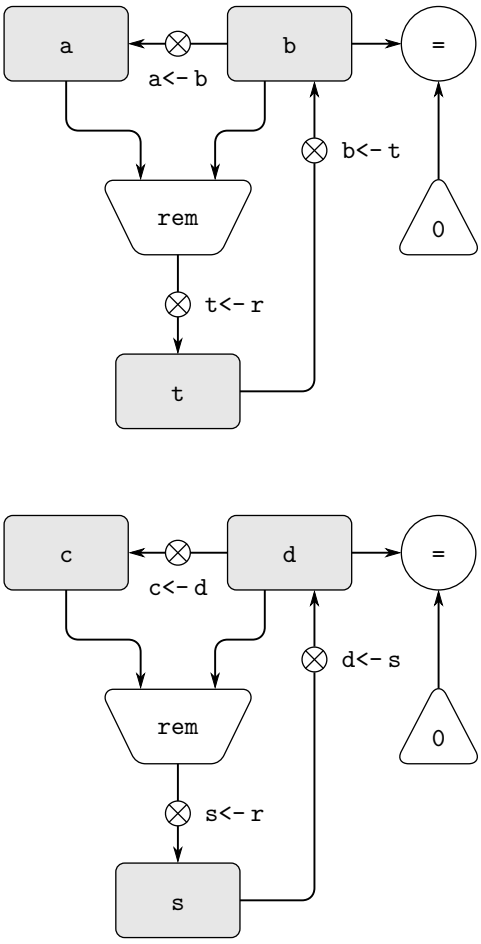
Figure 5.6 Controller instruction sequence for the GCD machine in figure 5.5.

change the machine so that it uses registers `a` and `b`, rather than registers `c` and `d`, in computing the second GCD as well as the first. If we do this, we obtain the controller sequence shown in figure 5.8.

We have removed the duplicate data-path components (so that the data paths are again as in figure 5.1), but the controller now has two GCD sequences that differ only in their entry-point labels. It would be better to replace these two sequences by branches to a single sequence—a `gcd` *subroutine*—at the end of which we branch back to the correct place in the main instruction sequence. We can accomplish this as follows: Before branching to `gcd`, we place a distinguishing value (such as 0 or 1) into a special register, `continue`. At the end of the `gcd` subroutine we return either to `after_gcd_1` or to `after_gcd_2`, depending on the value of the `continue` register. Figure 5.9 shows the relevant portion of the resulting controller sequence, which includes only a single copy of the `gcd` instructions.

This is a reasonable approach for handling small problems, but it would be awkward if there were many instances of GCD computations in the controller sequence. To decide where to continue executing after the `gcd` subroutine, we would need tests in the data paths and branch instructions in the controller for all the places that use `gcd`. A more powerful method for implementing subroutines is to have the `continue` register hold the label of the entry point in the controller sequence at which execution should continue when the subroutine is finished. Implementing this strategy requires a new kind of connection between the data paths and the controller of a register machine: There must be a way to assign to a register a label in the controller sequence in such a way that this value can be fetched from the register and used to continue execution at the designated entry point.

To reflect this ability, we will extend the `assign` instruction of the register-machine language to allow a register to be assigned as value a label from the controller sequence (as a special kind of constant). We will also extend the `go_to` instruction to allow execution to continue at the entry point described by the contents of a register rather than only at an entry point described by a constant label. Using




```
"gcd_1",
  test(list(op("="),
            reg("b"),
            constant(0))),
  branch(label("after_gcd_1")),
  assign("t", list(op("rem"),
                   reg("a"),
                   reg("b"))),
  assign("a", reg("b")),
  assign("b", reg("t")),
  go_to(label("gcd_1")),
"after_gcd_1",

   .
   .
   .

 "gcd_2",
   test(list(op("="),
             reg("d"),
             constant(0))),
   branch(label("after_gcd_2")),
   assign("s", list(op("rem"),
                    reg("c"),
                    reg("d"))),
   assign("c", reg("d")),
   assign("d", reg("s")),
   go_to(label("gcd_2")),
 "after_gcd_2"
```

Figure 5.7 Portions of the data paths and controller sequence for a machine with two GCD computations.

these new constructs we can terminate the `gcd` subroutine with a branch to the location stored in the `continue` register. This leads to the controller sequence shown in figure 5.10.

A machine with more than one subroutine could use multiple continuation registers (e.g., `gcd_continue`, `factorial_continue`) or we could have all subroutines share a single `continue` register. Sharing is more economical, but we must be careful if we have a subroutine (`sub1`) that calls another subroutine (`sub2`). Unless `sub1` saves the contents of `continue` in some other register before setting up `continue` for the call to `sub2`, `sub1` will not know where to go when it is finished. The mechanism developed in the next section to handle recursion also provides a better solution to this problem of nested subroutine calls.

```
"gcd_1",
  test(list(op("="), reg("b"), constant(0))),
  branch(label("after_gcd_1")),
  assign("t", list(op("rem"), reg("a"), reg("b"))),
  assign("a", reg("b")),
  assign("b", reg("t")),
  go_to(label("gcd_1")),
"after_gcd_1",
  ⋮
"gcd_2",
  test(list(op("="), reg("b"), constant(0))),
  branch(label("after_gcd_2")),
  assign("t", list(op("rem"), reg("a"), reg("b"))),
  assign("a", reg("b")),
  assign("b", reg("t")),
  go_to(label("gcd_2")),
"after_gcd_2"
```

Figure 5.8 Portions of the controller sequence for a machine that uses the same datapath components for two different GCD computations.

```
"gcd",
  test(list(op("="), reg("b"), constant(0))),
  branch(label("gcd_done")),
  assign("t", list(op("rem"), reg("a"), reg("b"))),
  assign("a", reg("b")),
  assign("b", reg("t")),
  go_to(label("gcd")),
"gcd_done",
  test(list(op("="), reg("continue"), constant(0))),
  branch(label("after_gcd_1")),
  go_to(label("after_gcd_2")),
  ⋮
  // Before branching to gcd from the first place where
  // it is needed, we place 0 in the continue register
  assign("continue", constant(0)),
  go_to(label("gcd")),
"after_gcd_1",
  ⋮
  // Before the second use of gcd, we place 1 in the continue register
  assign("continue", constant(1)),
  go_to(label("gcd")),
"after_gcd_2"
```

Figure 5.9 Using a `continue` register to avoid the duplicate controller sequence in figure 5.8.

```
"gcd",
  test(list(op("="), reg("b"), constant(0))),
  branch(label("gcd_done")),
  assign("t", list(op("rem"), reg("a"), reg("b"))),
  assign("a", reg("b")),
  assign("b", reg("t")),
  go_to(label("gcd")),
"gcd_done",
  go_to(reg("continue")),
  ⋮
  // Before calling gcd, we assign to continue
  // the label to which gcd should return.
  assign("continue", label("after_gcd_1"))),
  go_to(label("gcd")),
"after_gcd_1",
  ⋮
  // Here is the second call to gcd, with a different continuation.
  assign("continue", label("after_gcd_2")),
  go_to(label("gcd")),
"after_gcd_2"
```

Figure 5.10 Assigning labels to the `continue` register simplifies and generalizes the strategy shown in figure 5.9.

5.1.4 Using a Stack to Implement Recursion

With the ideas illustrated so far, we can implement any iterative process by specifying a register machine that has a register corresponding to each state variable of the process. The machine repeatedly executes a controller loop, changing the contents of the registers, until some termination condition is satisfied. At each point in the controller sequence, the state of the machine (representing the state of the iterative process) is completely determined by the contents of the registers (the values of the state variables).

Implementing recursive processes, however, requires an additional mechanism. Consider the following recursive method for computing factorials, which we first examined in section 1.2.1:

```
function factorial(n) {
    return n === 1
           ? 1
           : n * factorial(n - 1);
}
```

As we see from the function, computing $n!$ requires computing $(n-1)!$. Our GCD machine, modeled on the function

```
function gcd(a, b) {
    return b === 0 ? a : gcd(b, a % b);
}
```

similarly had to compute another GCD. But there is an important difference between the `gcd` function, which reduces the original computation to a new GCD computation,

and `factorial`, which requires computing another factorial as a subproblem. In GCD, the answer to the new GCD computation is the answer to the original problem. To compute the next GCD, we simply place the new arguments in the input registers of the GCD machine and reuse the machine's data paths by executing the same controller sequence. When the machine is finished solving the final GCD problem, it has completed the entire computation.

In the case of factorial (or any recursive process) the answer to the new factorial subproblem is not the answer to the original problem. The value obtained for $(n-1)!$ must be multiplied by n to get the final answer. If we try to imitate the GCD design, and solve the factorial subproblem by decrementing the `n` register and rerunning the factorial machine, we will no longer have available the old value of `n` by which to multiply the result. We thus need a second factorial machine to work on the subproblem. This second factorial computation itself has a factorial subproblem, which requires a third factorial machine, and so on. Since each factorial machine contains another factorial machine within it, the total machine contains an infinite nest of similar machines and hence cannot be constructed from a fixed, finite number of parts.

Nevertheless, we can implement the factorial process as a register machine if we can arrange to use the same components for each nested instance of the machine. Specifically, the machine that computes $n!$ should use the same components to work on the subproblem of computing $(n-1)!$, on the subproblem for $(n-2)!$, and so on. This is plausible because, although the factorial process dictates that an unbounded number of copies of the same machine are needed to perform a computation, only one of these copies needs to be active at any given time. When the machine encounters a recursive subproblem, it can suspend work on the main problem, reuse the same physical parts to work on the subproblem, then continue the suspended computation.

In the subproblem, the contents of the registers will be different than they were in the main problem. (In this case the `n` register is decremented.) In order to be able to continue the suspended computation, the machine must save the contents of any registers that will be needed after the subproblem is solved so that these can be restored to continue the suspended computation. In the case of factorial, we will save the old value of `n`, to be restored when we are finished computing the factorial of the decremented `n` register.[3]

Since there is no a priori limit on the depth of nested recursive calls, we may need to save an arbitrary number of register values. These values must be restored in the reverse of the order in which they were saved, since in a nest of recursions the last subproblem to be entered is the first to be finished. This dictates the use of a *stack*, or "last in, first out" data structure, to save register values. We can extend the register-machine language to include a stack by adding two kinds of instructions:

---

3. One might argue that we don't need to save the old `n`; after we decrement it and solve the subproblem, we could simply increment it to recover the old value. Although this strategy works for factorial, it cannot work in general, since the old value of a register cannot always be computed from the new one.

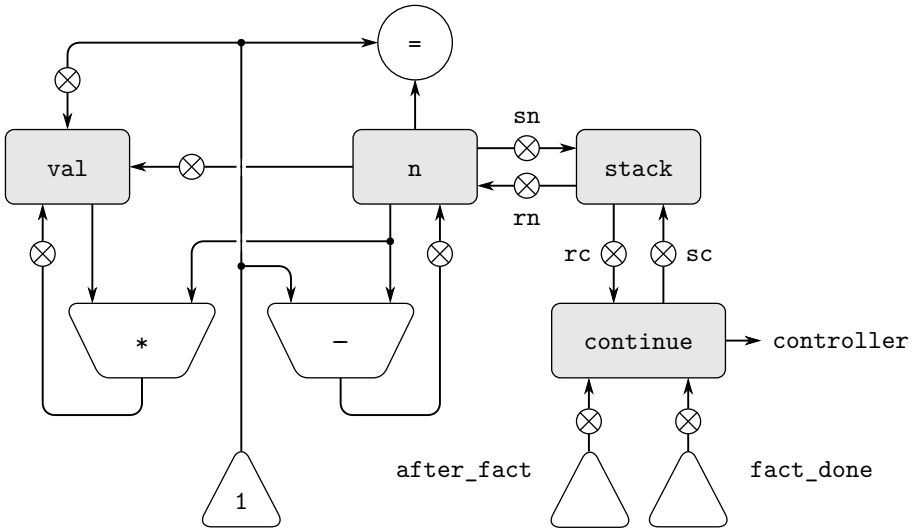
Values are placed on the stack using a `save` instruction and restored from the stack using a `restore` instruction. After a sequence of values has been `saved` on the stack, a sequence of `restores` will retrieve these values in reverse order.[4]

With the aid of the stack, we can reuse a single copy of the factorial machine's data paths for each factorial subproblem. There is a similar design issue in reusing the controller sequence that operates the data paths. To reexecute the factorial computation, the controller cannot simply loop back to the beginning, as with an iterative process, because after solving the $(n-1)!$ subproblem the machine must still multiply the result by n. The controller must suspend its computation of $n!$, solve the $(n-1)!$ subproblem, then continue its computation of $n!$. This view of the factorial computation suggests the use of the subroutine mechanism described in section 5.1.3, which has the controller use a `continue` register to transfer to the part of the sequence that solves a subproblem and then continue where it left off on the main problem. We can thus make a factorial subroutine that returns to the entry point stored in the `continue` register. Around each subroutine call, we save and restore `continue` just as we do the `n` register, since each "level" of the factorial computation will use the same `continue` register. That is, the factorial subroutine must put a new value in `continue` when it calls itself for a subproblem, but it will need the old value in order to return to the place that called it to solve a subproblem.

Figure 5.11 shows the data paths and controller for a machine that implements the recursive `factorial` function. The machine has a stack and three registers, called `n`, `val`, and `continue`. To simplify the data-path diagram, we have not named the register-assignment buttons, only the stack-operation buttons (`sc` and `sn` to save registers, `rc` and `rn` to restore registers). To operate the machine, we put in register `n` the number whose factorial we wish to compute and start the machine. When the machine reaches `fact_done`, the computation is finished and the answer will be found in the `val` register. In the controller sequence, `n` and `continue` are saved before each recursive call and restored upon return from the call. Returning from a call is accomplished by branching to the location stored in `continue`. The register `continue` is initialized when the machine starts so that the last return will go to `fact_done`. The `val` register, which holds the result of the factorial computation, is not saved before the recursive call, because the old contents of `val` is not useful after the subroutine returns. Only the new value, which is the value produced by the subcomputation, is needed.

Although in principle the factorial computation requires an infinite machine, the machine in figure 5.11 is actually finite except for the stack, which is potentially unbounded. Any particular physical implementation of a stack, however, will be of finite size, and this will limit the depth of recursive calls that can be handled by the machine. This implementation of factorial illustrates the general strategy for realizing recursive algorithms as ordinary register machines augmented by stacks. When a recursive subproblem is encountered, we save on the stack the registers whose current values will be required after the subproblem is solved, solve the recursive subproblem, then restore the saved registers and continue execution on the main problem. The `continue` register must always be saved. Whether there are

4. In section 5.3 we will see how to implement a stack in terms of more primitive operations.




```
controller(
  list(
      assign("continue", label("fact_done")), // set up final return address
    "fact_loop",
      test(list(op("="), reg("n"), constant(1))),
      branch(label("base_case")),
      // Set up for recursive call by saving n and continue.
      // Set up continue so that the computation will continue
      // at after_fact when the subroutine returns.
      save("continue"),
      save("n"),
      assign("n", list(op("-"), reg("n"), constant(1))),
      assign("continue", label("after_fact")),
      go_to(label("fact_loop")),
    "after_fact",
      restore("n"),
      restore("continue"),
      assign("val",                      // val now contains n(n-1)!
             list(op("*"), reg("n"), reg("val"))),
      go_to(reg("continue")),            // return to caller
    "base_case",
      assign("val", constant(1)),        // base case: 1! = 1
      go_to(reg("continue")),            // return to caller
    "fact_done"))
```

Figure 5.11 A recursive factorial machine.

other registers that need to be saved depends on the particular machine, since not all recursive computations need the original values of registers that are modified during solution of the subproblem (see exercise 5.4).

A double recursion

Let us examine a more complex recursive process, the tree-recursive computation of the Fibonacci numbers, which we introduced in section 1.2.2:

```
function fib(n) {
    return n === 0
           ? 0
           : n === 1
           ? 1
           : fib(n - 1) + fib(n - 2);
}
```

Just as with factorial, we can implement the recursive Fibonacci computation as a register machine with registers `n`, `val`, and `continue`. The machine is more complex than the one for factorial, because there are two places in the controller sequence where we need to perform recursive calls—once to compute Fib($n-1$) and once to compute Fib($n-2$). To set up for each of these calls, we save the registers whose values will be needed later, set the `n` register to the number whose Fib we need to compute recursively ($n-1$ or $n-2$), and assign to `continue` the entry point in the main sequence to which to return (`afterfib_n_1` or `afterfib_n_2`, respectively). We then go to `fib_loop`. When we return from the recursive call, the answer is in `val`. Figure 5.12 shows the controller sequence for this machine.

Exercise 5.4

Specify register machines that implement each of the following functions. For each machine, write a controller instruction sequence and draw a diagram showing the data paths.

a. Recursive exponentiation:

```
function expt(b, n) {
    return n === 0
           ? 1
           : b * expt(b, n - 1);
}
```

b. Iterative exponentiation:

```
function expt(b, n) {
    function expt_iter(counter, product) {
        return counter === 0
               ? product
               : expt_iter(counter - 1, b * product);
    }
    return expt_iter(n, 1);
}
```

Exercise 5.5

Hand-simulate the factorial and Fibonacci machines, using some nontrivial input (requiring execution of at least one recursive call). Show the contents of the stack at each significant point in the execution.

```
controller(
  list(
    assign("continue", label("fib_done")),
  "fib_loop",
    test(list(op("<"), reg("n"), constant(2))),
    branch(label("immediate_answer")),
    // set up to compute Fib(n − 1)
    save("continue"),
    assign("continue", label("afterfib_n_1")),
    save("n"),                          // save old value of n
    assign("n", list(op("-"), reg("n"), constant(1))), // clobber n to n − 1
    go_to(label("fib_loop")),           // perform recursive call
  "afterfib_n_1",                       // upon return, val contains Fib(n − 1)
    restore("n"),
    restore("continue"),
    // set up to compute Fib(n − 2)
    assign("n", list(op("-"), reg("n"), constant(2))),
    save("continue"),
    assign("continue", label("afterfib_n_2")),
    save("val"),                        // save Fib(n − 1)
    go_to(label("fib_loop")),
  "afterfib_n_2",                       // upon return, val contains Fib(n − 2)
    assign("n", reg("val")),            // n now contains Fib(n − 2)
    restore("val"),                     // val now contains Fib(n − 1)
    restore("continue"),
    assign("val",                       // Fib(n − 1) + Fib(n − 2)
      list(op("+"), reg("val"), reg("n"))),
    go_to(reg("continue")),             // return to caller, answer in val
  "immediate_answer",
    assign("val", reg("n")),            // base case: Fib(n) = n
    go_to(reg("continue")),
  "fib_done"))
```

Figure 5.12 Controller for a machine to compute Fibonacci numbers.

Exercise 5.6

Ben Bitdiddle observes that the Fibonacci machine's controller sequence has an extra `save` and an extra `restore`, which can be removed to make a faster machine. Where are these instructions?

5.1.5 Instruction Summary

A controller instruction in our register-machine language has one of the following forms, where each *input*$_i$ is `reg(`*register-name*`)` or `constant(`*constant-value*`)`.

These instructions were introduced in section 5.1.1:

`assign(`*register-name*`, reg(`*register-name*`))`

`assign(`*register-name*`, constant(`*constant-value*`))`

`assign(`*register-name*`, list(op(`*operation-name*`),` $input_1$`, ..., `$input_n$`))`

`perform(list(op(`*operation-name*`),` $input_1$`, ..., `$input_n$`))`

`test(list(op(`*operation-name*`),` $input_1$`, ..., `$input_n$`))`

`branch(label(`*label-name*`))`

`go_to(label(`*label-name*`))`

The use of registers to hold labels was introduced in section 5.1.3:

`assign(`*register-name*`, label(`*label-name*`))`

`go_to(reg(`*register-name*`))`

Instructions to use the stack were introduced in section 5.1.4:

`save(`*register-name*`)`

`restore(`*register-name*`)`

The only kind of *constant-value* we have seen so far is a number, but later we will also use strings and lists. For example, `constant("abc")` is the string `"abc"`, `constant(null)` is the empty list, and `constant(list("a", "b", "c"))` is the list `list("a", "b", "c")`.

5.2 A Register-Machine Simulator

In order to gain a good understanding of the design of register machines, we must test the machines we design to see if they perform as expected. One way to test a design is to hand-simulate the operation of the controller, as in exercise 5.5. But this is extremely tedious for all but the simplest machines. In this section we construct a simulator for machines described in the register-machine language. The simulator is a JavaScript program with four interface functions. The first uses a description of a register machine to construct a model of the machine (a data structure whose parts correspond to the parts of the machine to be simulated), and the other three allow us to simulate the machine by manipulating the model:

- `make_machine(`*register-names*`,` *operations*`,` *controller*`)`
  constructs and returns a model of the machine with the given registers, operations, and controller.
- `set_register_contents(`*machine-model*`,` *register-name*`,` *value*`)`
  stores a value in a simulated register in the given machine.
- `get_register_contents(`*machine-model*`,` *register-name*`)`
  returns the contents of a simulated register in the given machine.
- `start(`*machine-model*`)`
  simulates the execution of the given machine, starting from the beginning of the controller sequence and stopping when it reaches the end of the sequence.

As an example of how these functions are used, we can define `gcd_machine` to be a model of the GCD machine of section 5.1.1 as follows:

```
const gcd_machine =
    make_machine(
        list("a", "b", "t"),
        list(list("rem", (a, b) => a % b),
             list("=", (a, b) => a === b)),
        list(
          "test_b",
            test(list(op("="), reg("b"), constant(0))),
            branch(label("gcd_done")),
            assign("t", list(op("rem"), reg("a"), reg("b"))),
            assign("a", reg("b")),
            assign("b", reg("t")),
            go_to(label("test_b")),
          "gcd_done"));
```

The first argument to `make_machine` is a list of register names. The next argument is a table (a list of two-element lists) that pairs each operation name with a JavaScript function that implements the operation (that is, produces the same output value given the same input values). The last argument specifies the controller as a list of labels and machine instructions, as in section 5.1.

To compute GCDs with this machine, we set the input registers, start the machine, and examine the result when the simulation terminates:

```
set_register_contents(gcd_machine, "a", 206);
"done"

set_register_contents(gcd_machine, "b", 40);
"done"

start(gcd_machine);
"done"

get_register_contents(gcd_machine, "a");
2
```

This computation will run much more slowly than a `gcd` function written in JavaScript, because we will simulate low-level machine instructions, such as `assign`, by much more complex operations.

Exercise 5.7

Use the simulator to test the machines you designed in exercise 5.4.

5.2.1 The Machine Model

The machine model generated by `make_machine` is represented as a function with local state using the message-passing techniques developed in chapter 3. To build this model, `make_machine` begins by calling the function `make_new_machine` to construct the parts of the machine model that are common to all register machines.

This basic machine model constructed by `make_new_machine` is essentially a container for some registers and a stack, together with an execution mechanism that processes the controller instructions one by one.

The function `make_machine` then extends this basic model (by sending it messages) to include the registers, operations, and controller of the particular machine being defined. First it allocates a register in the new machine for each of the supplied register names and installs the designated operations in the machine. Then it uses an *assembler* (described below in section 5.2.2) to transform the controller list into instructions for the new machine and installs these as the machine's instruction sequence. The function `make_machine` returns as its value the modified machine model.

```
function make_machine(register_names, ops, controller) {
    const machine = make_new_machine();
    for_each(register_name =>
                machine("allocate_register")(register_name),
             register_names);
    machine("install_operations")(ops);
    machine("install_instruction_sequence")
           (assemble(controller, machine));
    return machine;
}
```

Registers

We will represent a register as a function with local state, as in chapter 3. The function `make_register` creates a register that holds a value that can be accessed or changed:

```
function make_register(name) {
    let contents = "*unassigned*";
    function dispatch(message) {
        return message === "get"
               ? contents
               : message === "set"
               ? value => { contents = value; }
               : error(message, "unknown request -- make_register");
    }
    return dispatch;
}
```

The following functions are used to access registers:

```
function get_contents(register) {
    return register("get");
}
function set_contents(register, value) {
    return register("set")(value);
}
```

The stack

We can also represent a stack as a function with local state. The function `make_stack` creates a stack whose local state consists of a list of the items on the stack.

A stack accepts requests to `push` an item onto the stack, to `pop` the top item off the stack and return it, and to `initialize` the stack to empty.

```
function make_stack() {
    let stack = null;
    function push(x) {
        stack = pair(x, stack);
        return "done";
    }
    function pop() {
        if (is_null(stack)) {
            error("empty stack -- pop");
        } else {
            const top = head(stack);
            stack = tail(stack);
            return top;
        }
    }
    function initialize() {
        stack = null;
        return "done";
    }
    function dispatch(message) {
        return message === "push"
               ? push
               : message === "pop"
               ? pop()
               : message === "initialize"
               ? initialize()
               : error(message, "unknown request -- stack");
    }
    return dispatch;
}
```

The following functions are used to access stacks:

```
function pop(stack) {
    return stack("pop");
}
function push(stack, value) {
    return stack("push")(value);
}
```

The basic machine

The `make_new_machine` function, shown in figure 5.13, constructs an object whose local state consists of a stack, an initially empty instruction sequence, a list of operations that initially contains an operation to initialize the stack, and a *register table* that initially contains two registers, named `flag` and `pc` (for "program counter"). The internal function `allocate_register` adds new entries to the register table, and the internal function `lookup_register` looks up registers in the table.

The `flag` register is used to control branching in the simulated machine. Our `test` instructions set the contents of `flag` to the result of the test (true or false). Our

```
function make_new_machine() {
    const pc = make_register("pc");
    const flag = make_register("flag");
    const stack = make_stack();
    let the_instruction_sequence = null;
    let the_ops = list(list("initialize_stack", () => stack("initialize")));
    let register_table = list(list("pc", pc), list("flag", flag));
    function allocate_register(name) {
        if (is_undefined(assoc(name, register_table))) {
            register_table = pair(list(name, make_register(name)),
                                  register_table);
        } else {
            error(name, "multiply defined register");
        }
        return "register allocated";
    }
    function lookup_register(name) {
        const val = assoc(name, register_table);
        return is_undefined(val)
               ? error(name, "unknown register")
               : head(tail(val));
    }
    function execute() {
        const insts = get_contents(pc);
        if (is_null(insts)) {
            return "done";
        } else {
            inst_execution_fun(head(insts))();
            return execute();
        }
    }
    function dispatch(message) {
        function start() {
            set_contents(pc, the_instruction_sequence);
            return execute();
        }
        return message === "start"
               ? start()
               : message === "install_instruction_sequence"
               ? seq => { the_instruction_sequence = seq; }
               : message === "allocate_register"
               ? allocate_register
               : message === "get_register"
               ? lookup_register
               : message === "install_operations"
               ? ops => { the_ops = append(the_ops, ops); }
               : message === "stack"
               ? stack
               : message === "operations"
               ? the_ops
               : error(message, "unknown request -- machine");
    }
    return dispatch;
}
```

Figure 5.13 The `make_new_machine` function implements the basic machine model.

branch instructions decide whether or not to branch by examining the contents of flag.

The pc register determines the sequencing of instructions as the machine runs. This sequencing is implemented by the internal function execute. In the simulation model, each machine instruction is a data structure that includes a function of no arguments, called the *instruction execution function*, such that calling this function simulates executing the instruction. As the simulation runs, pc points to the place in the instruction sequence beginning with the next instruction to be executed. The function execute gets that instruction, executes it by calling the instruction execution function, and repeats this cycle until there are no more instructions to execute (i.e., until pc points to the end of the instruction sequence).

As part of its operation, each instruction execution function modifies pc to indicate the next instruction to be executed. The instructions branch and go_to change pc to point to the new destination. All other instructions simply advance pc, making it point to the next instruction in the sequence. Observe that each call to execute calls execute again, but this does not produce an infinite loop because running the instruction execution function changes the contents of pc.

The function make_new_machine returns a dispatch function that implements message-passing access to the internal state. Notice that starting the machine is accomplished by setting pc to the beginning of the instruction sequence and calling execute.

For convenience, we provide an alternate interface to a machine's start operation, as well as functions to set and examine register contents, as specified at the beginning of section 5.2:

```
function start(machine) {
    return machine("start");
}
function get_register_contents(machine, register_name) {
    return get_contents(get_register(machine, register_name));
}
function set_register_contents(machine, register_name, value) {
    set_contents(get_register(machine, register_name), value);
    return "done";
}
```

These functions (and many functions in sections 5.2.2 and 5.2.3) use the following to look up the register with a given name in a given machine:

```
function get_register(machine, reg_name) {
    return machine("get_register")(reg_name);
}
```

5.2.2 The Assembler

The assembler transforms the sequence of controller instructions for a machine into a corresponding list of machine instructions, each with its execution function. Overall, the assembler is much like the evaluators we studied in chapter 4—there is an input language (in this case, the register-machine language) and we must perform an appropriate action for each type of component in the language.

The technique of producing an execution function for each instruction is just what we used in section 4.1.7 to speed up the evaluator by separating analysis from

runtime execution. As we saw in chapter 4, much useful analysis of JavaScript expressions could be performed without knowing the actual values of names. Here, analogously, much useful analysis of register-machine-language expressions can be performed without knowing the actual contents of machine registers. For example, we can replace references to registers by pointers to the register objects, and we can replace references to labels by pointers to the place in the instruction sequence that the label designates.

Before it can generate the instruction execution functions, the assembler must know what all the labels refer to, so it begins by scanning the controller sequence to separate the labels from the instructions. As it scans the controller, it constructs both a list of instructions and a table that associates each label with a pointer into that list. Then the assembler augments the instruction list by inserting the execution function for each instruction.

The `assemble` function is the main entry to the assembler. It takes the controller sequence and the machine model as arguments and returns the instruction sequence to be stored in the model. The function `assemble` calls `extract_labels` to build the initial instruction list and label table from the supplied controller. The second argument to `extract_labels` is a function to be called to process these results: This function uses `update_insts` to generate the instruction execution functions and insert them into the instruction list, and returns the modified list.

```
function assemble(controller, machine) {
    return extract_labels(controller,
                          (insts, labels) => {
                              update_insts(insts, labels, machine);
                              return insts;
                          });
}
```

The function `extract_labels` takes a list `controller` and a function `receive` as arguments. The function `receive` will be called with two values: (1) a list `insts` of instruction data structures, each containing an instruction from `controller`; and (2) a table called `labels`, which associates each label from `controller` with the position in the list `insts` that the label designates.

```
function extract_labels(controller, receive) {
    return is_null(controller)
           ? receive(null, null)
           : extract_labels(
                 tail(controller),
                 (insts, labels) => {
                   const next_element = head(controller);
                   return is_string(next_element)
                          ? receive(insts,
                                    pair(make_label_entry(next_element,
                                                          insts),
                                         labels))
                          : receive(pair(make_inst(next_element),
                                         insts),
                                    labels);
                 });
}
```

The function `extract_labels` works by sequentially scanning the elements of the `controller` and accumulating the `insts` and the `labels`. If an element is a string (and thus a label) an appropriate entry is added to the `labels` table. Otherwise the element is accumulated onto the `insts` list.[5]

---

5. Using the `receive` function here is a way to get `extract_labels` to effectively return two values—`labels` and `insts`—without explicitly making a compound data structure to hold them. An alternative implementation, which returns an explicit pair of values, is

```
function extract_labels(controller) {
    if (is_null(controller)) {
        return pair(null, null);
    } else {
        const result = extract_labels(tail(controller));
        const insts = head(result);
        const labels = tail(result);
        const next_element = head(controller);
        return is_string(next_element)
               ? pair(insts,
                      pair(make_label_entry(next_element, insts), labels))
               : pair(pair(make_inst(next_element), insts),
                      labels);
    }
}
```

which would be called by `assemble` as follows:

```
function assemble(controller, machine) {
    const result = extract_labels(controller);
    const insts = head(result);
    const labels = tail(result);
    update_insts(insts, labels, machine);
    return insts;
}
```

You can consider our use of `receive` as demonstrating an elegant way to return multiple values, or simply an excuse to show off a programming trick. An argument like `receive` that is the next function to be invoked is called a "continuation." Recall that we also used continuations to implement the backtracking control structure in the `amb` evaluator in section 4.3.3.

The function update_insts modifies the instruction list, which initially contains only the controller instructions, to include the corresponding execution functions:

```
function update_insts(insts, labels, machine) {
    const pc = get_register(machine, "pc");
    const flag = get_register(machine, "flag");
    const stack = machine("stack");
    const ops = machine("operations");
    return for_each(inst => set_inst_execution_fun(
                                inst,
                                make_execution_function(
                                    inst_controller_instruction(inst),
                                    labels, machine, pc,
                                    flag, stack, ops)),
                    insts);
}
```

The machine instruction data structure simply pairs the controller instruction with the corresponding execution function. The execution function is not yet available when extract_labels constructs the instruction, and is inserted later by update_insts.

```
function make_inst(inst_controller_instruction) {
    return pair(inst_controller_instruction, null);
}
function inst_controller_instruction(inst) {
    return head(inst);
}
function inst_execution_fun(inst) {
    return tail(inst);
}
function set_inst_execution_fun(inst, fun) {
    set_tail(inst, fun);
}
```

The controller instruction is not used by our simulator, but is handy to keep around for debugging (see exercise 5.15).

Elements of the label table are pairs:

```
function make_label_entry(label_name, insts) {
    return pair(label_name, insts);
}
```

Entries will be looked up in the table with

```
function lookup_label(labels, label_name) {
    const val = assoc(label_name, labels);
    return is_undefined(val)
           ? error(label_name, "undefined label -- assemble")
           : tail(val);
}
```

Exercise 5.8

The following register-machine code is ambiguous, because the label `here` is defined more than once:

```
"start",
  go_to(label("here")),
"here",
  assign("a", constant(3)),
  go_to(label("there")),
"here",
  assign("a", constant(4)),
  go_to(label("there")),
"there",
```

With the simulator as written, what will the contents of register `a` be when control reaches `there`? Modify the `extract_labels` function so that the assembler will signal an error if the same label name is used to indicate two different locations.

5.2.3 Instructions and Their Execution Functions

The assembler calls `make_execution_function` to generate the execution function for a controller instruction. Like the `analyze` function in the evaluator of section 4.1.7, this dispatches on the type of instruction to generate the appropriate execution function. The details of these execution functions determine the meaning of the individual instructions in the register-machine language.

```
function make_execution_function(inst, labels, machine,
                                 pc, flag, stack, ops) {
    const inst_type = type(inst);
    return inst_type === "assign"
           ? make_assign_ef(inst, machine, labels, ops, pc)
           : inst_type === "test"
           ? make_test_ef(inst, machine, labels, ops, flag, pc)
           : inst_type === "branch"
           ? make_branch_ef(inst, machine, labels, flag, pc)
           : inst_type === "go_to"
           ? make_go_to_ef(inst, machine, labels, pc)
           : inst_type === "save"
           ? make_save_ef(inst, machine, stack, pc)
           : inst_type === "restore"
           ? make_restore_ef(inst, machine, stack, pc)
           : inst_type === "perform"
           ? make_perform_ef(inst, machine, labels, ops, pc)
           : error(inst, "unknown instruction type -- assemble");
}
```

The elements of the `controller` sequence received by `make_machine` and passed to `assemble` are strings (for labels) and tagged lists (for instructions). The tag in an instruction is a string that identifies the instruction type, such as `"go_to"`, and the remaining elements of the list contains the arguments, such as the destination of the `go_to`. The dispatch in `make_execution_function` uses

```
function type(instruction) { return head(instruction); }
```

The tagged lists are constructed when the `list` expression that is the third argument to `make_machine` is evaluated. Each argument to that `list` is either a string (which evaluates to itself) or a call to a constructor for an instruction tagged list. For example, `assign("b", reg("t"))` calls the constructor `assign` with arguments `"b"` and the result of calling the constructor `reg` with the argument `"t"`. The constructors and their arguments determine the syntax of the individual instructions in the register-machine language. The instruction constructors and selectors are shown below, along with the execution-function generators that use the selectors.

The instruction `assign`

The `make_assign_ef` function makes execution functions for `assign` instructions:

```
function make_assign_ef(inst, machine, labels, operations, pc) {
    const target = get_register(machine, assign_reg_name(inst));
    const value_exp = assign_value_exp(inst);
    const value_fun =
        is_operation_exp(value_exp)
        ? make_operation_exp_ef(value_exp, machine, labels, operations)
        : make_primitive_exp_ef(value_exp, machine, labels);
    return () => {
                set_contents(target, value_fun());
                advance_pc(pc);
            };
}
```

The function `assign` constructs `assign` instructions. The selectors `assign_reg_name` and `assign_value_exp` extract the register name and value expression from an `assign` instruction.

```
function assign(register_name, source) {
    return list("assign", register_name, source);
}
function assign_reg_name(assign_instruction) {
    return head(tail(assign_instruction));
}
function assign_value_exp(assign_instruction) {
    return head(tail(tail(assign_instruction)));
}
```

The function `make_assign_ef` looks up the register name with `get_register` to produce the target register object. The value expression is passed to `make_operation_exp_ef` if the value is the result of an operation, and it is passed to `make_primitive_exp_ef` otherwise. These functions (shown below) analyze the value expression and produce an execution function for the value. This is a function of no arguments, called `value_fun`, which will be evaluated during the simulation to produce the actual value to be assigned to the register. Notice that the work of looking up the register name and analyzing the value expression is performed just once, at assembly time, not every time the instruction is simulated. This saving of work is the reason we use execution functions, and corresponds directly to the saving

in work we obtained by separating program analysis from execution in the evaluator of section 4.1.7.

The result returned by `make_assign_ef` is the execution function for the `assign` instruction. When this function is called (by the machine model's `execute` function), it sets the contents of the target register to the result obtained by executing `value_fun`. Then it advances the `pc` to the next instruction by running the function

```
function advance_pc(pc) {
    set_contents(pc, tail(get_contents(pc)));
}
```

The function `advance_pc` is the normal termination for all instructions except `branch` and `go_to`.

The instructions `test`, `branch`, and `go_to`

The function `make_test_ef` handles `test` instructions in a similar way. It extracts the expression that specifies the condition to be tested and generates an execution function for it. At simulation time, the function for the condition is called, the result is assigned to the `flag` register, and the `pc` is advanced:

```
function make_test_ef(inst, machine, labels, operations, flag, pc) {
    const condition = test_condition(inst);
    if (is_operation_exp(condition)) {
        const condition_fun = make_operation_exp_ef(
                                  condition, machine,
                                  labels, operations);
        return () => {
                    set_contents(flag, condition_fun());
                    advance_pc(pc);
                };
    } else {
        error(inst, "bad test instruction -- assemble");
    }
}
```

The function `test` constructs `test` instructions. The selector `test_condition` extracts the condition from a test.

```
function test(condition) { return list("test", condition); }
function test_condition(test_instruction) {
    return head(tail(test_instruction));
}
```

The execution function for a `branch` instruction checks the contents of the `flag` register and either sets the contents of the `pc` to the branch destination (if the branch is taken) or else just advances the `pc` (if the branch is not taken). Notice that the indicated destination in a `branch` instruction must be a label, and the `make_branch_ef` function enforces this. Notice also that the label is looked up at assembly time, not each time the `branch` instruction is simulated.

```
function make_branch_ef(inst, machine, labels, flag, pc) {
    const dest = branch_dest(inst);
    if (is_label_exp(dest)) {
        const insts = lookup_label(labels, label_exp_label(dest));
        return () => {
                    if (get_contents(flag)) {
                        set_contents(pc, insts);
                    } else {
                        advance_pc(pc);
                    }
                };
    } else {
        error(inst, "bad branch instruction -- assemble");
    }
}
```

The function `branch` constructs `branch` instructions. The selector `branch_dest` extracts the destination from a branch.

```
function branch(label) { return list("branch", label); }
function branch_dest(branch_instruction) {
    return head(tail(branch_instruction));
}
```

A `go_to` instruction is similar to a branch, except that the destination may be specified either as a label or as a register, and there is no condition to check—the `pc` is always set to the new destination.

```
function make_go_to_ef(inst, machine, labels, pc) {
    const dest = go_to_dest(inst);
    if (is_label_exp(dest)) {
        const insts = lookup_label(labels, label_exp_label(dest));
        return () => set_contents(pc, insts);
    } else if (is_register_exp(dest)) {
        const reg = get_register(machine, register_exp_reg(dest));
        return () => set_contents(pc, get_contents(reg));
    } else {
        error(inst, "bad go_to instruction -- assemble");
    }
}
```

The function `go_to` constructs `go_to` instructions. The selector `go_to_dest` extracts the destination from a `go_to` instruction.

```
function go_to(label) { return list("go_to", label); }
function go_to_dest(go_to_instruction) {
    return head(tail(go_to_instruction));
}
```

Other instructions

The stack instructions `save` and `restore` simply use the stack with the designated register and advance the `pc`:

```
function make_save_ef(inst, machine, stack, pc) {
    const reg = get_register(machine, stack_inst_reg_name(inst));
    return () => {
                push(stack, get_contents(reg));
                advance_pc(pc);
            };
}
function make_restore_ef(inst, machine, stack, pc) {
    const reg = get_register(machine, stack_inst_reg_name(inst));
    return () => {
                set_contents(reg, pop(stack));
                advance_pc(pc);
            };
}
```

The functions `save` and `restore` construct `save` and `restore` instructions. The selector `stack_inst_reg_name` extracts the register name from such instructions.

```
function save(reg) { return list("save", reg); }
function restore(reg) { return list("restore", reg); }
function stack_inst_reg_name(stack_instruction) {
    return head(tail(stack_instruction));
}
```

The final instruction type, handled by `make_perform_ef`, generates an execution function for the action to be performed. At simulation time, the action function is executed and the `pc` advanced.

```
function make_perform_ef(inst, machine, labels, operations, pc) {
    const action = perform_action(inst);
    if (is_operation_exp(action)) {
        const action_fun = make_operation_exp_ef(action, machine,
                                                 labels, operations);
        return () => {
                    action_fun();
                    advance_pc(pc);
                };
    } else {
        error(inst, "bad perform instruction -- assemble");
    }
}
```

The function `perform` constructs `perform` instructions. The selector `perform_action` extracts the action from a `perform` instruction.

```
function perform(action) { return list("perform", action); }
function perform_action(perform_instruction) {
    return head(tail(perform_instruction));
}
```

Execution functions for subexpressions

The value of a `reg`, `label`, or `constant` expression may be needed for assignment to a register (`make_assign_ef`, above) or for input to an operation (`make_operation_exp_ef`, below). The following function generates execution functions to produce values for these expressions during the simulation:

```
function make_primitive_exp_ef(exp, machine, labels) {
    if (is_constant_exp(exp)) {
        const c = constant_exp_value(exp);
        return () => c;
    } else if (is_label_exp(exp)) {
        const insts = lookup_label(labels, label_exp_label(exp));
        return () => insts;
    } else if (is_register_exp(exp)) {
        const r = get_register(machine, register_exp_reg(exp));
        return () => get_contents(r);
    } else {
        error(exp, "unknown expression type -- assemble");
    }
}
```

The syntax of `reg`, `label`, and `constant` expressions is determined by the following constructor functions, along with corresponding predicates and selectors.

```
function reg(name) { return list("reg", name); }
function is_register_exp(exp) { return is_tagged_list(exp, "reg"); }
function register_exp_reg(exp) { return head(tail(exp)); }

function constant(value) { return list("constant", value); }
function is_constant_exp(exp) {
    return is_tagged_list(exp, "constant");
}
function constant_exp_value(exp) { return head(tail(exp)); }

function label(name) { return list("label", name); }
function is_label_exp(exp) { return is_tagged_list(exp, "label"); }
function label_exp_label(exp) { return head(tail(exp)); }
```

The instructions `assign`, `perform`, and `test` may include the application of a machine operation (specified by an `op` expression) to some operands (specified by `reg` and `constant` expressions). The following function produces an execution function for an "operation expression"—a list containing the operation and operand expressions from the instruction:

```
function make_operation_exp_ef(exp, machine, labels, operations) {
    const op = lookup_prim(operation_exp_op(exp), operations);
    const afuns = map(e => make_primitive_exp_ef(e, machine, labels),
                      operation_exp_operands(exp));
    return () => apply_in_underlying_javascript(
                     op, map(f => f(), afuns));
}
```

The syntax of operation expressions is determined by

```
function op(name) { return list("op", name); }
function is_operation_exp(exp) {
    return is_pair(exp) && is_tagged_list(head(exp), "op");
}
function operation_exp_op(op_exp) { return head(tail(head(op_exp))); }
function operation_exp_operands(op_exp) { return tail(op_exp); }
```

Observe that the treatment of operation expressions is very much like the treatment of function applications by the `analyze_application` function in the evaluator of section 4.1.7 in that we generate an execution function for each operand. At simulation time, we call the operand functions and apply the JavaScript function that simulates the operation to the resulting values. We make use of the function `apply_in_underlying_javascript`, as we did in `apply_primitive_function` in section 4.1.4. This is needed to apply `op` to all elements of the argument list `afuns` produced by the first `map`, as if they were separate arguments to `op`. Without this, `op` would have been restricted to be a unary function.

The simulation function is found by looking up the operation name in the operation table for the machine:

```
function lookup_prim(symbol, operations) {
    const val = assoc(symbol, operations);
    return is_undefined(val)
           ? error(symbol, "unknown operation -- assemble")
           : head(tail(val));
}
```

Exercise 5.9

The treatment of machine operations above permits them to operate on labels as well as on constants and the contents of registers. Modify the expression-processing functions to enforce the condition that operations can be used only with registers and constants.

Exercise 5.10

When we introduced `save` and restore in section 5.1.4, we didn't specify what would happen if you tried to restore a register that was not the last one saved, as in the sequence

```
save(y);
save(x);
restore(y);
```

There are several reasonable possibilities for the meaning of `restore`:

a. `restore(y)` puts into `y` the last value saved on the stack, regardless of what register that value came from. This is the way our simulator behaves. Show how to take advantage of this behavior to eliminate one instruction from the Fibonacci machine of section 5.1.4 (figure 5.12).

b. `restore(y)` puts into `y` the last value saved on the stack, but only if that value was saved from `y`; otherwise, it signals an error. Modify the simulator to behave this way. You will have to change `save` to put the register name on the stack along with the value.

c. `restore(y)` puts into `y` the last value saved from `y` regardless of what other registers were saved after `y` and not restored. Modify the simulator to behave this way. You will have to associate a separate stack with each register. You should make the `initialize_stack` operation initialize all the register stacks.

Exercise 5.11

The simulator can be used to help determine the data paths required for implementing a machine with a given controller. Extend the assembler to store the following information in the machine model:

- a list of all instructions, with duplicates removed, sorted by instruction type (`assign`, `go_to`, and so on);
- a list (without duplicates) of the registers used to hold entry points (these are the registers referenced by `go_to` instructions);
- a list (without duplicates) of the registers that are `saved` or `restored`;
- for each register, a list (without duplicates) of the sources from which it is assigned (for example, the sources for register `val` in the factorial machine of figure 5.11 are `constant(1)` and `list(op("*"), reg("n"), reg("val"))`).

Extend the message-passing interface to the machine to provide access to this new information. To test your analyzer, define the Fibonacci machine from figure 5.12 and examine the lists you constructed.

Exercise 5.12

Modify the simulator so that it uses the controller sequence to determine what registers the machine has rather than requiring a list of registers as an argument to `make_machine`. Instead of preallocating the registers in `make_machine`, you can allocate them one at a time when they are first seen during assembly of the instructions.

5.2.4 Monitoring Machine Performance

Simulation is useful not only for verifying the correctness of a proposed machine design but also for measuring the machine's performance. For example, we can install in our simulation program a "meter" that measures the number of stack operations used in a computation. To do this, we modify our simulated stack to keep track of the number of times registers are saved on the stack and the maximum depth reached by the stack, and add a message to the stack's interface that prints the statistics, as shown below. We also add an operation to the basic machine model to print the stack statistics, by initializing `the_ops` in `make_new_machine` to

```
list(list("initialize_stack",
          () => stack("initialize")),
     list("print_stack_statistics",
          () => stack("print_statistics")));
```

Here is the new version of `make_stack`:

```
function make_stack() {
    let stack = null;
    let number_pushes = 0;
    let max_depth = 0;
    let current_depth = 0;
    function push(x) {
        stack = pair(x, stack);
        number_pushes = number_pushes + 1;
        current_depth = current_depth + 1;
        max_depth = math_max(current_depth, max_depth);
        return "done";
    }
    function pop() {
        if (is_null(stack)) {
            error("empty stack -- pop");
        } else {
            const top = head(stack);
            stack = tail(stack);
            current_depth = current_depth - 1;
            return top;
        }
    }
    function initialize() {
        stack = null;
        number_pushes = 0;
        max_depth = 0;
        current_depth = 0;
        return "done";
    }
    function print_statistics() {
        display("total pushes = " + stringify(number_pushes));
        display("maximum depth = " + stringify(max_depth));
    }
    function dispatch(message) {
        return message === "push"
               ? push
               : message === "pop"
               ? pop()
               : message === "initialize"
               ? initialize()
               : message === "print_statistics"
               ? print_statistics()
               : error(message, "unknown request -- stack");
    }
    return dispatch;
}
```

Exercises 5.14 through 5.18 describe other useful monitoring and debugging features that can be added to the register-machine simulator.

Exercise 5.13

Measure the number of pushes and the maximum stack depth required to compute $n!$ for various small values of n using the factorial machine shown in figure 5.11. From your data determine formulas in terms of n for the total number of push operations and the maximum stack depth used in computing $n!$ for any $n > 1$. Note that each of these is a linear function of n and is thus determined by two constants. In order to get the statistics printed, you will have to augment the factorial machine with instructions to initialize the stack and print the statistics. You may want to also modify the machine so that it repeatedly reads a value for n, computes the factorial, and prints the result (as we did for the GCD machine in figure 5.4), so that you will not have to repeatedly invoke `get_register_contents`, `set_register_contents`, and `start`.

Exercise 5.14

Add *instruction counting* to the register machine simulation. That is, have the machine model keep track of the number of instructions executed. Extend the machine model's interface to accept a new message that prints the value of the instruction count and resets the count to zero.

Exercise 5.15

Augment the simulator to provide for *instruction tracing*. That is, before each instruction is executed, the simulator should print the instruction. Make the machine model accept `trace_on` and `trace_off` messages to turn tracing on and off.

Exercise 5.16

Extend the instruction tracing of exercise 5.15 so that before printing an instruction, the simulator prints any labels that immediately precede that instruction in the controller sequence. Be careful to do this in a way that does not interfere with instruction counting (exercise 5.14). You will have to make the simulator retain the necessary label information.

Exercise 5.17

Modify the `make_register` function of section 5.2.1 so that registers can be traced. Registers should accept messages that turn tracing on and off. When a register is traced, assigning a value to the register should print the name of the register, the old contents of the register, and the new contents being assigned. Extend the interface to the machine model to permit you to turn tracing on and off for designated machine registers.

Exercise 5.18

Alyssa P. Hacker wants a *breakpoint* feature in the simulator to help her debug her machine designs. You have been hired to install this feature for her. She wants to be able to specify a place in the controller sequence where the simulator will stop and allow her to examine the state of the machine. You are to implement a function

`set_breakpoint(`*machine*`, `*label*`, `*n*`)`

that sets a breakpoint just before the *n*th instruction after the given label. For example,

```
set_breakpoint(gcd_machine, "test_b", 4)
```

installs a breakpoint in `gcd_machine` just before the assignment to register `a`. When the simulator reaches the breakpoint it should print the label and the offset of the breakpoint and stop executing instructions. Alyssa can then use `get_register_contents` and `set_register_contents` to manipulate the state of the simulated machine. She should then be able to continue execution by saying

```
proceed_machine(machine)
```

She should also be able to remove a specific breakpoint by means of

```
cancel_breakpoint(machine, label, n)
```

or to remove all breakpoints by means of

```
cancel_all_breakpoints(machine)
```

5.3 Storage Allocation and Garbage Collection

In section 5.4, we will show how to implement a JavaScript evaluator as a register machine. In order to simplify the discussion, we will assume that our register machines can be equipped with a *list-structured memory*, in which the basic operations for manipulating list-structured data are primitive. Postulating the existence of such a memory is a useful abstraction when one is focusing on the mechanisms of control in an interpreter, but this does not reflect a realistic view of the actual primitive data operations of contemporary computers. To obtain a more complete picture of how systems can support list-structured memory efficiently, we must investigate how list structure can be represented in a way that is compatible with conventional computer memories.

There are two considerations in implementing list structure. The first is purely an issue of representation: how to represent the "box-and-pointer" structure of pairs, using only the storage and addressing capabilities of typical computer memories. The second issue concerns the management of memory as a computation proceeds. The operation of a JavaScript system depends crucially on the ability to continually create new data objects. These include objects that are explicitly created by the JavaScript functions being interpreted as well as structures created by the interpreter itself, such as environments and argument lists. Although the constant creation of new data objects would pose no problem on a computer with an infinite amount of rapidly addressable memory, computer memories are available only in finite sizes (more's the pity). JavaScript thus provide an *automatic storage allocation* facility to support the illusion of an infinite memory. When a data object is no longer needed, the memory allocated to it is automatically recycled and used to construct new data objects. There are various techniques for providing such automatic storage allocation. The method we shall discuss in this section is called *garbage collection*.

5.3.1 Memory as Vectors

A conventional computer memory can be thought of as an array of cubbyholes, each of which can contain a piece of information. Each cubbyhole has a unique name, called its *address* or *location*. Typical memory systems provide two primitive operations: one that fetches the data stored in a specified location and one that assigns new data to a specified location. Memory addresses can be incremented to support sequential access to some set of the cubbyholes. More generally, many important data operations require that memory addresses be treated as data, which can be stored in memory locations and manipulated in machine registers. The representation of list structure is one application of such *address arithmetic*.

To model computer memory, we use a new kind of data structure called a *vector*. Abstractly, a vector is a compound data object whose individual elements can be accessed by means of an integer index in an amount of time that is independent of the index.[6] In order to describe memory operations, we use two functions for manipulating vectors:[7]

- `vector_ref(`*vector*`, `*n*`)` returns the *n*th element of the vector.
- `vector_set(`*vector*`, `*n*`, `*value*`)` sets the *n*th element of the vector to the designated value.

For example, if `v` is a vector, then `vector_ref(v, 5)` gets the fifth entry in the vector `v` and `vector_set(v, 5, 7)` changes the value of the fifth entry of the vector `v` to 7.[8] For computer memory, this access can be implemented through the use of address arithmetic to combine a *base address* that specifies the beginning location of a vector in memory with an *index* that specifies the offset of a particular element of the vector.

Representing data

We can use vectors to implement the basic pair structures required for a list-structured memory. Let us imagine that computer memory is divided into two vectors: `the_heads` and `the_tails`. We will represent list structure as follows: A pointer to a pair is an index into the two vectors. The `head` of the pair is the entry in `the_heads` with the designated index, and the tail of the pair is the entry in `the_tails` with the designated index. We also need a representation for objects other than pairs (such as numbers and strings) and a way to distinguish one kind of data from another. There are many methods of accomplishing this, but they all reduce to using *typed pointers*,

---

6. We could represent memory as lists of items. However, the access time would then not be independent of the index, since accessing the *n*th element of a list requires $n-1$ `tail` operations.

7. As mentioned in section 4.1.4 (footnote 18), JavaScript supports vectors as data structures and calls them "arrays." We use the term *vector* in this book, as it is the more common terminology. The vector functions above are easily implemented using JavaScript's primitive array support.

8. For completeness, we should specify a `make_vector` operation that constructs vectors. However, in the present application we will use vectors only to model fixed divisions of the computer memory.

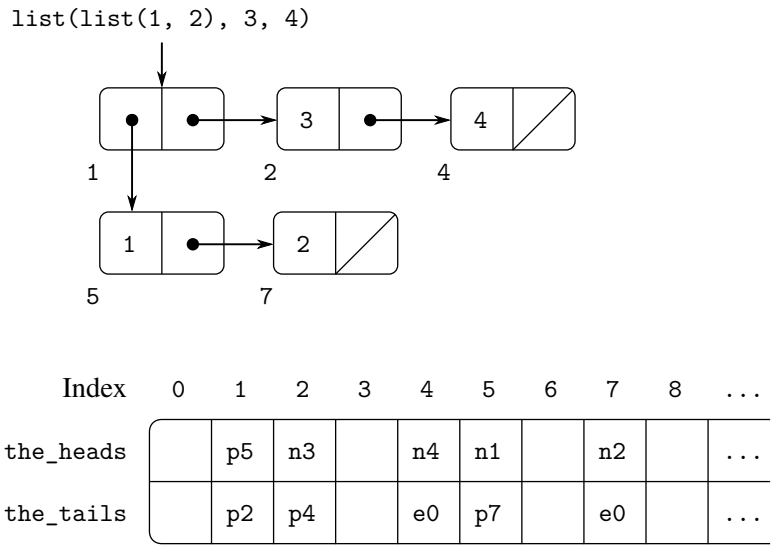


Figure 5.14 Box-and-pointer and memory-vector representations of the list `list(list(1, 2), 3, 4)`.

that is, to extending the notion of "pointer" to include information on data type.[9] The data type enables the system to distinguish a pointer to a pair (which consists of the "pair" data type and an index into the memory vectors) from pointers to other kinds of data (which consist of some other data type and whatever is being used to represent data of that type). Two data objects are considered to be the same (`===`) if their pointers are identical. Figure 5.14 illustrates the use of this method to represent `list(list(1, 2), 3, 4)`, whose box-and-pointer diagram is also shown. We use letter prefixes to denote the data-type information. Thus, a pointer to the pair with index 5 is denoted `p5`, the empty list is denoted by the pointer `e0`, and a pointer to the number 4 is denoted `n4`. In the box-and-pointer diagram, we have indicated at the lower left of each pair the vector index that specifies where the `head` and `tail` of the pair are stored. The blank locations in `the_heads` and `the_tails` may contain parts of other list structures (not of interest here).

---

9. This is precisely the same "tagged data" idea we introduced in chapter 2 for dealing with generic operations. Here, however, the data types are included at the primitive machine level rather than constructed through the use of lists.

Type information may be encoded in a variety of ways, depending on the details of the machine on which the JavaScript system is to be implemented. The execution efficiency of JavaScript programs will be strongly dependent on how cleverly this choice is made, but it is difficult to formulate general design rules for good choices. The most straightforward way to implement typed pointers is to allocate a fixed set of bits in each pointer to be a *type field* that encodes the data type. Important questions to be addressed in designing such a representation include the following: How many type bits are required? How large must the vector indices be? How efficiently can the primitive machine instructions be used to manipulate the type fields of pointers? Machines that include special hardware for the efficient handling of type fields are said to have *tagged architectures*.

A pointer to a number, such as `n4`, might consist of a type indicating numeric data together with the actual representation of the number 4.[10] To deal with numbers that are too large to be represented in the fixed amount of space allocated for a single pointer, we could use a distinct *bignum* data type, for which the pointer designates a list in which the parts of the number are stored.[11]

A string might be represented as a typed pointer that designates a sequence of the characters that form the string's printed representation. The parser constructs such a sequence when it encounters a string literal, and the string-concatenation operator `+` and string-producing primitive functions such as `stringify` construct such a sequence. Since we want two instances of a string to be recognized as the "same" string by `===` and we want `===` to be a simple test for equality of pointers, we must ensure that if the system sees the same string twice, it will use the same pointer (to the same sequence of characters) to represent both occurrences. To accomplish this, the system maintains a table, called the *string pool*, of all the strings it has ever encountered. When the system is about to construct a string, it checks the string pool to see if it has ever before seen the same string. If it has not, it constructs a new string (a typed pointer to a new character sequence) and enters this pointer in the string pool. If the system has seen the string before, it returns the string pointer stored in the string pool. This process of replacing strings by unique pointers is called *string interning*.

Implementing the primitive list operations

Given the above representation scheme, we can replace each "primitive" list operation of a register machine with one or more primitive vector operations. We will use two registers, `the_heads` and `the_tails`, to identify the memory vectors, and will assume that `vector_ref` and `vector_set` are available as primitive operations. We also assume that numeric operations on pointers (such as incrementing a pointer, using a pair pointer to index a vector, or adding two numbers) use only the index portion of the typed pointer.

For example, we can make a register machine support the instructions

`assign(`reg_1`, list(op("head"), reg(`reg_2`)))`

`assign(`reg_1`, list(op("tail"), reg(`reg_2`)))`

if we implement these, respectively, as

`assign(`reg_1`, list(op("vector_ref"), reg("the_heads"), reg(`reg_2`)))`

`assign(`reg_1`, list(op("vector_ref"), reg("the_tails"), reg(`reg_2`)))`

---

10. This decision on the representation of numbers determines whether `===`, which tests equality of pointers, can be used to test for equality of numbers. If the pointer contains the number itself, then equal numbers will have the same pointer. But if the pointer contains the index of a location where the number is stored, equal numbers will be guaranteed to have equal pointers only if we are careful never to store the same number in more than one location.

11. This is just like writing a number as a sequence of digits, except that each "digit" is a number between 0 and the largest number that can be stored in a single pointer.

The instructions

```
perform(list(op("set_head"), reg(reg1), reg(reg2)))
```

```
perform(list(op("set_tail"), reg(reg1), reg(reg2)))
```

are implemented as

```
perform(list(op("vector_set"), reg("the_heads"), reg(reg1), reg(reg2)))
```

```
perform(list(op("vector_set"), reg("the_tails"), reg(reg1), reg(reg2)))
```

The operation `pair` is performed by allocating an unused index and storing the arguments to `pair` in `the_heads` and `the_tails` at that indexed vector position. We presume that there is a special register, `free`, that always holds a pair pointer containing the next available index, and that we can increment the index part of that pointer to find the next free location.[12] For example, the instruction

```
assign(reg1, list(op("pair"), reg(reg2), reg(reg3)))
```

is implemented as the following sequence of vector operations:[13]

```
perform(list(op("vector_set"),
             reg("the_heads"), reg("free"), reg(reg2))),
perform(list(op("vector_set"),
             reg("the_tails"), reg("free"), reg(reg3))),
assign(reg1, reg("free")),
assign("free", list(op("+"), reg("free"), constant(1)))
```

The `===` operation

```
list(op("==="), reg(reg1), reg(reg2))
```

simply tests the equality of all fields in the registers, and predicates such as `is_pair`, `is_null`, `is_string`, and `is_number` need only check the type field.

Implementing stacks

Although our register machines use stacks, we need do nothing special here, since stacks can be modeled in terms of lists. The stack can be a list of the saved values, pointed to by a special register `the_stack`. Thus, `save(`*reg*`)` can be implemented as

```
assign("the_stack", list(op("pair"), reg(reg), reg("the_stack")))
```

Similarly, `restore(`*reg*`)` can be implemented as

```
assign(reg, list(op("head"), reg("the_stack")))
assign("the_stack", list(op("tail"), reg("the_stack")))
```

---

12. There are other ways of finding free storage. For example, we could link together all the unused pairs into a *free list*. Our free locations are consecutive (and hence can be accessed by incrementing a pointer) because we are using a compacting garbage collector, as we will see in section 5.3.2.

13. This is essentially the implementation of `pair` in terms of `set_head` and `set_tail`, as described in section 3.3.1. The operation `get_new_pair` used in that implementation is realized here by the `free` pointer.

and `perform(list(op("initialize_stack")))` can be implemented as

```
assign("the_stack", constant(null))
```

These operations can be further expanded in terms of the vector operations given above. In conventional computer architectures, however, it is usually advantageous to allocate the stack as a separate vector. Then pushing and popping the stack can be accomplished by incrementing or decrementing an index into that vector.

Exercise 5.19

Draw the box-and-pointer representation and the memory-vector representation (as in figure 5.14) of the list structure produced by

```
const x = pair(1, 2);
const y = list(x, x);
```

with the `free` pointer initially `p1`. What is the final value of `free` ? What pointers represent the values of `x` and `y`?

Exercise 5.20

Implement register machines for the following functions. Assume that the list-structure memory operations are available as machine primitives.

a. Recursive `count_leaves`:

```
function count_leaves(tree) {
    return is_null(tree)
           ? 0
           : ! is_pair(tree)
           ? 1
           : count_leaves(head(tree)) +
             count_leaves(tail(tree));
}
```

b. Recursive `count_leaves` with explicit counter:

```
function count_leaves(tree) {
    function count_iter(tree, n) {
        return is_null(tree)
               ? n
               : ! is_pair(tree)
               ? n + 1
               : count_iter(tail(tree),
                            count_iter(head(tree), n));
    }
    return count_iter(tree, 0);
}
```

Exercise 5.21

Exercise 3.12 of section 3.3.1 presented an `append` function that appends two lists to form a new list and an `append_mutator` function that splices two lists together. Design a register machine to implement each of these functions. Assume that the list-structure memory operations are available as primitive operations.

5.3.2 Maintaining the Illusion of Infinite Memory

The representation method outlined in section 5.3.1 solves the problem of implementing list structure, provided that we have an infinite amount of memory. With a real computer we will eventually run out of free space in which to construct new pairs.[14] However, most of the pairs generated in a typical computation are used only to hold intermediate results. After these results are accessed, the pairs are no longer needed—they are *garbage*. For instance, the computation

```
accumulate((x, y) => x + y,
           0,
           filter(is_odd, enumerate_interval(0, n)))
```

constructs two lists: the enumeration and the result of filtering the enumeration. When the accumulation is complete, these lists are no longer needed, and the allocated memory can be reclaimed. If we can arrange to collect all the garbage periodically, and if this turns out to recycle memory at about the same rate at which we construct new pairs, we will have preserved the illusion that there is an infinite amount of memory.

In order to recycle pairs, we must have a way to determine which allocated pairs are not needed (in the sense that their contents can no longer influence the future of the computation). The method we shall examine for accomplishing this is known as *garbage collection*. Garbage collection is based on the observation that, at any moment in an interpretation based on list-structured memory, the only objects that can affect the future of the computation are those that can be reached by some succession of `head` and `tail` operations starting from the pointers that are currently in the machine registers.[15] Any memory cell that is not so accessible may be recycled.

There are many ways to perform garbage collection. The method we shall examine here is called *stop-and-copy*. The basic idea is to divide memory into two halves: "working memory" and "free memory." When `pair` constructs pairs, it allocates these in working memory. When working memory is full, we perform garbage collection by locating all the useful pairs in working memory and copying these into consecutive locations in free memory. (The useful pairs are located by tracing all the `head` and `tail` pointers, starting with the machine registers.) Since we do not copy the garbage, there will presumably be additional free memory that we can use to allocate new pairs. In addition, nothing in the working memory is needed,

---

14. This may not be true eventually, because memories may get large enough so that it would be impossible to run out of free memory in the lifetime of the computer. For example, there are about 3×10^{16} nanoseconds in a year, so if we were to `pair` once per nanosecond we would need about 10^{18} cells of memory to build a machine that could operate for 30 years without running out of memory. That much memory seems absurdly large by today's standards, but it is not physically impossible. On the other hand, processors are getting faster and modern computers have increasingly large numbers of processors operating in parallel on a single memory, so it may be possible to use up memory much faster than we have postulated.

15. We assume here that the stack is represented as a list as described in section 5.3.1, so that items on the stack are accessible via the pointer in the stack register.

since all the useful pairs in it have been copied. Thus, if we interchange the roles of working memory and free memory, we can continue processing; new pairs will be allocated in the new working memory (which was the old free memory). When this is full, we can copy the useful pairs into the new free memory (which was the old working memory).[16]

Implementation of a stop-and-copy garbage collector

We now use our register-machine language to describe the stop-and-copy algorithm in more detail. We will assume that there is a register called `root` that contains a pointer to a structure that eventually points at all accessible data. This can be arranged by storing the contents of all the machine registers in a preallocated list pointed at by `root` just before starting garbage collection.[17] We also assume that, in addition to the current working memory, there is free memory available into which we can copy the useful data. The current working memory consists of vectors whose base addresses are in registers called `the_heads` and `the_tails`, and the free memory is in registers called `new_heads` and `new_tails`.

Garbage collection is triggered when we exhaust the free cells in the current working memory, that is, when a `pair` operation attempts to increment the `free` pointer beyond the end of the memory vector. When the garbage-collection process is complete, the `root` pointer will point into the new memory, all objects accessible from the `root` will have been moved to the new memory, and the `free` pointer will indicate the next place in the new memory where a new pair can be allocated.

---

16. This idea was invented and first implemented by Minsky, as part of the implementation of Lisp for the PDP-1 at the MIT Research Laboratory of Electronics. It was further developed by Fenichel and Yochelson (1969) for use in the Lisp implementation for the Multics time-sharing system. Later, Baker (1978) developed a "real-time" version of the method, which does not require the computation to stop during garbage collection. Baker's idea was extended by Hewitt, Lieberman, and Moon (see Lieberman and Hewitt 1983) to take advantage of the fact that some structure is more volatile and other structure is more permanent.

An alternative commonly used garbage-collection technique is the *mark-sweep* method. This consists of tracing all the structure accessible from the machine registers and marking each pair we reach. We then scan all of memory, and any location that is unmarked is "swept up" as garbage and made available for reuse. A full discussion of the mark-sweep method can be found in Allen 1978.

The Minsky-Fenichel-Yochelson algorithm is the dominant algorithm in use for large-memory systems because it examines only the useful part of memory. This is in contrast to mark-sweep, in which the sweep phase must check all of memory. A second advantage of stop-and-copy is that it is a *compacting* garbage collector. That is, at the end of the garbage-collection phase the useful data will have been moved to consecutive memory locations, with all garbage pairs compressed out. This can be an extremely important performance consideration in machines with virtual memory, in which accesses to widely separated memory addresses may require extra paging operations.

17. This list of registers does not include the registers used by the storage-allocation system: `root`, `the_heads`, `the_tails`, and the other registers that will be introduced in this section.

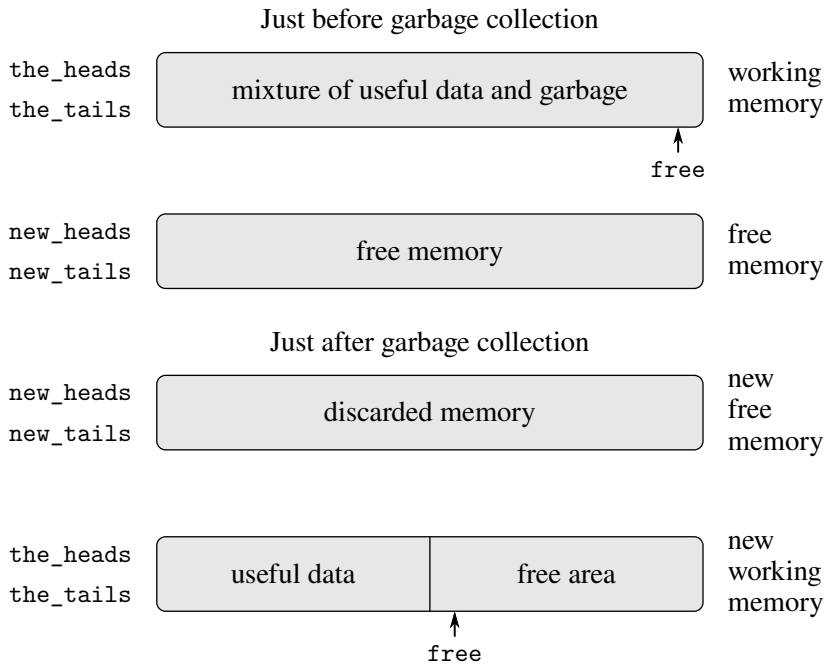


Figure 5.15 Reconfiguration of memory by the garbage-collection process.

In addition, the roles of working memory and new memory will have been interchanged—new pairs will be constructed in the new memory, beginning at the place indicated by `free`, and the (previous) working memory will be available as the new memory for the next garbage collection. Figure 5.15 shows the arrangement of memory just before and just after garbage collection.

The state of the garbage-collection process is controlled by maintaining two pointers: `free` and `scan`. These are initialized to point to the beginning of the new memory. The algorithm begins by relocating the pair pointed at by `root` to the beginning of the new memory. The pair is copied, the `root` pointer is adjusted to point to the new location, and the `free` pointer is incremented. In addition, the old location of the pair is marked to show that its contents have been moved. This marking is done as follows: In the `head` position, we place a special tag that signals that this is an already-moved object. (Such an object is traditionally called a *broken heart*.)[18] In the `tail` position we place a *forwarding address* that points at the location to which the object has been moved.

18. The term *broken heart* was coined by David Cressey, who wrote a garbage collector for MDL, a dialect of Lisp developed at MIT during the early 1970s.

After relocating the root, the garbage collector enters its basic cycle. At each step in the algorithm, the `scan` pointer (initially pointing at the relocated root) points at a pair that has been moved to the new memory but whose `head` and `tail` pointers still refer to objects in the old memory. These objects are each relocated, and the `scan` pointer is incremented. To relocate an object (for example, the object indicated by the `head` pointer of the pair we are scanning) we check to see if the object has already been moved (as indicated by the presence of a broken-heart tag in the `head` position of the object). If the object has not already been moved, we copy it to the place indicated by `free`, update `free`, set up a broken heart at the object's old location, and update the pointer to the object (in this example, the `head` pointer of the pair we are scanning) to point to the new location. If the object has already been moved, its forwarding address (found in the `tail` position of the broken heart) is substituted for the pointer in the pair being scanned. Eventually, all accessible objects will have been moved and scanned, at which point the `scan` pointer will overtake the `free` pointer and the process will terminate.

We can specify the stop-and-copy algorithm as a sequence of instructions for a register machine. The basic step of relocating an object is accomplished by a subroutine called `relocate_old_result_in_new`. This subroutine gets its argument, a pointer to the object to be relocated, from a register named `old`. It relocates the designated object (incrementing `free` in the process), puts a pointer to the relocated object into a register called `new`, and returns by branching to the entry point stored in the register `relocate_continue`. To begin garbage collection, we invoke this subroutine to relocate the `root` pointer, after initializing `free` and `scan`. When the relocation of `root` has been accomplished, we install the new pointer as the new `root` and enter the main loop of the garbage collector.

```
"begin_garbage_collection",
  assign("free", constant(0)),
  assign("scan", constant(0)),
  assign("old", reg("root")),
  assign("relocate_continue", label("reassign_root")),
  go_to(label("relocate_old_result_in_new")),
"reassign_root",
  assign("root", reg("new")),
  go_to(label("gc_loop")),
```

In the main loop of the garbage collector we must determine whether there are any more objects to be scanned. We do this by testing whether the `scan` pointer is coincident with the `free` pointer. If the pointers are equal, then all accessible objects have been relocated, and we branch to `gc_flip`, which cleans things up so that we can continue the interrupted computation. If there are still pairs to be scanned, we call the relocate subroutine to relocate the `head` of the next pair (by placing the `head` pointer in `old`). The `relocate_continue` register is set up so that the subroutine will return to update the `head` pointer.

```
"gc_loop",
  test(list(op("==="), reg("scan"), reg("free"))),
  branch(label("gc_flip")),
  assign("old", list(op("vector_ref"), reg("new_heads"), reg("scan"))),
  assign("relocate_continue", label("update_head")),
  go_to(label("relocate_old_result_in_new")),
```

At `update_head`, we modify the `head` pointer of the pair being scanned, then proceed to relocate the `tail` of the pair. We return to `update_tail` when that relocation has been accomplished. After relocating and updating the `tail`, we are finished scanning that pair, so we continue with the main loop.

```
"update_head",
  perform(list(op("vector_set"),
               reg("new_heads"), reg("scan"), reg("new"))),
  assign("old", list(op("vector_ref"),
                     reg("new_tails"), reg("scan"))),
  assign("relocate_continue", label("update_tail")),
  go_to(label("relocate_old_result_in_new")),

"update_tail",
  perform(list(op("vector_set"),
               reg("new_tails"), reg("scan"), reg("new"))),
  assign("scan", list(op("+"), reg("scan"), constant(1))),
  go_to(label("gc_loop")),
```

The subroutine `relocate_old_result_in_new` relocates objects as follows: If the object to be relocated (pointed at by `old`) is not a pair, then we return the same pointer to the object unchanged (in `new`). (For example, we may be scanning a pair whose `head` is the number 4. If we represent the `head` by `n4`, as described in section 5.3.1, then we want the "relocated" `head` pointer to still be `n4`.) Otherwise, we must perform the relocation. If the `head` position of the pair to be relocated contains a broken-heart tag, then the pair has in fact already been moved, so we retrieve the forwarding address (from the `tail` position of the broken heart) and return this in `new`. If the pointer in `old` points at a yet-unmoved pair, then we move the pair to the first free cell in new memory (pointed at by `free`) and set up the broken heart by storing a broken-heart tag and forwarding address at the old location. The subroutine `relocate_old_result_in_new` uses a register `oldht` to hold the `head` or the `tail` of the object pointed at by `old`.[19]

19. The garbage collector uses the low-level predicate `is_pointer_to_pair` instead of the list-structure `is_pair` operation because in a real system there might be various things that are treated as pairs for garbage-collection purposes. For example, a function object may be implemented as a special kind of "pair" that doesn't satisfy the `is_pair` predicate. For simulation purposes, `is_pointer_to_pair` can be implemented as `is_pair`.

```
"relocate_old_result_in_new",
  test(list(op("is_pointer_to_pair"), reg("old"))),
  branch(label("pair")),
  assign("new", reg("old")),
  go_to(reg("relocate_continue")),
"pair",
  assign("oldht", list(op("vector_ref"),
                       reg("the_heads"), reg("old"))),
  test(list(op("is_broken_heart"), reg("oldht"))),
  branch(label("already_moved")),
  assign("new", reg("free")),      // new location for pair
  // Update free pointer
  assign("free", list(op("+"), reg("free"), constant(1))),
  // Copy the head and tail to new memory
  perform(list(op("vector_set"),
               reg("new_heads"), reg("new"),
               reg("oldht"))),
  assign("oldht", list(op("vector_ref"),
                       reg("the_tails"), reg("old"))),
  perform(list(op("vector_set"),
               reg("new_tails"), reg("new"),
               reg("oldht"))),
  // Construct the broken heart
  perform(list(op("vector_set"),
               reg("the_heads"), reg("old"),
               constant("broken_heart"))),
  perform(list(op("vector_set"),
               reg("the_tails"), reg("old"),
               reg("new"))),
  go_to(reg("relocate_continue")),
"already_moved",
  assign("new", list(op("vector_ref"),
                     reg("the_tails"), reg("old"))),
  go_to(reg("relocate_continue")),
```

At the very end of the garbage collection process, we interchange the role of old and new memories by interchanging pointers: interchanging `the_heads` with `new_heads`, and `the_tails` with `new_tails`. We will then be ready to perform another garbage collection the next time memory runs out.

```
"gc_flip",
  assign("temp", reg("the_tails")),
  assign("the_tails", reg("new_tails")),
  assign("new_tails", reg("temp")),
  assign("temp", reg("the_heads")),
  assign("the_heads", reg("new_heads")),
  assign("new_heads", reg("temp"))
```

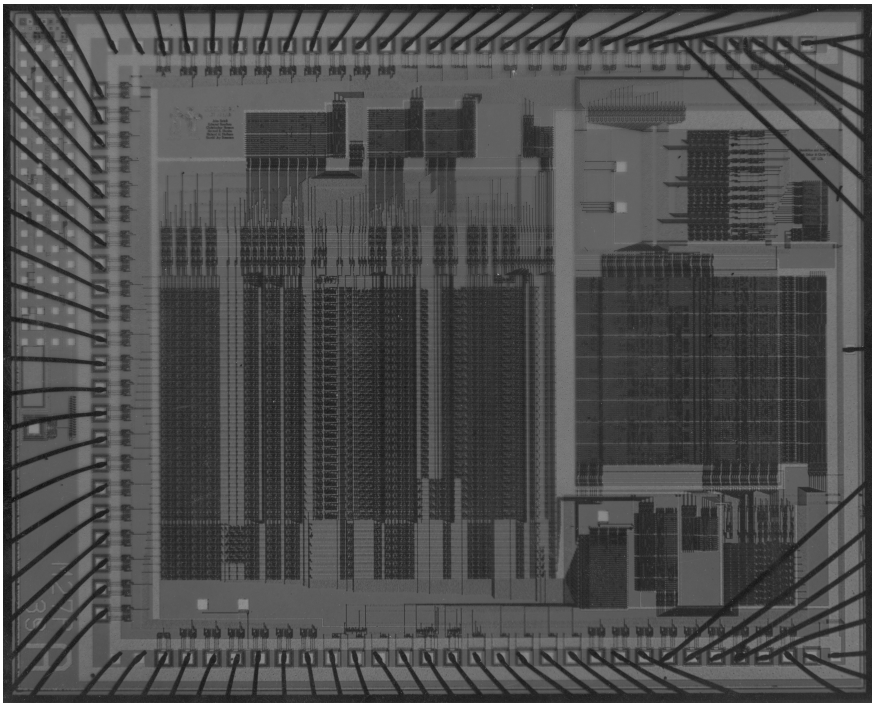


Figure 5.16 A silicon-chip implementation of an evaluator for Scheme.

5.4 The Explicit-Control Evaluator

In section 5.1 we saw how to transform simple JavaScript programs into descriptions of register machines. We will now perform this transformation on a more complex program, the metacircular evaluator of sections 4.1.1–4.1.4, which shows how the behavior of a JavaScript interpreter can be described in terms of the functions `evaluate` and `apply`. The *explicit-control evaluator* that we develop in this section shows how the underlying function-calling and argument-passing mechanisms used in the evaluation process can be described in terms of operations on registers and stacks. In addition, the explicit-control evaluator can serve as an implementation of a JavaScript interpreter, written in a language that is very similar to the native machine language of conventional computers. The evaluator can be executed by the register-machine simulator of section 5.2. Alternatively, it can be used as a starting point for building a machine-language implementation of a JavaScript evaluator, or even a special-purpose machine for evaluating JavaScript programs. Figure 5.16 shows such a hardware implementation: a silicon chip that acts as an evaluator for Scheme, the language used in place of JavaScript in the original edition of this book. The chip designers started with the data-path and controller specifications for a register machine similar to the evaluator described in this section and used design automation programs to construct the integrated-circuit layout.[20]

20. See Batali et al. 1982 for more information on the chip and the method by which it was designed.

Registers and operations

In designing the explicit-control evaluator, we must specify the operations to be used in our register machine. We described the metacircular evaluator in terms of abstract syntax, using functions such as `is_literal` and `make_function`. In implementing the register machine, we could expand these functions into sequences of elementary list-structure memory operations, and implement these operations on our register machine. However, this would make our evaluator very long, obscuring the basic structure with details. To clarify the presentation, we will include as primitive operations of the register machine the syntax functions given in section 4.1.2 and the functions for representing environments and other runtime data given in sections 4.1.3 and 4.1.4. In order to completely specify an evaluator that could be programmed in a low-level machine language or implemented in hardware, we would replace these operations by more elementary operations, using the list-structure implementation we described in section 5.3.

Our JavaScript evaluator register machine includes a stack and seven registers: `comp`, `env`, `val`, `continue`, `fun`, `argl`, and `unev`. The `comp` register is used to hold the component to be evaluated, and `env` contains the environment in which the evaluation is to be performed. At the end of an evaluation, `val` contains the value obtained by evaluating the component in the designated environment. The `continue` register is used to implement recursion, as explained in section 5.1.4. (The evaluator needs to call itself recursively, since evaluating a component requires evaluating its subcomponents.) The registers `fun`, `argl`, and `unev` are used in evaluating function applications.

We will not provide a data-path diagram to show how the registers and operations of the evaluator are connected, nor will we give the complete list of machine operations. These are implicit in the evaluator's controller, which will be presented in detail.

5.4.1 The Dispatcher and Basic Evaluation

The central element in the evaluator is the sequence of instructions beginning at `eval_dispatch`. This corresponds to the `evaluate` function of the metacircular evaluator described in section 4.1.1. When the controller starts at `eval_dispatch`, it evaluates the component specified by `comp` in the environment specified by `env`. When evaluation is complete, the controller will go to the entry point stored in `continue`, and the `val` register will hold the value of the component. As with the metacircular `evaluate`, the structure of `eval_dispatch` is a case analysis on the syntactic type of the component to be evaluated.[21]

---

21. In our controller, the dispatch is written as a sequence of `test` and `branch` instructions. Alternatively, it could have been written in a data-directed style, which avoids the need to perform sequential tests and facilitates the definition of new component types.

```
"eval_dispatch",
  test(list(op("is_literal"), reg("comp"))),
  branch(label("ev_literal")),
  test(list(op("is_name"), reg("comp"))),
  branch(label("ev_name")),
  test(list(op("is_application"), reg("comp"))),
  branch(label("ev_application")),
  test(list(op("is_operator_combination"), reg("comp"))),
  branch(label("ev_operator_combination")),
  test(list(op("is_conditional"), reg("comp"))),
  branch(label("ev_conditional")),
  test(list(op("is_lambda_expression"), reg("comp"))),
  branch(label("ev_lambda")),
  test(list(op("is_sequence"), reg("comp"))),
  branch(label("ev_sequence")),
  test(list(op("is_block"), reg("comp"))),
  branch(label("ev_block")),
  test(list(op("is_return_statement"), reg("comp"))),
  branch(label("ev_return")),
  test(list(op("is_function_declaration"), reg("comp"))),
  branch(label("ev_function_declaration")),
  test(list(op("is_declaration"), reg("comp"))),
  branch(label("ev_declaration")),
  test(list(op("is_assignment"), reg("comp"))),
  branch(label("ev_assignment")),
  go_to(label("unknown_component_type")),
```

Evaluating simple expressions

Numbers and strings, names, and lambda expressions have no subexpressions to be evaluated. For these, the evaluator simply places the correct value in the `val` register and continues execution at the entry point specified by `continue`. Evaluation of simple expressions is performed by the following controller code:

```
"ev_literal",
  assign("val", list(op("literal_value"), reg("comp"))),
  go_to(reg("continue")),

"ev_name",
  assign("val", list(op("symbol_of_name"), reg("comp"), reg("env"))),
  assign("val", list(op("lookup_symbol_value"),
                     reg("val"), reg("env"))),
  go_to(reg("continue")),

"ev_lambda",
  assign("unev", list(op("lambda_parameter_symbols"), reg("comp"))),
  assign("comp", list(op("lambda_body"), reg("comp"))),
  assign("val", list(op("make_function"),
                     reg("unev"), reg("comp"), reg("env"))),
  go_to(reg("continue")),
```

Observe how `ev_lambda` uses the `unev` and `comp` registers to hold the parameters and body of the lambda expression so that they can be passed to the `make_function` operation, along with the environment in `env`.

Conditionals

As with the metacircular evaluator, syntactic forms are handled by selectively evaluating fragments of the component. For a conditional, we must evaluate the predicate and decide, based on the value of predicate, whether to evaluate the consequent or the alternative.

Before evaluating the predicate, we save the conditional itself, which is in `comp`, so that we can later extract the consequent or alternative. To evaluate the predicate expression, we move it to the `comp` register and go to `eval_dispatch`. The environment in the `env` register is already the correct one in which to evaluate the predicate. However, we save `env` because we will need it later to evaluate the consequent or the alternative. We set up `continue` so that evaluation will resume at `ev_conditional_decide` after the predicate has been evaluated. First, however, we save the old value of `continue`, which we will need later in order to return to the evaluation of the statement that is waiting for the value of the conditional.

```
"ev_conditional",
  save("comp"), // save conditional for later
  save("env"),
  save("continue"),
  assign("continue", label("ev_conditional_decide")),
  assign("comp", list(op("conditional_predicate"), reg("comp"))),
  go_to(label("eval_dispatch")), // evaluate the predicate
```

When we resume at `ev_conditional_decide` after evaluating the predicate, we test whether it was true or false and, depending on the result, place either the consequent or the alternative in `comp` before going to `eval_dispatch`.[22] Notice that restoring `env` and `continue` here sets up `eval_dispatch` to have the correct environment and to continue at the right place to receive the value of the conditional.

```
"ev_conditional_decide",
  restore("continue"),
  restore("env"),
  restore("comp"),
  test(list(op("is_falsy"), reg("val"))),
  branch(label("ev_conditional_alternative")),
"ev_conditional_consequent",
  assign("comp", list(op("conditional_consequent"), reg("comp"))),
  go_to(label("eval_dispatch")),
"ev_conditional_alternative",
  assign("comp", list(op("conditional_alternative"), reg("comp"))),
  go_to(label("eval_dispatch")),
```

Sequence Evaluation

The portion of the explicit-control evaluator beginning at `ev_sequence`, which handles sequences of statements, is analogous to the metacircular evaluator's `eval_sequence` function.

---

22. In this chapter, we will use the function `is_falsy` to test the value of the predicate. This allows us to write the consequent and alternative branches in the same order as in a conditional, and simply fall through to the consequent branch when the predicate holds. The function `is_falsy` is declared as the opposite of the `is_truthy` function used to test predicates of conditionals in section 4.1.1.

The entries at ev_sequence_next and ev_sequence_continue form a loop that successively evaluates each statement in a sequence. The list of unevaluated statements is kept in unev. At ev_sequence we place the sequence of statements to be evaluated in unev. If the sequence is empty, we set val to undefined and jump to continue via ev_sequence_empty. Otherwise we start the sequence-evaluation loop, first saving the value of continue on the stack, because the continue register will be used for local flow of control in the loop, and the original value is needed for continuing after the statement sequence. Before evaluating each statement, we check to see if there are additional statements to be evaluated in the sequence. If so, we save the rest of the unevaluated statements (held in unev) and the environment in which these must be evaluated (held in env) and call eval_dispatch to evaluate the statement, which has been placed in comp. The two saved registers are restored after this evaluation, at ev_sequence_continue.

The final statement in the sequence is handled differently, at the entry point ev_sequence_last_statement. Since there are no more statements to be evaluated after this one, we need not save unev or env before going to eval_dispatch. The value of the whole sequence is the value of the last statement, so after the evaluation of the last statement there is nothing left to do except continue at the entry point that was saved at ev_sequence. Rather than setting up continue to arrange for eval_dispatch to return here and then restoring continue from the stack and continuing at that entry point, we restore continue from the stack before going to eval_dispatch, so that eval_dispatch will continue at that entry point after evaluating the statement.

```
"ev_sequence",
  assign("unev", list(op("sequence_statements"), reg("comp"))),
  test(list(op("is_empty_sequence"), reg("unev"))),
  branch(label("ev_sequence_empty")),
  save("continue"),
"ev_sequence_next",
  assign("comp", list(op("first_statement"), reg("unev"))),
  test(list(op("is_last_statement"), reg("unev"))),
  branch(label("ev_sequence_last_statement")),
  save("unev"),
  save("env"),
  assign("continue", label("ev_sequence_continue")),
  go_to(label("eval_dispatch")),
"ev_sequence_continue",
  restore("env"),
  restore("unev"),
  assign("unev", list(op("rest_statements"), reg("unev"))),
  go_to(label("ev_sequence_next")),
"ev_sequence_last_statement",
  restore("continue"),
  go_to(label("eval_dispatch")),

"ev_sequence_empty",
  assign("val", constant(undefined)),
  go_to(reg("continue")),
```

Unlike eval_sequence in the metacircular evaluator, ev_sequence does not need to check whether a return statement was evaluated so as to terminate the sequence evaluation. The "explicit control" in this evaluator allows a return statement

to jump directly to the continuation of the current function application without resuming the sequence evaluation. Thus sequence evaluation does not need to be concerned with returns, or even be aware of the existence of return statements in the language. Because a return statement jumps out of the sequence-evaluation code, the restores of saved registers at `ev_sequence_continue` won't be executed. We will see later how the return statement removes these values from the stack.

5.4.2 Evaluating Function Applications

A function application is specified by a combination containing a function expression and argument expressions. The function expression is a subexpression whose value is a function, and the argument expressions are subexpressions whose values are the arguments to which the function should be applied. The metacircular `evaluate` handles applications by calling itself recursively to evaluate each element of the combination, and then passing the results to `apply`, which performs the actual function application. The explicit-control evaluator does the same thing; these recursive calls are implemented by `go_to` instructions, together with use of the stack to save registers that will be restored after the recursive call returns. Before each call we will be careful to identify which registers must be saved (because their values will be needed later).[23]

As in the metacircular evaluator, operator combinations are transformed into applications of primitive functions corresponding to the operators. This takes place at `ev_operator_combination`, which performs this transformation in place in `comp` and falls through to `ev_application`.[24]

We begin the evaluation of an application by evaluating the function expression to produce a function, which will later be applied to the evaluated argument expressions. To evaluate the function expression, we move it to the `comp` register and go to `eval_dispatch`. The environment in the `env` register is already the correct one in which to evaluate the function expression. However, we save `env` because we will need it later to evaluate the argument expressions. We also extract the argument expressions into `unev` and save this on the stack. We set up `continue` so that `eval_dispatch` will resume at `ev_appl_did_function_expression` after the function expression has been evaluated. First, however, we save the old value of `continue`, which tells the controller where to continue after the application.

---

23. This is an important but subtle point in translating algorithms from a procedural language, such as JavaScript, to a register-machine language. As an alternative to saving only what is needed, we could save all the registers (except `val`) before each recursive call. This is called a *framed-stack* discipline. This would work but might save more registers than necessary; this could be an important consideration in a system where stack operations are expensive. Saving registers whose contents will not be needed later may also hold on to useless data that could otherwise be garbage-collected, freeing space to be reused.

24. We assume that the syntax transformer `operator_combination_to_application` is available as a machine operation. In an actual implementation built from scratch, we would use our explicit-control evaluator to interpret a JavaScript program that performs source-level transformations like this one and `function_decl_to_constant_decl` in a syntax phase that runs before execution.

```
"ev_operator_combination",
  assign("comp", list(op("operator_combination_to_application"),
                      reg("comp"), reg("env"))),
"ev_application",
  save("continue"),
  save("env"),
  assign("unev", list(op("arg_expressions"), reg("comp"))),
  save("unev"),
  assign("comp", list(op("function_expression"), reg("comp"))),
  assign("continue", label("ev_appl_did_function_expression")),
  go_to(label("eval_dispatch")),
```

Upon returning from evaluating the function expression, we proceed to evaluate the argument expressions of the application and to accumulate the resulting arguments in a list, held in `argl`. (This is like the evaluation of a sequence of statements, except that we collect the values.) First we restore the unevaluated argument expressions and the environment. We initialize `argl` to an empty list. Then we assign to the `fun` register the function that was produced by evaluating the function expression. If there are no argument expressions, we go directly to `apply_dispatch`. Otherwise we save `fun` on the stack and start the argument-evaluation loop:[25]

```
"ev_appl_did_function_expression",
  restore("unev"), // the argument expressions
  restore("env"),
  assign("argl", list(op("empty_arglist"))),
  assign("fun", reg("val")), // the function
  test(list(op("is_null"), reg("unev"))),
  branch(label("apply_dispatch")),
  save("fun"),
```

Each cycle of the argument-evaluation loop evaluates an argument expression from the list in `unev` and accumulates the result into `argl`. To evaluate an argument expression, we place it in the `comp` register and go to `eval_dispatch`, after setting `continue` so that execution will resume with the argument-accumulation phase. But first we save the arguments accumulated so far (held in `argl`), the environment (held in `env`), and the remaining argument expressions to be evaluated (held in `unev`). A special case is made for the evaluation of the last argument expression, which is handled at `ev_appl_last_arg`.

---

25. We add to the evaluator data-structure functions in section 4.1.3 the following two functions for manipulating argument lists:

```
function empty_arglist() { return null; }
function adjoin_arg(arg, arglist) {
    return append(arglist, list(arg));
}
```

We also make use of an additional syntax function to test for the last argument expression in an application:

```
function is_last_argument_expression(arg_expression) {
    return is_null(tail(arg_expression));
}
```

```
"ev_appl_argument_expression_loop",
  save("argl"),
  assign("comp", list(op("head"), reg("unev"))),
  test(list(op("is_last_argument_expression"), reg("unev"))),
  branch(label("ev_appl_last_arg")),
  save("env"),
  save("unev"),
  assign("continue", label("ev_appl_accumulate_arg")),
  go_to(label("eval_dispatch")),
```

When an argument expression has been evaluated, the value is accumulated into the list held in `argl`. The argument expression is then removed from the list of unevaluated argument expressions in `unev`, and the argument-evaluation loop continues.

```
"ev_appl_accumulate_arg",
  restore("unev"),
  restore("env"),
  restore("argl"),
  assign("argl", list(op("adjoin_arg"), reg("val"), reg("argl"))),
  assign("unev", list(op("tail"), reg("unev"))),
  go_to(label("ev_appl_argument_expression_loop")),
```

Evaluation of the last argument expression is handled differently, as is the last statement in a sequence. There is no need to save the environment or the list of unevaluated argument expressions before going to `eval_dispatch`, since they will not be required after the last argument expression is evaluated. Thus, we return from the evaluation to a special entry point `ev_appl_accum_last_arg`, which restores the argument list, accumulates the new argument, restores the saved function, and goes off to perform the application.[26]

```
"ev_appl_last_arg",
  assign("continue", label("ev_appl_accum_last_arg")),
  go_to(label("eval_dispatch")),
"ev_appl_accum_last_arg",
  restore("argl"),
  assign("argl", list(op("adjoin_arg"), reg("val"), reg("argl"))),
  restore("fun"),
  go_to(label("apply_dispatch")),
```

The details of the argument-evaluation loop determine the order in which the interpreter evaluates the argument expressions of a combination (e.g., left to right or right to left—see exercise 3.8). This order is not determined by the metacircular evaluator, which inherits its control structure from the underlying JavaScript in which it is

---

26. The optimization of treating the last argument expression specially is known as *evlis tail recursion* (see Wand 1980). We could be somewhat more efficient in the argument evaluation loop if we made evaluation of the first argument expression a special case too. This would permit us to postpone initializing `argl` until after evaluating the first argument expression, so as to avoid saving `argl` in this case. The compiler in section 5.5 performs this optimization. (Compare the `construct_arglist` function of section 5.5.3.)

implemented.[27] Because we use `head` in `ev_appl_argument_expression_loop` to extract successive argument expressions from `unev` and `tail` at `ev_appl_accumulate_arg` to extract the rest of the argument expressions, the explicit-control evaluator will evaluate the argument expressions of a combination in left-to-right order, as required by the ECMAScript specification.

Function Application

The entry point `apply_dispatch` corresponds to the `apply` function of the metacircular evaluator. By the time we get to `apply_dispatch`, the `fun` register contains the function to apply and `argl` contains the list of evaluated arguments to which it must be applied. The saved value of `continue` (originally passed to `eval_dispatch` and saved at `ev_application`), which tells where to return with the result of the function application, is on the stack. When the application is complete, the controller transfers to the entry point specified by the saved `continue`, with the result of the application in `val`. As with the metacircular `apply`, there are two cases to consider. Either the function to be applied is a primitive or it is a compound function.

```
"apply_dispatch",
  test(list(op("is_primitive_function"), reg("fun"))),
  branch(label("primitive_apply")),
  test(list(op("is_compound_function"), reg("fun"))),
  branch(label("compound_apply")),
  go_to(label("unknown_function_type")),
```

We assume that each primitive is implemented so as to obtain its arguments from `argl` and place its result in `val`. To specify how the machine handles primitives, we would have to provide a sequence of controller instructions to implement each primitive and arrange for `primitive_apply` to dispatch to the instructions for the primitive identified by the contents of `fun`. Since we are interested in the structure of the evaluation process rather than the details of the primitives, we will instead just use an `apply_primitive_function` operation that applies the function in fun to the arguments in `argl`. For the purpose of simulating the evaluator with the simulator of section 5.2 we use the function `apply_primitive_function`, which calls on the underlying JavaScript system to perform the application, just as we did for the metacircular evaluator in section 4.1.1. After computing the value of the primitive application, we restore `continue` and go to the designated entry point.

```
"primitive_apply",
  assign("val", list(op("apply_primitive_function"),
                     reg("fun"), reg("argl"))),
  restore("continue"),
  go_to(reg("continue")),
```

27. The order of argument-expression evaluation by the function `list_of_values` in the metacircular evaluator is determined by the order of evaluation of the arguments to `pair`, which is used to construct the argument list. The version of `list_of_values` in footnote 7 of section 4.1 calls `pair` directly; the version in the text uses `map`, which calls `pair`. (See exercise 4.1.)

The sequence of instructions labeled `compound_apply` specifies the application of compound functions. To apply a compound function, we proceed in a way similar to what we did in the metacircular evaluator. We construct a frame that binds the function's parameters to the arguments, use this frame to extend the environment carried by the function, and evaluate in this extended environment the body of the function.

At this point the compound function is in register `fun` and its arguments are in `argl`. We extract the function's parameters into `unev` and its environment into `env`. We then replace the environment in `env` with the environment constructed by extending it with bindings of the parameters to the given arguments. We then extract the body of the function into `comp`. The natural next step would be to restore the saved `continue` and proceed to `eval_dispatch` to evaluate the body and go to the restored continuation with the result in `val`, as is done for the last statement of a sequence. But there is a complication!

The complication has two aspects. One is that at any point in the evaluation of the body, a return statement may require the function to return the value of the return expression as the value of the body. But a return statement may be nested arbitrarily deeply in the body; so the stack at the moment the return statement is encountered is not necessarily the stack that is needed for a return from the function. One way to make it possible to adjust the stack for the return is to put a *marker* on the stack that can be found by the return code. This is implemented by the `push_marker_to_stack` instruction. The return code can then use the `revert_stack_to_marker` instruction to restore the stack to the place indicated by the marker before evaluating the return expression.[28]

The other aspect of the complication is that if the evaluation of the body terminates without executing a return statement, the value of the body must be `undefined`. To handle this, we set up the `continue` register to point to the entry point `return_undefined` before going off to `eval_dispatch` to evaluate the body. If a return statement is not encountered during evaluation of the body, evaluation of the body will continue at `return_undefined`.

```
"compound_apply",
  assign("unev", list(op("function_parameters"), reg("fun"))),
  assign("env", list(op("function_environment"), reg("fun"))),
  assign("env", list(op("extend_environment"),
                     reg("unev"), reg("argl"), reg("env"))),
  assign("comp", list(op("function_body"), reg("fun"))),
  push_marker_to_stack(),
  assign("continue", label("return_undefined")),
  go_to(label("eval_dispatch")),
```

The only places in the interpreter where the `env` register is assigned a new value are `compound_apply` and `ev_block` (section 5.4.3). Just as in the metacircular evaluator, the new environment for evaluation of a function body is constructed

28. The special instructions `push_marker_to_stack` and `revert_stack_to_marker` are not strictly necessary and could be implemented by explicitly pushing and popping a marker value onto and off the stack. Anything that could not be confused with a value in the program can be used as a marker. See exercise 5.23.

from the environment carried by the function, together with the argument list and the corresponding list of names to be bound.

When a return statement is evaluated at `ev_return`, we use the `revert_stack_to_marker` instruction to restore the stack to its state at the beginning of the function call by removing all values from the stack down to and including the marker. As a consequence, `restore("continue")` will restore the continuation of the function call, which was saved at `ev_application`. We then proceed to evaluate the return expression, whose result will be placed in `val` and thus be the value returned from the function when we continue after the evaluation of the return expression.

```
"ev_return",
  revert_stack_to_marker(),
  restore("continue"),
  assign("comp", list(op("return_expression"), reg("comp"))),
  go_to(label("eval_dispatch")),
```

If no return statement is encountered during evaluation of the function body, evaluation continues at `return_undefined`, the continuation that was set up at `compound_apply`. To return `undefined` from the function, we put `undefined` into `val` and go to the entry point that was put onto the stack at `ev_application`. Before we can restore that continuation from the stack, however, we must remove the marker that was saved at `compound_apply`.

```
"return_undefined",
  revert_stack_to_marker(),
  restore("continue"),
  assign("val", constant(undefined)),
  go_to(reg("continue")),
```

Return Statements and Tail Recursion

In chapter 1 we said that the process described by a function such as

```
function sqrt_iter(guess, x) {
    return is_good_enough(guess, x)
           ? guess
           : sqrt_iter(improve(guess, x), x);
}
```

is an iterative process. Even though the function is syntactically recursive (defined in terms of itself), it is not logically necessary for an evaluator to save information in passing from one call to `sqrt_iter` to the next.[29] An evaluator that can execute a function such as `sqrt_iter` without requiring increasing storage as the function continues to call itself is called a *tail-recursive* evaluator.

The metacircular implementation of the evaluator in chapter 4 isn't tail-recursive. It implements a return statement as a constructor of a return value object containing the value to be returned and inspects the result of a function call to see whether it is such an object. If the evaluation of a function body produces a return value object,

29. We saw in section 5.1 how to implement such a process with a register machine that had no stack; the state of the process was stored in a fixed set of registers.

the return value of the function is the contents of that object; otherwise, the return value is `undefined`. Both the construction of the return value object and the eventual inspection of the result of the function call are deferred operations, which lead to an accumulation of information on the stack.

Our explicit-control evaluator *is* tail-recursive, because it does not need to wrap up return values for inspection and thus avoids the buildup of stack from deferred operations. At `ev_return`, in order to evaluate the expression that computes the return value of a function, we transfer directly to `eval_dispatch` with nothing more on the stack than right before the function call. We accomplish this by undoing any saves to the stack by the function (which are useless because we are returning) using `revert_stack_to_marker`. Then, rather than arranging for `eval_dispatch` to come back here and *then* restoring `continue` from the stack and continuing at that entry point, we restore `continue` from the stack *before* going to `eval_dispatch` so that `eval_dispatch` will continue at that entry point after evaluating the expression. Finally, we transfer to `eval_dispatch` without saving any information on the stack. Thus, when we proceed to evaluate a return expression, the stack is the same as just before the call to the function whose return value we are about to compute. Hence, evaluating a return expression—even if it is a function call (as in `sqrt_iter`, where the conditional expression reduces to a call to `sqrt_iter`)—will not cause any information to accumulate on the stack.[30]

If we did not think to take advantage of the fact that it is unnecessary to hold on to the useless information on the stack while evaluating a return expression, we might have taken the straightforward approach of evaluating the return expression, coming back to restore the stack, and finally continuing at the entry point that is waiting for the result of the function call:

```
"ev_return",   // alternative implementation: not tail-recursive
  assign("comp", list(op("return_expression"), reg("comp"))),
  assign("continue", label("ev_restore_stack")),
  go_to(label("eval_dispatch")),
"ev_restore_stack",
  revert_stack_to_marker(),      // undo saves in current function
  restore("continue"),           // undo save at ev_application
  go_to(reg("continue")),
```

This may seem like a minor change to our previous code for evaluation of return statements: The only difference is that we delay undoing any register saves to the stack until after the evaluation of the return expression. The interpreter will still give the same value for any expression. But this change is fatal to the tail-recursive implementation, because we must now come back after evaluating the return expression in order to undo the (useless) register saves. These extra saves will accumulate during

30. This implementation of tail recursion is one variety of a well-known optimization technique used by many compilers. In compiling a function that ends with a function call, one can replace the call by a jump to the called function's entry point. Building this strategy into the interpreter, as we have done in this section, provides the optimization uniformly throughout the language.

a nest of function calls. Consequently, processes such as `sqrt_iter` will require space proportional to the number of iterations rather than requiring constant space. This difference can be significant. For example, with tail recursion, an infinite loop can be expressed using only the function-call and return mechanisms:

```
function count(n) {
    display(n);
    return count(n + 1);
}
```

Without tail recursion, such a function would eventually run out of stack space, and expressing a true iteration would require some control mechanism other than function call.

Note that our JavaScript implementation requires the use of **`return`** in order to be tail-recursive. Because the undoing of the register saves takes place at `ev_return`, removing **`return`** from the `count` function above will cause it to eventually run out of stack space. This explains the use of **`return`** in the infinite driver loops in chapter 4.

Exercise 5.22

Explain how the stack builds up if **`return`** is removed from `count`:

```
function count(n) {
    display(n);
    count(n + 1);
}
```

Exercise 5.23

Implement the equivalent of `push_marker_to_stack` by using `save` at `compound_apply` to store a special marker value on the stack. Implement the equivalent of `revert_stack_to_marker` at `ev_return` and `return_undefined` as a loop that repeatedly performs a `restore` until it hits the marker. Note that this will require restoring a value to a register other than the one it was saved from. (Although we are careful to avoid that in our evaluator, our stack implementation actually allows it. See exercise 5.10.) This is necessary because the only way to pop from the stack is by restoring to a register. Hint: You will need to create a unique constant to serve as the marker, for example with `const marker = list("marker")`. Because `list` creates a new pair, it cannot be `===` to anything else on the stack.

Exercise 5.24

Implement `push_marker_to_stack` and `revert_stack_to_marker` as register-machine instructions, following the implementation of `save` and `restore` in section 5.2.3. Add functions `push_marker` and `pop_marker` to access stacks, mirroring the implementation of `push` and `pop` in section 5.2.1. Note that you do not need to actually insert a marker into the stack. Instead, you can add a local state variable to the stack model to keep track of the position of the last `save` before each `push_marker_to_stack`. If you choose to put a marker on the stack, see the hint in exercise 5.23.

5.4.3 Blocks, Assignments, and Declarations

Blocks

The body of a block is evaluated with respect to the current environment extended by a frame that binds all local names to the value `"*unassigned*"`. We temporarily make use of the `val` register to hold the list of all variables declared in the block, which is obtained by `scan_out_declarations` from section 4.1.1. The functions `scan_out_declarations` and `list_of_unassigned` are assumed to be available as machine operations.[31]

```
"ev_block",
  assign("comp", list(op("block_body"), reg("comp"))),
  assign("val", list(op("scan_out_declarations"), reg("comp"))),

  save("comp"),     // so we can use it to temporarily hold *unassigned* values
  assign("comp", list(op("list_of_unassigned"), reg("val"))),
  assign("env", list(op("extend_environment"),
                     reg("val"), reg("comp"), reg("env"))),
  restore("comp"), // the block body
  go_to(label("eval_dispatch")),
```

Assignments and declarations

Assignments are handled by `ev_assignment`, reached from `eval_dispatch` with the assignment expression in `comp`. The code at `ev_assignment` first evaluates the value part of the expression and then installs the new value in the environment. The function `assign_symbol_value` is assumed to be available as a machine operation.

```
"ev_assignment",
  assign("unev", list(op("assignment_symbol"), reg("comp"))),
  save("unev"), // save variable for later
  assign("comp", list(op("assignment_value_expression"), reg("comp"))),
  save("env"),
  save("continue"),
  assign("continue", label("ev_assignment_install")),
  go_to(label("eval_dispatch")), // evaluate assignment value
"ev_assignment_install",
  restore("continue"),
  restore("env"),
  restore("unev"),
  perform(list(op("assign_symbol_value"),
               reg("unev"), reg("val"), reg("env"))),
  go_to(reg("continue")),
```

Declarations of variables and constants are handled in a similar way. Note that whereas the value of an assignment is the value that was assigned, the value of a declaration is `undefined`. This is handled by setting `val` to `undefined` before continuing. As in the metacircular evaluator, we transform a function declaration

31. Footnote 24 suggests that an actual implementation would perform syntax transformations before program execution. In the same vein, names declared in blocks should be scanned out in a preprocessing step rather than each time a block is evaluated.

into a constant declaration whose value expression is a lambda expression. This happens at `ev_function_declaration`, which makes the transformation in place in `comp` and falls through to `ev_declaration`.

```
"ev_function_declaration",
  assign("comp",
         list(op("function_decl_to_constant_decl"), reg("comp"))),
"ev_declaration",
  assign("unev", list(op("declaration_symbol"), reg("comp"))),
  save("unev"), // save declared name
  assign("comp",
         list(op("declaration_value_expression"), reg("comp"))),
  save("env"),
  save("continue"),
  assign("continue", label("ev_declaration_assign")),
  go_to(label("eval_dispatch")), // evaluate declaration value
"ev_declaration_assign",
  restore("continue"),
  restore("env"),
  restore("unev"),
  perform(list(op("assign_symbol_value"),
               reg("unev"), reg("val"), reg("env"))),
  assign("val", constant(undefined)),
  go_to(reg("continue")),
```

Exercise 5.25

Extend the evaluator to handle while loops, by translating them to applications of a function `while_loop`, as shown in exercise 4.7. You can paste the declaration of the function `while_loop` in front of user programs. You may "cheat" by assuming that the syntax transformer `while_to_application` is available as a machine operation. Refer to exercise 4.7 to discuss whether this approach works if return, break, and continue statements are allowed inside the while loop. If not, how can you modify the explicit-control evaluator to run programs with while loops that include these statements?

Exercise 5.26

Modify the evaluator so that it uses normal-order evaluation, based on the lazy evaluator of section 4.2.

5.4.4 Running the Evaluator

With the implementation of the explicit-control evaluator we come to the end of a development, begun in chapter 1, in which we have explored successively more precise models of the evaluation process. We started with the relatively informal substitution model, then extended this in chapter 3 to the environment model, which enabled us to deal with state and change. In the metacircular evaluator of chapter 4, we used JavaScript itself as a language for making more explicit the environment structure constructed during evaluation of an component. Now, with register machines, we have taken a close look at the evaluator's mechanisms for storage management, argument passing, and control. At each new level of description, we have had to raise issues and resolve ambiguities that were not apparent at the previous, less

precise treatment of evaluation. To understand the behavior of the explicit-control evaluator, we can simulate it and monitor its performance.

We will install a driver loop in our evaluator machine. This plays the role of the `driver_loop` function of section 4.1.4. The evaluator will repeatedly print a prompt, read a program, evaluate the program by going to `eval_dispatch`, and print the result. If nothing is entered at the prompt, we jump to the label `evaluator_done`, which is the last entry point in the controller. The following instructions form the beginning of the explicit-control evaluator's controller sequence:[32]

```
"read_evaluate_print_loop",
  perform(list(op("initialize_stack"))),
  assign("comp", list(op("user_read"),
                      constant("EC-evaluate input:"))),
  assign("comp", list(op("parse"), reg("comp"))),
  test(list(op("is_null"), reg("comp"))),
  branch(label("evaluator_done")),
  assign("env", list(op("get_current_environment"))),
  assign("val", list(op("scan_out_declarations"), reg("comp"))),
  save("comp"),    // so we can use it to temporarily hold *unassigned* values
  assign("comp", list(op("list_of_unassigned"), reg("val"))),
  assign("env", list(op("extend_environment"),
                     reg("val"), reg("comp"), reg("env"))),
  perform(list(op("set_current_environment"), reg("env"))),
  restore("comp"), // the program
  assign("continue", label("print_result")),
  go_to(label("eval_dispatch")),
"print_result",
  perform(list(op("user_print"),
               constant("EC-evaluate value:"), reg("val"))),
  go_to(label("read_evaluate_print_loop")),
```

We store the current environment, initially the global environment, in the variable `current_environment` and update it each time around the loop to remember past declarations. The operations `get_current_environment` and `set_current_environment` simply get and set this variable.

```
let current_environment = the_global_environment;
function get_current_environment() {
    return current_environment;
}
function set_current_environment(env) {
    current_environment = env;
}
```

32. We assume here that `user_read`, `parse`, and the various printing operations are available as primitive machine operations, which is useful for our simulation, but completely unrealistic in practice. These are actually extremely complex operations. In practice, reading and printing would be implemented using low-level input-output operations such as transferring single characters to and from a device.

When we encounter an error in a function (such as the "unknown function type" error indicated at `apply_dispatch`), we print an error message and return to the driver loop.[33]

```
"unknown_component_type",
  assign("val", constant("unknown syntax")),
  go_to(label("signal_error")),

"unknown_function_type",
  restore("continue"), // clean up stack (from apply_dispatch)
  assign("val", constant("unknown function type")),
  go_to(label("signal_error")),

"signal_error",
  perform(list(op("user_print"),
               constant("EC-evaluator error:"), reg("val"))),
  go_to(label("read_evaluate_print_loop")),
```

For the purposes of the simulation, we initialize the stack each time through the driver loop, since it might not be empty after an error (such as an undeclared name) interrupts an evaluation.[34]

If we combine all the code fragments presented in sections 5.4.1–5.4.4, we can create an evaluator machine model that we can run using the register-machine simulator of section 5.2.

```
const eceval = make_machine(list("comp", "env", "val", "fun",
                                 "argl", "continue", "unev"),
                            eceval_operations,
                            list("read_evaluate_print_loop",
                                 ⟨entire machine controller as given above⟩
                                 "evaluator_done"));
```

We must define JavaScript functions to simulate the operations used as primitives by the evaluator. These are the same functions we used for the metacircular evaluator in section 4.1, together with the few additional ones defined in footnotes throughout section 5.4.

```
const eceval_operations = list(list("is_literal", is_literal),
                               ⟨complete list of operations for eceval machine⟩);
```

Finally, we can initialize the global environment and run the evaluator:

```
const the_global_environment = setup_environment();
start(eceval);
```

---

33. There are other errors that we would like the interpreter to handle, but these are not so simple. See exercise 5.31.

34. We could perform the stack initialization only after errors, but doing it in the driver loop will be convenient for monitoring the evaluator's performance, as described below.

```
EC-evaluate input:
function append(x, y) {
    return is_null(x)
           ? y
           : pair(head(x), append(tail(x), y));
}

EC-evaluate value:
undefined

EC-evaluate input:
append(list("a", "b", "c"), list("d", "e", "f"));

EC-evaluate value:
["a", ["b", ["c", ["d", ["e", ["f", null]]]]]]
```

Of course, evaluating programs in this way will take much longer than if we had directly typed them into JavaScript, because of the multiple levels of simulation involved. Our programs are evaluated by the explicit-control-evaluator machine, which is being simulated by a JavaScript program, which is itself being evaluated by the JavaScript interpreter.

Monitoring the performance of the evaluator

Simulation can be a powerful tool to guide the implementation of evaluators. Simulations make it easy not only to explore variations of the register-machine design but also to monitor the performance of the simulated evaluator. For example, one important factor in performance is how efficiently the evaluator uses the stack. We can observe the number of stack operations required to evaluate various programs by defining the evaluator register machine with the version of the simulator that collects statistics on stack use (section 5.2.4), and adding an instruction at the evaluator's `print_result` entry point to print the statistics:

```
"print_result",
  perform(list(op("print_stack_statistics"))), // added instruction
  // rest is same as before
  perform(list(op("user_print"),
               constant("EC-evaluate value:"), reg("val"))),
  go_to(label("read_evaluate_print_loop")),
```

Interactions with the evaluator now look like this:

```
EC-evaluate input:
function factorial (n) {
    return n === 1
           ? 1
           : factorial(n - 1) * n;
}
```

```
total pushes = 4
maximum depth = 3
EC-evaluate value:
undefined

EC-evaluate input:
factorial(5);

total pushes = 151
maximum depth = 28
EC-evaluate value:
120
```

Note that the driver loop of the evaluator reinitializes the stack at the start of each interaction, so that the statistics printed will refer only to stack operations used to evaluate the previous program.

Exercise 5.27

Use the monitored stack to explore the tail-recursive property of the evaluator (section 5.4.2). Start the evaluator and define the iterative `factorial` function from section 1.2.1:

```
function factorial(n) {
    function iter(product, counter) {
        return counter > n
               ? product
               : iter(counter * product,
                      counter + 1);
    }
    return iter(1, 1);
}
```

Run the function with some small values of n. Record the maximum stack depth and the number of pushes required to compute $n!$ for each of these values.

a. You will find that the maximum depth required to evaluate $n!$ is independent of n. What is that depth?

b. Determine from your data a formula in terms of n for the total number of push operations used in evaluating $n!$ for any $n \geq 1$. Note that the number of operations used is a linear function of n and is thus determined by two constants.

Exercise 5.28

For comparison with exercise 5.27, explore the behavior of the following function for computing factorials recursively:

```
function factorial(n) {
    return n === 1
           ? 1
           : factorial(n - 1) * n;
}
```

By running this function with the monitored stack, determine, as a function of n, the maximum depth of the stack and the total number of pushes used in evaluating $n!$ for $n \geq 1$. (Again, these functions will be linear.) Summarize your experiments by filling in the following table with the appropriate expressions in terms of n:

	Maximum depth	Number of pushes
Recursive factorial		
Iterative factorial		

The maximum depth is a measure of the amount of space used by the evaluator in carrying out the computation, and the number of pushes correlates well with the time required.

Exercise 5.29

Modify the definition of the evaluator by changing `ev_return` as described in section 5.4.2 so that the evaluator is no longer tail-recursive. Rerun your experiments from exercises 5.27 and 5.28 to demonstrate that both versions of the `factorial` function now require space that grows linearly with their input.

Exercise 5.30

Monitor the stack operations in the tree-recursive Fibonacci computation:

```
function fib(n) {
    return n < 2 ? n : fib(n - 1) + fib(n - 2);
}
```

a. Give a formula in terms of n for the maximum depth of the stack required to compute Fib(n) for $n \geq 2$. Hint: In section 1.2.2 we argued that the space used by this process grows linearly with n.

b. Give a formula for the total number of pushes used to compute Fib(n) for $n \geq 2$. You should find that the number of pushes (which correlates well with the time used) grows exponentially with n. Hint: Let $S(n)$ be the number of pushes used in computing Fib(n). You should be able to argue that there is a formula that expresses $S(n)$ in terms of $S(n-1)$, $S(n-2)$, and some fixed "overhead" constant k that is independent of n. Give the formula, and say what k is. Then show that $S(n)$ can be expressed as $a\text{Fib}(n+1)+b$ and give the values of a and b.

Exercise 5.31

Our evaluator currently catches and signals only two kinds of errors—unknown component types and unknown function types. Other errors will take us out of the evaluator read-evaluate-print loop. When we run the evaluator using the register-machine simulator, these errors are caught by the underlying JavaScript system. This is analogous to the computer

crashing when a user program makes an error.[35] It is a large project to make a real error system work, but it is well worth the effort to understand what is involved here.

a. Errors that occur in the evaluation process, such as an attempt to access an unbound name, could be caught by changing the lookup operation to make it return a distinguished condition code, which cannot be a possible value of any user name. The evaluator can test for this condition code and then do what is necessary to go to `signal_error`. Find all of the places in the evaluator where such a change is necessary and fix them. This is lots of work.

b. Much worse is the problem of handling errors that are signaled by applying primitive functions such as an attempt to divide by zero or an attempt to extract the `head` of a string. In a professionally written high-quality system, each primitive application is checked for safety as part of the primitive. For example, every call to `head` could first check that the argument is a pair. If the argument is not a pair, the application would return a distinguished condition code to the evaluator, which would then report the failure. We could arrange for this in our register-machine simulator by making each primitive function check for applicability and returning an appropriate distinguished condition code on failure. Then the `primitive_apply` code in the evaluator can check for the condition code and go to `signal_error` if necessary. Build this structure and make it work. This is a major project.

5.5 Compilation

The explicit-control evaluator of section 5.4 is a register machine whose controller interprets JavaScript programs. In this section we will see how to run JavaScript programs on a register machine whose controller is not a JavaScript interpreter.

The explicit-control evaluator machine is universal—it can carry out any computational process that can be described in JavaScript. The evaluator's controller orchestrates the use of its data paths to perform the desired computation. Thus, the evaluator's data paths are universal: They are sufficient to perform any computation we desire, given an appropriate controller.[36]

---

35. This manifests itself as, for example, a "kernel panic" or a "blue screen of death" or even a reboot. Automatic rebooting is an approach typically used on phones and tablets. Most modern operating systems do a decent job of preventing user programs from causing an entire machine to crash.

36. This is a theoretical statement. We are not claiming that the evaluator's data paths are a particularly convenient or efficient set of data paths for a general-purpose computer. For example, they are not very good for implementing high-performance floating-point calculations or calculations that intensively manipulate bit vectors.

Commercial general-purpose computers are register machines organized around a collection of registers and operations that constitute an efficient and convenient universal set of data paths. The controller for a general-purpose machine is an interpreter for a register-machine language like the one we have been using. This language is called the *native language* of the machine, or simply *machine language*. Programs written in machine language are sequences of instructions that use the machine's data paths. For example, the explicit-control evaluator's instruction sequence can be thought of as a machine-language program for a general-purpose computer rather than as the controller for a specialized interpreter machine.

There are two common strategies for bridging the gap between higher-level languages and register-machine languages. The explicit-control evaluator illustrates the strategy of interpretation. An interpreter written in the native language of a machine configures the machine to execute programs written in a language (called the *source language*) that may differ from the native language of the machine performing the evaluation. The primitive functions of the source language are implemented as a library of subroutines written in the native language of the given machine. A program to be interpreted (called the *source program*) is represented as a data structure. The interpreter traverses this data structure, analyzing the source program. As it does so, it simulates the intended behavior of the source program by calling appropriate primitive subroutines from the library.

In this section, we explore the alternative strategy of *compilation*. A compiler for a given source language and machine translates a source program into an equivalent program (called the *object program*) written in the machine's native language. The compiler that we implement in this section translates programs written in JavaScript into sequences of instructions to be executed using the explicit-control evaluator machine's data paths.[37]

Compared with interpretation, compilation can provide a great increase in the efficiency of program execution, as we will explain below in the overview of the compiler. On the other hand, an interpreter provides a more powerful environment for interactive program development and debugging, because the source program being executed is available at run time to be examined and modified. In addition, because the entire library of primitives is present, new programs can be constructed and added to the system during debugging.

In view of the complementary advantages of compilation and interpretation, modern program-development environments pursue a mixed strategy. These systems are generally organized so that interpreted functions and compiled functions can call each other. This enables a programmer to compile those parts of a program that are assumed to be debugged, thus gaining the efficiency advantage of compilation, while

37. Actually, the machine that runs compiled code can be simpler than the interpreter machine, because we won't use the `comp` and `unev` registers. The interpreter used these to hold pieces of unevaluated components. With the compiler, however, these components get built into the compiled code that the register machine will run. For the same reason, we don't need the machine operations that deal with component syntax. But compiled code will use a few additional machine operations (to represent compiled function objects) that didn't appear in the explicit-control evaluator machine.

retaining the interpretive mode of execution for those parts of the program that are in the flux of interactive development and debugging.[38] In section 5.5.7, after we have implemented the compiler, we will show how to interface it with our interpreter to produce an integrated interpreter-compiler system.

An overview of the compiler

Our compiler is much like our interpreter, both in its structure and in the function it performs. Accordingly, the mechanisms used by the compiler for analyzing components will be similar to those used by the interpreter. Moreover, to make it easy to interface compiled and interpreted code, we will design the compiler to generate code that obeys the same conventions of register usage as the interpreter: The environment will be kept in the `env` register, argument lists will be accumulated in `argl`, a function to be applied will be in `fun`, functions will return their answers in `val`, and the location to which a function should return will be kept in `continue`. In general, the compiler translates a source program into an object program that performs essentially the same register operations as would the interpreter in evaluating the same source program.

This description suggests a strategy for implementing a rudimentary compiler: We traverse the component in the same way the interpreter does. When we encounter a register instruction that the interpreter would perform in evaluating the component, we do not execute the instruction but instead accumulate it into a sequence. The resulting sequence of instructions will be the object code. Observe the efficiency advantage of compilation over interpretation. Each time the interpreter evaluates a component—for example, `f(96, 22)`—it performs the work of classifying the component (discovering that this is a function application) and testing for the end of the list of argument expressions (discovering that there are two argument expressions). With a compiler, the component is analyzed only once, when the instruction sequence is generated at compile time. The object code produced by the compiler contains only the instructions that evaluate the function expression and the two argument expressions, assemble the argument list, and apply the function (in `fun`) to the arguments (in `argl`).

This is the same kind of optimization we implemented in the analyzing evaluator of section 4.1.7. But there are further opportunities to gain efficiency in compiled code. As the interpreter runs, it follows a process that must be applicable to any component in the language. In contrast, a given segment of compiled code is meant to execute some particular component. This can make a big difference, for example in the use of the stack to save registers. When the interpreter evaluates a component, it must be prepared for any contingency. Before evaluating a subcomponent, the interpreter saves all registers that will be needed later, because the subcomponent might require an arbitrary evaluation. A compiler, on the other hand, can exploit the

---

38. Language implementations often delay the compilation of program parts even when they are assumed to be debugged, until there is enough evidence that compiling them would lead to an overall efficiency advantage. The evidence is obtained at run time by monitoring the number of times the program parts are being interpreted. This technique is called *just-in-time compilation*.

structure of the particular component it is processing to generate code that avoids unnecessary stack operations.

As a case in point, consider the application `f(96, 22)`. Before the interpreter evaluates the function expression of the application, it prepares for this evaluation by saving the registers containing the argument expressions and the environment, whose values will be needed later. The interpreter then evaluates the function expression to obtain the result in `val`, restores the saved registers, and finally moves the result from `val` to `fun`. However, in the particular expression we are dealing with, the function expression is the name `f`, whose evaluation is accomplished by the machine operation `lookup_symbol_value`, which does not alter any registers. The compiler that we implement in this section will take advantage of this fact and generate code that evaluates the function expression using the instruction

```
assign("fun",
       list(op("lookup_symbol_value"), constant("f"), reg("env")))
```

where the argument to `lookup_symbol_value` is extracted at compile time from the parser's representation of `f(96, 22)`. This code not only avoids the unnecessary saves and restores but also assigns the value of the lookup directly to `fun`, whereas the interpreter would obtain the result in `val` and then move this to `fun`.

A compiler can also optimize access to the environment. Having analyzed the code, the compiler can know in which frame the value of a particular name will be located and access that frame directly, rather than performing the `lookup_symbol_value` search. We will discuss how to implement such lexical addressing in section 5.5.6. Until then, however, we will focus on the kind of register and stack optimizations described above. There are many other optimizations that can be performed by a compiler, such as coding primitive operations "in line" instead of using a general `apply` mechanism (see exercise 5.41); but we will not emphasize these here. Our main goal in this section is to illustrate the compilation process in a simplified (but still interesting) context.

5.5.1 Structure of the Compiler

In section 4.1.7 we modified our original metacircular interpreter to separate analysis from execution. We analyzed each component to produce an execution function that took an environment as argument and performed the required operations. In our compiler, we will do essentially the same analysis. Instead of producing execution functions, however, we will generate sequences of instructions to be run by our register machine.

The function `compile` is the top-level dispatch in the compiler. It corresponds to the `evaluate` function of section 4.1.1, the `analyze` function of section 4.1.7, and the `eval_dispatch` entry point of the explicit-control-evaluator in section 5.4.1. The compiler, like the interpreters, uses the component-syntax functions defined in section 4.1.2.[39] The function `compile` performs a case analysis on the syntactic type

39. Notice, however, that our compiler is a JavaScript program, and the syntax functions that it uses to manipulate expressions are the actual JavaScript functions used with the metacircular evaluator. For the explicit-control evaluator, in contrast, we assumed that equivalent syntax operations were available as operations for the register machine. (Of course, when we simulated

of the component to be compiled. For each type of component, it dispatches to a specialized *code generator*:

```
function compile(component, target, linkage) {
    return is_literal(component)
           ? compile_literal(component, target, linkage)
           : is_name(component)
           ? compile_name(component, target, linkage)
           : is_application(component)
           ? compile_application(component, target, linkage)
           : is_operator_combination(component)
           ? compile(operator_combination_to_application(component),
                     target, linkage)
           : is_conditional(component)
           ? compile_conditional(component, target, linkage)
           : is_lambda_expression(component)
           ? compile_lambda_expression(component, target, linkage)
           : is_sequence(component)
           ? compile_sequence(sequence_statements(component),
                              target, linkage)
           : is_block(component)
           ? compile_block(component, target, linkage)
           : is_return_statement(component)
           ? compile_return_statement(component, target, linkage)
           : is_function_declaration(component)
           ? compile(function_decl_to_constant_decl(component),
                     target, linkage)
           : is_declaration(component)
           ? compile_declaration(component, target, linkage)
           : is_assignment(component)
           ? compile_assignment(component, target, linkage)
           : error(component, "unknown component type -- compile");
}
```

Targets and linkages

The function `compile` and the code generators that it calls take two arguments in addition to the component to compile. There is a *target*, which specifies the register in which the compiled code is to return the value of the component. There is also a *linkage descriptor*, which describes how the code resulting from the compilation of the component should proceed when it has finished its execution. The linkage descriptor can require the code to do one of the following three things:

- proceed to the next instruction in sequence (this is specified by the linkage descriptor `"next"`),
- jump to the current value of the `continue` register as part of returning from a function call (this is specified by the linkage descriptor `"return"`), or
- jump to a named entry point (this is specified by using the designated label as the linkage descriptor).

---

the register machine in JavaScript, we used the actual JavaScript functions in our register machine simulation.)

For example, compiling the literal `5` with a target of the `val` register and a linkage of `"next"` should produce the instruction

```
assign("val", constant(5))
```

Compiling the same expression with a linkage of `"return"` should produce the instructions

```
assign("val", constant(5)),
go_to(reg("continue"))
```

In the first case, execution will continue with the next instruction in the sequence. In the second case, we will jump to whatever entry point is stored in the `continue` register. In both cases, the value of the expression will be placed into the target `val` register. Our compiler uses the `"return"` linkage when compiling the return expression of a return statement. Just as in the explicit-control evaluator, returning from a function call happens in three steps:

1. reverting the stack to the marker and restoring `continue` (which holds a continuation set up at the beginning of the function call)
2. computing the return value and placing it in `val`
3. jumping to the entry point in `continue`

Compilation of a return statement explicitly generates code for reverting the stack and restoring `continue`. The return expression is compiled with target `val` and linkage `"return"` so that the generated code for computing the return value places the return value in `val` and ends by jumping to `continue`.

Instruction sequences and stack usage

Each code generator returns an *instruction sequence* containing the object code it has generated for the component. Code generation for a compound component is accomplished by combining the output from simpler code generators for subcomponents, just as evaluation of a compound component is accomplished by evaluating the subcomponents.

The simplest method for combining instruction sequences is a function called `append_instruction_sequences`, which takes as arguments two instruction sequences that are to be executed sequentially. It appends them and returns the combined sequence. That is, if seq_1 and seq_2 are sequences of instructions, then evaluating

`append_instruction_sequences(`seq_1`,` seq_2`)`

produces the sequence

seq_1
seq_2

Whenever registers might need to be saved, the compiler's code generators use `preserving`, which is a more subtle method for combining instruction sequences. The function `preserving` takes three arguments: a set of registers and two instruction sequences that are to be executed sequentially. It appends the sequences in such a way that the contents of each register in the set is preserved over the execution of

the first sequence, if this is needed for the execution of the second sequence. That is, if the first sequence modifies the register and the second sequence actually needs the register's original contents, then `preserving` wraps a `save` and a `restore` of the register around the first sequence before appending the sequences. Otherwise, `preserving` simply returns the appended instruction sequences. Thus, for example,

`preserving(list(`reg_1`,` reg_2`),` seq_1`,` seq_2`)`

produces one of the following four sequences of instructions, depending on how seq_1 and seq_2 use reg_1 and reg_2:

seq_1	`save(`reg_1`),`	`save(`reg_2`),`	`save(`reg_2`),`
seq_2	seq_1	seq_1	`save(`reg_1`),`
	`restore(`reg_1`),`	`restore(`reg_2`),`	seq_1
	seq_2	seq_2	`restore(`reg_1`),`
			`restore(`reg_2`),`
			seq_2

By using `preserving` to combine instruction sequences the compiler avoids unnecessary stack operations. This also isolates the details of whether or not to generate `save` and `restore` instructions within the `preserving` function, separating them from the concerns that arise in writing each of the individual code generators. In fact no `save` or `restore` instructions are explicitly produced by the code generators, except that the code for calling a function saves `continue` and the code for returning from a function restores it: These corresponding `save` and `restore` instructions are explicitly generated by different calls to `compile`, not as a matched pair by `preserving` (as we will see in section 5.5.3).

In principle, we could represent an instruction sequence simply as a list of instructions. The function `append_instruction_sequences` could then combine instruction sequences by performing an ordinary list `append`. However, `preserving` would then be a complex operation, because it would have to analyze each instruction sequence to determine how the sequence uses its registers. The function `preserving` would be inefficient as well as complex, because it would have to analyze each of its instruction sequence arguments, even though these sequences might themselves have been constructed by calls to `preserving`, in which case their parts would have already been analyzed. To avoid such repetitious analysis we will associate with each instruction sequence some information about its register use. When we construct a basic instruction sequence we will provide this information explicitly, and the functions that combine instruction sequences will derive register-use information for the combined sequence from the information associated with the sequences being combined.

An instruction sequence will contain three pieces of information:

- the set of registers that must be initialized before the instructions in the sequence are executed (these registers are said to be *needed* by the sequence),

- the set of registers whose values are modified by the instructions in the sequence, and
- the actual instructions in the sequence.

We will represent an instruction sequence as a list of its three parts. The constructor for instruction sequences is thus

```
function make_instruction_sequence(needs, modifies, instructions) {
    return list(needs, modifies, instructions);
}
```

For example, the two-instruction sequence that looks up the value of the symbol `"x"` in the current environment, assigns the result to `val`, and then proceeds to the continuation, requires registers `env` and `continue` to have been initialized, and modifies register `val`. This sequence would therefore be constructed as

```
make_instruction_sequence(list("env", "continue"), list("val"),
    list(assign("val",
                list(op("lookup_symbol_value"), constant("x"),
                     reg("env"))),
         go_to(reg("continue"))));
```

The functions for combining instruction sequences are shown in section 5.5.4.

Exercise 5.32

In evaluating a function application, the explicit-control evaluator always saves and restores the `env` register around the evaluation of the function expression, saves and restores `env` around the evaluation of each argument expression (except the final one), saves and restores `argl` around the evaluation of each argument expression, and saves and restores `fun` around the evaluation of the argument-expression sequence. For each of the following applications, say which of these `save` and `restore` operations are superfluous and thus could be eliminated by the compiler's `preserving` mechanism:

```
f("x", "y")

f()("x", "y")

f(g("x"), y)

f(g("x"), "y")
```

Exercise 5.33

Using the `preserving` mechanism, the compiler will avoid saving and restoring `env` around the evaluation of the function expression of an application in the case where the function expression is a name. We could also build such optimizations into the evaluator. Indeed, the explicit-control evaluator of section 5.4 already performs a similar optimization, by treating applications with no arguments as a special case.

a. Extend the explicit-control evaluator to recognize as a separate class of components applications whose function expression is a name, and to take advantage of this fact in evaluating such components.

b. Alyssa P. Hacker suggests that by extending the evaluator to recognize more and more special cases we could incorporate all the compiler's optimizations, and that this would eliminate the advantage of compilation altogether. What do you think of this idea?

5.5.2 Compiling Components

In this section and the next we implement the code generators to which the `compile` function dispatches.

Compiling linkage code

In general, the output of each code generator will end with instructions—generated by the function `compile_linkage`—that implement the required linkage. If the linkage is `"return"` then we must generate the instruction `go_to(reg("continue"))`. This needs the `continue` register and does not modify any registers. If the linkage is `"next"`, then we needn't include any additional instructions. Otherwise, the linkage is a label, and we generate a `go_to` to that label, an instruction that does not need or modify any registers.

```
function compile_linkage(linkage) {
    return linkage === "return"
           ? make_instruction_sequence(list("continue"), null,
                                       list(go_to(reg("continue"))))
           : linkage === "next"
           ? make_instruction_sequence(null, null, null)
           : make_instruction_sequence(null, null,
                                       list(go_to(label(linkage))));
}
```

The linkage code is appended to an instruction sequence by `preserving` the `continue` register, since a `"return"` linkage will require the `continue` register: If the given instruction sequence modifies `continue` and the linkage code needs it, `continue` will be saved and restored.

```
function end_with_linkage(linkage, instruction_sequence) {
    return preserving(list("continue"),
                      instruction_sequence,
                      compile_linkage(linkage));
}
```

Compiling simple components

The code generators for literal expressions and names construct instruction sequences that assign the required value to the target register and then proceed as specified by the linkage descriptor.

The literal value is extracted at compile time from the component being compiled and put into the constant part of the `assign` instruction. For a name, an instruction is generated to use the `lookup_symbol_value` operation when the compiled program is run, to look up the value associated with a symbol in the current environment. Like a literal value, the symbol is extracted at compile time from the component being compiled. Thus `symbol_of_name(component)` is executed only once, when the program is being compiled, and the symbol appears as a constant in the `assign` instruction.

```
function compile_literal(component, target, linkage) {
    const literal = literal_value(component);
    return end_with_linkage(linkage,
               make_instruction_sequence(null, list(target),
                   list(assign(target, constant(literal)))));
}
function compile_name(component, target, linkage) {
    const symbol = symbol_of_name(component);
    return end_with_linkage(linkage,
               make_instruction_sequence(list("env"), list(target),
                   list(assign(target,
                               list(op("lookup_symbol_value"),
                                    constant(symbol),
                                    reg("env"))))));
}
```

These assignment instructions modify the target register, and the one that looks up a symbol needs the `env` register.

Assignments and declarations are handled much as they are in the interpreter. The function `compile_assignment_declaration` recursively generates code that computes the value to be associated with the symbol and appends to it a two-instruction sequence that updates the value associated with the symbol in the environment and assigns the value of the whole component (the assigned value for an assignment or `undefined` for a declaration) to the target register. The recursive compilation has target `val` and linkage `"next"` so that the code will put its result into `val` and continue with the code that is appended after it. The appending is done preserving `env`, since the environment is needed for updating the symbol–value association and the code for computing the value could be the compilation of a complex expression that might modify the registers in arbitrary ways.

```
function compile_assignment(component, target, linkage) {
    return compile_assignment_declaration(
               assignment_symbol(component),
               assignment_value_expression(component),
               reg("val"),
               target, linkage);
}
function compile_declaration(component, target, linkage) {
    return compile_assignment_declaration(
               declaration_symbol(component),
               declaration_value_expression(component),
               constant(undefined),
               target, linkage);
}
function compile_assignment_declaration(
             symbol, value_expression, final_value,
             target, linkage) {
    const get_value_code = compile(value_expression, "val", "next");
    return end_with_linkage(linkage,
               preserving(list("env"),
                   get_value_code,
                   make_instruction_sequence(list("env", "val"),
                                             list(target),
                       list(perform(list(op("assign_symbol_value"),
                                         constant(symbol),
                                         reg("val"),
                                         reg("env"))),
                            assign(target, final_value)))));
}
```

The appended two-instruction sequence requires `env` and `val` and modifies the target. Note that although we preserve `env` for this sequence, we do not preserve `val`, because the `get_value_code` is designed to explicitly place its result in `val` for use by this sequence. (In fact, if we did preserve `val`, we would have a bug, because this would cause the previous contents of `val` to be restored right after the `get_value_code` is run.)

Compiling conditionals

The code for a conditional compiled with a given target and linkage has the form

```
⟨compilation of predicate, target val, linkage "next"⟩
  test(list(op("is_falsy"), reg("val"))),
  branch(label("false_branch")),
"true_branch",
  ⟨compilation of consequent with given target and given linkage or after_cond⟩
"false_branch",
  ⟨compilation of alternative with given target and linkage⟩
"after_cond"
```

To generate this code, we compile the predicate, consequent, and alternative, and combine the resulting code with instructions to test the predicate result and with newly generated labels to mark the true and false branches and the end of the conditional.[40] In this arrangement of code, we must branch around the true branch if the test is false. The only slight complication is in how the linkage for the true branch should be handled. If the linkage for the conditional is `"return"` or a label, then the true and false branches will both use this same linkage. If the linkage is `"next"`, the true branch ends with a jump around the code for the false branch to the label at the end of the conditional.

```
function compile_conditional(component, target, linkage) {
    const t_branch = make_label("true_branch");
    const f_branch = make_label("false_branch");
    const after_cond = make_label("after_cond");
    const consequent_linkage =
            linkage === "next" ? after_cond : linkage;
    const p_code = compile(conditional_predicate(component),
                           "val", "next");
    const c_code = compile(conditional_consequent(component),
                           target, consequent_linkage);
    const a_code = compile(conditional_alternative(component),
                           target, linkage);
    return preserving(list("env", "continue"),
             p_code,
             append_instruction_sequences(
               make_instruction_sequence(list("val"), null,
                 list(test(list(op("is_falsy"), reg("val"))),
                      branch(label(f_branch)))),
               append_instruction_sequences(
                 parallel_instruction_sequences(
                   append_instruction_sequences(t_branch, c_code),
                   append_instruction_sequences(f_branch, a_code)),
                 after_cond)));
}
```

40. We can't just use the labels `true_branch`, `false_branch`, and `after_cond` as shown above, because there might be more than one conditional in the program. The compiler uses the function `make_label` to generate labels. The function `make_label` takes a string as argument and returns a new string that begins with the given string. For example, successive calls to `make_label("a")` would return `"a1"`, `"a2"`, and so on. The function `make_label` can be implemented similarly to the generation of unique variable names in the query language, as follows:

```
let label_counter = 0;
function new_label_number() {
    label_counter = label_counter + 1;
    return label_counter;
}
function make_label(string) {
    return string + stringify(new_label_number());
}
```

The env register is preserved around the predicate code because it could be needed by the true and false branches, and continue is preserved because it could be needed by the linkage code in those branches. The code for the true and false branches (which are not executed sequentially) is appended using a special combiner parallel_instruction_sequences described in section 5.5.4.

Compiling sequences

The compilation of statement sequences parallels their evaluation in the explicit-control evaluator with one exception: If a return statement appears anywhere in a sequence, we treat it as if it were the last statement. Each statement of the sequence is compiled—the last statement (or a return statement) with the linkage specified for the sequence, and the other statements with linkage "next" (to execute the rest of the sequence). The instruction sequences for the individual statements are appended to form a single instruction sequence, such that env (needed for the rest of the sequence) and continue (possibly needed for the linkage at the end of the sequence) are preserved.[41]

```
function compile_sequence(seq, target, linkage) {
    return is_empty_sequence(seq)
           ? compile_literal(make_literal(undefined), target, linkage)
           : is_last_statement(seq) ||
                 is_return_statement(first_statement(seq))
           ? compile(first_statement(seq), target, linkage)
           : preserving(list("env", "continue"),
                 compile(first_statement(seq), target, "next"),
                 compile_sequence(rest_statements(seq),
                                  target, linkage));
}
```

Treating a return statement as if it were the last statement in a sequence avoids compiling any "dead code" after the return statement that can never be executed. Removing the is_return_statement check does not change the behavior of the object program; however, there are many reasons not to compile dead code, which are beyond the scope of this book (security, compilation time, size of the object code, etc.), and many compilers give warnings for dead code.[42]

Compiling blocks

A block is compiled by prepending an assign instruction to the compiled body of the block. The assignment extends the current environment by a frame that binds the names declared in the block to the value "*unassigned*". This operation both needs and modifies the env register.

---

41. The continue register would be needed for a "return" linkage, which can result from a compilation by compile_and_go (section 5.5.7).

42. Our compiler does not detect all dead code. For example, a conditional statement whose consequent and alternative branches both end in a return statement will not stop subsequent statements from being compiled. See exercises 5.34 and 5.35.

```
function compile_block(stmt, target, linkage) {
    const body = block_body(stmt);
    const locals = scan_out_declarations(body);
    const unassigneds = list_of_unassigned(locals);
    return append_instruction_sequences(
               make_instruction_sequence(list("env"), list("env"),
                   list(assign("env", list(op("extend_environment"),
                                           constant(locals),
                                           constant(unassigneds),
                                           reg("env"))))),
               compile(body, target, linkage));
}
```

Compiling lambda expressions

Lambda expressions construct functions. The object code for a lambda expression must have the form

⟨*construct function object and assign it to target register*⟩
⟨*linkage*⟩

When we compile the lambda expression, we also generate the code for the function body. Although the body won't be executed at the time of function construction, it is convenient to insert it into the object code right after the code for the lambda expression. If the linkage for the lambda expression is a label or `"return"`, this is fine. But if the linkage is `"next"`, we will need to skip around the code for the function body by using a linkage that jumps to a label that is inserted after the body. The object code thus has the form

⟨*construct function object and assign it to target register*⟩
⟨*code for given linkage*⟩ *or* `go_to(label("after_lambda"))`
⟨*compilation of function body*⟩
`"after_lambda"`

The function `compile_lambda_expression` generates the code for constructing the function object followed by the code for the function body. The function object will be constructed at run time by combining the current environment (the environment at the point of declaration) with the entry point to the compiled function body (a newly generated label).[43]

---

43. We need machine operations to implement a data structure for representing compiled functions, analogous to the structure for compound functions described in section 4.1.3:

```
function make_compiled_function(entry, env) {
    return list("compiled_function", entry, env);
}
function is_compiled_function(fun) {
    return is_tagged_list(fun, "compiled_function");
}
function compiled_function_entry(c_fun) {
    return head(tail(c_fun));
}
function compiled_function_env(c_fun) {
    return head(tail(tail(c_fun)));
}
```

```
function compile_lambda_expression(exp, target, linkage) {
    const fun_entry = make_label("entry");
    const after_lambda = make_label("after_lambda");
    const lambda_linkage =
            linkage === "next" ? after_lambda : linkage;
    return append_instruction_sequences(
               tack_on_instruction_sequence(
                   end_with_linkage(lambda_linkage,
                       make_instruction_sequence(list("env"),
                                                 list(target),
                           list(assign(target,
                                       list(op("make_compiled_function"),
                                            label(fun_entry),
                                            reg("env"))))))),
                   compile_lambda_body(exp, fun_entry)),
               after_lambda);
}
```

The function `compile_lambda_expression` uses the special combiner `tack_on_instruction_sequence` (from section 5.5.4) rather than `append_instruction_sequences` to append the function body to the lambda expression code, because the body is not part of the sequence of instructions that will be executed when the combined sequence is entered; rather, it is in the sequence only because that was a convenient place to put it.

The function `compile_lambda_body` constructs the code for the body of the function. This code begins with a label for the entry point. Next come instructions that will cause the runtime evaluation environment to switch to the correct environment for evaluating the function body—namely, the environment of the function, extended to include the bindings of the parameters to the arguments with which the function is called. After this comes the code for the function body, augmented to ensure that it ends with a return statement. The augmented body is compiled with target `val` so that its return value will be placed in `val`. The linkage descriptor passed to this compilation is irrelevant, as it will be ignored.[44] Since a linkage argument is required, we arbitrarily pick `"next"`.

---

44. The augmented function body is a sequence ending with a return statement. Compilation of a sequence of statements uses the linkage `"next"` for all its component statements except the last, for which it uses the given linkage. In this case, the last statement is a return statement, and as we will see in section 5.5.3, a return statement always uses the `"return"` linkage descriptor for its return expression. Thus all function bodies will end with a `"return"` linkage, not the `"next"` we pass as the linkage argument to `compile` in `compile_lambda_body`.

```
function compile_lambda_body(exp, fun_entry) {
    const params  = lambda_parameter_symbols(exp);
    return append_instruction_sequences(
        make_instruction_sequence(list("env", "fun", "argl"),
                                  list("env"),
            list(fun_entry,
                 assign("env",
                        list(op("compiled_function_env"),
                             reg("fun"))),
                 assign("env",
                        list(op("extend_environment"),
                             constant(params),
                             reg("argl"),
                             reg("env")))),
        compile(append_return_undefined(lambda_body(exp)),
                "val", "next"));
}
```

To ensure that all functions end by executing a return statement, `compile_lambda_body` appends to the lambda body a return statement whose return expression is the literal `undefined`. To do so, it uses the function `append_return_undefined`, which constructs the parser's tagged-list representation (from section 4.1.2) of a sequence consisting of the body and a `return undefined;` statement.

```
function append_return_undefined(body) {
    return list("sequence", list(body,
                                 list("return_statement",
                                      list("literal", undefined))));
}
```

This simple transformation of lambda bodies is a third way to ensure that a function that does not return explicitly has the return value `undefined`. In the metacircular evaluator, we used a return-value object, which also played a role in stopping a sequence evaluation. In the explicit-control evaluator, functions that did not return explicitly continued to an entry point that stored `undefined` in `val`. See exercise 5.35 for a more elegant way to handle insertion of return statements.

Exercise 5.34

Footnote 42 pointed out that the compiler does not identify all instances of dead code. What would be required of a compiler to detect all instances of dead code?

Hint: The answer depends on how we define dead code. One possible (and useful) definition is "code following a return statement in a sequence"—but what about code in the consequent branch of `if (false) ...` or code following a call to `run_forever()` in exercise 4.15?

Exercise 5.35

The current design of `append_return_undefined` is a bit crude: It always appends a `return undefined;` to a lambda body, even if there is already a return statement in every execution path of the body. Rewrite `append_return_undefined` so that it inserts `return undefined;` at the end of only those paths that do not contain a return statement. Test your solution on the functions below, substituting any expressions for e_1 and e_2 and any (non-return) statements for s_1 and s_2. In `t`, a return statement should be added either at both `(*)`'s or just at `(**)`. In `w` and `h`, a return statement should be added at one of the `(*)`'s. In `m`, no return statement should be added.

```
function t(b) {
   if (b) {
      s1
      (*)
   } else {
      s2
      (*)
   }
   (**)
}
```

```
function w(b) {
   if (b) {
      return e1;
   } else {
      s1
      (*)
   }
   (*)
}
```

```
function m(b) {
   if (b) {
      return e1;
   } else {
      return e2;
   }
}
```

```
function h(b1, b2) {
   if (b1) {
      return e1;
   } else {
      if (b2) {
         s1
         (*)
      } else {
         return e2;
      }
      (*)
   }
   (*)
}
```

5.5.3 Compiling Applications and Return Statements

The essence of the compilation process is the compilation of function applications. The code for an application compiled with a given target and linkage has the form

⟨*compilation of function expression*, *target* `fun`, *linkage* `"next"`⟩
⟨*evaluate argument expressions and construct argument list in* `argl`⟩
⟨*compilation of function call with given target and linkage*⟩

The registers `env`, `fun`, and `argl` may have to be saved and restored during evaluation of the function and argument expressions. Note that this is the only place in the compiler where a target other than `val` is specified.

The required code is generated by `compile_application`. This recursively compiles the function expression, to produce code that puts the function to be applied into `fun`, and compiles the argument expressions, to produce code that evaluates the individual argument expressions of the application. The instruction sequences for the argument expressions are combined (by `construct_arglist`) with code that constructs the list of arguments in `argl`, and the resulting argument-list code is combined with the function code and the code that performs the function call (produced by `compile_function_call`). In appending the code sequences, the

env register must be preserved around the evaluation of the function expression (since evaluating the function expression might modify env, which will be needed to evaluate the argument expressions), and the fun register must be preserved around the construction of the argument list (since evaluating the argument expressions might modify fun, which will be needed for the actual function application). The continue register must also be preserved throughout, since it is needed for the linkage in the function call.

```
function compile_application(exp, target, linkage) {
    const fun_code = compile(function_expression(exp), "fun", "next");
    const argument_codes = map(arg => compile(arg, "val", "next"),
                               arg_expressions(exp));
    return preserving(list("env", "continue"),
                      fun_code,
                      preserving(list("fun", "continue"),
                          construct_arglist(argument_codes),
                          compile_function_call(target, linkage)));
}
```

The code to construct the argument list will evaluate each argument expression into val and then combine that value with the argument list being accumulated in argl using pair. Since we adjoin the arguments to the front of argl in sequence, we must start with the last argument and end with the first, so that the arguments will appear in order from first to last in the resulting list. Rather than waste an instruction by initializing argl to the empty list to set up for this sequence of evaluations, we make the first code sequence construct the initial argl. The general form of the argument-list construction is thus as follows:

⟨*compilation of last argument*, *targeted to* val⟩
assign("argl", list(op("list"), reg("val"))),
⟨*compilation of next argument*, *targeted to* val⟩
assign("argl", list(op("pair"), reg("val"), reg("argl"))),
...
⟨*compilation of first argument*, *targeted to* val⟩
assign("argl", list(op("pair"), reg("val"), reg("argl"))),

The argl register must be preserved around each argument evaluation except the first (so that arguments accumulated so far won't be lost), and env must be preserved around each argument evaluation except the last (for use by subsequent argument evaluations).

Compiling this argument code is a bit tricky, because of the special treatment of the first argument expression to be evaluated and the need to preserve argl and env in different places. The construct_arglist function takes as arguments the code that evaluates the individual argument expressions. If there are no argument expressions at all, it simply emits the instruction

```
assign(argl, constant(null))
```

Otherwise, `construct_arglist` creates code that initializes `argl` with the last argument, and appends code that evaluates the rest of the arguments and adjoins them to `argl` in succession. In order to process the arguments from last to first, we must reverse the list of argument code sequences from the order supplied by `compile_application`.

```
function construct_arglist(arg_codes) {
    if (is_null(arg_codes)) {
        return make_instruction_sequence(null, list("argl"),
                   list(assign("argl", constant(null))));
    } else {
        const rev_arg_codes = reverse(arg_codes);
        const code_to_get_last_arg =
            append_instruction_sequences(
                head(rev_arg_codes),
                make_instruction_sequence(list("val"), list("argl"),
                    list(assign("argl",
                                list(op("list"), reg("val"))))));
        return is_null(tail(rev_arg_codes))
               ? code_to_get_last_arg
               : preserving(list("env"),
                     code_to_get_last_arg,
                     code_to_get_rest_args(tail(rev_arg_codes)));
    }
}
function code_to_get_rest_args(arg_codes) {
    const code_for_next_arg =
        preserving(list("argl"),
            head(arg_codes),
            make_instruction_sequence(list("val", "argl"), list("argl"),
                list(assign("argl", list(op("pair"),
                                         reg("val"), reg("argl"))))));
    return is_null(tail(arg_codes))
           ? code_for_next_arg
           : preserving(list("env"),
                        code_for_next_arg,
                        code_to_get_rest_args(tail(arg_codes)));
}
```

Applying functions

After evaluating the elements of a function application, the compiled code must apply the function in `fun` to the arguments in `argl`. The code performs essentially the same dispatch as the `apply` function in the metacircular evaluator of section 4.1.1 or the `apply_dispatch` entry point in the explicit-control evaluator of section 5.4.2. It checks whether the function to be applied is a primitive function or a compiled function. For a primitive function, it uses `apply_primitive_function`; we will

see shortly how it handles compiled functions. The function-application code has the following form:

```
  test(list(op("primitive_function"), reg("fun"))),
  branch(label("primitive_branch")),
"compiled_branch",
  ⟨code to apply compiled function with given target and appropriate linkage⟩
"primitive_branch",
  assign(target,
         list(op("apply_primitive_function"), reg("fun"), reg("argl"))),
  ⟨linkage⟩
"after_call"
```

Observe that the compiled branch must skip around the primitive branch. Therefore, if the linkage for the original function call was `"next"`, the compound branch must use a linkage that jumps to a label that is inserted after the primitive branch. (This is similar to the linkage used for the true branch in `compile_conditional`.)

```
function compile_function_call(target, linkage) {
    const primitive_branch = make_label("primitive_branch");
    const compiled_branch = make_label("compiled_branch");
    const after_call = make_label("after_call");
    const compiled_linkage = linkage === "next" ? after_call : linkage;
    return append_instruction_sequences(
        make_instruction_sequence(list("fun"), null,
            list(test(list(op("is_primitive_function"), reg("fun"))),
                 branch(label(primitive_branch)))),
        append_instruction_sequences(
            parallel_instruction_sequences(
                append_instruction_sequences(
                    compiled_branch,
                    compile_fun_appl(target, compiled_linkage)),
                append_instruction_sequences(
                    primitive_branch,
                    end_with_linkage(linkage,
                        make_instruction_sequence(list("fun", "argl"),
                                                  list(target),
                            list(assign(
                                   target,
                                   list(op("apply_primitive_function"),
                                        reg("fun"), reg("argl")))))))),
            after_call));
}
```

The primitive and compound branches, like the true and false branches in `compile_conditional`, are appended using `parallel_instruction_sequences` rather than the ordinary `append_instruction_sequences`, because they will not be executed sequentially.

Applying compiled functions

The handling of function application and return is the most subtle part of the compiler. A compiled function (as constructed by `compile_lambda_expression`) has an entry point, which is a label that designates where the code for the function starts. The code at this entry point computes a result in `val` and ends by executing the instructions from a compiled return statement.

The code for a compiled-function application uses the stack in the same way as the explicit-control evaluator (section 5.4.2): before jumping to the compiled function's entry point, it saves the continuation of the function call to the stack, followed by a mark that allows reverting the stack to the state right before the call with the continuation on top.

```
// set up for return from function
save("continue"),
push_marker_to_stack(),
// jump to the function's entry point
assign("val", list(op("compiled_function_entry"), reg("fun"))),
go_to(reg("val")),
```

Compiling a return statement (with `compile_return_statement`) generates corresponding code for reverting the stack and restoring and jumping to `continue`.

```
revert_stack_to_marker(),
restore("continue"),
⟨evaluate the return expression and store the result in val⟩
go_to(reg("continue")), // "return"-linkage code
```

Unless a function enters an infinite loop, it will end by executing the above return code, resulting from either a return statement in the program or one inserted by `compile_lambda_body` to return `undefined`.[45]

Straightforward code for a compiled-function application with a given target and linkage would set up `continue` to make the function return to a local label instead of to the final linkage, to copy the function value from `val` to the target register if necessary. It would look like this if the linkage is a label:

```
assign("continue", label("fun_return")), // where function should return to
save("continue"),          // will be restored by the function
push_marker_to_stack(), // allows the function to revert stack to find fun_return
assign("val", list(op("compiled_function_entry"), reg("fun"))),
go_to(reg("val")),      // eventually reverts stack, restores and jumps to continue
"fun_return",              // the function returns to here
assign(target, reg("val")), // included if target is not val
go_to(label(linkage)),     // linkage code
```

45. Because the execution of a function body always ends with a return, there is no need here for a mechanism like the `return_undefined` entry point from section 5.4.2.

or like this—saving the caller's continuation at the start in order to restore and go to it at the end—if the linkage is `"return"` (that is, if the application is in a return statement and its value is the result to be returned):

```
  save("continue"),        // save the caller's continuation
  assign("continue", label("fun_return")), // where function should return to
  save("continue"),        // will be restored by the function
  push_marker_to_stack(), // allows the function to revert stack to find fun_return
  assign("val", list(op("compiled_function_entry"), reg("fun"))),
  go_to(reg("val")),     // eventually reverts stack, restores and jumps to continue
"fun_return",              // the function returns to here
  assign(target, reg("val")), // included if target is not val
  restore("continue"),     // restore the caller's continuation
  go_to(reg("continue")), // linkage code
```

This code sets up `continue` so that the function will return to a label `fun_return` and jumps to the function's entry point. The code at `fun_return` transfers the function's result from `val` to the target register (if necessary) and then jumps to the location specified by the linkage. (The linkage is always `"return"` or a label, because `compile_function_call` replaces a `"next"` linkage for the compound-function branch by an `after_call` label.) Before jumping to the function's entry point, we save `continue` and execute `push_marker_to_stack()` to enable the function to return to the intended location in the program with the expected stack. Matching `revert_stack_to_marker()` and `restore("continue")` instructions are generated by `compile_return_statement` for each return statement in the body of the function.[46]

In fact, if the target is not `val`, the above is exactly the code our compiler will generate.[47] Usually, however, the target is `val` (the only time the compiler specifies a different register is when targeting the evaluation of a function expression to `fun`), so the function result is put directly into the target register and there is no need to jump to a special location that copies it. Instead we simplify the code by setting up `continue` so that the called function will "return" directly to the place specified by the caller's linkage:

---

46. Elsewhere in the compiler, all saves and restores of registers are generated by `preserving` to preserve a register's value across a sequence of instructions by saving it before those instructions and restoring it after—for example over the evaluation of the predicate of a conditional. But this mechanism cannot generate instructions to save and restore `continue` for a function application and the corresponding return, because these are compiled separately and are not contiguous. Instead, these saves and restores must be explicitly generated by `compile_fun_appl` and `compile_return_statement`.

47. Actually, we signal an error when the target is not `val` and the linkage is `"return"`, since the only place we request a `"return"` linkage is in compiling return expressions, and our convention is that functions return their values in `val`.

⟨*set up* `continue` *for linkage and push the marker*⟩
```
assign("val", list(op("compiled_function_entry"), reg("fun"))),
go_to(reg("val")),
```

If the linkage is a label, we set up `continue` so that the function will continue at that label. (That is, the `go_to(reg("continue"))` the called function ends with becomes equivalent to the `go_to(label(`*linkage*`))` at `fun_return` above.)

```
assign("continue", label(linkage)),
save("continue"),
push_marker_to_stack(),
assign("val", list(op("compiled_function_entry"), reg("fun"))),
go_to(reg("val")),
```

If the linkage is `"return"`, we don't need to assign `continue`: It already holds the desired location. (That is, the `go_to(reg("continue"))` the called function ends with goes directly to the place where the `go_to(reg("continue"))` at `fun_return` would have gone.)

```
save("continue"),
push_marker_to_stack(),
assign("val", list(op("compiled_function_entry"), reg("fun"))),
go_to(reg("val")),
```

With this implementation of the `"return"` linkage, the compiler generates tail-recursive code. A function call in a return statement whose value is the result to be returned does a direct transfer, without saving unnecessary information on the stack.

Suppose instead that we had handled the case of a function call with a linkage of `"return"` and a target of `val` in the same way as for a non-`val` target. This would destroy tail recursion. Our system would still return the same value for any function call. But each time we called a function, we would save `continue` and return after the call to undo the (useless) save. These extra saves would accumulate during a nest of function calls.[48]

48. Making a compiler generate tail-recursive code is desirable, especially in the functional paradigm. However, compilers for common languages, including C and C++, do not always do this, and therefore these languages cannot represent iterative processes in terms of function call alone. The difficulty with tail recursion in these languages is that their implementations use the stack to store function arguments and local names as well as return addresses. The JavaScript implementations described in this book store arguments and names in memory to be garbage-collected. The reason for using the stack for names and arguments is that it avoids the need for garbage collection in languages that would not otherwise require it, and is generally believed to be more efficient. Sophisticated compilers can, in fact, use the stack for arguments without destroying tail recursion. (See Hanson 1990 for a description.) There is also some debate about whether stack allocation is actually more efficient than garbage collection in the first place, but the details seem to hinge on fine points of computer architecture. (See Appel 1987 and Miller and Rozas 1994 for opposing views on this issue.)

The function `compile_fun_appl` generates the above function-application code by considering four cases, depending on whether the target for the call is `val` and whether the linkage is `"return"`. Observe that the instruction sequences are declared to modify all the registers, since executing the function body can change the registers in arbitrary ways.[49]

```
function compile_fun_appl(target, linkage) {
    const fun_return = make_label("fun_return");
    return target === "val" && linkage !== "return"
           ? make_instruction_sequence(list("fun"), all_regs,
                 list(assign("continue", label(linkage)),
                      save("continue"),
                      push_marker_to_stack(),
                      assign("val", list(op("compiled_function_entry"),
                                         reg("fun"))),
                      go_to(reg("val"))))
           : target !== "val" && linkage !== "return"
           ? make_instruction_sequence(list("fun"), all_regs,
                 list(assign("continue", label(fun_return)),
                      save("continue"),
                      push_marker_to_stack(),
                      assign("val", list(op("compiled_function_entry"),
                                         reg("fun"))),
                      go_to(reg("val")),
                      fun_return,
                      assign(target, reg("val")),
                      go_to(label(linkage))))
           : target === "val" && linkage === "return"
           ? make_instruction_sequence(list("fun", "continue"),
                                       all_regs,
                 list(save("continue"),
                      push_marker_to_stack(),
                      assign("val", list(op("compiled_function_entry"),
                                         reg("fun"))),
                      go_to(reg("val"))))
           : //target !== "val" && linkage === "return"
             error(target, "return linkage, target not val -- compile");
}
```

We have shown how to generate tail-recursive linkage code for a function application when the linkage is `"return"`—that is, when the application is in a return statement and its value is the result to be returned. Similarly, as explained in section 5.4.2, the stack-marker mechanism used here (and in the explicit-control evaluator) for the call and return produces tail-recursive behavior only in that situation. These two aspects of the code generated for function application combine to ensure that when a function ends by returning the value of a function call, no stack accumulates.

---

49. The constant `all_regs` is bound to the list of names of all the registers:

```
const all_regs = list("env", "fun", "val", "argl", "continue");
```

Compiling return statements

The code for a return statement takes the following form, regardless of the given linkage and target:

```
revert_stack_to_marker(),
restore("continue"),    // saved by compile_fun_appl
⟨evaluate the return expression and store the result in val⟩
go_to(reg("continue")) // "return"-linkage code
```

The instructions to revert the stack using the marker and then restore `continue` correspond to the instructions generated by `compile_fun_appl` to save `continue` and mark the stack. The final jump to `continue` is generated by the use of the `"return"` linkage when compiling the return expression. The function `compile_return_statement` is different from all other code generators in that it ignores the target and linkage arguments—it always compiles the return expression with target `val` and linkage `"return"`.

```
function compile_return_statement(stmt, target, linkage) {
    return append_instruction_sequences(
               make_instruction_sequence(null, list("continue"),
                   list(revert_stack_to_marker(),
                        restore("continue"))),
               compile(return_expression(stmt), "val", "return"));
}
```

5.5.4 Combining Instruction Sequences

This section describes the details on how instruction sequences are represented and combined. Recall from section 5.5.1 that an instruction sequence is represented as a list of the registers needed, the registers modified, and the actual instructions. We will also consider a label (string) to be a degenerate case of an instruction sequence, which doesn't need or modify any registers. So to determine the registers needed and modified by instruction sequences we use the selectors

```
function registers_needed(s) {
    return is_string(s) ? null : head(s);
}
function registers_modified(s) {
    return is_string(s) ? null : head(tail(s));
}
function instructions(s) {
    return is_string(s) ? list(s) : head(tail(tail(s)));
}
```

and to determine whether a given sequence needs or modifies a given register we use the predicates

```
function needs_register(seq, reg) {
    return ! is_null(member(reg, registers_needed(seq)));
}
function modifies_register(seq, reg) {
    return ! is_null(member(reg, registers_modified(seq)));
}
```

In terms of these predicates and selectors, we can implement the various instruction sequence combiners used throughout the compiler.

The basic combiner is `append_instruction_sequences`. This takes as arguments two instruction sequences that are to be executed sequentially and returns an instruction sequence whose statements are the statements of the two sequences appended together. The subtle point is to determine the registers that are needed and modified by the resulting sequence. It modifies those registers that are modified by either sequence; it needs those registers that must be initialized before the first sequence can be run (the registers needed by the first sequence), together with those registers needed by the second sequence that are not initialized (modified) by the first sequence.

The function `append_instruction_sequences` is given two instruction sequences `seq1` and `seq2` and returns the instruction sequence whose instructions are the instructions of `seq1` followed by the instructions of `seq2`, whose modified registers are those registers that are modified by either `seq1` or `seq2`, and whose needed registers are the registers needed by `seq1` together with those registers needed by `seq2` that are not modified by `seq1`. (In terms of set operations, the new set of needed registers is the union of the set of registers needed by `seq1` with the set difference of the registers needed by `seq2` and the registers modified by `seq1`.) Thus, `append_instruction_sequences` is implemented as follows:

```
function append_instruction_sequences(seq1, seq2) {
    return make_instruction_sequence(
               list_union(registers_needed(seq1),
                          list_difference(registers_needed(seq2),
                                          registers_modified(seq1))),
               list_union(registers_modified(seq1),
                          registers_modified(seq2)),
               append(instructions(seq1), instructions(seq2)));
}
```

This function uses some simple operations for manipulating sets represented as lists, similar to the (unordered) set representation described in section 2.3.3:

```
function list_union(s1, s2) {
    return is_null(s1)
           ? s2
           : is_null(member(head(s1), s2))
           ? pair(head(s1), list_union(tail(s1), s2))
           : list_union(tail(s1), s2);
}
function list_difference(s1, s2) {
    return is_null(s1)
           ? null
           : is_null(member(head(s1), s2))
           ? pair(head(s1), list_difference(tail(s1), s2))
           : list_difference(tail(s1), s2);
}
```

The function `preserving`, the second major instruction sequence combiner, takes a list of registers `regs` and two instruction sequences `seq1` and `seq2` that

are to be executed sequentially. It returns an instruction sequence whose instructions are the instructions of `seq1` followed by the instructions of `seq2`, with appropriate `save` and `restore` instructions around `seq1` to protect the registers in `regs` that are modified by `seq1` but needed by `seq2`. To accomplish this, `preserving` first creates a sequence that has the required `save`s followed by the instructions of `seq1` followed by the required `restore`s. This sequence needs the registers being saved and restored in addition to the registers needed by `seq1`, and modifies the registers modified by `seq1` except for the ones being saved and restored. This augmented sequence and `seq2` are then appended in the usual way. The following function implements this strategy recursively, walking down the list of registers to be preserved:

```
function preserving(regs, seq1, seq2) {
    if (is_null(regs)) {
        return append_instruction_sequences(seq1, seq2);
    } else {
        const first_reg = head(regs);
        return needs_register(seq2, first_reg) &&
               modifies_register(seq1, first_reg)
               ? preserving(tail(regs),
                     make_instruction_sequence(
                         list_union(list(first_reg),
                                    registers_needed(seq1)),
                         list_difference(registers_modified(seq1),
                                         list(first_reg)),
                         append(list(save(first_reg)),
                                append(instructions(seq1),
                                       list(restore(first_reg))))),
                     seq2)
               : preserving(tail(regs), seq1, seq2);
    }
}
```

Another sequence combiner, `tack_on_instruction_sequence`, is used by `compile_lambda_expression` to append a function body to another sequence. Because the function body is not "in line" to be executed as part of the combined sequence, its register use has no impact on the register use of the sequence in which it is embedded. We thus ignore the function body's sets of needed and modified registers when we tack it onto the other sequence.

```
function tack_on_instruction_sequence(seq, body_seq) {
    return make_instruction_sequence(
               registers_needed(seq),
               registers_modified(seq),
               append(instructions(seq), instructions(body_seq)));
}
```

The functions `compile_conditional` and `compile_function_call` use a special combiner called `parallel_instruction_sequences` to append the two alternative branches that follow a test. The two branches will never be executed sequentially; for any particular evaluation of the test, one branch or the other will be entered. Because of this, the registers needed by the second branch are still needed by the combined sequence, even if these are modified by the first branch.

```
function parallel_instruction_sequences(seq1, seq2) {
    return make_instruction_sequence(
               list_union(registers_needed(seq1),
                          registers_needed(seq2)),
               list_union(registers_modified(seq1),
                          registers_modified(seq2)),
               append(instructions(seq1), instructions(seq2)));
}
```

5.5.5 An Example of Compiled Code

Now that we have seen all the elements of the compiler, let us examine an example of compiled code to see how things fit together. We will compile the declaration of a recursive `factorial` function by passing as first argument to `compile` the result of applying `parse` to a string representation of the program (here using back quotes `` `...` ``, which work like single and double quotation marks but allow the string to span multiple lines):

```
compile(parse(`
function factorial(n) {
    return n === 1
           ? 1
           : factorial(n - 1) * n;
}
              `),
        "val",
        "next");
```

We have specified that the value of the declaration should be placed in the `val` register. We don't care what the compiled code does after executing the declaration, so our choice of `"next"` as the linkage descriptor is arbitrary.

The function `compile` determines that it was given a function declaration, so it transforms it to a constant declaration and then calls `compile_declaration`. This compiles code to compute the value to be assigned (targeted to `val`), followed by code to install the declaration, followed by code to put the value of the declaration (which is the value `undefined`) into the target register, followed finally by the linkage code. The `env` register is preserved around the computation of the value, because it is needed in order to install the declaration. Because the linkage is `"next"`, there is no linkage code in this case. The skeleton of the compiled code is thus

⟨*save* `env` *if modified by code to compute value*⟩
⟨*compilation of declaration value*, *target* `val`, *linkage* `"next"`⟩
⟨*restore* `env` *if saved above*⟩

```
perform(list(op("assign_symbol_value"),
             constant("factorial"),
             reg("val"),
             reg("env"))),
assign("val", constant(undefined))
```

The expression that is compiled to produce the value for the name `factorial` is a lambda expression whose value is the function that computes factorials. The function `compile` handles this by calling `compile_lambda_expression`, which compiles

the function body, labels it as a new entry point, and generates the instruction that will combine the function body at the new entry point with the runtime environment and assign the result to `val`. The sequence then skips around the compiled function code, which is inserted at this point. The function code itself begins by extending the function's declaration environment by a frame that binds the parameter `n` to the function argument. Then comes the actual function body. Since this code for the value of the name doesn't modify the `env` register, the optional `save` and `restore` shown above aren't generated. (The function code at `entry1` isn't executed at this point, so its use of `env` is irrelevant.) Therefore, the skeleton for the compiled code becomes

```
  assign("val", list(op("make_compiled_function"),
                     label("entry1"),
                     reg("env"))),
  go_to(label("after_lambda2")),
"entry1",
  assign("env", list(op("compiled_function_env"), reg("fun"))),
  assign("env", list(op("extend_environment"),
                     constant(list("n")),
                     reg("argl"),
                     reg("env"))),
  ⟨compilation of function body⟩
"after_lambda2",
  perform(list(op("assign_symbol_value"),
               constant("factorial"),
               reg("val"),
               reg("env"))),
  assign("val", constant(undefined))
```

A function body is always compiled (by `compile_lambda_body`) with target `val` and linkage `"next"`. The body in this case consists of a single return statement:[50]

```
return n === 1
       ? 1
       : factorial(n - 1) * n;
```

The function `compile_return_statement` generates code to revert the stack using the marker and to restore the `continue` register, and then compiles the return expression with target `val` and linkage `"return"`, because its value is to be returned from the function. The return expression is a conditional expression, for which `compile_conditional` generates code that first computes the predicate (targeted to `val`), then checks the result and branches around the true branch if the predicate is false. Registers `env` and `continue` are preserved around the predicate code, since they may be needed for the rest of the conditional expression. The true and false branches are both compiled with target `val` and linkage `"return"`. (That is, the

50. Because of the `append_return_undefined` in `compile_lambda_body`, the body actually consists of a sequence with two return statements. However, the dead-code check in `compile_sequence` will stop after the compilation of the first return statement, so the body effectively consists of only a single return statement.

value of the conditional, which is the value computed by either of its branches, is the value of the function.)

```
  revert_stack_to_marker(),
  restore("continue"),
  ⟨save continue, env if modified by predicate and needed by branches⟩
  ⟨compilation of predicate, target val, linkage "next"⟩
  ⟨restore continue, env if saved above⟩
  test(list(op("is_falsy"), reg("val"))),
  branch(label("false_branch4")),
"true_branch3",
  ⟨compilation of true branch, target val, linkage "return"⟩
"false_branch4",
  ⟨compilation of false branch, target val, linkage "return"⟩
"after_cond5",
```

The predicate `n === 1` is a function application (after transformation of the operator combination). This looks up the function expression (the symbol `"==="`) and places this value in `fun`. It then assembles the arguments `1` and the value of `n` into `argl`. Then it tests whether `fun` contains a primitive or a compound function, and dispatches to a primitive branch or a compound branch accordingly. Both branches resume at the `after_call` label. The compound branch must set up `continue` to jump past the primitive branch and push a marker to the stack to match the revert operation in the compiled return statement of the function. The requirements to preserve registers around the evaluation of the function and argument expressions don't result in any saving of registers, because in this case those evaluations don't modify the registers in question.

```
  assign("fun", list(op("lookup_symbol_value"),
                     constant("==="), reg("env"))),
  assign("val", constant(1)),
  assign("argl", list(op("list"), reg("val"))),
  assign("val", list(op("lookup_symbol_value"),
                     constant("n"), reg("env"))),
  assign("argl", list(op("pair"), reg("val"), reg("argl"))),
  test(list(op("is_primitive_function"), reg("fun"))),
  branch(label("primitive_branch6")),
"compiled_branch7",
  assign("continue", label("after_call8")),
  save("continue"),
  push_marker_to_stack(),
  assign("val", list(op("compiled_function_entry"), reg("fun"))),
  go_to(reg("val")),
"primitive_branch6",
  assign("val", list(op("apply_primitive_function"),
                     reg("fun"),
                     reg("argl"))),
"after_call8",
```

The true branch, which is the constant 1, compiles (with target `val` and linkage `"return"`) to

```
  assign("val", constant(1)),
  go_to(reg("continue")),
```

The code for the false branch is another function call, where the function is the value of the symbol `"*"`, and the arguments are `n` and the result of another function call (a call to `factorial`). Each of these calls sets up `fun` and `argl` and its own primitive and compound branches. Figure 5.17 shows the complete compilation of the declaration of the `factorial` function. Notice that the possible `save` and `restore` of `continue` and `env` around the predicate, shown above, are in fact generated, because these registers are modified by the function call in the predicate and needed for the function call and the `"return"` linkage in the branches.

Exercise 5.36

Consider the following declaration of a factorial function, which is slightly different from the one given above:

```
function factorial_alt(n) {
    return n === 1
           ? 1
           : n * factorial_alt(n - 1);
}
```

Compile this function and compare the resulting code with that produced for `factorial`. Explain any differences you find. Does either program execute more efficiently than the other?

Exercise 5.37

Compile the iterative factorial function

```
function factorial(n) {
    function iter(product, counter) {
        return counter > n
               ? product
               : iter(product * counter, counter + 1);
    }
    return iter(1, 1);
}
```

Annotate the resulting code, showing the essential difference between the code for iterative and recursive versions of `factorial` that makes one process build up stack space and the other run in constant stack space.

Exercise 5.38

What program was compiled to produce the code shown in figure 5.18?

Exercise 5.39

What order of evaluation does our compiler produce for arguments of an application? Is it left-to-right (as mandated by the ECMAScript specification), right-to-left, or some other order? Where in the compiler is this order determined? Modify the compiler so that it produces some other order of evaluation. (See the discussion of order of evaluation for the explicit-control evaluator in section 5.4.1.) How does changing the order of argument evaluation affect the efficiency of the code that constructs the argument list?

```
// construct the function and skip over the code for the function body
  assign("val", list(op("make_compiled_function"),
                     label("entry1"), reg("env"))),
  go_to(label("after_lambda2")),
"entry1",                              // calls to factorial will enter here
  assign("env", list(op("compiled_function_env"), reg("fun"))),
  assign("env", list(op("extend_environment"), constant(list("n")),
                     reg("argl"), reg("env"))),
// begin actual function body
  revert_stack_to_marker(),            // starts with a return statement
  restore("continue"),
  save("continue"),                    // preserve registers across predicate
  save("env"),
// compute n === 1
  assign("fun", list(op("lookup_symbol_value"), constant("==="), reg("env"))),
  assign("val", constant(1)),
  assign("argl", list(op("list"), reg("val"))),
  assign("val", list(op("lookup_symbol_value"), constant("n"), reg("env"))),
  assign("argl", list(op("pair"), reg("val"), reg("argl"))),
  test(list(op("is_primitive_function"), reg("fun"))),
  branch(label("primitive_branch6")),
"compiled_branch7",
  assign("continue", label("after_call8")),
  save("continue"),
  push_marker_to_stack(),
  assign("val", list(op("compiled_function_entry"), reg("fun"))),
  go_to(reg("val")),
"primitive_branch6",
  assign("val", list(op("apply_primitive_function"), reg("fun"), reg("argl"))),
"after_call8",                         // val now contains result of n === 1
  restore("env"),
  restore("continue"),
  test(list(op("is_falsy"), reg("val"))),
  branch(label("false_branch4")),
"true_branch3",                        // return 1
  assign("val", constant(1)),
  go_to(reg("continue")),
"false_branch4",
// compute and return factorial(n - 1) * n
  assign("fun", list(op("lookup_symbol_value"), constant("*"), reg("env"))),
  save("continue"),
  save("fun"),                         // save * function
  assign("val", list(op("lookup_symbol_value"), constant("n"), reg("env"))),
  assign("argl", list(op("list"), reg("val"))),
  save("argl"),                        // save partial argument list for *
// compute factorial(n - 1) which is the other argument for *
  assign("fun", list(op("lookup_symbol_value"),
                     constant("factorial"), reg("env"))),
  save("fun"),                         // save factorial function
```

Figure 5.17 Compilation of the declaration of the `factorial` function (continued on next page).

```
// compute n - 1 which is the argument for factorial
  assign("fun", list(op("lookup_symbol_value"), constant("-"), reg("env"))),
  assign("val", constant(1)),
  assign("argl", list(op("list"), reg("val"))),
  assign("val", list(op("lookup_symbol_value"), constant("n"), reg("env"))),
  assign("argl", list(op("pair"), reg("val"), reg("argl"))),
  test(list(op("is_primitive_function"), reg("fun"))),
  branch(label("primitive_branch10")),
"compiled_branch11",
  assign("continue", label("after_call12")),
  save("continue"),
  push_marker_to_stack(),
  assign("val", list(op("compiled_function_entry"), reg("fun"))),
  go_to(reg("val")),
"primitive_branch10",
  assign("val", list(op("apply_primitive_function"), reg("fun"), reg("argl"))),
"after_call12",                    // val now contains result of n - 1
  assign("argl", list(op("list"), reg("val"))),
  restore("fun"),                  // restore factorial
// apply factorial
  test(list(op("is_primitive_function"), reg("fun"))),
  branch(label("primitive_branch14")),
"compiled_branch15",
  assign("continue", label("after_call16")),
  save("continue"),                // set up for compiled function –
  push_marker_to_stack(),          //   return in function will restore stack
  assign("val", list(op("compiled_function_entry"), reg("fun"))),
  go_to(reg("val")),
"primitive_branch14",
  assign("val", list(op("apply_primitive_function"), reg("fun"), reg("argl"))),
"after_call16",                    // val now contains result of factorial(n - 1)
  restore("argl"),                 // restore partial argument list for *
  assign("argl", list(op("pair"), reg("val"), reg("argl"))),
  restore("fun"),                  // restore *
  restore("continue"),
// apply * and return its value
  test(list(op("is_primitive_function"), reg("fun"))),
  branch(label("primitive_branch18")),
"compiled_branch19", // note that a compound function here is called tail-recursively
  save("continue"),
  push_marker_to_stack(),
  assign("val", list(op("compiled_function_entry"), reg("fun"))),
  go_to(reg("val")),
"primitive_branch18",
  assign("val", list(op("apply_primitive_function"), reg("fun"), reg("argl"))),
  go_to(reg("continue")),
"after_call20",
"after_cond5",
"after_lambda2",
// assign the function to the name factorial
  perform(list(op("assign_symbol_value"),
               constant("factorial"), reg("val"), reg("env"))),
  assign("val", constant(undefined))
```

Figure 5.17 (continued)

```
  assign("val", list(op("make_compiled_function"),
                     label("entry1"), reg("env"))),
"entry1"
  assign("env", list(op("compiled_function_env"), reg("fun"))),
  assign("env", list(op("extend_environment"),
                     constant(list("x")), reg("argl"), reg("env"))),
  revert_stack_to_marker(),
  restore("continue"),
  assign("fun", list(op("lookup_symbol_value"), constant("+"), reg("env"))),
  save("continue"),
  save("fun"),
  save("env"),
  assign("fun", list(op("lookup_symbol_value"), constant("g"), reg("env"))),
  save("fun"),
  assign("fun", list(op("lookup_symbol_value"), constant("+"), reg("env"))),
  assign("val", constant(2)),
  assign("argl", list(op("list"), reg("val"))),
  assign("val", list(op("lookup_symbol_value"), constant("x"), reg("env"))),
  assign("argl", list(op("pair"), reg("val"), reg("argl"))),
  test(list(op("is_primitive_function"), reg("fun"))),
  branch(label("primitive_branch3")),
"compiled_branch4"
  assign("continue", label("after_call5")),
  save("continue"),
  push_marker_to_stack(),
  assign("val", list(op("compiled_function_entry"), reg("fun"))),
  go_to(reg("val")),
"primitive_branch3",
  assign("val", list(op("apply_primitive_function"), reg("fun"), reg("argl"))),
"after_call5",
  assign("argl", list(op("list"), reg("val"))),
  restore("fun"),
  test(list(op("is_primitive_function"), reg("fun"))),
  branch(label("primitive_branch7")),
"compiled_branch8",
  assign("continue", label("after_call9")),
  save("continue"),
  push_marker_to_stack(),
  assign("val", list(op("compiled_function_entry"), reg("fun"))),
  go_to(reg("val")),
"primitive_branch7",
  assign("val", list(op("apply_primitive_function"), reg("fun"), reg("argl"))),
"after_call9",
  assign("argl", list(op("list"), reg("val"))),
  restore("env"),
  assign("val", list(op("lookup_symbol_value"), constant("x"), reg("env"))),
  assign("argl", list(op("pair"), reg("val"), reg("argl"))),
  restore("fun"),
  restore("continue"),
  test(list(op("is_primitive_function"), reg("fun"))),
  branch(label("primitive_branch11")),
```

Figure 5.18 An example of compiler output (continued on next page). See exercise 5.38.

```
"compiled_branch12",
  save("continue"),
  push_marker_to_stack(),
  assign("val", list(op("compiled_function_entry"), reg("fun"))),
  go_to(reg("val")),
"primitive_branch11",
  assign("val", list(op("apply_primitive_function"), reg("fun"), reg("argl"))),
  go_to(reg("continue")),
"after_call13",
"after_lambda2",
  perform(list(op("assign_symbol_value"),
               constant("f"), reg("val"), reg("env"))),
  assign("val", constant(undefined))
```

Figure 5.18 (continued)

Exercise 5.40

One way to understand the compiler's `preserving` mechanism for optimizing stack usage is to see what extra operations would be generated if we did not use this idea. Modify `preserving` so that it always generates the `save` and `restore` operations. Compile some simple expressions and identify the unnecessary stack operations that are generated. Compare the code to that generated with the `preserving` mechanism intact.

Exercise 5.41

Our compiler is clever about avoiding unnecessary stack operations, but it is not clever at all when it comes to compiling calls to the primitive functions of the language in terms of the primitive operations supplied by the machine. For example, consider how much code is compiled to compute `a + 1`: The code sets up an argument list in `argl`, puts the primitive addition function (which it finds by looking up the symbol `"+"` in the environment) into `fun`, and tests whether the function is primitive or compound. The compiler always generates code to perform the test, as well as code for primitive and compound branches (only one of which will be executed). We have not shown the part of the controller that implements primitives, but we presume that these instructions make use of primitive arithmetic operations in the machine's data paths. Consider how much less code would be generated if the compiler could *open-code* primitives—that is, if it could generate code to directly use these primitive machine operations. The expression `a + 1` might be compiled into something as simple as[51]

```
assign("val", list(op("lookup_symbol_value"), constant("a"), reg("env"))),
assign("val", list(op("+"), reg("val"), constant(1)))
```

In this exercise we will extend our compiler to support open coding of selected primitives. Special-purpose code will be generated for calls to these primitive functions instead of the general function-application code. In order to support this, we will augment our machine with special argument registers `arg1` and `arg2`. The primitive arithmetic operations of the

51. We have used the same symbol + here to denote both the source-language function and the machine operation. In general there will not be a one-to-one correspondence between primitives of the source language and primitives of the machine.

machine will take their inputs from `arg1` and `arg2`. The results may be put into `val`, `arg1`, or `arg2`.

The compiler must be able to recognize the application of an open-coded primitive in the source program. We will augment the dispatch in the `compile` function to recognize the names of these primitives in addition to the syntactic forms it currently recognizes. For each syntactic form our compiler has a code generator. In this exercise we will construct a family of code generators for the open-coded primitives.

a. The open-coded primitives, unlike the syntactic forms, all need their argument expressions evaluated. Write a code generator `spread_arguments` for use by all the open-coding code generators. The function `spread_arguments` should take a list of argument expressions and compile the given argument expressions targeted to successive argument registers. Note that an argument expression may contain a call to an open-coded primitive, so argument registers will have to be preserved during argument-expression evaluation.

b. The JavaScript operators `===`, `*`, `-`, and `+`, among others, are implemented in the register machine as primitive functions and are referred to in the global environment with the symbols `"==="`, `"*"`, `"-"`, and `"+"`. In JavaScript, it is not possible to redeclare these names, because they do not meet the syntactic restrictions for names. This means it is safe to open-code them. For each of the primitive functions `===`, `*`, `-`, and `+`, write a code generator that takes an application with a function expression that names that function, together with a target and a linkage descriptor, and produces code to spread the arguments into the registers and then perform the operation targeted to the given target with the given linkage. Make `compile` dispatch to these code generators.

c. Try your new compiler on the `factorial` example. Compare the resulting code with the result produced without open coding.

5.5.6 Lexical Addressing

One of the most common optimizations performed by compilers is the optimization of name lookup. Our compiler, as we have implemented it so far, generates code that uses the `lookup_symbol_value` operation of the evaluator machine. This searches for a name by comparing it with each name that is currently bound, working frame by frame outward through the runtime environment. This search can be expensive if the frames are deeply nested or if there are many names. For example, consider the problem of looking up the value of `x` while evaluating the expression `x * y * z` in an application of the function of five arguments that is returned by

```
((x, y) =>
   (a, b, c, d, e) =>
     ((y, z) => x * y * z)(a * b * x, c + d + x))(3, 4)
```

Each time `lookup_symbol_value` searches for `x`, it must determine that the symbol `"x"` is not equal to `"y"` or `"z"` (in the first frame), nor to `"a"`, `"b"`, `"c"`, `"d"`, or `"e"` (in the second frame). Because our language is lexically scoped, the runtime environment for any component will have a structure that parallels the lexical structure of the program in which the component appears. Thus, the compiler can know, when it analyzes the above expression, that each time the function is applied the binding

for x in x * y * z will be found two frames out from the current frame and will be the first binding in that frame.

We can exploit this fact by inventing a new kind of name-lookup operation, lexical_address_lookup, that takes as arguments an environment and a *lexical address* that consists of two numbers: a *frame number*, which specifies how many frames to pass over, and a *displacement number*, which specifies how many bindings to pass over in that frame. The operation lexical_address_lookup will produce the value of the name stored at that lexical address relative to the current environment. If we add the lexical_address_lookup operation to our machine, we can make the compiler generate code that references names using this operation, rather than lookup_symbol_value. Similarly, our compiled code can use a new lexical_address_assign operation instead of assign_symbol_value. With lexical addressing, there is no need to include any symbolic references to names in the object code, and frames do not need to include symbols at run time.

In order to generate such code, the compiler must be able to determine the lexical address of a name it is about to compile a reference to. The lexical address of a name in a program depends on where one is in the code. For example, in the following program, the address of x in expression e_1 is (2,0)—two frames back and the first name in the frame. At that point y is at address (0,0) and c is at address (1,2). In expression e_2, x is at (1,0), y is at (1,1), and c is at (0,2).

```
((x, y) =>
   (a, b, c, d, e) =>
     ((y, z) => e1)(e2, c + d + x))(3, 4);
```

One way for the compiler to produce code that uses lexical addressing is to maintain a data structure called a *compile-time environment*. This keeps track of which bindings will be at which positions in which frames in the runtime environment when a particular name-access operation is executed. The compile-time environment is a list of frames, each containing a list of symbols. There will be no values associated with the symbols, since values are not computed at compile time. (Exercise 5.47 will change this, as an optimization for constants.) The compile-time environment becomes an additional argument to compile and is passed along to each code generator. The top-level call to compile uses a compile-time-environment that includes the names of all primitive functions and primitive values. When the body of a lambda expression is compiled, compile_lambda_body extends the compile-time environment by a frame containing the function's parameters, so that the body is compiled with that extended environment. Similarly, when the body of a block is compiled, compile_block extends the compile-time environment by a frame containing the scanned-out local names of the body. At each point in the compilation, compile_name and compile_assignment_declaration use the compile-time environment in order to generate the appropriate lexical addresses.

Exercises 5.42 through 5.45 describe how to complete this sketch of the lexical-addressing strategy in order to incorporate lexical lookup into the compiler. Exercises 5.46 and 5.47 describe other uses for the compile-time environment.

Exercise 5.42

Write a function `lexical_address_lookup` that implements the new lookup operation. It should take two arguments—a lexical address and a runtime environment—and return the value of the name stored at the specified lexical address. The function `lexical_address_lookup` should signal an error if the value of the name is the string `"*unassigned*"`. Also write a function `lexical_address_assign` that implements the operation that changes the value of the name at a specified lexical address.

Exercise 5.43

Modify the compiler to maintain the compile-time environment as described above. That is, add a compile-time-environment argument to `compile` and the various code generators, and extend it in `compile_lambda_body` and `compile_block`.

Exercise 5.44

Write a function `find_symbol` that takes as arguments a symbol and a compile-time environment and returns the lexical address of the symbol with respect to that environment. For example, in the program fragment that is shown above, the compile-time environment during the compilation of expression e_1 is

```
list(list("y", "z"),
     list("a", "b", "c", "d", "e"),
     list("x", "y"))
```

The function `find_symbol` should produce

```
find_symbol("c", list(list("y", "z"),
                      list("a", "b", "c", "d", "e"),
                      list("x", "y")));
list(1, 2)

find_symbol("x", list(list("y", "z"),
                      list("a", "b", "c", "d", "e"),
                      list("x", "y")));
list(2, 0)

find_symbol("w", list(list("y", "z"),
                      list("a", "b", "c", "d", "e"),
                      list("x", "y")));
"not found"
```

Exercise 5.45

Using `find_symbol` from exercise 5.44, rewrite `compile_assignment_declaration` and `compile_name` to output lexical-address instructions. In cases where `find_symbol` returns `"not found"` (that is, where the name is not in the compile-time environment), you should report a compile-time error. Test the modified compiler on a few simple cases, such as the nested lambda combination at the beginning of this section.

Exercise 5.46

In JavaScript, an attempt to assign a new value to a name that is declared as a constant leads to an error. Exercise 4.11 shows how to detect such errors at run time. With the techniques presented in this section, we can detect attempts to assign a new value to a constant *at compile time*. For this purpose, extend the functions `compile_lambda_body` and `compile_block` to record in the compile-time environment whether a name is declared as a variable (using `let` or as a parameter), or as a constant (using `const` or `function`). Modify `compile_assignment` to report an appropriate error when it detects an assignment to a constant.

Exercise 5.47

Knowledge about constants at compile time opens the door to many optimizations that allow us to generate more efficient object code. In addition to the extension of the compile-time environment in exercise 5.46 to indicate names declared as constants, we may store the value of a constant if it is known at compile time, or other information that can help us optimize the code.

a. A constant declaration such as `const` *name* `=` *literal*`;` allows us to replace all occurrences of *name* within the scope of the declaration by *literal* so that *name* doesn't have to be looked up in the runtime environment. This optimization is called *constant propagation*. Use an extended compile-time environment to store literal constants, and modify `compile_name` to use the stored constant in the generated `assign` instruction instead of the `lookup_symbol_value` operation.

b. Function declaration is a derived component that expands to constant declaration. Let us assume that the names of primitive functions in the global environment are also considered constants. If we further extend our compile-time environment to keep track of which names refer to compiled functions and which ones to primitive functions, we can move the test that checks whether a function is compiled or primitive from run time to compile time. This makes the object code more efficient because it replaces a test that must be performed once per function application in the generated code by one that is performed by the compiler. Using such an extended compile-time environment, modify `compile_function_call` so that if it can be determined at compile time whether the called function is compiled or primitive, only the instructions in the `compiled_branch` or the `primitive_branch` are generated.

c. Replacing constant names with their literal values as in part (a) paves the way for another optimization, namely replacing applications of primitive functions to literal values with the compile-time computed result. This optimization, called *constant folding*, replaces expressions such as `40 + 2` by `42` by performing the addition in the compiler. Extend the compiler to perform constant folding for arithmetic operations on numbers and for string concatenation.

5.5.7 Interfacing Compiled Code to the Evaluator

We have not yet explained how to load compiled code into the evaluator machine or how to run it. We will assume that the explicit-control-evaluator machine has been defined as in section 5.4.4, with the additional operations specified in footnote 43

(section 5.5.2). We will implement a function `compile_and_go` that compiles a JavaScript program, loads the resulting object code into the evaluator machine, and causes the machine to run the code in the evaluator global environment, print the result, and enter the evaluator's driver loop. We will also modify the evaluator so that interpreted components can call compiled functions as well as interpreted ones. We can then put a compiled function into the machine and use the evaluator to call it:

```
compile_and_go(parse(`
function factorial(n) {
    return n === 1
           ? 1
           : factorial(n - 1) * n;
}
                     `));
EC-evaluate value:
undefined

EC-evaluate input:
factorial(5);

EC-evaluate value:
120
```

To allow the evaluator to handle compiled functions (for example, to evaluate the call to `factorial` above), we need to change the code at `apply_dispatch` (section 5.4.2) so that it recognizes compiled functions (as distinct from compound or primitive functions) and transfers control directly to the entry point of the compiled code:[52]

```
"apply_dispatch",
  test(list(op("is_primitive_function"), reg("fun"))),
  branch(label("primitive_apply")),
  test(list(op("is_compound_function"), reg("fun"))),
  branch(label("compound_apply")),
  test(list(op("is_compiled_function"), reg("fun"))),
  branch(label("compiled_apply")),
  go_to(label("unknown_function_type")),

"compiled_apply",
  push_marker_to_stack(),
  assign("val", list(op("compiled_function_entry"), reg("fun"))),
  go_to(reg("val")),
```

At `compiled_apply`, as at `compound_apply`, we push a marker to the stack so that a return statement in the compiled function can revert the stack to this state. Note that there is no save of `continue` at `compiled_apply` before the marking of the stack, because the evaluator was arranged so that at `apply_dispatch`, the continuation would be at the top of the stack.

---

52. Of course, compiled functions as well as interpreted functions are compound (nonprimitive). For compatibility with the terminology used in the explicit-control evaluator, in this section we will use "compound" to mean interpreted (as opposed to compiled).

To enable us to run some compiled code when we start the evaluator machine, we add a `branch` instruction at the beginning of the evaluator machine, which causes the machine to go to a new entry point if the `flag` register is set.[53]

```
  branch(label("external_entry")), // branches if flag is set
"read_evaluate_print_loop",
  perform(list(op("initialize_stack"))),
  ...
```

The code at `external_entry` assumes that the machine is started with `val` containing the location of an instruction sequence that puts a result into `val` and ends with `go_to(reg("continue"))`. Starting at this entry point jumps to the location designated by `val`, but first assigns `continue` so that execution will return to `print_result`, which prints the value in `val` and then goes to the beginning of the evaluator's read-evaluate-print loop.[54]

```
"external_entry",
  perform(list(op("initialize_stack"))),
  assign("env", list(op("get_current_environment"))),
  assign("continue", label("print_result")),
  go_to(reg("val")),
```

Now we can use the following function to compile a function declaration, execute the compiled code, and run the read-evaluate-print loop so we can try the function. Because we want the compiled code to proceed to the location in `continue` with its result in `val`, we compile the program with a target of `val` and a linkage of `"return"`. In order to transform the object code produced by the compiler into executable instructions for the evaluator register machine, we use the function `assemble` from

---

53. Now that the evaluator machine starts with a `branch`, we must always initialize the `flag` register before starting the evaluator machine. To start the machine at its ordinary read-evaluate-print loop, we could use

```
function start_eceval() {
    set_register_contents(eceval, "flag", false);
    return start(eceval);
}
```

54. Since a compiled function is an object that the system may try to print, we also modify the system print operation `user_print` (from section 4.1.4) so that it will not attempt to print the components of a compiled function:

```
function user_print(string, object) {
    function prepare(object) {
        return is_compound_function(object)
               ? "< compound function >"
               : is_primitive_function(object)
               ? "< primitive function >"
               : is_compiled_function(object)
               ? "< compiled function >"
               : is_pair(object)
               ? pair(prepare(head(object)),
                      prepare(tail(object)))
               : object;
    }
    display(string + " " + stringify(prepare(object)));
}
```

the register-machine simulator (section 5.2.2). For the interpreted program to refer to the names that are declared at top level in the compiled program, we scan out the top-level names and extend the global environment by binding these names to `"*unassigned*"`, knowing that the compiled code will assign them the correct values. We then initialize the `val` register to point to the list of instructions, set the `flag` so that the evaluator will go to `external_entry`, and start the evaluator.

```
function compile_and_go(program) {
    const instrs = assemble(instructions(compile(program,
                                                 "val", "return")),
                            eceval);
    const toplevel_names = scan_out_declarations(program);
    const unassigneds = list_of_unassigned(toplevel_names);
    set_current_environment(extend_environment(
                               toplevel_names,
                               unassigneds,
                               the_global_environment));
    set_register_contents(eceval, "val", instrs);
    set_register_contents(eceval, "flag", true);
    return start(eceval);
}
```

If we have set up stack monitoring, as at the end of section 5.4.4, we can examine the stack usage of compiled code:

```
compile_and_go(parse(`
function factorial(n) {
    return n === 1
           ? 1
           : factorial(n - 1) * n;
}
                     `));
total pushes = 0
maximum depth = 0
EC-evaluate value:
undefined

EC-evaluate input:
factorial(5);

total pushes = 36
maximum depth = 14
EC-evaluate value:
120
```

Compare this example with the evaluation of `factorial(5)` using the interpreted version of the same function, shown at the end of section 5.4.4. The interpreted version required 151 pushes and a maximum stack depth of 28. This illustrates the optimization that results from our compilation strategy.

Interpretation and compilation

With the programs in this section, we can now experiment with the alternative execution strategies of interpretation and compilation.[55] An interpreter raises the machine to the level of the user program; a compiler lowers the user program to the level of the machine language. We can regard the JavaScript language (or any programming language) as a coherent family of abstractions erected on the machine language. Interpreters are good for interactive program development and debugging because the steps of program execution are organized in terms of these abstractions, and are therefore more intelligible to the programmer. Compiled code can execute faster, because the steps of program execution are organized in terms of the machine language, and the compiler is free to make optimizations that cut across the higher-level abstractions.[56]

The alternatives of interpretation and compilation also lead to different strategies for porting languages to new computers. Suppose that we wish to implement JavaScript for a new machine. One strategy is to begin with the explicit-control evaluator of section 5.4 and translate its instructions to instructions for the new machine. A different strategy is to begin with the compiler and change the code generators so that they generate code for the new machine. The second strategy allows us to run any JavaScript program on the new machine by first compiling it with the compiler running on our original JavaScript system, and linking it with a compiled version of the runtime library.[57] Better yet, we can compile the compiler itself, and run this on

---

55. We can do even better by extending the compiler to allow compiled code to call interpreted functions. See exercise 5.50.

56. Independent of the strategy of execution, we incur significant overhead if we insist that errors encountered in execution of a user program be detected and signaled, rather than being allowed to kill the system or produce wrong answers. For example, an out-of-bounds array reference can be detected by checking the validity of the reference before performing it. The overhead of checking, however, can be many times the cost of the array reference itself, and a programmer should weigh speed against safety in determining whether such a check is desirable. A good compiler should be able to produce code with such checks, should avoid redundant checks, and should allow programmers to control the extent and type of error checking in the compiled code.

Compilers for popular languages, such as C and C++, put hardly any error-checking operations into running code, so as to make things run as fast as possible. As a result, it falls to programmers to explicitly provide error checking. Unfortunately, people often neglect to do this, even in critical applications where speed is not a constraint. Their programs lead fast and dangerous lives. For example, the notorious "Worm" that paralyzed the Internet in 1988 exploited the UNIX™ operating system's failure to check whether the input buffer has overflowed in the finger daemon. (See Spafford 1989.)

57. Of course, with either the interpretation or the compilation strategy we must also implement for the new machine storage allocation, input and output, and all the various operations that we took as "primitive" in our discussion of the evaluator and compiler. One strategy for minimizing work here is to write as many of these operations as possible in JavaScript and then compile them for the new machine. Ultimately, everything reduces to a small kernel (such as garbage collection and the mechanism for applying actual machine primitives) that is hand-coded for the new machine.

the new machine to compile other JavaScript programs.[58] Or we can compile one of the interpreters of section 4.1 to produce an interpreter that runs on the new machine.

Exercise 5.48

By comparing the stack operations used by compiled code to the stack operations used by the evaluator for the same computation, we can determine the extent to which the compiler optimizes use of the stack, both in speed (reducing the total number of stack operations) and in space (reducing the maximum stack depth). Comparing this optimized stack use to the performance of a special-purpose machine for the same computation gives some indication of the quality of the compiler.

a. Exercise 5.28 asked you to determine, as a function of n, the number of pushes and the maximum stack depth needed by the evaluator to compute $n!$ using the recursive factorial function given above. Exercise 5.13 asked you to do the same measurements for the special-purpose factorial machine shown in figure 5.11. Now perform the same analysis using the compiled `factorial` function.

   Take the ratio of the number of pushes in the compiled version to the number of pushes in the interpreted version, and do the same for the maximum stack depth. Since the number of operations and the stack depth used to compute $n!$ are linear in n, these ratios should approach constants as n becomes large. What are these constants? Similarly, find the ratios of the stack usage in the special-purpose machine to the usage in the interpreted version.

   Compare the ratios for special-purpose versus interpreted code to the ratios for compiled versus interpreted code. You should find that the special-purpose machine is much more efficient than the compiled code, since the hand-tailored controller code should be much better than what is produced by our rudimentary general-purpose compiler.

b. Can you suggest improvements to the compiler that would help it generate code that would come closer in performance to the hand-tailored version?

Exercise 5.49

Carry out an analysis like the one in exercise 5.48 to determine the effectiveness of compiling the tree-recursive Fibonacci function

```
function fib(n) {
    return n < 2 ? n : fib(n - 1) + fib(n - 2);
}
```

compared to the effectiveness of using the special-purpose Fibonacci machine of figure 5.12. (For measurement of the interpreted performance, see exercise 5.30.) For Fibonacci, the time resource used is not linear in n; hence the ratios of stack operations will not approach a limiting value that is independent of n.

---

58. This strategy leads to amusing tests of correctness of the compiler, such as checking whether the compilation of a program on the new machine, using the compiled compiler, is identical with the compilation of the program on the original JavaScript system. Tracking down the source of differences is fun but often frustrating, because the results are extremely sensitive to minuscule details.

Exercise 5.50

This section described how to modify the explicit-control evaluator so that interpreted code can call compiled functions. Show how to modify the compiler so that compiled functions can call not only primitive functions and compiled functions, but interpreted functions as well. This requires modifying `compile_function_call` to handle the case of compound (interpreted) functions. Be sure to handle all the same `target` and `linkage` combinations as in `compile_fun_appl`. To do the actual function application, the code needs to jump to the evaluator's `compound_apply` entry point. This label cannot be directly referenced in object code (since the assembler requires that all labels referenced by the code it is assembling be defined there), so we will add a register called `compapp` to the evaluator machine to hold this entry point, and add an instruction to initialize it:

```
  assign("compapp", label("compound_apply")),
  branch(label("external_entry")),      // branches if flag is set
"read_evaluate_print_loop",
  ...
```

To test your code, start by declaring a function `f` that calls a function `g`. Use `compile_and_go` to compile the declaration of `f` and start the evaluator. Now, typing at the evaluator, declare `g` and try to call `f`.

Exercise 5.51

The `compile_and_go` interface implemented in this section is awkward, since the compiler can be called only once (when the evaluator machine is started). Augment the compiler–interpreter interface by providing a `compile_and_run` primitive that can be called from within the explicit-control evaluator as follows:

```
EC-evaluate input:
compile_and_run(parse(`
function factorial(n) {
    return n === 1
           ? 1
           : factorial(n - 1) * n;
}
                      `));

EC-evaluate value:
undefined

EC-evaluate input:
factorial(5)

EC-Eval value:
120
```

Exercise 5.52

As an alternative to using the explicit-control evaluator's read-evaluate-print loop, design a register machine that performs a read-compile-execute-print loop. That is, the machine should run a loop that reads a program, compiles it, assembles and executes the resulting code, and prints the result. This is easy to run in our simulated setup, since we can arrange to call the functions `compile` and `assemble` as "register-machine operations."

Exercise 5.53

Use the compiler to compile the metacircular evaluator of section 4.1 and run this program using the register-machine simulator. Because the parser takes a string as input, you will need to convert the program into a string. The simplest way to do this is to use the back quotes (`), as we have done for the example inputs to `compile_and_go` and `compile_and_run`. The resulting interpreter will run very slowly because of the multiple levels of interpretation, but getting all the details to work is an instructive exercise.

Exercise 5.54

Develop a rudimentary implementation of JavaScript in C (or some other low-level language of your choice) by translating the explicit-control evaluator of section 5.4 into C. In order to run this code you will need to also provide appropriate storage-allocation routines and other runtime support.

Exercise 5.55

As a counterpoint to exercise 5.54, modify the compiler so that it compiles JavaScript functions into sequences of C instructions. Compile the metacircular evaluator of section 4.1 to produce a JavaScript interpreter written in C.

References

Abelson, Harold, Andrew Berlin, Jacob Katzenelson, William McAllister, Guillermo Rozas, Gerald Jay Sussman, and Jack Wisdom. 1992. The Supercomputer Toolkit: A general framework for special-purpose computing. *International Journal of High-Speed Electronics* 3(3):337–361.

Allen, John. 1978. *Anatomy of Lisp.* New York: McGraw-Hill.

Appel, Andrew W. 1987. Garbage collection can be faster than stack allocation. *Information Processing Letters* 25(4):275–279.

Backus, John. 1978. Can programming be liberated from the von Neumann style? *Communications of the ACM* 21(8):613–641.

Baker, Henry G., Jr. 1978. List processing in real time on a serial computer. *Communications of the ACM* 21(4):280–293.

Batali, John, Neil Mayle, Howard Shrobe, Gerald Jay Sussman, and Daniel Weise. 1982. The Scheme-81 architecture—System and chip. In *Proceedings of the MIT Conference on Advanced Research in VLSI,* edited by Paul Penfield, Jr. Dedham, MA: Artech House.

Borning, Alan. 1977. ThingLab—An object-oriented system for building simulations using constraints. In *Proceedings of the 5th International Joint Conference on Artificial Intelligence.*

Borodin, Alan, and Ian Munro. 1975. *The Computational Complexity of Algebraic and Numeric Problems.* New York: American Elsevier.

Chaitin, Gregory J. 1975. Randomness and mathematical proof. *Scientific American* 232(5): 47–52.

Church, Alonzo. 1941. *The Calculi of Lambda-Conversion.* Princeton, N.J.: Princeton University Press.

Clark, Keith L. 1978. Negation as failure. In *Logic and Data Bases.* New York: Plenum Press, pp. 293–322.

Clinger, William. 1982. Nondeterministic call by need is neither lazy nor by name. In *Proceedings of the ACM Symposium on Lisp and Functional Programming,* pp. 226–234.

Colmerauer A., H. Kanoui, R. Pasero, and P. Roussel. 1973. Un système de communication homme-machine en français. Technical report, Groupe d'Intelligence Artificielle, Université d'Aix-Marseille II, Luminy.

Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2022. *Introduction to Algorithms.* 4th edition. Cambridge, MA: MIT Press.

Crockford, Douglas. 2008. *JavaScript: The Good Parts.* Sebastopol, CA: O'Reilly Media.

Darlington, John, Peter Henderson, and David Turner. 1982. *Functional Programming and Its Applications.* New York: Cambridge University Press.

Dijkstra, Edsger W. 1968a. The structure of the "THE" multiprogramming system. *Communications of the ACM* 11(5):341–346.

Dijkstra, Edsger W. 1968b. Cooperating sequential processes. In *Programming Languages*, edited by F. Genuys. New York: Academic Press, pp. 43–112.

Dinesman, Howard P. 1968. *Superior Mathematical Puzzles*. New York: Simon and Schuster.

de Kleer, Johan, Jon Doyle, Guy Steele, and Gerald J. Sussman. 1977. AMORD: Explicit control of reasoning. In *Proceedings of the ACM Symposium on Artificial Intelligence and Programming Languages,* pp. 116–125.

Doyle, Jon. 1979. A truth maintenance system. *Artificial Intelligence* 12:231–272.

ECMA. 1997. ECMAScript: A general purpose, cross-platform programming language. 1st edition, edited by Guy L. Steele Jr. *Ecma International.*

ECMA. 2015. ECMAScript: A general purpose, cross-platform programming language. 6th edition, edited by Allen Wirfs-Brock. *Ecma International.*

ECMA. 2020. ECMAScript: A general purpose, cross-platform programming language. 11th edition, edited by Jordan Harband. *Ecma International.*

Edwards, A. W. F. 2019. *Pascal's Arithmetical Triangle*. Mineola, New York: Dover Publications.

Feeley, Marc. 1986. Deux approches à l'implantation du language Scheme. Masters thesis, Université de Montréal.

Feeley, Marc and Guy Lapalme. 1987. Using closures for code generation. *Journal of Computer Languages* 12(1):47–66.

Feigenbaum, Edward, and Howard Shrobe. 1993. The Japanese National Fifth Generation Project: Introduction, survey, and evaluation. In *Future Generation Computer Systems,* vol. 9, pp. 105–117.

Feller, William. 1957. *An Introduction to Probability Theory and Its Applications,* volume 1. New York: John Wiley & Sons.

Fenichel, R., and J. Yochelson. 1969. A Lisp garbage collector for virtual memory computer systems. *Communications of the ACM* 12(11):611–612.

Floyd, Robert. 1967. Nondeterministic algorithms. *JACM,* 14(4):636–644.

Forbus, Kenneth D., and Johan de Kleer. 1993. *Building Problem Solvers.* Cambridge, MA: MIT Press.

Friedman, Daniel P., and David S. Wise. 1976. CONS should not evaluate its arguments. In *Automata, Languages, and Programming: Third International Colloquium,* edited by S. Michaelson and R. Milner, pp. 257–284.

Friedman, Daniel P., Mitchell Wand, and Christopher T. Haynes. 1992. *Essentials of Programming Languages.* Cambridge, MA: MIT Press/McGraw-Hill.

Gabriel, Richard P. 1988. The Why of *Y*. *Lisp Pointers* 2(2):15–25.

Goldberg, Adele, and David Robson. 1983. *Smalltalk-80: The Language and Its Implementation.* Reading, MA: Addison-Wesley.

Gordon, Michael, Robin Milner, and Christopher Wadsworth. 1979. *Edinburgh LCF.* Lecture Notes in Computer Science, volume 78. New York: Springer-Verlag.

Gray, Jim, and Andreas Reuter. 1993. *Transaction Processing: Concepts and Models.* San Mateo, CA: Morgan-Kaufman.

Green, Cordell. 1969. Application of theorem proving to problem solving. In *Proceedings of the International Joint Conference on Artificial Intelligence,* pp. 219–240.

Green, Cordell, and Bertram Raphael. 1968. The use of theorem-proving techniques in question-answering systems. In *Proceedings of the ACM National Conference,* pp. 169–181.

Guttag, John V. 1977. Abstract data types and the development of data structures. *Communications of the ACM* 20(6):397–404.

Hamming, Richard W. 1980. *Coding and Information Theory.* Englewood Cliffs, N.J.: Prentice-Hall.

Hanson, Christopher P. 1990. Efficient stack allocation for tail-recursive languages. In *Proceedings of ACM Conference on Lisp and Functional Programming,* pp. 106–118.

Hanson, Christopher P. 1991. A syntactic closures macro facility. *Lisp Pointers,* 4(4):9–16.

Hardy, Godfrey H. 1921. Srinivasa Ramanujan. *Proceedings of the London Mathematical Society* XIX(2).

Hardy, Godfrey H., and E. M. Wright. 1960. *An Introduction to the Theory of Numbers.* 4th edition. New York: Oxford University Press.

Havender, J. 1968. Avoiding deadlocks in multi-tasking systems. *IBM Systems Journal* 7(2):74–84.

Henderson, Peter. 1980. *Functional Programming: Application and Implementation.* Englewood Cliffs, N.J.: Prentice-Hall.

Henderson. Peter. 1982. Functional Geometry. In *Conference Record of the 1982 ACM Symposium on Lisp and Functional Programming,* pp. 179–187.

Hewitt, Carl E. 1969. PLANNER: A language for proving theorems in robots. In *Proceedings of the International Joint Conference on Artificial Intelligence,* pp. 295–301.

Hewitt, Carl E. 1977. Viewing control structures as patterns of passing messages. *Journal of Artificial Intelligence* 8(3):323–364.

Hoare, C. A. R. 1972. Proof of correctness of data representations. *Acta Informatica* 1(1):271–281.

Hodges, Andrew. 1983. *Alan Turing: The Enigma.* New York: Simon and Schuster.

Hofstadter, Douglas R. 1979. *Gödel, Escher, Bach: An Eternal Golden Braid.* New York: Basic Books.

Hughes, R. J. M. 1990. Why functional programming matters. In *Research Topics in Functional Programming*, edited by David Turner. Reading, MA: Addison-Wesley, pp. 17–42.

IEEE Std 1178-1990. 1990. *IEEE Standard for the Scheme Programming Language.*

Ingerman, Peter, Edgar Irons, Kirk Sattley, and Wallace Feurzeig; assisted by M. Lind, Herbert Kanner, and Robert Floyd. 1960. THUNKS: A way of compiling procedure statements, with some comments on procedure declarations. Unpublished manuscript. (Also, private communication from Wallace Feurzeig.)

Jaffar, Joxan, and Peter J. Stuckey. 1986. Semantics of infinite tree logic programming. *Theoretical Computer Science* 46:141–158.

Kaldewaij, Anne. 1990. *Programming: The Derivation of Algorithms.* New York: Prentice-Hall.

Knuth, Donald E. 1997a. *Fundamental Algorithms.* Volume 1 of *The Art of Computer Programming.* 3rd edition. Reading, MA: Addison-Wesley.

Knuth, Donald E. 1997b. *Seminumerical Algorithms.* Volume 2 of *The Art of Computer Programming.* 3rd edition. Reading, MA: Addison-Wesley.

Konopasek, Milos, and Sundaresan Jayaraman. 1984. *The TK!Solver Book: A Guide to Problem-Solving in Science, Engineering, Business, and Education.* Berkeley, CA: Osborne/McGraw-Hill.

Kowalski, Robert. 1973. Predicate logic as a programming language. Technical report 70, Department of Computational Logic, School of Artificial Intelligence, University of Edinburgh.

Kowalski, Robert. 1979. *Logic for Problem Solving.* New York: North-Holland.

Lamport, Leslie. 1978. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM* 21(7):558–565.

Lampson, Butler, J. J. Horning, R. London, J. G. Mitchell, and G. K. Popek. 1981. Report on the programming language Euclid. Technical report, Computer Systems Research Group, University of Toronto.

Landin, Peter. 1965. A correspondence between Algol 60 and Church's lambda notation: Part I. *Communications of the ACM* 8(2):89–101.

Lieberman, Henry, and Carl E. Hewitt. 1983. A real-time garbage collector based on the lifetimes of objects. *Communications of the ACM* 26(6):419–429.

Liskov, Barbara H., and Stephen N. Zilles. 1975. Specification techniques for data abstractions. *IEEE Transactions on Software Engineering* 1(1):7–19.

McAllester, David Allen. 1978. A three-valued truth-maintenance system. Memo 473, MIT Artificial Intelligence Laboratory.

McAllester, David Allen. 1980. An outlook on truth maintenance. Memo 551, MIT Artificial Intelligence Laboratory.

McCarthy, John. 1967. A basis for a mathematical theory of computation. In *Computer Programing and Formal Systems*, edited by P. Braffort and D. Hirschberg. North-Holland, pp. 33–70.

McDermott, Drew, and Gerald Jay Sussman. 1972. Conniver reference manual. Memo 259, MIT Artificial Intelligence Laboratory.

Miller, Gary L. 1976. Riemann's Hypothesis and tests for primality. *Journal of Computer and System Sciences* 13(3):300–317.

Miller, James S., and Guillermo J. Rozas. 1994. Garbage collection is fast, but a stack is faster. Memo 1462, MIT Artificial Intelligence Laboratory.

Moon, David. 1978. MacLisp reference manual, Version 0. Technical report, MIT Laboratory for Computer Science.

Morris, J. H., Eric Schmidt, and Philip Wadler. 1980. Experience with an applicative string processing language. In *Proceedings of the 7th Annual ACM SIGACT/SIGPLAN Symposium on the Principles of Programming Languages.*

O'Neill, Melissa E. 2009. The genuine sieve of Eratosthenes. *Journal of Functional Programming* 19(1):95–106.

Phillips, Hubert. 1934. *The Sphinx Problem Book*. London: Faber and Faber.

Phillips, Hubert. 1961. *My Best Puzzles in Logic and Reasoning*. New York: Dover Publications.

Rabin, Michael O. 1980. Probabilistic algorithm for testing primality. *Journal of Number Theory* 12:128–138.

Raymond, Eric. 1996. *The New Hacker's Dictionary.* 3rd edition. Cambridge, MA: MIT Press.

Raynal, Michel. 1986. *Algorithms for Mutual Exclusion.* Cambridge, MA: MIT Press.

Rees, Jonathan A., and Norman I. Adams IV. 1982. T: A dialect of Lisp or, lambda: The ultimate software tool. In *Conference Record of the 1982 ACM Symposium on Lisp and Functional Programming,* pp. 114–122.

Rivest, Ronald L., Adi Shamir, and Leonard M. Adleman. 1978. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM,* 21(2):120–126.

Robinson, J. A. 1965. A machine-oriented logic based on the resolution principle. *Journal of the ACM* 12(1):23.

Robinson, J. A. 1983. Logic programming—Past, present, and future. *New Generation Computing* 1:107–124.

Spafford, Eugene H. 1989. The Internet Worm: Crisis and aftermath. *Communications of the ACM* 32(6):678–688.

Steele, Guy Lewis, Jr. 1977. Debunking the "expensive procedure call" myth. In *Proceedings of the National Conference of the ACM,* pp. 153–162.

Steele, Guy Lewis, Jr., and Gerald Jay Sussman. 1975. Scheme: An interpreter for the extended lambda calculus. Memo 349, MIT Artificial Intelligence Laboratory.

Steele, Guy Lewis, Jr., Donald R. Woods, Raphael A. Finkel, Mark R. Crispin, Richard M. Stallman, and Geoffrey S. Goodfellow. 1983. *The Hacker's Dictionary.* New York: Harper & Row.

Stoy, Joseph E. 1977. *Denotational Semantics.* Cambridge, MA: MIT Press.

Sussman, Gerald Jay, and Richard M. Stallman. 1975. Heuristic techniques in computer-aided circuit analysis. *IEEE Transactions on Circuits and Systems* CAS-22(11):857–865.

Sussman, Gerald Jay, and Guy Lewis Steele Jr. 1980. Constraints—A language for expressing almost-hierarchical descriptions. *AI Journal* 14:1–39.

Sussman, Gerald Jay, and Jack Wisdom. 1992. Chaotic evolution of the solar system. *Science* 257:256–262.

Sussman, Gerald Jay, Terry Winograd, and Eugene Charniak. 1971. Microplanner reference manual. Memo 203A, MIT Artificial Intelligence Laboratory.

Sutherland, Ivan E. 1963. SKETCHPAD: A man-machine graphical communication system. Technical report 296, MIT Lincoln Laboratory.

Thatcher, James W., Eric G. Wagner, and Jesse B. Wright. 1978. Data type specification: Parameterization and the power of specification techniques. In *Conference Record of the Tenth Annual ACM Symposium on Theory of Computing*, pp. 119–132.

Turner, David. 1981. The future of applicative languages. In *Proceedings of the 3rd European Conference on Informatics,* Lecture Notes in Computer Science, volume 123. New York: Springer-Verlag, pp. 334–348.

Wand, Mitchell. 1980. Continuation-based program transformation strategies. *Journal of the ACM* 27(1):164–180.

Waters, Richard C. 1979. A method for analyzing loop programs. *IEEE Transactions on Software Engineering* 5(3):237–247.

Winston, Patrick. 1992. *Artificial Intelligence*. 3rd edition. Reading, MA: Addison-Wesley.

Zabih, Ramin, David McAllester, and David Chapman. 1987. Non-deterministic Lisp with dependency-directed backtracking. *AAAI-87*, pp. 59–64.

Zippel, Richard. 1979. Probabilistic algorithms for sparse polynomials. Ph.D. dissertation, Department of Electrical Engineering and Computer Science, MIT.

Zippel, Richard. 1993. *Effective Polynomial Computation.* Boston, MA: Kluwer Academic Publishers.

Index

Page numbers for JavaScript declarations are in *italics*.
Page numbers followed by *n* indicate footnotes.

"" (double quote), 124, 125 (ex. 2.55)
' (single quote), 126 (ex. 2.55)
` (back quote), 546
+
  as numeric addition operator, 4
  as string concatenation operator, 75
-
  as numeric negation operator, 16
  as numeric subtraction operator, 5
* (multiplication operator), 4
/ (division operator), 4
"% (remainder operator), 39
&& (logical conjunction), 15
  as derived component, 336 (ex. 4.4)
  evaluation of, 15
  implementing in metacircular evaluator, 336 (ex. 4.4)
  parsing of, 336 (ex. 4.4)
  why a syntactic form, 15
"||" (logical disjunction), 15
  as derived component, 336 (ex. 4.4)
  evaluation of, 15
  implementing in metacircular evaluator, 336 (ex. 4.4)
  parsing of, 336 (ex. 4.4)
  why a syntactic form, 15
? :, 14, *see also* conditional expression
"! (logical negation operator), 15
"!==
  as numeric comparison operator, 15
  as string comparison operator, 124
=, 192, *see also* assignment
===
  as equality of pointers, 228, 489
  as general comparison operator, 228
  as numeric equality operator, 15, 490 *n*
  as string comparison operator, 124, 490
> (numeric comparison operator), 15
>= (numeric comparison operator), 14
< (numeric comparison operator), 15
<= (numeric comparison operator), 15
=>, 54, *see also* lambda expression
↦ notation for mathematical function, 61 *n*
[,] (box notation for pairs), 86
;, *see* semicolon
. . . (rest parameter and spread syntax), 277
$, pattern variables starting with, 403
// (for comments in programs), 108 *n*
$\theta(f(n))$ (theta of $f(n)$), 36
λ calculus (lambda calculus), 55 *n*
π, *see* pi
Σ (sigma) notation, 50

abs, *13, 14*
absolute value, 13
abstract data, 72, *see also* data abstraction
abstraction, *see also* data abstraction; higher-order functions; means of abstraction
  common pattern and, 50
  functional, 22
  metalinguistic, 318
  in register-machine design, 456–457
  of search in nondeterministic programming, 378
abstraction barriers, 71, 76–78, 147
  in complex-number system, 148
  in generic arithmetic system, 164
  in query language, 437
  in representing JavaScript syntax, 329
abstract models for data, 78 *n*
abstract syntax
  in metacircular evaluator, 322
  in query interpreter, 425
accelerated_sequence, *298*
accumulate, 53 (ex. 1.32), *100*
  same as fold_right, 105 (ex. 2.38)
accumulate_n, 104 (ex. 2.36)
accumulator, 100, 196 (ex. 3.1)
Ackermann's function, 31 (ex. 1.10)
acquire a mutex, 277
actions, in register machine, 454–455
actual_value, *364*

Ada, 411 (ex. 4.61)
Adams, Norman I., IV, 356 *n*
`add` (generic), *165*
  used for polynomial coefficients, 179, 180
`add_action`, 245, *248*
`add_complex`, *150*
`add_complex_to_javascript_num`, *169*
`addend`, *128*
`adder` (primitive constraint), *257*
adder
  full, 244
  half, 242
  ripple-carry, 246 (ex. 3.30)
`add_interval`, *81*
additivity, 72, 147, 156–162, 166
`add_lists`, *371*
`add_poly`, *178*
`add_rat`, *73*
address, 488
address arithmetic, 488
`add_rule_or_assertion`, *434*
`add_streams`, *291*
`add_terms`, *179*
`add_to_agenda`, 249, *252*
`add_vect`, 118 (ex. 2.46)
`adjoin_arg`, *505 n*
adjoining to a list with `pair`, 88
`adjoin_set`, 131
  binary-tree representation, *136*
  ordered-list representation, 135 (ex. 2.61)
  unordered-list representation, *132*
  for weighted sets, *145*
`adjoin_term`, 179, *182*
Adleman, Leonard, 46 *n*
administrative assistant, importance of, 403
`advance_pc`, *479*
`after_delay`, 245, *249*
agenda, *see* digital-circuit simulation
A'h-mose, 40 *n*
algebra, symbolic, *see* symbolic algebra
algebraic expression, 176
  differentiating, 126–131
  representing, 128–131
  simplifying, 129–130
algebraic specification for data, 79 *n*
Algol
  block structure, 26
  call-by-name argument passing, 287 *n*, 363 *n*
  thunks, 287 *n*, 363 *n*
algorithm
  optimal, 104 *n*
  probabilistic, 45–46, 188 *n*
aliasing, 204 *n*
Al-Karaji, 36 *n*
Allen, John, 494 *n*
`all_regs` (compiler), *542 n*
alternative
  of conditional expression, 14
  of conditional statement, 57
`always_true`, *428*
`amb`, *374*
`amb` evaluator, *see* nondeterministic evaluator
`ambeval`, *388*
analog computer, 307 (fig. 3.34)
`analyze`
  metacircular, *356*
  nondeterministic, 387
`analyze_...`
  metacircular, 356–359, 360 (ex. 4.21)
  nondeterministic, 389–392
`analyze_amb`, *394*
analyzing evaluator, 355–360
  as basis for nondeterministic evaluator, 386
`and` (query language), 405
  evaluation of, 413, 426, 446 (ex. 4.73)
and-gate, 242
  `and_gate`, *246*
`an_element_of`, *375*
`angle`
  data-directed, *160*
  polar representation, *152*
  rectangular representation, *151*
  with tagged data, *154*
`angle_polar`, *154*
`angle_rectangular`, *153*
`an_integer_starting_from`, *375*
Appel, Andrew W., 541 *n*
`append`, *88*, *225* (ex. 3.12)
  as accumulation, 103 (ex. 2.33)
  `append_mutator` vs., 225 (ex. 3.12)
  as register machine, 492 (ex. 5.21)
  "what is" (rules) vs. "how to" (function), 399–400

`append_instruction_sequences`, 524, *544*
`append_mutator`, *225* (ex. 3.12)
  as register machine, 492 (ex. 5.21)
`append_to_form` (rules), *410*
applicative-order evaluation, 13
  in JavaScript, 13
  normal order vs., 17 (ex. 1.5), 43 (ex. 1.20), 361–362
`apply` (lazy), *364*
`apply` (metacircular), *324*
  tail recursion and, 324 *n*
`apply` (primitive method), 346 *n*
`apply_a_rule`, *430*
`apply_dispatch`, *507*
  modified for compiled code, *558*
`apply_generic`, *160*
  with coercion, *171*, 174 (ex. 2.81)
  with coercion by raising, 175 (ex. 2.84)
  with coercion of multiple arguments, 175 (ex. 2.82)
  with coercion to simplify, 176 (ex. 2.85)
  with message passing, *163*
  with tower of types, 173
`apply_in_underlying_javascript`, 159 *n*, *346 n*
`apply_primitive_function`, 324, 340, *346*
`apply_rules`, *430*
arbiter, 279 *n*
arctangent, 151 *n*
`arg_expressions`, 331
`argl` register, 500
argument(s), 9
  arbitrary number of, 277
  delayed, 307
argument passing, *see* call-by-name argument passing; call-by-need argument passing
Aristotle's *De caelo* (Buridan's commentary on), 279 *n*
arithmetic
  address arithmetic, 488
  generic, 163, *see also* generic arithmetic operations
  on complex numbers, 148
  on intervals, 81–84
  on polynomials, *see* polynomial arithmetic
  on power series, 295 (ex. 3.60), 296 (ex. 3.62)
  on rational numbers, 72–76
  operators for, 4
array, *see* vector (data structure)
arrow function, *see* lambda expression
`articles`, *381*
ASCII code, 140
`assemble`, *474*, *475 n*
assembler, 470, 473–477
`assert` (query interpreter), 419
assertion, 401
  implicit, 407
`assertion_body`, *443*
`assign` (in register machine), 453
  instruction constructor, *478*
  simulating, 478
  storing label in register, 459
assignment, 190–206
  assignment expression, 192
  assignment operation, 190
  benefits of, 197–200
  bugs associated with, 204 *n*, 205
  constant/variable declaration vs., 192 *n*
  costs of, 200–206
  equality test vs., 192 *n*
  evaluation of, 210
  parsing of, 333
  value of, 192 *n*
`assignment_symbol`, *333*
`assignment_value_expression`, 333
`assign_reg_name`, *478*
`assign_symbol_value`, 341, *342*
`assign_value_exp`, *478*
`assoc`, *237*
associativity
  of conditional expression, 14
  of operators, 5
atomic operations supported in hardware, 279 *n*
atomic requirement for test_and_set, 278
`attach_tag`, *152*
  using JavaScript data types, 168 (ex. 2.78)
`augend`, *128*
automagically, 376
automatic search, 373, *see also* search
  history of, 376 *n*
automatic storage allocation, 487
`average`, *20*

`average_damp`, *63*
average damping, 61
`averager` (constraint), 262 (ex. 3.33)

back quotes, 546
backtracking, 376, *see also* nondeterministic computing
Backus, John, 315 *n*
Baker, Henry G., Jr., 494 *n*
balanced binary tree, 137, *see also* binary tree
balanced mobile, 96 (ex. 2.29)
bank account, 190, 220 (ex. 3.11)
  exchanging balances, 273
  joint, 203, 205 (ex. 3.7)
  joint, modeled with streams, 316 (fig. 3.38)
  joint, with concurrent access, 265
  password-protected, 196 (ex. 3.3)
  serialized, 270
  stream model, 314
  transferring money, 275 (ex. 3.44)
barrier synchronization, 280 *n*
Barth, John, 317
Batali, John Dean, 499 *n*
`below`, 112, 122 (ex. 2.51)
Bertrand's Hypothesis, 293 *n*
`beside`, 112, *122*
Bhaskara, 36 *n*
bignum, 490
binary numbers, addition of, *see* adder
binary operator, 16
binary search, 135
binary tree, 135
  balanced, 137
  converting a list to a, 137 (ex. 2.64)
  converting to a list, 137 (ex. 2.63)
  for Huffman encoding, 141
  represented with lists, 136
  set represented as, 135–138
  table structured as, 241 (ex. 3.26)
bind, 24
binding, 206
  deep, 343 *n*
`binding_in_frame`, *444*
`binding_value`, *444*
`binding_variable`, *444*
binomial coefficients, 36 *n*
black box, 22
block, 25
  empty, 252 *n*
  parsing of, 333
`block_body`, 333
blocked process, 278 *n*
block structure, 25–26, 351–355
  in environment model, 218–222
  in query language, 447 (ex. 4.76)
body of a function, 9
boolean values (true, false), 14
Borning, Alan, 253 *n*
Borodin, Alan, 104 *n*
bound name, 24
box-and-pointer notation, 84
  end-of-list marker, 85
box notation for pairs, 86
`branch` (in register machine), 452
  instruction constructor, *480*
  simulating, 479
`branch_dest`, *480*
branch of a tree, 7
`break` (keyword), 338 (ex. 4.7), *see also* while loop
breakpoint, 486 (ex. 5.18)
broken heart, 495
B-tree, 137 *n*
bug, 1
  capturing a free name, 24
  order of assignments, 205
  side effect with aliasing, 204 *n*
bureaucracy, 419
Buridan, Jean, 279 *n*
busy-waiting, 278 *n*

C
  compiling JavaScript into, 564 (ex. 5.55)
  error handling, 561 *n*
  JavaScript interpreter written in, 564 (ex. 5.54), 564 (ex. 5.55)
  recursive functions in, 30
cache-coherence protocols, 266 *n*
calculator, fixed points with, 61 *n*
call-by-name argument passing, 287 *n*, 363 *n*
call-by-need argument passing, 287 *n*, 363 *n*
`call_each`, *248*
camel case, 10 *n*
canonical form, for polynomials, 184
capturing a free name, 24

Carmichael numbers, 45 *n*, 48 (ex. 1.27)
case analysis, 13, *see also* conditional expression
  data-directed programming vs., 323
  general, 14
  as sequence of clauses, 15
cell, in serializer implementation, 277
`celsius_fahrenheit_converter`, *255*
  expression-oriented, *263* (ex. 3.37)
`center`, *83*
Chaitin, Gregory, 197 *n*
Chandah-sutra, 40 *n*
change and sameness
  meaning of, 202–204
  shared data and, 227
changing money, *see* counting change
Chapman, David, 376 *n*
character, ASCII encoding, 140
Charniak, Eugene, 376 *n*
Chebyshev, Pafnutii L'vovich, 293 *n*
`check_an_assertion`, *428*
chess, eight-queens puzzle, 108 (ex. 2.42), 380 (ex. 4.42)
chip implementation of Scheme, 499
chronological backtracking, 376
Church, Alonzo, 55 *n*, 80 (ex. 2.6)
Church numerals, 80 (ex. 2.6)
Church–Turing thesis, 349 *n*
circuit
  digital, *see* digital-circuit simulation
  modeled with streams, 304 (ex. 3.73), 309 (ex. 3.80)
Clark, Keith L., 422 *n*
clause of a case analysis, 15
Clinger, William, 363 *n*
closed world assumption, 422
closure, 71
  in abstract algebra, 85 *n*
  closure property of picture-language operations, 110, 113
  closure property of `pair`, 84
coal, bituminous, 112 *n*
code
  ASCII, 140
  fixed-length, 140
  Huffman, *see* Huffman code
  Morse, 140
  prefix, 140
  variable-length, 140
code generator, 523
  arguments of, 523
  value of, 524
`coeff`, 179, *182*
coercion, 170–176
  function, 170
  in algebraic manipulation, 184
  in polynomial arithmetic, 180
  table, 170
Colmerauer, Alain, 399 *n*
combination, 4, *see also* function application; operator combination
combination, means of, 3, *see also* closure
comments in programs, 108 *n*
`comp` register, 500
compacting garbage collector, 494 *n*
compilation, *see* compiler
`compile`, *523*
`compile_and_go`, 558, *560*
`compile_and_run`, 563 (ex. 5.51)
`compile_application`, *536*
`compile_assignment`, *529*
`compile_block`, *532*
`compile_conditional`, *530*
`compiled_apply`, *558*
`compile_declaration`, *529*
`compiled_function_entry`, *532 n*
`compiled_function_env`, *532 n*
`compile_fun_appl`, *542*
`compile_function_call`, *538*
`compile_lambda_body`, *534*
`compile_lambda_expression`, *533*
`compile_linkage`, *527*
`compile_literal`, *528*
`compile_name`, *528*
compiler, 519–521
  interpreter vs., 520–521, 561
  tail recursion, stack allocation, and garbage-collection, 541 *n*
`compile_return_statement`, *543*
compiler for JavaScript, 521–564, *see also* code generator; compile-time environment; instruction sequence; linkage descriptor; target register
  analyzing evaluator vs., 521, 522
  assignments, 528
  blocks, 531
  code generators, *see* `compile_...`
  combinations, 535–542
  conditionals, 529
  dead code analysis, 534 (ex. 5.34)

declarations, 528
efficiency, 521–522
example compilation, 546–549
explicit-control evaluator vs., 521–522, 526 (ex. 5.33), 560
expression-syntax functions, 522
function applications, 535–542
interfacing to evaluator, 557–564
label generation, 530 *n*
lambda expressions, 532
lexical addressing, 554–555
linkage code, 527
literals, 528
machine-operation use, 520 *n*
monitoring performance (stack use) of compiled code, 560, 562 (ex. 5.48), 562 (ex. 5.49)
names, 528
open coding of primitives, 553 (ex. 5.41)
order of argument evaluation, 549 (ex. 5.39)
register use, 520 *n*, 521, 542 *n*
return statements, 543
running compiled code, 557–564
scanning out internal declarations, 555
sequences of statements, 531
stack usage, 525, 526 (ex. 5.32), 553 (ex. 5.40)
structure of, 522–526
tail-recursive code generated by, 541
`compile_sequence`, *531*
compile-time environment, 555, 556 (ex. 5.43), 556 (ex. 5.44), 557 (ex. 5.47)
`complex` package, 167
complex-number arithmetic, 148
interfaced to generic arithmetic system, 167
structure of system, 155 (fig. 2.21)
complex numbers
polar representation, 151
rectangular representation, 151
rectangular vs. polar form, 149
represented as tagged data, 152–155
`complex_to_complex`, *175* (ex. 2.81)
composition of functions, 67 (ex. 1.42)
`compound_apply`, *508*
compound data, need for, 69–71
compound expression, 4, *see also* function application; operator combination; syntactic form
as function expression of application, 17 (ex. 1.4)
compound function, 8, *see also* function
used like primitive function, 10
compound query, 405–406
processing, 413–415, 426–428, 445 (ex. 4.72), 446 (ex. 4.73), 446 (ex. 4.74)
computability, 349 *n*, 351 *n*
computational process, 1, *see also* process
computer science, 318, 349 *n*
mathematics vs., 18, 398
concatenating strings, 75
`conclusion`, *444*
concrete data representation, 72
concurrency, 263–281
correctness of concurrent programs, 267–269
deadlock, 279–280
functional programming and, 315
mechanisms for controlling, 269–281
`concurrent_execute`, 270
`conditional_alternative`, 331
`conditional_consequent`, 331
conditional expression, 14, *see also* case analysis
as alternative of conditional expression, 14
evaluation of, 14
non-boolean value as predicate, 14 *n*
normal-order evaluation of, 18 (ex. 1.5)
as operand of operator combination, 17 (ex. 1.1)
parsing of, 331
precedence of, 17 (ex. 1.1)
right-associativity of, 14
why a syntactic form, 20 (ex. 1.6)
`conditional_predicate`, 331
conditional statement, 57
alternative statements of, 57
conditional instead of alternative block, 179
consequent statements of, 57
need for, 57
one-armed (without alternative), 252 *n*
parsing of, 331
predicate, consequent, and alternative of, 57

congruent modulo *n*, 44
`conjoin`, *426*
conjunction, 15, *see also* `&&` (logical conjunction)
`connect`, 256, *261*
connector(s), in constraint system, 254
  operations on, 256
  representing, 259
Conniver, 376 *n*
consciousness, expansion of, 325 *n*
consequent
  of clause, 15
  of conditional expression, 14
  of conditional statement, 57
`const` (keyword), 5, *see also* constant declaration
`constant` (in register machine), 453, *482*
  simulating, 482
  syntax of, 468
`constant` (primitive constraint), *259*
constant (in JavaScript), 5
  detecting assignment to, 343 (ex. 4.11), 557 (ex. 5.46)
  value of, 5
constant declaration, 5
  parsing of, 333
  why a syntactic form, 8
`constant_exp_value`, *482*
constraint(s)
  primitive, 254
  propagation of, 253–263
constraint network, 254
`construct_arglist`, *537*
constructing a list with `pair`, 88
constructor, 72
  as abstraction barrier, 76
`contents`, *152*, *443*
  using JavaScript data types, 168 (ex. 2.78)
continuation
  in nondeterministic evaluator, 386–388, *see also* failure continuation; success continuation
  in register-machine simulator, 475 *n*
`continue` (keyword), 338 (ex. 4.7), *see also* while loop
`continue` register, 459
  in explicit-control evaluator, 500
  recursion and, 464
continued fraction, 62 (ex. 1.37)
  e as, 62 (ex. 1.38)
  golden ratio as, 62 (ex. 1.37)
  tangent as, 63 (ex. 1.39)
controller for register machine, 450–452, *see also* register-machine language
  controller diagram, 451
control structure, 419
conventional interface, 71
  sequence as, 98–110
`convert`, *440*
`convert_to_query_syntax`, *438*
Cormen, Thomas H., 137 *n*
`corner_split`, *115*
correctness of a program, 18 *n*
cosine
  fixed point of, 61
  power series for, 295 (ex. 3.59)
cosmic radiation, 45 *n*
`count_change`, *35*
counting change, 34–35, 89 (ex. 2.19)
`count_leaves`, 93, *94*
  as accumulation, 104 (ex. 2.35)
  as register machine, 492 (ex. 5.20)
`count_pairs`, *229* (ex. 3.16)
credit-card accounts, international, 281 *n*
Cressey, David, 495 *n*
Crockford, Douglas, 339 *n*
cross-type operations, 169
cryptography, 46 *n*
CSE machine, 222
`cube`, *38* (ex. 1.15), *48*, *65*
`cube_root`, *64*
cube root
  as fixed point, 64
  by Newton's method, 21 (ex. 1.8)
`current_time`, 249, *251*
current time, for simulation agenda, 251
Curry, Haskell Brooks, 90 (ex. 2.20)
currying, 90 (ex. 2.20)
cycle in list, 226 (ex. 3.13)
  detecting, 229 (ex. 3.18)

Darlington, John, 315 *n*
data, 1, 3
  abstract, 72, *see also* data abstraction
  abstract models for, 78 *n*
  algebraic specification for, 79 *n*
  compound, 69–71
  concrete representation of, 72
  functional representation of, 78–80

hierarchical, 85, 92–96
list-structured, 74
meaning of, 78–80
mutable, *see* mutable data objects
numerical, 3
as program, 348–350
shared, 227–230
symbolic, 124
tagged, 152–155, 489 *n*
data abstraction, 70, 72, 147, 149, 329, *see also* metacircular evaluator
for queue, 232
data base
data-directed programming and, 162 (ex. 2.74)
Gargle personnel, 401–403
indexing, 413 *n*, 434
Insatiable Enterprises personnel, 161 (ex. 2.74)
logic programming and, 401
as set of records, 139
data-directed programming, 148, 156–162
case analysis vs., 323
in metacircular evaluator, 336 (ex. 4.3)
in query interpreter, 425
data-directed recursion, 180
data paths for register machine, 450–452
data-path diagram, 450
data types
in JavaScript, 168 (ex. 2.78)
in statically typed languages, 311 *n*
deadlock, 279–280
avoidance, 279
recovery, 280 *n*
debug, 1
declaration, *see also* internal declaration
of constant (`const`), 5
environment model of, 208
of function (`function`), 8
use of name before, 56 *n*, 222
of variable (`let`), 191
`declaration_symbol`, *333*
`declaration_value_expression`, *333*
declarative vs. imperative knowledge, 18, 398
logic programming and, 399–400, 419
nondeterministic computing and, 373 *n*
`decode`, *144*
decomposition of program into parts, 22
deep binding, 343 *n*
`deep_reverse`, 95 (ex. 2.27)
deferred operations, 29
definite integral, 51–52
estimated with Monte Carlo simulation, 200 (ex. 3.5), 313 (ex. 3.82)
de Kleer, Johan, 376 *n*, 421 *n*
delay, in digital circuit, 242
delayed argument, 307
delayed evaluation, 190, 282
assignment and, 289 (ex. 3.52)
explicit vs. automatic, 372
in lazy evaluator, 360–372
normal-order evaluation and, 311
printing and, 288 *n*
streams and, 306–310
delayed expression, 284
explicit, 307
explicit vs. automatic, 372
lazy evaluation and, 370
memoized, 287
`delay_it`, *366*
`delete_queue`, 232, *234*
`denom`, 72, *74*
axiom for, 78
reducing to lowest terms, *77*
dense polynomial, 181
dependency-directed backtracking, 376 *n*
`depends_on`, *433*
`deposit` message for bank account, *194*
`deposit`, with external serializer, *275*
depth-first search, 376
deque, 235 (ex. 3.23)
`deriv` (numerical), *65*
`deriv` (symbolic), *127*
data-directed, *161* (ex. 2.73)
derivative of a function, 64
derived component, 335
adding to explicit-control evaluator, 513 (ex. 5.25)
derived components in evaluator, 334–335
function declaration, 334
operator combination, 335
design, stratified, 123
differential equation, 307, *see also* `solve`
second-order, 309 (ex. 3.78), 309 (ex. 3.79)
differentiation
numerical, 64
rules for, 126, 130 (ex. 2.56)
symbolic, 126–131, 160 (ex. 2.73)

diffusion, simulation of, 268
digital-circuit simulation, 242–253
  agenda, 248–249
  agenda implementation, 251–253
  primitive function boxes, 245–246
  representing wires, 247–248
  sample simulation, 249–250
digital signal, 242
Dijkstra, Edsger Wybe, 277 *n*
Dinesman, Howard P., 378
Diophantus's *Arithmetic*, Fermat's copy of, 44 *n*
Dirichlet, Peter Gustav Lejeune, 198 *n*
`dirichlet_stream`, *313*
`dirichlet_test`, *198*
`disjoin`, *427*
  without delayed expression, *444* (ex. 4.68)
disjunction, 15, *see also* `"|"|` (logical disjunction)
dispatching
  comparing different styles, 163 (ex. 2.76)
  on type, 156, *see also* data-directed programming
`display` (primitive function), 46 (ex. 1.22), 75 *n*
`display` operation in register machine, 455
`display_stream`, *285*
`distinct`, *379 n*
`div` (generic), *165*
`div_complex`, *150*
`divides`, *43*
`div_interval`, *82*
  division by zero, 82 (ex. 2.10)
`div_poly`, 183 (ex. 2.91)
`div_rat`, *73*
`div_series`, 296 (ex. 3.62)
`div_terms`, *183* (ex. 2.91)
dog, perfectly rational behavior of, 279 *n*
`dot_product`, *105* (ex. 2.37)
Doyle, Jon, 376 *n*
`draw_line`, 119
`driver_loop`
  for lazy evaluator, *365*
  for metacircular evaluator, *347*
  for nondeterministic evaluator, *395*
driver loop
  in explicit-control evaluator, 514
  in lazy evaluator, 365
  in metacircular evaluator, 346
  in nondeterministic evaluator, 377, 394
  in query interpreter, 418, 424
duplicate parameters, 336 (ex. 4.5)

e
  as continued fraction, 62 (ex. 1.38)
  as solution to differential equation, 308
e^x, power series for, 295 (ex. 3.59)
`eceval`, *515*
ECMAScript, 2
  `Math` object, 10 *n*
`edge1_frame`, 117
`edge2_frame`, 117
Edwards, Anthony William Fairbank, 36 *n*
efficiency, *see also* order of growth
  of compilation, 521
  of data-base access, 413 *n*
  of evaluation, 355
  of query processing, 414
  of tree-recursive process, 35
Eich, Brendan, 2
EIEIO, 280 *n*
eight-queens puzzle, 108 (ex. 2.42), 380 (ex. 4.42)
electrical circuits, modeled with streams, 304 (ex. 3.73), 309 (ex. 3.80)
`element_expressions`, *442*
`else` (keyword), 57, *see also* conditional statement
embedded language, language design using, 360
`empty_arglist`, *505 n*
empty list, 87, *see also* **`null`**
  recognizing with `is_null`, 87
empty stream, 284
encapsulated name, 193 *n*
`enclosing_environment`, *341*
enclosing environment, 206
`encode`, *145* (ex. 2.68)
end-of-list marker, 85
`end_segment`, 77 (ex. 2.2), 120 (ex. 2.48)
`end_with_linkage`, *527*
engineering vs. mathematics, 45 *n*
`entry`, *136*
`enumerate_interval`, *101*
`enumerate_tree`, *101*
enumerator, 99
`env` register, 500

environment, 6, 206
  compile-time, *see* compile-time environment
  as context for evaluation, 7
  enclosing, 206
  global, *see* global environment
  lexical scoping and, 25 *n*
  program, *see* program environment
  in query interpreter, 447 (ex. 4.76)
  renaming vs., 447 (ex. 4.76)
environment model of evaluation, 190, 206–222
  environment structure, 207 (fig. 3.1)
  function application, 207
  function-application example, 210–213
  internal declarations, 218–222
  local state, 213–218
  message passing, 220 (ex. 3.11)
  metacircular evaluator and, 320
  rules for evaluation, 207–210
  tail recursion and, 213 *n*
`equal`, 125 (ex. 2.54)
equality
  in generic arithmetic system, 169 (ex. 2.79)
  of lists, 125 (ex. 2.54)
  of numbers, 15, 125 (ex. 2.54), 490 *n*
  referential transparency and, 203
  of strings, 124, 125 (ex. 2.54), 490
`equal_rat`, *73*
equation, solving, *see* half-interval method; Newton's method; `solve`
Eratosthenes, 290 *n*
`error` (primitive function), 60 *n*
  optional second argument, 79 *n*
error handling
  in compiled code, 561 *n*
  in explicit-control evaluator, 515, 518 (ex. 5.31)
Escher, Maurits Cornelis, 110 *n*
`estimate_integral`, 200 (ex. 3.5)
`estimate_pi`, *198, 199*
Euclid's Algorithm, 41–42, 450, *see also* greatest common divisor
  order of growth, 42
  for polynomials, 185 *n*
Euclid's *Elements*, 42 *n*
Euclid's proof of infinite number of primes, 293 *n*
Euclidean ring, 185 *n*
Euler, Leonhard, 62 (ex. 1.38)
  proof of Fermat's Little Theorem, 44 *n*
  series accelerator, 298
`euler_transform`, *298*
`eval` (primitive function in JavaScript), 350
`eval_assignment`, *327*
`eval_block`, *326*
`eval_conditional` (lazy), *365*
`eval_conditional` (metacircular), *325*
`eval_declaration`, *328*
`eval_dispatch`, *501*
`eval_return_statement`, *327*
`eval_sequence`, *326*
`evaluate` (lazy), 363
`evaluate` (metacircular), *323*
  analyzing version, *356*
  data-directed, 336 (ex. 4.3)
`evaluate_query`, 418, *426*
evaluation
  applicative-order, *see* applicative-order evaluation
  delayed, *see* delayed evaluation
  environment model of, *see* environment model of evaluation
  models of, 513
  normal-order, *see* normal-order evaluation
  of `&&`, 15
  of `"|"|`, 15
  of conditional expression, 14
  of function application, 9
  of operator combination, 6–7
  of primitive expression, 7
  order of subexpression evaluation, *see* order of evaluation
  substitution model of, *see* substitution model of function application
evaluator, 318, *see also* interpreter
  as abstract machine, 349
  metacircular, 320
  as universal machine, 349
evaluators, *see* metacircular evaluator; analyzing evaluator; lazy evaluator; nondeterministic evaluator; query interpreter; explicit-control evaluator
`ev_application`, *505*
`ev_assignment`, *512*
`ev_block`, *512*
`ev_conditional`, *502*

`ev_declaration`, *513*
`even_fibs`, *99*, *102*
event-driven simulation, 242
`ev_function_declaration`, *513*
`ev_lambda`, *501*
evlis tail recursion, 506 *n*
`ev_literal`, *501*
`ev_name`, *501*
`ev_operator_combination`, *505*
`ev_return`, *509*
`ev_sequence`, *503*
`exchange`, *273*
`execute`, 473
`execute_application`
  metacircular, *357*
  nondeterministic, *393*
execution function
  in analyzing evaluator, 356
  in nondeterministic evaluator, 386, 388
  in register-machine simulator, 473, 477–484
explicit-control evaluator for JavaScript, 499–519
  argument evaluation, 505–507
  assignments, 512
  blocks, 512
  combinations, 504–509
  compound functions, 508
  conditionals, 502
  controller, 500–515
  data paths, 500
  declarations, 512
  derived components, 513 (ex. 5.25)
  driver loop, 514
  error handling, 515, 518 (ex. 5.31)
  expressions with no subexpressions to evaluate, 501
  function application, 504–509
  as machine-language program, 520
  machine model, 515
  modified for compiled code, 557–559
  monitoring performance (stack use), 516–518
  normal-order evaluation, 513 (ex. 5.26)
  operations, 500
  optimizations (additional), 526 (ex. 5.33)
  primitive functions, 507
  registers, 500
  return statements, 508–511
  running, 513–516
  sequences of statements, 502–504
  stack usage, 504
  syntactic forms (additional), 513 (ex. 5.25)
  tail recursion, 511 (ex. 5.22), 509–511, 517 (ex. 5.27), 518 (ex. 5.29)
  as universal machine, 519
`expmod`, *44*, *47* (ex. 1.25), *47* (ex. 1.26)
exponential growth, 37
  of tree-recursive Fibonacci-number computation, 33
exponentiation, 38–40
  modulo *n*, 44
expression, 3, *see also* compound expression; primitive expression
  algebraic, *see* algebraic expressions
  literal, 322
  primitive boolean, 14
  symbolic, 71, *see also* string(s); symbol(s)
expression-oriented vs. imperative programming style, 263 *n*
expression statement, 3
  parsing of, 331
`expt`
  linear iterative version, *38*
  linear recursive version, *38*
  register machine for, 466 (ex. 5.4)
`extend`, *444*
`extend_environment`, 341, *341*
`extend_if_consistent`, *429*
`extend_if_possible`, *433*
`external_entry`, *559*
`extract_labels`, *475*, *475 n*

`factorial`
  as an abstract machine, 349
  compilation of, 546–549
  environment structure in evaluating, 212 (ex. 3.9)
  linear iterative version, *28*
  linear recursive version, *27*
  register machine for (iterative), 452 (ex. 5.1), 454 (ex. 5.2)
  register machine for (recursive), 462–464, 465 (fig. 5.11)
  stack usage, compiled, 562 (ex. 5.48)
  stack usage, interpreted, 517 (ex. 5.27), 517 (ex. 5.28)

stack usage, register machine, 486 (ex. 5.13)
with assignment, *205*
with higher-order functions, 52 (ex. 1.31)
with while loop, *338* (ex. 4.7)
factorial, 27, *see also* `factorial`
without declaration or assignment, 353 (ex. 4.18)
infinite stream, 293 (ex. 3.54)
failure, in nondeterministic computation, 374
bug vs., 389
searching and, 376
failure continuation (nondeterministic evaluator), 386, 388
constructed by `amb`, 394
constructed by assignment, 390
constructed by driver loop, 394
`false` (keyword), 14
falsiness, 339 *n*
`fast_expt`, *39*
`fast_is_prime`, *45*
feedback loop, modeled with streams, 306
Feeley, Marc, 356 *n*
Feigenbaum, Edward, 400 *n*
Fenichel, Robert, 494 *n*
Fermat, Pierre de, 44 *n*
Fermat's Little Theorem, 44
alternate form, 48 (ex. 1.28)
proof, 44 *n*
`fermat_test`, *45*
Fermat test for primality, 44–45
variant of, 48 (ex. 1.28)
`fetch_assertions`, *434*
`fetch_rules`, *434*
`fib`
linear iterative version, *33*
logarithmic version, *40* (ex. 1.19)
register machine for (tree-recursive), 466, 467 (fig. 5.12)
stack usage, compiled, 562 (ex. 5.49)
stack usage, interpreted, 518 (ex. 5.30)
tree-recursive version, *32*, *518* (ex. 5.30)
with memoization, *241* (ex. 3.27)
Fibonacci numbers, 32, *see also* `fib`
Euclid's GCD algorithm and, 42
infinite stream of, *see* `fibs`
`fibs` (infinite stream), *290*
implicit definition, *292*
FIFO buffer, 231
`filter`, *100*
filter, 53 (ex. 1.33), 100
`filtered_accumulate`, 53 (ex. 1.33)
`find_assertions`, *428*
`find_divisor`, *43*
`first_agenda_item`, 249, *253*
first-class elements in language, 67
`first_conjunct`, *443*
`first_disjunct`, *443*
`first_frame`, *341*
`first_segment`, *251*
`first_statement`, *332*
`first_term`, 179, *182*
fixed-length code, 140
`fixed_point`, *60*
as iterative improvement, 68 (ex. 1.46)
fixed point, 60–61
computing with calculator, 61 *n*
of cosine, 61
cube root as, 64
fourth root as, 68 (ex. 1.45)
golden ratio as, 62 (ex. 1.35)
as iterative improvement, 68 (ex. 1.46)
in Newton's method, 64
*n*th root as, 68 (ex. 1.45)
square root as, 61, 63, 66
of transformed function, 66
unification and, 432 *n*
`fixed_point_of_transform`, *66*
`flag` register, 471
`flatmap`, *107*
`flatten_stream`, *436*
`flip_horiz`, 112, 122 (ex. 2.50)
`flipped_pairs`, *113*, *116 n*, *117*
`flip_vert`, 112, *121*
Floyd, Robert, 376 *n*
`fold_left`, *106* (ex. 2.38)
`fold_right`, 105 (ex. 2.38)
Forbus, Kenneth D., 376 *n*
`force_it`, *366*
memoized version, *367*
forcing
tail of stream, 284
of thunk, 363
`for_each`, 92 (ex. 2.23), *369* (ex. 4.28)
`for_each_except`, *261*
`forget_value`, 256, *261*
formal parameters, *see* parameters

Fortran, 103 *n*
  inventor of, 315 *n*
forwarding address, 495
fourth root, as fixed point, 68 (ex. 1.45)
fraction, *see* rational number(s)
frame (environment model), 206
  as repository of local state, 213–218
  global, 206
frame (picture language), 110, 117
  coordinate map, 117
frame (query interpreter), 412, *see also* pattern matching; unification
  representation, 444
`frame_coord_map`, *118*
framed-stack discipline, 504 *n*
`frame_symbols`, *341*
`frame_values`, *341*
`free` register, 491, 495
free list, 491 *n*
free name, 24
  capturing, 24
  in internal declaration, 25
Friedman, Daniel P., 287 *n*, 319 *n*
`fringe`, 95 (ex. 2.28)
  as a tree enumeration, 101 *n*
`front_ptr`, *233*
`front_queue`, 232, *233*
full-adder, 244
  `full_adder`, *244*
`fun` register, 500
`function` (keyword), 8, *see also* function declaration
function (JavaScript), 3, 8
  anonymous, 54
  as argument, 49–53
  as black box, 22–23
  body of, 9
  compound, 8
  creating with function declaration, 8
  creating with lambda expression, 54, 208, 210
  declaration of, 8–9
  first-class, 67
  as general method, 58–63
  generic, 143, 148
  higher-order, *see* higher-order function
  mathematical function vs., 18
  memoized, 241 (ex. 3.27)
  monitored, 196 (ex. 3.2)
  name of, 9
  naming (with function declaration), 8
  parameters of, 9
  as pattern for local evolution of a process, 26
  primitive, 10
  as returned value, 63–68
  returning multiple values, 475 *n*
  scope of parameters, 24
  syntactic form vs., 362 (ex. 4.24)
  with any number of arguments, 277
function (mathematical)
  $\mapsto$ notation for, 61 *n*
  Ackermann's, 31 (ex. 1.10)
  composition of, 67 (ex. 1.42)
  derivative of, 64
  fixed point of, 60–61
  JavaScript function vs., 18
  rational, 184–188
  repeated application of, 68 (ex. 1.43)
  smoothing of, 68 (ex. 1.44)
functional abstraction, 22
functional programming, 200, 312–316
  concurrency and, 315
  functional programming languages, 315
  time and, 314–316
functional representation of data, 78–80
  mutable data, 230–231
function application
  compound expression as function expression of, 17 (ex. 1.4)
  environment model of, 207, 210–213
  evaluation of, 9
  as function expression of application, 63 *n*
  parsing of, 331
  substitution model of, *see* substitution model of function application
`function_body`, *340*
function box, in digital circuit, 242
function declaration, 8
  as derived component, 334
  hoisting of, 54 *n*, 352 (ex. 4.17)
  lambda expression vs., 54
  parsing of, 334
`function_declaration_body`, 334
`function_declaration_name`, 334
`function_declaration_parameters`, 334
`function_decl_to_constant_decl`, *334*

function_environment, *340*
function_expression, 331
function expression, 9
  application as, 63 *n*
  compound expression as, 17 (ex. 1.4)
  lambda expression as, 55
function_parameters, *340*

Gabriel, Richard P., 353 *n*
garbage collection, 493–498
  memoization and, 366 *n*
  mutation and, 223 *n*
  tail recursion and, 541 *n*
garbage collector
  compacting, 494 *n*
  mark-sweep, 494 *n*
  stop-and-copy, 493–498
Gargle, 378, 401
GCD, *see* greatest common divisor
gcd, *42*
  register machine for, 450–452, 469
gcd_machine, *469*
gcd_terms, *186*
general-purpose computer, as universal machine, 520
generate_huffman_tree, *146* (ex. 2.69)
generating sentences, 385 (ex. 4.47)
generic arithmetic operations, 164–169
  structure of system, 164 (fig. 2.23)
generic function, 143, 148
  generic selector, 154, 155
generic operation, 71
generic types, *see* polymorphic types
Genesis, 411 (ex. 4.61)
get, 157, *240*
get_contents, *470*
get_current_environment, *514*
get_register, *473*
get_register_contents, 468, *473*
get_signal, 245, *248*
get_time (primitive function), 46 (ex. 1.22)
get_value, 256, *261*
global environment, 207
  in metacircular evaluator, 344
global frame, 206
Goguen, Joseph, 78 *n*
golden ratio, 33
  as continued fraction, 62 (ex. 1.37)
  as fixed point, 62 (ex. 1.35)
good parts of JavaScript, 339 *n*
Gordon, Michael, 311 *n*
go_to (in register machine), 452
  destination in register, 459
  instruction constructor, *480*
  simulating, 480
go_to_dest, *480*
grammar, 381
graphics, *see* picture language
Gray, Jim, 280 *n*
greatest common divisor, 41–42, *see also* gcd
  generic, 186 (ex. 2.94)
  of polynomials, 185
  used to estimate π, 198
  used in rational-number arithmetic, 75
Green, Cordell, 399 *n*
Guttag, John Vogel, 79 *n*

half-adder, 242
  half_adder, *244*
  simulation of, 249–250
half-interval method, 58–60
  half_interval_method, 60
  Newton's method vs., 64 *n*
halting problem, 350 (ex. 4.15)
Halting Theorem, 351 *n*
Hamming, Richard Wesley, 142 *n*, 293 (ex. 3.56)
Hanson, Christopher P., 541 *n*
Hardy, Godfrey Harold, 293 *n*, 303 *n*
Haskell, 90 (ex. 2.20), 311 *n*
Hassle, 362 *n*
has_value, 256, *261*
Havender, J., 280 *n*
Haynes, Christopher T., 319 *n*
head (primitive function), 73
  axiom for, 79
  functional implementation of, *79*, *80* (ex. 2.4), *230*, *371*
  implemented with vectors, 490
  as list operation, 86
headed list, 236, 251 *n*
Henderson, Peter, 110 *n*, 291 *n*, 315 *n*
  Henderson diagram, 291
Henz, Martin, children of, 124
Heraclitus, 189
Heron of Alexandria, 19 *n*
Hewitt, Carl Eddie, 30 *n*, 376 *n*, 399 *n*, 494 *n*

hiding principle, 193 *n*
hierarchical data structures, 85, 92–96
hierarchy of types, 172–176
  inadequacy of, 173
  in symbolic algebra, 184
higher-order functions, 49
  function as argument, 49–53
  function as general method, 58–63
  function as returned value, 63–68
  in metacircular evaluator, 325 *n*
  static typing and, 311 *n*
high-level language, machine language vs., 318
Hilfinger, Paul, 138 *n*
Hoare, Charles Antony Richard, 78 *n*
Hodges, Andrew, 349 *n*
Hofstadter, Douglas R., 350 *n*
hoisting of function declarations, 54 *n*, 352 (ex. 4.17)
Horner, W. G., 104 *n*
Horner's rule, 103 (ex. 2.34)
"how to" vs. "what is" description, *see* imperative vs. declarative knowledge
Huffman, David, 141
Huffman code, 140–147
  optimality of, 142
  order of growth of encoding, 147 (ex. 2.72)
Hughes, R. J. M., 371 *n*

`identity`, *50*
`if` (keyword), 57, *see also* conditional statement
`imag_part`
  data-directed, *160*
  polar representation, *152*
  rectangular representation, *151*
  with tagged data, *154*
`imag_part_polar`, *154*
`imag_part_rectangular`, *153*
immediately invoked lambda expression, 217 (ex. 3.10)
imperative programming, 204
imperative vs. declarative knowledge, 18, 398
  logic programming and, 399–400, 419
  nondeterministic computing and, 373 *n*
imperative vs. expression-oriented programming style, 263 *n*
`inc`, *50*
incremental development of programs, 6
indentation, 335 (ex. 4.2)
indeterminate of a polynomial, 177
indexing a data base, 413 *n*, 434
inference, method of, 419
infinite series, 432 *n*
infinite stream(s), 289–296
  merging, 293 (ex. 3.56), 301, 303 (ex. 3.70), 316
  merging as a relation, 316 *n*
  of factorials, 293 (ex. 3.54)
  of Fibonacci numbers, *see* `fibs`
  of integers, *see* `integers`
  of pairs, 300–303
  of prime numbers, *see* `primes`
  of random numbers, 313
  representing power series, 295 (ex. 3.59)
  to model signals, 303–306
  to sum a series, 297
infix notation, 4
  prefix notation vs., 131 (ex. 2.58)
infix operator, 4
`inform_about_no_value`, *257*
`inform_about_value`, *257*
information retrieval, *see* data base
Ingerman, Peter, 363 *n*
`initialize_stack` operation in register machine, 471, 484
`insert`
  in one-dimensional table, *237*
  in two-dimensional table, *239*
`insert_queue`, 232, *234*
`install_complex_package`, *167*
`install_javascript_number_package`, *165*
`install_polar_package`, *159*
`install_polynomial_package`, *178*
`install_rational_package`, *166*
`install_rectangular_package`, *158*
instantiate a pattern, 404
`instantiate_expression`, 425, *439*
`instantiate_term`, *440*
`inst_controller_instruction`, *476*
`inst_execution_fun`, *476*
instruction counting, 486 (ex. 5.14)
instruction execution function, 473
`instructions`, *543*
instruction sequence, 524–526, 543–546
instruction tracing, 486 (ex. 5.15)

integer(s), 3 *n*
integerizing factor, 187
`integers` (infinite stream), *289*
  implicit definition, *291*
  lazy-list version, *371*
`integers_starting_from`, *289*
`integral`, *51*, *304*, *308* (ex. 3.77)
  with delayed argument, *308*
  with lambda expression, *54*
  lazy-list version, *372*
  need for delayed evaluation, 306
integral, *see also* definite integral; Monte Carlo integration
  of a power series, 295 (ex. 3.59)
integrated-circuit implementation of Scheme, 499
`integrate_series`, 295 (ex. 3.59)
integrator, for signals, 304
`interleave`, *302*
`interleave_delayed`, *435*
internal declaration, 25–26
  in environment model, 218–222
  free name in, 25
  names distinct from parameters, 336 (ex. 4.5)
  in nondeterministic evaluator, 391 *n*
  position of, 26 *n*
  restrictions on, 354
  scanning out, 352
  scope of name, 351–355
Internet "Worm", 561 *n*
interning strings, 490
interpreter, 2, *see also* evaluator
  compiler vs., 520–521, 561
  read-evaluate-print loop, 5
`intersection_set`, 131
  binary-tree representation, 138 (ex. 2.65)
  ordered-list representation, *134*
  unordered-list representation, *132*
interval arithmetic, 81–84
invariant quantity of an iterative process, 40 (ex. 1.16)
inverter, 242
  `inverter`, *245*
`is_`, in predicate names, 20 *n*
`is_amb`, *387*
`is_application`, 331
`is_assertion`, *443*
`is_assignment`, 333
`is_block`, 333
`is_boolean`, 339 *n*
`is_compiled_function`, *532 n*
`is_compound_function`, *340*
`is_conditional`, 331
`is_constant_exp`, *482*
`is_declaration`, *334*
`is_divisible`, *289*
`is_element_of_set`, 131
  binary-tree representation, *136*
  ordered-list representation, *134*
  unordered-list representation, *132*
`is_empty_agenda`, 249, *251*
`is_empty_conjunction`, *443*
`is_empty_disjunction`, *443*
`is_empty_queue`, 232, *233*
`is_empty_sequence`, *332*
`is_empty_termlist`, 179, *182*
`is_equal` (generic predicate), 169 (ex. 2.79)
`is_equal_to_zero` (generic), 169 (ex. 2.80)
  for polynomials, 182 (ex. 2.87)
`is_even`, *39*
`is_falsy`, *340*
  full JavaScript version, *339 n*
  why used in explicit-control evaluator, *502 n*
`is_function_declaration`, 334
`is_label_exp`, *482*
`is_lambda_expression`, 332
`is_last_argument_expression`, *505 n*
`is_last_statement`, *332*
`is_leaf`, *143*
`is_list_construction`, *442*
`is_literal`, *330*
`is_name`, 331
`is_null` (primitive function), 87
  implemented with typed pointers, 491
`is_number` (primitive function), 127
  data types and, 168 (ex. 2.78)
  implemented with typed pointers, 491
`is_operation_exp`, *483*
`is_pair` (primitive function), 94
  implemented with typed pointers, 491
`is_polar`, *153*
`is_prime`, *43*, *292*
`is_primitive_function`, 340, *345*
`is_product`, *128*
`is_rectangular`, *153*

`is_register_exp`, *482*
`is_return_statement`, 333
`is_return_value`, *340*
`is_rule`, *444*
`is_same_variable`, *128*, 178
`is_sequence`, 332
`is_string` (primitive function), 128
  data types and, 168 (ex. 2.78)
  implemented with typed pointers, 491
`is_sum`, *128*
`is_tagged_list`, *330*
`is_truthy`, 325, *340*
  full JavaScript version, *339 n*
`is_undefined` (primitive function), 157
`is_variable`
  for algebraic expressions, *128*
  in query system, *437*
iteration contructs, *see* looping constructs
iterative improvement, 68 (ex. 1.46)
iterative process, 29
  as a stream process, 296–300
  design of algorithm, 40 (ex. 1.16)
  implemented by function call, 20, 30, 511, *see also* tail recursion
  linear, 29, 37
  recursive process vs., 27–30, 212 (ex. 3.9), 462, 549 (ex. 5.37)
  register machine for, 462

Jaffar, Joxan, 432 *n*
Java, recursive functions in, 30
JavaScript
  applicative-order evaluation in, 13
  `eval` in, *350*
  first-class functions in, 67
  good parts, 339 *n*
  history of, 2
  internal type system, 168 (ex. 2.78)
  tail recursion in, 30 *n*
JavaScript environment used in this book, 10
`javascript_number` package, 165
`javascript_number_to_complex`, *170*
`javascript_number_to_javascript_number`, *175* (ex. 2.81)
JavaScript package `sicp`, 10 *n*
`javascript_predicate` (query interpreter), *427*
`javascript_predicate` (query language), 406, 422
  evaluation of, 415, 427, 446 (ex. 4.74)
`javascript_predicate_expression`, *443*
Jayaraman, Sundaresan, 253 *n*

Kaldewaij, Anne, 41 *n*
Karr, Alphonse, 189
Kepler, Johannes, 449
`key`, 139
key of a record
  in a data base, 139
  in a table, 236
  testing equality of, 240 (ex. 3.24)
keyword, 8
keywords
  `break`, 338 (ex. 4.7)
  `const`, 5
  `continue`, 338 (ex. 4.7)
  `else`, 57
  `false`, 14
  `function`, 8
  `if`, 57
  `let`, 191
  `null`, 85
  `return`, 9
  `true`, 14
  `while`, 337 (ex. 4.7)
Knuth, Donald E., 40 *n*, 42 *n*, 104 *n*, 197, 198 *n*
Kolmogorov, A. N., 197 *n*
Konopasek, Milos, 253 *n*
Kowalski, Robert, 399 *n*
KRC, 106 *n*, 301 *n*

`label` (in register machine), 452, *482*
  simulating, 482
`label_exp_label`, *482*
Lagrange interpolation formula, 177 *n*
λ calculus (lambda calculus), 55 *n*
`lambda_body`, 332
lambda expression, 54
  block as body of, 116 *n*
  as function expression of application, 55
  function declaration vs., 54
  immediately invoked, 217 (ex. 3.10)
  lazy evaluation and, 370
  parsing of, 332
  precedence of, 55
  value of, 210

lambda_parameter_symbols, *332*
Lambert, J.H., 63 (ex. 1.39)
Lamé, Gabriel, 42 *n*
Lamé's Theorem, 42
Lamport, Leslie, 281 *n*
Lampson, Butler, 204 *n*
Landin, Peter, 15 *n*, 287 *n*
language, *see* parsing natural language; programming language
Lapalme, Guy, 356 *n*
last_pair, 89 (ex. 2.17), *225* (ex. 3.12)
  rules, 411 (ex. 4.60)
lazy evaluation, 361
lazy evaluator, 360–370
lazy list, 370–372
lazy pair, 370–372
lazy tree, 371 *n*
least commitment, principle of, 152
left-associative, 5
left_branch, *136*, *143*
Leibniz, Baron Gottfried Wilhelm von
  proof of Fermat's Little Theorem, 44 *n*
  series for π, 49 *n*, 297
Leiserson, Charles E., 137 *n*, 303 *n*
length, 87
  as accumulation, 103 (ex. 2.33)
  iterative version, *88*
  recursive version, *88*
let (keyword), 191, *see also* variable, declaration
let* (Scheme variant of let), 336 (ex. 4.6)
lexical_address_assign, 555, 556 (ex. 5.42)
lexical addressing, 554–555
  lexical address, 555
lexical_address_lookup, 555, 556 (ex. 5.42)
lexical scoping, 25, *see also* scope of a name
  environment structure and, 554
Lieberman, Henry, 494 *n*
LIFO buffer, *see* stack
linear growth, 29, 37
linear iterative process, 29
  order of growth, 37
linear recursive process, 29
  order of growth, 37
line segment
  represented as pair of points, 77 (ex. 2.2)
  represented as pair of vectors, 120 (ex. 2.48)
linkage descriptor, 523
Liskov, Barbara Huberman, 79 *n*
Lisp, *see also* Scheme
  as ancestor of JavaScript, 2
  on DEC PDP-1, 494 *n*
  efficiency of, 5 *n*
  MDL dialect of, 495 *n*
list (primitive function), 86
list(s), 86, *see also* list structure
  adjoining to with pair, 88
  combining with append, 88
  constructing with pair, 88
  converting a binary tree to a, 137 (ex. 2.63)
  converting to a binary tree, 137 (ex. 2.64)
  empty, *see* empty list
  equality of, 125 (ex. 2.54)
  headed, 236, 251 *n*
  last pair of, 89 (ex. 2.17)
  lazy, 370–372
  length of, 87
  list structure vs., 86 *n*
  manipulation with head, tail, and pair, 86
  mapping over, 90–92
  *n*th element of, 87
  operations on, 87–90
  printed representation of, 86
  reversing, 89 (ex. 2.18)
  techniques for manipulating, 87–90
  walking down with tail, 87
list_difference, *544*
list notation for data, 87
list_of_arg_values, *364*
list_of_delayed_args, *364*
list_of_values, *325*
  without higher-order functions, *325 n*
list_ref, *87*, *371*
list structure, 74, *see also* list(s)
  list vs., 86 *n*
  mutable, 223–227
  represented using vectors, 488–492
list-structured memory, 487–498
list_to_tree, *138* (ex. 2.64)
list_union, *544*

literal expression, 322
  parsing of, 330
`literal_value`, *330*
`lives_near` (rule), *407*, 409 (ex. 4.58)
local evolution of a process, 27
local name, 23–24, 55–56
local state, 190–206
  maintained in frames, 213–218
local state variable, 190–197
location, 488
Locke, John, 1
logarithm, approximating ln 2, 300 (ex. 3.65)
logarithmic growth, 37, 39, 135 *n*
logical and (digital logic), 242
logical conjunction, 15, *see also* `&&` (logical conjunction)
logical disjunction, 15, *see also* `"|"|` (logical disjunction)
`logical_not`, *245*
logical or (digital logic), 242
logic programming, 398–400, *see also* query interpreter; query language
  computers for, 400 *n*
  history of, 398 *n*, 400 *n*
  in Japan, 400 *n*
  logic programming languages, 400
  mathematical logic vs., 419–424
logic puzzles, 378–380
`lookup`
  in one-dimensional table, *237*
  in set of records, *139*
  in two-dimensional table, *238*
`lookup_label`, *476*
`lookup_prim`, *483*
`lookup_symbol_value`, 341, *342*
  for scanned-out declarations, 344 (ex. 4.12)
looping constructs, 20, 30, *see also* while loop
`lower_bound`, 82 (ex. 2.7)

machine language, 520
  high-level language vs., 318
magician, *see* numerical analyst
`magnitude`
  data-directed, *160*
  polar representation, *152*
  rectangular representation, *151*
  with tagged data, *154*
`magnitude_polar`, *154*
`magnitude_rectangular`, *153*
`make_account`, *195*
  in environment model, 220 (ex. 3.11)
  with serialization, *271*, *272* (ex. 3.41), *273* (ex. 3.42)
`make_account_and_serializer`, *274*
`make_accumulator`, 196 (ex. 3.1)
`make_agenda`, 249, *251*
`make_application`, *331*
`make_assign_ef`, *478*
`make_binding`, *444*
`make_branch_ef`, *480*
`make_center_percent`, 83 (ex. 2.12)
`make_center_width`, *83*
`make_code_tree`, *143*
`make_compiled_function`, *532 n*
`make_complex_from_mag_ang`, *167*
`make_complex_from_real_imag`, *167*
`make_connector`, *260*
`make_constant_declaration`, *334*
`make_cycle`, *226* (ex. 3.13)
`make_decrementer`, *201*
`make_execution_function`, *477*
`make_frame`, 117, *119* (ex. 2.47), *341*
`make_from_mag_ang`, *155*, *160*
  message-passing, 163 (ex. 2.75)
  polar representation, *152*
  rectangular representation, *151*
`make_from_mag_ang_polar`, *154*
`make_from_mag_ang_rectangular`, *153*
`make_from_real_imag`, *155*, *160*
  message-passing, *162*
  polar representation, *152*
  rectangular representation, *151*
`make_from_real_imag_polar`, *154*
`make_from_real_imag_rectangular`, *153*
`make_function`, *340*
`make_go_to_ef`, *480*
`make_inst`, *476*
`make_instruction_sequence`, *526*
`make_interval`, 81, *82* (ex. 2.7)
`make_javascript_number`, *165*
`make_joint`, 205 (ex. 3.7)
`make_label`, *530 n*
`make_label_entry`, *476*
`make_lambda_expression`, *332*
`make_leaf`, *143*
`make_leaf_set`, *145*

make_literal, *330*
make_machine, 468, *470*
make_monitored, 196 (ex. 3.2)
make_mutex, *278*
make_name, *331*
make_new_machine, 472 (fig. 5.13)
make_new_variable, *438*
make_operation_exp_ef, *482*
make_perform_ef, *481*
make_point, 77 (ex. 2.2)
make_poly, *178*
make_polynomial, *182*
make_primitive_exp_ef, *482*
make_product, *128*, *130*
make_queue, 232, *233*
make_rat, 72, *74*, *77*
  axiom for, 78
  reducing to lowest terms, *75*
make_rational, *166*
make_register, *470*
make_restore_ef, *481*
make_return_value, *340*
make_save_ef, *481*
make_segment, 77 (ex. 2.2), 120 (ex. 2.48)
make_serializer, *277*
make_simplified_withdraw, *201*, *314*
make_stack, *471*
  with monitored stack, *485*
make_sum, *128, 129*
make_table
  message-passing implementation, *240*
  one-dimensional table, *237*
make_tableau, *298*
make_term, 179, *182*
make_test_ef, *479*
make_time_segment, *251*
make_tree, *136*
make_vect, 118 (ex. 2.46)
make_wire, 243, *247*, 250 (ex. 3.31)
make_withdraw, *194*
  in environment model, 213–218
  using immediately invoked lambda expression, *217* (ex. 3.10)
making change, *see* counting change
map, *91*, *371*
  as accumulation, 103 (ex. 2.33)
mapping
  over lists, 90–92
  nested, 106–110, 300–303
  as a transducer, 100
  over trees, 96–98
map_successive_pairs, *313*
mark-sweep garbage collector, 494 *n*
math_atan2 (primitive function), 151 *n*
math_cos (primitive function), 61
mathematical function, *see* function (mathematical)
mathematics
  computer science vs., 18, 398
  engineering vs., 45 *n*
math_floor (primitive function), 45
math_log (primitive function), 10, 62 (ex. 1.36)
math_max (primitive function), 81
math_min (primitive function), 81
math_random (primitive function), 44
  assignment needed for, 191 *n*
math_round (primitive function), 176 *n*
math_sin (primitive function), 61
math_trunc (primitive function), 294 (ex. 3.58)
matrix, represented as sequence, 105 (ex. 2.37)
matrix_times_matrix, 105 (ex. 2.37)
matrix_times_vector, 105 (ex. 2.37)
McAllester, David Allen, 376 *n*
McCarthy, John, 374 *n*
McDermott, Drew, 376 *n*
MDL, 495 *n*
means of abstraction, 3
  constant declaration as, 6
means of combination, 3, *see also* closure
measure in a Euclidean ring, 185 *n*
member, *125*
  extended to use pointer equality, 261 *n*
memo, *287*
memo_fib, *241* (ex. 3.27)
memoization, 35 *n*, 241 (ex. 3.27)
  garbage collection and, 366 *n*
  in stream tail, 287
  of thunks, 363
memoize, *242* (ex. 3.27)
memory
  in 1965, 375 *n*
  list-structured, 487–498
merge, *294* (ex. 3.56)
merge_weighted, 303 (ex. 3.70)
merging infinite streams, *see* infinite stream(s)

message passing, 80, 162–164
  environment model and, 220 (ex. 3.11)
  in bank account, 195
  in digital-circuit simulation, 247
  tail recursion and, 30 *n*
metacircular evaluator, 320
metacircular evaluator for JavaScript, 319–351
  `&&` (logical conjunction), 336 (ex. 4.4)
  `"|"|` (logical disjunction), 336 (ex. 4.4)
  analyzing version, 355–360
  compilation of, 564 (ex. 5.53), 564 (ex. 5.55)
  component representation, 322, 328–335
  data abstraction in, 320, 322, 342
  data-directed `evaluate`, 336 (ex. 4.3)
  derived components, 334–335
  driver loop, 346
  efficiency of, 355
  environment model of evaluation in, 320
  environment operations, 341
  `evaluate` and `apply`, 321–328
  `evaluate`–`apply` cycle, 320, 321 (fig. 4.1)
  global environment, 344
  higher-order functions in, 325 *n*
  implemented language
    vs. implementation language, 325
  job of, 320 *n*
  `let*` (Scheme variant of `let`), 336 (ex. 4.6)
  order of argument evaluation, 328 (ex. 4.1)
  parameters distinct from local names, 336 (ex. 4.5)
  preventing duplicate parameters, 336 (ex. 4.5)
  primitive functions, 344–346
  representation of environments, 341–343
  representation of functions, 340
  representation of true and false, 339
  return value, 324
  running, 344–348
  symbolic differentiation and, 329
  syntactic forms (additional), 336 (ex. 4.4), 336 (ex. 4.6), 337 (ex. 4.7)
  syntactic forms as derived components, 334–335
  syntax of evaluated language, 328–335
  tail recursion and, 324 *n*, 327 *n*, 509
  `undefined`, 344
  value of program at top level, 338 (ex. 4.8)
  while loop, 337 (ex. 4.7)
metalinguistic abstraction, 318
MicroPlanner, 376 *n*
`midpoint_segment`, 77 (ex. 2.2)
Miller, Gary L., 48 (ex. 1.28)
Miller, James S., 541 *n*
Miller–Rabin test for primality, 48 (ex. 1.28)
Milner, Robin, 311 *n*
Minsky, Marvin Lee, xxiv, 494 *n*
Miranda, 106 *n*
MIT, 399 *n*
  early history of, 112 *n*
  Research Laboratory of Electronics, 494 *n*
ML, 311 *n*
mobile, 96 (ex. 2.29)
Mocha, 2
modeling
  as a design strategy, 189
  in science and engineering, 12
models of evaluation, 513
modified registers, *see* instruction sequence
`modifies_register`, *543*
modularity, 102, 189
  along object boundaries, 316 *n*
  functional programs vs. objects, 312–316
  hiding principle, 193 *n*
  streams and, 296
  through dispatching on type, 156
  through infinite streams, 313
  through modeling with objects, 197
modulo *n*, 44
modus ponens, 419 *n*
money, changing, *see* counting change
monitored function, 196 (ex. 3.2)
`monte_carlo`, *198*
  infinite stream, *313*
Monte Carlo integration, 200 (ex. 3.5)
  stream formulation, 313 (ex. 3.82)
Monte Carlo simulation, 198

stream formulation, 312
Moon, David A., 494 *n*
Morris, J. H., 204 *n*
Morse code, 140
Mouse, Minnie and Mickey, 420
`mul` (generic), *165*
used for polynomial coefficients, 180
`mul_complex`, *150*
`mul_interval`, *81*
more efficient version, 82 (ex. 2.11)
`mul_poly`, *178*
`mul_rat`, *73*
`mul_series`, 295 (ex. 3.60)
`mul_streams`, 293 (ex. 3.54)
`mul_terms`, *180*
Multics time-sharing system, 494 *n*
`multiplicand`, *129*
multiplication by Russian peasant method, 40 *n*
`multiplier`
primitive constraint, *258*
selector, *129*
Munro, Ian, 104 *n*
mutable data objects, 222–231, *see also* queue; table
functional representation of, 230–231
implemented with assignment, 230–231
list structure, 223–227
pairs, 223–227
shared data, 228
mutator, 222
mutex, 277
mutual exclusion, 277 *n*
mutual recursion, 221
`mystery`, *226* (ex. 3.14)

name, *see* constant; local name; variable
bound, 24
encapsulated, 193 *n*
free, 24
of a function, 9
of a parameter, 24
parsing of, 330
scope of, 24, *see also* scope of a name
unbound, 206
value of, 206
naming
of computational objects, 5
of functions, 8
naming conventions
$ for pattern variables, 403
`is_` for predicates, 20 *n*
snake case, 10 *n*
NaN, not a typo, 339 *n*
native language of machine, 520
natural language parsing, *see* parsing natural language
needed registers, *see* instruction sequence
`needs_register`, *543*
`negate`, *427*
`negated_query`, *443*
negation
logical (`"!`), 15
numeric (`-`), 16
negation as failure, 422 *n*
nested declaration(s), *see* internal declaration
nested mapping(s), *see* mapping
nested operator combinations, 4
Netscape Communications Corporation, 2
Netscape Navigator, 2
`new` register, 496
`new_heads` register, 494
`new_tails` register, 494
Newton's method
for cube roots, 21 (ex. 1.8)
for differentiable functions, 64–65
half-interval method vs., 64 *n*
for square roots, 19–20, 65, 66
`newtons_method`, *65*
`newton_transform`, *65*
`new_withdraw`, *193*
`next` (linkage descriptor), 523
`next_to_in` (rules), *410* (ex. 4.59)
node of a tree, 7
noncomputable, 351 *n*
nondeterminism, in behavior of concurrent programs, 268 *n*, 316 *n*
nondeterministic choice point, 375
nondeterministic computing, 373–385
nondeterministic evaluator, 386–398
order of argument evaluation, 385 (ex. 4.44)
nondeterministic programming vs. JavaScript programming, 373, 380 (ex. 4.39), 380 (ex. 4.42)
nondeterministic programs
logic puzzles, 378–380
pairs with prime sums, 373
parsing natural language, 381–384

Pythagorean triples, 377 (ex. 4.33), 378 (ex. 4.34), 378 (ex. 4.35)
non-strict, 362
normal-order evaluation, 13
applicative order vs., 17 (ex. 1.5), 43 (ex. 1.20), 361–362
of conditional expressions, 18 (ex. 1.5)
delayed evaluation and, 311
in explicit-control evaluator, 513 (ex. 5.26)
normal-order evaluator, *see* lazy evaluator
`not` (query language), 406, 421
evaluation of, 415, 427, 446 (ex. 4.74)
notation in this book
box notation for data, 86
italic symbols in expression syntax, 9 *n*
list notation for data, 87
slanted characters for interpreter response, 4 *n*
`nouns`, *381*
*n*th root, as fixed point, 68 (ex. 1.45)
`null` (keyword)
as empty list, 87
as end-of-list marker, 85
recognizing with `is_null`, 87
number(s)
bignum, 490
comparison of, 14
equality of, 14, 125 (ex. 2.54), 490 *n*
in generic arithmetic system, 165
integer vs. real number, 3 *n*
in JavaScript, 3
`number_equal`, *130*
number theory, 44 *n*
`numer`, 72, *74*
axiom for, 78
reducing to lowest terms, *77*
numerical analysis, 3 *n*
numerical analyst, 59 *n*
numerical data, 3

object(s), 189
benefits of modeling with, 197
with time-varying state, 190
object-oriented programming languages, 174 *n*
object program, 520
Ocaml, 90 (ex. 2.20)
`office_move`, *379*
`old` register, 496
`oldht` register, 497
`ones` (infinite stream), *291*
lazy-list version, *371*
`op` (in register machine), 453
simulating, 482, *483*
open coding of primitives, 553 (ex. 5.41)
operands of a combination, 4
operation
cross-type, 169
generic, 71
in register machine, 450–452
operation-and-type table, 157
assignment needed for, 191 *n*
implementing, 240
`operation_exp_op`, *483*
`operation_exp_operands`, *483*
operator combination, 4
as function application, 335
as derived component, 335
evaluation of, 6–7
parsing of, 334
as a tree, 7
`operator_combination_to_application`, *335*
operator of a combination, 4
operator precedence, *see* precedence
operators (ECMAScript may allow additional operand type combinations)
`+` (for numeric addition), 4
`+` (for string concatenation), 75
`-` (numeric negation operator), 16
`-` (numeric subtraction operator), 5
`*` (multiplication), 4
`/` (division), 4
`"%` (remainder), 39
`"!` (logical negation), 15
`"!==` (for numeric comparison), 15
`"!==` (for string comparison), 124
`===` (for nonprimitive values), 228
`===` (for numeric comparison), 15
`===` (for string comparison), 124
`<` (numeric comparison), 15
`<=` (numeric comparison), 15
`>` (numeric comparison), 15
`>=` (numeric comparison), 14
optimality
of Horner's rule, 104 *n*
of Huffman code, 142
`or` (query language), 405

evaluation of, 414, 427
Oracle Corporation, 2
order, 179, *182*
ordered-list representation of sets, 133–135
order notation, 36
order of evaluation
assignment and, 206 (ex. 3.8)
in compiler, 549 (ex. 5.39)
in explicit-control evaluator, 506
in metacircular evaluator, 507 *n*
in JavaScript, 206 (ex. 3.8), 207 *n*
in metacircular evaluator, 328 (ex. 4.1)
order of events
decoupling apparent from actual, 287
indeterminacy in concurrent systems, 265
order of growth, 36–37
linear iterative process, 37
linear recursive process, 37
logarithmic, 39
tree-recursive process, 37
order of subexpression evaluation, *see* order of evaluation
ordinary numbers (in generic arithmetic system), 165
or-gate, 242
or_gate, 246 (ex. 3.28), 246 (ex. 3.29)
origin_frame, 117
Ostrowski, A. M., 104 *n*
outranked_by (rule), *408*, 422 (ex. 4.62)
overloaded operator +, 75

P operation on semaphore, 277 *n*
package, 157
complex-number, 167
JavaScript-number, 165
polar representation, 159
polynomial, 178
rational-number, 166
rectangular representation, 157
painter(s), 110
higher-order operations, 116
operations, 112
represented as functions, 119
transforming and combining, 120
pair (primitive function), 73
axiom for, 79
closure property of, 84
functional implementation of, *79*, *80* (ex. 2.4), *230*, *371*
implemented with mutators, 225, *225*
implemented with vectors, 491
as list operation, 86
pair(s), 73
axiomatic definition of, 79
box-and-pointer notation for, 84
box notation for, 86
functional representation of, 79–80, 230–231, 370
infinite stream of, 300–303
lazy, 370–372
mutable, 223–227
represented using vectors, 488–492
used to represent sequence, 85
used to represent tree, 92–96
pairs, *302*
Pan, V. Y., 104 *n*
parallel_instruction_sequences, *546*
parameter passing, *see* call-by-name argument passing; call-by-need argument passing
parameters, 9
distinct from local names, 336 (ex. 4.5)
duplicate, 336 (ex. 4.5)
names of, 24
scope of, 24
as variables, 192, 194
parentheses
in function declaration, 9
to group operator combinations, 4
around lambda expression, 55
around parameters of lambda expression, 54
around predicate of conditional statement, 57
parse, 328, *see also* parsing JavaScript
in query interpreter, 425
parse_..., 381–383
parsing JavaScript, 328–329
&& (logical conjunction), 336 (ex. 4.4)
"||" (logical disjunction), 336 (ex. 4.4)
assignment, 333
block, 333
conditional expression, 331
conditional statement, 331
constant declaration, 333
expression statement, 331

function application, 331
function declaration, 334
lambda expression, 332
literal expression, 330
name, 330
operator combination, 334
return statement, 333
sequence of statements, 332
variable declaration, 333
parsing natural language, 381–385
real language understanding vs. toy parser, 385 *n*
`partial_sums`, 293 (ex. 3.55)
Pascal, Blaise, 36 *n*
Pascal's triangle, 36 (ex. 1.12)
Pascal, lack of higher-order functions in, 311 *n*
password-protected bank account, 196 (ex. 3.3)
pattern, 403–404
`pattern_match`, *429*
pattern matching, 412
implementation, 428–429
unification vs., 416, 418 *n*
pattern variable, 403
representation of, 437
`pc` register, 471
`perform` (in register machine), 455
instruction constructor, *481*
simulating, 481
`perform_action`, *481*
Perlis, Alan J., xxiii
quips by, 5 *n*, 30 *n*
permutations of a set, 108
`permutations`, *108*
Peter, Paul and Mary, 268 (ex. 3.38)
Phillips, Hubert, 380 (ex. 4.40), 380 (ex. 4.41)
π (pi)
approximation with half-interval method, 60
approximation with Monte Carlo integration, 200 (ex. 3.5), 313 (ex. 3.82)
Dirichlet estimate for, 198, 312
Leibniz's series for, 49 *n*, 297
stream of approximations, 297–299
Wallis's formula for, 52 (ex. 1.31)
picture language, 110–123
Pingala, Áchárya, 40 *n*
pipelining, 264 *n*
`pi_stream`, *297*
`pi_sum`, *49*
with higher-order functions, *51*
with lambda expression, *54*
Planner, 376 *n*
point, represented as a pair, 77 (ex. 2.2)
pointer
in box-and-pointer notation, 84
typed, 488
`polar` package, 159
poly, 177
polymorphic types, 311 *n*
`polynomial` package, 178
polynomial(s), 177–188
canonical form, 184
dense, 181
evaluating with Horner's rule, 103 (ex. 2.34)
hierarchy of types, 184
indeterminate of, 177
sparse, 181
univariate, 177
polynomial arithmetic, 177–188
addition, 178–181
division, 183 (ex. 2.91)
Euclid's Algorithm, 185 *n*
greatest common divisor, 185–187, 188 *n*
interfaced to generic arithmetic system, 178
multiplication, 178–181
probabilistic algorithm for GCD, 188 *n*
rational functions, 184–188
subtraction, 182 (ex. 2.88)
`pop`, *471*
porting a language, 561
PowerPC, 280 *n*
power series, as stream, 295 (ex. 3.59)
adding, 295 (ex. 3.60)
dividing, 296 (ex. 3.62)
integrating, 295 (ex. 3.59)
multiplying, 295 (ex. 3.60)
precedence
of conditional expression, 17 (ex. 1.1)
of lambda expression, 55
of operators, 5
of unary operators, 16
predicate, 14
of clause, 15

of conditional expression, 14
of conditional statement, 57
naming convention for, 20 *n*
prefix code, 140
prefix notation, 128
infix notation vs., 131 (ex. 2.58)
prefix operator, 16
`prepositions`, *382*
`preserving`, 524, 526 (ex. 5.32), *545*, 553 (ex. 5.40)
pretty-printing, 335 (ex. 4.2)
prime number(s), 43–46
cryptography and, 46 *n*
Eratosthenes's sieve for, 290 *n*
Fermat test for, 44–45
infinite stream of, *see* `primes`
Miller–Rabin test for, 48 (ex. 1.28)
testing for, 43–48
`primes` (infinite stream), *290*
implicit definition, *292*
prime sieve
`sieve`, *290*
`prime_sum_pair`, *373*
`prime_sum_pairs`, *107*
infinite stream, 300
`primitive_apply`, *507*
primitive constraints, 254
primitive expression, 3
evaluation of, 7
name of constant, 5
number, 3
primitive function, 10
`primitive_function_objects`, *345*
primitive functions (ECMAScript equivalent in parentheses; those marked *ns* are not in the ECMAScript standard)
`apply` (`apply`), 346 *n*
`display` (*ns*), 75 *n*
`error` (*ns*), 60 *n*
`eval`, 350
`get_time` (`new Date().getTime`), 46 (ex. 1.22)
`head` (*ns*), 73
`is_null` (*ns*), 87
`is_number` (*ns*), 127
`is_pair` (*ns*), 94
`is_string` (*ns*), 128
`is_undefined`, 157
`list` (*ns*), 86
`math_atan2` (`Math.atan2`), 151 *n*
`math_cos` (`Math.cos`), 61
`math_floor` (`Math.floor`), 45
`math_log` (`Math.log`), 10, 62 (ex. 1.36)
`math_max` (`Math.max`), 81
`math_min` (`Math.min`), 81
`math_random` (`Math.random`), 44
`math_round` (`Math.round`), 176 *n*
`math_sin` (`Math.sin`), 61
`math_trunc` (`Math.trunc`), 294 (ex. 3.58)
`pair` (*ns*), 73
`prompt` (*ns*), 347
`set_head` (*ns*), 223
`set_tail` (*ns*), 223
`stringify` (`JSON.stringify`), 75
`tail` (*ns*), 73
`vector_ref` (*ns*), 488
`vector_set` (*ns*), 488
`primitive_function_symbols`, *345*
`primitive_implementation`, *345*
primitive query, *see* simple query
principle of least commitment, 152
`print_point`, *78* (ex. 2.2)
`print_queue`, 235 (ex. 3.21)
`print_rat`, *75*
`print_result`, *514*
monitored-stack version, *516*
`print_stack_statistics` operation in register machine, 484
probabilistic algorithm, 45–46, 188 *n*, 290 *n*
`probe`
in constraint system, *259*
in digital-circuit simulator, *249*
process, 1
iterative, 29
linear iterative, 29
linear recursive, 29
local evolution of, 27
order of growth of, 36
recursive, 29
resources required by, 36
shape of, 28
tree-recursive, 32–35
`product`, 52 (ex. 1.31)
as accumulation, 53 (ex. 1.32)
program, 1
as abstract machine, 348

comments in, 108 *n*
as data, 348–350
incremental development of, 6
structured with subroutines, 349 *n*
structure of, 6, 22, 24–26, *see also* abstraction barriers
value of, 338 (ex. 4.8)
program counter, 471
program environment, 6, 207, 221
programming
data-directed, *see* data-directed programming
demand-driven, 287
elements of, 3
functional, *see* functional programming
imperative, 204
odious style, 288 *n*
programming language, 1
design of, 360
functional, 315
logic, 400
object-oriented, 174 *n*
statically typed, 311 *n*
very high-level, 18 *n*
Prolog, 376 *n*, 399 *n*
promise to evaluate, 284
lazy evaluation and, 370
`prompt` (primitive function), 347
`prompt` operation in register machine, 454
prompts, 346
explicit-control evaluator, 514
lazy evaluator, 365
metacircular evaluator, 346
nondeterministic evaluator, 395
query interpreter, 425
`propagate`, *249*
propagation of constraints, 253–263
proving programs correct, 18 *n*
pseudodivision of polynomials, 187
pseudo-random sequence, 197 *n*
pseudoremainder of polynomials, 187
`push`, *471*
`push_marker_to_stack` (in register machine), 508, 511 (ex. 5.24)
`put`, 157, *240*
puzzles
eight-queens puzzle, 108 (ex. 2.42), 380 (ex. 4.42)
logic puzzles, 378–380
Pythagorean triples
with nondeterministic programs, 377 (ex. 4.33), 378 (ex. 4.34), 378 (ex. 4.35)
with streams, 302 (ex. 3.69)
Python, recursive functions in, 30

quantum mechanics, 316 *n*
`queens`, *109* (ex. 2.42)
query, 400, *see also* compound query; simple query
`query_driver_loop`, *425*
query interpreter, 400
adding rule or assertion, 419
compound query, *see* compound query
data base, 434–435
driver loop, 418, 424–425
environment structure in, 447 (ex. 4.76)
frame, 412, 444
improvements to, 423 (ex. 4.65), 446 (ex. 4.73), 446 (ex. 4.74)
infinite loops, 420–421, 423 (ex. 4.65)
instantiation, 425, 439–441
JavaScript interpreter vs., 417, 418, 447 (ex. 4.76)
as nondeterministic program, 447 (ex. 4.75)
overview, 411–419
pattern matching, 412, 428–429
pattern-variable representation, 437
problems with `not` and `javascript_predicate`, 421–422, 446 (ex. 4.74)
query evaluator, 418, 425–428
rule, *see* rule
simple query, *see* simple query
stream operations, 435–436
streams of frames, 412, 419 *n*
syntax of query language, 436–444
unification, 415–417, 431–433
query language, 400–411
abstraction in, 407
compound query, *see* compound query
data base, 401–403
equality testing in, 406 *n*
extensions to, 423 (ex. 4.64), 445 (ex. 4.72)
logical deductions, 409–411
mathematical logic vs., 419–424
rule, *see* rule
simple query, *see* simple query

query-language-specific representation, 436–444
  transforming JavaScript syntax into, 438
queue, 231–236
  double-ended, 235 (ex. 3.23)
  front of, 231
  functional implementation of, 235 (ex. 3.22)
  operations on, 232
  rear of, 231
  in simulation agenda, 251
quotation marks
  back quotes, 546
  double, 124
  single, 126 (ex. 2.55)

Rabin, Michael O., 48 (ex. 1.28)
radicand, 19
Ramanujan, Srinivasa, 303 *n*
Ramanujan numbers, 303 (ex. 3.71)
`rand`, *197*
  with reset, 200 (ex. 3.6)
`random_in_range`, *200* (ex. 3.5)
random-number generator, 191 *n*, 197
  in Monte Carlo simulation, 198
  in primality testing, 44
  with reset, 200 (ex. 3.6)
  with reset, stream version, 313 (ex. 3.81)
`random_numbers` (infinite stream), *313*
Raphael, Bertram, 399 *n*
`rational` package, 166
rational function, 184–188
  reducing to lowest terms, 187–188
rational number(s)
  arithmetic operations on, 72–76
  printing, 74
  reducing to lowest terms, 75, 77
  represented as pairs, 74
rational-number arithmetic, 72–76
  interfaced to generic arithmetic system, 166
  need for compound data, 69
rational tree, 432 *n*
Raymond, Eric, 361 *n*, 376 *n*
RC circuit, 304 (ex. 3.73)
`read_evaluate_print_loop`, *514*
read-evaluate-print loop, 5, *see also* driver loop
real number, 3 *n*
`real_part`
  data-directed, *160*
  polar representation, *152*
  rectangular representation, *151*
  with tagged data, *154*
`real_part_polar`, *154*
`real_part_rectangular`, *153*
`rear_ptr`, *233*
`receive` function, 475 *n*
record, in a data base, 139
rectangle, representing, 78 (ex. 2.3)
`rectangular` package, 157
recursion, 7
  data-directed, 180
  expressing complicated process, 7
  mutual, 221
  in rules, 408
  in working with trees, 93
recursion theory, 350 *n*
recursive function
  recursive function declaration, 22
  recursive process vs., 29
  specifying without declaration, 352 (ex. 4.18)
recursive process, 29
  iterative process vs., 27–30, 212 (ex. 3.9), 462, 549 (ex. 5.37)
  linear, 29, 37
  recursive function vs., 29
  register machine for, 462–467
  tree, 32–35, 37
red-black tree, 137 *n*
reducing to lowest terms, 75, 77, 187–188
Rees, Jonathan A., 356 *n*
referential transparency, 203
`reg` (in register machine), 453, *482*
  simulating, 482
register(s), 449
  representing, 470
  tracing, 486 (ex. 5.17)
`register_exp_reg`, *482*
register machine, 449
  actions, 454–455
  controller, 450–452
  controller diagram, 451
  data-path diagram, 450
  data paths, 450–452
  design of, 450–468
  language for describing, 452–455

monitoring performance, 484–487
simulator, 468–487
stack, 462–467
subroutine, 457–460
test operation, 451
register-machine language, 467–468
`assign`, 453, 468, *478*
`branch`, 452, 468, *480*
`constant`, 453, 467, 468, *482*
entry point, 452
`go_to`, 452, 467, *480*
instructions, 452, 467, 477–484
`label`, 452, 468, *482*
label, 452
`op`, 453, 468, *483*
`perform`, 455, 468, *481*
`push_marker_to_stack`, 508, 511 (ex. 5.24)
`reg`, 453, 467, *482*
`restore`, 464, 468, *481*
`revert_stack_to_marker`, 508, 511 (ex. 5.24)
`save`, 464, 468, *481*
`test`, 452, 468, *479*
register-machine simulator, 468–487
`registers_modified`, *543*
`registers_needed`, *543*
register table, in simulator, 471
relations, computing in terms of, 254, 398
relatively prime, 53 (ex. 1.33)
relativity, theory of, 281
release a mutex, 277
remainder
after integer division, 39
modulo *n*, 44
`remainder_terms`, 186 (ex. 2.94)
`remove`, *108*
`remove_first_agenda_item`, 249, *252*
`rename_variables_in`, *431*
`require`, *375*
as a syntactic form, 397 (ex. 4.52)
resistance
formula for parallel resistors, 81, 83
tolerance of resistors, 81
resolution, Horn-clause, 399 *n*
resolution principle, 398 *n*
`rest_conjuncts`, *443*
`rest_disjuncts`, *443*
`restore` (in register machine), 464, 483 (ex. 5.10)
implementing, 491
instruction constructor, *481*
simulating, 481
rest parameter and spread syntax, 277
`rest_segments`, *251*
`rest_statements`, *332*
`rest_terms`, 179, *182*
`retry`, *377*
`return` (linkage descriptor), 523
`return` (keyword), 9, *see also* return statement
`return_expression`, 333
returning multiple values, 475 *n*
return statement, 9
handling in compiler, 543
handling in explicit-control evaluator, 509
handling in metacircular evaluator, 327
parsing of, 333
tail recursion and, 509, 541, 542
`return_undefined`, *509*
return value, 9
representation in metacircular evaluator, 324, 340
`undefined` as, 324, 509, 534, 539
`return_value_content`, *340*
Reuter, Andreas, 280 *n*
`reverse`, 89 (ex. 2.18)
as folding, 106 (ex. 2.39)
rules, 424 (ex. 4.66)
`revert_stack_to_marker` (in register machine), 508, 511 (ex. 5.24)
Rhind Papyrus, 40 *n*
right-associative, 14
`right_branch`, *136*, *143*
`right_split`, *114*
ripple-carry adder, 246 (ex. 3.30)
Rivest, Ronald L., 46 *n*, 137 *n*
RLC circuit, 309 (ex. 3.80)
Robinson, J. A., 399 *n*
robustness, 123
rock songs, 1950s, 146 (ex. 2.70)
Rogers, William Barton, 112 *n*
`root` register, 494
roots of equation, *see* half-interval method; Newton's method
`rotate90`, *121*
roundoff error, 3 *n*, 149 *n*
Rozas, Guillermo Juan, 541 *n*
RSA algorithm, 46 *n*

rule (query language), 407–411
  applying, 417–418, 430–431, 447 (ex. 4.76)
  without body, 407 *n*, 410, 428
`rule_body`, *444*
Runkle, John Daniel, 112 *n*
Russian peasant method of multiplication, 40 *n*

`same` (rule), *407*
sameness and change
  meaning of, 202–204
  shared data and, 227
satisfy a compound query, 405–406
satisfy a pattern (simple query), 404
`save` (in register machine), 464, 483 (ex. 5.10)
  implementing, 491
  instruction constructor, *481*
  simulating, 481
`scale_list`, *90, 91*, *371*
`scale_stream`, *292*
`scale_tree`, *97*
`scale_vect`, 119 (ex. 2.46)
`scan` register, 495
scanning out declarations
  in compiler, 532, 560
  in explicit-control evaluator, 512
  in metacircular evaluator, 326, 352
  sequential declaration processing vs., 353–354
`scan_out_declarations`, *327*
Scheme
  evaluators written in, 319 *n*
  integrated-circuit implementation of, 499
  as JavaScript precursor, 2
  `let*` in, 336 (ex. 4.6)
  nondeterministic extension of, 376 *n*
  tail recursion in, 30 *n*
  use of "lambda" in, 55 *n*
  Y operator written in, 353 *n*
Scheme chip, 499
Schmidt, Eric, 204 *n*
scope of a name, 24, *see also* lexical scoping
  function's parameters, 24
  internal declaration, 351–355
`search`, *59*
search
  of binary tree, 135
  depth-first, 376
  systematic, 376
`segment_queue`, *251*
`segments`, *251*
`segments_to_painter`, *119*
`segment_time`, *251*
selector, 72
  as abstraction barrier, 76
  generic, 154, 155
Self, 2
semaphore, 277 *n*
  of size *n*, 279 (ex. 3.47)
semicolon (`;`)
  cancer of, 30 *n*
  ending statement, 3, 5, 9
separator code, 140
sequence(s), 85
  as conventional interface, 98–110
  as source of modularity, 102
  operations on, 100–106
  represented by pairs, 85
sequence accelerator, 297
sequence of statements, 192
  in block, 192 *n*
  in conditional statement, 57
  in function body, 9 *n*
  as body of lambda expression, 116 *n*
  parsing of, 332
sequential declaration processing vs. scanning out, 353–354
`serialized_exchange`, *275*
  with deadlock avoidance, 280 (ex. 3.48)
serializer, 269–273
  implementing, 277–279
  with multiple shared resources, 273–276
series, summation of, 50
  accelerating sequence of approximations, 297
  with streams, 297
set, 131
  data base as, 139
  operations on, 131–132
  permutations of, 108
  represented as binary tree, 135–138
  represented as ordered list, 133–135
  represented as unordered list, 132–133
  subsets of, 98 (ex. 2.32)
`set_contents`, *470*

`set_current_environment`, *514*
`set_current_time`, *251*
`set_front_ptr`, *233*
`set_head` (primitive function), 223
  functional implementation of, *230*
  implemented with vectors, 491
  value of, 223 *n*
`set_inst_execution_fun`, *476*
`set_rear_ptr`, *233*
`set_register_contents`, 468, *473*
`set_segments`, *251*
`set_signal`, 245, *248*
`set_tail` (primitive function), 223
  functional implementation of, *230*
  implemented with vectors, 491
  value of, 223 *n*
`setup_environment`, *345*
`set_value`, 256, *261*
shadow a binding, 207
Shamir, Adi, 46 *n*
shape of a process, 28
shared data, 227–230
shared resources, 273–276
shared state, 266
`shrink_to_upper_right`, *121*
Shrobe, Howard E., 400 *n*
SICP, xxi
`sicp` JavaScript package, 10 *n*
SICP JS, xxi
side-effect bug, 204 *n*
sieve of Eratosthenes, 290 *n*
Σ (sigma) notation, 50
signal, digital, 242
`signal_error`, *515*
signal-flow diagram, 99, 306 (fig. 3.33)
signal processing
  smoothing a function, 68 (ex. 1.44)
  smoothing a signal, 305 (ex. 3.75), 306 (ex. 3.76)
  stream model of, 303–306
  zero crossings of a signal, 305 (ex. 3.74), 305 (ex. 3.75), 306 (ex. 3.76)
signal-processing view of computation, 99
`simple_query`, *426*
  without delayed expression, *444* (ex. 4.68)
simple query, 403–404
  processing, 412, 413, 418, 426
simplification of algebraic expressions, 129
Simpson's Rule for numerical integration, 52 (ex. 1.29)
simulation
  of digital circuit, *see* digital-circuit simulation
  event-driven, 242
  as machine-design tool, 516
  for monitoring performance of register machine, 484
  Monte Carlo, *see* Monte Carlo simulation
  of register machine, 468–487
sine
  approximation for small angle, 37 (ex. 1.15)
  power series for, 295 (ex. 3.59)
`singleton_stream`, *436*
SKETCHPAD, 253 *n*
slash (double slash *//* for comments in programs), 108 *n*
`smallest_divisor`, *43*
  more efficient version, 47 (ex. 1.23)
Smalltalk, 253 *n*
smoothing a function, 68 (ex. 1.44)
smoothing a signal, 305 (ex. 3.75), 306 (ex. 3.76)
snake case, 10 *n*
snarf, 361 *n*
Solomonoff, Ray, 197 *n*
`solve` differential equation, *307, 308*
  lazy-list version, *372*
solving equation, *see* half-interval method; Newton's method; `solve`
source language, 520
source program, 520
Spafford, Eugene H., 561 *n*
sparse polynomial, 181
`split`, 117 (ex. 2.45)
spread and rest parameter syntax, 277
`sqrt`, *20*
  block structured, *26*
  in environment model, 218–220
  as fixed point, *61*, *64–66*
  as iterative improvement, 68 (ex. 1.46)
  with Newton's method, *65, 66*
  register machine for, 457 (ex. 5.3)
  as stream limit, *300* (ex. 3.64)
`sqrt_stream`, *297*
`square`, *8*
  in environment model, 208–209

square_limit, *116, 117*, 123 (ex. 2.52)
square_of_four, *116*, 123 (ex. 2.52)
squarer (constraint), 262 (ex. 3.34), 262 (ex. 3.35)
square root, 19–20, *see also* sqrt
  stream of approximations, 297
squash_inwards, *121*
stack, 29 *n*
  framed, 504 *n*
  for recursion in register machine, 462–467
  representing, 470, 491
stack allocation and tail recursion, 541 *n*
stack_inst_reg_name, *481*
Stallman, Richard M., 253 *n*, 376 *n*
start register machine, 468, *473*
start_eceval, *559 n*
start_segment, 77 (ex. 2.2), 120 (ex. 2.48)
state
  local, *see* local state
  shared, 266
  vanishes in stream formulation, 314
statement, 3
  value-producing and non-value-producing, 338 (ex. 4.8)
statement sequence, *see* sequence of statements
state variable, 29, 190
  local, 190–197
statically typed language, 311 *n*
Steele, Guy Lewis Jr., 30 *n*, 205 *n*, 253 *n*, 361 *n*, 376 *n*
Stein, Clifford, 137 *n*
stop-and-copy garbage collector, 493–498
Stoy, Joseph E., 12 *n*, 41 *n*, 353 *n*
Strachey, Christopher, 67 *n*
stratified design, 123
stream(s), 189, 281–316
  delayed evaluation and, 306–310
  empty, 284
  implemented as delayed lists, 282–284
  implemented as lazy lists, 370–372
  implicit definition, 291–293
  infinite, *see* infinite streams
  used in query interpreter, 412, 419 *n*
stream_append, *301*
stream_append_delayed, *435*
stream_enumerate_interval, *285*
stream_filter, *286*
stream_flatmap, *436*, 445 (ex. 4.71)
stream_for_each, *284*
stream_limit, 299 (ex. 3.64)
stream_map, *284*, *288*
stream_map_2, 288 (ex. 3.50)
stream_map_optimized, *288*
stream_ref, *284*
stream_tail, *284*
stream_withdraw, *314*
strict, 362
string(s), 124–126
  concatenation, 75
  equality of, 125 (ex. 2.54)
  interning, 490
  quotation marks for, *see* quotation marks
  representation of, 490
  typed over multiple lines, 546
  uniqueness of, 227 *n*
stringify (primitive function), 75
string pool, 490
Stuckey, Peter J., 432 *n*
sub (generic), *165*
sub_complex, *150*
sub_interval, 82 (ex. 2.8)
sub_rat, *73*
subroutine in register machine, 457–460
subsets of a set, *98* (ex. 2.32)
substitution model of function application, 11–13, 206
  inadequacy of, 200–202
  shape of process, 27–29
subtype, 172
  multiple, 173
sub_vect, 119 (ex. 2.46)
success continuation (nondeterministic evaluator), 386, 388
successive squaring, 38
sum, *50*
  as accumulation, 53 (ex. 1.32)
  iterative version, 52 (ex. 1.30)
sum_cubes, *49*
  with higher-order functions, *50*
sum_integers, *49*
  with higher-order functions, *51*
summation of a series, 50
  with streams, 297
sum_odd_squares, *98*, *101*
sum_of_squares, *10*
  in environment model, 210–212

`sum_primes`, *283*
supertype, 172
  multiple, 173
Sussman, Gerald Jay, 30 *n*, 253 *n*, 376 *n*
Sutherland, Ivan, 253 *n*
symbol(s)
  in environment operations, 341
  in global environment, 344
  in parsing of names, 330
  representing names in metacircular evaluator, 323
  in `unparse`, 441
symbolic algebra, 176–188
symbolic differentiation, 126–131, 160 (ex. 2.73)
symbolic expression, 71, *see also* string(s); symbol(s)
`symbol_leaf`, *143*
`symbol_of_name`, 331
`symbols`, *143*
SYNC, 280 *n*
synchronization, *see* concurrency
syntactic analysis, separated from execution
  in metacircular evaluator, 355–360
  in register-machine simulator, 474, 478
syntactic form, 8
  as derived component, 334–335
  function vs., 362 (ex. 4.24)
  need for, 20 (ex. 1.6)
syntactic forms
  assignment, 192
  block, 25
  break statement, 338 (ex. 4.7)
  conditional expression, 14
  conditional statement, 57
  constant declaration, 5
  continue statement, 338 (ex. 4.7)
  function declaration, 8
  lambda expression, 54
  logical conjunction (`&&`), 15
  logical disjunction (`"|"|`), 15
  return statement, 9
  variable declaration, 191
  while loop, 337 (ex. 4.7)
syntactic sugar, 15 *n*, 30 *n*
  `&&` and `"|"|` as, 15
  function vs. data as, 248 *n*
  looping constructs as, 30
syntax, *see also* syntactic form
  abstract, *see* abstract syntax
  of expressions, describing, 9 *n*
  of a programming language, 8
syntax interface, 248 *n*
systematic search, 376

table, 236–242
  backbone of, 236
  for coercion, 170
  for data-directed programming, 157
  local, 239–240
  n-dimensional, 241 (ex. 3.25)
  one-dimensional, 236–237
  operation-and-type, *see* operation-and-type table
  represented as binary tree vs. unordered list, 241 (ex. 3.26)
  testing equality of keys, 240 (ex. 3.24)
  two-dimensional, 237–239
  used in simulation agenda, 251
  used to store computed values, 241 (ex. 3.27)
tableau, 298
tabulation, 35 *n*, 241 (ex. 3.27)
`tack_on_instruction_sequence`, *545*
tagged architecture, 489 *n*
tagged data, 152–155, 489 *n*
`tail` (primitive function), 73
  axiom for, 79
  functional implementation of, *79*, 80 (ex. 2.4), *230*, *371*
  implemented with vectors, 490
  as list operation, 86
tail recursion, 30
  compiler and, 541
  environment model of evaluation and, 213 *n*
  explicit-control evaluator and, 511 (ex. 5.22), 509–511, 517 (ex. 5.27), 518 (ex. 5.29)
  garbage collection and, 541 *n*
  in JavaScript, 30 *n*
  metacircular evaluator and, 324 *n*, 327 *n*, 509
  return statement necessary for, 511, 542
  in Scheme, 30 *n*
tail-recursive evaluator, 509
tangent
  as continued fraction, 63 (ex. 1.39)
  power series for, 296 (ex. 3.62)

target register, 523
TDZ (temporal dead zone), 222 *n*, *see also* declaration, use of name before
Technological University of Eindhoven, 277 *n*
temporal dead zone (TDZ), 222 *n*, *see also* declaration, use of name before
terminal node of a tree, 7
`term_list`, *178*
term list of polynomial, 177
  representing, 181–183
ternary operator, *see* conditional expression
`test` (in register machine), 452
  instruction constructor, *479*
  simulating, 479
`test_and_set`, *278*
`test_condition`, *479*
test operation in register machine, 451
Thatcher, James W., 78 *n*
`the_empty_environment`, *341*
`the_empty_termlist`, 179, *182*
`the_global_environment`, *345*, *515*
`the_heads`
  register, 490, 494
  vector, 488
THE Multiprogramming System, 277 *n*
theorem proving (automatic), 398 *n*
$\theta(f(n))$ (theta of $f(n)$), 36
`the_tails`
  register, 490, 494
  vector, 488
thread, 264
thunk, 363
  call-by-name, 287 *n*
  call-by-need, 287 *n*
  forcing, 363
  implementation of, 366–367
  origin of name, 363 *n*
time
  assignment and, 264
  communication and, 281
  in concurrent systems, 265–269
  functional programming and, 314–316
  in nondeterministic computing, 374, 375
  purpose of, 265 *n*
`timed_prime_test`, 46 (ex. 1.22)
time segment, in agenda, 251
time slicing, 278
timing diagram, 266 (fig. 3.29)
TK"!Solver, 253 *n*
tower of types, 172 (fig. 2.25)
tracing
  instruction execution, 486 (ex. 5.15)
  register assignment, 486 (ex. 5.17)
`transform_painter`, *121*
transparency, referential, 203
`transpose` a matrix, 105 (ex. 2.37)
tree
  binary, 135, *see also* binary tree
  B-tree, 137 *n*
  combination viewed as, 7
  counting leaves of, 93
  enumerating leaves of, 101
  fringe of, 95 (ex. 2.28)
  Huffman, 141
  lazy, 371 *n*
  mapping over, 96–98
  rational, 432 *n*
  red-black, 137 *n*
  represented as pairs, 92–96
  reversing at all levels, 95 (ex. 2.27)
tree accumulation, 7
`tree_map`, 97 (ex. 2.31)
tree-recursive process, 32–35
  order of growth, 37
`tree_to_list_...`, *137* (ex. 2.63)
trigonometric relations, 151
`true` (keyword), 14
truncation error, 3 *n*
truthiness, 325, 339 *n*
truth maintenance, 376 *n*
Turing, Alan M., 349 *n*, 351 *n*
Turing machine, 349 *n*
Turner, David, 106 *n*, 301 *n*, 315 *n*
`type` in query system, *443*
`type` in register machine, *477*
type(s)
  cross-type operations, 169
  dispatching on, 156
  hierarchy in symbolic algebra, 184
  hierarchy of, 172–176
  lowering, 173, 176 (ex. 2.85)
  multiple subtype and supertype, 173
  polymorphic, 311 *n*
  raising, 173, 175 (ex. 2.83)
  subtype, 172
  supertype, 172
  tower of, 172 (fig. 2.25)

typed pointer, 488
type field, 489 *n*
type-inferencing mechanism, 311 *n*
`type_tag`, *152*
  using JavaScript data types, 168 (ex. 2.78)
type tag, 148, 152
  two-level, 168

unary operator, 16
unbound name, 206
`undefined` (predeclared name), 157
`unev` register, 500
unification, 415–417
  discovery of algorithm, 398 *n*
  implementation, 431–433
  pattern matching vs., 416, 418 *n*
`unify_match`, *431*
`union_set`, 131
  binary-tree representation, 138 (ex. 2.65)
  ordered-list representation, 135 (ex. 2.62)
  unordered-list representation, 133 (ex. 2.59)
`unique` (query language), 445 (ex. 4.72)
`unique_pairs`, 108 (ex. 2.40)
unit square, 117
univariate polynomial, 177
universal machine, 349
  explicit-control evaluator as, 519
  general-purpose computer as, 520
University of Edinburgh, 399 *n*
University of Marseille, 399 *n*
UNIX, 561 *n*
  epoch, 46 *n*
`unknown_component_type`, *515*
`unknown_function_type`, *515*
unordered-list representation of sets, 132–133
`unparse`
  as inverse of `parse`, 335 (ex. 4.2)
  in query interpreter, 425, *441*
`update_insts`, *476*
`upper_bound`, 82 (ex. 2.7)
`up_split`, 116 (ex. 2.44)
upward compatibility, 370 (ex. 4.29)
`user_print`, *347*
  modified for compiled code, *559 n*
`user_read`, *347*

V operation on semaphore, 277 *n*
`val` register, 500
value
  of an expression, 4, 5 *n*
  of a program, 338 (ex. 4.8)
`value_fun`, 478
`variable`, *178*
variable
  assignment to, 192
  declaration, 191
  declaration, parsing of, 333
  parameter as, 192
variable-length code, 140
vector (data structure), 488
  for arguments of `apply`, 346 *n*
  used in spread and rest parameter syntax, 277
vector (mathematical)
  operations on, 105 (ex. 2.37), 118 (ex. 2.46)
  in picture-language frame, 117
  represented as pair, 118 (ex. 2.46)
  represented as sequence, 105 (ex. 2.37)
`vector_ref` (primitive function), 488
`vector_set` (primitive function), 488
`verbs`, *381*
very high-level language, 18 *n*

Wadler, Philip, 204 *n*
Wadsworth, Christopher, 311 *n*
Wagner, Eric G., 79 *n*
Walker, Francis Amasa, 112 *n*
walking down a list with `tail`, 87
Wallis, John, 52 *n*
Wand, Mitchell, 319 *n*, 506 *n*
Waters, Richard C., 103 *n*
web browser, interpreting JavaScript, 2
`weight`, *143*
`weight_leaf`, *143*
Weyl, Hermann, 69
"what is" vs. "how to" description, *see* declarative vs. imperative knowledge
`wheel` (rule), *407*, 423 (ex. 4.63)
`while` (keyword), 337 (ex. 4.7), *see also* while loop
while loop
  implementing in analyzing evaluator, 359 (ex. 4.20)
  implementing in metacircular evaluator, 337 (ex. 4.7)

whitespace characters, 335 (ex. 4.2)
`width`, *83*
width of an interval, 82 (ex. 2.9)
Wilde, Oscar (Perlis's paraphrase of), 5 *n*
Wiles, Andrew, 44 *n*
Winograd, Terry, 376 *n*
Winston, Patrick Henry, 376 *n*, 385 *n*
wire, in digital circuit, 242
Wise, David S., 287 *n*
wishful thinking, 72, 127
`withdraw`, *191*
  problems in concurrent system, 265
world line of a particle, 282 *n*, 315 *n*
Wright, E. M., 293 *n*
Wright, Jesse B., 79 *n*
Wrigstad, Tobias, daughter of, 124

`xcor_vect`, 118 (ex. 2.46)
Xerox Palo Alto Research Center, 253 *n*

Y operator, 353 *n*
Yang Hui, 36 *n*
`ycor_vect`, 118 (ex. 2.46)
Yochelson, Jerome C., 494 *n*

Zabih, Ramin, 376 *n*
zero crossings of a signal, 305 (ex. 3.74), 305 (ex. 3.75), 306 (ex. 3.76)
zero test (generic), 169 (ex. 2.80)
  for polynomials, 182 (ex. 2.87)
Zilles, Stephen N., 78 *n*
Zippel, Richard E., 188 *n*

List of Exercises

1.1 16 **1.11** 35 **1.21** 46 **1.31** 52 **1.41** 67
1.2 17 **1.12** 36 **1.22** 46 **1.32** 53 **1.42** 67
1.3 17 **1.13** 36 **1.23** 47 **1.33** 53 **1.43** 68
1.4 17 **1.14** 37 **1.24** 47 **1.34** 58 **1.44** 68
1.5 17 **1.15** 37 **1.25** 47 **1.35** 62 **1.45** 68
1.6 20 **1.16** 40 **1.26** 47 **1.36** 62 **1.46** 68
1.7 21 **1.17** 40 **1.27** 48 **1.37** 62
1.8 21 **1.18** 40 **1.28** 48 **1.38** 62
1.9 30 **1.19** 40 **1.29** 52 **1.39** 63
1.10 31 **1.20** 43 **1.30** 52 **1.40** 67

2.1 76 **2.21** 91 **2.41** 108 **2.61** 135 **2.81** 174
2.2 77 **2.22** 92 **2.42** 108 **2.62** 135 **2.82** 175
2.3 78 **2.23** 92 **2.43** 110 **2.63** 137 **2.83** 175
2.4 80 **2.24** 94 **2.44** 116 **2.64** 137 **2.84** 175
2.5 80 **2.25** 95 **2.45** 117 **2.65** 138 **2.85** 176
2.6 80 **2.26** 95 **2.46** 118 **2.66** 139 **2.86** 176
2.7 82 **2.27** 95 **2.47** 119 **2.67** 145 **2.87** 182
2.8 82 **2.28** 95 **2.48** 120 **2.68** 145 **2.88** 182
2.9 82 **2.29** 96 **2.49** 120 **2.69** 146 **2.89** 182
2.10 82 **2.30** 97 **2.50** 122 **2.70** 146 **2.90** 183
2.11 82 **2.31** 97 **2.51** 122 **2.71** 146 **2.91** 183
2.12 83 **2.32** 98 **2.52** 123 **2.72** 147 **2.92** 184
2.13 83 **2.33** 103 **2.53** 125 **2.73** 160 **2.93** 185
2.14 84 **2.34** 103 **2.54** 125 **2.74** 161 **2.94** 186
2.15 84 **2.35** 104 **2.55** 125 **2.75** 163 **2.95** 186
2.16 84 **2.36** 104 **2.56** 130 **2.76** 163 **2.96** 187
2.17 89 **2.37** 105 **2.57** 130 **2.77** 168 **2.97** 187
2.18 89 **2.38** 105 **2.58** 131 **2.78** 168
2.19 89 **2.39** 106 **2.59** 133 **2.79** 169
2.20 89 **2.40** 108 **2.60** 133 **2.80** 169

3.1 196 **3.11** 220 **3.21** 235 **3.31** 250 **3.41** 272
3.2 196 **3.12** 225 **3.22** 235 **3.32** 253 **3.42** 272
3.3 196 **3.13** 226 **3.23** 235 **3.33** 262 **3.43** 275
3.4 197 **3.14** 226 **3.24** 240 **3.34** 262 **3.44** 275
3.5 200 **3.15** 229 **3.25** 241 **3.35** 262 **3.45** 276
3.6 200 **3.16** 229 **3.26** 241 **3.36** 262 **3.46** 279
3.7 205 **3.17** 229 **3.27** 241 **3.37** 263 **3.47** 279
3.8 206 **3.18** 229 **3.28** 246 **3.38** 268 **3.48** 280
3.9 212 **3.19** 230 **3.29** 246 **3.39** 271 **3.49** 280
3.10 217 **3.20** 231 **3.30** 246 **3.40** 271 **3.50** 288

3.51 288 **3.58** 294 **3.65** 300 **3.72** 303 **3.79** 309
3.52 289 **3.59** 295 **3.66** 302 **3.73** 304 **3.80** 309
3.53 293 **3.60** 295 **3.67** 302 **3.74** 305 **3.81** 313
3.54 293 **3.61** 296 **3.68** 302 **3.75** 305 **3.82** 313
3.55 293 **3.62** 296 **3.69** 302 **3.76** 306
3.56 293 **3.63** 299 **3.70** 303 **3.77** 308
3.57 294 **3.64** 299 **3.71** 303 **3.78** 309

4.1 328 **4.17** 352 **4.33** 377 **4.49** 396 **4.65** 423
4.2 335 **4.18** 352 **4.34** 378 **4.50** 396 **4.66** 424
4.3 336 **4.19** 354 **4.35** 378 **4.51** 397 **4.67** 424
4.4 336 **4.20** 359 **4.36** 379 **4.52** 397 **4.68** 444
4.5 336 **4.21** 360 **4.37** 379 **4.53** 404 **4.69** 445
4.6 336 **4.22** 360 **4.38** 380 **4.54** 406 **4.70** 445
4.7 337 **4.23** 362 **4.39** 380 **4.55** 408 **4.71** 445
4.8 338 **4.24** 362 **4.40** 380 **4.56** 408 **4.72** 445
4.9 343 **4.25** 367 **4.41** 380 **4.57** 408 **4.73** 446
4.10 343 **4.26** 368 **4.42** 380 **4.58** 409 **4.74** 446
4.11 343 **4.27** 368 **4.43** 385 **4.59** 410 **4.75** 447
4.12 344 **4.28** 368 **4.44** 385 **4.60** 411 **4.76** 447
4.13 344 **4.29** 370 **4.45** 385 **4.61** 411
4.14 348 **4.30** 372 **4.46** 385 **4.62** 422
4.15 350 **4.31** 372 **4.47** 385 **4.63** 423
4.16 352 **4.32** 372 **4.48** 396 **4.64** 423

5.1 452 **5.12** 484 **5.23** 511 **5.34** 534 **5.45** 556
5.2 454 **5.13** 486 **5.24** 511 **5.35** 535 **5.46** 557
5.3 457 **5.14** 486 **5.25** 513 **5.36** 549 **5.47** 557
5.4 466 **5.15** 486 **5.26** 513 **5.37** 549 **5.48** 562
5.5 466 **5.16** 486 **5.27** 517 **5.38** 549 **5.49** 562
5.6 467 **5.17** 486 **5.28** 517 **5.39** 549 **5.50** 563
5.7 469 **5.18** 486 **5.29** 518 **5.40** 553 **5.51** 563
5.8 477 **5.19** 492 **5.30** 518 **5.41** 553 **5.52** 563
5.9 483 **5.20** 492 **5.31** 518 **5.42** 556 **5.53** 564
5.10 483 **5.21** 492 **5.32** 526 **5.43** 556 **5.54** 564
5.11 484 **5.22** 511 **5.33** 526 **5.44** 556 **5.55** 564